

艾米地产项目人工智能调度管理系统

程序鉴别材料（源代码）

版本：V1.0

本项目共包含 220 个核心源代码文件。

涵盖以下核心层：

- 核心业务逻辑层（emily-core/emily_core）
- API 服务层（emily-core/api）
- 系统脚本层（scripts）
- 插件适配层（data/plugins/emily_agent）

文件目录索引

1. emily-core\emily_core\adapters\session\session_config.py
2. emily-core\emily_core\adapters\session\session_factory.py
3. emily-core\emily_core\adapters\session\session_pool.py
4. emily-core\emily_core\adapters\standard\command.py
5. emily-core\emily_core\adapters\standard\message.py
6. emily-core\emily_core\adapters\standard\reply.py
7. emily-core\emily_core\adapters\standard\result.py
8. emily-core\emily_core\adapters\standard\route_decision.py
9. emily-core\emily_core\agent\sop_parser.py
10. emily-core\emily_core\application_user_utils.py
11. emily-core\emily_core\application\event_app.py
12. emily-core\emily_core\application\file_app.py
13. emily-core\emily_core\application\meeting_app.py
14. emily-core\emily_core\application\node_app.py
15. emily-core\emily_core\application\permission_app.py
16. emily-core\emily_core\application\query_app.py
17. emily-core\emily_core\application\task_app.py
18. emily-core\emily_core\bootstrap.py
19. emily-core\emily_core\infrastructure\database\models.py
20. emily-core\emily_core\infrastructure\database\scripts\fill_users_required_fields.py
21. emily-core\emily_core\infrastructure\database\session.py
22. emily-core\emily_core\infrastructure\llm\client.py
23. emily-core\emily_core\infrastructure\llm\prompt_loader.py
24. emily-core\emily_core\infrastructure\logging\business_event_logger.py
25. emily-core\emily_core\infrastructure\logging\feedback_detector.py
26. emily-core\emily_core\infrastructure\logging\legacy_log_bridge.py
27. emily-core\emily_core\infrastructure\logging\llm_logger.py
28. emily-core\emily_core\infrastructure\logging\log_writer.py
29. emily-core\emily_core\infrastructure\logging\pipeline_logger.py

- 30. emily-core\emily_core\infrastructure\logging\rag_logger.py
- 31. emily-core\emily_core\infrastructure\logging\scheduler_logger.py
- 32. emily-core\emily_core\infrastructure\logging\session_lifecycle_logger.py
- 33. emily-core\emily_core\node_event_bus.py
- 34. emily-core\emily_core\outbound_bus.py
- 35. emily-core\emily_core\permission\auth_engine.py
- 36. emily-core\emily_core\permission\cache.py
- 37. emily-core\emily_core\permission\code_compiler.py
- 38. emily-core\emily_core\permission\level.py
- 39. emily-core\emily_core\permission\row_security.py
- 40. emily-core\emily_core\providers\email\base.py
- 41. emily-core\emily_core\providers\email\imap_provider.py
- 42. emily-core\emily_core\providers\email\smtp_provider.py
- 43. emily-core\emily_core\providers\rag\base.py
- 44. emily-core\emily_core\providers\rag\local_fallback.py
- 45. emily-core\emily_core\providers\rag\maxkb_provider.py
- 46. emily-core\emily_core\repositories\agent_reasoning_repo.py
- 47. emily-core\emily_core\repositories\chat_archive_repo.py
- 48. emily-core\emily_core\repositories\event_repo.py
- 49. emily-core\emily_core\repositories\evolution_repo.py
- 50. emily-core\emily_core\repositories\file_repo.py
- 51. emily-core\emily_core\repositories\llm_interaction_repo.py
- 52. emily-core\emily_core\repositories\meeting_repo.py
- 53. emily-core\emily_core\repositories\message_repo.py
- 54. emily-core\emily_core\repositories\node_repo.py
- 55. emily-core\emily_core\repositories\permission_grant_repo.py
- 56. emily-core\emily_core\repositories\permission_repo.py
- 57. emily-core\emily_core\repositories\scheduler_repo.py
- 58. emily-core\emily_core\repositories\session_accessible_file_repo.py
- 59. emily-core\emily_core\repositories\session_archive_repo.py
- 60. emily-core\emily_core\repositories\system_description_repo.py
- 61. emily-core\emily_core\repositories\task_repo.py
- 62. emily-core\emily_core\repositories\tool_call_repo.py
- 63. emily-core\emily_core\repositories\tool_registry_repo.py
- 64. emily-core\emily_core\repositories\user_repo.py
- 65. emily-core\emily_core\repositories\world_book_repo.py
- 66. emily-core\emily_core\scheduler\application.py
- 67. emily-core\emily_core\scheduler\commands.py
- 68. emily-core\emily_core\scheduler\engine.py
- 69. emily-core\emily_core\scheduler\handler_registry.py
- 70. emily-core\emily_core\scheduler\hook_registry.py
- 71. emily-core\emily_core\scheduler\jobs\daily_insight.py
- 72. emily-core\emily_core\scheduler\jobs\data_sync.py
- 73. emily-core\emily_core\scheduler\jobs\health_check.py
- 74. emily-core\emily_core\scheduler\jobs\morning_report.py
- 75. emily-core\emily_core\scheduler\jobs\node_deadlines.py

- 76. emily-core\emily_core\scheduler\jobs\patch_validator.py
- 77. emily-core\emily_core\scheduler\jobs\periodic_node.py
- 78. emily-core\emily_core\scheduler\jobs\rule_induction.py
- 79. emily-core\emily_core\scheduler\jobs\session_cleanup.py
- 80. emily-core\emily_core\scheduler\jobs\system_description_update.py
- 81. emily-core\emily_core\scheduler\jobs\webhook.py
- 82. emily-core\emily_core\scheduler\jobs\world_book_update.py
- 83. emily-core\emily_core\scheduler\service.py
- 84. emily-core\emily_core\services\agent_trace_service.py
- 85. emily-core\emily_core\services\chat_archive_service.py
- 86. emily-core\emily_core\services\cognition_drift_detector.py
- 87. emily-core\emily_core\services\domain_takeover_service.py
- 88. emily-core\emily_core\services\email_service.py
- 89. emily-core\emily_core\services\event_journal.py
- 90. emily-core\emily_core\services\event_service.py
- 91. emily-core\emily_core\services\evolution\insight_generator.py
- 92. emily-core\emily_core\services\evolution\morning_report_builder.py
- 93. emily-core\emily_core\services\evolution\patch_applier.py
- 94. emily-core\emily_core\services\evolution\patch_generator.py
- 95. emily-core\emily_core\services\evolution\patch_validator.py
- 96. emily-core\emily_core\services\evolution\rule_inductor.py
- 97. emily-core\emily_core\services\file_service.py
- 98. emily-core\emily_core\services\file_storage_service.py
- 99. emily-core\emily_core\services\initialization_checker.py
- 100. emily-core\emily_core\services\meeting_service.py
- 101. emily-core\emily_core\services\message_service.py
- 102. emily-core\emily_core\services\node_batch.py
- 103. emily-core\emily_core\services\node_batch_update.py
- 104. emily-core\emily_core\services\node_commands.py
- 105. emily-core\emily_core\services\node_service.py
- 106. emily-core\emily_core\services\node_state_machine.py
- 107. emily-core\emily_core\services\pending_issues.py
- 108. emily-core\emily_core\services\permission_service.py
- 109. emily-core\emily_core\services\query_service.py
- 110. emily-core\emily_core\services\rule_book_loader.py
- 111. emily-core\emily_core\services\schema_drift_detector.py
- 112. emily-core\emily_core\services\system_description_builder.py
- 113. emily-core\emily_core\services\system_description_service.py
- 114. emily-core\emily_core\services\task_service.py
- 115. emily-core\emily_core\services\user_binding_service.py
- 116. emily-core\emily_core\services\user_memory_service.py
- 117. emily-core\emily_core\services\world_book_builder.py
- 118. emily-core\emily_core\services\world_book_service.py
- 119. emily-core\emily_core\session\confirm_queue.py
- 120. emily-core\emily_core\session\fetchers\fetch_available_tools.py
- 121. emily-core\emily_core\session\fetchers\fetch_rag_info.py

- 122. emily-core\emily_core\session\fetchers\fetch_system_description.py
- 123. emily-core\emily_core\session\fetchers\fetch_visible_files.py
- 124. emily-core\emily_core\session\fetchers\fetch_visible_schema.py
- 125. emily-core\emily_core\session\focus_lock.py
- 126. emily-core\emily_core\session\session_agent.py
- 127. emily-core\emily_core\session\session_context.py
- 128. emily-core\emily_core\session\session_data_fetcher.py
- 129. emily-core\emily_core\session\session_state.py
- 130. emily-core\emily_core\skill\definition.py
- 131. emily-core\emily_core\skill\executor.py
- 132. emily-core\emily_core\skill\param_extractor.py
- 133. emily-core\emily_core\skill\parser.py
- 134. emily-core\emily_core\skill\registry.py
- 135. emily-core\emily_core\skill\validator.py
- 136. emily-core\emily_core\tools\business_flow_tools.py
- 137. emily-core\emily_core\tools\chat_archive_tool.py
- 138. emily-core\emily_core\tools\definitions.py
- 139. emily-core\emily_core\tools\email_tool.py
- 140. emily-core\emily_core\tools\event_tool.py
- 141. emily-core\emily_core\tools\file_tool.py
- 142. emily-core\emily_core\tools\knowledge_search_tool.py
- 143. emily-core\emily_core\tools\meeting_tool.py
- 144. emily-core\emily_core\tools\memory_tool.py
- 145. emily-core\emily_core\tools\node_task_tool.py
- 146. emily-core\emily_core\tools\node_tool.py
- 147. emily-core\emily_core\tools\node_voice_entry_tool.py
- 148. emily-core\emily_core\tools\pending_issue_tool.py
- 149. emily-core\emily_core\tools\query_tool.py
- 150. emily-core\emily_core\tools\registry.py
- 151. emily-core\emily_core\tools\scripts\search_files.py
- 152. emily-core\emily_core\tools\task_tool.py
- 153. emily-core\emily_core\workitem\injector.py
- 154. emily-core\emily_core\workitem\pipeline\bus.py
- 155. emily-core\emily_core\workitem\pipeline\context.py
- 156. emily-core\emily_core\workitem\pipeline\hook.py
- 157. emily-core\emily_core\workitem\pipeline\hook_registry.py
- 158. emily-core\emily_core\workitem\pipeline\interfaces\auth.py
- 159. emily-core\emily_core\workitem\pipeline\interfaces\execution.py
- 160. emily-core\emily_core\workitem\pipeline\interfaces\guardian.py
- 161. emily-core\emily_core\workitem\pipeline\interfaces\planning.py
- 162. emily-core\emily_core\workitem\pipeline\interfaces\risk.py
- 163. emily-core\emily_core\workitem\pipeline\interfaces\routing.py
- 164. emily-core\emily_core\workitem\pipeline\mocks\mock_execution.py
- 165. emily-core\emily_core\workitem\pipeline\mocks\mock_planning.py
- 166. emily-core\emily_core\workitem\pipeline\node.py
- 167. emily-core\emily_core\workitem\pipeline\real_guardian.py

- 168. emily-core\emily_core\workitem\scheduler.py
- 169. emily-core\emily_core\workitem\workitem.py
- 170. emily-core\emily_core\workitem\workitem_agent.py
- 171. emily-core\emily_core\workitem\workitem_state.py
- 172. emily-core\api\middleware\auth.py
- 173. emily-core\api\routes\evolution.py
- 174. emily-core\api\routes\health.py
- 175. emily-core\api\routes\message.py
- 176. emily-core\api\routes\meta_cognition.py
- 177. emily-core\api\routes\node.py
- 178. emily-core\api\routes\node_schemas.py
- 179. emily-core\api\routes\permission.py
- 180. emily-core\api\routes\session.py
- 181. emily-core\api\routes\skills.py
- 182. emily-core\api\server.py
- 183. emily-core\api\sse\node_events.py
- 184. emily-core\api\sse\outbound.py
- 185. scripts\build_system_description.py
- 186. scripts\build_world_book.py
- 187. scripts\check_initialization.py
- 188. scripts\cognition_cycle.py
- 189. scripts\cold_start.py
- 190. scripts\collect_session_data.py
- 191. scripts\detect_cognition_drift.py
- 192. scripts\evolution.py
- 193. scripts\evolution_anomaly.py
- 194. scripts\evolution_apply.py
- 195. scripts\evolution_insight.py
- 196. scripts\evolution_metrics.py
- 197. scripts\evolution_morning.py
- 198. scripts\evolution_node_close.py
- 199. scripts\evolution_patch.py
- 200. scripts\evolution_rules.py
- 201. scripts\evolution_validate.py
- 202. scripts\generate_session_prompt.py
- 203. scripts\manage_nodes.py
- 204. scripts\register_api.py
- 205. scripts\rescan_files.py
- 206. scripts\self_check.py
- 207. scripts\sop_to_skill.py
- 208. scripts\update_world_book.py
- 209. data\plugins\emily_agent_conf_schema.json
- 210. data\plugins\emily_agent\adapters\api_client.py
- 211. data\plugins\emily_agent\adapters\astrbot\inbound_adapter.py
- 212. data\plugins\emily_agent\adapters\astrbot\outbound_sender.py
- 213. data\plugins\emily_agent\adapters\sse_listener.py

- 214. data\plugins\emily_agent\adapters\standard\command.py
- 215. data\plugins\emily_agent\adapters\standard\message.py
- 216. data\plugins\emily_agent\adapters\standard\reply.py
- 217. data\plugins\emily_agent\adapters\standard\result.py
- 218. data\plugins\emily_agent\adapters\standard\route_decision.py
- 219. data\plugins\emily_agent\main.py
- 220. data\plugins\emily_agent\metadata.yaml

文件编号：1 文件路径：emily-core\emily_core\adapters\session\session_config.py

```
from __future__ import annotations
from dataclasses import dataclass
@dataclass
class SessionConfig:
    ttl_seconds: int = 600
    max_concurrent: int = 100
    max_workitems_per_session: int = 5
    sweep_interval_seconds: int = 60
    @classmethod
    def from_config(cls, config) -> "SessionConfig":
        return cls(
            ttl_seconds=getattr(config, "session_ttl_seconds", 600),
            max_concurrent=getattr(config, "session_max_concurrent", 100),
            max_workitems_per_session=getattr(config, "workitem_max_per_session",
5),
        )
```

文件编号：2 文件路径：emily-core\emily_core\adapters\session\session_factory.py

```
from __future__ import annotations
import logging
from typing import TYPE_CHECKING
from ...session.session_agent import SessionAgent
from ...session.session_context import SessionContext
if TYPE_CHECKING:
    from ...adapters.standard.message import StandardMessage
    from ...workitem.pipeline.bus import PipelineBUS
logger = logging.getLogger("emily.session_factory")
class SessionFactory:
    def __init__(self, bus: "PipelineBUS", core=None):
        self._bus = bus
        self._core = core
    def create(self, message: "StandardMessage", user_id: str = "") ->
SessionAgent:
        conv_id = message.conversation_id
        context = self._build_context(message, user_id)
        llm = None
        skill_registry = None
        if self._core is not None:
            llm = getattr(self._core, "_llm_client", None)
```

```

        skill_registry = getattr(self._core, "_skill_registry", None)
    agent = SessionAgent(
        conversation_id=conv_id,
        context=context,
        bus=self._bus,
        llm_client=llm,
        skill_registry=skill_registry,
    )
    logger.info(
        "SessionFactory created session: conv=%s user=%s llm=%s
skill_registry=%s",
        conv_id, user_id or "?",
        "yes" if llm else "no",
        "yes" if skill_registry else "no",
    )
    return agent
def _build_context(self, message: "StandardMessage", user_id: str) ->
SessionContext:
    return SessionContext.create(
        user_id=user_id,
        conversation_id=message.conversation_id,
        sender_name=message.sender_name or "",
        core=self._core,
    )

```

文件编号: 3 文件路径: emily-core\emily_core\adapters\session\session_pool.py

```

from __future__ import annotations
import asyncio
import logging
import time
from typing import TYPE_CHECKING
from .session_config import SessionConfig
from .session_factory import SessionFactory
if TYPE_CHECKING:
    from ...adapters.standard.message import StandardMessage
    from ...adapters.standard.reply import ReplyMessage
    from ...session.session_agent import SessionAgent
    from ...workitem.pipeline.bus import PipelineBUS
logger = logging.getLogger("emily.session_pool")
class _Entry:
    __slots__ = ("agent", "last_active", "lock")
    def __init__(self, agent):
        self.agent = agent
        self.last_active = time.time()
        self.lock = asyncio.Lock()
class SessionPoolManager:
    def __init__(
        self,
        bus: "PipelineBUS",
        config: SessionConfig | None = None,
        factory: SessionFactory | None = None,
    )

```

```

        core=None,
    ):
        self._config = config or SessionConfig()
        self._factory = factory or SessionFactory(bus, core=core)
        self._sessions: dict[str, _Entry] = {}
        self._start_time = time.time()
        self._sweeper_task: asyncio.Task | None = None
    async def _start_sweeper(self) -> None:
        if self._sweeper_task is not None:
            return
        interval = getattr(self._config, "sweep_interval_seconds", 300) or 300
        self._sweeper_task = asyncio.create_task(self._sweep_loop(interval))
        logger.info("SessionPool sweeper started (interval=%ds)", interval)
    async def _sweep_loop(self, interval: int) -> None:
        while True:
            await asyncio.sleep(interval)
            try:
                removed = self.sweep_expired()
                if removed:
                    logger.debug("SessionPool sweeper removed %d expired
sessions", removed)
            except Exception as e:
                logger.warning("SessionPool sweeper error: %s", e)
    async def _ensure_sweeper(self) -> None:
        if self._sweeper_task is None:
            await self._start_sweeper()
    async def route(self, message: "StandardMessage", user_id: str = "") ->
"ReplyMessage | None":
        conv_id = message.conversation_id
        entry = self._sessions.get(conv_id)
        await self._ensure_sweeper()
        if entry is None:
            if len(self._sessions) >= self._config.max_concurrent:
                self.sweep_expired()
            agent = self._factory.create(message, user_id=user_id)
            entry = _Entry(agent)
            self._sessions[conv_id] = entry
            logger.info("SessionPool created: conv=%s (pool size=%d)",
conv_id, len(self._sessions))
        else:
            logger.debug("SessionPool hit: conv=%s", conv_id)
        async with entry.lock:
            entry.last_active = time.time()
            return await entry.agent.handle(message)
    def lookup(self, conversation_id: str) -> "SessionAgent | None":
        entry = self._sessions.get(conversation_id)
        return entry.agent if entry else None
    async def terminate(self, conversation_id: str) -> bool:
        entry = self._sessions.get(conversation_id)
        if entry is None:
            return False
        try:
            await entry.agent.archive()
        except Exception as e:

```

```

        logger.warning("SessionPool terminate archive failed: %s", e)
        self._sessions.pop(conversation_id, None)
        logger.info("SessionPool terminated: conv=%s", conversation_id)
        return True
def sweep_expired(self) -> int:
    now = time.time()
    ttl = self._config.ttl_seconds
    expired = [
        cid for cid, e in self._sessions.items()
        if now - e.last_active > ttl
    ]
    for cid in expired:
        entry = self._sessions.pop(cid, None)
        if entry:
            try:
                asyncio.ensure_future(entry.agent.archive())
            except Exception as e:
                logger.warning("SessionPool sweep archive failed for %s: %s",
cid, e)
        if expired:
            logger.info("SessionPool swept %d expired session(s)", len(expired))
    return len(expired)
@property
def size(self) -> int:
    return len(self._sessions)
@property
def uptime_seconds(self) -> int:
    return int(time.time() - self._start_time)

```

文件编号: 4 文件路径: emily-core\emily_core\adapters\standard\command.py

```

from dataclasses import dataclass
@dataclass
class EventCommand:
    project_id: str | None = None
    project_name: str | None = None
    title: str = ""
    event_type: str = "general"
    category: str = "待分类"
    description: str = ""
    event_date: str | None = None
    creator_id: str = ""
    source_message_id: str = ""
    related_event_ids: list[str] | None = None
    conversation_id: str = ""
@dataclass
class TaskCommand:
    project_id: str | None = None
    project_name: str | None = None
    title: str = ""
    description: str = ""
    assignee_text: str = ""

```

```

    due_date: str | None = None
    due_text: str = ""
    creator_id: str = ""
    source_message_id: str = ""
@dataclass
class MeetingCommand:
    project_id: str | None = None
    project_name: str | None = None
    title: str = ""
    summary: str = ""
    attendees: list[str] | None = None
    creator_id: str = ""
    source_message_id: str = ""
@dataclass
class FileCommand:
    project_id: str | None = None
    project_name: str | None = None
    filename: str = ""
    file_type: str = ""
    file_size: int = 0
    storage_path: str = ""
    file_category: str = "OTHER"
    uploaded_by: str = ""
    source_message_id: str = ""
@dataclass
class QueryCommand:
    query_type: str = "event"
    project_id: str | None = None
    project_ids: list[str] | None = None
    project_name: str | None = None
    time_range: str = "all"
    status_filter: str | None = None
    assignee: str | None = None
    sender_name: str | None = None
    keyword: str | None = None
    intent: str | None = None
    file_type: str | None = None
    conversation_id: str | None = None
    limit: int = 50

```

文件编号: 5 文件路径: emily-core\emily_core\adapters\standard\message.py

```

from dataclasses import dataclass, field
from datetime import datetime
@dataclass
class StandardMessage:
    message_id: str = ""
    platform: str = ""
    conversation_type: str = ""
    conversation_id: str = ""
    sender_id: str = ""
    sender_name: str = ""

```

```

group_id: str | None = None
group_name: str | None = None
content: str = ""
is_at_bot: bool = False
mentioned_user_ids: list[str] = field(default_factory=list)
reply_to_message_id: str | None = None
timestamp: datetime | None = None
raw_event: dict | None = None
msg_type: int = 1
attachments: list[dict] = field(default_factory=list)

```

文件编号: 6 文件路径: emily-core\emily_core\adapters\standard\reply.py

```

from dataclasses import dataclass, field
@dataclass
class ReplyMessage:
    conversation_id: str
    content: str
    reply_to_message_id: str | None = None
    references: list[dict] = field(default_factory=list)
    metadata: dict = field(default_factory=dict)
    file_paths: list[dict] = field(default_factory=list)

```

文件编号: 7 文件路径: emily-core\emily_core\adapters\standard\result.py

```

from dataclasses import dataclass, field
@dataclass
class RouteResult:
    intent: str
    project_id: str | None = None
    project_name: str | None = None
    confidence: float = 0.0
    data: dict = field(default_factory=dict)
@dataclass
class HandlerResult:
    success: bool
    object_type: str | None = None
    object_id: str | None = None
    reply: str | None = None
    error_code: str | None = None
    pending_confirmation: bool = False
@dataclass
class AgentStep:
    step_index: int
    type: str
    detail: str
    timestamp: str = ""
@dataclass
class AgentResult:

```

```

success: bool
reply: str
steps: list[AgentStep] = field(default_factory=list)
error_code: str | None = None

```

文件编号: 8 文件路径: emily-core\emily_core\adapters\standard\route_decision.py

```

from dataclasses import dataclass
@dataclass
class RouteDecision:
    takeover: bool
    mode: str = "collaborate"
    intent: str | None = None
    confidence: float = 0.0
    handler: str | None = None
    should_reply: bool = True
    reason: str = ""

```

文件编号: 9 文件路径: emily-core\emily_core\agent\sop_parser.py

```

import re
from typing import Any
_md: Any = None
def _get_md() -> Any:
    global _md
    if _md is None:
        import mistune # type: ignore[import-untyped]
        _md = mistune.create_markdown(renderer="ast", plugins=["table"])
    return _md
def _parse_to_tokens(content: str) -> list[dict[str, Any]]:
    return _get_md()(content) # type: ignore[no-any-return]
def _extract_text(children: list[dict[str, Any]]) -> str:
    parts: list[str] = []
    for child in children:
        ttype = child.get("type", "")
        if ttype == "text":
            parts.append(child.get("raw", ""))
        elif ttype == "codespan":
            attrs = child.get("attrs", {})
            parts.append(attrs.get("text", child.get("raw", "")))
        elif ttype in (
            "strong", "emphasis", "link", "image",
            "block_text", "block_code", "paragraph",
        ):
            parts.append(_extract_text(child.get("children", [])))
        elif ttype in ("softbreak", "linebreak"):
            parts.append(" ")
        elif child.get("children"):
            parts.append(_extract_text(child["children"]))

```



```

    return "".join(parts)
def _extract_codespan_values(children: list[dict[str, Any]]) -> list[str]:
    codes: list[str] = []
    for child in children:
        ttype = child.get("type", "")
        if ttype == "codespan":
            attrs = child.get("attrs", {})
            codes.append(attrs.get("text", child.get("raw", "")))
        elif child.get("children"):
            codes.extend(_extract_codespan_values(child["children"]))
    return codes
def _has_strong_prefix(children: list[dict[str, Any]], text_prefix: str) -> bool:
    for child in children:
        if child.get("type") == "strong":
            txt = _extract_text(child.get("children", []))
            if txt.startswith(text_prefix):
                return True
    return False
def _collect_paragraph_texts(children: list[dict[str, Any]]) -> list[str]:
    lines: list[str] = []
    for child in children:
        if child.get("type") == "paragraph":
            lines.append(_extract_text(child.get("children", [])))
        elif child.get("children"):
            lines.extend(_collect_paragraph_texts(child["children"]))
    return lines
class _SOPBlockWalker:
    def __init__(self) -> None:
        self._h2_num: int | None = None
        self._h3_id: str | None = None
        self._in_must: bool = False
        self._in_deny: bool = False
        self._in_pos_example: bool = False
        self._in_neg_example: bool = False
        self.display_name: str = ""
        self.sop_id: str = ""
        self.sop_type: str = ""
        self.version: str = ""
        self.allow_roles: list[str] = []
        self.trigger_keywords: list[str] = []
        self.deny_conditions: list[str] = []
        self.positive_examples: list[str] = []
        self.negative_examples: list[str] = []
        self.allowed_tools: list[str] = []
    def walk(self, tokens: list[dict[str, Any]]) -> dict[str, Any]:
        for token in tokens:
            ttype: str = token.get("type", "")
            if ttype == "heading":
                self._on_heading(token)
            elif ttype == "table":
                self._on_table(token)
            elif ttype == "list":
                self._on_list(token)
            elif ttype == "block_quote":

```

```

        self._on_block_quote(token)
    elif ttype == "paragraph":
        self._on_paragraph(token)
    return {
        "display_name": self.display_name,
        "sop_id": self.sop_id,
        "sop_type": self.sop_type,
        "version": self.version,
        "allow_roles": tuple(self.allow_roles) if self.allow_roles else (),
        "trigger_keywords": tuple(self.trigger_keywords),
        "deny_conditions": tuple(self.deny_conditions),
        "positive_examples": tuple(self.positive_examples),
        "negative_examples": tuple(self.negative_examples),
        "allowed_tools": list(self.allowed_tools),
    }

def _on_heading(self, token: dict[str, Any]) -> None:
    children: list[dict[str, Any]] = token.get("children", [])
    text = _extract_text(children).strip()
    level: int = token.get("attrs", {}).get("level", 0)
    if level == 1:
        name = re.sub(r"\s*-\s*业务流服务手册\s*$", "", text)
        self.display_name = name.strip()
    elif level == 2:
        m = re.match(r"(\d+)\.", text)
        self._h2_num = int(m.group(1)) if m else None
        self._h3_id = None
        self._reset_section_state()
    elif level == 3:
        m = re.match(r"(\d+\.\d+)", text)
        self._h3_id = m.group(1) if m else None
        self._reset_section_state()

def _on_table(self, token: dict[str, Any]) -> None:
    if self._h2_num == 1:
        self._extract_chapter1_table(token)
    elif self._h2_num == 3 and self._h3_id == "3.2":
        self._extract_section_32_table(token)

def _on_list(self, token: dict[str, Any]) -> None:
    if self._h2_num != 2:
        return
    if self._h3_id not in ("2.1", None):
        return
    for item_token in token.get("children", []):
        if item_token.get("type") != "list_item":
            continue
        item_text = _extract_text(item_token.get("children", [])).strip()
        if not item_text:
            continue
        if self._in_must:
            sub_items = re.split(r"[\s;]", item_text)
            for sub in sub_items:
                sub = sub.strip()
                if sub and len(sub) > 2:
                    self.trigger_keywords.append(sub)
        elif self._in_deny:

```

```

        self.deny_conditions.append(item_text)
def _on_block_quote(self, token: dict[str, Any]) -> None:
    if self._h2_num != 2 or self._h3_id != "2.2":
        return
    lines = _collect_paragraph_texts(token.get("children", []))
    if not lines:
        return
    for line_text in lines:
        line_text = line_text.strip()
        if not line_text:
            continue
        if self._in_pos_example:
            quotes = re.findall(r"「([^\"]+)」", line_text)
            for q in quotes:
                q = q.strip()
                if q:
                    self.positive_examples.append(q)
        elif self._in_neg_example:
            quotes = re.findall(r"「([^\"]+)」", line_text)
            for q in quotes:
                q = q.strip()
                if q:
                    self.negative_examples.append(q)
            arrows = re.findall(r"「\s*→\s*(.+?)」(?=「|$)」", line_text)
            for a in arrows:
                a = a.strip()
                if a and a not in self.negative_examples:
                    self.negative_examples.append(a)
def _on_paragraph(self, token: dict[str, Any]) -> None:
    children: list[dict[str, Any]] = token.get("children", [])
    if _has_strong_prefix(children, "必须条件"):
        self._in_must = True
        self._in_deny = False
    elif _has_strong_prefix(children, "否定条件"):
        self._in_must = False
        self._in_deny = True
    elif _has_strong_prefix(children, "应触发此业务流"):
        self._in_pos_example = True
        self._in_neg_example = False
    elif _has_strong_prefix(children, "不应触发此业务流"):
        self._in_pos_example = False
        self._in_neg_example = True
def _extract_chapter1_table(self, token: dict[str, Any]) -> None:
    for section in token.get("children", []):
        for row_token in section.get("children", []):
            if row_token.get("type") != "table_row":
                continue
            cells: list[dict[str, Any]] = row_token.get("children", [])
            if len(cells) < 2:
                continue
            key = _extract_text(cells[0].get("children", [])).strip()
            val_children = cells[1].get("children", [])
            if key == "业务流程编号" and not self.sop_id:
                self.sop_id = _extract_text(val_children).strip()

```

```

        elif key == "版本" and not self.version:
            self.version = _extract_text(val_children).strip()
        elif key == "权限控制":
            roles = _extract_codespan_values(val_children)
            seen: set[str] = set()
            unique: list[str] = []
            for r in roles:
                if r not in seen:
                    seen.add(r)
                    unique.append(r)
            if unique:
                self.allow_roles = unique
        elif key == "业务类型" and not self.sop_type:
            self.sop_type = _extract_text(val_children).strip()
    def _extract_section_32_table(self, token: dict[str, Any]) -> None:
        for section in token.get("children", []):
            for row_token in section.get("children", []):
                if row_token.get("type") != "table_row":
                    continue
                cells: list[dict[str, Any]] = row_token.get("children", [])
                if not cells:
                    continue
                tool_names = _extract_codespan_values(cells[0].get("children",
[]))

                for name in tool_names:
                    if (
                        name.startswith(("record_", "submit_", "review_",
"query_", "invoke_", "manage_", "write_"))
                    ):
                        if name not in self.allowed_tools:
                            self.allowed_tools.append(name)
    def _reset_section_state(self) -> None:
        self._in_must = False
        self._in_deny = False
        self._in_pos_example = False
        self._in_neg_example = False
    def parse_sop_markdown(content: str, file_path: str, file_name: str) -> dict[str,
Any]:
        tokens = _parse_to_tokens(content)
        walker = _SOPBlockWalker()
        return walker.walk(tokens)
    def extract_allowed_tools_from_sop(sop_text: str) -> list[str]:
        tokens = _parse_to_tokens(sop_text)
        walker = _SOPBlockWalker()
        result = walker.walk(tokens)
        return result.get("allowed_tools", [])

```

文件编号: 10 文件路径: emily-core\emily_core\application_user_utils.py

```

import logging
logger = logging.getLogger("emily.app.user_utils")
def resolve_user_name(user_id: str) -> str:

```

```

if not user_id:
    return ""
try:
    from ..repositories.user_repo import UserRepository
    u = UserRepository.get(user_id)
    if u:
        return u.username or ""
except Exception as e:
    logger.debug("resolve_user_name failed for %s: %s", user_id, e)
return ""

```

文件编号: 11 文件路径: emily-core\emily_core\application\event_app.py

```

import asyncio
import json
import logging
from typing import Optional
from ..adapters.standard.result import RouteResult, HandlerResult
from ..adapters.standard.command import EventCommand
from ..services.event_service import EventService
from ..infrastructure.database.models import Event
logger = logging.getLogger("emily.app.event")
def _log_business_event(**kwargs) -> None:
    try:
        from ..infrastructure.logging.business_event_logger import
BusinessEventLogger
        ctx = BusinessEventLogger._current_context
        kwargs.setdefault("pipeline_run_id", ctx.get("pipeline_run_id", ""))
        kwargs.setdefault("conversation_id", ctx.get("conversation_id", ""))
        asyncio.ensure_future(BusinessEventLogger.log(**kwargs))
    except Exception:
        pass
class EventApplication:
    def __init__(self, event_service: EventService):
        self.event_service = event_service
        self._journal = None
    def set_journal(self, journal) -> None:
        self._journal = journal
    async def handle_event(
        self,
        route_result: RouteResult,
        user_id: str,
        message_id: str,
    ) -> HandlerResult:
        try:
            cmd = self._build_command(route_result, user_id, message_id)
            event = self.event_service.create_pending_event(cmd)
            project_name = route_result.project_name
            reply = EventService.format_confirmation_reply(event, project_name)
            _log_business_event(
                event_category="event",
                event_action="created",

```

```

        target_type="event",
        target_id=event.id,
        target_no=getattr(event, "event_no", "") or "",
        summary=f"创建事件: {event.title[:100]}",
        user_id=user_id,
        project_id=route_result.project_id or "",
    )
    return HandlerResult(
        success=True,
        object_type="event",
        object_id=event.id,
        reply=reply,
        pending_confirmation=True,
    )
except Exception as e:
    logger.error("Event handling failed: %s", e, exc_info=True)
    return HandlerResult(
        success=False,
        error_code="event_create_failed",
        reply=f"事件录入失败: {e}",
    )
def handle_confirmation(
    self,
    event_id: str,
    action: str,
) -> HandlerResult:
    if action == "confirm":
        event = self.event_service.confirm_event(event_id)
        if event:
            if self._journal is not None:
                from ._user_utils import resolve_user_name
                user_name = resolve_user_name(event.user_id) if event.user_id
            else ""

            self._journal.append(
                name=user_name or "用户",
                summary=f"确认录入事件: {event.title} ({event.event_no})",
            )
            _log_business_event(
                event_category="event",
                event_action="confirmed",
                target_type="event",
                target_id=event.id,
                target_no=getattr(event, "event_no", "") or "",
                summary=f"确认事件: {event.title[:100]}",
                user_id=event.user_id or "",
                project_id=getattr(event, "project_id", "") or "",
            )
            return HandlerResult(
                success=True,
                object_type="event",
                object_id=event.id,
                reply=f"✅ 已记录该事件 ({event.event_no})",
            )
        else:

```

```

        return HandlerResult(
            success=False,
            error_code="confirm_failed",
            reply="确认失败, 事件不存在或已处理",
        )
    elif action == "cancel":
        event = self.event_service.cancel_event(event_id)
        if event:
            return HandlerResult(
                success=True,
                object_type="event",
                object_id=event.id,
                reply="✗ 已取消录入",
            )
        else:
            return HandlerResult(
                success=False,
                error_code="cancel_failed",
                reply="取消失败, 事件不存在或已处理",
            )
    else:
        return HandlerResult(
            success=False,
            error_code="unknown_action",
            reply=f"未知操作: {action}",
        )

    @staticmethod
    def _build_command(
        route_result: RouteResult,
        user_id: str,
        message_id: str,
    ) -> EventCommand:
        data = route_result.data or {}
        related = data.get("related_event_ids")
        if isinstance(related, str):
            import json as _json
            try:
                related = _json.loads(related)
            except (_json.JSONDecodeError, TypeError):
                related = [related] if related else []
        return EventCommand(
            project_id=route_result.project_id,
            project_name=route_result.project_name,
            title=data.get("title", "未命名事件"),
            event_type=data.get("event_type", "general"),
            category="待分类",
            description=data.get("description", ""),
            event_date=data.get("event_date"),
            creator_id=user_id,
            source_message_id=message_id,
            related_event_ids=related if isinstance(related, list) else None,
            conversation_id=data.get("_conversation_id", ""),
        )

```

文件编号: 12 文件路径: emily-core\emily_core\application\file_app.py

```
import logging
from ..adapters.standard.result import RouteResult, HandlerResult
from ..adapters.standard.command import FileCommand
from ..services.file_service import FileService
logger = logging.getLogger("emily.app.file")
class FileApplication:
    def __init__(self, file_service: FileService, storage_service=None):
        self.file_service = file_service
        self.storage_service = storage_service
        self._journal = None
    def set_journal(self, journal) -> None:
        self._journal = journal
    def set_storage_service(self, storage_service) -> None:
        self.storage_service = storage_service
    async def handle_file(
        self, route_result: RouteResult, user_id: str, message_id: str,
        attachment_url: str = "", attachment_type: int = 0,
        source_filename: str = "",
    ) -> HandlerResult:
        try:
            data = route_result.data or {}
            filename = data.get("filename", "未命名文件")
            cmd = FileCommand(
                project_id=route_result.project_id,
                project_name=route_result.project_name,
                filename=filename,
                file_type=data.get("file_type", ""),
                uploaded_by=user_id,
                source_message_id=message_id,
                file_category=data.get("file_category", "OTHER"),
            )
            f = self.file_service.create_file_record(cmd)
            local_path = ""
            if attachment_url and self.storage_service:
                try:
                    store_result = self.storage_service.store_attachment(
                        message_id=message_id,
                        attachment_url=attachment_url,
                        attachment_type=attachment_type or 3,
                        source_filename=source_filename or filename,
                    )
                    if store_result:
                        local_path = store_result.get("local_path", "")
                        logger.info("File physically stored: %s", local_path)
                except Exception as e:
                    logger.warning("Physical file storage failed (non-blocking):
%s", e)
            if self._journal is not None:
                from ._user_utils import resolve_user_name
```



```

        user_name = resolve_user_name(cmd.uploaded_by) or "用户"
        summary = f"归档文件: {f.filename} ({f.file_no}) "
        if local_path:
            summary += f" → {local_path}"
        self._journal.append(name=user_name, summary=summary)
    reply = FileService.format_reply(f)
    if local_path:
        reply += f"\n文件已保存到本地存储。"
    return HandlerResult(
        success=True, object_type="file", object_id=f.id, reply=reply,
    )
except Exception as e:
    logger.error("File record creation failed: %s", e, exc_info=True)
    return HandlerResult(
        success=False, error_code="file_create_failed", reply=f"文件归档失
败: {e}",
    )
async def handle_list_by_category(
    self,
    file_category: str | None = None,
    project_id: str | None = None,
    project_ids: list[str] | None = None,
    keyword: str = "",
    limit: int = 10,
) -> HandlerResult:
    try:
        from ..infrastructure.database.models import FileCategory
        files = self.file_service.list_by_category(
            project_id=project_id,
            project_ids=project_ids,
            file_category=file_category,
            limit=limit,
        )
        if not files:
            cat_name = FileCategory.display(file_category) if file_category
else "文件"
            return HandlerResult(
                success=True,
                reply=f"🐼 {cat_name}类暂无文件记录。",
            )
        cat_name = FileCategory.display(file_category) if file_category else
"全部"
        lines = [f"🐼 {cat_name}类文件 (共 {len(files)} 份)",
"_____"]
        for i, f in enumerate(files[:10], 1):
            cat_display = FileCategory.display(getattr(f, 'file_category',
'OTHER'))
            lines.append(f"{i}. {f.filename} ({f.file_no}) [{cat_display}]")
        if len(files) > 10:
            lines.append(f"... 还有 {len(files) - 10} 份")
        return HandlerResult(
            success=True,
            reply="\n".join(lines),
            data={

```

```

        "total": len(files),
        "category": file_category,
        "files": [
            {
                "file_no": f.file_no,
                "filename": f.filename,
                "file_category": getattr(f, 'file_category', 'OTHER'),
            }
            for f in files[:20]
        ],
    },
)
except Exception as e:
    logger.error("List files by category failed: %s", e, exc_info=True)
    return HandlerResult(
        success=False,
        error_code="file_query_failed",
        reply=f"文件查询失败: {e}",
    )
async def handle_update_category(
    self,
    file_no: str,
    file_category: str,
    user_id: str = "",
) -> HandlerResult:
    try:
        from ..infrastructure.database.models import FileCategory
        f = self.file_service.repo.get_by_file_no(file_no)
        if f is None:
            return HandlerResult(
                success=False,
                reply=f"找不到文件编号 {file_no}",
            )
        old_category = getattr(f, 'file_category', 'OTHER')
        old_display = FileCategory.display(old_category)
        new_display = FileCategory.display(file_category)
        result = self.file_service.update_file_category(
            file_id=f.id,
            file_category=file_category,
            operator_id=user_id,
        )
        if result is None:
            return HandlerResult(
                success=False,
                reply=f"文件分类更新失败",
            )
        return HandlerResult(
            success=True,
            reply=f"✅ 文件「{f.filename}」已从「{old_display}」改到「{new_display}」",
            data={
                "file_no": file_no,
                "old_category": old_category,
                "new_category": file_category,
            }
        )
    except Exception as e:
        logger.error("Update file category failed: %s", e, exc_info=True)
        return HandlerResult(
            success=False,
            error_code="file_update_failed",
            reply=f"文件分类更新失败: {e}",
        )

```

```

    },
)
except Exception as e:
    logger.error("Update file category failed: %s", e, exc_info=True)
    return HandlerResult(
        success=False,
        error_code="file_category_update_failed",
        reply=f"分类更新失败: {e}",
    )

```

文件编号: 13 文件路径: emily-core\emily_core\application\meeting_app.py

```

import asyncio
import logging
from ..adapters.standard.result import RouteResult, HandlerResult
from ..adapters.standard.command import MeetingCommand
from ..services.meeting_service import MeetingService
logger = logging.getLogger("emily.app.meeting")
def _log_business_event(**kwargs) -> None:
    try:
        from ..infrastructure.logging.business_event_logger import
        BusinessEventLogger
        ctx = BusinessEventLogger._current_context
        kwargs.setdefault("pipeline_run_id", ctx.get("pipeline_run_id", ""))
        kwargs.setdefault("conversation_id", ctx.get("conversation_id", ""))
        asyncio.ensure_future(BusinessEventLogger.log(**kwargs))
    except Exception:
        pass
class MeetingApplication:
    def __init__(self, meeting_service: MeetingService):
        self.meeting_service = meeting_service
        self._journal = None
    def set_journal(self, journal) -> None:
        self._journal = journal
    async def handle_meeting(
        self, route_result: RouteResult, user_id: str, message_id: str
    ) -> HandlerResult:
        try:
            data = route_result.data or {}
            cmd = MeetingCommand(
                project_id=route_result.project_id,
                project_name=route_result.project_name,
                title=data.get("title", "未命名会议"),
                summary=data.get("summary", ""),
                attendees=data.get("attendees") or [],
                creator_id=user_id,
                source_message_id=message_id,
            )
            meeting = self.meeting_service.create_meeting(cmd)
            if self._journal is not None:
                from ._user_utils import resolve_user_name
                user_name = resolve_user_name(cmd.creator_id) or "用户"

```

```

        self._journal.append(
            name=user_name,
            summary=f"录入会议纪要: {meeting.title}
({meeting.meeting_no}) ",
        )
        from ._user_utils import resolve_user_name
        _uname = resolve_user_name(cmd.creator_id) or ""
        _log_business_event(
            event_category="meeting",
            event_action="created",
            target_type="meeting",
            target_id=meeting.id,
            target_no=getattr(meeting, "meeting_no", "") or "",
            summary=f"录入会议: {meeting.title[:100]}",
            user_id=user_id,
            user_name=_uname,
            project_id=route_result.project_id or "",
        )
        reply = MeetingService.format_reply(meeting)
        return HandlerResult(
            success=True, object_type="meeting", object_id=meeting.id,
reply=reply,
        )
    except Exception as e:
        logger.error("Meeting creation failed: %s", e, exc_info=True)
        return HandlerResult(
            success=False, error_code="meeting_create_failed", reply=f"会议归
档失败: {e}",
        )

```

文件编号: 14 文件路径: emily-core\emily_core\application\node_app.py

```

from __future__ import annotations
import logging
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from ..services.node_service import NodeService
    from ..services.node_commands import (
        CreateNodeCommand,
        UpdateNodeCommand,
        DiscardNodeCommand,
        ActivateNodeCommand,
        CreateDeliverableCommand,
        UpdateDeliverableProgressCommand,
        AddDependencyCommand,
        RemoveDependencyCommand,
        MountChildCommand,
        UnmountChildCommand,
    )
logger = logging.getLogger("emily.node_app")
class NodeApplication:
    def __init__(self, service: "NodeService", auth_engine=None):

```

```
self._service = service
self._auth_engine = auth_engine
async def _check_node_permission(self, node_id: str, operator_id: str,
                                operation: str = "write") -> bool:
    if not self._auth_engine or not operator_id:
        return True
    try:
        result = await self._auth_engine.check_access(
            perms=None,
            resource_type="NODE",
            resource_id=node_id,
            operation=operation,
        )
        return result.allowed
    except Exception:
        logger.warning("Permission check failed for node=%s user=%s", node_id,
operator_id)
        return True
def _require_permission(self, allowed: bool, node_id: str, operation: str) ->
None:
    if not allowed:
        raise PermissionError(f"无权限对节点 {node_id} 执行 {operation} 操作")
async def create_node(self, cmd: "CreateNodeCommand") -> dict:
    result = await self._service.create_node(cmd)
    return {
        "success": result.success,
        "node_id": result.node_id,
        "status": result.status,
        "progress": result.progress,
        "reply": result.message,
        "error_code": result.error_code,
    }
async def update_node(self, cmd: "UpdateNodeCommand") -> dict:
    if cmd.operator_id:
        allowed = await self._check_node_permission(cmd.node_id,
cmd.operator_id, "write")
        self._require_permission(allowed, cmd.node_id, "write")
    result = await self._service.update_node(cmd)
    return {
        "success": result.success,
        "node_id": result.node_id,
        "status": result.status,
        "reply": result.message,
        "error_code": result.error_code,
    }
async def discard_node(self, cmd: "DiscardNodeCommand") -> dict:
    if cmd.operator_id:
        allowed = await self._check_node_permission(cmd.node_id,
cmd.operator_id, "delete")
        self._require_permission(allowed, cmd.node_id, "delete")
    result = await self._service.discard_node(cmd)
    return {
        "success": result.success,
        "node_id": result.node_id,
```

```

        "reply": result.message,
        "error_code": result.error_code,
    }
    async def activate_node(self, cmd: "ActivateNodeCommand") -> dict:
        try:
            result = await self._service.activate_node(cmd)
            return {
                "success": result.success,
                "node_id": result.node_id,
                "status": result.status,
                "reply": result.message,
                "error_code": result.error_code,
            }
        except Exception as e:
            logger.error("activate_node failed: %s", e)
            return {"success": False, "node_id": cmd.node_id, "reply": f"节点激活失
败: {e}"}
    async def create_deliverable(self, cmd: "CreateDeliverableCommand") -> dict:
        result = await self._service.create_deliverable(cmd)
        return {
            "success": result.success,
            "node_id": result.node_id,
            "reply": result.message,
            "error_code": result.error_code,
        }
    async def update_deliverable_progress(self, cmd:
"UpdateDeliverableProgressCommand") -> dict:
        result = await self._service.update_deliverable_progress(cmd)
        return {
            "success": result.success,
            "node_id": result.node_id,
            "status": result.status,
            "progress": result.progress,
            "reply": result.message,
            "affected_ancestors": result.affected_downstream,
            "error_code": result.error_code,
        }
    async def add_dependency(self, cmd: "AddDependencyCommand") -> dict:
        result = await self._service.add_dependency(cmd)
        return {
            "success": result.success,
            "node_id": result.node_id,
            "reply": result.message,
            "error_code": result.error_code,
        }
    async def remove_dependency(self, cmd: "RemoveDependencyCommand") -> dict:
        result = await self._service.remove_dependency(cmd)
        return {
            "success": result.success,
            "node_id": result.node_id,
            "reply": result.message,
            "error_code": result.error_code,
        }
    async def mount_child(self, cmd: "MountChildCommand") -> dict:

```

```

        result = await self._service.mount_child(cmd)
        return {
            "success": result.success,
            "node_id": result.node_id,
            "reply": result.message,
            "error_code": result.error_code,
        }
    async def unmount_child(self, cmd: "UnmountChildCommand") -> dict:
        result = await self._service.unmount_child(cmd)
        return {
            "success": result.success,
            "node_id": result.node_id,
            "reply": result.message,
            "error_code": result.error_code,
        }
    async def get_node_detail(self, node_id: str) -> dict:
        detail = await self._service.get_node_detail(node_id)
        if detail is None:
            return {"success": False, "reply": f"节点 {node_id} 不存在"}
        return {"success": True, "data": detail, "reply": f"节点 [{detail['node_name']}] 查询成功"}

```

文件编号: 15 文件路径: emily-core\emily_core\application\permission_app.py

```

from __future__ import annotations
import logging
from typing import Optional
from emily_core.services.permission_service import PermissionService
logger = logging.getLogger("emily.permission.app")
class PermissionApplication:
    def __init__(self, service: PermissionService):
        self._service = service
    async def check_permission(self, user_id: str, sop_id: str) -> dict:
        try:
            result = await self._service.check(user_id, sop_id)
            result["success"] = True
            return result
        except Exception as e:
            logger.error("check_permission failed: %s", e)
            return {"success": False, "allowed": False, "reason": str(e), "reply": f"权限校验失败: {e}"}
    async def grant_permission(self, *, grantee_id: str, grantor_id: str,
                               perm_code: str, grant_type: str = "TEMP",
                               operations: str = '["read"]',
                               expire_time: Optional[str] = None,
                               remark: str = "", client_ip: str = "") -> dict:
        try:
            return await self._service.grant(
                grantee_id=grantee_id,
                grantor_id=grantor_id,
                perm_code=perm_code,
                grant_type=grant_type,

```

```

        operations=operations,
        expire_time=expire_time,
        remark=remark,
        client_ip=client_ip,
    )
except Exception as e:
    logger.error("grant_permission failed: %s", e)
    return {"success": False, "reply": f"授权操作失败: {e}"}
async def revoke_permission(self, *, grant_no: str, revoke_reason: str = "",
                             operator_id: str = "") -> dict:
    try:
        return await self._service.revoke(
            grant_no=grant_no,
            revoke_reason=revoke_reason,
            operator_id=operator_id,
        )
    except Exception as e:
        logger.error("revoke_permission failed: %s", e)
        return {"success": False, "reply": f"撤销操作失败: {e}"}
async def query_user_permissions(self, user_id: str) -> dict:
    try:
        result = await self._service.query_user_permissions(user_id)
        if result.get("success"):
            perms = result["permissions"]
            level_name = perms.get("level_name", "?")
            sop_list = perms.get("sop_allow", [])
            grants = perms.get("active_grants", [])
            reply_parts = [
                f"📋 权限清单",
                f" 权限层级: L{perms.get('level', 1)} {level_name}",
                f" 所属单位: {perms.get('company_name', '-')}",
                f" 企业类型: {perms.get('company_type', '-')}",
                f" 部门: {perms.get('department', '-')}",
                f" 信息密级: {perms.get('info_level', '-')}",
                f" 可用SOP ({len(sop_list)}): {' '.join(sop_list[:10])}",
                {'...' if len(sop_list) > 10 else ''},
                f" 活跃授权 ({len(grants)}): " + (
                    ', '.join(g['grant_no'] for g in grants[:5])
                    if grants else '无'
                ),
            ]
            result["reply"] = '\n'.join(reply_parts)
        return result
    except Exception as e:
        logger.error("query_user_permissions failed: %s", e)
        return {"success": False, "reply": f"查询权限失败: {e}"}
async def request_permission(self, *, requester_id: str, perm_code: str,
                              request_type: str = "TEMP_GRANT",
                              reason: str = "", priority: str = "NORMAL") ->
dict:
    import asyncio
    from emily_core.infrastructure.database.models import _utc_now
    from emily_core.repositories.permission_repo import PermissionRepository
    try:

```



```

repo = PermissionRepository()
user = await asyncio.to_thread(repo.get_user, requester_id)
if user is None:
    return {"success": False, "reply": f"用户 {requester_id} 不存在"}
supervisor_id = user.supervisor_id or ""
from datetime import datetime
from emily_core.infrastructure.database.models import BEIJING_TZ
today = datetime.now(BEIJING_TZ).strftime("%Y%m%d")
from emily_core.infrastructure.database.session import get_session
from emily_core.infrastructure.database.models import

PermissionRequest
def _create_request():
    with get_session() as session:
        prefix = f"PRQ-{today}-"
        count = session.query(PermissionRequest).filter(
            PermissionRequest.request_no.like(prefix + "%")
        ).count()
        request_no = f"{prefix}{count + 1:04d}"
        from datetime import timedelta
        if priority == "URGENT":
            expire_delta = timedelta(hours=2)
        else:
            expire_delta = timedelta(hours=24)
        expire_at = (datetime.now(BEIJING_TZ) +
expire_delta).isoformat()
        req = PermissionRequest(
            request_no=request_no,
            requester_id=requester_id,
            perm_code=perm_code,
            request_type=request_type,
            reason=reason,
            status="PENDING",
            current_approver_id=supervisor_id,
            approval_level=1,
            priority=priority,
            expire_at=expire_at,
        )
        session.add(req)
        session.flush()
        return request_no, supervisor_id
request_no, approver = await asyncio.to_thread(_create_request)
return {
    "success": True,
    "request_no": request_no,
    "approver_id": approver,
    "reply": f"权限申请已提交（编号 {request_no}），待 {approver or '管
理员'} 审批",
}
except Exception as e:
    logger.error("request_permission failed: %s", e)
    return {"success": False, "reply": f"权限申请失败: {e}"}
async def approve_request(self, *, request_no: str, approver_id: str,
                           approved: bool, remark: str = "") -> dict:
    import asyncio

```

```

from emily_core.infrastructure.database.session import get_session
from emily_core.infrastructure.database.models import (
    PermissionRequest, PermissionGrant, _utc_now,
)
try:
    def _process_approval():
        with get_session() as session:
            req = session.query(PermissionRequest).filter(
                PermissionRequest.request_no == request_no,
                PermissionRequest.is_deleted == False,
            ).first()
            if req is None:
                return None, "申请记录不存在"
            if req.status != "PENDING":
                return None, f"申请状态为 {req.status}, 无法审批"
            if req.current_approver_id and req.current_approver_id !=
approver_id:
                return None, "您不是该申请的当前审批人"
            if approved:
                req.status = "APPROVED"
                req.approver_id = approver_id
                req.approval_remark = remark
                req.approved_at = _utc_now()
                grant_no_prefix = f"PGR-{_utc_now()[10].replace('-',
'' )}}-"

                grant_count = session.query(PermissionGrant).filter(
                    PermissionGrant.grant_no.like(grant_no_prefix + "%")
                ).count()
                grant_no = f"{grant_no_prefix}{grant_count + 1:04d}"
                grant = PermissionGrant(
                    grant_no=grant_no,
                    grantee_id=req.requester_id,
                    grantor_id=approver_id,
                    perm_code=req.perm_code,
                    grant_type="TEMP" if req.request_type == "TEMP_GRANT"
else "PERMANENT",

                    operations='["read"]',
                    status="ACTIVE",
                    remark=f"审批通过 {request_no}: {remark}",
                )
                session.add(grant)
            else:
                req.status = "REJECTED"
                req.approver_id = approver_id
                req.approval_remark = remark
                req.approved_at = _utc_now()
                session.flush()
                return req.request_no, "approved" if approved else "rejected"
req_no, result = await asyncio.to_thread(_process_approval)
if req_no is None:
    return {"success": False, "reply": result}
action = "通过" if approved else "拒绝"
return {
    "success": True,

```

```

        "request_no": req_no,
        "reply": f"权限申请 {req_no} 已{action}",
    }
except Exception as e:
    logger.error("approve_request failed: %s", e)
    return {"success": False, "reply": f"审批操作失败: {e}"}

```

文件编号: 16 文件路径: emily-core\emily_core\application\query_app.py

```

import logging
from typing import Optional
from ..adapters.standard.result import RouteResult, HandlerResult
from ..adapters.standard.command import QueryCommand
from ..services.query_service import QueryService
logger = logging.getLogger("emily.app.query")
class QueryApplication:
    def __init__(self, query_service: QueryService):
        self.query_service = query_service
    async def handle_query(
        self, route_result: RouteResult, user_id: str, message_id: str
    ) -> HandlerResult:
        try:
            data = route_result.data or {}
            cmd = QueryCommand(
                query_type=data.get("query_type", "event"),
                project_id=route_result.project_id,
                project_name=route_result.project_name,
                time_range=data.get("time_range", "all"),
                status_filter=data.get("status_filter"),
                assignee=data.get("assignee"),
                sender_name=data.get("sender_name"),
                keyword=data.get("keyword"),
                intent=data.get("intent"),
                file_type=data.get("file_type"),
                conversation_id=data.get("conversation_id"),
                limit=data.get("limit", 50),
            )
            logger.info(
                "Query: type=%s, project=%s, time=%s, limit=%d",
                cmd.query_type, cmd.project_id, cmd.time_range, cmd.limit,
            )
            results = self.query_service.execute(cmd)
            reply = self.query_service.format_reply(cmd.query_type, results)
            logger.info(
                "Query result: type=%s, total=%s",
                cmd.query_type, results.get("total", "N/A"),
            )
            return HandlerResult(
                success=True,
                object_type="query",
                reply=reply,
            )

```

```

except Exception as e:
    logger.error("Query failed: %s", e, exc_info=True)
    return HandlerResult(
        success=False,
        error_code="query_failed",
        reply=f"查询失败: {e}",
    )

```

文件编号: 17 文件路径: emily-core\emily_core\application\task_app.py

```

import asyncio
import logging
from ..adapters.standard.result import RouteResult, HandlerResult
from ..adapters.standard.command import TaskCommand
from ..services.task_service import TaskService
logger = logging.getLogger("emily.app.task")
def _log_business_event(**kwargs) -> None:
    try:
        from ..infrastructure.logging.business_event_logger import
        BusinessEventLogger
        ctx = BusinessEventLogger._current_context
        kwargs.setdefault("pipeline_run_id", ctx.get("pipeline_run_id", ""))
        kwargs.setdefault("conversation_id", ctx.get("conversation_id", ""))
        asyncio.ensure_future(BusinessEventLogger.log(**kwargs))
    except Exception:
        pass
class TaskApplication:
    def __init__(self, task_service: TaskService):
        self.task_service = task_service
        self._journal = None
    def set_journal(self, journal) -> None:
        self._journal = journal
    async def handle_task(
        self, route_result: RouteResult, user_id: str, message_id: str
    ) -> HandlerResult:
        try:
            data = route_result.data or {}
            cmd = TaskCommand(
                project_id=route_result.project_id,
                project_name=route_result.project_name,
                title=data.get("title", "未命名任务"),
                description=data.get("description", ""),
                assignee_text=data.get("assignee", ""),
                due_date=data.get("due_date"),
                due_text=data.get("due_text", ""),
                creator_id=user_id,
                source_message_id=message_id,
            )
            task = self.task_service.create_task(cmd)
            if self._journal is not None:
                from ._user_utils import resolve_user_name
                user_name = resolve_user_name(cmd.creator_id) or "用户"

```

```

        assignee = cmd.assignee_text or ""
        summary = f"创建任务: {task.title} ({task.task_no}) "
        if assignee:
            summary += f", 负责人{assignee}"
        self._journal.append(name=user_name, summary=summary)
    from ._user_utils import resolve_user_name
    _uname = resolve_user_name(cmd.creator_id) or ""
    _log_business_event(
        event_category="task",
        event_action="created",
        target_type="task",
        target_id=task.id,
        target_no=getattr(task, "task_no", "") or "",
        summary=f"创建任务: {task.title[:100]}",
        user_id=user_id,
        user_name=_uname,
        project_id=route_result.project_id or "",
    )
    reply = f"✅ 已创建任务 ({task.task_no}) \n—————\n标题:
{task.title}"
    if task.owner_text:
        reply += f"\n负责人: {task.owner_text}"
    if task.due_date:
        reply += f"\n截止日期: {task.due_date}"
    return HandlerResult(
        success=True, object_type="task", object_id=task.id, reply=reply,
    )
except Exception as e:
    logger.error("Task creation failed: %s", e, exc_info=True)
    return HandlerResult(
        success=False, error_code="task_create_failed", reply=f"任务创建失
败: {e}",
    )

```

文件编号: 18 文件路径: emily-core\emily_core\bootstrap.py

```

import logging
import os
from datetime import datetime, timezone, timedelta
from .config import Config
BEIJING_TZ = timezone(timedelta(hours=8))
_logger = logging.getLogger("emily.bootstrap")
def _setup_logging(config: Config) -> None:
    root_logger = logging.getLogger("emily")
    root_logger.setLevel(getattr(logging, config.log_level.upper(), logging.INFO))
    fmt = logging.Formatter(
        "%(asctime)s [%(levelname)s] %(name)s: %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S",
    )
    if not any(isinstance(h, logging.StreamHandler) for h in
root_logger.handlers):
        console = logging.StreamHandler()

```

```

        console.setLevel(logging.DEBUG)
        console.setFormatter(fmt)
        root_logger.addHandler(console)
    if config.log_to_file:
        try:
            os.makedirs(config.log_dir, exist_ok=True)
            date_str = datetime.now(BEIJING_TZ).strftime("%Y%m%d")
            log_file = os.path.join(config.log_dir, f"emily_{date_str}.log")
            file_handler = logging.FileHandler(log_file, encoding="utf-8")
            file_handler.setLevel(logging.DEBUG)
            file_handler.setFormatter(fmt)
            root_logger.addHandler(file_handler)
        except Exception as e:
            _logger.warning("File logging disabled: %s", e)
def _config_from_env(config_data: dict | None) -> dict:
    data = dict(config_data or {})
    env_map = {
        "EMILY_DATABASE_URL": "database_url",
        "EMILY_LLM_API_KEY": "llm_api_key",
        "EMILY_LLM_BASE_URL": "llm_base_url",
        "EMILY_LLM_MODEL": "llm_model",
        "EMILY_STORAGE_ROOT": "storage_root",
        "EMILY_HOOK_CONFIG_PATH": "hook_config_path",
        "EMILY_SOP_REPOSITORY_DIR": "sop_repository_dir",
        "EMILY_EXECUTOR_MODE": "executor_mode",
        "EMILY_PLANNER_MODE": "planner_mode",
        "EMILY_AUTH_MODE": "auth_mode",
        "EMILY_RISK_MODE": "risk_mode",
        "EMILY_MAXKB_URL": "maxkb_url",
        "EMILY_MAXKB_ADMIN_PASSWORD": "maxkb_admin_password",
        "EMILY_MAXKB_KNOWLEDGE_ID": "maxkb_knowledge_id",
        "EMILY_KB_ENABLED": "kb_enabled",
        "EMILY_PROMPTS_DIR": "prompts_dir",
    }
    for env_key, cfg_key in env_map.items():
        val = os.environ.get(env_key)
        if val and not data.get(cfg_key):
            data[cfg_key] = val
    return data
def init(config_data: dict | None = None, rag_provider=None) -> "EmilyCore":
    from . import EmilyCore
    config = Config.from_dict(_config_from_env(config_data))
    _setup_logging(config)
    from .infrastructure.database.session import init_db, get_db_path
    db_url = config.database_url if config.database_url else None
    init_db(db_url)
    _logger.info("Database ready: %s", get_db_path())
    if rag_provider is None and config.kb_enabled:
        try:
            from .providers.rag.maxkb_provider import MaxKBragProvider
            if config.maxkb_admin_password and config.maxkb_knowledge_id:
                rag_provider = MaxKBragProvider(
                    base_url=config.maxkb_url,
                    admin_password=config.maxkb_admin_password,

```

```

        knowledge_id=config.maxkb_knowledge_id,
    )
    _logger.info("MaxKB RAG provider created: kb=%s",
config.maxkb_knowledge_id[:8])
    except Exception as e:
        _logger.warning("MaxKB RAG provider init failed: %s", e)
    _logger.info(
        "Emily Core initialized, mode=%s, bot_name=%s, llm=%s, kb=%s",
        config.takeover_mode,
        config.bot_name,
        "configured" if config.llm_api_key else "disabled (Mock brain)",
        "enabled" if rag_provider else "disabled",
    )
    return EmilyCore(config, rag_provider=rag_provider)

```

文件编号: 19 文件路径: emily-core\emily_core\infrastructure\database\models.py

```

from datetime import datetime, timezone, timedelta
import uuid
from sqlalchemy import Column, String, Integer, BigInteger, Float, ForeignKey,
UniqueConstraint, Boolean, Text, Index, text
from sqlalchemy.orm import DeclarativeBase, relationship
class Base(DeclarativeBase):
    pass
BEIJING_TZ = timezone(timedelta(hours=8))
def _new_uuid() -> str:
    return str(uuid.uuid4())
def _new_id(prefix: str = "") -> str:
    now = datetime.now(BEIJING_TZ)
    date_part = now.strftime("%Y%m%d")
    short_uuid = uuid.uuid4().hex[:8]
    if prefix:
        return f"{prefix.upper()}-{date_part}-{short_uuid}"
    return f"{date_part}-{short_uuid}"
def _utc_now() -> str:
    return datetime.now(timezone.utc).isoformat()
def _beijing_now() -> str:
    return datetime.now(BEIJING_TZ).isoformat()
class User(Base):
    __tablename__ = "users"
    id = Column(String, primary_key=True, default=_new_uuid)
    username = Column(String(100), nullable=False)
    phone = Column(String(50))
    email = Column(String(200))
    status = Column(String(50), default="active")
    is_admin = Column(Boolean, default=False)
    gender = Column(Integer, default=0)
    id_card = Column(String(50), default="")
    qq = Column(String(50), default="")
    wechat = Column(String(100), default="")
    remark = Column(String, default="")
    creator_id = Column(String, nullable=False)

```

```

is_deleted = Column(Boolean, default=False)
perm_list = Column(String, default="[]")
org_category = Column(Integer, default=0)
level = Column(Integer, default=1)
supervisor_id = Column(String, nullable=True)
company = Column(String, ForeignKey("company_info.id"), nullable=True)
project_id = Column(String, ForeignKey("projects.id"), nullable=True)
position = Column(String, default="[]")
long_term_memory = Column(String, default="")
conversation_summary = Column(String, default="")
created_at = Column(String, default=_utc_now, nullable=False)
updated_at = Column(String, default=_utc_now, onupdate=_utc_now,
nullable=False)
im_bindings = relationship("UserImBinding", back_populates="user",
lazy="selectin")
class UserImBinding(Base):
    __tablename__ = "user_im_bindings"
    id = Column(String, primary_key=True, default=_new_uuid)
    user_id = Column(String, ForeignKey("users.id"), nullable=False)
    im_platform = Column(String(50), nullable=False)
    im_user_id = Column(String(200), nullable=False)
    im_display_name = Column(String(200))
    status = Column(String(50), default="active")
    created_at = Column(String, default=_utc_now)
    updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
    __table_args__ = (UniqueConstraint("im_platform", "im_user_id",
name="uq_im_platform_user"),)
    user = relationship("User", back_populates="im_bindings")
class Conversation(Base):
    __tablename__ = "conversations"
    id = Column(String, primary_key=True, default=_new_uuid)
    im_platform = Column(String(50), nullable=False)
    conversation_type = Column(String(50), nullable=False)
    conversation_id = Column(String(200), nullable=False)
    group_id = Column(String(200))
    title = Column(String(500))
    project_id = Column(String)
    takeover_mode = Column(String(50), default="collaborate")
    created_at = Column(String, default=_utc_now)
    updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
    __table_args__ = (UniqueConstraint("im_platform", "conversation_id",
name="uq_im_platform_conv"),)
    messages = relationship("Message", back_populates="conversation",
lazy="selectin")
class Message(Base):
    __tablename__ = "messages"
    id = Column(String, primary_key=True, default=_new_uuid)
    event_id = Column(String(100), unique=True, nullable=False)
    message_uid = Column(String(200))
    conversation_id = Column(String, ForeignKey("conversations.id"))
    project_id = Column(String, nullable=True)
    sender_user_id = Column(String, ForeignKey("users.id"), nullable=True)
    sender_im_id = Column(String(200), nullable=False)
    sender_name = Column(String(200))

```



```

message_type = Column(String(50), default="text")
direction = Column(String(50), default="user_to_agent")
content = Column(String)
attachments = Column(String, default="[]")
is_at_bot = Column(Boolean, default=False)
takeover = Column(Boolean, default=False)
takeover_reason = Column(String(200))
intent = Column(String(100))
status = Column(String(50), default="received")
created_at = Column(String, default=_utc_now)
processed_at = Column(String)
conversation = relationship("Conversation", back_populates="messages")
msg_type = Column(Integer, default=1)
file_url = Column(String, default="")
receiver_id = Column(String(200), nullable=True)
group_id = Column(String(200), nullable=True)
attachments_rel = relationship("MessageAttachment", back_populates="message",
lazy="selectin")
class Project(Base):
    __tablename__ = "projects"
    id = Column(String, primary_key=True, default=_new_uuid)
    code = Column(String(50), unique=True)
    name = Column(String(200), nullable=False)
    description = Column(String)
    status = Column(String(50), default="active")
    created_at = Column(String, default=_utc_now)
    updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
    events = relationship("Event", back_populates="project", lazy="selectin")
    address = Column(String(500), default="")
    city = Column(String(100), default="")
    lifecycle_stage = Column(Integer, default=0)
    stage_updated_at = Column(String, nullable=True)
    creator_id = Column(String, nullable=True)
    is_deleted = Column(Boolean, default=False)
class Event(Base):
    __tablename__ = "events"
    id = Column(String, primary_key=True, default=_new_uuid)
    event_no = Column(String(50), unique=True, nullable=False)
    event_type = Column(String(100), nullable=False)
    category = Column(String(100), default="待分类")
    project_id = Column(String, ForeignKey("projects.id"), nullable=True)
    user_id = Column(String, ForeignKey("users.id"), nullable=True)
    message_id = Column(String, ForeignKey("messages.id"), nullable=True)
    title = Column(String(500), nullable=False)
    description = Column(String)
    event_date = Column(String)
    attachments = Column(String, default="[]")
    remarks = Column(String)
    payload = Column(String, default="{}")
    status = Column(String(50), default="pending")
    created_at = Column(String, default=_utc_now)
    confirmed_at = Column(String)
    related_event_ids = Column(String, default="[]")
    conversation_id = Column(String(200), nullable=True, index=True)

```

```

project = relationship("Project", back_populates="events")
class SessionArchive(Base):
    __tablename__ = "session_archives"
    id = Column(String, primary_key=True, default=_new_uuid)
    conversation_id = Column(String(200), nullable=False, index=True)
    user_id = Column(String, ForeignKey("users.id"), nullable=True)
    user_name = Column(String(200), default="")
    turn_count = Column(Integer, default=0)
    message_history_snapshot = Column(Text, default="")
    context_snapshot = Column(Text, default="")
    started_at = Column(String, nullable=True)
    archived_at = Column(String, default=_utc_now)
    archive_reason = Column(String(50), default="expired")
class Task(Base):
    __tablename__ = "tasks"
    id = Column(String, primary_key=True, default=_new_uuid)
    task_no = Column(String(50), unique=True, nullable=False)
    project_id = Column(String, ForeignKey("projects.id"), nullable=True)
    source_message_id = Column(String, ForeignKey("messages.id"), nullable=True)
    title = Column(String(500), nullable=False)
    description = Column(String)
    owner_id = Column(String, ForeignKey("users.id"), nullable=True)
    owner_text = Column(String(200))
    status = Column(String(50), default="todo")
    due_date = Column(String)
    due_text = Column(String(200))
    created_by = Column(String, ForeignKey("users.id"), nullable=True)
    created_at = Column(String, default=_utc_now)
    updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
class Meeting(Base):
    __tablename__ = "meetings"
    id = Column(String, primary_key=True, default=_new_uuid)
    meeting_no = Column(String(50), unique=True, nullable=False)
    project_id = Column(String, ForeignKey("projects.id"), nullable=True)
    title = Column(String(500))
    summary = Column(String)
    attendees = Column(String, default="[]")
    source_message_id = Column(String, ForeignKey("messages.id"), nullable=True)
    created_by = Column(String, ForeignKey("users.id"), nullable=True)
    created_at = Column(String, default=_utc_now)
    meeting_type = Column(Integer, default=0)
    meeting_date = Column(String, nullable=True)
    location = Column(String(500), default="")
    host_id = Column(String, nullable=True)
    attendee_names = Column(String, default="[]")
    conclusion = Column(String, default="")
    action_items = Column(String, default="[]")
    related_file_ids = Column(String, default="[]")
    status = Column(Integer, default=1)
    updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
    is_deleted = Column(Boolean, default=False)
class FileCategory:
    PROJECT_LICENSE = "PROJECT_LICENSE"
    CONTRACT = "CONTRACT"

```

```

WORK_RECORD = "WORK_RECORD"
PHASE_DELIVERABLE = "PHASE_DELIVERABLE"
PROCESS_DOC = "PROCESS_DOC"
MANAGEMENT_SPEC = "MANAGEMENT_SPEC"
OTHER = "OTHER"
ALL = [
    PROJECT_LICENSE, CONTRACT, WORK_RECORD,
    PHASE_DELIVERABLE, PROCESS_DOC, MANAGEMENT_SPEC, OTHER,
]
DISPLAY_NAMES = {
    PROJECT_LICENSE: "项目证照",
    CONTRACT: "承包合同",
    WORK_RECORD: "工作记录",
    PHASE_DELIVERABLE: "阶段成果",
    PROCESS_DOC: "过程文件",
    MANAGEMENT_SPEC: "管理规程",
    OTHER: "其他文件",
}
@classmethod
def validate(cls, value: str) -> str:
    if value in cls.ALL:
        return value
    return cls.OTHER
@classmethod
def display(cls, value: str) -> str:
    return cls.DISPLAY_NAMES.get(value, "其他文件")
class File(Base):
    __tablename__ = "files"
    id = Column(String, primary_key=True, default=_new_uuid)
    file_no = Column(String(50), unique=True, nullable=False)
    project_id = Column(String, ForeignKey("projects.id"), nullable=True)
    message_id = Column(String, ForeignKey("messages.id"), nullable=True)
    filename = Column(String(500), nullable=False)
    file_type = Column(String(100))
    bucket = Column(String(100))
    object_key = Column(String)
    storage_path = Column(String)
    file_size = Column(Integer)
    uploaded_by = Column(String, ForeignKey("users.id"), nullable=True)
    parse_status = Column(String(50), default="pending")
    created_at = Column(String, default=_utc_now)
    file_ext = Column(String(50), default="")
    file_url = Column(String(1000), default="")
    file_hash = Column(String(256), default="")
    related_company_ids = Column(String, default="[]")
    responsible_id = Column(String, nullable=True)
    version = Column(String(50), default="V1.0")
    is_latest = Column(Boolean, default=True)
    parent_file_id = Column(String, nullable=True)
    change_log = Column(String, default="[]")
    confidentiality = Column(Integer, default=0)
    creator_id = Column(String, nullable=True)
    updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
    is_deleted = Column(Boolean, default=False)

```

```

    source_attachment_id = Column(String, ForeignKey("message_attachments.id"),
    nullable=True)
    attachment_refs = relationship("MessageAttachment", back_populates="file",
    foreign_keys="MessageAttachment.file_id")
    source_module_id = Column(String(100), default="", comment="来源模块ID（节点ID/
    其他业务对象ID）")
    source_module_type = Column(String(50), default="", comment="来源模块类型：
    NODE_STARTUP_DOC/NODE_WORKLOAD_DOC/NODE_DELIVERABLE_DOC/NODE_ATTACHMENT")
    file_category = Column(String(50), default="OTHER", comment="文件业务分类：
    PROJECT_LICENSE/CONTRACT/WORK_RECORD/PHASE_DELIVERABLE/PROCESS_DOC/MANAGEMENT_SPEC
    /OTHER")
class CompanyInfo(Base):
    __tablename__ = "company_info"
    id = Column(String, primary_key=True, default=_new_uuid)
    company_name = Column(String(256), nullable=False)
    unified_code = Column(String(50), nullable=False)
    business_desc = Column(String, default="")
    project_leader_id = Column(String, nullable=False)
    creator_id = Column(String, nullable=False)
    type = Column(String(50), default="")
    status = Column(String(50), default="active")
    scope = Column(String, default="[]")
    partners = Column(String, default="[]")
    parent_id = Column(String, ForeignKey("company_info.id"), nullable=True)
    department = Column(String, default="[]")
    function_scope = Column(Text, default="{}")
    is_admin = Column(Boolean, default=False, comment="是否管理单位（建设单位/代建单
    位）")
    created_at = Column(String, default=_utc_now)
    updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
    is_deleted = Column(Boolean, default=False)
class ProjectIndicatorDetail(Base):
    __tablename__ = "project_indicator_details"
    id = Column(String, primary_key=True, default=_new_uuid)
    project_id = Column(String, nullable=False)
    indicator_name = Column(String(128), nullable=False)
    indicator_value = Column(String(256), nullable=False)
    unit = Column(String(50), default="")
    source_file_id = Column(String, nullable=True)
    description = Column(String, default="")
    is_constraint = Column(Boolean, default=False)
    creator_id = Column(String, nullable=False)
    created_at = Column(String, default=_utc_now)
    updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
    is_deleted = Column(Boolean, default=False)
class BusinessFlowOrder(Base):
    __tablename__ = "business_flow_orders"
    id = Column(String, primary_key=True, default=_new_uuid)
    flow_no = Column(String(100), unique=True, nullable=False)
    project_id = Column(String, nullable=True)
    title = Column(String(500), nullable=False)
    flow_type = Column(Integer, default=0)
    metrics = Column(String, default="{}")
    planned_finish_time = Column(String, nullable=True)

```

```
actual_finish_time = Column(String, nullable=True)
current_node = Column(String(100), default="")
current_handler_id = Column(String, nullable=True)
flow_records = Column(String, default="[]")
related_file_ids = Column(String, default="[]")
related_meeting_ids = Column(String, default="[]")
status = Column(Integer, default=0)
priority = Column(Integer, default=1)
creator_id = Column(String, nullable=False)
created_at = Column(String, default=_utc_now)
updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
is_deleted = Column(Boolean, default=False)

class InstructionOrder(Base):
    __tablename__ = "instruction_orders"
    id = Column(String, primary_key=True, default=_new_uuid)
    instruction_no = Column(String(100), unique=True, nullable=False)
    project_id = Column(String, nullable=True)
    title = Column(String(500), nullable=False)
    content = Column(String, nullable=False)
    instruction_type = Column(Integer, default=0)
    issuer_id = Column(String, nullable=False)
    executor_ids = Column(String, default="[]")
    deadline = Column(String, nullable=True)
    actual_finish_time = Column(String, nullable=True)
    feedback = Column(String, default="")
    related_file_ids = Column(String, default="[]")
    related_flow_id = Column(String, nullable=True)
    message_id = Column(String, nullable=True)
    status = Column(Integer, default=0)
    creator_id = Column(String, nullable=False)
    created_at = Column(String, default=_utc_now)
    updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
    is_deleted = Column(Boolean, default=False)

class ProjectPlan(Base):
    __tablename__ = "project_plans"
    id = Column(String, primary_key=True, default=_new_uuid)
    plan_no = Column(String(100), unique=True, nullable=False)
    project_id = Column(String, nullable=True)
    title = Column(String(500), nullable=False)
    plan_type = Column(Integer, default=0)
    start_date = Column(String, nullable=True)
    end_date = Column(String, nullable=True)
    creator_id = Column(String, nullable=False)
    status = Column(Integer, default=0)
    parent_plan_id = Column(String, nullable=True)
    remark = Column(String, default="")
    created_at = Column(String, default=_utc_now)
    updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
    is_deleted = Column(Boolean, default=False)

class PlanItem(Base):
    __tablename__ = "plan_items"
    id = Column(String, primary_key=True, default=_new_uuid)
    plan_id = Column(String, nullable=False)
    item_no = Column(Integer, nullable=False)
```

```

content = Column(String, nullable=False)
responsible_id = Column(String, nullable=True)
reviewer_id = Column(String, nullable=True)
planned_date = Column(String, nullable=True)
actual_date = Column(String, nullable=True)
is_completed = Column(Boolean, default=False)
is_covered = Column(Boolean, default=False)
related_instruction_ids = Column(String, default="[]")
related_flow_ids = Column(String, default="[]")
progress = Column(Integer, default=0)
remark = Column(String, default="")
creator_id = Column(String, nullable=False)
created_at = Column(String, default=_utc_now)
updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
is_deleted = Column(Boolean, default=False)

class SOPRoutingLog(Base):
    __tablename__ = "sop_routing_logs"
    id = Column(String, primary_key=True, default=_new_uuid)
    log_date = Column(String(10), nullable=False)
    log_time = Column(String(8), nullable=False)
    user_id = Column(String, nullable=True)
    conversation_id = Column(String, nullable=True)
    message_id = Column(String, nullable=True)
    message_content = Column(String, nullable=False)
    matched_sop_id = Column(String, nullable=True)
    is_hit = Column(Boolean, default=False)
    match_confidence = Column(String(10), default="none")
    fallback_action = Column(String(30), default="")
    llm_reasoning = Column(String, default="")
    execution_result = Column(String(20), default="")
    created_at = Column(String, default=_utc_now)

class MessageAttachment(Base):
    __tablename__ = "message_attachments"
    id = Column(String, primary_key=True, default=_new_uuid)
    message_id = Column(String, ForeignKey("messages.id"), nullable=False)
    file_id = Column(String, ForeignKey("files.id"), nullable=True)
    attachment_type = Column(Integer, default=0)
    file_url = Column(String(1000), default="")
    local_path = Column(String(1000), default="")
    file_size = Column(Integer, default=0)
    mime_type = Column(String(100), default="")
    thumbnail_url = Column(String(1000), default="")
    created_at = Column(String, default=_utc_now)
    message = relationship("Message", back_populates="attachments_rel")
    file = relationship("File", back_populates="attachment_refs",
                        foreign_keys=[file_id])

class AgentReasoningLog(Base):
    __tablename__ = "agent_reasoning_logs"
    id = Column(String, primary_key=True, default=_new_uuid)
    message_id = Column(String, ForeignKey("messages.id"), nullable=False)
    user_id = Column(String, ForeignKey("users.id"), nullable=True)
    conversation_id = Column(String, ForeignKey("conversations.id"),
    nullable=True)
    iteration_count = Column(Integer, default=0)

```



```

elapsed_ms = Column(Integer, default=0)
max_iterations_reached = Column(Boolean, default=False)
matched_sop_id = Column(String(50), nullable=True)
match_confidence = Column(String(10), default="none")
is_compound = Column(Boolean, default=False)
fallback = Column(Boolean, default=False)
execution_result = Column(String(20), default="")
reply_preview = Column(String(500), default="")
error_message = Column(String(500), default="")
steps_json = Column(Text, default="[]")
created_at = Column(String, default=_utc_now)

class LLMInteractionLog(Base):
    __tablename__ = "llm_interaction_logs"
    id = Column(String, primary_key=True, default=_new_uuid)
    reasoning_log_id = Column(String, ForeignKey("agent_reasoning_logs.id"),
    nullable=True)
    message_id = Column(String, ForeignKey("messages.id"), nullable=True)
    call_sequence = Column(Integer, default=0)
    call_type = Column(String(30), default="chat_with_tools")
    model = Column(String(100), default="")
    prompt_summary = Column(String(500), default="")
    user_message_count = Column(Integer, default=0)
    tool_count = Column(Integer, default=0)
    response_type = Column(String(20), default="")
    response_summary = Column(String(500), default="")
    finish_reason = Column(String(50), default="")
    prompt_tokens = Column(Integer, default=0)
    completion_tokens = Column(Integer, default=0)
    total_tokens = Column(Integer, default=0)
    latency_ms = Column(Integer, default=0)
    created_at = Column(String, default=_utc_now)

class ToolCallLog(Base):
    __tablename__ = "tool_call_logs"
    id = Column(String, primary_key=True, default=_new_uuid)
    reasoning_log_id = Column(String, ForeignKey("agent_reasoning_logs.id"),
    nullable=True)
    llm_interaction_id = Column(String, ForeignKey("llm_interaction_logs.id"),
    nullable=True)
    step_index = Column(Integer, default=0)
    tool_name = Column(String(100), nullable=False)
    tool_arguments = Column(Text, default="{}")
    tool_result_summary = Column(String(500), default="")
    is_success = Column(Boolean, default=True)
    error_message = Column(String(500), default="")
    elapsed_ms = Column(Integer, default=0)
    created_at = Column(String, default=_utc_now)

class HookExecutionLog(Base):
    __tablename__ = "hook_execution_logs"
    id = Column(String, primary_key=True, default=_new_uuid)
    hook_name = Column(String, nullable=False, index=True)
    mount_point = Column(String, nullable=False)
    pipeline_run_id = Column(String, nullable=False, index=True)
    phase = Column(String, nullable=False)
    decision = Column(String, nullable=False)

```

```

message = Column(String, default="")
duration_ms = Column(Integer, default=0)
metadata_json = Column(Text, default="{}")
user_id = Column(String, default="")
sop_id = Column(String, default="")
block_reason = Column(String(500), default="")
session_level = Column(Integer, nullable=True)
created_at = Column(String, nullable=False)

class PermissionGroup(Base):
    __tablename__ = "permission_groups"
    id = Column(String, primary_key=True, default=_new_uuid)
    name = Column(String(100), nullable=False)
    code = Column(String(50), unique=True, nullable=False)
    description = Column(String(500), default="")
    company_type = Column(String(50), nullable=False)
    department = Column(String(100), default="")
    org_level = Column(Integer, default=1)
    parent_group_id = Column(String, nullable=True)
    allowed_sop_types = Column(String, default="[]")
    min_level = Column(Integer, default=1)
    status = Column(String(50), default="active")
    is_system = Column(Boolean, default=False)
    creator_id = Column(String, nullable=True)
    created_at = Column(String, default=_utc_now)
    updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
    is_deleted = Column(Boolean, default=False)

class SOPBusinessFlow(Base):
    __tablename__ = "sop_business_flows"
    id = Column(String, primary_key=True, default=_new_uuid)
    sop_id = Column(String(50), unique=True, nullable=False)
    sop_file_name = Column(String(200), nullable=False)
    display_name = Column(String(200), nullable=False)
    description = Column(String(500), default="")
    sop_type = Column(String(50), nullable=False)
    category = Column(String(100), default="")
    default_permission_group_id = Column(String,
    ForeignKey("permission_groups.id"), nullable=True)
    min_level = Column(Integer, default=1)
    require_company_match = Column(Boolean, default=True)
    require_department_match = Column(Boolean, default=False)
    is_public = Column(Boolean, default=False)
    allowed_company_types = Column(String, default="[]")
    allowed_departments = Column(String, default="[]")
    security_level = Column(String(20), default="PUBLIC")
    required_node_ids = Column(String, default="[]")
    version = Column(String(50), default="v1.0")
    is_active = Column(Boolean, default=True)
    is_deprecated = Column(Boolean, default=False)
    deprecate_reason = Column(String(500), default="")
    creator_id = Column(String, nullable=True)
    created_at = Column(String, default=_utc_now)
    updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
    is_deleted = Column(Boolean, default=False)
    permission_group = relationship("PermissionGroup", foreign_keys=

```



```

[default_permission_group_id])
class SOPPermissionBinding(Base):
    __tablename__ = "sop_permission_bindings"
    id = Column(String, primary_key=True, default=_new_uuid)
    sop_business_flow_id = Column(String, ForeignKey("sop_business_flows.id"),
    nullable=False)
    permission_group_id = Column(String, ForeignKey("permission_groups.id"),
    nullable=False)
    binding_type = Column(String(50), default="allow")
    creator_id = Column(String, nullable=True)
    created_at = Column(String, default=_utc_now)
    is_deleted = Column(Boolean, default=False)
    __table_args__ = (
        UniqueConstraint("sop_business_flow_id", "permission_group_id",
name="uq_sop_permission_binding"),
    )
class PermissionDef(Base):
    __tablename__ = "permission_def"
    id = Column(String, primary_key=True, default=_new_uuid)
    perm_code = Column(String(256), unique=True, nullable=False)
    resource_type = Column(String(3), nullable=False)
    security_level = Column(String(12), nullable=False)
    project_id = Column(String, default="*")
    node_id = Column(String, default="*")
    resource_id = Column(String, default="*")
    description = Column(String(500), default="")
    created_at = Column(String, default=_utc_now)
    updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
    is_deleted = Column(Boolean, default=False)
class PermissionGrant(Base):
    __tablename__ = "permission_grants"
    id = Column(String, primary_key=True, default=_new_uuid)
    grant_no = Column(String(50), unique=True, nullable=False)
    grantee_id = Column(String, ForeignKey("users.id"), nullable=False)
    grantor_id = Column(String, ForeignKey("users.id"), nullable=True)
    perm_code = Column(String(256), nullable=False)
    grant_type = Column(String(12), nullable=False)
    operations = Column(String, default='["read"]')
    grant_time = Column(String, default=_utc_now)
    expire_time = Column(String, nullable=True)
    status = Column(String(20), default="ACTIVE")
    revoke_time = Column(String, nullable=True)
    revoke_reason = Column(String(500), default="")
    remark = Column(String(500), default="")
    client_ip = Column(String(64), default="")
    created_at = Column(String, default=_utc_now)
    updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
    is_deleted = Column(Boolean, default=False)
    __table_args__ = (
        Index("idx_pg_grantee_status", "grantee_id", "status"),
    )
class PermissionRequest(Base):
    __tablename__ = "permission_requests"
    id = Column(String, primary_key=True, default=_new_uuid)

```

```

request_no = Column(String(50), unique=True, nullable=False)
requester_id = Column(String, ForeignKey("users.id"), nullable=False)
perm_code = Column(String(256), nullable=False)
request_type = Column(String(20), nullable=False)
reason = Column(String(1000), default="")
status = Column(String(20), default="PENDING")
current_approver_id = Column(String, ForeignKey("users.id"), nullable=True)
approval_level = Column(Integer, default=1)
priority = Column(String(20), default="NORMAL")
expire_at = Column(String, nullable=True)
approved_at = Column(String, nullable=True)
approver_id = Column(String, ForeignKey("users.id"), nullable=True)
approval_remark = Column(String(500), default="")
source_data = Column(Text, default="{}")
agent_issue_id = Column(String, nullable=True)
created_at = Column(String, default=_utc_now)
updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
is_deleted = Column(Boolean, default=False)
__table_args__ = (
    Index("idx_prq_approver_status", "current_approver_id", "status"),
)
class PermissionAuditLog(Base):
    __tablename__ = "permission_audit_log"
    log_id = Column(BigInteger, primary_key=True, autoincrement=True)
    event_time = Column(String, default=_utc_now)
    grantor_id = Column(String, nullable=True)
    grantee_id = Column(String, nullable=True)
    perm_code = Column(String(256), default="")
    grant_type = Column(String(32), default="")
    duration = Column(Integer, nullable=True)
    session_id = Column(String(128), nullable=True)
    operation_type = Column(String(32), nullable=False)
    client_ip = Column(String(64), default="")
    user_agent = Column(Text, default="")
    remark = Column(String(512), default="")
    __table_args__ = (
        Index("idx_pal_grantee_time", "grantee_id", "event_time"),
        Index("idx_pal_op_time", "operation_type", "event_time"),
    )
class PublicFieldRegistry(Base):
    __tablename__ = "public_field_registry"
    id = Column(String, primary_key=True, default=_new_uuid)
    project_id = Column(String, nullable=True)
    model_name = Column(String(100), nullable=False)
    field_name = Column(String(100), nullable=False)
    description = Column(String(500), default="")
    created_at = Column(String, default=_utc_now)
    is_deleted = Column(Boolean, default=False)
class PendingData(Base):
    __tablename__ = "pending_data"
    id = Column(String, primary_key=True, default=_new_uuid)
    pending_no = Column(String(50), unique=True, nullable=False)
    user_id = Column(String, ForeignKey("users.id"), nullable=False)
    data_type = Column(String(50), nullable=False)

```

```

data_content = Column(Text, default="{}")
exception_reason = Column(String(1000), default="")
target_node_id = Column(String, default="")
approver_id = Column(String, ForeignKey("users.id"), nullable=True)
status = Column(String(20), default="PENDING")
expire_time = Column(String, nullable=True)
request_id = Column(String, nullable=True)
created_at = Column(String, default=_utc_now)
updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
is_deleted = Column(Boolean, default=False)
__table_args__ = (
    Index("idx_pnd_approver_status", "approver_id", "status"),
)

class DataMaskingRule(Base):
    __tablename__ = "data_masking_rules"
    id = Column(String, primary_key=True, default=_new_uuid)
    rule_code = Column(String(50), unique=True, nullable=False)
    field_pattern = Column(String(200), nullable=False)
    mask_type = Column(String(50), nullable=False)
    min_level_to_view = Column(Integer, default=5)
    params = Column(Text, default="{}")
    created_at = Column(String, default=_utc_now)
    updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
    is_deleted = Column(Boolean, default=False)

class PermissionReviewTask(Base):
    __tablename__ = "permission_review_tasks"
    id = Column(String, primary_key=True, default=_new_uuid)
    review_no = Column(String(50), unique=True, nullable=False)
    review_period = Column(String(20), nullable=False)
    scope_type = Column(String(20), default="ALL")
    scope_value = Column(String, default="")
    assignee_id = Column(String, ForeignKey("users.id"), nullable=True)
    status = Column(String(20), default="PENDING")
    deadline = Column(String, nullable=True)
    result_summary = Column(Text, default="{}")
    created_at = Column(String, default=_utc_now)
    updated_at = Column(String, default=_utc_now, onupdate=_utc_now)
    is_deleted = Column(Boolean, default=False)

class ProjectNode(Base):
    __tablename__ = "project_nodes"
    project_id = Column(String(100), nullable=False, comment="项目归属ID (FK→projects.id)")
    node_id = Column(String(100), nullable=False, comment="节点编号（业务主键），例：SG-JG-01-2026")
    node_name = Column(String(500), nullable=False, comment="节点名称")
    owner_dept_id = Column(String(100), nullable=False, default="项目总", comment="主责条线（FK→company_info.id）")
    related_company_id = Column(String(100), nullable=False, default="建设单位", comment="关联单位（FK→company_info.id）")
    deadline = Column(String(50), nullable=False, comment="截止时间（ISO8601）")
    land_parcel_id = Column(String(100), default="", comment="关联地块ID")
    remark = Column(Text, default="", comment="备注")
    parent_node_id = Column(String(100), default="", comment="父节点ID (FK→project_nodes.node_id)")

```

```

    stage_id = Column(Integer, default=0, comment="所属阶段ID（对齐
projects.lifecycle_stage）")
    child_weight = Column(String, default="1.0000", comment="作为子节点时在父节点中的
权重（DECIMAL(5,4），存为字符串避免精度问题）")
    startup_doc_id = Column(String(100), default="", comment="启动文档记录ID
（FK→files.id）")
    creator_id = Column(String(100), nullable=False, comment="录入人ID")
    created_at = Column(String(50), nullable=False, default=_utc_now, comment="录
入时间（ISO8601）")
    approver_id = Column(String(100), default="", comment="批准人ID")
    approved_at = Column(String(50), default="", comment="批准时间（ISO8601）")
    completed_at = Column(String(50), default="", comment="完成时间（ISO8601）")
    is_discarded = Column(Boolean, default=False, comment="是否被废弃")
    progress = Column(String, default="0.00", comment="整体进度（百分比 0.00-
100.00，存为字符串避免精度问题）")
    status = Column(String(20), default="NOT_ACTIVATED", comment="当前状态：
NOT_ACTIVATED / CONDITIONS_NOT_MET / IN_PROGRESS / COMPLETED")
    sort_order = Column(Integer, default=0, comment="排序序号")
    responsible_user_id = Column(String(100), nullable=False, default="",
comment="责任人（FK→users.id，创建时默认取 creator_id）")
    node_type = Column(String(20), nullable=False, default="WORK_PACKAGE",
comment="节点类型：MILESTONE / WORK_PACKAGE / TASK")
    visibility_mode = Column(
        String(30), nullable=False, default="specific",
        comment="文件可见模式：specific（按 node_accessible_files 绑定）/
all_project_files（全项目文件默认可见）"
    )
    updated_at = Column(String(50), nullable=False, default=_utc_now,
onupdate=_utc_now, comment="最后更新时间（ISO8601）")
    id = Column(String, primary_key=True, default=_new_uuid)
    __table_args__ = (
        Index("idx_nodes_project", "project_id"),
        Index("idx_nodes_status", "status"),
        Index("idx_nodes_stage", "stage_id"),
        Index("idx_nodes_owner", "owner_dept_id"),
        Index("idx_nodes_parent", "parent_node_id"),
    )
class NodeDependency(Base):
    __tablename__ = "node_dependencies"
    node_id = Column(String(100), nullable=False, comment="本节点（下游节点，
FK→project_nodes.node_id）")
    depends_on_deliverable_id = Column(String(100), nullable=False, comment="依赖
的成果ID（FK→node_deliverables.deliverable_id）")
    depends_on_node_id = Column(String(100), nullable=False, comment="成果所属上游
节点ID（冗余字段，FK→project_nodes.node_id）")
    dependency_type = Column(String(20), nullable=False, default="DELIVERABLE",
comment="依赖类型：DELIVERABLE / TIME")
    weight = Column(String, nullable=False, default="1.0000", comment="权重
（0.0000-1.0000，存为字符串）")
    created_at = Column(String(50), nullable=False, default=_utc_now, comment="创
建时间（ISO8601）")
    id = Column(String, primary_key=True, default=_new_uuid)
    __table_args__ = (
        UniqueConstraint("node_id", "depends_on_deliverable_id",

```

```

name="uq_dep_node_deliverable"),
    Index("idx_ndep_node", "node_id"),
    Index("idx_ndep_deliverable", "depends_on_deliverable_id"),
)
class NodeDeliverable(Base):
    __tablename__ = "node_deliverables"
    deliverable_id = Column(String(100), nullable=False, comment="成果编号（业务主键）")
    node_id = Column(String(100), nullable=False, comment="所属节点ID (FK→project_nodes.node_id)")
    deliverable_name = Column(String(500), nullable=False, comment="成果名称")
    target_amount = Column(String, nullable=False, comment="目标量（DECIMAL(12,2), 存为字符串）")
    current_amount = Column(String, nullable=False, default="0.00", comment="当前量（DECIMAL(12,2), 存为字符串）")
    unit = Column(String(50), nullable=False, comment="量纲（份/吨/平方米...）")
    is_required = Column(Boolean, nullable=False, default=True, comment="是否必需成果")
    file_id = Column(String(100), default="", comment="关联文件ID（FK→files.id）")
    completed_at = Column(String(50), default="", comment="完成时间（ISO8601）")
    created_at = Column(String(50), nullable=False, default=_utc_now, comment="创建时间（ISO8601）")
    submission_status = Column(String(20), nullable=False, default="PENDING", comment="提交状态: PENDING / SUBMITTED / CONFIRMED / RETURNED")
    submitted_by = Column(String(100), default="", comment="提交人（FK→users.id）")
    submitted_at = Column(String(50), default="", comment="提交时间（ISO8601）")
    confirmed_by = Column(String(100), default="", comment="确认人（FK→users.id）")
    confirmed_at = Column(String(50), default="", comment="确认时间（ISO8601）")
    return_reason = Column(String(500), default="", comment="退回原因")
    attachment_file_id = Column(String(100), default="", comment="提交附件 (FK→files.id)")
    id = Column(String, primary_key=True, default=_new_uuid)
    __table_args__ = (
        Index("idx_ndel_node", "node_id"),
    )
class NodeAccessibleFile(Base):
    __tablename__ = "node_accessible_files"
    node_id = Column(String(100), nullable=False, comment="节点ID (FK→project_nodes.node_id)")
    file_id = Column(String(100), nullable=False, comment="文件ID（FK→files.id）")
    added_by = Column(String(100), nullable=False, comment="授权人ID")
    added_at = Column(String(50), nullable=False, default=_utc_now, comment="授权时间（ISO8601）")
    id = Column(String, primary_key=True, default=_new_uuid)
    __table_args__ = (
        UniqueConstraint("node_id", "file_id", name="uq_naf_node_file"),
        Index("idx_naf_node", "node_id"),
        Index("idx_naf_file", "file_id"),
    )
class NodeEvent(Base):
    __tablename__ = "node_events"
    event_id = Column(String(100), nullable=False, comment="事件唯一ID")
    node_id = Column(String(100), nullable=False, comment="关关节点ID (FK→project_nodes.node_id)")

```

```

event_type = Column(String(50), nullable=False, comment="事件类型枚举")
old_value = Column(Text, default="", comment="变更前值 (JSON快照)")
new_value = Column(Text, default="", comment="变更后值 (JSON快照)")
operator_id = Column(String(100), default="", comment="操作人ID (系统自动触发则为空)")
remark = Column(Text, default="", comment="操作说明/备注")
created_at = Column(String(50), nullable=False, default=_utc_now, comment="事件发生时间 (ISO8601)")
id = Column(String, primary_key=True, default=_new_uuid)
__table_args__ = (
    Index("idx_nev_node", "node_id"),
    Index("idx_nev_type", "event_type"),
    Index("idx_nev_created", "created_at"),
)
class ToolRegistryModel(Base):
    __tablename__ = "tool_registry"
    id = Column(String, primary_key=True)
    signature = Column(Text, nullable=False, default="{}")
    display_name = Column(String(200), nullable=False)
    category = Column(String(20), nullable=False, default="base")
    permission_flag = Column(String(50), nullable=False, default="all")
    handler_module = Column(String(200), default="")
    is_active = Column(Boolean, nullable=False, default=True)
    registered_at = Column(String(50), nullable=False)
    updated_at = Column(String(50), nullable=False)
class SessionAccessibleFile(Base):
    __tablename__ = "session_accessible_files"
    id = Column(String, primary_key=True, default=_new_uuid)
    user_id = Column(String, nullable=False, index=True)
    file_id = Column(String, nullable=False, index=True)
    access_type = Column(String(50), nullable=False, default="project_scope")
    granted_by = Column(String, default="")
    granted_at = Column(String(50), nullable=False)
    expires_at = Column(String(50), default="")
    __table_args__ = (
        UniqueConstraint("user_id", "file_id", name="uq_saf_user_file"),
    )
class SchedulerJob(Base):
    __tablename__ = "scheduler_jobs"
    id = Column(String, primary_key=True, default=_new_uuid)
    job_no = Column(String(50), unique=True, nullable=False, comment="作业编号 JOB-YYYYMMDD-NNNN")
    name = Column(String(200), nullable=False, comment="作业名称")
    description = Column(String, default="", comment="作业描述")
    job_type = Column(String(20), nullable=False, default="ONCE", comment="调度类型: ONCE / CRON / INTERVAL")
    cron_expression = Column(String(100), default="", comment="cron 表达式 (CRON 模式)")
    interval_seconds = Column(Integer, default=0, comment="间隔秒数 (INTERVAL 模式)")
    deadline_rule = Column(String(500), default="", comment="自然语言描述 (LLM 推算, CRON 补充)")
    action_type = Column(String(50), nullable=False, comment="动作类型 (对应 JobHandler.action_type)")

```



```

    handler_module = Column(String(200), nullable=False, comment="Handler 模块路径
    (如 scheduler.jobs.morning_report)")
    action_params = Column(Text, default="{}", comment="JSON 参数")
    status = Column(String(20), nullable=False, default="DRAFT", comment="DRAFT /
    ACTIVE / INACTIVE")
    last_executed_at = Column(String(50), default="", comment="上次执行时间")
    next_execution_at = Column(String(50), default="", comment="下次执行时间")
    creator_id = Column(String, nullable=True, comment="创建人ID")
    created_at = Column(String, nullable=False, default=_utc_now, comment="创建时
    间")
    updated_at = Column(String, nullable=False, default=_utc_now,
    onupdate=_utc_now, comment="更新时间")
    __table_args__ = (
        Index("idx_sj_status", "status"),
        Index("idx_sj_next_execution", "next_execution_at"),
        Index("idx_sj_action_type", "action_type"),
    )
class SchedulerExecution(Base):
    __tablename__ = "scheduler_executions"
    id = Column(String, primary_key=True, default=_new_uuid)
    job_id = Column(String, ForeignKey("scheduler_jobs.id"), nullable=False,
    comment="关联作业ID")
    execution_no = Column(String(50), unique=True, nullable=False, comment="执行编
    号 SE-YYYYMMDD-NNNN")
    period_key = Column(String(100), default="", comment="周期标识 (如 2024-W25),
    用于幂等和追溯")
    status = Column(String(20), nullable=False, default="PENDING",
    comment="PENDING / RUNNING / SUCCESS / FAILED")
    started_at = Column(String(50), default="", comment="开始时间")
    finished_at = Column(String(50), default="", comment="结束时间")
    error_message = Column(Text, default="", comment="错误信息")
    result_summary = Column(Text, default="", comment="执行结果摘要")
    created_at = Column(String, nullable=False, default=_utc_now, comment="创建时
    间")
    __table_args__ = (
        Index("idx_se_job_status", "job_id", "status"),
        Index("idx_se_created_at", "created_at"),
        Index("idx_se_period", "job_id", "period_key"),
    )
class PipelineExecutionLog(Base):
    __tablename__ = "pipeline_execution_logs"
    id = Column(String, primary_key=True, default=_new_uuid)
    pipeline_run_id = Column(String, index=True)
    conversation_id = Column(String, index=True)
    user_id = Column(String, index=True)
    user_name = Column(String(200), default="")
    user_level = Column(Integer, default=1)
    matched_sop_id = Column(String, default="")
    match_confidence = Column(String(20), default="")
    is_compound = Column(Boolean, default=False)
    is_fallback = Column(Boolean, default=False)
    intent_reasoning = Column(String, default="")
    final_status = Column(String(20), default="")
    abort_reason = Column(String(500), default="")

```

```

result_text = Column(Text, default="")
tool_calls_json = Column(Text, default="")
step_results_json = Column(Text, default="")
hook_decisions_json = Column(Text, default="")
was_blocked = Column(Boolean, default=False)
block_hook_name = Column(String(200), default="")
started_at = Column(String, default="")
completed_at = Column(String, default="")
elapsed_ms = Column(Integer, default=0)
node1_ms = Column(Integer, default=0)
node2_ms = Column(Integer, default=0)
node3_ms = Column(Integer, default=0)
node4_ms = Column(Integer, default=0)
created_at = Column(String, default=_utc_now)
__table_args__ = (
    Index("idx_pel_run_id", "pipeline_run_id"),
    Index("idx_pel_user_created", "user_id", "created_at"),
)

class EvolutionLLMInteractionLog(Base):
    __tablename__ = "evolution_llm_interaction_logs"
    id = Column(String, primary_key=True, default=_new_uuid)
    pipeline_run_id = Column(String, index=True)
    conversation_id = Column(String, index=True)
    user_id = Column(String, default="")
    call_category = Column(String(30), default="")
    call_sequence = Column(Integer, default=0)
    model = Column(String(100), default="")
    message_count = Column(Integer, default=0)
    tool_count = Column(Integer, default=0)
    json_mode = Column(Boolean, default=False)
    response_type = Column(String(20), default="")
    response_summary = Column(String(500), default="")
    finish_reason = Column(String(50), default="")
    prompt_tokens = Column(Integer, default=0)
    completion_tokens = Column(Integer, default=0)
    total_tokens = Column(Integer, default=0)
    latency_ms = Column(Integer, default=0)
    is_error = Column(Boolean, default=False)
    error_summary = Column(String(500), default="")
    created_at = Column(String, default=_utc_now)
    __table_args__ = (
        Index("idx_elil_run_id", "pipeline_run_id"),
        Index("idx_elil_category_created", "call_category", "created_at"),
    )

class RAGRetrievalLog(Base):
    __tablename__ = "rag_retrieval_logs"
    id = Column(String, primary_key=True, default=_new_uuid)
    pipeline_run_id = Column(String, index=True)
    conversation_id = Column(String, default="")
    user_id = Column(String, default="")
    query_text = Column(String(500), default="")
    provider = Column(String(30), default="")
    hit_count = Column(Integer, default=0)
    top_score = Column(Float, default=0.0)

```



```

avg_score = Column(Float, default=0.0)
results_summary = Column(Text, default="")
was_used_by_llm = Column(Boolean, default=True)
latency_ms = Column(Integer, default=0)
error_summary = Column(String(500), default="")
created_at = Column(String, default=_utc_now)
__table_args__ = (
    Index("idx_rrl_run_id", "pipeline_run_id"),
    Index("idx_rrl_provider_created", "provider", "created_at"),
)

class SessionLifecycleLog(Base):
    __tablename__ = "session_lifecycle_logs"
    id = Column(String, primary_key=True, default=_new_uuid)
    conversation_id = Column(String, index=True)
    user_id = Column(String, index=True)
    event_type = Column(String(20), default="")
    detail_json = Column(Text, default="")
    message_count = Column(Integer, default=0)
    duration_ms = Column(Integer, default=0)
    created_at = Column(String, default=_utc_now)
    __table_args__ = (
        Index("idx_sll_conv_created", "conversation_id", "created_at"),
    )

class SchedulerJobLog(Base):
    __tablename__ = "scheduler_job_logs"
    id = Column(String, primary_key=True, default=_new_uuid)
    job_id = Column(String, index=True)
    action_type = Column(String(100), default="")
    params_json = Column(Text, default="")
    success = Column(Boolean, default=True)
    summary = Column(String(500), default="")
    elapsed_ms = Column(Integer, default=0)
    error_detail = Column(Text, default="")
    started_at = Column(String, default="")
    completed_at = Column(String, default="")
    created_at = Column(String, default=_utc_now)
    __table_args__ = (
        Index("idx_sjl_action_created", "action_type", "created_at"),
    )

class UserFeedbackSignal(Base):
    __tablename__ = "user_feedback_signals"
    id = Column(String, primary_key=True, default=_new_uuid)
    pipeline_run_id = Column(String, index=True)
    conversation_id = Column(String, default="")
    user_id = Column(String, default="")
    signal_type = Column(String(30), default="")
    signal_strength = Column(Float, default=0.0)
    trigger_message = Column(String(500), default="")
    context_summary = Column(String(500), default="")
    created_at = Column(String, default=_utc_now)
    __table_args__ = (
        Index("idx_ufs_type_created", "signal_type", "created_at"),
        Index("idx_ufs_user_created", "user_id", "created_at"),
    )

```

```

class BusinessEventLog(Base):
    __tablename__ = "business_event_logs"
    id = Column(String, primary_key=True, default=_new_uuid)
    project_id = Column(String, index=True)
    user_id = Column(String, index=True)
    user_name = Column(String(200), default="")
    event_category = Column(String(30), default="")
    event_action = Column(String(30), default="")
    target_type = Column(String(50), default="")
    target_id = Column(String, default="")
    target_no = Column(String(50), default="")
    summary = Column(String(500), default="")
    detail_json = Column(Text, default="")
    pipeline_run_id = Column(String, default="")
    created_at = Column(String, default=_utc_now)
    __table_args__ = (
        Index("idx_bel_category_created", "event_category", "created_at"),
        Index("idx_bel_project_created", "project_id", "created_at"),
    )

class EvolutionDailyInsight(Base):
    __tablename__ = "evolution_daily_insights"
    id = Column(String, primary_key=True, default=_new_uuid)
    insight_date = Column(String, unique=True, nullable=False, comment="1天=YYYY-MM-DD, 多天=YYYY-MM-DD~YYYY-MM-DD")
    analysis_days = Column(Integer, default=1, comment="复盘天数（默认1，最小1）")
    total_messages = Column(Integer, default=0, comment="本周消息数")
    total_pipeline_runs = Column(Integer, default=0, comment="本周 Pipeline 执行数")
    sop_hit_rate = Column(Float, default=0.0, comment="SOP 命中率 0-1")
    fallback_rate = Column(Float, default=0.0, comment="Fallback 率 0-1")
    top_sop_ids = Column(Text, default="[]", comment="JSON: [{sop_id, count}] Top 5")
    feedback_summary = Column(Text, default="", comment="用户反馈信号汇总文本")
    anomaly_flags = Column(Text, default="[]", comment='JSON: ["high_fallback", "low_rag_hit"]')
    insight_text = Column(Text, default="", comment="LLM 生成的完整 JSON 洞察")
    metrics_json = Column(Text, default="{}", comment="完整指标快照 JSON")
    health_score = Column(Integer, default=0, comment="健康评分 0-100")
    created_at = Column(String, default=_utc_now)
    __table_args__ = (
        Index("idx_edi_date", "insight_date"),
    )

class EvolutionRule(Base):
    __tablename__ = "evolution_rules"
    id = Column(String, primary_key=True, default=_new_uuid)
    rule_no = Column(String(20), unique=True, nullable=False, comment="规则编号 R-001")
    title = Column(String(200), nullable=False, comment="规则标题")
    description = Column(Text, default="", comment="规则详细描述")
    evidence_insight_ids = Column(Text, default="[]", comment="JSON: 支撑此规则的 Insight ID 列表")
    category = Column(String(30), default="", comment="routing/prompt/sop/hook/user_memory")
    confidence = Column(Float, default=0.0, comment="置信度 0-1")

```

```

    status = Column(String(20), nullable=False, default="DRAFT",
comment="DRAFT/CONFIRMED/SUPERSEDED/DISCARDED")
    superseded_by = Column(String(20), default="", comment="被哪条规则替代")
    suggested_action = Column(Text, default="", comment="建议的具体改进动作")
    impact_estimate = Column(String(500), default="", comment="预计改进后的指标变化")
    created_at = Column(String, default=_utc_now)
    confirmed_at = Column(String, default="", comment="确认时间")
    __table_args__ = (
        Index("idx_er_status", "status"),
        Index("idx_er_category", "category"),
    )
class EvolutionPatch(Base):
    __tablename__ = "evolution_patches"
    id = Column(String, primary_key=True, default=_new_uuid)
    patch_no = Column(String(20), unique=True, nullable=False, comment="补丁编号 EP-001")
    rule_no = Column(String(20), default="", comment="来源规则编号")
    target_type = Column(String(30), default="",
comment="prompt/sop/skill/hook/user_memory")
    target_path = Column(String(500), default="", comment="目标文件路径（相对 emily-data/）")
    patch_content = Column(Text, default="", comment="变更内容")
    patch_type = Column(String(30), default="",
comment="append/replace_section/insert_after")
    search_anchor = Column(String(500), default="", comment="定位锚点")
    risk_level = Column(String(10), default="", comment="low/medium/high")
    risk_reasoning = Column(String(500), default="", comment="风险等级判定理由")
    validation_criteria = Column(Text, default="", comment="验证标准")
    expected_effect = Column(String(500), default="", comment="预期效果")
    status = Column(String(20), nullable=False, default="DRAFT",
comment="DRAFT/APPLIED/CONFIRMED/ROLLED_BACK/REJECTED")
    applied_at = Column(String, default="")
    validated_at = Column(String, default="")
    validation_result = Column(Text, default="", comment="验证结果 JSON")
    rollback_snapshot = Column(Text, default="", comment="应用前原始内容（用于回滚）")
    created_at = Column(String, default=_utc_now)
    __table_args__ = (
        Index("idx_ep_status", "status"),
        Index("idx_ep_rule", "rule_no"),
    )
class ProjectWorldBook(Base):
    __tablename__ = "project_world_books"
    id = Column(String, primary_key=True, default=_new_uuid)
    project_id = Column(String, ForeignKey("projects.id"), nullable=False,
unique=True, comment="项目归属（FK→projects.id，每项目唯一）")
    version = Column(Integer, default=1, comment="整体版本号（递增）")
    content_json = Column(Text, default="{}", comment="七层结构化 JSON（机器可解析）")
    content_text = Column(Text, default="", comment="纯文本摘要（直接注入 prompt，~400 tokens）")
    layer_versions = Column(Text, default="{}", comment='JSON：每层独立版本号 {"ontology":1,"personnel":1,...}')

```

```

    initialization_tier = Column(Integer, default=0, comment="当前初始化层级 0-4
    (0=未开始, 4=充分运转)")
    initialization_status = Column(Text, default="{}", comment="JSON: 各必备项完成
    情况")
    is_activated = Column(Boolean, default=False, comment="是否达到 T3 可运转级")
    token_count = Column(Integer, default=0, comment="估算 token 数")
    generated_at = Column(String, default=_utc_now, comment="最近生成时间")
    generated_by = Column(String(50), default="manual", comment="生成来源: startup
    / scheduler_data / scheduler_llm / manual")
    created_at = Column(String, default=_utc_now, comment="首次创建时间")
    updated_at = Column(String, default=_utc_now, onupdate=_utc_now, comment="最近
    更新时间")
    __table_args__ = (
        Index("idx_wb_project", "project_id"),
    )
class SystemDescription(Base):
    __tablename__ = "system_descriptions"
    id = Column(String, primary_key=True, default=_new_uuid)
    version = Column(Integer, default=1, comment="整体版本号 (递增)")
    content_json = Column(Text, default="{}", comment="三域结构化 JSON (机器可解
    析)")
    content_text = Column(Text, default="", comment="纯文本摘要 (直接注入 prompt,
    ~400 tokens)")
    domain_versions = Column(Text, default="{}", comment='JSON: 每域独立版本号
    {"database":1,"file":1,"permission":1}')
    schema_hash = Column(String(64), default="", comment="Base.metadata 的 SHA-256
    hash")
    permission_hash = Column(String(64), default="", comment="PermissionLevel +
    INHERITANCE_CHAIN 的 SHA-256 hash")
    file_model_hash = Column(String(64), default="", comment="FileCategory 枚举 +
    File 模型的 SHA-256 hash")
    token_count = Column(Integer, default=0, comment="估算 token 数")
    generated_at = Column(String, default=_utc_now, comment="最近生成时间")
    generated_by = Column(String(50), default="manual", comment="生成来源: startup
    / scheduler / manual")
    created_at = Column(String, default=_utc_now, comment="首次创建时间")
    updated_at = Column(String, default=_utc_now, onupdate=_utc_now, comment="最近
    更新时间")

```

文件编号: 20 文件路径: emily-core\emily_core\infrastructure\database\scripts\fill_users_required_fields.py

```

import sys
import logging
from datetime import datetime, timezone
from pathlib import Path
project_root = Path(__file__).parent.parent.parent.parent.parent
sys.path.insert(0, str(project_root))
from emily_core.infrastructure.database.session import get_session, init_db
from emily_core.infrastructure.database.models import User
from sqlalchemy import text, func
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger("fill_users_fields")

```

```
def _utc_now() -> str:
    return datetime.now(timezone.utc).isoformat()

def get_or_create_system_user(session) -> str:
    system_user = session.query(User).filter(User.username == "system").first()
    if not system_user:
        logger.info("创建系统用户...")
        now = _utc_now()
        system_user = User(
            username="system",
            phone="13800000000",
            email="system@emily.local",
            status="active",
            is_admin=True,
            gender=1,
            id_card="",
            qq="",
            wechat="",
            remark="系统内置用户，用于数据迁移和自动操作",
            creator_id="",
            is_deleted=False,
            perm_list='["*"]',
            org_category=4,
            level=6,
            created_at=now,
            updated_at=now,
        )
        session.add(system_user)
        session.flush()
        system_user.creator_id = system_user.id
        session.add(system_user)
        session.flush()
        logger.info(f"系统用户已创建，ID: {system_user.id}")
    else:
        logger.info(f"系统用户已存在，ID: {system_user.id}")
    return system_user.id

def fill_created_at(session):
    now = _utc_now()
    result = session.query(User).filter(
        (User.created_at.is_(None)) |
        (User.created_at == "") |
        (User.created_at == "None")
    ).update(
        {"created_at": now},
        synchronize_session=False
    )
    session.flush()
    logger.info(f"填充 created_at: {result} 条记录")
    return result

def fill_updated_at(session):
    users_to_update = session.query(User).filter(
        (User.updated_at.is_(None)) |
        (User.updated_at == "") |
        (User.updated_at == "None")
    ).all()
```

```

count = 0
for user in users_to_update:
    user.updated_at = user.created_at or _utc_now()
    session.add(user)
    count += 1
session.flush()
logger.info(f"填充 updated_at: {count} 条记录")
return count

def fill_creator_id(session, system_user_id: str):
    system_result = session.query(User).filter(
        User.username == "system",
        (User.creator_id.is_(None)) | (User.creator_id == "") | (User.creator_id
== "None")
    ).update(
        {"creator_id": system_user_id},
        synchronize_session=False
    )
    other_result = session.query(User).filter(
        User.username != "system",
        (User.creator_id.is_(None)) | (User.creator_id == "") | (User.creator_id
== "None")
    ).update(
        {"creator_id": system_user_id},
        synchronize_session=False
    )
    session.flush()
    total = system_result + other_result
    logger.info(f"填充 creator_id: {total} 条记录")
    return total

def generate_username(user: User, existing_usernames: set) -> str:
    if user.phone and user.phone.strip() and user.phone.strip().lower() != "none":
        digits = ''.join(c for c in user.phone if c.isdigit())
        if len(digits) >= 4:
            base = f"user_{digits[-4:]}"
        else:
            base = f"user_{digits}"
    elif user.email and user.email.strip() and user.email.strip().lower() !=
"none":
        email_prefix = user.email.split('@')[0]
        import re
        base = re.sub(r'^[a-zA-Z0-9_]', '_', email_prefix)
        if len(base) > 20:
            base = base[:20]
    else:
        base = f"user_{user.id[:8]}"
    final_name = base
    counter = 0
    while final_name in existing_usernames:
        counter += 1
        final_name = f"{base}_{counter}"
    existing_usernames.add(final_name)
    return final_name

def fill_username(session):
    existing_usernames = set()

```

```
for row in session.query(User.username).filter(
    User.username.is_not(None),
    User.username != "",
    User.username != "None"
).all():
    existing_usernames.add(row.username)
users_to_update = session.query(User).filter(
    (User.username.is_(None)) |
    (User.username == "") |
    (User.username == "None")
).order_by(User.created_at).all()
count = 0
for user in users_to_update:
    new_username = generate_username(user, existing_usernames)
    user.username = new_username
    session.add(user)
    logger.info(f" 用户 {user.id} -> 用户名: {new_username}")
    count += 1
session.flush()
logger.info(f"填充 username: {count} 条记录")
return count

def verify_fill_result(session):
    logger.info("\n" + "="*60)
    logger.info("验证填充结果")
    logger.info("="*60)
    null_username = session.query(User).filter(
        (User.username.is_(None)) | (User.username == "") | (User.username ==
"None")
    ).count()
    null_creator_id = session.query(User).filter(
        (User.creator_id.is_(None)) | (User.creator_id == "") | (User.creator_id
== "None")
    ).count()
    null_created_at = session.query(User).filter(
        (User.created_at.is_(None)) | (User.created_at == "") | (User.created_at
== "None")
    ).count()
    null_updated_at = session.query(User).filter(
        (User.updated_at.is_(None)) | (User.updated_at == "") | (User.updated_at
== "None")
    ).count()
    if all(x == 0 for x in [null_username, null_creator_id, null_created_at,
null_updated_at]):
        logger.info("✅ 所有必填字段填充完成！")
    else:
        logger.warning("⚠️ 仍存在空值记录：")
        logger.warning(f" - username: {null_username} 条")
        logger.warning(f" - creator_id: {null_creator_id} 条")
        logger.warning(f" - created_at: {null_created_at} 条")
        logger.warning(f" - updated_at: {null_updated_at} 条")
    logger.info("\n" + "="*60)
    logger.info("数据摘要（前20条）：")
    logger.info("="*60)
    users = session.query(User).order_by(User.created_at.desc()).limit(20).all()
```



```

    for user in users:
        logger.info(f" {user.username:<20} | created: {user.created_at[:19]} |
creator: {user.creator_id[:8]}")
    total = session.query(User).count()
    logger.info(f"\n总用户数: {total}")
def main():
    logger.info("="*60)
    logger.info("开始填充 users 表必填字段")
    logger.info("="*60)
    init_db()
    with get_session() as session:
        system_user_id = get_or_create_system_user(session)
        fill_created_at(session)
        fill_updated_at(session)
        fill_creator_id(session, system_user_id)
        fill_username(session)
        verify_fill_result(session)
        logger.info("\n" + "="*60)
        logger.info("填充完成! ")
        logger.info("="*60)
if __name__ == "__main__":
    main()

```

文件编号: 21 文件路径: emily-core\emily_core\infrastructure\database\session.py

```

import logging
from contextlib import contextmanager
from urllib.parse import quote_plus
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, Session
from .models import Base
logger = logging.getLogger("emily.db")
_DEFAULT_PG_HOST = "emily-postgres"
_DEFAULT_PG_PORT = 5432
_DEFAULT_PG_DB = "emily"
_DEFAULT_PG_USER = "emily"
_DEFAULT_PG_PASSWORD = "emily_secret_2026"
_engine = None
_SessionLocal = None
def _create_pg_engine(
    host: str,
    port: int,
    db: str,
    user: str,
    password: str,
):
    url = f"postgresql://{user}:{quote_plus(password)}@{host}:{port}/{db}"
    return create_engine(
        url,
        echo=False,
        pool_size=5,
        max_overflow=10,
    )

```



```

        pool_pre_ping=True,
        pool_recycle=3600,
    )
def _ensure_columns(engine) -> None:
    _PENDING_COLUMNS = {
        "hook_execution_logs": [
            ("user_id", "VARCHAR", ""),
            ("sop_id", "VARCHAR", ""),
            ("block_reason", "VARCHAR(500)", ""),
            ("session_level", "INTEGER", "NULL"),
        ],
    }
    from sqlalchemy import text as sa_text
    with engine.connect() as conn:
        for table_name, columns in _PENDING_COLUMNS.items():
            table_exists = conn.execute(
                sa_text(
                    "SELECT EXISTS ("
                    "  SELECT 1 FROM information_schema.tables"
                    "  WHERE table_name = :tbl"
                    ")"
                ),
                {"tbl": table_name},
            ).scalar()
            if not table_exists:
                continue
            existing_rows = conn.execute(
                sa_text(
                    "SELECT column_name FROM information_schema.columns"
                    "  WHERE table_name = :tbl"
                ),
                {"tbl": table_name},
            ).fetchall()
            existing = {r[0] for r in existing_rows}
            for col_name, col_type, col_default in columns:
                if col_name in existing:
                    continue
                default_clause = ""
                if col_default != "NULL":
                    default_clause = f" DEFAULT {col_default}"
                else:
                    default_clause = ""
                conn.execute(
                    sa_text(
                        f"ALTER TABLE {table_name} "
                        f"ADD COLUMN {col_name} {col_type}{default_clause}"
                    )
                )
            conn.commit()
            logger.info(
                "Schema migration: added column %s to %s",
                col_name, table_name,
            )
def init_db(

```

```

    db_url: str | None = None,
    *,
    pg_host: str = _DEFAULT_PG_HOST,
    pg_port: int = _DEFAULT_PG_PORT,
    pg_db: str = _DEFAULT_PG_DB,
    pg_user: str = _DEFAULT_PG_USER,
    pg_password: str = _DEFAULT_PG_PASSWORD,
) -> None:
    global _engine, _SessionLocal
    if _engine is not None:
        return
    if db_url:
        _engine = create_engine(
            db_url,
            echo=False,
            pool_size=5,
            max_overflow=10,
            pool_pre_ping=True,
            pool_recycle=3600,
        )
        logger.info("Database engine: PostgreSQL (from URL)")
    else:
        _engine = _create_pg_engine(pg_host, pg_port, pg_db, pg_user, pg_password)
        logger.info("Database engine: PostgreSQL (%s:%d/%s)", pg_host, pg_port,
pg_db)
    _SessionLocal = sessionmaker(
        autocommit=False,
        autoflush=False,
        bind=_engine,
        expire_on_commit=False,
    )
    Base.metadata.create_all(bind=_engine)
    _ensure_columns(_engine)
    logger.info(
        "Database initialized (PostgreSQL): %d tables",
        len(Base.metadata.tables),
    )

@contextmanager
def get_session() -> Session:
    if _SessionLocal is None:
        init_db()
    session = _SessionLocal()
    try:
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
        session.close()

def get_session_raw() -> Session:
    if _SessionLocal is None:
        init_db()
    return _SessionLocal()

```

```
def get_db_path() -> str:
    if _engine is not None:
        return str(_engine.url)
    return "postgresql://emily@emily-postgres:5432/emily"
```

文件编号: 22 文件路径: emily-core\emily_core\infrastructure\llm\client.py

```
import json
import logging
import time
from typing import Callable, Optional
from openai import AsyncOpenAI
logger = logging.getLogger("emily.llm")
class LLMClient:
    def __init__(
        self,
        api_key: str,
        base_url: str = "https://api.deepseek.com",
        model: str = "deepseek-chat",
        temperature: float = 0.1,
        max_tokens: int = 1024,
    ):
        self.model = model
        self.temperature = temperature
        self.max_tokens = max_tokens
        self._client = AsyncOpenAI(
            api_key=api_key,
            base_url=base_url,
        )
        self._trace_callback: Callable | None = None
        logger.info(
            "LLMClient initialized: model=%s, base_url=%s, temperature=%.2f",
            model, base_url, temperature,
        )
    def set_trace_callback(self, callback: Callable | None):
        self._trace_callback = callback
    async def chat_messages(
        self,
        messages: list[dict],
        *,
        json_mode: bool = False,
        tools: list[dict] | None = None,
        temperature: float | None = None,
        max_tokens: int | None = None,
    ) -> dict:
        call_seq = getattr(self, "_trace_call_seq", 0) + 1
        self._trace_call_seq = call_seq
        call_type = "chat_messages" + ("_json" if json_mode else "")
        if self._trace_callback:
            try:
                self._trace_callback({
                    "phase": "start",
```

```

        "call_type": call_type,
        "call_sequence": call_seq,
        "model": self.model,
        "message_count": len(messages),
        "tool_count": len(tools) if tools else 0,
        "json_mode": json_mode,
    })
    except Exception:
        pass
    t0 = time.time()
    kwargs = dict(
        model=self.model,
        messages=messages,
        temperature=temperature if temperature is not None else
self.temperature,
        max_tokens=max_tokens if max_tokens is not None else self.max_tokens,
    )
    if json_mode:
        kwargs["response_format"] = {"type": "json_object"}
    if tools:
        kwargs["tools"] = tools
    try:
        response = await self._client.chat.completions.create(**kwargs)
    except Exception as e:
        if tools and "tools" in str(e).lower():
            logger.warning("Tools not supported, falling back: %s", e)
            del kwargs["tools"]
            try:
                response = await
self._client.chat.completions.create(**kwargs)
            except Exception as e2:
                logger.error("LLM chat_messages fallback failed: %s", e2)
                raise
        elif json_mode and "response_format" in str(e).lower():
            logger.warning("LLM json_mode not supported, falling back: %s", e)
            del kwargs["response_format"]
            response = await self._client.chat.completions.create(**kwargs)
        else:
            logger.error("LLM chat_messages failed: %s", e)
            raise
    choice = response.choices[0]
    finish_reason = choice.finish_reason or ""
    reasoning_content = getattr(choice.message, "reasoning_content", None) or
""

    elapsed_ms = int((time.time() - t0) * 1000)
    if choice.message.tool_calls:
        tc = choice.message.tool_calls[0]
        try:
            arguments = json.loads(tc.function.arguments)
        except json.JSONDecodeError:
            logger.warning("Failed to parse tool arguments: %s",
tc.function.arguments)
            arguments = {}
        logger.debug("LLM tool call: %s(%s)", tc.function.name, str(arguments))

```

```

[:200])
    if self._trace_callback:
        try:
            self._trace_callback({
                "phase": "end", "call_type": call_type, "call_sequence":
call_seq,
                "response_type": "tool_call",
                "response_summary": f"{tc.function.name}({str(arguments)
[:200]})",
                "finish_reason": finish_reason,
                "prompt_tokens": getattr(response.usage, "prompt_tokens",
0) if response.usage else 0,
                "completion_tokens": getattr(response.usage,
"completion_tokens", 0) if response.usage else 0,
                "total_tokens": getattr(response.usage, "total_tokens", 0)
if response.usage else 0,
                "latency_ms": elapsed_ms,
            })
        except Exception:
            pass
    return {
        "type": "tool_call",
        "tool_name": tc.function.name,
        "tool_arguments": arguments,
        "tool_call_id": tc.id,
        "finish_reason": finish_reason,
        "reasoning_content": reasoning_content,
    }
    content = choice.message.content or ""
    logger.debug("LLM chat_messages: %d chars, finish=%s, elapsed=%dms",
        len(content), finish_reason, elapsed_ms)
    if self._trace_callback:
        try:
            self._trace_callback({
                "phase": "end", "call_type": call_type, "call_sequence":
call_seq,
                "response_type": "json" if json_mode else "text",
                "response_summary": content[:500],
                "finish_reason": finish_reason,
                "prompt_tokens": getattr(response.usage, "prompt_tokens", 0)
if response.usage else 0,
                "completion_tokens": getattr(response.usage,
"completion_tokens", 0) if response.usage else 0,
                "total_tokens": getattr(response.usage, "total_tokens", 0) if
response.usage else 0,
                "latency_ms": elapsed_ms,
            })
        except Exception:
            pass
    if json_mode:
        data = self._parse_json_response(content)
        return {"type": "json", "data": data, "finish_reason": finish_reason}
    return {"type": "text", "content": content, "finish_reason":
finish_reason}

```

```

@staticmethod
def _parse_json_response(raw: str) -> dict:
    try:
        return json.loads(raw)
    except json.JSONDecodeError:
        import re
        match = re.search(r"```(?:json)?\s*\n?(.*?)\n?```", raw, re.DOTALL)
        if match:
            try:
                return json.loads(match.group(1).strip())
            except json.JSONDecodeError:
                pass
        logger.error("Failed to parse LLM response as JSON: %s", raw[:500])
        raise ValueError(f"LLM response is not valid JSON: {raw[:200]}")
    async def chat(self, system_prompt: str, user_message: str) -> str:
        result = await self.chat_messages([
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_message},
        ])
        return result["content"]
    async def chat_json(self, system_prompt: str, user_message: str) -> dict:
        result = await self.chat_messages([
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": user_message},
        ], json_mode=True)
        return result["data"]
    async def chat_with_tools(
        self,
        messages: list[dict],
        tools: list[dict],
        temperature: float | None = None,
        max_tokens: int | None = None,
    ) -> dict:
        return await self.chat_messages(
            messages,
            tools=tools,
            temperature=temperature,
            max_tokens=max_tokens,
        )

```

文件编号: 23 文件路径: emily-core\emily_core\infrastructure\llm\prompt_loader.py

```

from __future__ import annotations
import logging
import os
from pathlib import Path
logger = logging.getLogger("emily.prompt_loader")
_cache: dict[str, str] = {}
_DEFAULTS: dict[str, str] = {
    "routing": None,
}
def _find_prompt_path(name: str, prompts_dir: str = "") -> Path | None:

```

```

filename = f"{name}.md"
candidates: list[Path] = []
if prompts_dir:
    candidates.append(Path(prompts_dir) / filename)
env_dir = os.environ.get("EMILY_PROMPTS_DIR")
if env_dir:
    candidates.append(Path(env_dir) / filename)
candidates.append(Path("/app/prompts") / filename)
candidates.append(
    Path(__file__).resolve().parents[4] / "emily-data" / "prompts" / filename
)
for p in candidates:
    if p.exists():
        return p
return None
def _read_file(path: Path) -> str:
    lines: list[str] = []
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            stripped = line.strip()
            if stripped.startswith("<!--") and stripped.endswith("-->"):
                continue
            lines.append(line)
    return "".join(lines)
def load_prompt(name: str, prompts_dir: str = "") -> str:
    if name in _cache:
        return _cache[name]
    path = _find_prompt_path(name, prompts_dir)
    if path is not None:
        try:
            content = _read_file(path)
            _cache[name] = content
            logger.debug("Prompt '%s' loaded from %s", name, path)
            return content
        except Exception as e:
            logger.warning("Failed to read prompt '%s' from %s: %s", name, path,
e)
            fallback = _DEFAULTS.get(name)
            if fallback is not None:
                _cache[name] = fallback
                logger.warning(
                    "Prompt '%s' file not found, using hardcoded default", name
                )
                return fallback
            if name == "routing":
                logger.info("Prompt 'routing' deprecated – redirecting to 'session'")
                return load_prompt("session", prompts_dir)
            raise FileNotFoundError(
                f"Prompt '{name}' not found on disk or in defaults. "
                f"Checked: prompts_dir={prompts_dir or '(auto-resolved)'}"
            )
def reload_prompt(name: str) -> str:
    _cache.pop(name, None)
    return load_prompt(name)

```

```
def clear_cache() -> None:
    _cache.clear()
    logger.debug("Prompt cache cleared")
```

文件编号: 24 文件路径: emily-core\emily_core\infrastructure\logging\business_event_logger.py

```
from __future__ import annotations
import logging
from typing import Any
logger = logging.getLogger("emily.evolution.business_event_logger")
class BusinessEventLogger:
    _current_context: dict[str, str] = {}
    @classmethod
    def set_context(cls, pipeline_run_id: str = "", conversation_id: str = "") ->
None:
        cls._current_context = {
            "pipeline_run_id": pipeline_run_id,
            "conversation_id": conversation_id,
        }
    @classmethod
    def clear_context(cls) -> None:
        cls._current_context = {}
    @classmethod
    async def log(
        cls,
        *,
        event_category: str,
        event_action: str,
        target_type: str = "",
        target_id: str = "",
        target_no: str = "",
        summary: str = "",
        detail_json: str = "",
        project_id: str = "",
        user_id: str = "",
        user_name: str = "",
        pipeline_run_id: str = "",
        conversation_id: str = "",
    ) -> None:
        ctx = cls._current_context
        if not pipeline_run_id:
            pipeline_run_id = ctx.get("pipeline_run_id", "")
        if not conversation_id:
            conversation_id = ctx.get("conversation_id", "")
        from .log_writer import EvolutionLogWriter
        from ...infrastructure.database.models import BusinessEventLog
        await EvolutionLogWriter.write(
            BusinessEventLog,
            project_id=project_id,
            user_id=user_id,
            user_name=user_name,
            event_category=event_category,
```



```

        event_action=event_action,
        target_type=target_type,
        target_id=target_id,
        target_no=target_no,
        summary=summary,
        detail_json=detail_json,
        pipeline_run_id=pipeline_run_id,
    )

```

文件编号: 25 文件路径: emily-core\emily_core\infrastructure\logging\feedback_detector.py

```

from __future__ import annotations
import logging
logger = logging.getLogger("emily.evolution.feedback_detector")
class FeedbackSignalDetector:
    _PATTERNS = {
        "explicit_correction": {
            "keywords": ["不对", "错了", "不是这个", "重新", "搞错了", "不是我要的",
"搞反了"],
            "strength": 0.9,
        },
        "truncation_followup": {
            "keywords": ["继续", "然后呢", "还有吗", "后面呢", "接着说", "说完"],
            "strength": 0.6,
        },
        "positive": {
            "keywords": ["好的", "谢谢", "对", "收到", "可以", "没问题", "确认"],
            "strength": 0.5,
        },
        "repeat_request": {
            "keywords": [],
            "strength": 0.8,
        },
    }
    @staticmethod
    def detect(
        user_message: str,
        *,
        pipeline_run_id: str = "",
        conversation_id: str = "",
        user_id: str = "",
        assistant_reply: str = "",
    ) -> list[dict]:
        if not user_message or len(user_message.strip()) < 2:
            return []
        text = user_message.strip().lower()
        signals = []
        for signal_type, config in FeedbackSignalDetector._PATTERNS.items():
            if signal_type == "repeat_request":
                continue
            for kw in config["keywords"]:
                if kw in text:

```

```

        signals.append({
            "signal_type": signal_type,
            "signal_strength": config["strength"],
            "trigger_message": user_message[:200],
            "context_summary": (assistant_reply or "")[:200],
        })
    break
    return signals
@staticmethod
async def log_signals(
    signals: list[dict],
    *,
    pipeline_run_id: str = "",
    conversation_id: str = "",
    user_id: str = "",
) -> None:
    if not signals:
        return
    from .log_writer import EvolutionLogWriter
    from ...infrastructure.database.models import UserFeedbackSignal
    for sig in signals:
        await EvolutionLogWriter.write(
            UserFeedbackSignal,
            pipeline_run_id=pipeline_run_id,
            conversation_id=conversation_id,
            user_id=user_id,
            signal_type=sig.get("signal_type", ""),
            signal_strength=sig.get("signal_strength", 0.0),
            trigger_message=sig.get("trigger_message", ""),
            context_summary=sig.get("context_summary", ""),
        )

```

文件编号: 26 文件路径: emily-core\emily_core\infrastructure\logging\legacy_log_bridge.py

```

from __future__ import annotations
import json
import logging
from datetime import datetime, timezone, timedelta
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from ...workitem.pipeline.context import BusContext
logger = logging.getLogger("emily.legacy_log_bridge")
BEIJING_TZ = timezone(timedelta(hours=8))
async def write_legacy_logs(context: "BusContext", started_at: str) -> None:
    wi = context.work_item
    if wi is None:
        return
    from .log_writer import EvolutionLogWriter
    from ...infrastructure.database.models import (
        SOPRoutingLog, AgentReasoningLog, LLMInteractionLog, ToolCallLog,
    )
    intent = context.intent

```

```

sop_id = getattr(intent, "sop_id", None) or wi.sop_id or ""
confidence = getattr(intent, "confidence", "none") if intent else "none"
is_hit = bool(sop_id) and sop_id != "__FALLBACK_SOP__"
now_beijing = datetime.now(BEIJING_TZ)
message_content = wi.user_input or ""
try:
    await EvolutionLogWriter.write(
        SOPRoutingLog,
        log_date=now_beijing.strftime("%Y-%m-%d"),
        log_time=now_beijing.strftime("%H:%M:%S"),
        user_id=context.user_id or None,
        conversation_id=context.message.conversation_id if context.message
else None,
        message_id=None,
        message_content=message_content[:500],
        matched_sop_id=sop_id or None,
        is_hit=is_hit,
        match_confidence=confidence,
        fallback_action="" if is_hit else "fallback",
        llm_reasoning=getattr(intent, "reasoning", "")[:200] if intent else
"",
        execution_result="",
    )
except Exception as e:
    logger.warning("Legacy sop_routing_logs write failed: %s", e)
try:
    db_msg_id = context.db_message_id if hasattr(context, "db_message_id")
else None
    elapsed_ms = 0
    for sr in (wi.step_results or []):
        for tc in (sr.tool_calls or []):
            elapsed_ms += tc.elapsed_ms
    steps_json = []
    for sr in (wi.step_results or []):
        steps_json.append({
            "step_id": sr.step_id,
            "success": sr.success,
            "output": (sr.output or "")[:200],
        })
    reply_preview = (wi.result_text or "")[:500]
    error_msg = wi.error_message or ""
    await EvolutionLogWriter.write(
        AgentReasoningLog,
        message_id=db_msg_id or "",
        user_id=context.user_id or None,
        conversation_id=None,
        iteration_count=wi.llm_call_count,
        elapsed_ms=elapsed_ms,
        max_iterations_reached=False,
        matched_sop_id=sop_id,
        match_confidence=confidence,
        is_compound=getattr(intent, "is_compound", False) if intent else
False,
        fallback=not is_hit,

```

```

        execution_result="success" if not error_msg else "failed",
        reply_preview=reply_preview,
        error_message=error_msg[:500],
        steps_json=json.dumps(steps_json, ensure_ascii=False),
    )
except Exception as e:
    logger.warning("Legacy agent_reasoning_logs write failed: %s", e)
try:
    for sr in (wi.step_results or []):
        for tc in (sr.tool_calls or []):
            try:
                await EvolutionLogWriter.write(
                    ToolCallLog,
                    reasoning_log_id=None,
                    llm_interaction_id=None,
                    step_index=0,
                    tool_name=tc.tool_name,
                    tool_arguments=json.dumps(tc.tool_input,
ensure_ascii=False, default=str)[:5000],
                    tool_result_summary=str(tc.tool_output)[:500] if
tc.tool_output else "",
                    is_success=tc.success,
                    error_message="",
                    elapsed_ms=tc.elapsed_ms,
                )
            except Exception as e:
                logger.warning("Legacy tool_call_logs write failed: %s", e)
if sr.tool_calls:
    try:
        await EvolutionLogWriter.write(
            LLMInteractionLog,
            reasoning_log_id=None,
            message_id=context.db_message_id if hasattr(context,
"db_message_id") else None,
            call_sequence=0,
            call_type="chat_with_tools",
            model="",
            prompt_summary=sr.output[:500] if sr.output else "",
            user_message_count=1,
            tool_count=len(sr.tool_calls),
            response_type="tool_call",
            response_summary=sr.output[:500] if sr.output else "",
            finish_reason="stop",
            prompt_tokens=0,
            completion_tokens=0,
            total_tokens=0,
            latency_ms=sum(tc.elapsed_ms for tc in sr.tool_calls),
        )
    except Exception as e:
        logger.warning("Legacy llm_interaction_logs write failed: %s",
e)
except Exception as e:
    logger.warning("Legacy log bridge tool/llm loop failed: %s", e)

```

文件编号: 27 文件路径: emily-core\emily_core\infrastructure\logging\llm_logger.py

```
from __future__ import annotations
import logging
from typing import Any
logger = logging.getLogger("emily.evolution.llm_logger")
class LLMInteractionLogger:
    _current_context: dict[str, str] = {}
    @classmethod
    def set_context(cls, pipeline_run_id: str = "",
                  conversation_id: str = "", user_id: str = "",
                  call_category: str = "") -> None:
        cls._current_context = {
            "pipeline_run_id": pipeline_run_id,
            "conversation_id": conversation_id,
            "user_id": user_id,
            "call_category": call_category,
        }
    @classmethod
    def clear_context(cls) -> None:
        cls._current_context = {}
    @classmethod
    def make_callback(cls):
        def callback(data: dict) -> None:
            if data.get("phase") != "end":
                return
            cls._on_llm_call_end(data)
        return callback
    @classmethod
    def _on_llm_call_end(cls, data: dict) -> None:
        from .log_writer import EvolutionLogWriter
        from ...infrastructure.database.models import EvolutionLLMInteractionLog
        ctx = cls._current_context
        call_category = ctx.get("call_category", "")
        if not call_category:
            call_type = data.get("call_type", "")
            if "intent" in call_type or "json" in call_type:
                call_category = "intent"
            elif "plan" in call_type:
                call_category = "planning"
            else:
                call_category = "execution"
        is_error = data.get("finish_reason") == "error" or
data.get("response_type") == "error"
        error_summary = ""
        if is_error:
            error_summary = str(data.get("error", ""))[:500]
        EvolutionLogWriter.write_sync(
            EvolutionLLMInteractionLog,
            pipeline_run_id=ctx.get("pipeline_run_id", ""),
            conversation_id=ctx.get("conversation_id", ""),
```

```

        user_id=ctx.get("user_id", ""),
        call_category=call_category,
        call_sequence=data.get("call_sequence", 0),
        model=data.get("model", ""),
        message_count=data.get("message_count", 0),
        tool_count=data.get("tool_count", 0),
        json_mode=data.get("json_mode", False),
        response_type=data.get("response_type", ""),
        response_summary=(data.get("response_summary", "") or "")[:500],
        finish_reason=data.get("finish_reason", ""),
        prompt_tokens=data.get("prompt_tokens", 0),
        completion_tokens=data.get("completion_tokens", 0),
        total_tokens=data.get("total_tokens", 0),
        latency_ms=data.get("latency_ms", 0),
        is_error=is_error,
        error_summary=error_summary,
    )

```

文件编号: 28 文件路径: emily-core\emily_core\infrastructure\logging\log_writer.py

```

from __future__ import annotations
import asyncio
import logging
from datetime import datetime, timezone
from typing import Any
logger = logging.getLogger("emily.evolution_log_writer")
class EvolutionLogWriter:
    TEXT_LIMIT = 5000
    STRING_LIMIT = 500
    _LONG_FIELD_SUFFIXES = ("_json", "_summary", "_detail", "_text")
    _LONG_FIELD_NAMES = ("detail_json", "results_summary", "tool_calls_json",
                          "step_results_json", "hook_decisions_json",
                          "error_detail",
                          "params_json", "context_summary")

    @staticmethod
    async def write(model_class, **kwargs):
        try:
            kwargs = EvolutionLogWriter._truncate(kwargs)
            kwargs = EvolutionLogWriter._fill_defaults(kwargs)
            await asyncio.to_thread(EvolutionLogWriter._sync_write, model_class,
            kwargs)
        except Exception as e:
            logger.warning("EvolutionLogWriter.write failed for %s: %s",
                           getattr(model_class, "__tablename__", "?"), e)

    @staticmethod
    def write_sync(model_class, **kwargs):
        try:
            kwargs = EvolutionLogWriter._truncate(kwargs)
            kwargs = EvolutionLogWriter._fill_defaults(kwargs)
            EvolutionLogWriter._sync_write(model_class, kwargs)
        except Exception as e:
            logger.warning("EvolutionLogWriter.write_sync failed for %s: %s",

```

```

        getattr(model_class, "__tablename__", "?"), e)

    @staticmethod
    def _sync_write(model_class, kwargs: dict):
        from ..database.session import get_session
        with get_session() as session:
            entry = model_class(**kwargs)
            session.add(entry)
            session.commit()

    @staticmethod
    def _truncate(kwargs: dict) -> dict:
        truncated = {}
        for k, v in kwargs.items():
            if isinstance(v, str):
                limit = EvolutionLogWriter.TEXT_LIMIT if (
                    any(k.endswith(s) for s in
                        EvolutionLogWriter._LONG_FIELD_SUFFIXES)
                    or k in EvolutionLogWriter._LONG_FIELD_NAMES
                ) else EvolutionLogWriter.STRING_LIMIT
                truncated[k] = v[:limit]
            else:
                truncated[k] = v
        return truncated

    @staticmethod
    def _fill_defaults(kwargs: dict) -> dict:
        if "created_at" not in kwargs:
            kwargs["created_at"] = datetime.now(timezone.utc).isoformat()
        return kwargs

```

文件编号: 29 文件路径: emily-core\emily_core\infrastructure\logging\pipeline_logger.py

```

from __future__ import annotations
import json
import logging
from datetime import datetime, timezone
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from ...workitem.pipeline.context import BusContext
logger = logging.getLogger("emily.evolution.pipeline_logger")
class PipelineExecutionLogger:
    @staticmethod
    async def log(context: "BusContext", started_at: str, node_timings: dict) ->
None:
        from .log_writer import EvolutionLogWriter
        from ...infrastructure.database.models import PipelineExecutionLog
        wi = context.work_item
        if wi is None:
            return
        intent = context.intent
        sop_id = getattr(intent, "sop_id", None) or wi.sop_id or ""
        confidence = getattr(intent, "confidence", "none") if intent else "none"
        is_compound = getattr(intent, "is_compound", False) if intent else False
        is_fallback = getattr(intent, "fallback", False) if intent else (wi.sop_id

```

```

is None)
    from ...workitem.workitem_state import WorkItemState
    final_status = "DONE"
    if context.should_abort:
        final_status = "ABORTED"
    elif wi.state == WorkItemState.FAILED:
        final_status = "FAILED"
    tool_calls_data = []
    step_results_data = []
    for sr in (wi.step_results or []):
        sr_dict = {
            "step_id": sr.step_id,
            "success": sr.success,
            "output": (sr.output or "")[:200],
        }
        step_results_data.append(sr_dict)
        for tc in (sr.tool_calls or []):
            tool_calls_data.append({
                "tool_name": tc.tool_name,
                "success": tc.success,
                "elapsed_ms": tc.elapsed_ms,
            })
    hook_decisions = []
    for w in (context.warnings or []):
        hook_decisions.append({"decision": "WARN", "message": w[:200]})
    session_ctx = context.get_session_context()
    user_name = getattr(session_ctx, "user_name", "") if session_ctx else ""
    user_level = getattr(session_ctx, "level", 1) if session_ctx else 1
    completed_at = datetime.now(timezone.utc).isoformat()
    elapsed_ms = sum(node_timings.values())
    await EvolutionLogWriter.write(
        PipelineExecutionLog,
        pipeline_run_id=context.pipeline_run_id,
        conversation_id=context.message.conversation_id if context.message
    else "",
        user_id=context.user_id or "",
        user_name=user_name,
        user_level=user_level,
        matched_sop_id=sop_id,
        match_confidence=confidence,
        is_compound=is_compound,
        is_fallback=is_fallback,
        intent_reasoning=getattr(intent, "reasoning", "")[:500] if intent else
    "",
        final_status=final_status,
        abort_reason=(context.abort_reason or "")[:500],
        result_text=(wi.result_text or "")[:1000],
        tool_calls_json=json.dumps(tool_calls_data, ensure_ascii=False),
        step_results_json=json.dumps(step_results_data, ensure_ascii=False),
        hook_decisions_json=json.dumps(hook_decisions, ensure_ascii=False),
        was_blocked=context.should_abort,
        block_hook_name="",
        started_at=started_at,
        completed_at=completed_at,

```



```
        elapsed_ms=elapsed_ms,  
        node1_ms=node_timings.get("wi_node1", 0),  
        node2_ms=node_timings.get("wi_node2", 0),  
        node3_ms=node_timings.get("wi_node3", 0),  
        node4_ms=node_timings.get("wi_node4", 0),  
    )
```

文件编号: 30 文件路径: emily-core\emily_core\infrastructure\logging\rag_logger.py

```
from __future__ import annotations  
import json  
import logging  
logger = logging.getLogger("emily.evolution.rag_logger")  
class RAGRetrievalLogger:  
    @staticmethod  
    async def log(  
        *,  
        query_text: str,  
        provider: str,  
        hit_count: int = 0,  
        top_score: float = 0.0,  
        avg_score: float = 0.0,  
        results_summary: str = "",  
        was_used_by_llm: bool = True,  
        latency_ms: int = 0,  
        error_summary: str = "",  
        pipeline_run_id: str = "",  
        conversation_id: str = "",  
        user_id: str = "",  
    ) -> None:  
        from .log_writer import EvolutionLogWriter  
        from ...infrastructure.database.models import RAGRetrievalLog  
        await EvolutionLogWriter.write(  
            RAGRetrievalLog,  
            pipeline_run_id=pipeline_run_id,  
            conversation_id=conversation_id,  
            user_id=user_id,  
            query_text=query_text,  
            provider=provider,  
            hit_count=hit_count,  
            top_score=top_score,  
            avg_score=avg_score,  
            results_summary=results_summary,  
            was_used_by_llm=was_used_by_llm,  
            latency_ms=latency_ms,  
            error_summary=error_summary,  
        )
```

文件编号: 31 文件路径: emily-core\emily_core\infrastructure\logging\scheduler_logger.py

```

from __future__ import annotations
import json
import logging
logger = logging.getLogger("emily.evolution.scheduler_logger")
class SchedulerJobLogger:
    @staticmethod
    def log_sync(
        *,
        job_id: str,
        action_type: str,
        params_json: str = "",
        success: bool = True,
        summary: str = "",
        elapsed_ms: int = 0,
        error_detail: str = "",
        started_at: str = "",
        completed_at: str = "",
    ) -> None:
        from .log_writer import EvolutionLogWriter
        from ...infrastructure.database.models import SchedulerJobLog
        EvolutionLogWriter.write_sync(
            SchedulerJobLog,
            job_id=job_id,
            action_type=action_type,
            params_json=params_json,
            success=success,
            summary=summary,
            elapsed_ms=elapsed_ms,
            error_detail=error_detail,
            started_at=started_at,
            completed_at=completed_at,
        )

```

文件编号：32 文件路径：emily-core\emily_core\infrastructure\logging\session_lifecycle_logger.py

```

from __future__ import annotations
import logging
logger = logging.getLogger("emily.evolution.session_lifecycle_logger")
class SessionLifecycleLogger:
    @staticmethod
    async def log(
        *,
        conversation_id: str,
        user_id: str = "",
        event_type: str = "",
        detail_json: str = "",
        message_count: int = 0,
        duration_ms: int = 0,
    ) -> None:
        from .log_writer import EvolutionLogWriter

```

```
from ...infrastructure.database.models import SessionLifecycleLog
await EvolutionLogWriter.write(
    SessionLifecycleLog,
    conversation_id=conversation_id,
    user_id=user_id,
    event_type=event_type,
    detail_json=detail_json,
    message_count=message_count,
    duration_ms=duration_ms,
)
```

文件编号: 33 文件路径: emily-core\emily_core\node_event_bus.py

```
from __future__ import annotations
import asyncio
import json
import logging
from dataclasses import dataclass, field
from typing import Callable, Awaitable
logger = logging.getLogger("emily.node_event_bus")
@dataclass
class NodeEvent:
    event_id: str
    node_id: str
    event_type: str
    project_id: str = ""
    stage_id: int = 0
    old_value: str = ""
    new_value: str = ""
    operator_id: str = ""
    remark: str = ""
    created_at: str = ""
    def to_outbound(self) -> dict:
        return {
            "event_id": self.event_id,
            "node_id": self.node_id,
            "event_type": self.event_type,
            "project_id": self.project_id,
            "old_value": self.old_value,
            "new_value": self.new_value,
            "operator_id": self.operator_id,
            "created_at": self.created_at,
        }
@dataclass
class SubscriptionFilter:
    node_id: str | None = None
    event_types: list[str] | None = None
    project_id: str | None = None
    stage_id: int | None = None
    owner_dept_id: str | None = None
class NodeEventBus:
    def __init__(self, max_queue: int = 1000):
```

```

        self._subscribers: dict[str, tuple[asyncio.Queue, SubscriptionFilter |
None]] = {}
        self._max_queue = max_queue
        self._outbound_bus = None
        def set_outbound_bus(self, bus) -> None:
            self._outbound_bus = bus
        def subscribe(self, sub_id: str, filter_: SubscriptionFilter | None = None) ->
asyncio.Queue:
            queue: asyncio.Queue = asyncio.Queue(maxsize=self._max_queue)
            self._subscribers[sub_id] = (queue, filter_)
            logger.debug("NodeEventBus: subscriber '%s' added (total=%d)", sub_id,
len(self._subscribers))
            return queue
        def unsubscribe(self, sub_id: str) -> None:
            self._subscribers.pop(sub_id, None)
            logger.debug("NodeEventBus: subscriber '%s' removed (total=%d)",
len(self._subscribers))
        async def publish(self, event: NodeEvent) -> None:
            dropped = 0
            for sub_id, (q, filter_) in list(self._subscribers.items()):
                if filter_ and not self._matches_filter(event, filter_):
                    continue
                try:
                    q.put_nowait(event)
                except asyncio.QueueFull:
                    dropped += 1
                    logger.warning("NodeEventBus: subscriber '%s' queue full, event
dropped", sub_id)
            if dropped:
                logger.warning("NodeEventBus: %d subscriber(s) dropped event",
dropped)
            if self._outbound_bus is not None:
                self._outbound_bus.publish("node_event", event.to_outbound())
        @staticmethod
        def _matches_filter(event: NodeEvent, filter_: SubscriptionFilter) -> bool:
            if filter_.node_id is not None and event.node_id != filter_.node_id:
                return False
            if filter_.event_types is not None and event.event_type not in
filter_.event_types:
                return False
            if filter_.project_id is not None and event.project_id !=
filter_.project_id:
                return False
            if filter_.stage_id is not None and event.stage_id != filter_.stage_id:
                return False
            return True
        @property
        def subscriber_count(self) -> int:
            return len(self._subscribers)

```

文件编号: 34 文件路径: emily-core\emily_core\outbound_bus.py

```

from __future__ import annotations
import asyncio
import logging
logger = logging.getLogger("emily.outbound_bus")
class OutboundEventBus:
    def __init__(self, max_queue: int = 1000):
        self._subscribers: set[asyncio.Queue] = set()
        self._max_queue = max_queue
    def subscribe(self) -> asyncio.Queue:
        queue: asyncio.Queue = asyncio.Queue(maxsize=self._max_queue)
        self._subscribers.add(queue)
        logger.debug("OutboundBus: subscriber added (total=%d)",
len(self._subscribers))
        return queue
    def unsubscribe(self, queue: asyncio.Queue) -> None:
        self._subscribers.discard(queue)
        logger.debug("OutboundBus: subscriber removed (total=%d)",
len(self._subscribers))
    def publish(self, event_type: str, data: dict) -> None:
        event = {"type": event_type, "data": data}
        dropped = 0
        for q in list(self._subscribers):
            try:
                q.put_nowait(event)
            except asyncio.QueueFull:
                dropped += 1
        if dropped:
            logger.warning("OutboundBus: %d subscriber queue(s) full, event
dropped", dropped)
            if not self._subscribers:
                logger.debug("OutboundBus: no subscribers, event '%s' dropped",
event_type)
    @property
    def subscriber_count(self) -> int:
        return len(self._subscribers)

```

文件编号: 35 文件路径: emily-core\emily_core\permission\auth_engine.py

```

from __future__ import annotations
import json
import logging
from dataclasses import dataclass, field
from typing import Optional, TYPE_CHECKING
from .level import can_access, is_admin, LEVEL_NAME
from .code_compiler import compile_code, code_matches_any, can_view_security_level
if TYPE_CHECKING:
    from ..workitem.pipeline.context import BusContext
logger = logging.getLogger("emily.permission.auth_engine")
@dataclass
class AccessCheckResult:

```

```

    allowed: bool
    reason: str = ""
    matched_details: dict = field(default_factory=dict)
    suggested_approver: str = ""
def _build_sop_code(sop_id: str, security_level: str = "INTERNAL",
                    project_id: str = "*", node_id: str = "*") -> str:
    return f"SOP-{security_level}-{project_id}-{node_id}-{sop_id}"
class PermissionAuthEngine:
    def __init__(self, cache=None, audit_repo=None):
        self._cache = cache
        self._audit_repo = audit_repo
    async def check_sop_access(
        self,
        perms: dict,
        sop_id: str,
        context: Optional["BusContext"] = None,
    ) -> AccessCheckResult:
        deny_result = self._check_deny_codes(perms, sop_id)
        if deny_result is not None:
            await self._log_access_denied(perms, sop_id, deny_result.reason)
            return deny_result
        grant_result = self._check_granted_codes(perms, sop_id)
        if grant_result is not None:
            return grant_result
        matrix_result = await self._check_sop_matrix(perms, sop_id)
        if matrix_result is not None:
            if not matrix_result.allowed:
                await self._log_access_denied(perms, sop_id, matrix_result.reason)
                return matrix_result
        return AccessCheckResult(
            allowed=False,
            reason="SOP 未配置权限规则，默认拒绝（请联系管理员配置）",
            matched_details={"source": "default_deny"},
        )
    def _check_deny_codes(self, perms: dict,
                        sop_id: str) -> Optional[AccessCheckResult]:
        denied_codes = perms.get("denied_codes", [])
        if not denied_codes:
            return None
        for security_level in ["PUBLIC", "INTERNAL", "PRIVATE", "CONFIDENTIAL"]:
            sop_code = _build_sop_code(sop_id, security_level)
            if code_matches_any(sop_code, denied_codes):
                return AccessCheckResult(
                    allowed=False,
                    reason=f"权限显式拒绝（SOP {sop_id} 在拒绝列表中）",
                    matched_details={"denied_sop_id": sop_id, "sop_code":
sop_code},
                    suggested_approver=perms.get("supervisor_id", ""),
                )
        return None
    def _check_granted_codes(self, perms: dict,
                            sop_id: str) -> Optional[AccessCheckResult]:
        granted_codes = perms.get("granted_codes", [])
        if not granted_codes:

```

```

        return None
    for security_level in ["PUBLIC", "INTERNAL", "PRIVATE", "CONFIDENTIAL"]:
        sop_code = _build_sop_code(sop_id, security_level)
        if code_matches_any(sop_code, granted_codes):
            return AccessCheckResult(
                allowed=True,
                matched_details={
                    "source": "granted_code",
                    "sop_code": sop_code,
                    "grant_type": "TEMP/PERMANENT",
                },
            )
        return None
    async def _check_sop_matrix(self, perms: dict,
                               sop_id: str) -> Optional[AccessCheckResult]:
        sop_flow = self._get_sop_flow(sop_id)
        if sop_flow is None:
            return None
        perm_level = perms.get("level", 1)
        info_level = perms.get("info_level", "public")
        company_type = perms.get("company_type", "")
        department = perms.get("department", [])
        authorized_node_ids = perms.get("authorized_node_ids", [])
        supervisor_id = perms.get("supervisor_id", "")
        if sop_flow.is_public:
            return AccessCheckResult(
                allowed=True,
                matched_details={"source": "public_sop"},
            )
        if not can_access(perm_level, sop_flow.min_level):
            user_level_name = LEVEL_NAME.get(perm_level, f"L{perm_level}")
            required_name = LEVEL_NAME.get(sop_flow.min_level,
            f"L{sop_flow.min_level}")
            return AccessCheckResult(
                allowed=False,
                reason=f"权限层级不足（当前 {user_level_name}，需
{required_name}）",
                matched_details={
                    "check": "level",
                    "user_level": perm_level,
                    "required_level": sop_flow.min_level,
                },
                suggested_approver=supervisor_id,
            )
        sop_security = sop_flow.security_level or "PUBLIC"
        if not can_view_security_level(perm_level, sop_security):
            return AccessCheckResult(
                allowed=False,
                reason=f"密级不足（SOP 密级 {sop_security}，用户可见
{info_level}）",
                matched_details={
                    "check": "security_level",
                    "sop_level": sop_security,
                    "user_max_level": info_level,

```

```

        },
        suggested_approver=supervisor_id,
    )
    if sop_flow.require_company_match:
        allowed_types = json.loads(sop_flow.allowed_company_types) \
            if sop_flow.allowed_company_types else []
        if allowed_types and company_type not in allowed_types:
            return AccessCheckResult(
                allowed=False,
                reason=f"企业类型不匹配（SOP 要求 {allowed_types}, 用户 {company_type}）",
                matched_details={
                    "check": "company_type",
                    "allowed_types": allowed_types,
                    "user_type": company_type,
                },
                suggested_approver=supervisor_id,
            )
    if sop_flow.require_department_match:
        allowed_depts = json.loads(sop_flow.allowed_departments) \
            if sop_flow.allowed_departments else []
        user_departments = department if isinstance(department, list) else [department]
        if department else []
        if allowed_depts and not (set(user_departments) & set(allowed_depts)):
            return AccessCheckResult(
                allowed=False,
                reason=f"部门不匹配（SOP 要求 {allowed_depts}, 用户 {user_departments}）",
                matched_details={
                    "check": "department",
                    "allowed_depts": allowed_depts,
                    "user_dept": user_departments,
                },
                suggested_approver=supervisor_id,
            )
    required_nodes = json.loads(sop_flow.required_node_ids) \
        if sop_flow.required_node_ids else []
    if required_nodes:
        if not authorized_node_ids and "*" not in required_nodes:
            return AccessCheckResult(
                allowed=False,
                reason="无可访问的全景节点权限",
                matched_details={
                    "check": "node_scope",
                    "required_nodes": required_nodes,
                    "user_nodes": authorized_node_ids,
                },
                suggested_approver=supervisor_id,
            )
    user_set = set(authorized_node_ids)
    req_set = set(required_nodes)
    if "*" not in user_set and not (user_set & req_set):
        return AccessCheckResult(
            allowed=False,

```



```

        reason="节点范围不匹配",
        matched_details={
            "check": "node_scope",
            "required_nodes": required_nodes,
            "user_nodes": authorized_node_ids,
        },
        suggested_approver=supervisor_id,
    )
    return AccessCheckResult(
        allowed=True,
        matched_details={"source": "matrix_check_passed"},
    )
def _get_sop_flow(self, sop_id: str):
    if self._cache is None:
        return None
    matrix = self._cache.get_matrix()
    for flow in matrix.sop_flows:
        if flow.sop_id == sop_id:
            return flow
    return None
async def _log_access_denied(self, perms: dict,
                             sop_id: str, reason: str) -> None:
    audit_user_id = perms.get("user_id", "")
    if self._audit_repo is None:
        logger.info("ACCESS_DENIED user=%s sop=%s reason=%s (no audit repo)",
                    audit_user_id or "?", sop_id, reason)
        return
    try:
        import asyncio
        await asyncio.to_thread(
            self._audit_repo.log_access_denied,
            grantee_id=audit_user_id,
            perm_code=f"SOP-INTERNAL-*--{sop_id}-*",
            reason=reason,
        )
    except Exception as e:
        logger.warning("Failed to write access_denied audit log: %s", e)
async def check_access(
    self,
    perms: dict,
    resource_type: str,
    resource_id: str,
    operation: str = "read",
) -> AccessCheckResult:
    denied_codes = perms.get("denied_codes", [])
    granted_codes = perms.get("granted_codes", [])
    perm_level = perms.get("level", 1)
    supervisor_id = perms.get("supervisor_id", "")
    if denied_codes:
        code = f"{resource_type}-*--*--{resource_id}"
        if code_matches_any(code, denied_codes):
            return AccessCheckResult(
                allowed=False,
                reason=f"权限显式拒绝（资源 {resource_id}）",

```

```

        suggested_approver=supervisor_id,
    )
    if granted_codes:
        code = f"{resource_type}-*-*-*-{resource_id}"
        if code_matches_any(code, granted_codes):
            return AccessCheckResult(allowed=True)
    if is_admin(perm_level):
        return AccessCheckResult(allowed=True)
    return AccessCheckResult(
        allowed=False,
        reason=f"无权访问资源 {resource_type}:{resource_id}",
        suggested_approver=supervisor_id,
    )

```

文件编号: 36 文件路径: emily-core\emily_core\permission\cache.py

```

from __future__ import annotations
import logging
import threading
import time
from dataclasses import dataclass, field
from typing import Optional
from ..infrastructure.database.session import get_session
from ..infrastructure.database.models import (
    PermissionGroup,
    SOPBusinessFlow,
    SOPPermissionBinding,
)
logger = logging.getLogger("emily.permission.cache")
@dataclass
class PermissionMatrix:
    groups: list = field(default_factory=list)
    sop_flows: list = field(default_factory=list)
    bindings: list = field(default_factory=list)
    loaded_at: float = 0.0
    version: int = 0
class PermissionCache:
    def __init__(self, ttl_seconds: int = 300):
        self._ttl = ttl_seconds
        self._matrix: Optional[PermissionMatrix] = None
        self._matrix_lock = threading.RLock()
        self._user_cache: dict[str, tuple[list[str], list[str], int, float]] = {}
        self._user_lock = threading.RLock()
    def get_matrix(self) -> PermissionMatrix:
        with self._matrix_lock:
            if self._matrix is not None:
                age = time.monotonic() - self._matrix.loaded_at
                if age < self._ttl:
                    return self._matrix
            matrix = self._load_matrix_from_db()
            if matrix is not None:
                self._matrix = matrix

```

```

        return self._matrix
    if self._matrix is not None:
        logger.warning("Permission matrix reload failed, using stale
cache")
        return self._matrix
    logger.warning("Permission matrix not available, returning empty
matrix")
    return PermissionMatrix()
def _load_matrix_from_db(self) -> Optional[PermissionMatrix]:
    try:
        with get_session() as session:
            groups = (
                session.query(PermissionGroup)
                .filter(PermissionGroup.status == "active",
                    PermissionGroup.is_deleted == False)
                .all()
            )
            sop_flows = (
                session.query(SOPBusinessFlow)
                .filter(SOPBusinessFlow.is_active == True,
                    SOPBusinessFlow.is_deleted == False)
                .all()
            )
            bindings = (
                session.query(SOPPermissionBinding)
                .filter(SOPPermissionBinding.is_deleted == False)
                .all()
            )
            old_version = self._matrix.version if self._matrix else 0
            new_version = old_version + 1
            matrix = PermissionMatrix(
                groups=list(groups),
                sop_flows=list(sop_flows),
                bindings=list(bindings),
                loaded_at=time.monotonic(),
                version=new_version,
            )
            logger.info(
                "Permission matrix loaded: %d groups, %d flows, %d bindings,
version=%d",
                len(groups), len(sop_flows), len(bindings), new_version,
            )
            return matrix
    except Exception as e:
        logger.error("Failed to load permission matrix from DB: %s", e)
        return None
def invalidate(self) -> None:
    with self._matrix_lock:
        self._matrix = None
    with self._user_lock:
        self._user_cache.clear()
    logger.info("Permission cache invalidated (L1 + L2)")
def get_version(self) -> int:
    with self._matrix_lock:

```

```

        if self._matrix is None:
            return 0
        return self._matrix.version
def get_user_whitelist(
    self, user_id: str, user_level: int,
    company_type: str, department: str,
) -> tuple[list[str], list[str]]:
    matrix = self.get_matrix()
    current_version = matrix.version
    with self._user_lock:
        cached = self._user_cache.get(user_id)
        if cached is not None:
            cached_allow, cached_deny, cached_version, _loaded_at = cached
            if cached_version == current_version:
                return cached_allow, cached_deny
    sop_allow, denied_sop_ids = self._compute_user_whitelist(
        matrix, user_level, company_type, department,
    )
    with self._user_lock:
        self._user_cache[user_id] = (
            sop_allow, denied_sop_ids, current_version, time.monotonic(),
        )
    return sop_allow, denied_sop_ids
@staticmethod
def _compute_user_whitelist(
    matrix: PermissionMatrix,
    user_level: int,
    company_type: str,
    department: str | list[str],
) -> tuple[list[str], list[str]]:
    from .level import can_access
    matched_group_ids: set[str] = set()
    for g in matrix.groups:
        if g.company_type and g.company_type != company_type:
            continue
        user_depts = department if isinstance(department, list) else
[department] if department else []
        if g.department and g.department not in user_depts:
            continue
        matched_group_ids.add(g.id)
    sop_allow: list[str] = []
    denied_sop_ids: list[str] = []
    for flow in matrix.sop_flows:
        flow_bindings = [b for b in matrix.bindings
            if b.sop_business_flow_id == flow.id]
        if any(b.binding_type == "deny" and b.permission_group_id in
matched_group_ids
            for b in flow_bindings):
            denied_sop_ids.append(flow.sop_id)
            continue
        if flow.is_public:
            sop_allow.append(flow.sop_id)
            continue
        if not can_access(user_level, flow.min_level):

```

```

        continue
    if flow.require_company_match:
        import json
        allowed_types = json.loads(flow.allowed_company_types) if
flow.allowed_company_types else []
        if allowed_types and company_type not in allowed_types:
            continue
    if flow.require_department_match:
        import json
        allowed_depts = json.loads(flow.allowed_departments) if
flow.allowed_departments else []
        if allowed_depts and department not in allowed_depts:
            continue
    sop_allow.append(flow.sop_id)
    return sop_allow, denied_sop_ids
def invalidate_user(self, user_id: str) -> None:
    with self._user_lock:
        self._user_cache.pop(user_id, None)
def stats(self) -> dict:
    with self._matrix_lock:
        matrix_info = {
            "loaded": self._matrix is not None,
            "version": self._matrix.version if self._matrix else 0,
            "age_seconds": (
                round(time.monotonic() - self._matrix.loaded_at, 1)
                if self._matrix else 0
            ),
            "ttl_seconds": self._ttl,
        }
    with self._user_lock:
        user_info = {
            "cached_users": len(self._user_cache),
        }
    return {"l1_matrix": matrix_info, "l2_users": user_info}

```

文件编号: 37 文件路径: emily-core\emily_core\permission\code_compiler.py

```

from __future__ import annotations
import re
from dataclasses import dataclass
from typing import Iterable
RESOURCE_TYPES = frozenset({"DOC", "DB", "SOP", "MSG", "SYS"})
SECURITY_LEVELS = ["PUBLIC", "INTERNAL", "PRIVATE", "CONFIDENTIAL"]
SECURITY_LEVEL_INDEX = {lvl: i for i, lvl in enumerate(SECURITY_LEVELS)}
SECURITY_LEVEL_NAME = {
    "PUBLIC": "公开",
    "INTERNAL": "内部",
    "PRIVATE": "私有",
    "CONFIDENTIAL": "机密",
}
LEVEL_MAX_SECURITY_INDEX: dict[int, int] = {
    1: 0,
}

```

```

2: 1,
3: 1,
4: 1,
5: 3,
6: 3,
}
@dataclass(frozen=True)
class CompiledCode:
    raw: str
    resource_type: str
    security_level: str
    project_id: str
    node_id: str
    resource_id: str
    def matches(self, target: "CompiledCode") -> bool:
        if self.resource_type != target.resource_type:
            return False
        if self.security_level != target.security_level:
            return False
        return (
            _segment_match(self.project_id, target.project_id)
            and _segment_match(self.node_id, target.node_id)
            and _segment_match(self.resource_id, target.resource_id)
        )
    def _segment_match(pattern: str, value: str) -> bool:
        if pattern == "*":
            return True
        if pattern.endswith("*"):
            return value.startswith(pattern[:-1])
        return pattern == value
_CODE_RE = re.compile(r"^([A-Z]+)-([A-Z]+)-([^-]*)-([^-]*)-(.+)$")
def compile_code(code: str) -> CompiledCode | None:
    if not code:
        return None
    m = _CODE_RE.match(code.strip())
    if m is None:
        return None
    rtype, slevel, pid, nid, rid = m.groups()
    if rtype not in RESOURCE_TYPES:
        return None
    if slevel not in SECURITY_LEVEL_INDEX:
        return None
    return CompiledCode(
        raw=code.strip(),
        resource_type=rtype,
        security_level=slevel,
        project_id=pid or "*",
        node_id=nid or "*",
        resource_id=rid or "*"
    )
def code_matches_any(code: str, patterns: Iterable[str]) -> bool:
    target = compile_code(code)
    if target is None:
        return False

```

```

    for p in patterns:
        pat = compile_code(p)
        if pat is not None and pat.matches(target):
            return True
    return False
def can_view_security_level(user_level: int, security_level: str) -> bool:
    idx = SECURITY_LEVEL_INDEX.get(security_level)
    if idx is None:
        return False
    max_idx = LEVEL_MAX_SECURITY_INDEX.get(user_level, 0)
    return idx <= max_idx

```

文件编号: 38 文件路径: emily-core\emily_core\permission\level.py

```

from __future__ import annotations
from enum import Enum
class PermissionLevel(Enum):
    GUEST = 1
    PARTICIPANT_EXEC = 2
    PARTICIPANT_MGR = 3
    OWNER_SUPERVISOR = 4
    ADMIN = 5
    SYS_ADMIN = 6
    INHERITANCE_CHAIN: dict[int, frozenset[int]] = {
        1: frozenset({1}),
        2: frozenset({2, 1}),
        3: frozenset({3, 2, 1}),
        4: frozenset({4, 1}),
        5: frozenset({5, 4, 1}),
        6: frozenset({6, 5, 4, 1}),
    }
    LEVEL_NAME: dict[int, str] = {
        1: "访客",
        2: "参建执行",
        3: "参建管理",
        4: "建设主管",
        5: "管理员",
        6: "系统管理员",
    }
def effective_levels(user_level: int) -> frozenset[int]:
    return INHERITANCE_CHAIN.get(user_level,
    INHERITANCE_CHAIN[PermissionLevel.GUEST.value])
def can_access(user_level: int, required_level: int) -> bool:
    return required_level in effective_levels(user_level)
def is_admin(user_level: int) -> bool:
    return user_level >= PermissionLevel.ADMIN.value
def is_sys_admin(user_level: int) -> bool:
    return user_level >= PermissionLevel.SYS_ADMIN.value
def level_label(level: int) -> str:
    return f"{LEVEL_NAME.get(level, '未知')}(L{level})"

```

文件编号: 39 文件路径: emily-core\emily_core\permission\row_security.py

```
from __future__ import annotations
import logging
import threading
from typing import Optional
from sqlalchemy import event
from sqlalchemy.orm import Session
from sqlalchemy.sql import select
logger = logging.getLogger("emily.permission.row_security")
_FILTERABLE_TABLES: dict[str, str] = {
    "events": "company_id",
    "tasks": "company_id",
    "files": "company_id",
    "messages": "company_id",
}
_local = threading.local()
def set_current_permission_snapshot(snapshot) -> None:
    _local.permission_snapshot = snapshot
def get_current_permission_snapshot():
    return getattr(_local, "permission_snapshot", None)
def skip_auth_injection() -> None:
    _local.skip_auth = True
def restore_auth_injection() -> None:
    _local.skip_auth = False
def _is_skip() -> bool:
    return getattr(_local, "skip_auth", False)
_auth_listener_registered = False
def register_row_security_listener() -> None:
    global _auth_listener_registered
    if _auth_listener_registered:
        return
    _auth_listener_registered = True
    @event.listens_for(Session, "do_orm_execute")
    def _inject_company_filter(orm_execute_state):
        if _is_skip():
            return
        if not orm_execute_state.is_select:
            return
        perms = get_current_permission_snapshot()
        if perms is None:
            return
        allowed_ids = _build_allowed_company_ids(perms)
        if not allowed_ids:
            return
        try:
            _try_inject_orm_filter(orm_execute_state, allowed_ids)
        except Exception as e:
            logger.warning(
                "Row security injection failed, fail-open: %s",
                e,
            )
def _build_allowed_company_ids(perms) -> list[str]:
```



```

ids: list[str] = []
if perms.company_id:
    ids.append(perms.company_id)
if perms.partner_ids:
    ids.extend(pid for pid in perms.partner_ids if pid and pid not in ids)
from .level import is_admin
if is_admin(perms.level):
    return []
if getattr(perms, 'is_management_unit', False) and getattr(perms,
'project_ids', []):
    return _load_project_member_company_ids(perms.project_ids)
return ids
def _load_project_member_company_ids(project_ids: list[str]) -> list[str]:
    try:
        from ..infrastructure.database import get_session
        from ..infrastructure.database.models import User
        with get_session() as session:
            companies = session.query(User.company).filter(
                User.project_id.in_(project_ids),
                User.company != None,
                User.is_deleted == False,
            ).distinct().all()
            return [c[0] for c in companies if c[0]]
    except Exception as e:
        logger.warning("_load_project_member_company_ids failed: %s", e)
        return []
def _try_inject_orm_filter(orm_execute_state, allowed_ids: list[str]):
    from sqlalchemy import and_
    for entity in orm_execute_state.select_statement.column_slices:
        table = _get_entity_table(entity)
        if table is None:
            continue
        table_name = table.name if hasattr(table, 'name') else str(table)
        if table_name not in _FILTERABLE_TABLES:
            continue
        company_col_name = _FILTERABLE_TABLES[table_name]
        company_col = _find_table_column(table, company_col_name)
        if company_col is None:
            continue
        condition = company_col.in_(allowed_ids)
        existing_where = orm_execute_state.select_statement.whereclause
        if existing_where is not None:
            orm_execute_state.select_statement =
orm_execute_state.select_statement.where(
                and_(existing_where, condition)
            )
        else:
            orm_execute_state.select_statement =
orm_execute_state.select_statement.where(condition)
def _get_entity_table(entity):
    if hasattr(entity, 'mapper'):
        mapper = entity.mapper
        if hasattr(mapper, 'local_table'):
            return mapper.local_table

```

```

    if hasattr(entity, '__clause_element__'):
        el = entity.__clause_element__()
        if hasattr(el, 'table'):
            return el.table
    if hasattr(entity, 'table'):
        return entity.table
    return None

def _find_table_column(table, col_name: str) -> Optional:
    if hasattr(table, 'columns') and hasattr(table.columns, col_name):
        return table.columns[col_name]
    if hasattr(table, 'c') and hasattr(table.c, col_name):
        return table.c[col_name]
    return None

class PermissionAuditLogRepository:
    @staticmethod
    def log_access_denied(grantee_id: str, perm_code: str, *,
                        reason: str = "", session=None) -> None:
        from ..infrastructure.database.models import PermissionAuditLog, _utc_now
        from ..infrastructure.database.session import get_session as _get_session
        from typing import Optional as _Opt
        from sqlalchemy.orm import Session as _Session
        def _impl(sess: _Session):
            log = PermissionAuditLog(
                event_time=_utc_now(),
                grantor_id="",
                grantee_id=grantee_id,
                perm_code=perm_code,
                grant_type="",
                operation_type="ACCESS_DENIED",
                remark=reason,
            )
            sess.add(log)
            sess.flush()
        if session is not None:
            _impl(session)
        else:
            with _get_session() as sess:
                _impl(sess)

    @staticmethod
    def log_grant(grantor_id: str, grantee_id: str, perm_code: str,
                grant_type: str, *, duration: int = None,
                session_id: str = "", remark: str = "", session=None) -> None:
        from ..infrastructure.database.models import PermissionAuditLog, _utc_now
        from ..infrastructure.database.session import get_session as _get_session
        from sqlalchemy.orm import Session as _Session
        def _impl(sess: _Session):
            log = PermissionAuditLog(
                event_time=_utc_now(),
                grantor_id=grantor_id,
                grantee_id=grantee_id,
                perm_code=perm_code,
                grant_type=grant_type,
                duration=duration,
                session_id=session_id,

```

```

        operation_type="GRANT",
        remark=remark,
    )
    sess.add(log)
    sess.flush()
    if session is not None:
        _impl(session)
    else:
        with _get_session() as sess:
            _impl(sess)
    @staticmethod
    def log_revoke(grantor_id: str, grantee_id: str, perm_code: str, *,
                  remark: str = "", session=None) -> None:
        from ..infrastructure.database.models import PermissionAuditLog, _utc_now
        from ..infrastructure.database.session import get_session as _get_session
        from sqlalchemy.orm import Session as _Session
        def _impl(sess: _Session):
            log = PermissionAuditLog(
                event_time=_utc_now(),
                grantor_id=grantor_id,
                grantee_id=grantee_id,
                perm_code=perm_code,
                operation_type="REVOKE",
                remark=remark,
            )
            sess.add(log)
            sess.flush()
        if session is not None:
            _impl(session)
        else:
            with _get_session() as sess:
                _impl(sess)
    @staticmethod
    def query_logs(grantee_id: str = "", operation_type: str = "",
                  limit: int = 100, *, session=None) -> list:
        from ..infrastructure.database.models import PermissionAuditLog
        from ..infrastructure.database.session import get_session as _get_session
        from sqlalchemy.orm import Session as _Session
        def _impl(sess: _Session):
            q = sess.query(PermissionAuditLog)
            if grantee_id:
                q = q.filter(PermissionAuditLog.grantee_id == grantee_id)
            if operation_type:
                q = q.filter(PermissionAuditLog.operation_type == operation_type)
            q = q.order_by(PermissionAuditLog.log_id.desc())
            return q.limit(limit).all()
        if session is not None:
            return _impl(session)
        with _get_session() as sess:
            return _impl(sess)

```

```
from __future__ import annotations
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from datetime import datetime
from typing import Optional
@dataclass
class EmailCredentials:
    smtp_host: str
    smtp_port: int
    imap_host: str
    imap_port: int
    username: str
    password: str
    use_ssl: bool = True
@dataclass
class EmailAttachment:
    filename: str
    content: bytes
    content_type: str
@dataclass
class EmailEnvelope:
    uid: str
    message_id: str
    sender: str
    recipient: str
    subject: str
    body_plain: str
    body_html: Optional[str] = None
    date: Optional[datetime] = None
    attachments: list[EmailAttachment] = field(default_factory=list)
    is_order: bool = False
@dataclass
class SendResult:
    success: bool
    message_id: Optional[str] = None
    error: Optional[str] = None
class EmailProvider(ABC):
    @abstractmethod
    async def send(
        self,
        creds: EmailCredentials,
        to: str,
        subject: str,
        body: str,
        html: bool = False,
        attachments: Optional[list[EmailAttachment]] = None,
    ) -> SendResult:
        ...
    @abstractmethod
    async def fetch_inbox(
        self,
        creds: EmailCredentials,
```

```

        subject_filter: Optional[str] = None,
        from_self: bool = False,
        unread_only: bool = True,
        limit: int = 20,
    ) -> list[EmailEnvelope]:
        ...
    @abstractmethod
    async def fetch_orders(
        self,
        creds: EmailCredentials,
        since_uid: Optional[str] = None,
    ) -> list[EmailEnvelope]:
        ...
    @abstractmethod
    async def mark_read(
        self,
        creds: EmailCredentials,
        uids: list[str],
    ) -> bool:
        ...
    @abstractmethod
    async def is_available(self) -> bool:
        ...

```

文件编号: 41 文件路径: emily-core\emily_core\providers\email\imap_provider.py

```

from __future__ import annotations
import logging
import re
from email import message_from_bytes, policy
from email.utils import parsedate_to_datetime
from typing import Optional
import aioimaplib
from .base import (
    EmailAttachment,
    EmailCredentials,
    EmailEnvelope,
    EmailProvider,
)
logger = logging.getLogger("emily.email.imap")
_UID_RE = re.compile(r"\d+")
class IMAPEmailProvider(EmailProvider):
    def __init__(self):
        self._available: Optional[bool] = None
    async def is_available(self) -> bool:
        if self._available is not None:
            return self._available
        try:
            import aioimaplib
            self._available = True
        except ImportError:
            self._available = False

```

```

        return self._available
    async def fetch_inbox(
        self,
        creds: EmailCredentials,
        subject_filter: Optional[str] = None,
        from_self: bool = False,
        unread_only: bool = True,
        limit: int = 20,
    ) -> list[EmailEnvelope]:
        if not creds.username or not creds.password:
            logger.warning("IMAP fetch_inbox: 凭证不完整")
            return []
        masked_user = _mask_email(creds.username)
        try:
            imap = aioimaplib.IMAP4_SSL(
                host=creds.imap_host,
                port=creds.imap_port,
                timeout=15,
            )
            await imap.wait_hello_from_server()
            await imap.login(creds.username, creds.password)
            logger.debug("IMAP login ok: user=%s", masked_user)
            await imap.select("INBOX")
            criteria = []
            if unread_only:
                criteria.append("UNSEEN")
            if subject_filter:
                criteria.append(f'SUBJECT "{subject_filter}"')
            if from_self:
                criteria.append(f"FROM {creds.username}")
                criteria.append(f"TO {creds.username}")
            search_cmd = " ".join(criteria) if criteria else "ALL"
            logger.debug("IMAP SEARCH: %s", search_cmd)
            if criteria:
                search_resp = await imap.search(None, *criteria)
            else:
                search_resp = await imap.search(None, "ALL")
            if search_resp.result != "OK":
                logger.warning("IMAP SEARCH failed: result=%s",
search_resp.result)
                await imap.logout()
                return []
            seq_nums = _parse_uids(search_resp.lines)
            if not seq_nums:
                await imap.logout()
                return []
            seq_nums = seq_nums[-limit:] if len(seq_nums) > limit else seq_nums
            seq_set = ",".join(seq_nums)
            uid_resp = await imap.fetch(seq_set, "(UID)")
            uid_list = _extract_uids_from_fetch(uid_resp.lines)
            if not uid_list:
                await imap.logout()
                return []
            uid_set = ",".join(uid_list)

```

```

        fetch_resp = await imap.uid("FETCH", uid_set, "(RFC822)")
        if fetch_resp.result != "OK":
            logger.warning("IMAP FETCH failed: result=%s", fetch_resp.result)
            await imap.logout()
            return []
        envelopes = _parse_fetch_response(fetch_resp.lines, subject_filter,
uid_list)
        await imap.logout()
        logger.info("IMAP fetch_inbox: %d envelopes, user=%s", len(envelopes),
masked_user)
        return envelopes
    except aioimaplib.Abort as e:
        logger.error("IMAP abort: user=%s error=%s", masked_user, e)
        return []
    except (OSError, TimeoutError) as e:
        logger.warning("IMAP connection failed: host=%s:%d error=%s",
            creds.imap_host, creds.imap_port, e)
        return []
    except Exception as e:
        logger.error("IMAP fetch_inbox unknown error: user=%s error=%s",
            masked_user, e, exc_info=True)
        return []
    async def fetch_orders(
        self,
        creds: EmailCredentials,
        since_uid: Optional[str] = None,
    ) -> list[EmailEnvelope]:
        envelopes = await self.fetch_inbox(
            creds=creds,
            subject_filter="order",
            from_self=True,
            unread_only=True,
            limit=50,
        )
        for env in envelopes:
            env.is_order = True
        if since_uid and envelopes:
            try:
                since_uid_int = int(since_uid)
                envelopes = [e for e in envelopes if int(e.uid) > since_uid_int]
            except (ValueError, TypeError):
                pass
        return envelopes
    async def mark_read(
        self,
        creds: EmailCredentials,
        uids: list[str],
    ) -> bool:
        if not uids:
            return True
        if not creds.username or not creds.password:
            return False
        masked_user = _mask_email(creds.username)
        try:

```

```

        imap = aioimaplib.IMAP4_SSL(
            host=creds.imap_host,
            port=creds.imap_port,
            timeout=15,
        )
        await imap.wait_hello_from_server()
        await imap.login(creds.username, creds.password)
        await imap.select("INBOX")
        uid_set = ",".join(uids)
        result, _ = await imap.uid("STORE", uid_set, "+FLAGS", "\\Seen")
        await imap.logout()
        ok = result == "OK"
        logger.debug("IMAP mark_read: %d uids, ok=%s, user=%s", len(uids), ok,
masked_user)
        return ok
    except Exception as e:
        logger.error("IMAP mark_read failed: uids=%s error=%s", uids, e,
exc_info=True)
        return False
    async def send(self, *args, **kwargs):
        raise NotImplementedError("IMAPEmailProvider 不支持发送, 请使用
SMTPEmailProvider")
def _parse_uids(data) -> list[str]:
    if not data or not isinstance(data, list):
        return []
    for item in data:
        if isinstance(item, (bytes, bytearray)):
            raw = item.decode("utf-8", errors="replace")
            nums = _UID_RE.findall(raw)
            if nums:
                return nums
        elif isinstance(item, str):
            nums = _UID_RE.findall(item)
            if nums:
                return nums
    return []
def _extract_uids_from_fetch(fetch_lines) -> list[str]:
    uids = []
    for line in fetch_lines:
        if isinstance(line, (bytes, bytearray)):
            text = line.decode("utf-8", errors="replace")
        elif isinstance(line, tuple):
            for item in line:
                if isinstance(item, (bytes, bytearray)):
                    text = item.decode("utf-8", errors="replace")
                    break
            else:
                continue
        else:
            text = str(line)
        import re
        match = re.search(r"UID\s+(\d+)", text, re.IGNORECASE)
        if match:
            uids.append(match.group(1))

```



```

    return uids if uids else _parse_uids(fetch_lines)
def _parse_fetch_response(fetch_lines: list, subject_filter: Optional[str] = None,
uid_list: list = None) -> list[EmailEnvelope]:
    envelopes = []
    filter_lower = subject_filter.lower() if subject_filter else None
    emails_raw = _split_fetch_to_emails(fetch_lines)
    for i, raw_email in enumerate(emails_raw):
        try:
            msg = message_from_bytes(raw_email, policy=policy.default)
            env = _envelope_from_message(msg, filter_lower)
            if env is not None:
                if uid_list and i < len(uid_list):
                    env.uid = uid_list[i]
                envelopes.append(env)
        except Exception as e:
            logger.warning("Failed to parse email: %s", e)
    return envelopes
def _split_fetch_to_emails(fetch_data: list) -> list[bytes]:
    emails = []
    current = b""
    for item in fetch_data:
        chunk = b""
        if isinstance(item, (bytes, bytearray)):
            chunk = bytes(item)
        elif isinstance(item, tuple):
            for sub in reversed(item):
                if isinstance(sub, (bytes, bytearray)):
                    chunk = bytes(sub)
                    break
        if not chunk:
            continue
        stripped = chunk.strip()
        if b"FETCH" in stripped and b"RFC822" in stripped:
            if current:
                emails.append(current)
            current = b""
            continue
        current += chunk
    if current:
        emails.append(current)
    return emails
def _envelope_from_message(msg, filter_lower: Optional[str] = None) ->
Optional[EmailEnvelope]:
    subject = msg.get("Subject", "") or ""
    if filter_lower and filter_lower not in subject.lower():
        return None
    uid = msg.get("UID", "") or msg.get("X-UID", "") or ""
    body_plain = ""
    body_html = None
    if msg.is_multipart():
        for part in msg.walk():
            content_type = part.get_content_type()
            disposition = str(part.get("Content-Disposition", ""))
            if "attachment" in disposition:

```

```

        continue
    if content_type == "text/plain":
        payload = _decode_payload(part)
        if payload:
            body_plain += payload
    elif content_type == "text/html" and body_html is None:
        body_html = _decode_payload(part)
else:
    payload = _decode_payload(msg)
    content_type = msg.get_content_type()
    if content_type == "text/html":
        body_html = payload
    else:
        body_plain = payload or ""
attachments = []
for part in msg.walk():
    disposition = str(part.get("Content-Disposition", ""))
    if "attachment" in disposition:
        filename = part.get_filename() or "attachment"
        content = part.get_payload(decode=True)
        content_type = part.get_content_type()
        if content:
            attachments.append(EmailAttachment(
                filename=filename,
                content=content,
                content_type=content_type,
            ))
date = None
date_str = msg.get("Date", "")
if date_str:
    try:
        date = parsedate_to_datetime(date_str)
    except Exception:
        pass
return EmailEnvelope(
    uid=uid,
    message_id=msg.get("Message-ID", "") or "",
    sender=msg.get("From", "") or "",
    recipient=msg.get("To", "") or "",
    subject=subject,
    body_plain=body_plain.strip(),
    body_html=body_html,
    date=date,
    attachments=attachments,
    is_order=False,
)
def _decode_payload(part) -> str:
    try:
        payload = part.get_payload(decode=True)
        if payload is None:
            return ""
        charset = part.get_content_charset() or "utf-8"
        return payload.decode(charset, errors="replace")
    except Exception:

```

```
        return ""
def _mask_email(email: str) -> str:
    if "@" not in email:
        return "***"
    user, domain = email.split("@", 1)
    masked_user = user[:2] + "*****" if len(user) > 2 else "*****"
    return f"{masked_user}@{domain}"
```

文件编号: 42 文件路径: emily-core\emily_core\providers\email\smtp_provider.py

```
from __future__ import annotations
import logging
import uuid
from email.mime.base import MIMEBase
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from email import encoders as email_encoders
from email.utils import formatdate
from typing import Optional
import aiosmtplib
from aiosmtplib.errors import (
    SMTPAuthenticationError,
    SMTPConnectError,
    SMTPResponseException,
    SMTPTimeoutError,
)
from .base import (
    EmailAttachment,
    EmailCredentials,
    EmailProvider,
    SendResult,
)
logger = logging.getLogger("emily.email.smtp")
class SMTPEmailProvider(EmailProvider):
    def __init__(self):
        self._available: Optional[bool] = None
    async def is_available(self) -> bool:
        if self._available is not None:
            return self._available
        try:
            import aiosmtplib
            self._available = True
        except ImportError:
            self._available = False
        return self._available
    async def send(
        self,
        creds: EmailCredentials,
        to: str,
        subject: str,
        body: str,
        html: bool = False,
```

```

        attachments: Optional[list[EmailAttachment]] = None,
    ) -> SendResult:
        if not creds.username or not creds.password:
            return SendResult(success=False, error="AUTH_FAILED: 凭证不完整")
        if not to or "@" not in to:
            return SendResult(success=False, error=f"INVALID_RECIPIENT: {to}")
        try:
            msg = self._build_mime(
                from_addr=creds.username,
                to=to,
                subject=subject,
                body=body,
                html=html,
                attachments=attachments,
            )
        except Exception as e:
            logger.error("MIME construction failed: %s", e, exc_info=True)
            return SendResult(success=False, error=f"MIME_ERROR: {e}")
        masked_user = _mask_email(creds.username)
        try:
            smtp = aiosmtplib.SMTP(
                hostname=creds.smtp_host,
                port=creds.smtp_port,
                use_tls=creds.use_ssl,
                timeout=15,
            )
            await smtp.connect()
            logger.debug("SMTP connected: host=%s port=%d user=%s",
                          creds.smtp_host, creds.smtp_port, masked_user)
            await smtp.login(creds.username, creds.password)
            logger.debug("SMTP login ok: user=%s", masked_user)
            errors, response_msg = await smtp.send_message(msg)
            await smtp.quit()
            if errors:
                error_details = "; ".join(f"{addr}: {resp}" for addr, resp in
errors.items())
                logger.warning("SMTP send partial errors: %s", error_details)
            logger.info("Email sent: from=%s to=%s subject=%.80r",
                          masked_user, _mask_email(to), subject)
            msg_id = msg.get("Message-ID", "") if hasattr(msg, "get") else ""
            return SendResult(
                success=True,
                message_id=msg_id,
            )
        except SMTPAuthenticationError as e:
            logger.warning("SMTP auth failed: user=%s error=%s", masked_user, e)
            return SendResult(success=False, error="AUTH_FAILED")
        except (SMTPConnectError, SMTPTimeoutError, OSError) as e:
            logger.warning("SMTP connection failed: host=%s:%d error=%s",
                          creds.smtp_host, creds.smtp_port, e)
            return SendResult(success=False, error="CONNECTION_TIMEOUT")
        except SMTPResponseException as e:
            if e.code == 550 or e.code == 554:
                logger.warning("SMTP rejected: code=%d msg=%s", e.code, e.message)

```

```

        return SendResult(success=False, error=f"RATE_LIMITED:
{e.message}")
        logger.warning("SMTP response error: code=%d msg=%s", e.code,
e.message)
        return SendResult(success=False, error=f"SMTP_ERROR_{e.code}:
{e.message}")
    except Exception as e:
        logger.error("SMTP send unknown error: %s", e, exc_info=True)
        return SendResult(success=False, error=str(e))
    async def fetch_inbox(self, *args, **kwargs):
        raise NotImplementedError("SMTPEmailProvider 不支持收件, 请使用
IMAPEmailProvider")
    async def fetch_orders(self, *args, **kwargs):
        raise NotImplementedError("SMTPEmailProvider 不支持收件, 请使用
IMAPEmailProvider")
    async def mark_read(self, *args, **kwargs):
        raise NotImplementedError("SMTPEmailProvider 不支持标记已读, 请使用
IMAPEmailProvider")
    def _build_mime(
        self,
        from_addr: str,
        to: str,
        subject: str,
        body: str,
        html: bool = False,
        attachments: Optional[list[EmailAttachment]] = None,
    ) -> MIMEMultipart:
        msg = MIMEMultipart()
        msg["From"] = from_addr
        msg["To"] = to
        msg["Subject"] = subject
        msg["Date"] = formatdate(localtime=True)
        msg["Message-ID"] = f"<{uuid.uuid4()}@emily>"
        subtype = "html" if html else "plain"
        msg.attach(MIMEText(body, subtype, "utf-8"))
        if attachments:
            for att in attachments:
                part = MIMEBase(*att.content_type.split("/", 1),
name=att.filename)
                part.set_payload(att.content)
                email_encoders.encode_base64(part)
                part.add_header(
                    "Content-Disposition",
                    "attachment",
                    filename=("utf-8", "", att.filename),
                )
                msg.attach(part)
        return msg
    def _mask_email(email: str) -> str:
        if "@" not in email:
            return "***"
        user, domain = email.split("@", 1)

```

```
masked_user = user[:2] + "****" if len(user) > 2 else "****"
return f"{masked_user}@{domain}"
```

文件编号: 43 文件路径: emily-core\emily_core\providers\rag\base.py

```
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
@dataclass
class SearchResult:
    content: str
    score: float = 0.0
    source_document: str = ""
    source_kb: str = ""
    metadata: dict = field(default_factory=dict)
@dataclass
class RagSearchResponse:
    query: str
    results: list[SearchResult]
    context_text: str
    total: int
    provider_name: str
class RagProvider(ABC):
    @abstractmethod
    async def search(
        self, query: str, top_k: int = 5,
        stage: str | None = None,
        role: str | None = None,
    ) -> RagSearchResponse:
    @abstractmethod
    async def is_available(self) -> bool:
```

文件编号: 44 文件路径: emily-core\emily_core\providers\rag\local_fallback.py

```
import logging
import os
import re
from pathlib import Path
from typing import Optional
from .base import RagProvider, SearchResult, RagSearchResponse
logger = logging.getLogger("emily.rag.local")
_DEFAULT_SEARCH_DIR = "项目资料"
class LocalFileRagProvider(RagProvider):
    def __init__(self, search_dir: str | None = None):
        project_root = Path(__file__).resolve().parents[7]
        self._search_dir = Path(search_dir) if search_dir else project_root /
_DEFAULT_SEARCH_DIR
        self._chunks: list[dict] = []
        self._loaded: bool = False
    def _load(self) -> None:
```

```

if self._loaded:
    return
if not self._search_dir.exists():
    logger.info("LocalFileRag: 目录不存在 %s", self._search_dir)
    self._loaded = True
    return
for file_path in self._search_dir.rglob("*"):
    if file_path.suffix.lower() not in (".md", ".txt", ".markdown"):
        continue
    try:
        content = file_path.read_text(encoding="utf-8")
    except Exception:
        continue
    source_name = str(file_path.relative_to(self._search_dir))
    sections = _split_by_headings(content)
    for title, body in sections:
        if body.strip():
            metadata = _extract_metadata(source_name, title, body)
            self._chunks.append({
                "title": title or file_path.name,
                "body": body.strip(),
                "source": source_name,
                **metadata,
            })
self._loaded = True
logger.info(
    "LocalFileRag: 已加载 %d 个文本块 (目录: %s)",
    len(self._chunks), self._search_dir,
)
async def is_available(self) -> bool:
    self._load()
    return len(self._chunks) > 0
async def search(
    self, query: str, top_k: int = 5,
    stage: str | None = None,
    role: str | None = None,
) -> RagSearchResponse:
    self._load()
    if not self._chunks:
        return RagSearchResponse(
            query=query, results=[], context_text="",
            total=0, provider_name="LocalFile (empty)",
        )
    tokens = _tokenize(query)
    if not tokens:
        return RagSearchResponse(
            query=query, results=[], context_text="",
            total=0, provider_name="LocalFile (empty query)",
        )
    scored = []
    for chunk in self._chunks:
        if stage and not _match_stage(chunk, stage):
            continue
        if role and not _match_role(chunk, role):

```

```

        continue
    body_lower = chunk["body"].lower()
    score = sum(body_lower.count(t.lower()) for t in tokens)
    if score > 0:
        scored.append((score / max(len(body_lower), 1), chunk))
    scored.sort(key=lambda x: x[0], reverse=True)
    results = []
    for score, chunk in scored[:top_k]:
        results.append(SearchResult(
            content=chunk["body"][:1000],
            score=round(score * 100, 2),
            source_document=chunk["source"],
            metadata={
                "title": chunk.get("title", ""),
                "stage": chunk.get("stage", ""),
                "chunk_id": chunk.get("chunk_id", ""),
                "keywords": chunk.get("keywords", []),
            },
        ))
    parts = []
    for r in results:
        parts.append(f"
context_text = "\n\n---\n\n".join(parts)
return RagSearchResponse(
    query=query,
    results=results,
    context_text=context_text,
    total=len(results),
    provider_name="LocalFile",
)

def _split_by_headings(content: str) -> list[tuple[str, str]]:
    sections: list[tuple[str, str]] = []
    heading_pattern = re.compile(r"^(
matches = list(heading_pattern.finditer(content))
if not matches:
    sections.append(("", content))
    return sections
for i, m in enumerate(matches):
    start = m.end()
    end = matches[i + 1].start() if i + 1 < len(matches) else len(content)
    sections.append((m.group(2).strip(), content[start:end]))
if matches and matches[0].start() > 0:
    sections.insert(0, ("", content[:matches[0].start()]))
return sections

def _tokenize(text: str) -> list[str]:
    tokens: list[str] = []
    chinese = re.findall(r"[-\u4e00-\u9fff]", text)
    tokens.extend(chinese)
    words = re.findall(r"[a-zA-Z0-9]{2,}", text)
    tokens.extend(words)
    return tokens

_STAGE_KEYWORDS: dict[str, str] = {
    "投资决策": "投资决策",
    "投资": "投资决策",

```



```

"专项审查": "专项审查",
"专项": "专项审查",
"规划报建": "规划报建",
"报建": "规划报建",
"规划": "规划报建",
"招标采购": "招标采购",
"招标": "招标采购",
"采购": "招标采购",
"施工建设": "施工建设",
"施工": "施工建设",
"建设": "施工建设",
"竣工验收": "竣工验收",
"验收": "竣工验收",
"竣工": "竣工验收",
"交付售后": "交付售后",
"交付": "交付售后",
"售后": "交付售后",
}
_ROLE_KEYWORDS = [
    "项目总经理", "设计部经理", "开发部经理", "工程部经理", "成本部经理",
    "营销部经理", "财务经理", "客服部经理", "行政人事经理",
    "投资拓展主管", "法务主管", "审计主管",
    "建筑设计主管", "精装设计主管", "景观设计主管", "给排水设计主管",
    "强弱电设计主管", "暖通设计主管",
    "规划报建专员", "工程报建专员", "配套报建专员",
    "土建工程主管", "精装工程主管", "景观工程主管", "水电工程主管",
    "安全总监", "资料员",
    "土建造价主管", "安装造价主管", "招标采购专员", "合同管理员",
    "策划师", "销售主管", "置业顾问", "渠道专员",
    "会计", "出纳",
    "维保主管", "产证专员",
    "行政专员", "人事专员",
]
def _extract_metadata(source_name: str, title: str, body: str) -> dict:
    stage = ""
    for kw, stage_name in _STAGE_KEYWORDS.items():
        if kw in source_name or kw in title or kw in body[:200]:
            stage = stage_name
            break
    chunk_id = ""
    id_match = re.search(r"\b(LS-\d+-\d+)\b", title + body[:200])
    if id_match:
        chunk_id = id_match.group(1)
    else:
        id_match = re.search(r"\b(DOC-\d+-\d+)\b", title + body[:200])
        if id_match:
            chunk_id = id_match.group(1)
    responsible_role = ""
    for role_name in _ROLE_KEYWORDS:
        if role_name in title or role_name in body[:500]:
            responsible_role = role_name
            break
    keywords = _extract_keywords(title, body)
    return {

```

```

        "stage": stage,
        "chunk_id": chunk_id,
        "keywords": keywords,
        "responsible_role": responsible_role,
    }
def _extract_keywords(title: str, body: str, max_kw: int = 10) -> list[str]:
    text = f"{title} {body[:500]}"
    kw_set: set[str] = set()
    chinese_chars = re.findall(r"[-\u4e00-\u9fff]{2,6}", text)
    for kw in chinese_chars:
        if len(kw) >= 2:
            kw_set.add(kw)
    eng_words = re.findall(r"[a-zA-Z0-9]{2,}", text)
    for w in eng_words:
        kw_set.add(w)
    return list(kw_set)[:max_kw]
def _match_stage(chunk: dict, stage: str) -> bool:
    chunk_stage = chunk.get("stage", "")
    return stage in chunk_stage or stage in chunk.get("source", "")
def _match_role(chunk: dict, role: str) -> bool:
    chunk_role = chunk.get("responsible_role", "")
    if role in chunk_role:
        return True
    return role in chunk.get("title", "") or role in chunk.get("body", "")[:500]

```

文件编号：45 文件路径：emily-core\emily_core\providers\rag\maxkb_provider.py

```

import logging
from typing import Optional
import aiohttp
from .base import RagProvider, SearchResult, RagSearchResponse
logger = logging.getLogger("emily.rag.maxkb")
_ADMIN_USER = "admin"
_WORKSPACE = "default"
class MaxKBProvider(RagProvider):
    def __init__(
        self,
        base_url: str = "http://maxkb:8080",
        admin_password: str = "",
        knowledge_id: str = "",
        search_mode: str = "embedding",
        similarity: float = 0.3,
        timeout: int = 60,
    ):
        self._base_url = base_url.rstrip("/")
        self._admin_password = admin_password
        self._knowledge_id = knowledge_id
        self._search_mode = search_mode
        self._similarity = similarity
        self._timeout = timeout
        self._available: Optional[bool] = None
        self._auth_token: Optional[str] = None

```

```
    async def _login(self) -> Optional[str]:
        try:
            async with aiohttp.ClientSession() as session:
                async with session.post(
                    f"{self._base_url}/admin/api/user/login",
                    json={"username": _ADMIN_USER, "password":
self._admin_password},
                    timeout=aiohttp.ClientTimeout(total=10),
                ) as resp:
                    if resp.status != 200:
                        logger.warning("MaxKB admin login failed: HTTP %d",
resp.status)

                        return None
                    data = await resp.json()
                    token = data.get("data", {}).get("token")
                    if token:
                        logger.info("MaxKB admin login successful")
                        return token
                    logger.warning("MaxKB admin login response missing token: %s",
                                str(data)[:200])
                        return None
        except Exception as e:
            logger.warning("MaxKB admin login error: %s", e)
            return None
    async def is_available(self) -> bool:
        if self._available is not None:
            return self._available
        if not self._admin_password or not self._knowledge_id:
            logger.debug("MaxKB: admin_password 或 knowledge_id 未配置")
            self._available = False
            return False
        token = await self._login()
        self._available = token is not None
        if token:
            self._auth_token = token
        return self._available
    async def search(
        self, query: str, top_k: int = 5,
        stage: str | None = None,
        role: str | None = None,
    ) -> RagSearchResponse:
        if not await self.is_available():
            return RagSearchResponse(
                query=query, results=[], context_text="",
                total=0, provider_name="MaxKB (unavailable)",
            )
        augmented_query = query
        filters = []
        if stage:
            filters.append(f"阶段={stage}")
        if role:
            filters.append(f"角色={role}")
        if filters:
            augmented_query = f"{query} (过滤条件: {'、'.join(filters)}) "
```

```

results, context_text = await self._do_hit_test(
    self._auth_token, augmented_query, top_k,
)
if results is None:
    logger.info("MaxKB token expired, re-logging in...")
    self._available = None
    new_token = await self._login()
    if new_token:
        self._auth_token = new_token
        self._available = True
        results, context_text = await self._do_hit_test(
            new_token, augmented_query, top_k,
        )
        if results is None:
            results, context_text = [], ""
if results is None:
    results, context_text = [], ""
return RagSearchResponse(
    query=query,
    results=results,
    context_text=context_text,
    total=len(results) if results else 0,
    provider_name=f"MaxKB (hit_test, mode={self._search_mode})",
)
async def _do_hit_test(
    self, token: str, query: str, top_k: int,
) -> tuple[list[SearchResult] | None, str]:
    try:
        async with aiohttp.ClientSession() as session:
            async with session.post(
                f"{self._base_url}/admin/api/workspace/{_WORKSPACE}"
                f"/knowledge/{self._knowledge_id}/hit_test",
                headers={
                    "Authorization": f"Bearer {token}",
                    "Content-Type": "application/json",
                },
                json={
                    "query_text": query,
                    "top_number": top_k,
                    "similarity": self._similarity,
                    "search_mode": self._search_mode,
                },
                timeout=aiohttp.ClientTimeout(total=self._timeout),
            ) as resp:
                if resp.status == 401:
                    return None, ""
                if resp.status != 200:
                    text = await resp.text()
                    logger.error("MaxKB hit_test HTTP %d: %s",
                                resp.status, text[:300])
                    return [], ""
                data = await resp.json()
                if data.get("code") != 200:
                    logger.error("MaxKB hit_test API error: %s",

```

```

        str(data)[:300])
    return [], ""
hits = data.get("data", [])
if not hits:
    return [], ""
results = []
contexts = []
for i, hit in enumerate(hits):
    content = hit.get("content", "").strip()
    similarity = hit.get("similarity", 0.0)
    title = hit.get("title", "")
    doc_name = hit.get("document_name", "")
    source = doc_name or title or "未知来源"
    results.append(SearchResult(
        content=content,
        score=round(similarity, 4),
        source_document=source,
        source_kb="Emily-地产知识库",
        metadata={
            "title": title,
            "document_name": doc_name,
            "similarity": similarity,
            "index": i,
        },
    ))
    contexts.append(
        f"
    )
    context_text = "\n\n---\n\n".join(contexts)
    return results, context_text
except aiohttp.ClientError as e:
    logger.error("MaxKB hit_test HTTP error: %s", e)
    return [], ""
except Exception as e:
    logger.error("MaxKB hit_test unknown error: %s", e, exc_info=True)
    return [], ""

```

文件编号：46 文件路径：emily-core\emily_core\repositories\agent_reasoning_repo.py

```

import json
import logging
from typing import Optional
from ..infrastructure.database.session import get_session
from ..infrastructure.database.models import AgentReasoningLog
logger = logging.getLogger("emily.repo.agent_reasoning")
class AgentReasoningRepository:
    @staticmethod
    def create(
        message_id: str,
        user_id: str = "",
        conversation_id: str = "",
        matched_sop_id: str | None = None,
    )

```

```
        match_confidence: str = "none",
        is_compound: bool = False,
        fallback: bool = False,
    ) -> str:
        from ..infrastructure.database.models import _new_uuid
        reason_id = _new_uuid()
        with get_session() as session:
            conv_uuid = None
            if conversation_id:
                conv_uuid = AgentReasoningRepository._resolve_conversation_id(
                    session, conversation_id
                )
            log = AgentReasoningLog(
                id=reason_id,
                message_id=message_id,
                user_id=user_id or None,
                conversation_id=conv_uuid,
                matched_sop_id=matched_sop_id,
                match_confidence=match_confidence,
                is_compound=is_compound,
                fallback=fallback,
            )
            session.add(log)
            session.flush()
            logger.debug("ReasoningLog created: %s", reason_id)
        return reason_id

    @staticmethod
    def _resolve_conversation_id(session, business_conv_id: str) -> str | None:
        from ..infrastructure.database.models import Conversation
        conv = (
            session.query(Conversation)
            .filter(Conversation.conversation_id == business_conv_id)
            .first()
        )
        if conv:
            return conv.id
        conv = Conversation(
            im_platform="",
            conversation_type="private",
            conversation_id=business_conv_id,
            takeover_mode="collaborate",
        )
        session.add(conv)
        session.flush()
        return conv.id

    @staticmethod
    def finalize(
        reasoning_log_id: str,
        *,
        iteration_count: int = 0,
        elapsed_ms: int = 0,
        matched_sop_id: str | None = None,
        match_confidence: str = "none",
        is_compound: bool = False,
```

```

        fallback: bool = False,
        execution_result: str = "",
        reply_preview: str = "",
        steps: list | None = None,
        error_message: str = "",
        max_iterations_reached: bool = False,
    ) -> None:
        with get_session() as session:
            log = session.query(AgentReasoningLog).filter(
                AgentReasoningLog.id == reasoning_log_id
            ).first()
            if log is None:
                logger.warning("ReasoningLog not found: %s", reasoning_log_id)
                return
            log.iteration_count = iteration_count
            log.elapsed_ms = elapsed_ms
            log.max_iterations_reached = max_iterations_reached
            log.matched_sop_id = matched_sop_id
            log.match_confidence = match_confidence
            log.is_compound = is_compound
            log.fallback = fallback
            log.execution_result = execution_result
            log.reply_preview = (reply_preview or "")[:500]
            log.error_message = (error_message or "")[:500]
            if steps:
                try:
                    log.steps_json = json.dumps(
                        [s.to_dict() if hasattr(s, 'to_dict') else s for s in
steps],
                        ensure_ascii=False,
                        default=str,
                    )
                except Exception:
                    log.steps_json = json.dumps(steps, ensure_ascii=False,
default=str)
            @staticmethod
            def get_by_message_id(message_id: str) -> Optional[AgentReasoningLog]:
                with get_session() as session:
                    return (
                        session.query(AgentReasoningLog)
                            .filter(AgentReasoningLog.message_id == message_id)
                            .first()
                    )
            @staticmethod
            def get_complete_trace(message_id: str) -> dict:
                from .llm_interaction_repo import LLMInteractionRepository
                from .tool_call_repo import ToolCallRepository
                reasoning = AgentReasoningRepository.get_by_message_id(message_id)
                if reasoning is None:
                    return {"found": False}
                llm_logs = LLMInteractionRepository.get_by_reasoning_id(reasoning.id)
                tool_logs = ToolCallRepository.get_by_reasoning_id(reasoning.id)
                return {
                    "found": True,

```

```

        "reasoning": {
            "id": reasoning.id,
            "message_id": reasoning.message_id,
            "iteration_count": reasoning.iteration_count,
            "elapsed_ms": reasoning.elapsed_ms,
            "matched_sop_id": reasoning.matched_sop_id,
            "match_confidence": reasoning.match_confidence,
            "is_compound": reasoning.is_compound,
            "fallback": reasoning.fallback,
            "execution_result": reasoning.execution_result,
            "reply_preview": reasoning.reply_preview,
            "error_message": reasoning.error_message,
            "steps_json": reasoning.steps_json,
            "created_at": str(reasoning.created_at) if reasoning.created_at
        }
    else None,
    },
    "llm_interactions": llm_logs,
    "tool_calls": tool_logs,
}

```

文件编号: 47 文件路径: emily-core\emily_core\repositories\chat_archive_repo.py

```

import logging
from typing import Optional
from ..infrastructure.database.session import get_session
from ..infrastructure.database.models import (
    Message, MessageAttachment, File, Conversation,
)
logger = logging.getLogger("emily.repo.chat_archive")
class ChatArchiveRepository:
    @staticmethod
    def create_attachment(
        *,
        message_id: str,
        attachment_type: int = 0,
        file_url: str = "",
        file_size: int = 0,
        mime_type: str = "",
        thumbnail_url: str = "",
    ) -> MessageAttachment:
        from ..infrastructure.database.models import _new_uuid
        with get_session() as session:
            att = MessageAttachment(
                id=_new_uuid(),
                message_id=message_id,
                attachment_type=attachment_type,
                file_url=file_url,
                file_size=file_size,
                mime_type=mime_type,
                thumbnail_url=thumbnail_url,
            )
            session.add(att)

```



```
        session.flush()
        logger.debug(
            "Attachment created: msg=%s, type=%d, url=%s",
            message_id, attachment_type, file_url[:80],
        )
    )
    return att

@staticmethod
def get_attachments_for_message(message_id: str) -> list[dict]:
    with get_session() as session:
        rows = (
            session.query(MessageAttachment)
            .filter(MessageAttachment.message_id == message_id)
            .all()
        )
    return [
        {
            "id": r.id,
            "attachment_type": r.attachment_type,
            "file_url": r.file_url,
            "local_path": r.local_path,
            "file_size": r.file_size,
            "mime_type": r.mime_type,
            "thumbnail_url": r.thumbnail_url,
            "file_id": r.file_id,
        }
        for r in rows
    ]

@staticmethod
def get_files_for_conversation(conversation_id: str) -> list[dict]:
    with get_session() as session:
        rows = (
            session.query(MessageAttachment, File, Message)
            .join(Message, MessageAttachment.message_id == Message.id)
            .outerjoin(File, MessageAttachment.file_id == File.id)
            .filter(Message.conversation_id == conversation_id)
            .order_by(MessageAttachment.created_at.desc())
            .all()
        )
    results = []
    for att, file, msg in rows:
        results.append({
            "attachment_id": att.id,
            "attachment_type": att.attachment_type,
            "file_url": att.file_url,
            "local_path": att.local_path,
            "file_size": att.file_size,
            "mime_type": att.mime_type,
            "file_id": file.id if file else None,
            "file_no": file.file_no if file else None,
            "filename": file.filename if file else None,
            "message_id": msg.id,
            "message_preview": (msg.content or "")[:80],
            "created_at": str(att.created_at) if att.created_at else None,
```

```
    })  
    return results
```

文件编号: 48 文件路径: emily-core\emily_core\repositories\event_repo.py

```
import logging  
from datetime import datetime, timezone  
from typing import Optional  
from ..infrastructure.database.session import get_session  
from ..infrastructure.database.models import Event, Project  
logger = logging.getLogger("emily.repo.event")  
class EventRepository:  
    @staticmethod  
    def generate_event_no() -> str:  
        today_str = datetime.now(timezone.utc).strftime("%Y%m%d")  
        prefix = f"EVT-{today_str}-"  
        with get_session() as session:  
            last = (  
                session.query(Event)  
                .filter(Event.event_no.like(f"{prefix}%"))  
                .order_by(Event.event_no.desc())  
                .first()  
            )  
            if last is None:  
                return f"{prefix}0001"  
            seq_str = last.event_no[len(prefix):]  
            try:  
                seq = int(seq_str) + 1  
            except ValueError:  
                seq = 1  
            return f"{prefix}{seq:04d}"  
    @staticmethod  
    def create(  
        *,  
        event_no: str,  
        event_type: str,  
        title: str,  
        project_id: Optional[str] = None,  
        user_id: Optional[str] = None,  
        message_id: Optional[str] = None,  
        category: str = "待分类",  
        description: Optional[str] = None,  
        event_date: Optional[str] = None,  
        payload: str = "{}",  
        status: str = "pending",  
        related_event_ids: list[str] | None = None,  
        conversation_id: Optional[str] = None,  
    ) -> Event:  
        with get_session() as session:  
            import json as _json  
            event = Event(  
                event_no=event_no,
```

```

        event_type=event_type,
        category=category,
        project_id=project_id,
        user_id=user_id,
        message_id=message_id,
        title=title,
        description=description,
        event_date=event_date,
        payload=payload,
        status=status,
        related_event_ids=_json.dumps(related_event_ids,
ensure_ascii=False)
        if related_event_ids
        else "[]",
        conversation_id=conversation_id,
    )
    session.add(event)
    session.flush()
    logger.info(
        "Event created: id=%s, no=%s, type=%s, status=%s",
        event.id, event_no, event_type, status,
    )
    return event

@staticmethod
def get_by_id(event_id: str) -> Optional[Event]:
    with get_session() as session:
        return session.query(Event).filter(Event.id == event_id).first()

@staticmethod
def get_by_event_no(event_no: str) -> Optional[Event]:
    with get_session() as session:
        return session.query(Event).filter(Event.event_no == event_no).first()

@staticmethod
def update_status(event_id: str, status: str) -> None:
    with get_session() as session:
        event = session.query(Event).filter(Event.id == event_id).first()
        if event:
            event.status = status
            if status == "confirmed":
                event.confirmed_at = datetime.now(timezone.utc).isoformat()
            logger.info(
                "Event status updated: id=%s, status=%s", event_id, status,
            )

@staticmethod
def update_remarks(event_id: str, remarks: str) -> None:
    with get_session() as session:
        event = session.query(Event).filter(Event.id == event_id).first()
        if event:
            event.remarks = remarks

@staticmethod
def find_pending_by_conversation(conversation_id: str) -> Optional[Event]:
    with get_session() as session:
        return (
            session.query(Event)
            .join(Event.message_id) # noqa: 不做 join, 改用子查询

```

```

        .filter(Event.status == "pending")
        .order_by(Event.created_at.desc())
        .first()
    )
    @staticmethod
    def find_pending_by_conversation_id(conversation_id: str) -> Optional[Event]:
        with get_session() as session:
            return (
                session.query(Event)
                .filter(
                    Event.conversation_id == conversation_id,
                    Event.status == "pending",
                )
                .order_by(Event.created_at.desc())
                .first()
            )
    @staticmethod
    def query_events(
        *,
        project_id: str | None = None,
        project_ids: list[str] | None = None,
        time_range: str = "all",
        status: str | None = None,
        limit: int = 50,
    ) -> list[Event]:
        from datetime import datetime, timezone, timedelta
        from ..infrastructure.database.models import Message
        with get_session() as session:
            q = session.query(Event)
            if project_ids:
                q = q.filter(Event.project_id.in_(project_ids))
            elif project_id:
                q = q.filter(Event.project_id == project_id)
            if status:
                q = q.filter(Event.status == status)
            if time_range != "all":
                now = datetime.now(timezone.utc)
                if time_range == "today":
                    start = now.replace(hour=0, minute=0, second=0, microsecond=0)
                elif time_range == "this_week":
                    start = now - timedelta(days=now.weekday())
                    start = start.replace(hour=0, minute=0, second=0,
microsecond=0)
                elif time_range == "this_month":
                    start = now.replace(day=1, hour=0, minute=0, second=0,
microsecond=0)
                else:
                    start = None
            if start:
                q = q.filter(Event.created_at >= start.isoformat())
            q = q.order_by(Event.created_at.desc()).limit(limit)
            return q.all()
    @staticmethod
    def count_by_status(project_id: str | None = None) -> dict[str, int]:

```

```

        with get_session() as session:
            q = session.query(Event)
            if project_id:
                q = q.filter(Event.project_id == project_id)
            rows = q.all()
            counts = {}
            for row in rows:
                s = row.status or "unknown"
                counts[s] = counts.get(s, 0) + 1
            return counts

    @staticmethod
    def find_project_by_name(name: str) -> Optional[Project]:
        with get_session() as session:
            return session.query(Project).filter(Project.name == name).first()

    @staticmethod
    def find_project_by_code(code: str) -> Optional[Project]:
        with get_session() as session:
            return session.query(Project).filter(Project.code == code).first()

    @staticmethod
    def list_projects(status: str = "active") -> list[Project]:
        with get_session() as session:
            return session.query(Project).filter(Project.status == status).all()

```

文件编号: 49 文件路径: emily-core\emily_core\repositories\evolution_repo.py

```

from __future__ import annotations
import logging
from datetime import datetime, timedelta
from typing import Optional
from sqlalchemy import func, text, or_
from sqlalchemy.orm import Session
from emily_core.infrastructure.database.models import (
    BEIJING_TZ,
    EvolutionDailyInsight,
    EvolutionRule,
    EvolutionPatch,
    PipelineExecutionLog,
    SOPRoutingLog,
    UserFeedbackSignal,
    RAGRetrievalLog,
    BusinessEventLog,
    SessionLifecycleLog,
    AgentReasoningLog,
    ToolCallLog,
    ProjectNode,
    Task,
    User,
    _new_uuid,
    _utc_now,
)
from emily_core.infrastructure.database.session import get_session
logger = logging.getLogger("emily.evolution_repo")

```

```
class EvolutionRepo:
    @staticmethod
    def create_insight(
        insight_date: str,
        *,
        analysis_days: int = 1,
        total_messages: int = 0,
        total_pipeline_runs: int = 0,
        sop_hit_rate: float = 0.0,
        fallback_rate: float = 0.0,
        top_sop_ids: str = "[]",
        feedback_summary: str = "",
        anomaly_flags: str = "[]",
        insight_text: str = "",
        metrics_json: str = "{}",
        health_score: int = 0,
        session: Optional[Session] = None,
    ) -> EvolutionDailyInsight:
        def _impl(sess: Session) -> EvolutionDailyInsight:
            existing = sess.query(EvolutionDailyInsight).filter(
                EvolutionDailyInsight.insight_date == insight_date
            ).first()
            if existing:
                existing.analysis_days = analysis_days
                existing.total_messages = total_messages
                existing.total_pipeline_runs = total_pipeline_runs
                existing.sop_hit_rate = sop_hit_rate
                existing.fallback_rate = fallback_rate
                existing.top_sop_ids = top_sop_ids
                existing.feedback_summary = feedback_summary
                existing.anomaly_flags = anomaly_flags
                existing.insight_text = insight_text
                existing.metrics_json = metrics_json
                existing.health_score = health_score
                sess.flush()
                return existing
            row = EvolutionDailyInsight(
                insight_date=insight_date,
                analysis_days=analysis_days,
                total_messages=total_messages,
                total_pipeline_runs=total_pipeline_runs,
                sop_hit_rate=sop_hit_rate,
                fallback_rate=fallback_rate,
                top_sop_ids=top_sop_ids,
                feedback_summary=feedback_summary,
                anomaly_flags=anomaly_flags,
                insight_text=insight_text,
                metrics_json=metrics_json,
                health_score=health_score,
            )
            sess.add(row)
            sess.flush()
            return row
        if session is not None:
```

```

        return _impl(session)
    with get_session() as sess:
        return _impl(sess)
    @staticmethod
    def get_insight_by_date(date: str, *, session: Optional[Session] = None) ->
Optional[EvolutionDailyInsight]:
        def _impl(sess: Session):
            return sess.query(EvolutionDailyInsight).filter(
                EvolutionDailyInsight.insight_date == date
            ).first()
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)
    @staticmethod
    def get_insights_range(start_date: str, end_date: str, *, session:
Optional[Session] = None) -> list[EvolutionDailyInsight]:
        def _impl(sess: Session):
            return sess.query(EvolutionDailyInsight).filter(
                EvolutionDailyInsight.insight_date >= start_date,
                EvolutionDailyInsight.insight_date <= end_date,
            ).order_by(EvolutionDailyInsight.insight_date).all()
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)
    @staticmethod
    def generate_rule_no(*, session: Optional[Session] = None) -> str:
        def _impl(sess: Session) -> str:
            count = sess.query(EvolutionRule).filter(
                EvolutionRule.rule_no.like("R-%")
            ).count()
            return f"R-{{count + 1:03d}}"
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)
    @staticmethod
    def create_rule(
        rule_no: str,
        title: str,
        *,
        description: str = "",
        evidence_insight_ids: str = "[]",
        category: str = "",
        confidence: float = 0.0,
        status: str = "DRAFT",
        suggested_action: str = "",
        impact_estimate: str = "",
        session: Optional[Session] = None,
    ) -> EvolutionRule:
        def _impl(sess: Session) -> EvolutionRule:
            row = EvolutionRule(
                rule_no=rule_no,

```

```

        title=title,
        description=description,
        evidence_insight_ids=evidence_insight_ids,
        category=category,
        confidence=confidence,
        status=status,
        suggested_action=suggested_action,
        impact_estimate=impact_estimate,
    )
    sess.add(row)
    sess.flush()
    return row
    if session is not None:
        return _impl(session)
    with get_session() as sess:
        return _impl(sess)
    @staticmethod
    def get_rule_by_no(rule_no: str, *, session: Optional[Session] = None) ->
Optional[EvolutionRule]:
        def _impl(sess: Session):
            return sess.query(EvolutionRule).filter(
                EvolutionRule.rule_no == rule_no
            ).first()
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)
    @staticmethod
    def get_rules_by_status(status: str, *, session: Optional[Session] = None) ->
list[EvolutionRule]:
        def _impl(sess: Session):
            return sess.query(EvolutionRule).filter(
                EvolutionRule.status == status
            ).order_by(EvolutionRule.created_at).all()
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)
    @staticmethod
    def update_rule_status(rule_no: str, status: str, *, confirmed_at: str = "",
superseded_by: str = "", session: Optional[Session] = None) -> bool:
        def _impl(sess: Session) -> bool:
            row = sess.query(EvolutionRule).filter(
                EvolutionRule.rule_no == rule_no
            ).first()
            if row is None:
                return False
            row.status = status
            if confirmed_at:
                row.confirmed_at = confirmed_at
            if superseded_by:
                row.superseded_by = superseded_by
            sess.flush()
            return True

```



```

        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)
    @staticmethod
    def generate_patch_no(*, session: Optional[Session] = None) -> str:
        def _impl(sess: Session) -> str:
            count = sess.query(EvolutionPatch).filter(
                EvolutionPatch.patch_no.like("EP-%")
            ).count()
            return f"EP-{{count + 1:03d}}"
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)
    @staticmethod
    def create_patch(
        patch_no: str,
        rule_no: str,
        *,
        target_type: str = "",
        target_path: str = "",
        patch_content: str = "",
        patch_type: str = "",
        search_anchor: str = "",
        risk_level: str = "",
        risk_reasoning: str = "",
        validation_criteria: str = "",
        expected_effect: str = "",
        status: str = "DRAFT",
        session: Optional[Session] = None,
    ) -> EvolutionPatch:
        def _impl(sess: Session) -> EvolutionPatch:
            row = EvolutionPatch(
                patch_no=patch_no,
                rule_no=rule_no,
                target_type=target_type,
                target_path=target_path,
                patch_content=patch_content,
                patch_type=patch_type,
                search_anchor=search_anchor,
                risk_level=risk_level,
                risk_reasoning=risk_reasoning,
                validation_criteria=validation_criteria,
                expected_effect=expected_effect,
                status=status,
            )
            sess.add(row)
            sess.flush()
            return row
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)

```

```

    @staticmethod
    def get_patch_by_no(patch_no: str, *, session: Optional[Session] = None) ->
Optional[EvolutionPatch]:
        def _impl(sess: Session):
            return sess.query(EvolutionPatch).filter(
                EvolutionPatch.patch_no == patch_no
            ).first()
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)

    @staticmethod
    def get_patches_by_status(status: str, *, session: Optional[Session] = None) -
> list[EvolutionPatch]:
        def _impl(sess: Session):
            return sess.query(EvolutionPatch).filter(
                EvolutionPatch.status == status
            ).order_by(EvolutionPatch.created_at).all()
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)

    @staticmethod
    def update_patch_status(patch_no: str, status: str, *, applied_at: str = "",
validated_at: str = "", validation_result: str = "", rollback_snapshot: str = "",
session: Optional[Session] = None) -> bool:
        def _impl(sess: Session) -> bool:
            row = sess.query(EvolutionPatch).filter(
                EvolutionPatch.patch_no == patch_no
            ).first()
            if row is None:
                return False
            row.status = status
            if applied_at:
                row.applied_at = applied_at
            if validated_at:
                row.validated_at = validated_at
            if validation_result:
                row.validation_result = validation_result
            if rollback_snapshot:
                row.rollback_snapshot = rollback_snapshot
            sess.flush()
            return True
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)

    @staticmethod
    def _date_range_filter(start_date: str, end_date: str):
        start = f"{start_date}T00:00:00"
        end_dt = datetime.fromisoformat(end_date) + timedelta(days=1)
        end = end_dt.isoformat()
        return start, end

    @staticmethod

```

```

def aggregate_pipeline_logs(start_date: str, end_date: str, *, session:
Optional[Session] = None) -> dict:
    def _impl(sess: Session) -> dict:
        start, end = EvolutionRepo._date_range_filter(start_date, end_date)
        total = sess.query(func.count(PipelineExecutionLog.id)).filter(
            PipelineExecutionLog.created_at >= start,
            PipelineExecutionLog.created_at < end,
        ).scalar() or 0
        hit = sess.query(func.count(PipelineExecutionLog.id)).filter(
            PipelineExecutionLog.created_at >= start,
            PipelineExecutionLog.created_at < end,
            PipelineExecutionLog.is_fallback == False,
        ).scalar() or 0
        fallback = total - hit
        sop_hit_rate = hit / total if total > 0 else 0.0
        fallback_rate = fallback / total if total > 0 else 0.0
        sop_rows = sess.query(
            PipelineExecutionLog.matched_sop_id,
            func.count(PipelineExecutionLog.id).label("cnt"),
        ).filter(
            PipelineExecutionLog.created_at >= start,
            PipelineExecutionLog.created_at < end,
            PipelineExecutionLog.is_fallback == False,
            PipelineExecutionLog.matched_sop_id != "",
        ).group_by(PipelineExecutionLog.matched_sop_id).order_by(text("cnt
DESC")).limit(10).all()
        status_rows = sess.query(
            PipelineExecutionLog.final_status,
            func.count(PipelineExecutionLog.id).label("cnt"),
        ).filter(
            PipelineExecutionLog.created_at >= start,
            PipelineExecutionLog.created_at < end,
        ).group_by(PipelineExecutionLog.final_status).all()
        avg_elapsed =
sess.query(func.avg(PipelineExecutionLog.elapsed_ms)).filter(
            PipelineExecutionLog.created_at >= start,
            PipelineExecutionLog.created_at < end,
        ).scalar() or 0
        max_elapsed =
sess.query(func.max(PipelineExecutionLog.elapsed_ms)).filter(
            PipelineExecutionLog.created_at >= start,
            PipelineExecutionLog.created_at < end,
        ).scalar() or 0
        blocked = sess.query(func.count(PipelineExecutionLog.id)).filter(
            PipelineExecutionLog.created_at >= start,
            PipelineExecutionLog.created_at < end,
            PipelineExecutionLog.was_blocked == True,
        ).scalar() or 0
        conf_rows = sess.query(
            PipelineExecutionLog.match_confidence,
            func.count(PipelineExecutionLog.id).label("cnt"),
        ).filter(
            PipelineExecutionLog.created_at >= start,
            PipelineExecutionLog.created_at < end,

```

```

        ).group_by(PipelineExecutionLog.match_confidence).all()
    failed_rows = sess.query(
        PipelineExecutionLog.final_status,
        PipelineExecutionLog.abort_reason,
    ).filter(
        PipelineExecutionLog.created_at >= start,
        PipelineExecutionLog.created_at < end,
        PipelineExecutionLog.final_status.in_(["FAILED", "ABORTED"]),
    ).limit(20).all()
    low_conf = sess.query(PipelineExecutionLog.intent_reasoning).filter(
        PipelineExecutionLog.created_at >= start,
        PipelineExecutionLog.created_at < end,
        PipelineExecutionLog.match_confidence.in_(["low", "none"]),
        PipelineExecutionLog.intent_reasoning != "",
    ).limit(10).all()
    return {
        "total": total,
        "hit": hit,
        "fallback": fallback,
        "sop_hit_rate": round(sop_hit_rate, 4),
        "fallback_rate": round(fallback_rate, 4),
        "sop_distribution": [{"sop_id": r.matched_sop_id, "count": r.cnt}
    for r in sop_rows],
        "status_distribution": {r.final_status: r.cnt for r in
    status_rows},
        "avg_elapsed_ms": int(avg_elapsed),
        "max_elapsed_ms": int(max_elapsed),
        "blocked": blocked,
        "confidence_distribution": {r.match_confidence: r.cnt for r in
    conf_rows},
        "abort_reasons": [{"status": r.final_status, "reason":
    r.abort_reason} for r in failed_rows],
        "low_confidence_reasons": [r.intent_reasoning for r in low_conf],
    }
    if session is not None:
        return _impl(session)
    with get_session() as sess:
        return _impl(sess)

    @staticmethod
    def aggregate_sop_routing_logs(start_date: str, end_date: str, *, session:
Optional[Session] = None) -> dict:
        def _impl(sess: Session) -> dict:
            not_hit = sess.query(func.count(SOPRoutingLog.id)).filter(
                SOPRoutingLog.log_date >= start_date,
                SOPRoutingLog.log_date <= end_date,
                SOPRoutingLog.is_hit == False,
            ).scalar() or 0
            miss_samples = sess.query(
                SOPRoutingLog.message_content,
                SOPRoutingLog.llm_reasoning,
                SOPRoutingLog.match_confidence,
            ).filter(
                SOPRoutingLog.log_date >= start_date,
                SOPRoutingLog.log_date <= end_date,

```

```

        SOPRoutingLog.is_hit == False,
    ).order_by(SOPRoutingLog.created_at.desc()).limit(10).all()
    fallback_rows = sess.query(
        SOPRoutingLog.fallback_action,
        func.count(SOPRoutingLog.id).label("cnt"),
    ).filter(
        SOPRoutingLog.log_date >= start_date,
        SOPRoutingLog.log_date <= end_date,
        SOPRoutingLog.is_hit == False,
    ).group_by(SOPRoutingLog.fallback_action).all()
    return {
        "not_hit_count": not_hit,
        "miss_samples": [
            {"message": r.message_content, "reasoning": r.llm_reasoning,
"confidence": r.match_confidence}
            for r in miss_samples
        ],
        "fallback_distribution": {r.fallback_action: r.cnt for r in
fallback_rows},
    }
    if session is not None:
        return _impl(session)
    with get_session() as sess:
        return _impl(sess)

    @staticmethod
    def aggregate_feedback_signals(start_date: str, end_date: str, *, session:
Optional[Session] = None) -> dict:
        def _impl(sess: Session) -> dict:
            start, end = EvolutionRepo._date_range_filter(start_date, end_date)
            type_rows = sess.query(
                UserFeedbackSignal.signal_type,
                func.count(UserFeedbackSignal.id).label("cnt"),
                func.avg(UserFeedbackSignal.signal_strength).label("avg_strength"),
            ).filter(
                UserFeedbackSignal.created_at >= start,
                UserFeedbackSignal.created_at < end,
            ).group_by(UserFeedbackSignal.signal_type).all()
            corrections = sess.query(
                UserFeedbackSignal.trigger_message,
                UserFeedbackSignal.context_summary,
            ).filter(
                UserFeedbackSignal.created_at >= start,
                UserFeedbackSignal.created_at < end,
                UserFeedbackSignal.signal_type == "explicit_correction",
            ).order_by(UserFeedbackSignal.signal_strength.desc()).limit(5).all()
            return {
                "type_distribution": [
                    {"type": r.signal_type, "count": r.cnt, "avg_strength":
round(float(r.avg_strength or 0), 2)}
                    for r in type_rows
                ],
                "correction_samples": [
                    {"message": r.trigger_message, "context": r.context_summary}

```

```

        for r in corrections
    ],
    }
    if session is not None:
        return _impl(session)
    with get_session() as sess:
        return _impl(sess)
    @staticmethod
    def aggregate_rag_logs(start_date: str, end_date: str, *, session:
Optional[Session] = None) -> dict:
        def _impl(sess: Session) -> dict:
            start, end = EvolutionRepo._date_range_filter(start_date, end_date)
            total = sess.query(func.count(RAGRetrievalLog.id)).filter(
                RAGRetrievalLog.created_at >= start,
                RAGRetrievalLog.created_at < end,
            ).scalar() or 0
            hit = sess.query(func.count(RAGRetrievalLog.id)).filter(
                RAGRetrievalLog.created_at >= start,
                RAGRetrievalLog.created_at < end,
                RAGRetrievalLog.hit_count > 0,
            ).scalar() or 0
            avg_top_score =
sess.query(func.avg(RAGRetrievalLog.top_score)).filter(
                RAGRetrievalLog.created_at >= start,
                RAGRetrievalLog.created_at < end,
            ).scalar() or 0
            avg_latency = sess.query(func.avg(RAGRetrievalLog.latency_ms)).filter(
                RAGRetrievalLog.created_at >= start,
                RAGRetrievalLog.created_at < end,
            ).scalar() or 0
            zero_hit = sess.query(
                RAGRetrievalLog.query_text,
                RAGRetrievalLog.provider,
            ).filter(
                RAGRetrievalLog.created_at >= start,
                RAGRetrievalLog.created_at < end,
                RAGRetrievalLog.hit_count == 0,
            ).order_by(RAGRetrievalLog.created_at.desc()).limit(10).all()
            return {
                "total": total,
                "hit": hit,
                "zero_hit_count": total - hit,
                "zero_hit_rate": round((total - hit) / total, 4) if total > 0 else
0.0,
                "avg_top_score": round(float(avg_top_score), 4),
                "avg_latency_ms": int(avg_latency),
                "zero_hit_samples": [{"query": r.query_text, "provider":
r.provider} for r in zero_hit],
            }
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)
    @staticmethod

```

```

def aggregate_business_events(start_date: str, end_date: str, *, session:
Optional[Session] = None) -> dict:
    def _impl(sess: Session) -> dict:
        start, end = EvolutionRepo._date_range_filter(start_date, end_date)
        cat_rows = sess.query(
            BusinessEventLog.event_category,
            func.count(BusinessEventLog.id).label("cnt"),
        ).filter(
            BusinessEventLog.created_at >= start,
            BusinessEventLog.created_at < end,
        ).group_by(BusinessEventLog.event_category).all()
        action_rows = sess.query(
            BusinessEventLog.event_action,
            func.count(BusinessEventLog.id).label("cnt"),
        ).filter(
            BusinessEventLog.created_at >= start,
            BusinessEventLog.created_at < end,
        ).group_by(BusinessEventLog.event_action).all()
        user_rows = sess.query(
            BusinessEventLog.user_name,
            func.count(BusinessEventLog.id).label("cnt"),
        ).filter(
            BusinessEventLog.created_at >= start,
            BusinessEventLog.created_at < end,
            BusinessEventLog.user_name != "",
        ).group_by(BusinessEventLog.user_name).order_by(text("cnt
DESC")).limit(10).all()
        return {
            "category_distribution": {r.event_category: r.cnt for r in
cat_rows},
            "action_distribution": {r.event_action: r.cnt for r in
action_rows},
            "top_users": [{"name": r.user_name, "count": r.cnt} for r in
user_rows],
        }
    if session is not None:
        return _impl(session)
    with get_session() as sess:
        return _impl(sess)

@staticmethod
def aggregate_session_lifecycle(start_date: str, end_date: str, *, session:
Optional[Session] = None) -> dict:
    def _impl(sess: Session) -> dict:
        start, end = EvolutionRepo._date_range_filter(start_date, end_date)
        created = sess.query(func.count(SessionLifecycleLog.id)).filter(
            SessionLifecycleLog.created_at >= start,
            SessionLifecycleLog.created_at < end,
            SessionLifecycleLog.event_type == "created",
        ).scalar() or 0
        archived_rows = sess.query(
            func.count(SessionLifecycleLog.id).label("cnt"),
            func.sum(SessionLifecycleLog.message_count).label("total_msgs"),
            func.avg(SessionLifecycleLog.duration_ms).label("avg_duration"),
        ).filter(

```

```

        SessionLifecycleLog.created_at >= start,
        SessionLifecycleLog.created_at < end,
        SessionLifecycleLog.event_type == "archived",
    ).first()
    return {
        "sessions_created": created,
        "sessions_archived": archived_rows.cnt if archived_rows else 0,
        "total_messages_in_archived": int(archived_rows.total_msgs or 0)
    if archived_rows else 0,
        "avg_duration_ms": int(archived_rows.avg_duration or 0) if
archived_rows else 0,
    }
    if session is not None:
        return _impl(session)
    with get_session() as sess:
        return _impl(sess)

    @staticmethod
    def aggregate_agent_reasoning(start_date: str, end_date: str, *, session:
Optional[Session] = None) -> dict:
        def _impl(sess: Session) -> dict:
            start, end = EvolutionRepo._date_range_filter(start_date, end_date)
            fallback = sess.query(func.count(AgentReasoningLog.id)).filter(
                AgentReasoningLog.created_at >= start,
                AgentReasoningLog.created_at < end,
                AgentReasoningLog.fallback == True,
            ).scalar() or 0
            result_rows = sess.query(
                AgentReasoningLog.execution_result,
                func.count(AgentReasoningLog.id).label("cnt"),
            ).filter(
                AgentReasoningLog.created_at >= start,
                AgentReasoningLog.created_at < end,
            ).group_by(AgentReasoningLog.execution_result).all()
            avg_iterations =
sess.query(func.avg(AgentReasoningLog.iteration_count)).filter(
                AgentReasoningLog.created_at >= start,
                AgentReasoningLog.created_at < end,
            ).scalar() or 0
            max_iter_reached =
sess.query(func.count(AgentReasoningLog.id)).filter(
                AgentReasoningLog.created_at >= start,
                AgentReasoningLog.created_at < end,
                AgentReasoningLog.max_iterations_reached == True,
            ).scalar() or 0
            errors = sess.query(AgentReasoningLog.error_message).filter(
                AgentReasoningLog.created_at >= start,
                AgentReasoningLog.created_at < end,
                AgentReasoningLog.execution_result == "failed",
                AgentReasoningLog.error_message != "",
            ).limit(10).all()
            return {
                "fallback_count": fallback,
                "result_distribution": {r.execution_result: r.cnt for r in
result_rows},

```



```

        "avg_iterations": round(float(avg_iterations), 2),
        "max_iterations_reached": max_iter_reached,
        "error_samples": [r.error_message for r in errors],
    }
    if session is not None:
        return _impl(session)
    with get_session() as sess:
        return _impl(sess)
    @staticmethod
    def aggregate_tool_calls(start_date: str, end_date: str, *, session:
Optional[Session] = None) -> dict:
        def _impl(sess: Session) -> dict:
            start, end = EvolutionRepo._date_range_filter(start_date, end_date)
            tool_rows = sess.query(
                ToolCallLog.tool_name,
                func.count(ToolCallLog.id).label("cnt"),
            ).filter(
                ToolCallLog.created_at >= start,
                ToolCallLog.created_at < end,
            ).group_by(ToolCallLog.tool_name).order_by(text("cnt DESC")).all()
            fail_rows = sess.query(
                ToolCallLog.tool_name,
                func.count(ToolCallLog.id).label("cnt"),
                ToolCallLog.error_message,
            ).filter(
                ToolCallLog.created_at >= start,
                ToolCallLog.created_at < end,
                ToolCallLog.is_success == False,
            ).group_by(ToolCallLog.tool_name, ToolCallLog.error_message).all()
            return {
                "tool_distribution": {r.tool_name: r.cnt for r in tool_rows},
                "failure_details": [
                    {"tool": r.tool_name, "count": r.cnt, "error":
r.error_message}
                    for r in fail_rows
                ],
            }
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)
    @staticmethod
    def aggregate_project_nodes(date_str: str = "", *, session: Optional[Session]
= None) -> dict:
        def _impl(sess: Session) -> dict:
            status_rows = sess.query(
                ProjectNode.status,
                func.count(ProjectNode.id).label("cnt"),
            ).filter(
                ProjectNode.is_discarded == False,
            ).group_by(ProjectNode.status).all()
            progress_nums = sess.query(ProjectNode.progress).filter(
                ProjectNode.status == "IN_PROGRESS",
                ProjectNode.is_discarded == False,

```

```

    ).all()
    avg_progress = 0.0
    if progress_nums:
        vals = []
        for p in progress_nums:
            try:
                vals.append(float(p.progress))
            except (ValueError, TypeError):
                pass
        if vals:
            avg_progress = sum(vals) / len(vals)
    type_rows = sess.query(
        ProjectNode.node_type,
        func.count(ProjectNode.id).label("cnt"),
    ).filter(
        ProjectNode.is_discarded == False,
    ).group_by(ProjectNode.node_type).all()
    now_str = datetime.now(BEIJING_TZ).strftime("%Y-%m-%d")
    future_str = (datetime.now(BEIJING_TZ) +
timedelta(days=7)).strftime("%Y-%m-%d")
    overdue = sess.query(ProjectNode).filter(
        ProjectNode.status == "IN_PROGRESS",
        ProjectNode.is_discarded == False,
        ProjectNode.deadline < now_str,
    ).all()
    upcoming = sess.query(ProjectNode).filter(
        ProjectNode.status == "IN_PROGRESS",
        ProjectNode.is_discarded == False,
        ProjectNode.deadline >= now_str,
        ProjectNode.deadline < future_str,
    ).all()
    progress_changes = []
    if date_str:
        start = f"{date_str}T00:00:00"
        end_dt = datetime.fromisoformat(date_str) + timedelta(days=1)
        end = end_dt.isoformat()
        updated = sess.query(ProjectNode).filter(
            ProjectNode.updated_at >= start,
            ProjectNode.updated_at < end,
            ProjectNode.is_discarded == False,
        ).all()
        progress_changes = [
            {"node_id": n.node_id, "name": n.node_name, "progress":
n.progress}
            for n in updated
        ]
    return {
        "status_distribution": {r.status: r.cnt for r in status_rows},
        "avg_progress_in_progress": round(avg_progress, 2),
        "type_distribution": {r.node_type: r.cnt for r in type_rows},
        "overdue_nodes": [{"node_id": n.node_id, "name": n.node_name,
"deadline": n.deadline} for n in overdue],
        "upcoming_deadlines": [{"node_id": n.node_id, "name": n.node_name,
"deadline": n.deadline} for n in upcoming],

```

```

        "progress_changes": progress_changes,
    }
    if session is not None:
        return _impl(session)
    with get_session() as sess:
        return _impl(sess)
    @staticmethod
    def aggregate_cognition_drift(end_date: str, *, session: Optional[Session] =
None) -> dict:
        from emily_core.services.cognition_drift_detector import
CognitionDriftDetector
        from emily_core.infrastructure.database.models import Project,
ProjectWorldBook
        def _impl(sess: Session) -> dict:
            detector = CognitionDriftDetector()
            all_projects = sess.query(Project).filter(
                or_(Project.is_deleted == False, Project.is_deleted == None),
            ).all()
            all_project_ids = [p.id for p in all_projects]
            all_wbs = sess.query(ProjectWorldBook).all()
            wb_map = {wb.project_id: wb for wb in all_wbs}
            project_drifts = []
            projects_with_drift = 0
            total_stale_layers = 0
            stale_layer_distribution: dict[str, int] = {
                "ontology": 0,
                "personnel": 0,
                "structure": 0,
                "temporal": 0,
                "relation": 0,
                "knowledge": 0,
                "introspection": 0,
            }
            for project_id in all_project_ids:
                drift_result = detector.detect(project_id)
                if not drift_result.get("has_world_book"):
                    continue
                has_drift = drift_result.get("has_drift", False)
                stale_layers = drift_result.get("stale_layers", [])
                if has_drift:
                    projects_with_drift += 1
                    total_stale_layers += len(stale_layers)
                    for layer_name in stale_layers:
                        if layer_name in stale_layer_distribution:
                            stale_layer_distribution[layer_name] += 1
                project_drifts.append({
                    "project_id": project_id,
                    "project_name": next((p.name for p in all_projects if p.id ==
project_id), ""),
                    "has_drift": has_drift,
                    "stale_layers": stale_layers,
                    "drift_summary": {
                        k: {"stale": v.get("stale", False), "signals":
v.get("signals", [])}

```

```

        for k, v in drift_result.get("drift", {}).items()
    },
    })
    projects_with_wb = len(wb_map)
    projects_without_wb = len(all_project_ids) - projects_with_wb
    return {
        "total_projects": len(all_project_ids),
        "projects_with_world_book": projects_with_wb,
        "projects_without_world_book": projects_without_wb,
        "world_book_coverage": round(projects_with_wb /
len(all_project_ids), 4) if all_project_ids else 0.0,
        "projects_with_drift": projects_with_drift,
        "drift_rate": round(projects_with_drift / projects_with_wb, 4) if
projects_with_wb else 0.0,
        "total_stale_layers": total_stale_layers,
        "avg_stale_layers_per_drifted_project": round(total_stale_layers /
projects_with_drift, 2) if projects_with_drift else 0.0,
        "stale_layer_distribution": stale_layer_distribution,
        "project_drifts": project_drifts,
    }
    if session is not None:
        return _impl(session)
    with get_session() as sess:
        return _impl(sess)
    @staticmethod
    def get_active_users(*, session: Optional[Session] = None) -> list[User]:
        def _impl(sess: Session):
            return sess.query(User).filter(
                User.status == "active",
                User.is_deleted == False,
            ).all()
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)
    @staticmethod
    def get_user_pending_tasks(user_id: str, *, session: Optional[Session] = None)
-> list[Task]:
        def _impl(sess: Session):
            return sess.query(Task).filter(
                Task.owner_id == user_id,
                Task.status.in_([ "WAITING", "SUBMITTED", "RETURNED", "todo",
"in_progress" ])),
            ).all()
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)
    @staticmethod
    def get_user_nodes(user_id: str, *, session: Optional[Session] = None) ->
list[ProjectNode]:
        def _impl(sess: Session):
            return sess.query(ProjectNode).filter(
                ProjectNode.responsible_user_id == user_id,

```

```

        ProjectNode.status.in_([ "IN_PROGRESS", "CONDITIONS_NOT_MET" ]),
        ProjectNode.is_discarded == False,
    ).all()
    if session is not None:
        return _impl(session)
    with get_session() as sess:
        return _impl(sess)
    @staticmethod
    def get_user_recent_events(user_id: str, date_str: str, *, session:
Optional[Session] = None) -> list[BusinessEventLog]:
        def _impl(sess: Session):
            start = f"{date_str}T00:00:00"
            end_dt = datetime.fromisoformat(date_str) + timedelta(days=1)
            end = end_dt.isoformat()
            return sess.query(BusinessEventLog).filter(
                BusinessEventLog.user_id == user_id,
                BusinessEventLog.created_at >= start,
                BusinessEventLog.created_at < end,
            ).order_by(BusinessEventLog.created_at.desc()).limit(10).all()
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)

```

文件编号: 50 文件路径: emily-core\emily_core\repositories\file_repo.py

```

import logging
from datetime import datetime, timezone
from typing import Optional
from ..infrastructure.database.session import get_session
from ..infrastructure.database.models import File
logger = logging.getLogger("emily.repo.file")
class FileRepository:
    @staticmethod
    def generate_file_no() -> str:
        today_str = datetime.now(timezone.utc).strftime("%Y%m%d")
        prefix = f"FIL-{today_str}-"
        with get_session() as session:
            last = (
                session.query(File)
                .filter(File.file_no.like(f"{prefix}%"))
                .order_by(File.file_no.desc())
                .first()
            )
            if last is None:
                return f"{prefix}0001"
            seq_str = last.file_no[len(prefix):]
            try:
                seq = int(seq_str) + 1
            except ValueError:
                seq = 1
            return f"{prefix}{seq:04d}"

```

```

@staticmethod
def create(
    *,
    file_no: str,
    filename: str,
    project_id: Optional[str] = None,
    message_id: Optional[str] = None,
    file_type: Optional[str] = None,
    bucket: Optional[str] = None,
    object_key: Optional[str] = None,
    storage_path: Optional[str] = None,
    file_size: int = 0,
    uploaded_by: Optional[str] = None,
    parse_status: str = "pending",
    file_category: str = "OTHER",
) -> File:
    with get_session() as session:
        f = File(
            file_no=file_no,
            filename=filename,
            project_id=project_id,
            message_id=message_id,
            file_type=file_type,
            bucket=bucket,
            object_key=object_key,
            storage_path=storage_path,
            file_size=file_size,
            uploaded_by=uploaded_by,
            parse_status=parse_status,
            file_category=file_category,
        )
        session.add(f)
        session.flush()
        logger.info("File created: no=%s, filename=%s, category=%s", file_no,
filename, file_category)
    return f

@staticmethod
def get_by_id(file_id: str) -> Optional[File]:
    with get_session() as session:
        return session.query(File).filter(File.id == file_id).first()

@staticmethod
def get_by_file_no(file_no: str) -> Optional[File]:
    with get_session() as session:
        return session.query(File).filter(File.file_no == file_no).first()

@staticmethod
def query_files(
    *,
    project_id: str | None = None,
    project_ids: list[str] | None = None,
    file_type: str | None = None,
    limit: int = 50,
) -> list[File]:
    with get_session() as session:
        q = session.query(File)

```

```

        if project_ids:
            q = q.filter(File.project_id.in_(project_ids))
        elif project_id:
            q = q.filter(File.project_id == project_id)
        if file_type:
            q = q.filter(File.file_type == file_type)
        q = q.order_by(File.created_at.desc()).limit(limit)
        return q.all()

    @staticmethod
    def query_by_category(
        *,
        project_id: str | None = None,
        project_ids: list[str] | None = None,
        file_category: str | None = None,
        limit: int = 50,
    ) -> list[File]:
        with get_session() as session:
            q = session.query(File).filter(File.is_deleted == False)
            if project_id:
                q = q.filter(File.project_id == project_id)
            if project_ids:
                q = q.filter(File.project_id.in_(project_ids))
            if file_category:
                q = q.filter(File.file_category == file_category)
            return q.order_by(File.created_at.desc()).limit(limit).all()

    @staticmethod
    def update_category(file_id: str, file_category: str) -> File | None:
        with get_session() as session:
            f = session.query(File).filter(File.id == file_id, File.is_deleted ==
False).first()
            if f is None:
                return None
            f.file_category = file_category
            session.commit()
            logger.info("File %s category updated: %s", f.file_no, file_category)
            return f

    @staticmethod
    def count_by_category(project_id: str | None = None) -> dict[str, int]:
        with get_session() as session:
            q = session.query(File).filter(File.is_deleted == False)
            if project_id:
                q = q.filter(File.project_id == project_id)
            files = q.all()
            counts: dict[str, int] = {}
            for f in files:
                cat = f.file_category or "OTHER"
                counts[cat] = counts.get(cat, 0) + 1
            return counts

```

文件编号: 51 文件路径: emily-core\emily_core\repositories\llm_interaction_repo.py

```

import logging
from typing import Optional
from ..infrastructure.database.session import get_session
from ..infrastructure.database.models import LLMInteractionLog
logger = logging.getLogger("emily.repo.llm_interaction")
class LLMInteractionRepository:
    @staticmethod
    def create(
        reasoning_log_id: str,
        call_sequence: int,
        call_type: str,
        model: str,
        user_message_count: int = 0,
        tool_count: int = 0,
    ) -> str:
        from ..infrastructure.database.models import _new_uuid
        log_id = _new_uuid()
        with get_session() as session:
            log = LLMInteractionLog(
                id=log_id,
                reasoning_log_id=reasoning_log_id,
                call_sequence=call_sequence,
                call_type=call_type,
                model=model,
                user_message_count=user_message_count,
                tool_count=tool_count,
            )
            session.add(log)
            session.flush()
            logger.debug("LLMInteractionLog created: %s (seq=%d)", log_id,
call_sequence)
            return log_id
    @staticmethod
    def update_response(
        llm_interaction_id: str,
        response_type: str,
        response_summary: str = "",
        finish_reason: str = "",
        prompt_tokens: int = 0,
        completion_tokens: int = 0,
        total_tokens: int = 0,
        latency_ms: int = 0,
    ) -> None:
        with get_session() as session:
            log = session.query(LLMInteractionLog).filter(
                LLMInteractionLog.id == llm_interaction_id
            ).first()
            if log is None:
                logger.warning("LLMInteractionLog not found: %s",
llm_interaction_id)
            return
            log.response_type = response_type

```



```

log.response_summary = (response_summary or "")[:500]
log.finish_reason = finish_reason[:50] if finish_reason else ""
log.prompt_tokens = prompt_tokens
log.completion_tokens = completion_tokens
log.total_tokens = total_tokens
log.latency_ms = latency_ms

@staticmethod
def get_by_reasoning_id(reasoning_log_id: str) -> list[dict]:
    with get_session() as session:
        rows = (
            session.query(LLMInteractionLog)
            .filter(LLMInteractionLog.reasoning_log_id == reasoning_log_id)
            .order_by(LLMInteractionLog.call_sequence)
            .all()
        )
    return [
        {
            "id": r.id,
            "call_sequence": r.call_sequence,
            "call_type": r.call_type,
            "model": r.model,
            "response_type": r.response_type,
            "response_summary": r.response_summary,
            "finish_reason": r.finish_reason,
            "prompt_tokens": r.prompt_tokens,
            "completion_tokens": r.completion_tokens,
            "total_tokens": r.total_tokens,
            "latency_ms": r.latency_ms,
            "created_at": str(r.created_at) if r.created_at else None,
        }
        for r in rows
    ]

@staticmethod
def get_usage_stats(time_range: str = "7d") -> dict:
    from datetime import datetime, timezone, timedelta
    with get_session() as session:
        now = datetime.now(timezone.utc)
        if time_range == "today":
            cutoff = now.strftime("%Y-%m-%d")
        elif time_range == "week":
            cutoff = (now - timedelta(days=7)).isoformat()
        elif time_range == "month":
            cutoff = (now - timedelta(days=30)).isoformat()
        else:
            cutoff = None
        q = session.query(LLMInteractionLog)
        if cutoff:
            q = q.filter(LLMInteractionLog.created_at >= cutoff)
        rows = q.all()
        total_calls = len(rows)
        total_tokens = sum(r.total_tokens or 0 for r in rows)
        avg_latency = sum(r.latency_ms or 0 for r in rows) / max(total_calls,
1)
by_model: dict[str, dict] = {}

```

```

for r in rows:
    model = r.model or "unknown"
    if model not in by_model:
        by_model[model] = {"calls": 0, "total_tokens": 0}
    by_model[model]["calls"] += 1
    by_model[model]["total_tokens"] += (r.total_tokens or 0)
by_type: dict[str, int] = {}
for r in rows:
    t = r.call_type or "unknown"
    by_type[t] = by_type.get(t, 0) + 1
return {
    "total_calls": total_calls,
    "total_tokens": total_tokens,
    "avg_latency_ms": round(avg_latency, 1),
    "by_model": by_model,
    "by_type": by_type,
}

```

文件编号: 52 文件路径: emily-core\emily_core\repositories\meeting_repo.py

```

import json
import logging
from datetime import datetime, timezone
from typing import Optional
from ..infrastructure.database.session import get_session
from ..infrastructure.database.models import Meeting
logger = logging.getLogger("emily.repo.meeting")
class MeetingRepository:
    @staticmethod
    def generate_meeting_no() -> str:
        today_str = datetime.now(timezone.utc).strftime("%Y%m%d")
        prefix = f"MTG-{today_str}-"
        with get_session() as session:
            last = (
                session.query(Meeting)
                .filter(Meeting.meeting_no.like(f"{prefix}%"))
                .order_by(Meeting.meeting_no.desc())
                .first()
            )
            if last is None:
                return f"{prefix}0001"
            seq_str = last.meeting_no[len(prefix):]
            try:
                seq = int(seq_str) + 1
            except ValueError:
                seq = 1
            return f"{prefix}{seq:04d}"
    @staticmethod
    def create(
        *,
        meeting_no: str,
        title: str = "",

```

```

        project_id: Optional[str] = None,
        summary: Optional[str] = None,
        attendees: Optional[list[str]] = None,
        source_message_id: Optional[str] = None,
        created_by: Optional[str] = None,
    ) -> Meeting:
        with get_session() as session:
            meeting = Meeting(
                meeting_no=meeting_no,
                project_id=project_id,
                title=title,
                summary=summary,
                attendees=json.dumps(attendees or [], ensure_ascii=False),
                source_message_id=source_message_id,
                created_by=created_by,
            )
            session.add(meeting)
            session.flush()
            logger.info("Meeting created: no=%s, title=%s", meeting_no, title)
            return meeting

    @staticmethod
    def get_by_id(meeting_id: str) -> Optional[Meeting]:
        with get_session() as session:
            return session.query(Meeting).filter(Meeting.id == meeting_id).first()

    @staticmethod
    def get_by_meeting_no(meeting_no: str) -> Optional[Meeting]:
        with get_session() as session:
            return session.query(Meeting).filter(Meeting.meeting_no ==
meeting_no).first()

    @staticmethod
    def query_meetings(
        *,
        project_id: str | None = None,
        project_ids: list[str] | None = None,
        time_range: str = "all",
        limit: int = 50,
    ) -> list[Meeting]:
        from datetime import datetime, timezone
        from ..repositories.task_repo import _resolve_time_start
        with get_session() as session:
            q = session.query(Meeting)
            if project_ids:
                q = q.filter(Meeting.project_id.in_(project_ids))
            elif project_id:
                q = q.filter(Meeting.project_id == project_id)
            if time_range != "all":
                now = datetime.now(timezone.utc)
                start = _resolve_time_start(time_range, now)
                if start:
                    q = q.filter(Meeting.created_at >= start.isoformat())
            q = q.order_by(Meeting.created_at.desc()).limit(limit)
            return q.all()

```

文件编号: 53 文件路径: emily-core\emily_core\repositories\message_repo.py

```
import json
import logging
from datetime import datetime, timezone
from typing import List, Optional
from ..infrastructure.database.session import get_session
from ..infrastructure.database.models import Message, Conversation
from ..adapters.standard.message import StandardMessage
from ..adapters.standard.route_decision import RouteDecision
logger = logging.getLogger("emily.repo.message")
class MessageRepository:
    @staticmethod
    def create_from_standard(
        event_id: str,
        msg: StandardMessage,
        decision: RouteDecision,
        status: str = "received",
    ) -> Message:
        with get_session() as session:
            conv = session.query(Conversation).filter(
                Conversation.im_platform == msg.platform,
                Conversation.conversation_id == msg.conversation_id,
            ).first()
            if conv is None:
                conv = Conversation(
                    im_platform=msg.platform,
                    conversation_type=msg.conversation_type,
                    conversation_id=msg.conversation_id,
                    group_id=msg.group_id,
                    title=msg.group_name or msg.group_id or msg.sender_name,
                    takeover_mode=decision.mode,
                )
                session.add(conv)
                session.flush()
            _msg_type = getattr(msg, "msg_type", 1) or 1
            _attachments_list = getattr(msg, "attachments", []) or []
            _file_url = _attachments_list[0].get("url", "") if _attachments_list
        else ""
            _attachments_json = json.dumps(_attachments_list, ensure_ascii=False)
        if _attachments_list else "[]"
            _receiver_id = getattr(msg, "receiver_id", "") or ""
        db_msg = Message(
            event_id=event_id,
            message_uid=msg.message_id or None,
            conversation_id=conv.id,
            sender_im_id=msg.sender_id,
            sender_name=msg.sender_name,
            content=msg.content or "",
            is_at_bot=msg.is_at_bot,
            takeover=decision.takeover,
            takeover_reason=decision.reason or "",
            status=status,
```

```

        msg_type=msg_type,
        file_url=_file_url,
        attachments=_attachments_json,
        receiver_id=_receiver_id,
        group_id=msg.group_id,
    )
    session.add(db_msg)
    session.flush()
    logger.info(
        "Message saved: id=%s, sender=%s, at_bot=%s, takeover=%s",
        db_msg.id, msg.sender_id, msg.is_at_bot, decision.takeover,
    )
    return db_msg

@staticmethod
def _resolve_conversation_id(session, business_conv_id: str) -> str:
    conv = (
        session.query(Conversation)
        .filter(Conversation.conversation_id == business_conv_id)
        .first()
    )
    if conv:
        return conv.id
    conv = Conversation(
        im_platform="",
        conversation_type="private",
        conversation_id=business_conv_id,
        takeover_mode="collaborate",
    )
    session.add(conv)
    session.flush()
    return conv.id

@staticmethod
def get_by_event_id(event_id: str) -> Optional[Message]:
    with get_session() as session:
        return session.query(Message).filter(Message.event_id ==
event_id).first()

@staticmethod
def mark_processed(message_id: str) -> None:
    with get_session() as session:
        msg = session.query(Message).filter(Message.id == message_id).first()
        if msg:
            msg.status = "processed"
            msg.processed_at = datetime.now(timezone.utc).isoformat()
            logger.info("Message marked processed: id=%s", message_id)

@staticmethod
def list_by_conversation(conversation_id: str, limit: int = 100) ->
List[Message]:
    with get_session() as session:
        return (
            session.query(Message)
            .filter(Message.conversation_id == conversation_id)
            .order_by(Message.created_at.desc())
            .limit(limit)
            .all()

```

```

    )
    @staticmethod
    def update_sender_user_id(message_id: str, user_id: str) -> None:
        with get_session() as session:
            msg = session.query(Message).filter(Message.id == message_id).first()
            if msg:
                msg.sender_user_id = user_id
    @staticmethod
    def update_intent(message_id: str, intent: str) -> None:
        with get_session() as session:
            msg = session.query(Message).filter(Message.id == message_id).first()
            if msg:
                msg.intent = intent
                logger.info("Message intent updated: id=%s, intent=%s",
message_id, intent)
    @staticmethod
    def update_project_id(message_id: str, project_id: str) -> None:
        with get_session() as session:
            msg = session.query(Message).filter(Message.id == message_id).first()
            if msg:
                msg.project_id = project_id
                logger.info("Message project_id updated: id=%s, project_id=%s",
message_id, project_id)
    @staticmethod
    def query_messages(
        *,
        project_id: str | None = None,
        time_range: str = "all",
        conversation_id: str | None = None,
        sender_name: str | None = None,
        keyword: str | None = None,
        intent: str | None = None,
        limit: int = 50,
    ) -> list[Message]:
        from datetime import datetime, timezone
        from ..repositories.task_repo import _resolve_time_start
        with get_session() as session:
            q = session.query(Message)
            if project_id:
                q = q.filter(Message.project_id == project_id)
            if conversation_id:
                q = q.filter(Message.conversation_id == conversation_id)
            if sender_name:
                q = q.filter(Message.sender_name.like(f"%{sender_name}%"))
            if keyword:
                q = q.filter(Message.content.like(f"%{keyword}%"))
            if intent:
                q = q.filter(Message.intent == intent)
            if time_range != "all":
                now = datetime.now(timezone.utc)
                start = _resolve_time_start(time_range, now)
                if start:
                    q = q.filter(Message.created_at >= start.isoformat())
            q = q.order_by(Message.created_at.desc()).limit(limit)

```

```

        return q.all()
    @staticmethod
    def get_active_conversations(
        time_range: str = "all", limit: int = 20
    ) -> list[dict]:
        from datetime import datetime, timezone
        from ..repositories.task_repo import _resolve_time_start
        from sqlalchemy import func
        with get_session() as session:
            q = session.query(
                Message.conversation_id,
                func.count(Message.id).label("count"),
                func.max(Message.created_at).label("last_active"),
            )
            if time_range != "all":
                now = datetime.now(timezone.utc)
                start = _resolve_time_start(time_range, now)
                if start:
                    q = q.filter(Message.created_at >= start.isoformat())
            q = (
                q.group_by(Message.conversation_id)
                .order_by(func.count(Message.id).desc())
                .limit(limit)
            )
            rows = q.all()
            results = []
            for row in rows:
                conv = session.query(Conversation).filter(
                    Conversation.id == row.conversation_id
                ).first()
                results.append({
                    "conversation_id": row.conversation_id,
                    "title": conv.title if conv else "未知会话",
                    "count": row.count,
                    "last_active": row.last_active,
                })
            return results
    @staticmethod
    def count_recent_turns(
        conversation_id: str, ttl_seconds: int = 600
    ) -> int:
        from datetime import datetime, timezone, timedelta
        with get_session() as session:
            cutoff = (datetime.now(timezone.utc) -
                timedelta(seconds=ttl_seconds)).isoformat()
            count = (
                session.query(Message)
                .filter(
                    Message.conversation_id == conversation_id,
                    Message.created_at >= cutoff,
                )
                .count()
            )
            return count

```

```
@staticmethod
def search_content(
    keyword: str, time_range: str = "all", limit: int = 50
) -> list[Message]:
    from datetime import datetime, timezone
    from ..repositories.task_repo import _resolve_time_start
    with get_session() as session:
        q = session.query(Message).filter(
            Message.content.like(f"%{keyword}%")
        )
        if time_range != "all":
            now = datetime.now(timezone.utc)
            start = _resolve_time_start(time_range, now)
            if start:
                q = q.filter(Message.created_at >= start.isoformat())
        q = q.order_by(Message.created_at.desc()).limit(limit)
        return q.all()

@staticmethod
def create_outbound(
    conversation_id: str,
    content: str,
    *,
    sender_im_id: str = "",
    direction: str = "agent_to_user",
    message_type: str = "",
    reply_to_message_id: str | None = None,
) -> Message:
    with get_session() as session:
        conv_uuid = MessageRepository._resolve_conversation_id(
            session, conversation_id
        )
        db_msg = Message(
            event_id=f"outbound_{_new_uuid_short()}",
            conversation_id=conv_uuid,
            sender_im_id=sender_im_id,
            sender_name="Emy",
            content=content,
            direction=direction,
            message_type=message_type,
            status="sent",
            takeover=True,
            is_at_bot=False,
        )
        if reply_to_message_id:
            db_msg.message_uid = reply_to_message_id
        session.add(db_msg)
        session.flush()
        return db_msg

@staticmethod
def list_by_conversation_full(
    conversation_id: str,
    limit: int = 100,
    offset: int = 0,
    include_progress: bool = False,
```



```

) -> list[Message]:
    with get_session() as session:
        q = session.query(Message).filter(
            Message.conversation_id == conversation_id
        )
        if not include_progress:
            q = q.filter(Message.message_type != "progress")
        q = q.order_by(Message.created_at.asc()).offset(offset).limit(limit)
        return q.all()

@staticmethod
def search_messages_fulltext(
    keyword: str,
    *,
    conversation_id: str | None = None,
    user_id: str | None = None,
    time_range: str = "all",
    limit: int = 50,
) -> list[Message]:
    from ..repositories.task_repo import _resolve_time_start
    with get_session() as session:
        q = session.query(Message)
        q = q.filter(Message.content.ilike(f"%{keyword}%"))
        if conversation_id:
            q = q.filter(Message.conversation_id == conversation_id)
        if user_id:
            q = q.filter(Message.sender_user_id == user_id)
        if time_range != "all":
            now = datetime.now(timezone.utc)
            start = _resolve_time_start(time_range, now)
            if start:
                q = q.filter(Message.created_at >= start.isoformat())
        q = q.order_by(Message.created_at.desc()).limit(limit)
        return q.all()

@staticmethod
def get_user_history(
    user_id: str,
    platform: str | None = None,
    limit: int = 50,
) -> list[Message]:
    with get_session() as session:
        q = session.query(Message).filter(
            Message.sender_user_id == user_id
        )
        if platform:
            q = q.join(Conversation).filter(
                Conversation.im_platform == platform
            )
        q = q.order_by(Message.created_at.desc()).limit(limit)
        return q.all()

@staticmethod
def get_daily_stats(date_str: str) -> dict:
    from sqlalchemy import func
    with get_session() as session:
        total = (

```

```
        session.query(func.count(Message.id))
        .filter(Message.created_at.like(f"{date_str}%"))
        .scalar()
    ) or 0
    conv_q = (
        session.query(
            Message.conversation_id,
            func.count(Message.id).label("count"),
        )
        .filter(Message.created_at.like(f"{date_str}%"))
        .group_by(Message.conversation_id)
        .order_by(func.count(Message.id).desc())
        .limit(20)
        .all()
    )
    return {
        "total": total,
        "by_conversation": [
            {"conversation_id": cid, "count": cnt}
            for cid, cnt in conv_q
        ],
    }
}

@staticmethod
def get_recent_by_user_id(user_id: str, limit: int = 20) -> list[dict]:
    with get_session() as session:
        rows = (
            session.query(
                Message.content,
                Message.created_at,
                Message.direction,
                Message.sender_name,
            )
            .filter(Message.sender_user_id == user_id)
            .order_by(Message.created_at.desc())
            .limit(limit)
            .all()
        )
        turns = []
        for row in reversed(rows):
            role = "user" if row.direction == "user_to_agent" else "agent"
            turns.append({
                "role": role,
                "time": row.created_at or "",
                "content": row.content or "",
                "sender_name": row.sender_name or "",
            })
        return turns

def _new_uuid_short() -> str:
    import uuid
    return str(uuid.uuid4())[:12]
```

```

from __future__ import annotations
import json
import logging
from datetime import datetime, timezone, timedelta
from ..infrastructure.database.models import (
    ProjectNode,
    NodeDependency,
    NodeDeliverable,
    NodeAccessibleFile,
    NodeEvent,
)
from ..infrastructure.database.session import get_session
logger = logging.getLogger("emily.node_repo")
BEIJING_TZ = timezone(timedelta(hours=8))
def _to_decimal_str(value: float, precision: int = 4) -> str:
    if precision == 2:
        return f"{value:.2f}"
    return f"{value:.4f}"
def _parse_decimal(value: str) -> float:
    try:
        return float(value)
    except (ValueError, TypeError):
        return 0.0
class ProjectNodeRepo:
    @staticmethod
    def create(**kwargs) -> ProjectNode:
        with get_session() as session:
            node = ProjectNode(**kwargs)
            session.add(node)
            session.flush()
            logger.info("ProjectNode created: %s (project=%s)", node.node_id,
node.project_id)
            return node
    @staticmethod
    def get_by_id(node_uuid: str) -> ProjectNode | None:
        with get_session() as session:
            return (
                session.query(ProjectNode)
                    .filter(ProjectNode.id == node_uuid, ProjectNode.is_discarded ==
False)
                    .first()
            )
    @staticmethod
    def get_by_node_id(node_id: str, project_id: str | None = None) -> ProjectNode
| None:
        with get_session() as session:
            q = (
                session.query(ProjectNode)
                    .filter(ProjectNode.node_id == node_id, ProjectNode.is_discarded
== False)
            )
            if project_id:

```

```
        q = q.filter(ProjectNode.project_id == project_id)
    return q.first()

    @staticmethod
    def find_by_project(project_id: str, status: str | None = None, limit: int =
200) -> list[ProjectNode]:
        with get_session() as session:
            q = (
                session.query(ProjectNode)
                    .filter(ProjectNode.project_id == project_id,
ProjectNode.is_discarded == False)
            )
            if status:
                q = q.filter(ProjectNode.status == status)
            return q.order_by(ProjectNode.sort_order,
ProjectNode.created_at).limit(limit).all()

    @staticmethod
    def find_by_parent(parent_node_id: str) -> list[ProjectNode]:
        with get_session() as session:
            return (
                session.query(ProjectNode)
                    .filter(
                        ProjectNode.parent_node_id == parent_node_id,
                        ProjectNode.is_discarded == False,
                    )
                    .order_by(ProjectNode.sort_order)
                    .all()
            )

    @staticmethod
    def find_by_stage(project_id: str, stage_id: int) -> list[ProjectNode]:
        with get_session() as session:
            return (
                session.query(ProjectNode)
                    .filter(
                        ProjectNode.project_id == project_id,
                        ProjectNode.stage_id == stage_id,
                        ProjectNode.parent_node_id == "",
                        ProjectNode.is_discarded == False,
                    )
                    .order_by(ProjectNode.sort_order)
                    .all()
            )

    @staticmethod
    def find_by_owner(owner_dept_id: str, project_id: str | None = None) ->
list[ProjectNode]:
        with get_session() as session:
            q = (
                session.query(ProjectNode)
                    .filter(ProjectNode.owner_dept_id == owner_dept_id,
ProjectNode.is_discarded == False)
            )
            if project_id:
                q = q.filter(ProjectNode.project_id == project_id)
            return q.order_by(ProjectNode.created_at.desc()).limit(200).all()

    @staticmethod
```

```

def update_fields(node_id: str, **kwargs) -> ProjectNode | None:
    with get_session() as session:
        node = (
            session.query(ProjectNode)
                .filter(ProjectNode.node_id == node_id, ProjectNode.is_discarded
== False)
                .first()
        )
        if node is None:
            return None
        for key, value in kwargs.items():
            if hasattr(node, key):
                setattr(node, key, value)
        node.updated_at = datetime.now(timezone.utc).isoformat()
        session.commit()
        logger.info("ProjectNode updated: %s fields=%s", node_id,
list(kwargs.keys()))
        return node

    @staticmethod
    def update_status(node_id: str, new_status: str) -> ProjectNode | None:
        with get_session() as session:
            node = (
                session.query(ProjectNode)
                    .filter(ProjectNode.node_id == node_id, ProjectNode.is_discarded
== False)
                    .first()
            )
            if node is None:
                return None
            node.status = new_status
            node.updated_at = datetime.now(timezone.utc).isoformat()
            if new_status == "COMPLETED":
                node.completed_at = datetime.now(timezone.utc).isoformat()
            session.commit()
            logger.info("ProjectNode status: %s -> %s", node_id, new_status)
            return node

    @staticmethod
    def update_progress(node_id: str, progress: float) -> ProjectNode | None:
        with get_session() as session:
            node = (
                session.query(ProjectNode)
                    .filter(ProjectNode.node_id == node_id, ProjectNode.is_discarded
== False)
                    .first()
            )
            if node is None:
                return None
            node.progress = _to_decimal_str(progress, precision=2)
            node.updated_at = datetime.now(timezone.utc).isoformat()
            session.commit()
            return node

    @staticmethod
    def discard(node_id: str) -> ProjectNode | None:
        with get_session() as session:

```

```

        node = (
            session.query(ProjectNode)
                .filter(ProjectNode.node_id == node_id, ProjectNode.is_discarded
== False)
                .first()
        )
        if node is None:
            return None
        node.is_discarded = True
        node.updated_at = datetime.now(timezone.utc).isoformat()
        session.commit()
        logger.info("ProjectNode discarded: %s", node_id)
        return node

    @staticmethod
    def count_children(parent_node_id: str) -> int:
        with get_session() as session:
            return (
                session.query(ProjectNode)
                    .filter(
                        ProjectNode.parent_node_id == parent_node_id,
                        ProjectNode.is_discarded == False,
                    )
                    .count()
            )

    @staticmethod
    def find_by_status(status: str, project_id: str | None = None,
                      owner_dept_id: str | None = None,
                      limit: int = 200) -> list[ProjectNode]:
        with get_session() as session:
            q = (
                session.query(ProjectNode)
                    .filter(
                        ProjectNode.status == status,
                        ProjectNode.is_discarded == False,
                    )
            )
            if project_id:
                q = q.filter(ProjectNode.project_id == project_id)
            if owner_dept_id:
                q = q.filter(ProjectNode.owner_dept_id == owner_dept_id)
            return q.order_by(ProjectNode.created_at.desc()).limit(limit).all()

    @staticmethod
    def find_pending_approval(owner_dept_id: str | None = None,
                              project_id: str | None = None,
                              limit: int = 200) -> list[ProjectNode]:
        return ProjectNodeRepo.find_by_status(
            "NOT_ACTIVATED",
            project_id=project_id,
            owner_dept_id=owner_dept_id,
            limit=limit,
        )

    @staticmethod
    def get_ancestor_chain(node_id: str, max_depth: int = 3) -> list[ProjectNode]:
        ancestors = []

```

```

        current_id = node_id
        for _ in range(max_depth):
            with get_session() as session:
                node = (
                    session.query(ProjectNode)
                    .filter(
                        ProjectNode.node_id == current_id,
                        ProjectNode.is_discarded == False,
                    )
                    .first()
                )
            if node is None or not node.parent_node_id:
                break
            parent = ProjectNodeRepo.get_by_node_id(node.parent_node_id)
            if parent is None:
                break
            ancestors.append(parent)
            current_id = parent.node_id
        return ancestors

    @staticmethod
    def find_by_responsible_user(
        responsible_user_id: str,
        project_id: str | None = None,
        node_type: str | None = None,
        status: str | None = None,
        limit: int = 100,
    ) -> list[ProjectNode]:
        with get_session() as session:
            q = (
                session.query(ProjectNode)
                .filter(
                    ProjectNode.responsible_user_id == responsible_user_id,
                    ProjectNode.is_discarded == False,
                )
            )
            if project_id:
                q = q.filter(ProjectNode.project_id == project_id)
            if node_type:
                q = q.filter(ProjectNode.node_type == node_type)
            if status:
                q = q.filter(ProjectNode.status == status)
            return q.order_by(ProjectNode.deadline.asc()).limit(limit).all()

    @staticmethod
    def find_by_node_type(
        node_type: str,
        project_id: str | None = None,
        limit: int = 200,
    ) -> list[ProjectNode]:
        with get_session() as session:
            q = (
                session.query(ProjectNode)
                .filter(ProjectNode.node_type == node_type,
                    ProjectNode.is_discarded == False)
            )

```

```

        if project_id:
            q = q.filter(ProjectNode.project_id == project_id)
        return q.order_by(ProjectNode.created_at.desc()).limit(limit).all()

    @staticmethod
    def find_near_deadline(before_minutes: int = 60, limit: int = 100) ->
list[ProjectNode]:
        now = datetime.now(timezone.utc)
        window_end = (now + timedelta(minutes=before_minutes)).isoformat()
        with get_session() as session:
            return (
                session.query(ProjectNode)
                    .filter(
                        ProjectNode.deadline != "",
                        ProjectNode.deadline.isnot(None),
                        ProjectNode.deadline <= window_end,
                        ProjectNode.deadline > now.isoformat(),
                        ProjectNode.status != "COMPLETED",
                        ProjectNode.is_discarded == False,
                    )
                    .order_by(ProjectNode.deadline.asc())
                    .limit(limit)
                    .all()
            )

    @staticmethod
    def find_overdue(limit: int = 100) -> list[ProjectNode]:
        now_iso = datetime.now(timezone.utc).isoformat()
        with get_session() as session:
            return (
                session.query(ProjectNode)
                    .filter(
                        ProjectNode.deadline != "",
                        ProjectNode.deadline.isnot(None),
                        ProjectNode.deadline < now_iso,
                        ProjectNode.status != "COMPLETED",
                        ProjectNode.is_discarded == False,
                    )
                    .order_by(ProjectNode.deadline.asc())
                    .limit(limit)
                    .all()
            )

class NodeDependencyRepo:
    @staticmethod
    def create(**kwargs) -> NodeDependency:
        with get_session() as session:
            dep = NodeDependency(**kwargs)
            session.add(dep)
            session.flush()
            logger.info(
                "NodeDependency created: %s depends on %s",
                dep.node_id, dep.depends_on_deliverable_id,
            )
        return dep

    @staticmethod
    def find_by_node(node_id: str) -> list[NodeDependency]:

```



```

        with get_session() as session:
            return (
                session.query(NodeDependency)
                .filter(NodeDependency.node_id == node_id)
                .all()
            )

    @staticmethod
    def find_downstream(depends_on_node_id: str) -> list[NodeDependency]:
        with get_session() as session:
            return (
                session.query(NodeDependency)
                .filter(NodeDependency.depends_on_node_id == depends_on_node_id)
                .all()
            )

    @staticmethod
    def get_by_id(dep_id: str) -> NodeDependency | None:
        with get_session() as session:
            return session.query(NodeDependency).filter(NodeDependency.id ==
dep_id).first()

    @staticmethod
    def delete(dep_id: str) -> bool:
        with get_session() as session:
            dep = session.query(NodeDependency).filter(NodeDependency.id ==
dep_id).first()
            if dep is None:
                return False
            session.delete(dep)
            session.commit()
            logger.info("NodeDependency deleted: %s", dep_id)
            return True

    @staticmethod
    def exists(node_id: str, depends_on_deliverable_id: str) -> bool:
        with get_session() as session:
            return (
                session.query(NodeDependency)
                .filter(
                    NodeDependency.node_id == node_id,
                    NodeDependency.depends_on_deliverable_id ==
depends_on_deliverable_id,
                )
                .first()
                is not None
            )

class NodeDeliverableRepo:
    @staticmethod
    def generate_deliverable_id(node_id: str, seq: int) -> str:
        return f"{node_id}-DELV-{seq:03d}"

    @staticmethod
    def get_next_seq(node_id: str) -> int:
        with get_session() as session:
            existing = (
                session.query(NodeDeliverable)
                .filter(NodeDeliverable.node_id == node_id)
                .all()
            )

```

```

    )
    return len(existing) + 1
@staticmethod
def create(**kwargs) -> NodeDeliverable:
    with get_session() as session:
        deliv = NodeDeliverable(**kwargs)
        session.add(deliv)
        session.flush()
        logger.info("NodeDeliverable created: %s for node %s",
deliv.deliverable_id, deliv.node_id)
        return deliv
@staticmethod
def get_by_deliverable_id(deliverable_id: str) -> NodeDeliverable | None:
    with get_session() as session:
        return (
            session.query(NodeDeliverable)
                .filter(NodeDeliverable.deliverable_id == deliverable_id)
                .first()
        )
@staticmethod
def find_by_node(node_id: str) -> list[NodeDeliverable]:
    with get_session() as session:
        return (
            session.query(NodeDeliverable)
                .filter(NodeDeliverable.node_id == node_id)
                .all()
        )
@staticmethod
def update_progress(deliverable_id: str, current_amount: str, file_id: str =
"") -> NodeDeliverable | None:
    with get_session() as session:
        deliv = (
            session.query(NodeDeliverable)
                .filter(NodeDeliverable.deliverable_id == deliverable_id)
                .first()
        )
        if deliv is None:
            return None
        deliv.current_amount = current_amount
        if file_id:
            deliv.file_id = file_id
        if _parse_decimal(current_amount) >=
_parse_decimal(deliv.target_amount):
            deliv.completed_at = datetime.now(timezone.utc).isoformat()
            session.commit()
            logger.info("NodeDeliverable progress: %s -> %s", deliverable_id,
current_amount)
        return deliv
@staticmethod
def get_completion_ratio(node_id: str) -> float:
    with get_session() as session:
        deliverables = (
            session.query(NodeDeliverable)
                .filter(

```

```

        NodeDeliverable.node_id == node_id,
        NodeDeliverable.is_required == True,
    )
    .all()
)
if not deliverables:
    return 1.0
total_ratio = 0.0
for d in deliverables:
    target = max(_parse_decimal(d.target_amount), 0.001)
    current = min(_parse_decimal(d.current_amount), target)
    total_ratio += current / target
return total_ratio / len(deliverables)

@staticmethod
def update_file_id(deliverable_id: str, file_id: str) -> NodeDeliverable |
None:
    with get_session() as session:
        deliv = (
            session.query(NodeDeliverable)
            .filter(NodeDeliverable.deliverable_id == deliverable_id)
            .first()
        )
        if deliv is None:
            return None
        deliv.file_id = file_id
        session.commit()
        logger.info("NodeDeliverable file_id updated: %s -> %s",
deliverable_id, file_id or "(cleared)")
        return deliv

@staticmethod
def update_submission_status(
    deliverable_id: str,
    submission_status: str,
    submitted_by: str = "",
    confirmed_by: str = "",
    return_reason: str = "",
    attachment_file_id: str = "",
) -> NodeDeliverable | None:
    with get_session() as session:
        deliv = (
            session.query(NodeDeliverable)
            .filter(NodeDeliverable.deliverable_id == deliverable_id)
            .first()
        )
        if deliv is None:
            return None
        deliv.submission_status = submission_status
        if submitted_by:
            deliv.submitted_by = submitted_by
            deliv.submitted_at = datetime.now(timezone.utc).isoformat()
        if confirmed_by:
            deliv.confirmed_by = confirmed_by
            deliv.confirmed_at = datetime.now(timezone.utc).isoformat()
        if return_reason:

```

```

        deliv.return_reason = return_reason
    if attachment_file_id:
        deliv.attachment_file_id = attachment_file_id
    session.commit()
    logger.info("NodeDeliverable submission: %s -> %s", deliverable_id,
submission_status)
    return deliv

@staticmethod
def find_by_submission_status(
    node_id: str,
    submission_status: str,
) -> list[NodeDeliverable]:
    with get_session() as session:
        return (
            session.query(NodeDeliverable)
                .filter(
                    NodeDeliverable.node_id == node_id,
                    NodeDeliverable.submission_status == submission_status,
                )
                .all()
        )

@staticmethod
def get_by_node_and_name(
    node_id: str,
    deliverable_name: str,
) -> NodeDeliverable | None:
    with get_session() as session:
        return (
            session.query(NodeDeliverable)
                .filter(
                    NodeDeliverable.node_id == node_id,
                    NodeDeliverable.deliverable_name == deliverable_name,
                )
                .first()
        )

@staticmethod
def find_pending_by_responsible_user(
    responsible_user_id: str,
    project_id: str | None = None,
    submission_status: str = "",
    node_status: str = "IN_PROGRESS",
    limit: int = 20,
    offset: int = 0,
) -> list[tuple]:
    with get_session() as session:
        q = (
            session.query(NodeDeliverable, ProjectNode)
                .join(ProjectNode, NodeDeliverable.node_id == ProjectNode.node_id)
                .filter(
                    ProjectNode.responsible_user_id == responsible_user_id,
                    ProjectNode.is_discarded == False,
                )
        )
        if node_status:

```

```

        q = q.filter(ProjectNode.status == node_status)
    if project_id:
        q = q.filter(ProjectNode.project_id == project_id)
    if submission_status:
        q = q.filter(NodeDeliverable.submission_status ==
submission_status)
    return (
        q.order_by(ProjectNode.deadline.asc())
        .offset(offset)
        .limit(limit)
        .all()
    )
@staticmethod
def count_pending_by_responsible_user(
    responsible_user_id: str,
    project_id: str | None = None,
    submission_status: str = "",
    node_status: str = "IN_PROGRESS",
) -> int:
    with get_session() as session:
        from sqlalchemy import func
        q = (
            session.query(func.count(NodeDeliverable.id))
            .join(ProjectNode, NodeDeliverable.node_id == ProjectNode.node_id)
            .filter(
                ProjectNode.responsible_user_id == responsible_user_id,
                ProjectNode.is_discarded == False,
            )
        )
        if node_status:
            q = q.filter(ProjectNode.status == node_status)
        if project_id:
            q = q.filter(ProjectNode.project_id == project_id)
        if submission_status:
            q = q.filter(NodeDeliverable.submission_status ==
submission_status)
        return q.scalar() or 0
class NodeAccessibleFileRepo:
    @staticmethod
    def create(**kwargs) -> NodeAccessibleFile:
        with get_session() as session:
            naf = NodeAccessibleFile(**kwargs)
            session.add(naf)
            session.flush()
            logger.info("NodeAccessibleFile added: node=%s file=%s", naf.node_id,
naf.file_id)
        return naf
    @staticmethod
    def find_by_node(node_id: str) -> list[NodeAccessibleFile]:
        with get_session() as session:
            return (
                session.query(NodeAccessibleFile)
                .filter(NodeAccessibleFile.node_id == node_id)
                .all()
            )

```

```

    )
    @staticmethod
    def find_by_file(file_id: str) -> list[NodeAccessibleFile]:
        with get_session() as session:
            return (
                session.query(NodeAccessibleFile)
                .filter(NodeAccessibleFile.file_id == file_id)
                .all()
            )
    @staticmethod
    def remove(node_id: str, file_id: str) -> bool:
        with get_session() as session:
            naf = (
                session.query(NodeAccessibleFile)
                .filter(
                    NodeAccessibleFile.node_id == node_id,
                    NodeAccessibleFile.file_id == file_id,
                )
                .first()
            )
            if naf is None:
                return False
            session.delete(naf)
            session.commit()
            logger.info("NodeAccessibleFile removed: node=%s file=%s", node_id,
file_id)
            return True
    @staticmethod
    def batch_add(node_id: str, file_ids: list[str], added_by: str) -> int:
        count = 0
        with get_session() as session:
            for file_id in file_ids:
                existing = (
                    session.query(NodeAccessibleFile)
                    .filter(
                        NodeAccessibleFile.node_id == node_id,
                        NodeAccessibleFile.file_id == file_id,
                    )
                    .first()
                )
                if existing:
                    continue
                naf = NodeAccessibleFile(
                    node_id=node_id,
                    file_id=file_id,
                    added_by=added_by,
                )
                session.add(naf)
                count += 1
            session.commit()
            logger.info("NodeAccessibleFile batch_add: node=%s count=%d", node_id,
count)
            return count
    @staticmethod

```

```
def exists(node_id: str, file_id: str) -> bool:
    with get_session() as session:
        return (
            session.query(NodeAccessibleFile)
                .filter(
                    NodeAccessibleFile.node_id == node_id,
                    NodeAccessibleFile.file_id == file_id,
                )
                .first()
                is not None
        )

class NodeEventRepo:
    @staticmethod
    def create(**kwargs) -> NodeEvent:
        with get_session() as session:
            event = NodeEvent(**kwargs)
            session.add(event)
            session.flush()
            logger.info("NodeEvent created: %s type=%s node=%s", event.event_id,
event.event_type, event.node_id)
            return event

    @staticmethod
    def find_by_node(node_id: str, event_type: str | None = None, limit: int =
100) -> list[NodeEvent]:
        with get_session() as session:
            q = session.query(NodeEvent).filter(NodeEvent.node_id == node_id)
            if event_type:
                q = q.filter(NodeEvent.event_type == event_type)
            return q.order_by(NodeEvent.created_at.desc()).limit(limit).all()

    @staticmethod
    def find_by_project(project_id: str, limit: int = 200) -> list[NodeEvent]:
        with get_session() as session:
            return (
                session.query(NodeEvent)
                    .join(ProjectNode, NodeEvent.node_id == ProjectNode.node_id)
                    .filter(ProjectNode.project_id == project_id)
                    .order_by(NodeEvent.created_at.desc())
                    .limit(limit)
                    .all()
            )

    @staticmethod
    def find_by_operator(operator_id: str, limit: int = 100) -> list[NodeEvent]:
        with get_session() as session:
            return (
                session.query(NodeEvent)
                    .filter(NodeEvent.operator_id == operator_id)
                    .order_by(NodeEvent.created_at.desc())
                    .limit(limit)
                    .all()
            )
```

```

from datetime import datetime
from typing import Optional
from sqlalchemy.orm import Session
from emily_core.infrastructure.database.models import (
    BEIJING_TZ,
    PermissionGrant,
    _utc_now,
)
from emily_core.infrastructure.database.session import get_session
class PermissionGrantRepository:
    @staticmethod
    def generate_grant_no(*, session: Optional[Session] = None) -> str:
        def _impl(sess: Session) -> str:
            today = datetime.now(BEIJING_TZ).strftime("%Y%m%d")
            prefix = f"PGR-{today}-"
            count = sess.query(PermissionGrant).filter(
                PermissionGrant.grant_no.like(prefix + "%")
            ).count()
            return f"{prefix}{count + 1:04d}"
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)
    @staticmethod
    def create(grant_no: str, grantee_id: str, perm_code: str, grant_type: str, *,
               grantor_id: str = "", operations: str = '["read"]',
               expire_time: Optional[str] = None, remark: str = "", client_ip: str
               = "",
               session: Optional[Session] = None) -> PermissionGrant:
        def _impl(sess: Session) -> PermissionGrant:
            grant = PermissionGrant(
                grant_no=grant_no,
                grantee_id=grantee_id,
                grantor_id=grantor_id,
                perm_code=perm_code,
                grant_type=grant_type,
                operations=operations,
                expire_time=expire_time,
                status="ACTIVE",
                remark=remark,
                client_ip=client_ip,
            )
            sess.add(grant)
            sess.flush()
            return grant
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)
    @staticmethod
    def get_by_grant_no(grant_no: str, *, session: Optional[Session] = None) ->
    Optional[PermissionGrant]:

```



```

def _impl(sess: Session):
    return sess.query(PermissionGrant).filter(
        PermissionGrant.grant_no == grant_no,
        PermissionGrant.is_deleted == False,
    ).first()
if session is not None:
    return _impl(session)
with get_session() as sess:
    return _impl(sess)

@staticmethod
def get_active_grants(user_id: str, *, session: Optional[Session] = None) ->
list[PermissionGrant]:
    def _impl(sess: Session):
        return sess.query(PermissionGrant).filter(
            PermissionGrant.grantee_id == user_id,
            PermissionGrant.status == "ACTIVE",
            PermissionGrant.is_deleted == False,
        ).all()
    if session is not None:
        return _impl(session)
    with get_session() as sess:
        return _impl(sess)

@staticmethod
def revoke(grant_no: str, revoke_reason: str = "", *,
           session: Optional[Session] = None) -> Optional[PermissionGrant]:
    def _impl(sess: Session):
        grant = sess.query(PermissionGrant).filter(
            PermissionGrant.grant_no == grant_no,
            PermissionGrant.is_deleted == False,
        ).first()
        if grant is None:
            return None
        grant.status = "REVOKED"
        grant.revoke_reason = revoke_reason
        grant.revoke_time = _utc_now()
        sess.flush()
        return grant
    if session is not None:
        return _impl(session)
    with get_session() as sess:
        return _impl(sess)

@staticmethod
def expire_overdue(*, session: Optional[Session] = None) -> int:
    def _impl(sess: Session) -> int:
        now = _utc_now()
        rows = sess.query(PermissionGrant).filter(
            PermissionGrant.status == "ACTIVE",
            PermissionGrant.expire_time.isnot(None),
            PermissionGrant.expire_time < now,
            PermissionGrant.is_deleted == False,
        ).all()
        for g in rows:
            g.status = "EXPIRED"
        sess.flush()

```

```

        return len(rows)
    if session is not None:
        return _impl(session)
    with get_session() as sess:
        return _impl(sess)
    @staticmethod
    def cascade_revoke_above_level(user_id: str, new_level: int, *,
                                   session: Optional[Session] = None) -> int:
        def _impl(sess: Session) -> int:
            rows = sess.query(PermissionGrant).filter(
                PermissionGrant.grantee_id == user_id,
                PermissionGrant.status == "ACTIVE",
                PermissionGrant.grant_type.in_(["TEMP", "PERMANENT"]),
                PermissionGrant.is_deleted == False,
            ).all()
            count = 0
            for g in rows:
                if new_level <= 2:
                    g.status = "REVOKED"
                    g.revoke_reason = f"级联撤销：用户权限降级至 L{new_level}"
                    g.revoke_time = _utc_now()
                    count += 1
            sess.flush()
            return count
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)

```

文件编号：56 文件路径：emily-core\emily_core\repositories\permission_repo.py

```

from typing import Optional
from sqlalchemy.orm import Session
from emily_core.infrastructure.database.models import (
    CompanyInfo,
    PermissionGroup,
    PermissionGrant,
    SOPBusinessFlow,
    SOPPermissionBinding,
    User,
)
from emily_core.infrastructure.database.session import get_session
class PermissionRepository:
    @staticmethod
    def get_user(user_id: str, *, session: Optional[Session] = None) ->
Optional[User]:
        def _impl(sess: Session):
            return sess.query(User).filter(
                User.id == user_id,
                User.is_deleted == False,
            ).first()
        if session is not None:

```

```

        return _impl(session)
    with get_session() as sess:
        return _impl(sess)
    @staticmethod
    def get_company(company_id: str, *, session: Optional[Session] = None) ->
Optional[CompanyInfo]:
        def _impl(sess: Session):
            if not company_id:
                return None
            return sess.query(CompanyInfo).filter(
                CompanyInfo.id == company_id,
                CompanyInfo.is_deleted == False,
            ).first()
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)
    @staticmethod
    def get_active_grants(user_id: str, *, session: Optional[Session] = None) ->
list[PermissionGrant]:
        def _impl(sess: Session):
            return sess.query(PermissionGrant).filter(
                PermissionGrant.grantee_id == user_id,
                PermissionGrant.status == "ACTIVE",
                PermissionGrant.is_deleted == False,
            ).all()
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)
    @staticmethod
    def list_active_sop_flows(*, session: Optional[Session] = None) ->
list[SOPBusinessFlow]:
        def _impl(sess: Session):
            return sess.query(SOPBusinessFlow).filter(
                SOPBusinessFlow.is_active == True,
                SOPBusinessFlow.is_deleted == False,
            ).all()
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)
    @staticmethod
    def list_sop_bindings(*, session: Optional[Session] = None) ->
list[SOPPermissionBinding]:
        def _impl(sess: Session):
            return sess.query(SOPPermissionBinding).filter(
                SOPPermissionBinding.is_deleted == False,
            ).all()
        if session is not None:
            return _impl(session)
        with get_session() as sess:
            return _impl(sess)
    @staticmethod

```

```

def list_permission_groups(*, session: Optional[Session] = None) ->
list[PermissionGroup]:
    def _impl(sess: Session):
        return sess.query(PermissionGroup).filter(
            PermissionGroup.status == "active",
            PermissionGroup.is_deleted == False,
        ).all()
    if session is not None:
        return _impl(session)
    with get_session() as sess:
        return _impl(sess)

```

文件编号: 57 文件路径: emily-core\emily_core\repositories\scheduler_repo.py

```

from __future__ import annotations
import logging
from datetime import datetime, timezone, timedelta
from ..infrastructure.database.models import (
    SchedulerJob,
    SchedulerExecution,
)
from ..infrastructure.database.session import get_session
logger = logging.getLogger("emily.scheduler_repo")
BEIJING_TZ = timezone(timedelta(hours=8))
class SchedulerRepo:
    @staticmethod
    def generate_job_no() -> str:
        date_part = datetime.now(BEIJING_TZ).strftime("%Y%m%d")
        with get_session() as session:
            latest = (
                session.query(SchedulerJob.job_no)
                .filter(SchedulerJob.job_no.like(f"JOB-{date_part}-%"))
                .order_by(SchedulerJob.job_no.desc())
                .first()
            )
            if latest:
                seq = int(latest[0].rsplit("-", 1)[-1]) + 1
            else:
                seq = 1
            return f"JOB-{date_part}-{seq:04d}"
    @staticmethod
    def generate_execution_no() -> str:
        date_part = datetime.now(BEIJING_TZ).strftime("%Y%m%d")
        with get_session() as session:
            latest = (
                session.query(SchedulerExecution.execution_no)
                .filter(SchedulerExecution.execution_no.like(f"SE-{date_part}-%"))
                .order_by(SchedulerExecution.execution_no.desc())
                .first()
            )
            if latest:
                seq = int(latest[0].rsplit("-", 1)[-1]) + 1

```

```

        else:
            seq = 1
            return f"SE-{date_part}-{seq:04d}"
    @staticmethod
    def create_job(**kwargs) -> SchedulerJob:
        with get_session() as session:
            job = SchedulerJob(**kwargs)
            session.add(job)
            session.flush()
            logger.info("SchedulerJob created: %s", job.job_no)
            return job
    @staticmethod
    def get_job_by_id(job_id: str) -> SchedulerJob | None:
        with get_session() as session:
            return session.query(SchedulerJob).filter(SchedulerJob.id ==
job_id).first()
    @staticmethod
    def get_job_by_no(job_no: str) -> SchedulerJob | None:
        with get_session() as session:
            return session.query(SchedulerJob).filter(SchedulerJob.job_no ==
job_no).first()
    @staticmethod
    def list_jobs(status: str = "") -> list[SchedulerJob]:
        with get_session() as session:
            q = session.query(SchedulerJob)
            if status:
                q = q.filter(SchedulerJob.status == status)
            return q.order_by(SchedulerJob.created_at.desc()).all()
    @staticmethod
    def list_active_jobs() -> list[SchedulerJob]:
        with get_session() as session:
            return (
                session.query(SchedulerJob)
                .filter(SchedulerJob.status == "ACTIVE")
                .all()
            )
    @staticmethod
    def update_job_status(job_id: str, status: str) -> SchedulerJob | None:
        with get_session() as session:
            job = session.query(SchedulerJob).filter(SchedulerJob.id ==
job_id).first()
            if job is None:
                return None
            job.status = status
            job.updated_at = datetime.now(timezone.utc).isoformat()
            session.commit()
            logger.info("SchedulerJob status: %s -> %s", job_id, status)
            return job
    @staticmethod
    def update_job_next_execution(job_id: str, next_execution_at: str) -> None:
        with get_session() as session:
            job = session.query(SchedulerJob).filter(SchedulerJob.id ==
job_id).first()
            if job is None:

```

```

        return
    job.next_execution_at = next_execution_at
    job.updated_at = datetime.now(timezone.utc).isoformat()
    session.commit()

    @staticmethod
    def update_job_last_executed(job_id: str, last_executed_at: str,
                                next_execution_at: str = "") -> None:
        with get_session() as session:
            job = session.query(SchedulerJob).filter(SchedulerJob.id ==
job_id).first()
            if job is None:
                return
            job.last_executed_at = last_executed_at
            if next_execution_at:
                job.next_execution_at = next_execution_at
            job.updated_at = datetime.now(timezone.utc).isoformat()
            session.commit()

    @staticmethod
    def create_execution(**kwargs) -> SchedulerExecution:
        with get_session() as session:
            execution = SchedulerExecution(**kwargs)
            session.add(execution)
            session.flush()
            logger.info("SchedulerExecution created: %s", execution.execution_no)
            return execution

    @staticmethod
    def update_execution_status(execution_id: str, status: str,
                                error_message: str = "",
                                result_summary: str = "") -> SchedulerExecution |
None:
        with get_session() as session:
            execution = (
                session.query(SchedulerExecution)
                    .filter(SchedulerExecution.id == execution_id)
                    .first()
            )
            if execution is None:
                return None
            execution.status = status
            if status == "RUNNING":
                execution.started_at = datetime.now(timezone.utc).isoformat()
            if status in ("SUCCESS", "FAILED"):
                execution.finished_at = datetime.now(timezone.utc).isoformat()
            if error_message:
                execution.error_message = error_message
            if result_summary:
                execution.result_summary = result_summary
            session.commit()
            logger.info("SchedulerExecution status: %s -> %s", execution_id,
status)

            return execution

    @staticmethod
    def find_executions_by_job(job_id: str, limit: int = 50) ->
list[SchedulerExecution]:

```

```

        with get_session() as session:
            return (
                session.query(SchedulerExecution)
                .filter(SchedulerExecution.job_id == job_id)
                .order_by(SchedulerExecution.created_at.desc())
                .limit(limit)
                .all()
            )
    @staticmethod
    def has_period_execution(job_id: str, period_key: str) -> bool:
        if not period_key:
            return False
        with get_session() as session:
            return (
                session.query(SchedulerExecution)
                .filter(
                    SchedulerExecution.job_id == job_id,
                    SchedulerExecution.period_key == period_key,
                    SchedulerExecution.status.in_([ "SUCCESS", "RUNNING" ]),
                )
                .first()
                is not None
            )

```

文件编号: 58 文件路径: emily-core\emily_core\repositories\session_accessible_file_repo.py

```

from __future__ import annotations
import logging
from datetime import datetime, timezone
from typing import Optional
from ..infrastructure.database import get_session
from ..infrastructure.database.models import SessionAccessibleFile, File,
ProjectNode, NodeAccessibleFile
from sqlalchemy import or_, and_
logger = logging.getLogger("emily.repo.accessible_file")
class SessionAccessibleFileRepo:
    @staticmethod
    def sync_for_user(
        user_id: str,
        project_ids: list[str],
        info_level: str = "public",
        authorized_node_ids: Optional[list[str]] = None,
    ) -> int:
        authorized_node_ids = authorized_node_ids or []
        info_level_map = {"public": 0, "internal": 1, "confidential": 2, "secret":
3}
        max_conf = info_level_map.get(info_level, 0)
        total = 0
        try:
            now = _now_iso()
            with get_session() as session:
                session.query(SessionAccessibleFile).filter(

```

```

        SessionAccessibleFile.user_id == user_id,
        SessionAccessibleFile.access_type.in_(["project_scope",
"node_linked"])),
    ).delete(synchronize_session=False)
    if project_ids:
        files = session.query(File).filter(
            File.project_id.in_(project_ids),
            File.is_deleted == False,
            File.confidentiality <= max_conf,
        ).all()
        for f in files:
            saf = SessionAccessibleFile(
                user_id=user_id,
                file_id=f.id,
                access_type="project_scope",
                granted_by="system",
                granted_at=now,
            )
            session.add(saf)
            total += 1
    if authorized_node_ids:
        nodes = session.query(ProjectNode).filter(
            ProjectNode.node_id.in_(authorized_node_ids)
        ).all()
        all_project_nodes = [n for n in nodes if n.visibility_mode ==
"all_project_files"]
        specific_nodes = [n for n in nodes if n.visibility_mode !=
"all_project_files"]
        if all_project_nodes:
            all_project_project_ids = list({n.project_id for n in
all_project_nodes if n.project_id})
            if all_project_project_ids:
                already_visible =
session.query(SessionAccessibleFile.file_id).filter(
                    SessionAccessibleFile.user_id == user_id,
                ).subquery()
                extra_files = session.query(File).filter(
                    File.project_id.in_(all_project_project_ids),
                    File.is_deleted == False,
                    ~File.id.in_(already_visible),
                ).all()
                for f in extra_files:
                    saf = SessionAccessibleFile(
                        user_id=user_id,
                        file_id=f.id,
                        access_type="node_linked",
                        granted_by="system",
                        granted_at=now,
                    )
                    session.add(saf)
                    total += 1
    if specific_nodes:
        specific_node_ids = [n.node_id for n in specific_nodes]
        node_file_links =

```



```

session.query(NodeAccessibleFile).filter(
    NodeAccessibleFile.node_id.in_(specific_node_ids)
).all()
for link in node_file_links:
    existing =
session.query(SessionAccessibleFile).filter(
    SessionAccessibleFile.user_id == user_id,
    SessionAccessibleFile.file_id == link.file_id,
).first()
if not existing:
    saf = SessionAccessibleFile(
        user_id=user_id,
        file_id=link.file_id,
        access_type="node_linked",
        granted_by=link.added_by or "system",
        granted_at=link.added_at or now,
    )
    session.add(saf)
    total += 1
session.commit()
logger.info("sync_for_user(%s): %d files synced", user_id, total)
return total
except Exception as e:
    logger.error("sync_for_user(%s) failed: %s", user_id, e)
    return total

@staticmethod
def get_file_summary(user_id: str) -> dict:
    try:
        with get_session() as session:
            rows = session.query(
                SessionAccessibleFile.file_id,
            ).filter(
                SessionAccessibleFile.user_id == user_id,
            ).all()
            file_ids = [r.file_id for r in rows]
            if not file_ids:
                return {"count": 0, "by_type": {}, "by_category": {}, "files":
[]}

            files = session.query(File).filter(
                File.id.in_(file_ids),
                File.is_deleted == False,
            ).all()
            by_type: dict[str, int] = {}
            by_category: dict[str, int] = {}
            files_list: list[dict] = []
            for f in files:
                ft = f.file_type or "其他"
                by_type[ft] = by_type.get(ft, 0) + 1
                cat = getattr(f, 'file_category', 'OTHER') or 'OTHER'
                by_category[cat] = by_category.get(cat, 0) + 1
                files_list.append({
                    "file_id": f.id,
                    "filename": f.filename,
                    "file_type": f.file_type or "",

```

```

        "project_id": f.project_id or "",
        "file_category": getattr(f, 'file_category', 'OTHER') or
'OTHER',
    })
    return {"count": len(files), "by_type": by_type, "by_category":
by_category, "files": files_list}
except Exception as e:
    logger.error("get_file_summary(%s) failed: %s", user_id, e)
    return {"count": 0, "by_type": {}, "by_category": {}, "files": []}
@staticmethod
def search(user_id: str, keyword: str, top_k: int = 5) -> list[dict]:
    keywords = [k.strip() for k in keyword.replace(" ", " ").split(" ") if
k.strip()]
    if not keywords:
        return []
    try:
        with get_session() as session:
            accessible = session.query(SessionAccessibleFile.file_id).filter(
                SessionAccessibleFile.user_id == user_id
            ).subquery()
            query = session.query(File).filter(
                File.id.in_(accessible),
                File.is_deleted == False,
            )
            conditions = []
            for kw in keywords:
                conditions.append(File.filename.ilike(f"%{kw}%"))
                conditions.append(File.file_type.ilike(f"%{kw}%"))
            files = query.filter(or_(*conditions)).limit(top_k * 2).all()
            scored = []
            for f in files:
                score = 0
                for kw in keywords:
                    if kw.lower() in (f.filename or "").lower():
                        score += 10
                    if kw.lower() in (f.file_type or "").lower():
                        score += 3
                scored.append({
                    "file_id": f.id,
                    "filename": f.filename or "",
                    "file_type": f.file_type or "",
                    "description": f.file_type or "",
                    "score": score,
                })
            scored.sort(key=lambda x: x["score"], reverse=True)
            return scored[:top_k]
    except Exception as e:
        logger.error("search(%s, %s) failed: %s", user_id, keyword, e)
        return []
@staticmethod
def add_explicit_grant(user_id: str, file_id: str, granted_by: str = "admin")
-> bool:
    try:
        now = _now_iso()

```

```

        with get_session() as session:
            existing = session.query(SessionAccessibleFile).filter(
                SessionAccessibleFile.user_id == user_id,
                SessionAccessibleFile.file_id == file_id,
            ).first()
            if existing:
                existing.access_type = "explicit"
                existing.granted_by = granted_by
                existing.granted_at = now
                existing.expires_at = ""
            else:
                saf = SessionAccessibleFile(
                    user_id=user_id,
                    file_id=file_id,
                    access_type="explicit",
                    granted_by=granted_by,
                    granted_at=now,
                )
                session.add(saf)
                session.commit()
            return True
    except Exception as e:
        logger.error("add_explicit_grant(%s, %s) failed: %s", user_id,
file_id, e)
        return False

    @staticmethod
    def revoke(user_id: str, file_id: str) -> bool:
        try:
            with get_session() as session:
                deleted = session.query(SessionAccessibleFile).filter(
                    SessionAccessibleFile.user_id == user_id,
                    SessionAccessibleFile.file_id == file_id,
                ).delete()
                session.commit()
            return deleted > 0
        except Exception as e:
            logger.error("revoke(%s, %s) failed: %s", user_id, file_id, e)
            return False

    def _now_iso() -> str:
        return datetime.now(timezone.utc).isoformat()

```

文件编号: 59 文件路径: emily-core\emily_core\repositories\session_archive_repo.py

```

import logging
from typing import Optional
from ..infrastructure.database.session import get_session
from ..infrastructure.database.models import SessionArchive
logger = logging.getLogger("emily.repo.session_archive")
class SessionArchiveRepo:
    @staticmethod
    def create(
        *,

```

```

        conversation_id: str,
        user_id: Optional[str] = None,
        user_name: str = "",
        turn_count: int = 0,
        message_history_snapshot: str = "",
        context_snapshot: str = "",
        started_at: Optional[str] = None,
        archive_reason: str = "expired",
    ) -> SessionArchive:
        with get_session() as session:
            archive = SessionArchive(
                conversation_id=conversation_id,
                user_id=user_id,
                user_name=user_name,
                turn_count=turn_count,
                message_history_snapshot=message_history_snapshot,
                context_snapshot=context_snapshot,
                started_at=started_at,
                archive_reason=archive_reason,
            )
            session.add(archive)
            session.flush()
            logger.info(
                "SessionArchive created: conv=%s user=%s turns=%d reason=%s",
                conversation_id, user_id or "?", turn_count, archive_reason,
            )
            return archive

    @staticmethod
    def get_by_conversation_id(conversation_id: str) -> Optional[SessionArchive]:
        with get_session() as session:
            return (
                session.query(SessionArchive)
                .filter(SessionArchive.conversation_id == conversation_id)
                .order_by(SessionArchive.archived_at.desc())
                .first()
            )

    @staticmethod
    def list_by_user(user_id: str, limit: int = 20) -> list[SessionArchive]:
        with get_session() as session:
            return (
                session.query(SessionArchive)
                .filter(SessionArchive.user_id == user_id)
                .order_by(SessionArchive.archived_at.desc())
                .limit(limit)
                .all()
            )

```

文件编号: 60 文件路径: emily-core\emily_core\repositories\system_description_repo.py

```

from __future__ import annotations
import json
import logging

```

```

from typing import Optional
from ..infrastructure.database.models import SystemDescription, _utc_now
from ..infrastructure.database.session import get_session
logger = logging.getLogger("emily.system_description_repo")
class SystemDescriptionRepo:
    @staticmethod
    def create(
        content_json: str = "{}",
        content_text: str = "",
        domain_versions: str = "{}",
        schema_hash: str = "",
        permission_hash: str = "",
        file_model_hash: str = "",
        token_count: int = 0,
        generated_by: str = "manual",
    ) -> SystemDescription:
        with get_session() as session:
            desc = SystemDescription(
                content_json=content_json,
                content_text=content_text,
                domain_versions=domain_versions,
                schema_hash=schema_hash,
                permission_hash=permission_hash,
                file_model_hash=file_model_hash,
                token_count=token_count,
                generated_by=generated_by,
            )
            session.add(desc)
            session.flush()
            logger.info("SystemDescription created: version=%d generated_by=%s",
desc.version, generated_by)
            return desc
    @staticmethod
    def get_latest() -> Optional[SystemDescription]:
        with get_session() as session:
            return (
                session.query(SystemDescription)
                    .order_by(SystemDescription.version.desc())
                    .first()
            )
    @staticmethod
    def get_by_id(desc_id: str) -> Optional[SystemDescription]:
        with get_session() as session:
            return session.query(SystemDescription).filter(SystemDescription.id ==
desc_id).first()
    @staticmethod
    def update_content(
        content_json: str = None,
        content_text: str = None,
        domain_versions: str = None,
        version: int = None,
        schema_hash: str = None,
        permission_hash: str = None,
        file_model_hash: str = None,

```

```

        token_count: int = None,
        generated_by: str = None,
    ) -> Optional[SystemDescription]:
        with get_session() as session:
            desc = (
                session.query(SystemDescription)
                .order_by(SystemDescription.version.desc())
                .first()
            )
            if desc is None:
                return None
            if content_json is not None:
                desc.content_json = content_json
            if content_text is not None:
                desc.content_text = content_text
            if domain_versions is not None:
                desc.domain_versions = domain_versions
            if version is not None:
                desc.version = version
            if schema_hash is not None:
                desc.schema_hash = schema_hash
            if permission_hash is not None:
                desc.permission_hash = permission_hash
            if file_model_hash is not None:
                desc.file_model_hash = file_model_hash
            if token_count is not None:
                desc.token_count = token_count
            if generated_by is not None:
                desc.generated_by = generated_by
            desc.updated_at = _utc_now()
            session.flush()
            logger.info("SystemDescription updated: version=%d", desc.version)
            return desc

    @staticmethod
    def list_all() -> list[SystemDescription]:
        with get_session() as session:
            return
            session.query(SystemDescription).order_by(SystemDescription.version.desc()).all()

```

文件编号: 61 文件路径: emily-core\emily_core\repositories\task_repo.py

```

import logging
from datetime import datetime, timezone, timedelta
from typing import Optional
from ..infrastructure.database.session import get_session
from ..infrastructure.database.models import Task
logger = logging.getLogger("emily.repo.task")
class TaskRepository:
    @staticmethod
    def generate_task_no() -> str:
        today_str = datetime.now(timezone.utc).strftime("%Y%m%d")
        prefix = f"TSK-{today_str}-"

```

```

        with get_session() as session:
            last = (
                session.query(Task)
                .filter(Task.task_no.like(f"{prefix}%"))
                .order_by(Task.task_no.desc())
                .first()
            )
            if last is None:
                return f"{prefix}0001"
            seq_str = last.task_no[len(prefix):]
            try:
                seq = int(seq_str) + 1
            except ValueError:
                seq = 1
            return f"{prefix}{seq:04d}"

    @staticmethod
    def create(
        *,
        task_no: str,
        title: str,
        project_id: Optional[str] = None,
        source_message_id: Optional[str] = None,
        description: Optional[str] = None,
        owner_id: Optional[str] = None,
        owner_text: Optional[str] = None,
        status: str = "todo",
        due_date: Optional[str] = None,
        due_text: Optional[str] = None,
        created_by: Optional[str] = None,
    ) -> Task:
        with get_session() as session:
            task = Task(
                task_no=task_no,
                project_id=project_id,
                source_message_id=source_message_id,
                title=title,
                description=description,
                owner_id=owner_id,
                owner_text=owner_text,
                status=status,
                due_date=due_date,
                due_text=due_text,
                created_by=created_by,
            )
            session.add(task)
            session.flush()
            logger.info("Task created: no=%s, title=%s", task_no, title)
            return task

    @staticmethod
    def get_by_id(task_id: str) -> Optional[Task]:
        with get_session() as session:
            return session.query(Task).filter(Task.id == task_id).first()

    @staticmethod
    def get_by_task_no(task_no: str) -> Optional[Task]:

```

```

        with get_session() as session:
            return session.query(Task).filter(Task.task_no == task_no).first()
    @staticmethod
    def query_tasks(
        *,
        project_id: str | None = None,
        project_ids: list[str] | None = None,
        time_range: str = "all",
        status: str | None = None,
        assignee: str | None = None,
        limit: int = 50,
    ) -> list[Task]:
        with get_session() as session:
            q = session.query(Task)
            if project_ids:
                q = q.filter(Task.project_id.in_(project_ids))
            elif project_id:
                q = q.filter(Task.project_id == project_id)
            if status:
                q = q.filter(Task.status == status)
            if assignee:
                q = q.filter(Task.owner_text.like(f"%{assignee}%"))
            if time_range != "all":
                now = datetime.now(timezone.utc)
                start = _resolve_time_start(time_range, now)
                if start:
                    q = q.filter(Task.created_at >= start.isoformat())
            q = q.order_by(Task.created_at.desc()).limit(limit)
            return q.all()
    @staticmethod
    def count_by_status(project_id: str | None = None) -> dict[str, int]:
        with get_session() as session:
            q = session.query(Task)
            if project_id:
                q = q.filter(Task.project_id == project_id)
            rows = q.all()
            counts = {}
            for row in rows:
                s = row.status or "unknown"
                counts[s] = counts.get(s, 0) + 1
            return counts
    def _resolve_time_start(time_range: str, now) -> Optional[datetime]:
        if time_range == "today":
            return now.replace(hour=0, minute=0, second=0, microsecond=0)
        elif time_range == "this_week":
            start = now - timedelta(days=now.weekday())
            return start.replace(hour=0, minute=0, second=0, microsecond=0)
        elif time_range == "this_month":
            return now.replace(day=1, hour=0, minute=0, second=0, microsecond=0)
        return None

```



```

import logging
from typing import Optional
from ..infrastructure.database.session import get_session
from ..infrastructure.database.models import ToolCallLog
logger = logging.getLogger("emily.repo.tool_call")
class ToolCallRepository:
    @staticmethod
    def create(
        reasoning_log_id: str,
        llm_interaction_id: str,
        step_index: int,
        tool_name: str,
        tool_arguments: dict,
    ) -> str:
        from ..infrastructure.database.models import _new_uuid
        import json
        log_id = _new_uuid()
        try:
            args_json = json.dumps(tool_arguments, ensure_ascii=False)
        except Exception:
            args_json = str(tool_arguments)
        with get_session() as session:
            log = ToolCallLog(
                id=log_id,
                reasoning_log_id=reasoning_log_id,
                llm_interaction_id=llm_interaction_id,
                step_index=step_index,
                tool_name=tool_name,
                tool_arguments=args_json,
            )
            session.add(log)
            session.flush()
            logger.debug("ToolCallLog created: %s (%s)", log_id, tool_name)
            return log_id
    @staticmethod
    def update_result(
        tool_call_log_id: str,
        result_summary: str = "",
        is_success: bool = True,
        error_message: str = "",
        elapsed_ms: int = 0,
    ) -> None:
        with get_session() as session:
            log = session.query(ToolCallLog).filter(
                ToolCallLog.id == tool_call_log_id
            ).first()
            if log is None:
                logger.warning("ToolCallLog not found: %s", tool_call_log_id)
                return
            log.tool_result_summary = (result_summary or "")[:500]
            log.is_success = is_success
            log.error_message = (error_message or "")[:500]

```

```

        log.elapsed_ms = elapsed_ms
    @staticmethod
    def get_by_reasoning_id(reasoning_log_id: str) -> list[dict]:
        with get_session() as session:
            rows = (
                session.query(ToolCallLog)
                .filter(ToolCallLog.reasoning_log_id == reasoning_log_id)
                .order_by(ToolCallLog.step_index)
                .all()
            )
            return [
                {
                    "id": r.id,
                    "llm_interaction_id": r.llm_interaction_id,
                    "step_index": r.step_index,
                    "tool_name": r.tool_name,
                    "tool_arguments": r.tool_arguments,
                    "tool_result_summary": r.tool_result_summary,
                    "is_success": r.is_success,
                    "error_message": r.error_message,
                    "elapsed_ms": r.elapsed_ms,
                    "created_at": str(r.created_at) if r.created_at else None,
                }
                for r in rows
            ]

```

文件编号: 63 文件路径: emily-core\emily_core\repositories\tool_registry_repo.py

```

from __future__ import annotations
import json
import logging
from typing import Optional
from ..infrastructure.database import get_session
from ..infrastructure.database.models import ToolRegistryModel
logger = logging.getLogger("emily.repo.tool_registry")
class ToolRegistryRepo:
    @staticmethod
    def upsert(
        api_id: str,
        signature: str = "{}",
        display_name: str = "",
        category: str = "base",
        permission_flag: str = "all",
        handler_module: str = "",
    ) -> bool:
        try:
            now = _now_iso()
            with get_session() as session:
                existing = session.query(ToolRegistryModel).filter(
                    ToolRegistryModel.id == api_id
                ).first()
                if existing:

```

```
        existing.signature = signature
        existing.display_name = display_name
        existing.category = category
        existing.permission_flag = permission_flag
        existing.handler_module = handler_module
        existing.updated_at = now
    else:
        row = ToolRegistryModel(
            id=api_id,
            signature=signature,
            display_name=display_name,
            category=category,
            permission_flag=permission_flag,
            handler_module=handler_module,
            is_active=True,
            registered_at=now,
            updated_at=now,
        )
        session.add(row)
        session.commit()
        return True
    except Exception as e:
        logger.error("ToolRegistryRepo.upsert(%s) failed: %s", api_id, e)
        return False

@staticmethod
def get_available(
    level: int = 0,
    sop_allow: Optional[list[str]] = None,
) -> list[dict]:
    try:
        with get_session() as session:
            query = session.query(ToolRegistryModel).filter(
                ToolRegistryModel.is_active == True
            )
            rows = query.all()
            available: list[dict] = []
            for row in rows:
                if row.category == "base":
                    available.append(_row_to_dict(row))
                    continue
                if row.category == "project":
                    if level >= 5:
                        available.append(_row_to_dict(row))
                        continue
                if row.permission_flag == "all":
                    available.append(_row_to_dict(row))
                elif row.permission_flag == "admin" and level >= 5:
                    available.append(_row_to_dict(row))
                elif row.permission_flag == "write" and level >= 3:
                    available.append(_row_to_dict(row))
            return available
    except Exception as e:
        logger.error("ToolRegistryRepo.get_available failed: %s", e)
    return []
```

```
@staticmethod
def get_all_active() -> list[dict]:
    try:
        with get_session() as session:
            rows = session.query(ToolRegistryModel).filter(
                ToolRegistryModel.is_active == True
            ).all()
            return [_row_to_dict(row) for row in rows]
    except Exception as e:
        logger.error("ToolRegistryRepo.get_all_active failed: %s", e)
        return []

@staticmethod
def deactivate(api_id: str) -> bool:
    try:
        with get_session() as session:
            row = session.query(ToolRegistryModel).filter(
                ToolRegistryModel.id == api_id
            ).first()
            if row:
                row.is_active = False
                row.updated_at = _now_iso()
                session.commit()
                return True
            return False
    except Exception as e:
        logger.error("ToolRegistryRepo.deactivate(%s) failed: %s", api_id, e)
        return False

@staticmethod
def get_all() -> list[dict]:
    try:
        with get_session() as session:
            rows = session.query(ToolRegistryModel).all()
            return [_row_to_dict(row) for row in rows]
    except Exception as e:
        logger.error("ToolRegistryRepo.get_all failed: %s", e)
        return []

def _row_to_dict(row: ToolRegistryModel) -> dict:
    return {
        "api_id": row.id,
        "display_name": row.display_name,
        "signature": row.signature,
        "category": row.category,
        "permission_flag": row.permission_flag,
        "handler_module": row.handler_module,
        "is_active": row.is_active,
        "registered_at": row.registered_at,
        "updated_at": row.updated_at,
    }

def _now_iso() -> str:
    from datetime import datetime, timezone
    return datetime.now(timezone.utc).isoformat()
```

```
import logging
from typing import Tuple
from ..infrastructure.database.session import get_session
from ..infrastructure.database.models import User, UserImBinding
logger = logging.getLogger("emily.repo.user")
class UserRepository:
    @staticmethod
    def get_by_im(im_platform: str, im_user_id: str) -> User | None:
        with get_session() as session:
            binding = (
                session.query(UserImBinding)
                .filter(
                    UserImBinding.im_platform == im_platform,
                    UserImBinding.im_user_id == im_user_id,
                    UserImBinding.status == "active",
                )
                .first()
            )
            if binding:
                return session.query(User).filter(User.id ==
binding.user_id).first()
            return None
    @staticmethod
    def create_user_and_bind(
        im_platform: str,
        im_user_id: str,
        im_display_name: str | None = None,
    ) -> Tuple[User, UserImBinding]:
        with get_session() as session:
            user = User(
                username=im_display_name or im_user_id,
            )
            session.add(user)
            session.flush()
            binding = UserImBinding(
                user_id=user.id,
                im_platform=im_platform,
                im_user_id=im_user_id,
                im_display_name=im_display_name,
            )
            session.add(binding)
            session.flush()
            logger.info(
                "Created user %s (%s) and bound to %s/%s",
                user.id, user.username, im_platform, im_user_id,
            )
            return user, binding
    @staticmethod
    def get(user_id: str) -> User | None:
        with get_session() as session:
            return session.query(User).filter(User.id == user_id).first()
    @staticmethod
```

```

def get_by_id(user_id: str) -> User | None:
    return UserRepository.get(user_id)
@staticmethod
def find_by_name(name: str) -> User | None:
    if not name:
        return None
    with get_session() as session:
        return (
            session.query(User)
                .filter(User.username == name, User.is_deleted == False)
                .first()
        )
@staticmethod
def update_user(user_id: str, **kwargs) -> User | None:
    with get_session() as session:
        user = session.query(User).filter(User.id == user_id).first()
        if user:
            for k, v in kwargs.items():
                if hasattr(user, k):
                    setattr(user, k, v)
            return user
@staticmethod
def get_binding(im_platform: str, im_user_id: str) -> UserImBinding | None:
    with get_session() as session:
        return (
            session.query(UserImBinding)
                .filter(
                    UserImBinding.im_platform == im_platform,
                    UserImBinding.im_user_id == im_user_id,
                )
                .first()
        )
@staticmethod
def list_users(status: str = "active", limit: int = 50) -> list[User]:
    with get_session() as session:
        q = session.query(User)
        if status:
            q = q.filter(User.status == status)
        return q.order_by(User.created_at.desc()).limit(limit).all()
@staticmethod
def count_users(status: str = "active") -> int:
    with get_session() as session:
        q = session.query(User)
        if status:
            q = q.filter(User.status == status)
        return q.count()

```

文件编号: 65 文件路径: emily-core\emily_core\repositories\world_book_repo.py

```

from __future__ import annotations
import json
import logging

```

```

from typing import Optional
from ..infrastructure.database.models import ProjectWorldBook
from ..infrastructure.database.session import get_session
logger = logging.getLogger("emily.world_book_repo")
class ProjectWorldBookRepo:
    @staticmethod
    def create(
        project_id: str,
        content_json: str = "{}",
        content_text: str = "",
        layer_versions: str = "{}",
        initialization_tier: int = 0,
        initialization_status: str = "{}",
        is_activated: bool = False,
        token_count: int = 0,
        generated_by: str = "manual",
    ) -> ProjectWorldBook:
        with get_session() as session:
            wb = ProjectWorldBook(
                project_id=project_id,
                content_json=content_json,
                content_text=content_text,
                layer_versions=layer_versions,
                initialization_tier=initialization_tier,
                initialization_status=initialization_status,
                is_activated=is_activated,
                token_count=token_count,
                generated_by=generated_by,
            )
            session.add(wb)
            session.flush()
            logger.info("ProjectWorldBook created: project=%s tier=%d",
project_id, initialization_tier)
            return wb

    @staticmethod
    def get_by_project(project_id: str) -> Optional[ProjectWorldBook]:
        with get_session() as session:
            return (
                session.query(ProjectWorldBook)
                .filter(ProjectWorldBook.project_id == project_id)
                .first()
            )

    @staticmethod
    def get_by_id(wb_id: str) -> Optional[ProjectWorldBook]:
        with get_session() as session:
            return session.query(ProjectWorldBook).filter(ProjectWorldBook.id ==
wb_id).first()

    @staticmethod
    def update_content(
        project_id: str,
        content_json: str = None,
        content_text: str = None,
        layer_versions: str = None,
        version: int = None,

```

```

        initialization_tier: int = None,
        initialization_status: str = None,
        is_activated: bool = None,
        token_count: int = None,
        generated_by: str = None,
    ) -> Optional[ProjectWorldBook]:
        from ..infrastructure.database.models import _utc_now
        with get_session() as session:
            wb = (
                session.query(ProjectWorldBook)
                .filter(ProjectWorldBook.project_id == project_id)
                .first()
            )
            if wb is None:
                return None
            if content_json is not None:
                wb.content_json = content_json
            if content_text is not None:
                wb.content_text = content_text
            if layer_versions is not None:
                wb.layer_versions = layer_versions
            if version is not None:
                wb.version = version
            if initialization_tier is not None:
                wb.initialization_tier = initialization_tier
            if initialization_status is not None:
                wb.initialization_status = initialization_status
            if is_activated is not None:
                wb.is_activated = is_activated
            if token_count is not None:
                wb.token_count = token_count
            if generated_by is not None:
                wb.generated_by = generated_by
            wb.updated_at = _utc_now()
            session.flush()
            logger.info("ProjectWorldBook updated: project=%s version=%d",
project_id, wb.version)
            return wb

    @staticmethod
    def list_all() -> list[ProjectWorldBook]:
        with get_session() as session:
            return session.query(ProjectWorldBook).all()

    @staticmethod
    def delete_by_project(project_id: str) -> bool:
        with get_session() as session:
            deleted = (
                session.query(ProjectWorldBook)
                .filter(ProjectWorldBook.project_id == project_id)
                .delete()
            )
            logger.info("ProjectWorldBook deleted: project=%s count=%d",
project_id, deleted)
            return deleted > 0

```


文件编号: 66 文件路径: emily-core\emily_core\scheduler\application.py

```
from __future__ import annotations
import logging
from .commands import (
    CreateJobCommand,
    ActivateJobCommand,
    TriggerJobCommand,
    SchedulerOperationResult,
)
from .service import SchedulerService
logger = logging.getLogger("emily.scheduler.application")
class SchedulerApplication:
    def __init__(self, service: SchedulerService, engine=None):
        self._service = service
        self._engine = engine
    async def create_job(self, cmd: CreateJobCommand) -> dict:
        result = await self._service.create_job(cmd)
        return {"success": result.success, "job_id": result.job_id, "reply":
result.message}
    async def activate_job(self, cmd: ActivateJobCommand) -> dict:
        result = await self._service.activate_job(cmd)
        return {"success": result.success, "reply": result.message}
    async def trigger_job(self, cmd: TriggerJobCommand) -> dict:
        if self._engine is None:
            return {"success": False, "reply": "调度引擎未初始化"}
        try:
            await self._engine.trigger_job(cmd.job_id)
            return {"success": True, "reply": "已触发作业执行"}
        except Exception as e:
            return {"success": False, "reply": f"触发失败: {e}"}
    async def get_job_status(self, job_id: str) -> dict:
        job = await self._service.get_job(job_id)
        if job is None:
            return {"success": False, "reply": "作业不存在"}
        return {
            "success": True,
            "data": {
                "job_no": job.job_no,
                "name": job.name,
                "status": job.status,
                "action_type": job.action_type,
                "last_executed_at": job.last_executed_at,
                "next_execution_at": job.next_execution_at,
            },
        }
    async def list_jobs(self, status: str = "") -> dict:
        jobs = await self._service.list_jobs(status)
        return {
            "success": True,
            "data": [
```

```

        {
            "job_no": j.job_no,
            "name": j.name,
            "status": j.status,
            "action_type": j.action_type,
            "job_type": j.job_type,
            "enabled": j.status == "ACTIVE",
        }
        for j in jobs
    ],
}

```

文件编号: 67 文件路径: emily-core\emily_core\scheduler\commands.py

```

from dataclasses import dataclass, field
@dataclass
class CreateJobCommand:
    name: str
    action_type: str
    handler_module: str
    job_type: str = "ONCE"
    cron_expression: str = ""
    interval_seconds: int = 0
    deadline_rule: str = ""
    action_params: str = "{}"
    creator_id: str = ""
@dataclass
class ActivateJobCommand:
    job_id: str
    activate: bool = True
    operator_id: str = ""
@dataclass
class TriggerJobCommand:
    job_id: str
    operator_id: str = ""
@dataclass
class SchedulerOperationResult:
    success: bool = True
    job_id: str = ""
    execution_no: str = ""
    message: str = ""
    error_code: str = ""

```

文件编号: 68 文件路径: emily-core\emily_core\scheduler\engine.py

```

from __future__ import annotations
import asyncio
import json
import logging

```

```

from datetime import datetime, timezone
from dataclasses import dataclass
from .handler_registry import JobHandlerRegistry, JobResult
from .hook_registry import SchedulerHookRegistry, HookDecision
logger = logging.getLogger("emily.scheduler.engine")
@dataclass
class SchedulerContext:
    job_id: str = ""
    job_no: str = ""
    action_type: str = ""
    action_params: dict = None
    execution_id: str = ""
    execution_no: str = ""
    period_key: str = ""
    result: JobResult = None
    def __post_init__(self):
        if self.action_params is None:
            self.action_params = {}
class SchedulerEngine:
    def __init__(
        self,
        service,
        handler_registry: JobHandlerRegistry,
        hook_registry: SchedulerHookRegistry,
        config,
        outbound_bus=None,
    ):
        self._service = service
        self._handlers = handler_registry
        self._hooks = hook_registry
        self._config = config
        self._outbound = outbound_bus
        self._task: asyncio.Task | None = None
        self._running = False
    async def start(self):
        if not getattr(self._config, 'scheduler_enabled', True):
            logger.info("SchedulerEngine: disabled by config")
            return
        self._running = True
        self._task = asyncio.create_task(self._loop())
        tick = getattr(self._config, 'scheduler_tick_seconds', 60)
        logger.info("SchedulerEngine: started (tick=%ds)", tick)
    async def stop(self):
        self._running = False
        if self._task:
            self._task.cancel()
            try:
                await self._task
            except asyncio.CancelledError:
                pass
        logger.info("SchedulerEngine: stopped")
    async def _loop(self):
        while self._running:
            try:

```

```

        await self._tick()
    except asyncio.CancelledError:
        break
    except Exception as e:
        logger.error("Scheduler tick failed: %s", e, exc_info=True)
        tick_seconds = getattr(self._config, 'scheduler_tick_seconds', 60)
        await asyncio.sleep(tick_seconds)
async def _tick(self):
    from ..infrastructure.database.session import get_session_raw
    from sqlalchemy import text
    lock_key = "scheduler_engine:global_tick"
    lock_session = get_session_raw()
    try:
        acquired = lock_session.execute(
            text("SELECT pg_try_advisory_lock(hashtext(:key))"),
            {"key": lock_key},
        ).scalar()
        if not acquired:
            return
        try:
            active_jobs = await self._service.list_active_jobs()
            now = datetime.now(timezone.utc)
            for job in active_jobs:
                if self._is_due(job, now):
                    await self._execute_job(job)
        finally:
            try:
                lock_session.execute(
                    text("SELECT pg_advisory_unlock(hashtext(:key))"),
                    {"key": lock_key},
                )
            except Exception:
                pass
    finally:
        lock_session.close()
def _is_due(self, job, now: datetime) -> bool:
    next_exec = getattr(job, 'next_execution_at', '')
    if not next_exec:
        return True
    try:
        next_dt = datetime.fromisoformat(next_exec.replace("Z", "+00:00"))
        return now >= next_dt
    except (ValueError, AttributeError):
        return True
async def _execute_job(self, job):
    period_key = self._calc_period_key(job)
    ctx = SchedulerContext(
        job_id=job.id,
        job_no=job.job_no,
        action_type=job.action_type,
        action_params=json.loads(getattr(job, 'action_params', '{}') or '{}'),
        period_key=period_key,
    )
    exec_result = await self._service.create_execution(job.id, period_key)

```

```

    if exec_result.success:
        ctx.execution_no = exec_result.execution_no
    if not await self._fire_before_hooks(ctx):
        logger.info("Job %s blocked by before:execute hook", job.job_no)
        return
    handler = self._handlers.get(job.action_type)
    if handler is None:
        logger.error("No handler for action_type=%s", job.action_type)
        return
    try:
        result = await handler.execute(ctx.action_params)
        ctx.result = result
    except Exception as e:
        logger.error("Handler %s failed: %s", job.action_type, e,
exc_info=True)
        ctx.result = JobResult(success=False, summary=str(e))
        await self._fire_error_hooks(ctx, e)
        await self._fire_after_hooks(ctx)
    try:
        from ..infrastructure.logging.scheduler_logger import
SchedulerJobLogger
        from datetime import datetime, timezone
        now = datetime.now(timezone.utc).isoformat()
        SchedulerJobLogger.log_sync(
            job_id=job.id,
            action_type=job.action_type,
            params_json=getattr(job, 'action_params', '{}') or '{}',
            success=ctx.result.success if ctx.result else False,
            summary=(ctx.result.summary or "")[:500] if ctx.result else "",
            elapsed_ms=0,
            error_detail="" if (ctx.result and ctx.result.success) else
(ctx.result.summary if ctx.result else ""),
            started_at="",
            completed_at=now,
        )
    except Exception as e:
        logger.warning("Scheduler job log write failed: %s", e)
    async def trigger_job(self, job_id: str):
        job = await self._service.get_job(job_id)
        if job is None:
            raise ValueError(f"作业 {job_id} 不存在")
        await self._execute_job(job)
    async def _fire_before_hooks(self, ctx: SchedulerContext) -> bool:
        for hook in self._hooks.get_enabled("before:execute"):
            try:
                result = await hook.execute(ctx)
                if result.is_blocked:
                    logger.info("Blocked by hook '%s': %s", hook.name,
result.message)
                    return False
            except Exception as e:
                logger.error("Before hook '%s' failed: %s", hook.name, e)
                return False
        return True

```

```

    async def _fire_after_hooks(self, ctx: SchedulerContext) -> None:
        for hook in self._hooks.get_enabled("after:execute"):
            try:
                await hook.execute(ctx)
            except Exception as e:
                logger.warning("After hook '%s' failed: %s", hook.name, e)
    async def _fire_error_hooks(self, ctx: SchedulerContext, error: Exception) ->
None:
        for hook in self._hooks.get_enabled("on_error:execute"):
            try:
                await hook.execute(ctx)
            except Exception as e:
                logger.error("Error hook '%s' also failed: %s", hook.name, e)
    @staticmethod
    def _calc_period_key(job) -> str:
        now = datetime.now(timezone.utc)
        job_type = getattr(job, 'job_type', 'ONCE')
        cron = getattr(job, 'cron_expression', '').lower()
        if job_type == "WEEKLY" or (job_type == "CRON" and "week" in cron):
            return now.strftime("%Y-W%W")
        elif job_type == "MONTHLY" or (job_type == "CRON" and "month" in cron):
            return now.strftime("%Y-M%m")
        return now.strftime("%Y-%m-%d")

```

文件编号: 69 文件路径: emily-core\emily_core\scheduler\handler_registry.py

```

from __future__ import annotations
import logging
from abc import ABC, abstractmethod
from dataclasses import dataclass
logger = logging.getLogger("emily.scheduler.handler_registry")
@dataclass
class JobResult:
    success: bool = True
    summary: str = ""
    data: dict = None
    def __post_init__(self):
        if self.data is None:
            self.data = {}
class SchedulerJobHandler(ABC):
    @property
    @abstractmethod
    def action_type(self) -> str:
    @property
    @abstractmethod
    def description(self) -> str:
    @abstractmethod
    async def execute(self, params: dict) -> JobResult:
class JobHandlerRegistry:
    def __init__(self):
        self._handlers: dict[str, SchedulerJobHandler] = {}
    def register(self, handler: SchedulerJobHandler) -> None:

```

```

        if handler.action_type in self._handlers:
            raise ValueError(f"SchedulerJobHandler '{handler.action_type}' 已注册")

        self._handlers[handler.action_type] = handler
        logger.info("JobHandler registered: %s (%s)", handler.action_type,
handler.description)
    def get(self, action_type: str) -> SchedulerJobHandler | None:
        return self._handlers.get(action_type)
    def has(self, action_type: str) -> bool:
        return action_type in self._handlers
    def list_all(self) -> list[dict]:
        return [
            {"action_type": h.action_type, "description": h.description}
            for h in self._handlers.values()
        ]
    def __len__(self) -> int:
        return len(self._handlers)

```

文件编号: 70 文件路径: emily-core\emily_core\scheduler\hook_registry.py

```

from __future__ import annotations
import logging
from abc import ABC, abstractmethod
from dataclasses import dataclass
from enum import Enum
logger = logging.getLogger("emily.scheduler.hook_registry")
class HookDecision(str, Enum):
    ALLOW = "ALLOW"
    WARN = "WARN"
    BLOCK = "BLOCK"
@dataclass
class HookResult:
    decision: HookDecision = HookDecision.ALLOW
    message: str = ""
    @property
    def is_blocked(self) -> bool:
        return self.decision == HookDecision.BLOCK
class SchedulerHook(ABC):
    @property
    @abstractmethod
    def name(self) -> str:
    @property
    def priority(self) -> int:
        return 10
    @property
    def enabled(self) -> bool:
        return True
    @abstractmethod
    async def execute(self, context: "SchedulerContext") -> HookResult:
class SchedulerHookRegistry:
    def __init__(self):
        self._hooks: dict[str, list[SchedulerHook]] = {}

```

```

def register(self, mount_point: str, hook: SchedulerHook) -> None:
    if mount_point not in self._hooks:
        self._hooks[mount_point] = []
    self._hooks[mount_point].append(hook)
    self._hooks[mount_point].sort(key=lambda h: h.priority)
    logger.info("SchedulerHook registered: %s at %s (priority=%d)", hook.name,
mount_point, hook.priority)
def get_enabled(self, mount_point: str) -> list[SchedulerHook]:
    return [h for h in self._hooks.get(mount_point, []) if h.enabled]
def hook_count(self) -> int:
    return sum(len(hooks) for hooks in self._hooks.values())

```

文件编号: 71 文件路径: emily-core\emily_core\scheduler\jobs\daily_insight.py

```

from __future__ import annotations
import logging
from ..handler_registry import SchedulerJobHandler, JobResult
logger = logging.getLogger("emily.scheduler.jobs.daily_insight")
class DailyInsightHandler(SchedulerJobHandler):
    action_type = "generate_daily_insight"
    description = "生成洞察（指标聚合+异常检测+LLM复盘，支持可变周期）"
    def __init__(self, llm_client=None):
        self._llm = llm_client
    async def execute(self, params: dict) -> JobResult:
        from datetime import datetime, timezone, timedelta
        beijing_tz = timezone(timedelta(hours=8))
        date = params.get("date", datetime.now(beijing_tz).strftime("%Y-%m-%d"))
        days = max(1, params.get("days", 1))
        try:
            from emily_core.services.evolution.insight_generator import
InsightGenerator
            gen = InsightGenerator(llm_client=self._llm)
            result = await gen.generate(date, days=days)
            period_label = f"{days}天" if days > 1 else "日"
            return JobResult(
                success=True,
                summary=f"{period_label}洞察 {date} 生成完成, 健康评分=
{result.get('insight', {}).get('health_score', 'N/A')}",
            )
        except Exception as e:
            logger.error("DailyInsightHandler failed: %s", e, exc_info=True)
            return JobResult(success=False, summary=str(e))

```

文件编号: 72 文件路径: emily-core\emily_core\scheduler\jobs\data_sync.py

```

from __future__ import annotations
import logging
from ..handler_registry import SchedulerJobHandler, JobResult
logger = logging.getLogger("emily.scheduler.jobs.data_sync")

```



```
class DataSyncHandler(SchedulerJobHandler):
    action_type = "sync_external_data"
    description = "定时同步外部数据源"
    def __init__(self, file_service=None, rag_provider=None):
        self._file_service = file_service
        self._rag_provider = rag_provider
    async def execute(self, params: dict) -> JobResult:
        sync_type = params.get("sync_type", "files")
        logger.info("Data sync triggered: type=%s", sync_type)
        return JobResult(success=True, summary=f"数据同步完成（类型: {sync_type}）")
```

文件编号: 73 文件路径: emily-core\emily_core\scheduler\jobs\health_check.py

```
from __future__ import annotations
import logging
from ..handler_registry import SchedulerJobHandler, JobResult
logger = logging.getLogger("emily.scheduler.jobs.health_check")
class HealthCheckHandler(SchedulerJobHandler):
    action_type = "system_health_check"
    description = "系统健康自检（DB/LLM/磁盘）"
    def __init__(self, outbound_bus=None, session_factory=None):
        self._outbound_bus = outbound_bus
        self._session_factory = session_factory
    async def execute(self, params: dict) -> JobResult:
        checks = []
        try:
            from ....infrastructure.database.session import get_session_raw
            session = (self._session_factory() if self._session_factory
                       else get_session_raw())
            session.execute(__import__('sqlalchemy').text("SELECT 1"))
            session.close()
            checks.append("DB: OK")
        except Exception as e:
            checks.append(f"DB: FAIL ({e})")
        logger.info("Health check: %s", ", ".join(checks))
        return JobResult(
            success=all("FAIL" not in c for c in checks),
            summary="; ".join(checks),
        )
```

文件编号: 74 文件路径: emily-core\emily_core\scheduler\jobs\morning_report.py

```
from __future__ import annotations
import logging
from ..handler_registry import SchedulerJobHandler, JobResult
logger = logging.getLogger("emily.scheduler.jobs.morning_report")
class MorningReportHandler(SchedulerJobHandler):
    action_type = "generate_morning_report"
    description = "生成晨报并推送到指定群"
```

```

def __init__(self, outbound_bus=None, llm_client=None):
    self._outbound_bus = outbound_bus
    self._llm = llm_client
    async def execute(self, params: dict) -> JobResult:
        push_group = params.get("push_to_group", "项目群")
        report_text = f"📅 {push_group} 晨报（待接入完整逻辑）"
        try:
            from datetime import datetime, timezone, timedelta
            beijing_tz = timezone(timedelta(hours=8))
            yesterday = (datetime.now(beijing_tz) -
timedelta(days=1)).strftime("%Y-%m-%d")
            from emily_core.repositories.evolution_repo import EvolutionRepo
            insight = EvolutionRepo.get_insight_by_date(yesterday)
            draft_patches = EvolutionRepo.get_patches_by_status("DRAFT")
            if insight or draft_patches:
                evolution_section = "\n\n📊 系统进化摘要（昨日）\n"
                if insight:
                    evolution_section += f"健康评分: {insight.health_score}/100\n"
                    evolution_section += f"SOP 命中率:
{insight.sop_hit_rate:.1%}\n"
                if draft_patches:
                    evolution_section += "\n🔧 待审批补丁\n"
                    for p in draft_patches:
                        evolution_section += f"    {p.patch_no}: {p.rule_no}
[{p.risk_level}] - {p.target_path}\n"
                report_text += evolution_section
            except Exception as e:
                logger.warning("晨报进化信息追加失败: %s", e)
            if self._outbound_bus:
                try:
                    self._outbound_bus.publish("reply", {
                        "content": report_text,
                        "source": "scheduler:morning_report",
                    })
                except Exception as e:
                    logger.error("晨报推送失败: %s", e)
                    return JobResult(success=False, summary=f"推送失败: {e}")
            logger.info("Morning report sent to %s", push_group)
            return JobResult(success=True, summary=f"晨报已推送到 {push_group}")

```

文件编号: 75 文件路径: emily-core\emily_core\scheduler\jobs\node_deadlines.py

```

from __future__ import annotations
import logging
from ..handler_registry import SchedulerJobHandler, JobResult
logger = logging.getLogger("emily.scheduler.jobs.node_deadlines")
class NodeDeadlineHandler(SchedulerJobHandler):
    action_type = "check_node_deadlines"
    description = "节点截止时间提醒（临期 + 超期）"
    def __init__(self, node_service=None, outbound_bus=None):
        self._node_service = node_service
        self._outbound_bus = outbound_bus

```

```

    async def execute(self, params: dict) -> JobResult:
        if self._node_service is None:
            return JobResult(success=False, summary="NodeService 未注入")
        before_minutes = params.get("before_minutes", 60)
        check_overdue = params.get("overdue_check", True)
        reminders = []
        near = await
self._node_service.find_near_deadline(before_minutes=before_minutes)
        for node in near:
            msg = f"⚠️ 节点「{node.node_name}」即将到期（截止：{node.deadline}）"
            reminders.append(msg)
        if check_overdue:
            overdue = await self._node_service.find_overdue()
            for node in overdue:
                msg = f"🔴 节点「{node.node_name}」已超期（截止：{node.deadline}）"
                reminders.append(msg)
        if reminders and self._outbound_bus:
            full_text = "\n".join(reminders)
            try:
                self._outbound_bus.publish("reply", {
                    "content": full_text,
                    "source": "scheduler:node_deadlines",
                })
            except Exception as e:
                logger.error("节点提醒推送失败: %s", e)
                return JobResult(success=False, summary=f"推送失败: {e}")
            summary = f"临期 {len(near)} 个, 超期 {len(overdue)} if check_overdue else
0} 个"
            logger.info("Node deadline check: %s", summary)
            return JobResult(success=True, summary=summary)

```

文件编号: 76 文件路径: emily-core\emily_core\scheduler\jobs\patch_validator.py

```

from __future__ import annotations
import logging
from ..handler_registry import SchedulerJobHandler, JobResult
logger = logging.getLogger("emily.scheduler.jobs.patch_validator")
class PatchValidationHandler(SchedulerJobHandler):
    action_type = "validate_evolution_patches"
    description = "验证已应用 >= 7天的补丁效果"
    def __init__(self, llm_client=None):
        self._llm = llm_client
    async def execute(self, params: dict) -> JobResult:
        try:
            from emily_core.services.evolution.patch_validator import
PatchValidator
            validator = PatchValidator()
            results = await validator.validate()
            confirmed = sum(1 for r in results if r.get("status") == "confirmed")
            rolled_back = sum(1 for r in results if r.get("status") ==
"rolled_back")
            return JobResult(

```

```

        success=True,
        summary=f"补丁验证完成: {len(results)} 个, CONFIRMED={confirmed},
ROLLED_BACK={rolled_back}",
    )
except Exception as e:
    logger.error("PatchValidationHandler failed: %s", e, exc_info=True)
    return JobResult(success=False, summary=str(e))

```

文件编号: 77 文件路径: emily-core\emily_core\scheduler\jobs\periodic_node.py

```

from __future__ import annotations
import logging
from ..handler_registry import SchedulerJobHandler, JobResult
logger = logging.getLogger("emily.scheduler.jobs.periodic_node")
class PeriodicNodeHandler(SchedulerJobHandler):
    action_type = "create_periodic_node"
    description = "定期创建叶子节点(循环业务任务)"
    def __init__(self, node_service=None):
        self._node_service = node_service
    async def execute(self, params: dict) -> JobResult:
        if self._node_service is None:
            return JobResult(success=False, summary="NodeService 未注入")
        from ...services.node_commands import CreateTaskNodeCommand
        cmd = CreateTaskNodeCommand(
            project_id=params.get("project_id", ""),
            node_name=params.get("node_name", "周期任务"),
            responsible_user_id=params.get("responsible_user_id", ""),
            deadline=params.get("deadline", ""),
            parent_node_id=params.get("parent_node_id", ""),
            owner_dept_id=params.get("owner_dept_id", "项目总"),
            description=params.get("description", ""),
            creator_id=params.get("creator_id", "scheduler"),
        )
        result = await self._node_service.create_node(cmd)
        if result.success:
            logger.info("Periodic node created: %s", result.node_id)
            return JobResult(success=True, summary=f"节点「{cmd.node_name}」创建成功")
        else:
            logger.error("Periodic node failed: %s", result.message)
            return JobResult(success=False, summary=result.message)

```

文件编号: 78 文件路径: emily-core\emily_core\scheduler\jobs\rule_induction.py

```

from __future__ import annotations
import logging
from ..handler_registry import SchedulerJobHandler, JobResult
logger = logging.getLogger("emily.scheduler.jobs.rule_induction")
class RuleInductionHandler(SchedulerJobHandler):

```

```

action_type = "induct_evolution_rules"
description = "从近N天洞察中归纳进化规则（默认7天）"
def __init__(self, llm_client=None):
    self._llm = llm_client
async def execute(self, params: dict) -> JobResult:
    from datetime import datetime, timezone, timedelta
    beijing_tz = timezone(timedelta(hours=8))
    end_date = params.get("end_date", datetime.now(beijing_tz).strftime("%Y-%m-%d"))
    days = params.get("days", 7)
    try:
        from emily_core.services.evolution.rule_inductor import RuleInductor
        inductor = RuleInductor(llm_client=self._llm)
        rules = await inductor.induct(end_date, days=days)
        return JobResult(success=True, summary=f"规则归纳完成: {len(rules)} 条")
    except Exception as e:
        logger.error("RuleInductionHandler failed: %s", e, exc_info=True)
        return JobResult(success=False, summary=str(e))

```

文件编号: 79 文件路径: emily-core\emily_core\scheduler\jobs\session_cleanup.py

```

from __future__ import annotations
import logging
from ..handler_registry import SchedulerJobHandler, JobResult
logger = logging.getLogger("emily.scheduler.jobs.session_cleanup")
class SessionCleanupHandler(SchedulerJobHandler):
    action_type = "cleanup_stale_sessions"
    description = "巡检超时 Session 并下发关闭指令"
    def __init__(self, session_pool=None, outbound_bus=None):
        self._session_pool = session_pool
        self._outbound_bus = outbound_bus
    async def execute(self, params: dict) -> JobResult:
        timeout_minutes = params.get("timeout_minutes", 30)
        count = 0
        if self._session_pool:
            logger.info("Session cleanup: timeout=%dmin", timeout_minutes)
        else:
            logger.warning("SessionCleanup: SessionPool 未注入")
        return JobResult(success=True, summary=f"已关闭 {count} 个超时 session")

```

文件编号: 80 文件路径: emily-core\emily_core\scheduler\jobs\system_description_update.py

```

from __future__ import annotations
import logging
from ..handler_registry import SchedulerJobHandler, JobResult
logger = logging.getLogger("emily.scheduler.jobs.system_description_update")
class SystemDescriptionUpdateHandler(SchedulerJobHandler):
    action_type = "system_description_update"

```

```

description = "系统描述偏差检测 + 自动更新"
def __init__(self, system_description_service=None):
    self._service = system_description_service
    async def execute(self, params: dict) -> JobResult:
        try:
            if self._service is None:
                from ...services.system_description_service import
SystemDescriptionService
                self._service = SystemDescriptionService()
            result = await self._service.check_and_update()
            status = result.get("status", "unknown")
            if status == "no_drift":
                summary = "系统描述无需更新: 与当前代码结构一致"
            elif status in ("built", "updated"):
                stale = result.get("updated_domains", [])
                summary = f"系统描述已更新: 偏差域 {stale}"
            elif status == "preview":
                summary = "系统描述预览模式"
            else:
                summary = f"系统描述更新状态: {status}"
            return JobResult(
                success=True,
                summary=summary,
                data=result,
            )
        except Exception as e:
            logger.error("SystemDescriptionUpdateHandler failed: %s", e,
exc_info=True)
            return JobResult(success=False, summary=str(e))

```

文件编号: 81 文件路径: emily-core\emily_core\scheduler\jobs\webhook.py

```

from __future__ import annotations
import logging
from ..handler_registry import SchedulerJobHandler, JobResult
logger = logging.getLogger("emily.scheduler.jobs.webhook")
class WebhookHandler(SchedulerJobHandler):
    action_type = "call_external_webhook"
    description = "定时调用外部 Webhook URL"
    async def execute(self, params: dict) -> JobResult:
        url = params.get("url", "")
        method = params.get("method", "POST")
        if not url:
            return JobResult(success=False, summary="未配置 webhook URL")
        logger.info("Webhook call: %s %s", method, url)
        return JobResult(success=True, summary=f"Webhook 已调用: {method} {url}")

```

文件编号: 82 文件路径: emily-core\emily_core\scheduler\jobs\world_book_update.py

```

from __future__ import annotations
import logging
from ..handler_registry import SchedulerJobHandler, JobResult
logger = logging.getLogger("emily.scheduler.jobs.world_book_update")
class WorldBookUpdateHandler(SchedulerJobHandler):
    action_type = "world_book_update"
    description = "认知偏差检测 + 世界书增量更新"
    def __init__(self, world_book_service=None):
        self._service = world_book_service
    async def execute(self, params: dict) -> JobResult:
        try:
            if self._service is None:
                from ...services.world_book_service import ProjectWorldBookService
                self._service = ProjectWorldBookService()
            results = await self._service.update_all()
            updated = sum(1 for r in results if r.get("status") == "updated")
            no_drift = sum(1 for r in results if r.get("status") == "no_drift")
            errors = sum(1 for r in results if r.get("status") == "error")
            return JobResult(
                success=True,
                summary=f"世界书更新: {updated}个更新, {no_drift}个无偏差, {errors}
个失败",
                data={"results": results},
            )
        except Exception as e:
            logger.error("WorldBookUpdateHandler failed: %s", e, exc_info=True)
            return JobResult(success=False, summary=str(e))

```

文件编号: 83 文件路径: emily-core\emily_core\scheduler\service.py

```

from __future__ import annotations
import asyncio
import logging
from datetime import datetime, timezone
from .commands import (
    CreateJobCommand,
    ActivateJobCommand,
    TriggerJobCommand,
    SchedulerOperationResult,
)
logger = logging.getLogger("emily.scheduler.service")
BEIJING_TZ = timezone(datetime.now(timezone.utc).astimezone().utcoffset() or
__import__('datetime').timedelta(hours=8))
class SchedulerService:
    def __init__(self, repo=None):
        from ..repositories.scheduler_repo import SchedulerRepo
        self._repo = repo or SchedulerRepo()
    async def create_job(self, cmd: CreateJobCommand) -> SchedulerOperationResult:
        job_no = self._repo.generate_job_no()
        job = await asyncio.to_thread(

```

```

        self._repo.create_job,
        job_no=job_no,
        name=cmd.name,
        action_type=cmd.action_type,
        handler_module=cmd.handler_module,
        job_type=cmd.job_type,
        cron_expression=cmd.cron_expression,
        interval_seconds=cmd.interval_seconds,
        deadline_rule=cmd.deadline_rule,
        action_params=cmd.action_params,
        creator_id=cmd.creator_id,
    )
    return SchedulerOperationResult(
        success=True, job_id=job.id, message=f"作业「{cmd.name}」创建成功（编号: {job_no}）",
    )

    async def activate_job(self, cmd: ActivateJobCommand) -> SchedulerOperationResult:
        target = "ACTIVE" if cmd.activate else "INACTIVE"
        job = await asyncio.to_thread(self._repo.update_job_status, cmd.job_id, target)
        if job is None:
            return SchedulerOperationResult(success=False, job_id=cmd.job_id, message="作业不存在")
        return SchedulerOperationResult(
            success=True, job_id=cmd.job_id, message=f"作业已{'激活' if cmd.activate else '停用'}",
        )

    async def get_job(self, job_id: str):
        return await asyncio.to_thread(self._repo.get_job_by_id, job_id)
    async def list_jobs(self, status: str = "") -> list:
        return await asyncio.to_thread(self._repo.list_jobs, status)
    async def list_active_jobs(self) -> list:
        return await asyncio.to_thread(self._repo.list_active_jobs)
    async def create_execution(self, job_id: str, period_key: str = "") -> SchedulerOperationResult:
        execution_no = self._repo.generate_execution_no()
        execution = await asyncio.to_thread(
            self._repo.create_execution,
            execution_no=execution_no,
            job_id=job_id,
            period_key=period_key,
        )
        return SchedulerOperationResult(
            success=True, execution_no=execution_no,
        )

    async def update_execution_status(self, execution_id: str, status: str, error_message: str = "", result_summary: str = "") -> None:
        await asyncio.to_thread(
            self._repo.update_execution_status,
            execution_id, status, error_message, result_summary,
        )

```


文件编号: 84 文件路径: emily-core\emily_core\services\agent_trace_service.py

```
import logging
logger = logging.getLogger("emily.service.agent_trace")
class AgentTraceService:
    def __init__(self):
        pass
    def create_reasoning_log(
        self,
        message_id: str,
        user_id: str = "",
        conversation_id: str = "",
        matched_sop_id: str | None = None,
        match_confidence: str = "none",
        is_compound: bool = False,
        fallback: bool = False,
    ) -> str | None:
        try:
            from ..repositories.agent_reasoning_repo import
AgentReasoningRepository
            return AgentReasoningRepository.create(
                message_id=message_id,
                user_id=user_id,
                conversation_id=conversation_id,
                matched_sop_id=matched_sop_id,
                match_confidence=match_confidence,
                is_compound=is_compound,
                fallback=fallback,
            )
        except Exception as e:
            logger.warning("Failed to create reasoning log: %s", e)
            return None
    def finalize_reasoning_log(
        self,
        reasoning_log_id: str,
        *,
        iteration_count: int = 0,
        elapsed_ms: int = 0,
        matched_sop_id: str | None = None,
        match_confidence: str = "none",
        is_compound: bool = False,
        fallback: bool = False,
        execution_result: str = "",
        reply_preview: str = "",
        steps: list | None = None,
        error_message: str = "",
        max_iterations_reached: bool = False,
    ) -> None:
        try:
            from ..repositories.agent_reasoning_repo import
AgentReasoningRepository
```

```
        AgentReasoningRepository.finalize(
            reasoning_log_id=reasoning_log_id,
            iteration_count=iteration_count,
            elapsed_ms=elapsed_ms,
            matched_sop_id=matched_sop_id,
            match_confidence=match_confidence,
            is_compound=is_compound,
            fallback=fallback,
            execution_result=execution_result,
            reply_preview=reply_preview,
            steps=steps,
            error_message=error_message,
            max_iterations_reached=max_iterations_reached,
        )
    except Exception as e:
        logger.warning("Failed to finalize reasoning log %s: %s",
reasoning_log_id, e)
    def create_llm_interaction_log(
        self,
        reasoning_log_id: str,
        call_sequence: int,
        call_type: str,
        model: str,
        user_message_count: int = 0,
        tool_count: int = 0,
    ) -> str | None:
        try:
            from ..repositories.llm_interaction_repo import
LLMInteractionRepository
            return LLMInteractionRepository.create(
                reasoning_log_id=reasoning_log_id,
                call_sequence=call_sequence,
                call_type=call_type,
                model=model,
                user_message_count=user_message_count,
                tool_count=tool_count,
            )
        except Exception as e:
            logger.warning("Failed to create LLM interaction log: %s", e)
            return None
    def update_llm_response(
        self,
        llm_interaction_id: str,
        response_type: str,
        response_summary: str = "",
        finish_reason: str = "",
        prompt_tokens: int = 0,
        completion_tokens: int = 0,
        total_tokens: int = 0,
        latency_ms: int = 0,
    ) -> None:
        try:
            from ..repositories.llm_interaction_repo import
LLMInteractionRepository
```

```

        LLMInteractionRepository.update_response(
            llm_interaction_id=llm_interaction_id,
            response_type=response_type,
            response_summary=response_summary,
            finish_reason=finish_reason,
            prompt_tokens=prompt_tokens,
            completion_tokens=completion_tokens,
            total_tokens=total_tokens,
            latency_ms=latency_ms,
        )
    except Exception as e:
        logger.warning("Failed to update LLM response %s: %s",
            llm_interaction_id, e)
    def create_tool_call_log(
        self,
        reasoning_log_id: str,
        llm_interaction_id: str,
        step_index: int,
        tool_name: str,
        tool_arguments: dict,
    ) -> str | None:
        try:
            from ..repositories.tool_call_repo import ToolCallRepository
            return ToolCallRepository.create(
                reasoning_log_id=reasoning_log_id,
                llm_interaction_id=llm_interaction_id,
                step_index=step_index,
                tool_name=tool_name,
                tool_arguments=tool_arguments,
            )
        except Exception as e:
            logger.warning("Failed to create tool call log: %s", e)
            return None
    def update_tool_result(
        self,
        tool_call_log_id: str,
        result_summary: str = "",
        is_success: bool = True,
        error_message: str = "",
        elapsed_ms: int = 0,
    ) -> None:
        try:
            from ..repositories.tool_call_repo import ToolCallRepository
            ToolCallRepository.update_result(
                tool_call_log_id=tool_call_log_id,
                result_summary=result_summary,
                is_success=is_success,
                error_message=error_message,
                elapsed_ms=elapsed_ms,
            )
        except Exception as e:
            logger.warning("Failed to update tool result %s: %s",
                tool_call_log_id, e)
    def get_complete_agent_trace(self, message_id: str) -> dict:

```

```

        try:
            from ..repositories.agent_reasoning_repo import
AgentReasoningRepository
            return AgentReasoningRepository.get_complete_trace(message_id)
        except Exception as e:
            logger.warning("Failed to get agent trace for %s: %s", message_id, e)
            return {"found": False, "error": str(e)}
    def query_llm_usage_stats(self, time_range: str = "7d") -> dict:
        try:
            from ..repositories.llm_interaction_repo import
LLMInteractionRepository
            return LLMInteractionRepository.get_usage_stats(time_range)
        except Exception as e:
            logger.warning("Failed to query LLM usage stats: %s", e)
            return {}

```

文件编号: 85 文件路径: emily-core\emily_core\services\chat_archive_service.py

```

import json
import logging
from datetime import datetime, timezone
from typing import Optional
from ..repositories.message_repo import MessageRepository
from ..repositories.chat_archive_repo import ChatArchiveRepository
from ..adapters.standard.message import StandardMessage
from ..adapters.standard.route_decision import RouteDecision
from ..infrastructure.database.models import Message
logger = logging.getLogger("emily.service.chat_archive")
class ChatArchiveService:
    def __init__(self, storage_root: str = ""):
        self._storage_root = storage_root
    def record_inbound_message(
        self,
        event_id: str,
        msg: StandardMessage,
        decision: RouteDecision,
    ) -> Message:
        return MessageRepository.create_from_standard(event_id, msg, decision)
    def record_outbound_reply(
        self,
        conversation_id: str,
        content: str,
        *,
        sender_im_id: str = "",
        reply_to_message_id: str | None = None,
    ) -> Message:
        return MessageRepository.create_outbound(
            conversation_id=conversation_id,
            content=content,
            sender_im_id=sender_im_id,
            direction="agent_to_user",
            message_type="",

```

```

        reply_to_message_id=reply_to_message_id,
    )
def record_progress_message(
    self,
    conversation_id: str,
    content: str,
    sender_im_id: str = "",
) -> Message:
    return MessageRepository.create_outbound(
        conversation_id=conversation_id,
        content=content,
        sender_im_id=sender_im_id,
        direction="agent_to_user",
        message_type="progress",
    )
def add_attachments(self, message_id: str, attachments: list[dict]) -> int:
    if not attachments:
        return 0
    count = 0
    for att in attachments:
        try:
            ChatArchiveRepository.create_attachment(
                message_id=message_id,
                attachment_type=att.get("type", 0),
                file_url=att.get("url", ""),
                file_size=att.get("file_size", 0),
                mime_type=att.get("mime_type", ""),
                thumbnail_url=att.get("thumb", ""),
            )
            count += 1
        except Exception as e:
            logger.warning("Failed to add attachment for message %s: %s",
message_id, e)
    return count
def get_conversation_history(
    self,
    conversation_id: str,
    limit: int = 100,
    offset: int = 0,
    include_progress: bool = False,
) -> list[Message]:
    return MessageRepository.list_by_conversation_full(
        conversation_id=conversation_id,
        limit=limit,
        offset=offset,
        include_progress=include_progress,
    )
def get_user_history(
    self,
    user_id: str,
    platform: str | None = None,
    limit: int = 50,
) -> list[Message]:
    return MessageRepository.get_user_history(

```

```

        user_id=user_id,
        platform=platform,
        limit=limit,
    )
def search_messages(
    self,
    keyword: str,
    *,
    conversation_id: str | None = None,
    user_id: str | None = None,
    time_range: str = "all",
    limit: int = 50,
) -> list[Message]:
    return MessageRepository.search_messages_fulltext(
        keyword=keyword,
        conversation_id=conversation_id,
        user_id=user_id,
        time_range=time_range,
        limit=limit,
    )
def get_attachments_for_message(self, message_id: str) -> list[dict]:
    return ChatArchiveRepository.get_attachments_for_message(message_id)
def get_files_for_conversation(self, conversation_id: str) -> list[dict]:
    return ChatArchiveRepository.get_files_for_conversation(conversation_id)
def get_daily_stats(self, date_str: str) -> dict:
    return MessageRepository.get_daily_stats(date_str)

```

文件编号: 86 文件路径: emily-core\emily_core\services\cognition_drift_detector.py

```

from __future__ import annotations
import json
import logging
from datetime import datetime, timezone, timedelta
from typing import Optional
from ..repositories.world_book_repo import ProjectWorldBookRepo
from ..infrastructure.database.session import get_session
from ..infrastructure.database.models import (
    Project, User, CompanyInfo, ProjectNode, NodeDependency, Event,
)
logger = logging.getLogger("emily.cognition_drift_detector")
BEIJING_TZ = timezone(timedelta(hours=8))
class CognitionDriftDetector:
    def detect(self, project_id: str) -> dict:
        wb = ProjectWorldBookRepo.get_by_project(project_id)
        if wb is None:
            return {
                "project_id": project_id,
                "has_world_book": False,
                "drift": {},
                "stale_layers": [],
                "has_drift": False,
                "message": "项目无世界书, 需首次生成",
            }

```

```

    }
    try:
        layers = json.loads(wb.content_json or "{}")
    except (json.JSONDecodeError, TypeError):
        layers = {}
    drift = {}
    drift["ontology"] = self._check_ontology(project_id,
layers.get("ontology", {}), wb.updated_at)
    drift["personnel"] = self._check_personnel(project_id,
layers.get("personnel", {}), wb.updated_at)
    drift["structure"] = self._check_structure(project_id,
layers.get("structure", {}))
    drift["temporal"] = self._check_temporal(project_id,
layers.get("temporal", {}))
    drift["relation"] = self._check_relation(project_id,
layers.get("relation", {}), wb.updated_at)
    drift["introspection"] = self._check_introspection(project_id,
layers.get("introspection", {}))
    drift["knowledge"] = {"stale": False, "signals": [], "note": "层6不常驻，按
需检测"}
    stale_layers = [k for k, v in drift.items() if v.get("stale", False)]
    return {
        "project_id": project_id,
        "has_world_book": True,
        "drift": drift,
        "stale_layers": stale_layers,
        "has_drift": len(stale_layers) > 0,
    }
def _check_ontology(self, project_id: str, layer: dict, wb_updated: str) ->
dict:
    signals = []
    stale = False
    try:
        with get_session() as session:
            project = session.query(Project).filter(Project.id ==
project_id).first()
            if project is None:
                return {"stale": True, "signals": ["项目不存在"]}
            current_stage = project.lifecycle_stage or 0
            recorded_stage = layer.get("lifecycle_stage", -1)
            if current_stage != recorded_stage:
                signals.append(f"lifecycle_stage: {recorded_stage}->
{current_stage}")
                stale = True
            users = session.query(User).filter(User.project_id == project_id,
User.is_deleted == False).all()
            company_ids = list(set(u.company for u in users if u.company))
            current_company_count = 0
            if company_ids:
                current_company_count =
session.query(CompanyInfo).filter(CompanyInfo.id.in_(company_ids)).count()
                recorded_company_count = len(layer.get("organizations", []))
                if current_company_count > recorded_company_count:
                    signals.append(f"新增参建单位: {recorded_company_count}->

```

```

{current_company_count}")
        stale = True
    except Exception as e:
        signals.append(f"检测异常: {e}")
    return {"stale": stale, "signals": signals}

def _check_personnel(self, project_id: str, layer: dict, wb_updated: str) ->
dict:
    signals = []
    stale = False
    try:
        with get_session() as session:
            users = session.query(User).filter(User.project_id == project_id,
User.is_deleted == False).all()
            current_count = len(users)
            recorded_count = layer.get("total_users", 0)
            if current_count != recorded_count:
                signals.append(f"用户数变化: {recorded_count}->
{current_count}")
                stale = True
    except Exception as e:
        signals.append(f"检测异常: {e}")
    return {"stale": stale, "signals": signals}

def _check_structure(self, project_id: str, layer: dict) -> dict:
    signals = []
    stale = False
    try:
        with get_session() as session:
            nodes = session.query(ProjectNode).filter(
                ProjectNode.project_id == project_id,
                ProjectNode.is_discarded == False,
            ).all()
            current_total = len(nodes)
            recorded_total = layer.get("total_nodes", 0)
            if current_total != recorded_total:
                signals.append(f"节点数变化: {recorded_total}->
{current_total}")
                stale = True
            if current_total > 0:
                current_progress = sum(float(n.progress or "0") for n in
nodes) / current_total
                recorded_progress = float(layer.get("overall_progress",
"0%").replace("%", ""))
                if abs(current_progress - recorded_progress) > 5:
                    signals.append(f"进度偏差: {recorded_progress:.1f}%->
{current_progress:.1f}%")
                    stale = True
            now_beijing = datetime.now(BEIJING_TZ)
            current_overdue = 0
            for n in nodes:
                if n.status != "COMPLETED" and n.deadline:
                    try:
                        dl = datetime.fromisoformat(n.deadline)
                        if dl.tzinfo is None:
                            dl = dl.replace(tzinfo=BEIJING_TZ)

```



```

        if dl < now_beijing:
            current_overdue += 1
        except (ValueError, TypeError):
            pass
        recorded_overdue = layer.get("overdue", 0)
        if current_overdue != recorded_overdue:
            signals.append(f"逾期数变化: {recorded_overdue}->
{current_overdue}")
        stale = True
    except Exception as e:
        signals.append(f"检测异常: {e}")
    return {"stale": stale, "signals": signals}
def _check_temporal(self, project_id: str, layer: dict) -> dict:
    signals = []
    stale = False
    try:
        with get_session() as session:
            recent_count = session.query(Event).filter(
                Event.project_id == project_id,
            ).count()
            recorded_events = len(layer.get("recent_events", []))
            if recent_count > recorded_events + 3:
                signals.append(f"新事件: 至少{recent_count - recorded_events}
条")
            stale = True
    except Exception as e:
        signals.append(f"检测异常: {e}")
    return {"stale": stale, "signals": signals}
def _check_relation(self, project_id: str, layer: dict, wb_updated: str) ->
dict:
    signals = []
    stale = False
    try:
        with get_session() as session:
            nodes = session.query(ProjectNode).filter(
                ProjectNode.project_id == project_id,
                ProjectNode.is_discarded == False,
            ).all()
            node_ids = [n.node_id for n in nodes]
            if node_ids:
                dep_count = session.query(NodeDependency).filter(
                    NodeDependency.node_id.in_(node_ids)
                ).count()
                recorded_deps = len(layer.get("key_dependencies", []))
                if dep_count != recorded_deps:
                    signals.append(f"依赖数变化: {recorded_deps}->{dep_count}")
                    stale = True
    except Exception as e:
        signals.append(f"检测异常: {e}")
    return {"stale": stale, "signals": signals}
def _check_introspection(self, project_id: str, layer: dict) -> dict:
    signals = []
    stale = False
    try:

```

```
from .initialization_checker import InitializationChecker
checker = InitializationChecker()
current_result = checker.check(project_id)
current_tier = current_result["tier"]
recorded_tier = layer.get("initialization_tier", -1)
if current_tier != recorded_tier:
    signals.append(f"初始化层级变化: T{recorded_tier}->T{current_tier}")
    stale = True
except Exception as e:
    signals.append(f"检测异常: {e}")
return {"stale": stale, "signals": signals}
```

文件编号: 87 文件路径: emily-core\emily_core\services\domain_takeover_service.py

```
import logging
from ..adapters.standard.message import StandardMessage
from ..adapters.standard.route_decision import RouteDecision
from ..config import Config
logger = logging.getLogger("emily.domain_takeover")
class DomainTakeoverService:
    def __init__(self, config: Config):
        self.config = config
    def decide(self, message: StandardMessage) -> RouteDecision:
        mode = self.config.takeover_mode
        if mode == "observe":
            logger.debug("takeover=false, reason=observe_mode")
            return RouteDecision(
                takeover=False,
                mode=mode,
                should_reply=False,
                reason="observe_mode",
            )
        if mode == "managed":
            logger.info("takeover=true, reason=managed_mode")
            return RouteDecision(
                takeover=True,
                mode=mode,
                reason="managed_mode",
            )
        if message.conversation_type == "private":
            logger.info("takeover=true, reason=private_message")
            return RouteDecision(
                takeover=True,
                mode=mode,
                reason="private_message",
            )
        if message.is_at_bot:
            logger.info("takeover=true, reason=at_bot_in_group")
            return RouteDecision(
                takeover=True,
                mode=mode,
```

```

        reason="at_bot_in_group",
    )
    logger.debug("takeover=false, reason=group_message_not_at_bot")
    return RouteDecision(
        takeover=False,
        mode=mode,
        should_reply=False,
        reason="group_message_not_at_bot",
    )

```

文件编号: 88 文件路径: emily-core\emily_core\services\email_service.py

```

from __future__ import annotations
from typing import Optional
from ..providers.email.base import (
    EmailAttachment,
    EmailCredentials,
    EmailEnvelope,
    SendResult,
)
from ..providers.email.smtp_provider import SMTPEmailProvider
from ..providers.email.imap_provider import IMAPEmailProvider
class EmailService:
    def __init__(self, smtp: SMTPEmailProvider, imap: IMAPEmailProvider):
        self._smtp = smtp
        self._imap = imap
    async def send(
        self,
        creds: EmailCredentials,
        to: str,
        subject: str,
        body: str,
        html: bool = False,
        attachments: Optional[list[EmailAttachment]] = None,
    ) -> SendResult:
        return await self._smtp.send(
            creds=creds,
            to=to,
            subject=subject,
            body=body,
            html=html,
            attachments=attachments,
        )
    async def fetch_inbox(
        self,
        creds: EmailCredentials,
        subject_filter: Optional[str] = None,
        from_self: bool = False,
        unread_only: bool = True,
        limit: int = 20,
    ) -> list[EmailEnvelope]:
        return await self._imap.fetch_inbox(

```

```

        creds=creds,
        subject_filter=subject_filter,
        from_self=from_self,
        unread_only=unread_only,
        limit=limit,
    )
    async def fetch_orders(
        self,
        creds: EmailCredentials,
        since_uid: Optional[str] = None,
    ) -> list[EmailEnvelope]:
        return await self._imap.fetch_orders(creds=creds, since_uid=since_uid)
    async def mark_read(
        self,
        creds: EmailCredentials,
        uids: list[str],
    ) -> bool:
        return await self._imap.mark_read(creds=creds, uids=uids)

```

文件编号: 89 文件路径: emily-core\emily_core\services\event_journal.py

```

import os
import logging
from datetime import datetime, timezone
logger = logging.getLogger("emily.service.journal")
class EventJournal:
    def __init__(self, path: str = "", enabled: bool = True):
        self.enabled = enabled
        self.path = path or ""
        if not self.path:
            self.path = os.path.join(
                os.path.dirname(os.path.dirname(os.path.dirname(
                    os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
                ))),
                "tem_log", "项目日志.md",
            )
        if self.enabled:
            self._ensure_file()
    def _ensure_file(self) -> None:
        os.makedirs(os.path.dirname(self.path), exist_ok=True)
        if not os.path.exists(self.path):
            try:
                with open(self.path, "w", encoding="utf-8") as f:
                    f.write("
话)。 \n")
                    f.write("> 格式: [日期] 姓名 操作摘要\n\n")
                    f.write("---\n\n")
                    logger.info("Created journal file: %s", self.path)
            except OSError as e:
                logger.error("Failed to create journal file %s: %s", self.path, e)
                self.enabled = False

```

```

def append(self, name: str, summary: str) -> bool:
    if not self.enabled:
        return False
    date_str = datetime.now(timezone.utc).strftime("%Y-%m-%d")
    try:
        from datetime import timedelta
        local = datetime.now(timezone.utc) + timedelta(hours=8)
        date_str = local.strftime("%Y-%m-%d")
    except Exception:
        pass
    line = f"[{date_str}] {name} {summary}\n"
    try:
        with open(self.path, "a", encoding="utf-8") as f:
            f.write(line)
        logger.info("Journal appended: %s", line.strip())
        return True
    except OSError as e:
        logger.error("Failed to append journal: %s", e)
        return False

def read(self, limit: int = 100) -> list[str]:
    if not self.enabled or not os.path.exists(self.path):
        return []
    try:
        with open(self.path, "r", encoding="utf-8") as f:
            lines = f.readlines()
        entries = [
            line.strip() for line in lines
            if line.strip() and not line.startswith("
")
            and not line.startswith(">") and not line.startswith("---")
        ]
        return entries[-limit:] if len(entries) > limit else entries
    except OSError as e:
        logger.error("Failed to read journal: %s", e)
        return []

def search(self, keyword: str = "", limit: int = 50) -> list[str]:
    entries = self.read(limit=10000)
    if not keyword:
        return entries[-limit:]
    filtered = [e for e in entries if keyword in e]
    return filtered[-limit:]

```

文件编号: 90 文件路径: emily-core\emily_core\services\event_service.py

```

import json
import logging
from typing import Optional
from ..repositories.event_repo import EventRepository
from ..adapters.standard.command import EventCommand
from ..infrastructure.database.models import Event, Project
logger = logging.getLogger("emily.service.event")
class EventService:
    def __init__(self):

```

```
self.repo = EventRepository()
def create_pending_event(self, cmd: EventCommand) -> Event:
    event_no = self.repo.generate_event_no()
    project_id = cmd.project_id
    if not project_id and cmd.project_name:
        project = self.repo.find_project_by_name(cmd.project_name)
        if project:
            project_id = project.id
    payload = {
        "project_name": cmd.project_name,
        "source": "llm_extract",
    }
    event = self.repo.create(
        event_no=event_no,
        event_type=cmd.event_type,
        title=cmd.title,
        project_id=project_id,
        user_id=cmd.creator_id or None,
        message_id=cmd.source_message_id or None,
        category=cmd.category,
        description=cmd.description,
        event_date=cmd.event_date,
        payload=json.dumps(payload, ensure_ascii=False),
        status="pending",
        related_event_ids=cmd.related_event_ids,
        conversation_id=cmd.conversation_id or None,
    )
    logger.info(
        "Pending event created: no=%s, title=%s, project=%s",
        event_no, cmd.title, cmd.project_name,
    )
    return event
def confirm_event(self, event_id: str) -> Optional[Event]:
    event = self.repo.get_by_id(event_id)
    if not event:
        logger.warning("Event not found: %s", event_id)
        return None
    if event.status != "pending":
        logger.warning("Event not pending: id=%s, status=%s", event_id,
event.status)
        return None
    self.repo.update_status(event_id, "confirmed")
    logger.info("Event confirmed: id=%s", event_id)
    return self.repo.get_by_id(event_id)
def cancel_event(self, event_id: str) -> Optional[Event]:
    event = self.repo.get_by_id(event_id)
    if not event:
        logger.warning("Event not found: %s", event_id)
        return None
    if event.status != "pending":
        logger.warning("Event not pending: id=%s, status=%s", event_id,
event.status)
        return None
    self.repo.update_status(event_id, "cancelled")
```

```

        logger.info("Event cancelled: id=%s", event_id)
        return self.repo.get_by_id(event_id)
    def find_pending_by_conversation(self, conversation_id: str) ->
Optional[Event]:
        return self.repo.find_pending_by_conversation_id(conversation_id)
    @staticmethod
    def format_confirmation_reply(event: Event, project_name: Optional[str] =
None) -> str:
        if not project_name:
            try:
                payload = json.loads(event.payload) if event.payload else {}
                project_name = payload.get("project_name", "未指定")
            except (json.JSONDecodeError, TypeError):
                project_name = "未指定"
        event_date = event.event_date or "未指定"
        reply = (
            f"📄 事件录入确认\n"
            f"—————\n"
            f"简述: {event.title}\n"
            f"项目: {project_name}\n"
            f"时间: {event_date}\n"
            f"编号: {event.event_no}\n"
            f"—————\n"
            f"回复\"确认\"录入, 回复\"取消\"放弃"
        )
        return reply
    def get_by_id(self, event_id: str) -> Optional[Event]:
        return self.repo.get_by_id(event_id)
    def get_project_list_text(self) -> str:
        projects = self.repo.list_projects(status="active")
        if not projects:
            return "暂无项目"
        lines = []
        for p in projects:
            code = p.code or ""
            lines.append(f"- {p.name}" + (f" ({code}) " if code else ""))
        return "\n".join(lines)

```

文件编号: 91 文件路径: emily-core\emily_core\services\evolution\insight_generator.py

```

from __future__ import annotations
import asyncio
import json
import logging
import os
from datetime import datetime, timedelta
from pathlib import Path
from typing import Optional
logger = logging.getLogger("emily.insight_generator")
class InsightGenerator:
    def __init__(self, llm_client=None, data_dir: str = ""):
        self._llm = llm_client

```

```

        self._data_dir = data_dir or os.environ.get("EMILY_DATA_DIR", "")
    async def generate(self, end_date: str, *, days: int = 1, dry_run: bool =
False) -> dict:
        from scripts.evolution_metrics import collect_metrics
        from scripts.evolution_anomaly import detect_anomalies
        import sys
        _HERE = Path(__file__).resolve().parent
        _SCRIPTS_DIR = _HERE.parent.parent.parent.parent.parent / "scripts"
        if str(_SCRIPTS_DIR) not in sys.path:
            sys.path.insert(0, str(_SCRIPTS_DIR))
        metrics = await collect_metrics(end_date, days=days)
        anomalies = detect_anomalies(metrics, days=days)
        start_dt = datetime.fromisoformat(end_date) - timedelta(days=days - 1)
        start_date = start_dt.strftime("%Y-%m-%d")
        if self._llm is None or dry_run:
            return {
                "end_date": end_date,
                "start_date": start_date,
                "days": days,
                "metrics": metrics,
                "anomalies": anomalies,
                "insight": None,
                "status": "preview",
            }
        try:
            from emily_core.infrastructure.llm.prompt_loader import load_prompt
            template = load_prompt("evolution_insight")
            analysis_period = f"{start_date} ~ {end_date} ({days}天) " if days > 1
        else end_date
            user_message = template.replace("{analysis_period}", analysis_period)
            user_message = user_message.replace("{metrics_json}",
json.dumps(metrics, ensure_ascii=False, default=str))
            user_message = user_message.replace("{anomaly_flags}",
json.dumps(anomalies, ensure_ascii=False, default=str))
            user_message = user_message.replace("{sop_distribution}",
json.dumps(metrics.get("pipeline", {}).get("sop_distribution", []),
ensure_ascii=False))
            user_message = user_message.replace("{fallback_messages}",
json.dumps(metrics.get("sop_routing", {}).get("miss_samples", [])[:10],
ensure_ascii=False))
            user_message = user_message.replace("{correction_details}",
json.dumps(metrics.get("feedback", {}).get("correction_samples", []),
ensure_ascii=False))
            user_message = user_message.replace("{rag_misses}",
json.dumps(metrics.get("rag", {}).get("zero_hit_samples", []),
ensure_ascii=False))
            user_message = user_message.replace("{tool_failures}",
json.dumps(metrics.get("tool_calls", {}).get("failure_details", []),
ensure_ascii=False))
            user_message = user_message.replace("{node_progress_changes}",
json.dumps(metrics.get("project_nodes", {}).get("progress_changes", []),
ensure_ascii=False))
            user_message = user_message.replace("{business_events_summary}",
json.dumps(metrics.get("business_events", {}), ensure_ascii=False, default=str))

```



```

        insight = await self._llm.chat_json(template, user_message)
    except Exception as e:
        logger.error("LLM insight generation failed: %s", e, exc_info=True)
        return {
            "end_date": end_date,
            "start_date": start_date,
            "days": days,
            "metrics": metrics,
            "anomalies": anomalies,
            "insight": None,
            "status": "llm_error",
            "error": str(e),
        }
    insight_date_key = f"{start_date}~{end_date}" if days > 1 else end_date
    try:
        from emily_core.repositories.evolution_repo import EvolutionRepo
        await asyncio.to_thread(
            EvolutionRepo.create_insight,
            insight_date_key,
            analysis_days=days,
            total_messages=metrics["pipeline"]["total"],
            total_pipeline_runs=metrics["pipeline"]["total"],
            sop_hit_rate=metrics["pipeline"]["sop_hit_rate"],
            fallback_rate=metrics["pipeline"]["fallback_rate"],
            top_sop_ids=json.dumps(metrics["pipeline"]["sop_distribution"]
[:5], ensure_ascii=False),
            feedback_summary=json.dumps(metrics.get("feedback",
{})).get("type_distribution", []), ensure_ascii=False),
            anomaly_flags=json.dumps([a["flag"] for a in anomalies],
ensure_ascii=False),
            insight_text=json.dumps(insight, ensure_ascii=False),
            metrics_json=json.dumps(metrics, ensure_ascii=False, default=str),
            health_score=insight.get("health_score", 0),
        )
    except Exception as e:
        logger.error("Failed to write insight to DB: %s", e, exc_info=True)
    try:
        self._write_md(start_date, end_date, days, insight, metrics,
anomalies)
    except Exception as e:
        logger.error("Failed to write insight MD: %s", e, exc_info=True)
    return {
        "end_date": end_date,
        "start_date": start_date,
        "days": days,
        "metrics": metrics,
        "anomalies": anomalies,
        "insight": insight,
        "status": "generated",
    }

def _write_md(self, start_date: str, end_date: str, days: int, insight: dict,
metrics: dict, anomalies: list) -> None:
    data_dir = self._data_dir
    if not data_dir:

```

```

        candidate =
Path(__file__).resolve().parent.parent.parent.parent.parent / "emily-data"
        if candidate.exists():
            data_dir = str(candidate)
        if not data_dir:
            logger.warning("Cannot determine emily-data directory, skipping MD
output")
            return
        insights_dir = Path(data_dir) / "evolution" / "insights"
        insights_dir.mkdir(parents=True, exist_ok=True)
        if days > 1:
            filename = f"{start_date}_{end_date}.md"
        else:
            filename = f"{end_date}.md"
        p = metrics["pipeline"]
        lines = []
        period_label = f"{end_date} ({days}天) " if days > 1 else end_date
        lines.append(f"
lines.append("")
lines.append("
lines.append(f"- 处理消息: {p['total']} 条")
lines.append(f"- Pipeline 执行: {p['total']} 次")
lines.append(f"- SOP 命中率: {p['sop_hit_rate']:.0%} | Fallback 率:
{p['fallback_rate']:.0%}")
lines.append(f"- 健康评分: {insight.get('health_score', 0)}/100")
lines.append("")
if anomalies:
    lines.append("
    for a in anomalies:
        sev = "HIGH" if a["severity"] == "high" else "MED"
        lines.append(f"- [{sev}] {a['flag']}: {a['message']}")
    lines.append("")
if insight.get("key_findings"):
    lines.append("
    for i, f in enumerate(insight["key_findings"], 1):
        lines.append(f"{i}. [{f.get('category', '')}] {f.get('finding',
''}})")
    lines.append("")
if insight.get("node_review"):
    nr = insight["node_review"]
    lines.append("
    if nr.get("progress_highlights"):
        for h in nr["progress_highlights"]:
            lines.append(f"- {h}")
    if nr.get("overdue_risks"):
        for r in nr["overdue_risks"]:
            lines.append(f"- {r}")
    lines.append("")
if insight.get("improvement_suggestions"):
    lines.append("
    for s in insight["improvement_suggestions"]:
        lines.append(f"- [{s.get('target', '')}]{s.get('priority', '')}]
{s.get('suggestion', '')}")
    lines.append("")

```

```

if insight.get("summary"):
    lines.append(f"
    lines.append(insight["summary"])
    lines.append("")
filepath = insights_dir / filename
filepath.write_text("\n".join(lines), encoding="utf-8")
logger.info("Insight MD written to %s", filepath)

```

文件编号: 92 文件路径: emily-core\emily_core\services\evolution\morning_report_builder.py

```

from __future__ import annotations
import asyncio
import logging
from datetime import datetime, timedelta
logger = logging.getLogger("emily.morning_report_builder")
class MorningReportBuilder:
    def __init__(self):
        pass
    async def build_for_user(self, user, date: str, *, is_admin: bool = False) ->
dict:
        from emily_core.repositories.evolution_repo import EvolutionRepo
        user_id = user.id
        user_name = user.username
        nodes = await asyncio.to_thread(EvolutionRepo.get_user_nodes, user_id)
        tasks = await asyncio.to_thread(EvolutionRepo.get_user_pending_tasks,
user_id)
        yesterday = (datetime.fromisoformat(date) -
timedelta(days=1)).strftime("%Y-%m-%d")
        events = await asyncio.to_thread(EvolutionRepo.get_user_recent_events,
user_id, yesterday)
        report_text = self._format_report(user_name, date, nodes, tasks, events,
is_admin)
        sections = {
            "nodes": [{"node_id": n.node_id, "name": n.node_name, "progress":
n.progress, "deadline": n.deadline, "status": n.status} for n in nodes],
            "tasks": [{"title": t.title, "status": t.status, "due_date":
t.due_date} for t in tasks],
            "events": [{"summary": e.summary, "action": e.event_action} for e in
events],
        }
        return {"user_name": user_name, "report_text": report_text, "sections":
sections}
    async def build_for_date(self, date: str, *, is_admin: bool = False) ->
list[dict]:
        from emily_core.repositories.evolution_repo import EvolutionRepo
        users = await asyncio.to_thread(EvolutionRepo.get_active_users)
        reports = []
        for user in users:
            report = await self.build_for_user(user, date, is_admin=(user.level >=
5))
            reports.append(report)
        return reports

```

```

def _format_report(self, user_name: str, date: str, nodes: list, tasks: list,
events: list, is_admin: bool) -> str:
    dt = datetime.fromisoformat(date)
    weekdays_cn = ["一", "二", "三", "四", "五", "六", "日"]
    weekday = weekdays_cn[dt.weekday()]
    lines = []
    lines.append(f"\u2600\u2013 早啊{user_name}! 今天是{dt.month}月{dt.day}号星
期{weekday}, 新的一天开工啦~")
    lines.append("")
    if nodes:
        lines.append("\U0001f4cb 你的节点进度")
        lines.append("")
        for n in nodes:
            try:
                progress = float(n.progress)
            except (ValueError, TypeError):
                progress = 0.0
            icon = "\U0001f7e2" if progress >= 70 else ("\U0001f7e1" if
progress >= 30 else "\u26aa")
            lines.append(f"{icon} {n.node_name} ({n.node_id}) -
{n.progress}%, 截止{n.deadline}")
            lines.append("")
    if tasks:
        lines.append("\U0001f4cc 待办任务")
        lines.append("")
        for t in tasks:
            due = t.due_date or ""
            urgency = ""
            if due and due <= date:
                icon = "\U0001f534"
                urgency = "- 截止今天! "
            elif due and due <= (datetime.fromisoformat(date) +
timedelta(days=3)).strftime("%Y-%m-%d"):
                icon = "\U0001f7e1"
                urgency = f"- 截止{due}"
            else:
                icon = "\U0001f7e2"
                urgency = f"- 截止{due}" if due else ""
            lines.append(f"{icon} {t.title} {urgency}")
            lines.append("")
    if events:
        lines.append("\U0001f4ca 昨日动态")
        lines.append("")
        for e in events[:5]:
            lines.append(f"\u2022 {e.event_action}: {e.summary}")
            lines.append("")
    overdue_nodes = [n for n in nodes if n.deadline and n.deadline < date]
    upcoming = [n for n in nodes if n.deadline and date <= n.deadline <=
(datetime.fromisoformat(date) + timedelta(days=7)).strftime("%Y-%m-%d")]
    if overdue_nodes or upcoming:
        lines.append("\u26a0\u2013 注意")
        lines.append("")
        for n in overdue_nodes:
            lines.append(f"\U0001f534 {n.node_name} 已逾期! 截止 {n.deadline}")

```

```

        for n in upcoming:
            days_left = (datetime.fromisoformat(n.deadline) -
datetime.fromisoformat(date)).days
            lines.append(f"\U0001f7e1 {n.node_name} 距截止仅 {days_left} 天")
            urgent_count = len(overdue_nodes) + len([n for n in upcoming if n.deadline
<= (datetime.fromisoformat(date) + timedelta(days=3)).strftime("%Y-%m-%d")])
            if urgent_count > 0:
                lines.append(f"\n今天有{urgent_count}件事比较急，盯紧点\U0001f4aa 早上先
搞定最紧迫的吧")
            else:
                lines.append(f"\n今天节奏不错，继续保持\U0001f44d")
        return "\n".join(lines)
    async def build_admin_section(self, date: str) -> str:
        from emily_core.repositories.evolution_repo import EvolutionRepo
        yesterday = (datetime.fromisoformat(date) -
timedelta(days=1)).strftime("%Y-%m-%d")
        lines = []
        lines.append("\n\n\U0001f4ca 系统进化摘要（昨日）")
        try:
            insight = await asyncio.to_thread(EvolutionRepo.get_insight_by_date,
yesterday)
            if insight:
                lines.append(f"健康评分: {insight.health_score}/100")
                lines.append(f"SOP 命中率: {insight.sop_hit_rate:.1%}")
                lines.append(f"Fallback 率: {insight.fallback_rate:.1%}")
            draft_patches = await
asyncio.to_thread(EvolutionRepo.get_patches_by_status, "DRAFT")
            if draft_patches:
                lines.append("\n\U0001f527 待审批补丁")
                for p in draft_patches:
                    lines.append(f"  {p.patch_no}: {p.rule_no} [{p.risk_level}] -
{p.target_path}")
            except Exception as e:
                logger.warning("Failed to build admin section: %s", e)
        return "\n".join(lines)

```

文件编号: 93 文件路径: emily-core\emily_core\services\evolution\patch_applier.py

```

from __future__ import annotations
import asyncio
import json
import logging
import os
from datetime import datetime
from pathlib import Path
logger = logging.getLogger("emily.patch_applier")
class PatchApplier:
    def __init__(self, data_dir: str = ""):
        self._data_dir = data_dir or os.environ.get("EMILY_DATA_DIR", "")
    def _resolve_data_dir(self) -> str:
        data_dir = self._data_dir
        if not data_dir:

```

```

        candidate =
Path(__file__).resolve().parent.parent.parent.parent.parent / "emily-data"
        if candidate.exists():
            data_dir = str(candidate)
        return data_dir
    async def apply(self, patch_no: str, *, dry_run: bool = False) -> dict:
        from emily_core.repositories.evolution_repo import EvolutionRepo
        patch = await asyncio.to_thread(EvolutionRepo.get_patch_by_no, patch_no)
        if patch is None:
            return {"status": "error", "message": f"Patch {patch_no} not found"}
        if patch.status != "DRAFT":
            return {"status": "error", "message": f"Patch {patch_no} is not in
DRAFT status (current: {patch.status})"}
        data_dir = self._resolve_data_dir()
        if not data_dir:
            return {"status": "error", "message": "Cannot resolve emily-data
directory"}
        target_path = Path(data_dir) / patch.target_path
        if not target_path.exists():
            return {"status": "error", "message": f"Target file not found:
{target_path}"}
        original_content = target_path.read_text(encoding="utf-8")
        rollback_snapshot = original_content
        if dry_run:
            return {
                "status": "preview",
                "patch_no": patch_no,
                "target_path": str(target_path),
                "patch_type": patch.patch_type,
                "search_anchor": patch.search_anchor,
                "patch_content_preview": patch.patch_content[:200],
                "file_size": len(original_content),
            }
        try:
            new_content = self._apply_patch_to_content(
                original_content,
                patch.patch_type,
                patch.search_anchor,
                patch.patch_content,
            )
        except Exception as e:
            logger.error("Failed to apply patch content: %s", e)
            return {"status": "error", "message": f"Failed to apply: {e}"}
        target_path.write_text(new_content, encoding="utf-8")
        now_str = datetime.now().isoformat()
        await asyncio.to_thread(
            EvolutionRepo.update_patch_status,
            patch_no,
            "APPLIED",
            applied_at=now_str,
            rollback_snapshot=rollback_snapshot,
        )
        logger.info("Patch %s applied to %s", patch_no, target_path)
        return {"status": "applied", "patch_no": patch_no, "target_path":

```

```

str(target_path))
    def _apply_patch_to_content(self, content: str, patch_type: str,
search_anchor: str, patch_content: str) -> str:
        if patch_type == "append":
            if search_anchor and search_anchor in content:
                return content.replace(search_anchor, search_anchor + "\n" +
patch_content)
            else:
                return content.rstrip("\n") + "\n" + patch_content + "\n"
        elif patch_type == "replace_section":
            if search_anchor and search_anchor in content:
                before, _, after = content.partition(search_anchor)
                after_section = after.split("\n#
                if len(after_section) > 1:
                    after = "\n#
                else:
                    after = ""
                return before + search_anchor + "\n" + patch_content + after
            else:
                raise ValueError(f"Search anchor not found: {search_anchor[:50]}")
        elif patch_type == "insert_after":
            if search_anchor and search_anchor in content:
                before, _, after = content.partition(search_anchor)
                return before + search_anchor + "\n" + patch_content + "\n" +
after
            else:
                raise ValueError(f"Search anchor not found: {search_anchor[:50]}")
        else:
            raise ValueError(f"Unknown patch_type: {patch_type}")
    async def rollback(self, patch_no: str, *, dry_run: bool = False) -> dict:
        from emily_core.repositories.evolution_repo import EvolutionRepo
        patch = await asyncio.to_thread(EvolutionRepo.get_patch_by_no, patch_no)
        if patch is None:
            return {"status": "error", "message": f"Patch {patch_no} not found"}
        if patch.status != "APPLIED":
            return {"status": "error", "message": f"Patch {patch_no} is not
APPLIED (current: {patch.status})"}
        if not patch.rollback_snapshot:
            return {"status": "error", "message": "No rollback snapshot
available"}
        data_dir = self._resolve_data_dir()
        target_path = Path(data_dir) / patch.target_path
        if dry_run:
            return {
                "status": "preview",
                "patch_no": patch_no,
                "target_path": str(target_path),
                "rollback_content_preview": patch.rollback_snapshot[:200],
            }
        target_path.write_text(patch.rollback_snapshot, encoding="utf-8")
        await asyncio.to_thread(
            EvolutionRepo.update_patch_status,
            patch_no,
            "ROLLED_BACK",

```

```

    )
    logger.info("Patch %s rolled back from %s", patch_no, target_path)
    return {"status": "rolled_back", "patch_no": patch_no}
async def approve(self, patch_no: str) -> dict:
    from emily_core.repositories.evolution_repo import EvolutionRepo
    patch = await asyncio.to_thread(EvolutionRepo.get_patch_by_no, patch_no)
    if patch is None:
        return {"status": "error", "message": f"Patch {patch_no} not found"}
    logger.info("Patch %s approved", patch_no)
    return {"status": "approved", "patch_no": patch_no}
async def reject(self, patch_no: str, reason: str = "") -> dict:
    from emily_core.repositories.evolution_repo import EvolutionRepo
    success = await asyncio.to_thread(
        EvolutionRepo.update_patch_status,
        patch_no,
        "REJECTED",
    )
    if not success:
        return {"status": "error", "message": f"Patch {patch_no} not found"}
    logger.info("Patch %s rejected: %s", patch_no, reason)
    return {"status": "rejected", "patch_no": patch_no}

```

文件编号: 94 文件路径: emily-core\emily_core\services\evolution\patch_generator.py

```

from __future__ import annotations
import asyncio
import json
import logging
import os
from pathlib import Path
logger = logging.getLogger("emily.patch_generator")
class PatchGenerator:
    CATEGORY_TO_TARGET = {
        "prompt": "prompts/session.md",
        "routing": "prompts/session.md",
        "sop": "sops/",
        "skill": "skills/",
        "hook": "config/hook_config.json",
        "user_memory": "user_memory/",
    }
    def __init__(self, llm_client=None, data_dir: str = ""):
        self.llm = llm_client
        self.data_dir = data_dir or os.environ.get("EMILY_DATA_DIR", "")
    async def generate(self, rule_nos: list[str] | None = None, *, dry_run: bool = False) -> list[dict]:
        from emily_core.repositories.evolution_repo import EvolutionRepo
        if rule_nos:
            rules = []
            for rn in rule_nos:
                rule = await asyncio.to_thread(EvolutionRepo.get_rule_by_no, rn)
                if rule:
                    rules.append(rule)

```



```

        else:
            rules = await asyncio.to_thread(EvolutionRepo.get_rules_by_status,
"CONFIRMED")
            if not rules:
                logger.info("No rules to generate patches for")
                return []
            data_dir = self._data_dir
            if not data_dir:
                candidate =
Path(__file__).resolve().parent.parent.parent.parent.parent / "emily-data"
                if candidate.exists():
                    data_dir = str(candidate)
            if not data_dir:
                logger.warning("Cannot determine emily-data directory")
                if dry_run:
                    return [{"status": "preview", "rules_count": len(rules)}]
                return []
            patches = []
            for rule in rules:
                category = rule.category
                target_path = self.CATEGORY_TO_TARGET.get(category, "")
                current_content = ""
                if target_path and data_dir:
                    full_path = Path(data_dir) / target_path
                    if full_path.exists() and full_path.is_file():
                        current_content = full_path.read_text(encoding="utf-8")[:5000]
                    else:
                        current_content = f"(文件不存在: {target_path})"
                if not self._llm or dry_run:
                    patches.append({
                        "status": "preview",
                        "rule_no": rule.rule_no,
                        "category": category,
                        "target_path": target_path,
                        "current_content_preview": current_content[:200],
                    })
                    continue
                try:
                    from emily_core.infrastructure.llm.prompt_loader import
load_prompt

                    template = load_prompt("evolution_patch")
                    user_message = template.replace("{rule_no}", rule.rule_no)
                    user_message = user_message.replace("{rule_title}", rule.title)
                    user_message = user_message.replace("{rule_description}",
rule.description)
                    user_message = user_message.replace("{rule_category}", category)
                    user_message = user_message.replace("{rule_suggested_action}",
rule.suggested_action)
                    user_message = user_message.replace("{current_target_content}",
current_content)
                    user_message = user_message.replace("{target_path}", target_path)
                    result = await self._llm.chat_json(template, user_message)
                except Exception as e:
                    logger.error("LLM patch generation failed for rule %s: %s",

```

```

rule.rule_no, e, exc_info=True)
        patches.append({"status": "llm_error", "rule_no": rule.rule_no,
"error": str(e)})
        continue
    try:
        patch_no = await
asyncio.to_thread(EvolutionRepo.generate_patch_no)
        await asyncio.to_thread(
            EvolutionRepo.create_patch,
            patch_no,
            rule.rule_no,
            target_type=result.get("target_type", category),
            target_path=result.get("target_path", target_path),
            patch_content=result.get("patch_content", ""),
            patch_type=result.get("patch_type", ""),
            search_anchor=result.get("search_anchor", ""),
            risk_level=result.get("risk_level", ""),
            risk_reasoning=result.get("risk_reasoning", ""),
            validation_criteria=result.get("validation_criteria", ""),
            expected_effect=result.get("expected_effect", ""),
        )
        result["patch_no"] = patch_no
        result["rule_no"] = rule.rule_no
        result["status"] = "generated"
        patches.append(result)
    except Exception as e:
        logger.error("Failed to save patch for rule %s: %s", rule.rule_no,
e, exc_info=True)
        patches.append({"status": "save_error", "rule_no": rule.rule_no,
"error": str(e)})
    return patches

```

文件编号: 95 文件路径: emily-core\emily_core\services\evolution\patch_validator.py

```

from __future__ import annotations
import asyncio
import json
import logging
from datetime import datetime, timedelta
logger = logging.getLogger("emily.patch_validator")
class PatchValidator:
    async def validate(self, patch_nos: list[str] | None = None, *, dry_run: bool
= False) -> list[dict]:
        from emily_core.repositories.evolution_repo import EvolutionRepo
        if patch_nos:
            patches = []
            for pn in patch_nos:
                p = await asyncio.to_thread(EvolutionRepo.get_patch_by_no, pn)
                if p:
                    patches.append(p)
        else:
            all_applied = await

```

```

asyncio.to_thread(EvolutionRepo.get_patches_by_status, "APPLIED")
    cutoff = (datetime.now() - timedelta(days=7)).isoformat()
    patches = [p for p in all_applied if p.applied_at and p.applied_at <
cutoff]
    if not patches:
        logger.info("No patches to validate")
        return []
    results = []
    for patch in patches:
        try:
            criteria = patch.validation_criteria
            applied_date = patch.applied_at[:10] if patch.applied_at else ""
            if not applied_date:
                result = {"status": "error", "patch_no": patch.patch_no,
"message": "No applied_at date"}
                results.append(result)
                continue
            applied_dt = datetime.fromisoformat(applied_date)
            before_start = (applied_dt - timedelta(days=7)).strftime("%Y-%m-
%d")

            before_end = applied_date
            after_start = applied_date
            after_dt = applied_dt + timedelta(days=7)
            after_end = after_dt.strftime("%Y-%m-%d")
            before_insights = await asyncio.to_thread(
                EvolutionRepo.get_insights_range, before_start, before_end
            )
            after_insights = await asyncio.to_thread(
                EvolutionRepo.get_insights_range, after_start, after_end
            )
            before_scores = [i.health_score for i in before_insights if
i.health_score > 0]
            after_scores = [i.health_score for i in after_insights if
i.health_score > 0]
            avg_before = sum(before_scores) / len(before_scores) if
before_scores else 0
            avg_after = sum(after_scores) / len(after_scores) if after_scores
else 0

            if dry_run:
                results.append({
                    "status": "preview",
                    "patch_no": patch.patch_no,
                    "avg_health_before": round(avg_before, 1),
                    "avg_health_after": round(avg_after, 1),
                    "criteria": criteria,
                })
                continue
            if avg_after >= avg_before or not before_scores:
                new_status = "CONFIRMED"
                decision = "improved_or_stable"
            else:
                new_status = "ROLLED_BACK"
                decision = "degraded"
            try:

```

```

        from emily_core.services.evolution.patch_applier import
PatchApplier

        applier = PatchApplier()
        await applier.rollback(patch.patch_no)
        logger.warning("Auto-rolled back patch %s due to
degradation", patch.patch_no)
        except Exception as e:
            logger.error("Failed to auto-rollback patch %s: %s",
patch.patch_no, e)
        now_str = datetime.now().isoformat()
        validation_result = json.dumps({
            "decision": decision,
            "avg_health_before": round(avg_before, 1),
            "avg_health_after": round(avg_after, 1),
            "criteria": criteria,
        })
        await asyncio.to_thread(
            EvolutionRepo.update_patch_status,
            patch.patch_no,
            new_status,
            validated_at=now_str,
            validation_result=validation_result,
        )
        results.append({
            "status": new_status.lower(),
            "patch_no": patch.patch_no,
            "decision": decision,
            "avg_health_before": round(avg_before, 1),
            "avg_health_after": round(avg_after, 1),
        })
    except Exception as e:
        logger.error("Failed to validate patch %s: %s", patch.patch_no, e,
exc_info=True)
        results.append({"status": "error", "patch_no": patch.patch_no,
"error": str(e)})
    return results

```

文件编号：96 文件路径：emily-core\emily_core\services\evolution\rule_inductor.py

```

from __future__ import annotations
import asyncio
import json
import logging
import os
from datetime import datetime, timedelta
from pathlib import Path
logger = logging.getLogger("emily.rule_inductor")
class RuleInductor:
    CATEGORY_TARGET_MAP = {
        "prompt": "prompts/session.md",
        "sop": "sops/",
        "skill": "skills/",

```

```

        "hook": "config/hook_config.json",
        "user_memory": "user_memory/",
        "routing": "prompts/session.md",
    }
    def __init__(self, llm_client=None, data_dir: str = ""):
        self._llm = llm_client
        self._data_dir = data_dir or os.environ.get("EMILY_DATA_DIR", "")
        async def induct(self, end_date: str, *, days: int = 7, dry_run: bool = False)
-> list[dict]:
        from emily_core.repositories.evolution_repo import EvolutionRepo
        start_dt = datetime.fromisoformat(end_date) - timedelta(days=days - 1)
        start_date = start_dt.strftime("%Y-%m-%d")
        insights = await asyncio.to_thread(
            EvolutionRepo.get_insights_range, start_date, end_date
        )
        if not insights:
            logger.info("No insights found in range %s ~ %s", start_date,
end_date)
            return []
        trend_data = self._compute_trends(insights)
        recurring_anomalies = self._find_recurring_anomalies(insights)
        insights_data = []
        for ins in insights:
            insight_json = {}
            try:
                insight_json = json.loads(ins.insight_text) if ins.insight_text
            else {}
            except json.JSONDecodeError:
                pass
            insights_data.append({
                "date": ins.insight_date,
                "sop_hit_rate": ins.sop_hit_rate,
                "fallback_rate": ins.fallback_rate,
                "health_score": ins.health_score,
                "summary": insight_json.get("summary", ""),
                "key_findings": insight_json.get("key_findings", []),
            })
        if not self._llm or dry_run:
            logger.info("Preview mode: %d insights loaded, %d recurring
anomalies", len(insights), len(recurring_anomalies))
            return [{"status": "preview", "trend_data": trend_data,
"recurring_anomalies": recurring_anomalies}]
        try:
            from emily_core.infrastructure.llm.prompt_loader import load_prompt
            template = load_prompt("evolution_rule")
            date_range = f"{start_date} ~ {end_date} ({days}天)"
            user_message = template.replace("{date_range}", date_range)
            user_message = user_message.replace("{insights_json}",
json.dumps(insights_data, ensure_ascii=False, default=str))
            user_message = user_message.replace("{trend_data}",
json.dumps(trend_data, ensure_ascii=False))
            user_message = user_message.replace("{recurring_anomalies}",
json.dumps(recurring_anomalies, ensure_ascii=False))
            user_message = user_message.replace("{days}", str(days))

```

```

        result = await self._llm.chat_json(template, user_message)
        rules = result.get("rules", [])
    except Exception as e:
        logger.error("LLM rule induction failed: %s", e, exc_info=True)
        return [{"status": "llm_error", "error": str(e)}]
    saved_rules = []
    for rule in rules:
        try:
            rule_no = await asyncio.to_thread(EvolutionRepo.generate_rule_no)
            await asyncio.to_thread(
                EvolutionRepo.create_rule,
                rule_no,
                rule.get("title", ""),
                description=rule.get("description", ""),
                evidence_insight_ids=json.dumps(rule.get("evidence_dates",
                    [])), ensure_ascii=False),
                category=rule.get("category", ""),
                confidence=rule.get("confidence", 0.0),
                suggested_action=rule.get("suggested_action", ""),
                impact_estimate=rule.get("impact_estimate", ""),
            )
            rule["rule_no"] = rule_no
            saved_rules.append(rule)
            self._write_yaml(rule_no, rule)
        except Exception as e:
            logger.error("Failed to save rule: %s", e, exc_info=True)
    return saved_rules

def _compute_trends(self, insights) -> dict:
    if not insights:
        return {}
    dates = [i.insight_date for i in insights]
    sop_rates = [i.sop_hit_rate for i in insights]
    fallback_rates = [i.fallback_rate for i in insights]
    health_scores = [i.health_score for i in insights]
    return {
        "period_dates": dates,
        "sop_hit_rate_trend": sop_rates,
        "fallback_rate_trend": fallback_rates,
        "health_score_trend": health_scores,
        "sop_hit_rate_change": round(sop_rates[-1] - sop_rates[0], 4) if
len(sop_rates) > 1 else 0,
    }

def _find_recurring_anomalies(self, insights) -> list[dict]:
    anomaly_map = {}
    for ins in insights:
        flags = []
        try:
            flags = json.loads(ins.anomaly_flags) if ins.anomaly_flags else []
        except json.JSONDecodeError:
            pass
        for flag in flags:
            if flag not in anomaly_map:
                anomaly_map[flag] = []
            anomaly_map[flag].append(ins.insight_date)

```

```

        recurring = []
        for flag, dates in anomaly_map.items():
            if len(dates) >= 2:
                recurring.append({"flag": flag, "dates": dates, "count":
len(dates)})
        return recurring
    def _write_yaml(self, rule_no: str, rule: dict) -> None:
        data_dir = self._data_dir
        if not data_dir:
            candidate =
Path(__file__).resolve().parent.parent.parent.parent.parent / "emily-data"
            if candidate.exists():
                data_dir = str(candidate)
        if not data_dir:
            return
        rules_dir = Path(data_dir) / "evolution" / "rules"
        rules_dir.mkdir(parents=True, exist_ok=True)
        yaml_lines = [
            f"rule_no: {rule_no}",
            f"title: \"{rule.get('title', '')}\"",
            f"description: |",
        ]
        desc = rule.get("description", "")
        for line in desc.split("\n"):
            yaml_lines.append(f"  {line}")
        yaml_lines.append(f"category: {rule.get('category', '')}")
        yaml_lines.append(f"confidence: {rule.get('confidence', 0.0)}")
        yaml_lines.append(f"evidence:")
        for d in rule.get("evidence_dates", []):
            yaml_lines.append(f"  - date: \"{d}\"")
        yaml_lines.append(f"suggested_action: \"{rule.get('suggested_action',
'')}\"")
        yaml_lines.append(f"impact_estimate: \"{rule.get('impact_estimate',
'')}\"")
        yaml_lines.append(f"status: DRAFT")
        yaml_lines.append(f"created_at: \"{datetime.now().isoformat()}\"")
        filepath = rules_dir / f"{rule_no}.yaml"
        filepath.write_text("\n".join(yaml_lines), encoding="utf-8")

```

文件编号: 97 文件路径: emily-core\emily_core\services\file_service.py

```

import logging
from typing import Optional
from ..repositories.file_repo import FileRepository
from ..adapters.standard.command import FileCommand
from ..infrastructure.database.models import File as FileModel
logger = logging.getLogger("emily.service.file")
class FileService:
    def __init__(self):
        self.repo = FileRepository()
    def create_file_record(self, cmd: FileCommand) -> FileModel:
        file_no = self.repo.generate_file_no()

```

```

    f = self.repo.create(
        file_no=file_no,
        filename=cmd.filename,
        project_id=cmd.project_id or None,
        message_id=cmd.source_message_id or None,
        file_type=cmd.file_type or None,
        storage_path=cmd.storage_path or None,
        file_size=cmd.file_size,
        uploaded_by=cmd.uploaded_by or None,
        parse_status="pending",
        file_category=cmd.file_category or "OTHER",
    )
    logger.info("File %s recorded: %s", file_no, cmd.filename)
    return f

@staticmethod
def format_reply(f: FileModel) -> str:
    return (
        f"📁 文件已归档\n"
        f"-----\n"
        f"编号: {f.file_no}\n"
        f"文件名: {f.filename}\n"
        f"-----"
    )

def list_by_category(
    self,
    project_id: str | None = None,
    project_ids: list[str] | None = None,
    file_category: str | None = None,
    limit: int = 50,
) -> list[FileModel]:
    return self.repo.query_by_category(
        project_id=project_id,
        project_ids=project_ids,
        file_category=file_category,
        limit=limit,
    )

def update_file_category(self, file_id: str, file_category: str, operator_id:
str = "") -> FileModel | None:
    from ..infrastructure.database.models import FileCategory
    validated = FileCategory.validate(file_category)
    result = self.repo.update_category(file_id, validated)
    if result:
        logger.info("File category updated: %s → %s by %s", result.file_no,
validated, operator_id)
    return result

def get_category_summary(self, project_id: str | None = None) -> dict:
    from ..infrastructure.database.models import FileCategory
    counts = self.repo.count_by_category(project_id=project_id)
    return {
        "by_category": {
            FileCategory.display(cat): count
            for cat, count in counts.items()
        },
    },

```



```

        "total": sum(counts.values()),
    }

```

文件编号: 98 文件路径: emily-core\emily_core\services\file_storage_service.py

```

import logging
import os
from datetime import datetime, timezone
from pathlib import Path
from typing import Optional
from ..repositories.file_repo import FileRepository
logger = logging.getLogger("emily.service.file_storage")
class FileStorageService:
    def __init__(self, storage_root: str = "", platform: str = ""):
        if not storage_root:
            storage_root = str(
                Path(__file__).parent.parent.parent / "data" / "files"
            )
        self._storage_root = Path(storage_root)
        self._platform = platform or "napcat"
    def ensure_dir(self, date_str: str | None = None) -> Path:
        if date_str is None:
            date_str = datetime.now(timezone.utc).strftime("%Y-%m")
        elif len(date_str) > 7:
            date_str = date_str[:7]
        target = self._storage_root / self._platform / date_str
        target.mkdir(parents=True, exist_ok=True)
        return target
    @staticmethod
    def generate_file_no() -> str:
        return FileRepository.generate_file_no()
    async def download_from_url(self, url: str) -> bytes | None:
        try:
            import aiohttp
        except ImportError:
            logger.warning("aiohttp not available, falling back to urllib")
            import urllib.request
        try:
            with urllib.request.urlopen(url, timeout=30) as resp:
                return resp.read()
        except Exception as e:
            logger.warning("Download failed (urllib): %s - %s", url[:100], e)
            return None
        try:
            async with aiohttp.ClientSession() as session:
                async with session.get(url,
                    timeout=aiohttp.ClientTimeout(total=30)) as resp:
                    if resp.status == 200:
                        return await resp.read()
                    logger.warning("Download failed: HTTP %d - %s", resp.status,
                        url[:100])
            return None

```

```

        except Exception as e:
            logger.warning("Download failed (aiohttp): %s - %s", url[:100], e)
            return None
def store_attachment(
    self,
    message_id: str,
    attachment_url: str,
    attachment_type: int,
    source_filename: str = "",
    mime_type: str = "",
    file_size: int = 0,
) -> dict | None:
    import urllib.request
    file_no = self.generate_file_no()
    today_str = datetime.now(timezone.utc).strftime("%Y-%m")
    target_dir = self.ensure_dir(today_str)
    ext = self._infer_extension(source_filename, mime_type)
    filename = f"{file_no}{ext}"
    local_path = target_dir / filename
    try:
        with urllib.request.urlopen(attachment_url, timeout=30) as resp:
            data = resp.read()
    except Exception as e:
        logger.warning("Attachment download failed: %s - %s",
attachment_url[:100], e)
        return None
    try:
        local_path.write_bytes(data)
        logger.info("File saved: %s (%d bytes)", local_path, len(data))
    except Exception as e:
        logger.warning("File write failed: %s - %s", local_path, e)
        return None
    return self._finalize_store(
        data=data,
        file_no=file_no,
        message_id=message_id,
        attachment_url=attachment_url,
        attachment_type=attachment_type,
        source_filename=source_filename,
        mime_type=mime_type,
        file_size=file_size,
        local_path=local_path,
    )
async def store_attachment_async(
    self,
    message_id: str,
    attachment_url: str,
    attachment_type: int,
    source_filename: str = "",
    mime_type: str = "",
    file_size: int = 0,
) -> dict | None:
    data = await self.download_from_url(attachment_url)
    if data is None:

```

```
        return None
    file_no = self.generate_file_no()
    today_str = datetime.now(timezone.utc).strftime("%Y-%m")
    target_dir = self.ensure_dir(today_str)
    ext = self._infer_extension(source_filename, mime_type)
    filename = f"{file_no}{ext}"
    local_path = target_dir / filename
    try:
        local_path.write_bytes(data)
        logger.info("File saved: %s (%d bytes)", local_path, len(data))
    except Exception as e:
        logger.warning("File write failed: %s - %s", local_path, e)
        return None
    result = self._finalize_store(
        data=data,
        file_no=file_no,
        message_id=message_id,
        attachment_url=attachment_url,
        attachment_type=attachment_type,
        source_filename=source_filename,
        mime_type=mime_type,
        file_size=file_size,
        local_path=local_path,
    )
    if result:
        result["attachment_type"] = attachment_type
    return result

@staticmethod
def _infer_extension(source_filename: str, mime_type: str) -> str:
    if source_filename and "." in source_filename:
        return os.path.splitext(source_filename)[1]
    if mime_type:
        ext_map = {
            "image/jpeg": ".jpg", "image/png": ".png", "image/gif": ".gif",
            "application/pdf": ".pdf", "application/zip": ".zip",
            "audio/amr": ".amr", "audio/mp3": ".mp3",
            "video/mp4": ".mp4",
        }
        return ext_map.get(mime_type, "")
    return ""

def _finalize_store(
    self,
    data: bytes,
    file_no: str,
    message_id: str,
    attachment_url: str,
    attachment_type: int,
    source_filename: str,
    mime_type: str,
    file_size: int,
    local_path: Path,
) -> dict | None:
    actual_size = len(data)
    try:
```

```

        file_record = FileRepository.create(
            file_no=file_no,
            filename=source_filename or local_path.name,
            project_id=None,
            message_id=message_id,
            file_type=_attachment_type_label(attachment_type),
            storage_path=str(local_path.relative_to(self._storage_root)),
            file_size=actual_size or file_size,
        )
    except Exception as e:
        logger.warning("File record creation failed: %s", e)
        return None
    try:
        from ..repositories.chat_archive_repo import ChatArchiveRepository
        ChatArchiveRepository.create_attachment(
            message_id=message_id,
            attachment_type=attachment_type,
            file_url=attachment_url,
            file_size=actual_size or file_size,
            mime_type=mime_type,
        )
        from ..infrastructure.database.session import get_session
        from ..infrastructure.database.models import MessageAttachment as MA
        with get_session() as session:
            att = (
                session.query(MA)
                .filter(MA.message_id == message_id, MA.file_url ==
attachment_url)
                .order_by(MA.created_at.desc())
                .first()
            )
            if att:
                att.file_id = file_record.id
                att.local_path =
str(local_path.relative_to(self._storage_root))
    except Exception as e:
        logger.warning("Attachment record update failed: %s", e)
    return {
        "file_id": file_record.id,
        "file_no": file_no,
        "local_path": str(local_path),
        "file_size": actual_size or file_size,
    }

def get_local_path(self, file_no: str) -> str | None:
    from ..infrastructure.database.session import get_session
    from ..infrastructure.database.models import File as FileModel
    with get_session() as session:
        f = session.query(FileModel).filter(FileModel.file_no ==
file_no).first()
        if f and f.storage_path:
            return str(self._storage_root / f.storage_path)
    return None

def list_files_for_conversation(self, conversation_id: str) -> list[dict]:
    from ..repositories.chat_archive_repo import ChatArchiveRepository

```

```

        return ChatArchiveRepository.get_files_for_conversation(conversation_id)
def _attachment_type_label(t: int) -> str:
    return {1: "image", 2: "image", 3: "file", 4: "voice", 5: "video"}.get(t,
"file")

```

文件编号: 99 文件路径: emily-core\emily_core\services\initialization_checker.py

```

from __future__ import annotations
import json
import logging
from datetime import datetime, timezone, timedelta
from typing import Optional
from ..infrastructure.database.session import get_session
from sqlalchemy import or_
from ..infrastructure.database.models import (
    Project, User, CompanyInfo, ProjectNode, NodeDependency,
)
logger = logging.getLogger("emily.initialization_checker")
BEIJING_TZ = timezone(timedelta(hours=8))
class InitializationChecker:
    def check(self, project_id: str) -> dict:
        with get_session() as session:
            project = session.query(Project).filter(
                Project.id == project_id,
                or_(Project.is_deleted == False, Project.is_deleted == None),
            ).first()
            if project is None:
                return {
                    "project_id": project_id,
                    "tier": 0,
                    "tier_label": "未开始（项目不存在）",
                    "is_activated": False,
                    "items": {},
                    "missing": ["项目不存在"],
                    "summary_by_tier": {},
                }
            t1 = {}
            t1["T1_project_name"] = bool(project.name and project.name.strip() and
project.name != "未命名项目")
            t1["T1_project_code"] = bool(project.code and project.code.strip())
            t1["T1_project_address"] = bool(project.address and
project.address.strip())
            desc = (project.description or "").strip()
            t1["T1_project_type"] = bool(desc) or (project.lifecycle_stage or 0)
            != 0
            t1["T1_lifecycle_stage"] = (project.lifecycle_stage or 0) != 0
            admins = session.query(User).filter(
                User.project_id == project_id,
                User.is_deleted == False,
                User.is_admin == True,
            ).all()
            t1["T1_admin_user"] = len(admins) > 0

```

```

admins)
    t1["T1_admin_email"] = any(u.email and u.email.strip() for u in
    t1_done = sum(1 for v in t1.values() if v)
    t1_total = len(t1)
    t2 = {}
    users_in_project = session.query(User).filter(
        User.project_id == project_id,
        User.is_deleted == False,
    ).all()
    company_ids = list(set(u.company for u in users_in_project if
    u.company))
    companies =
    session.query(CompanyInfo).filter(CompanyInfo.id.in_(company_ids)).all() if
    company_ids else []
    company_types = [c.type for c in companies if c.type]
    t2["T2_builder_company"] = "建设单位" in company_types
    t2["T2_management_company"] = any(t in company_types for t in ["代建单
    位", "项目管理", "建设单位"])
    t2["T2_general_contractor"] = any(t in company_types for t in ["施工单
    位", "总包", "施工总承包"])
    t2["T2_supervisor_company"] = any(t in company_types for t in ["监理单
    位", "监理"])
    has_pm = False
    for u in users_in_project:
        try:
            positions = json.loads(u.position or "[]")
            if "项目经理" in positions:
                has_pm = True
                break
        except (json.JSONDecodeError, TypeError):
            pass
    t2["T2_project_manager"] = has_pm
    has_cs = False
    for u in users_in_project:
        try:
            positions = json.loads(u.position or "[]")
            if any(p in positions for p in ["总监理工程师", "总监理"]):
                has_cs = True
                break
        except (json.JSONDecodeError, TypeError):
            pass
    t2["T2_chief_supervisor"] = has_cs
    t2_done = sum(1 for v in t2.values() if v)
    t2_total = len(t2)
    t3 = {}
    nodes = session.query(ProjectNode).filter(
        ProjectNode.project_id == project_id,
        ProjectNode.is_discarded == False,
    ).all()
    milestones = [n for n in nodes if getattr(n, 'node_type', '') ==
    'MILESTONE']
    t3["T3_node_tree_created"] = len(milestones) > 0
    t3["T3_milestone_deadlines"] = len(milestones) > 0 and all(m.deadline
    for m in milestones)

```

```

wp_and_ms = [n for n in nodes if getattr(n, 'node_type', '') in
('MILESTONE', 'WORK_PACKAGE')]
t3["T3_node_responsible_persons"] = len(wp_and_ms) > 0 and
all(n.responsible_user_id for n in wp_and_ms)
sop_count = 0
try:
    from ..skill.registry import SkillRegistry
    from pathlib import Path
    skill_dir = "/app/skills"
    if not Path(skill_dir).exists():
        skill_dir = ""
    if not skill_dir:
        dev_dir = str(Path(__file__).resolve().parents[2] / "emily-
data" / "skills")
        if Path(dev_dir).exists():
            skill_dir = dev_dir
    if skill_dir:
        reg = SkillRegistry(skill_directory=skill_dir)
        reg.load()
        sop_count = len(reg.list_sop_ids())
except Exception:
    pass
t3["T3_sop_adapted"] = sop_count > 0
t3["T3_pm_im_bound"] = False
if has_pm:
    for u in users_in_project:
        try:
            positions = json.loads(u.position or "[]")
            if "项目经理" in positions and u.im_bindings:
                t3["T3_pm_im_bound"] = True
                break
        except (json.JSONDecodeError, TypeError):
            pass
t3_done = sum(1 for v in t3.values() if v)
t3_total = len(t3)
t4 = {}
expected_types = {"建设单位", "代建单位", "施工单位", "监理单位", "设计单
位"}
t4["T4_all_companies"] = expected_types.issubset(set(company_types))
active_nodes = [n for n in nodes if n.status not in
("NOT_ACTIVATED",)]
t4["T4_all_node_responsible"] = len(active_nodes) > 0 and
all(n.responsible_user_id for n in active_nodes)
wp_nodes = [n for n in nodes if getattr(n, 'node_type', '') ==
'WORK_PACKAGE']
wp_node_ids = [n.node_id for n in wp_nodes]
dep_count = 0
if wp_node_ids:
    deps = session.query(NodeDependency).filter(
        NodeDependency.node_id.in_(wp_node_ids)
    ).count()
    dep_count = deps
t4["T4_dependency_coverage"] = len(wp_nodes) > 0 and dep_count >=
len(wp_nodes) * 0.5

```

```
file_count = 0
try:
    from ..infrastructure.database.models import File
    file_count = session.query(File).filter(File.project_id ==
project_id).count()
except Exception:
    pass
t4["T4_knowledge_filled"] = file_count >= 5
t4["T4_morning_report_sent"] = False
try:
    from ..infrastructure.database.models import SchedulerJobLog
    log = session.query(SchedulerJobLog).filter(
        SchedulerJobLog.action_type == "morning_report",
        SchedulerJobLog.status == "success",
    ).first()
    t4["T4_morning_report_sent"] = log is not None
except Exception:
    pass
t4_done = sum(1 for v in t4.values() if v)
t4_total = len(t4)
all_items = {**t1, **t2, **t3, **t4}
total_items = len(all_items)
done_items = sum(1 for v in all_items.values() if v)
tier = 0
if t1_done >= t1_total:
    tier = 1
if tier >= 1 and t2_done >= t2_total:
    tier = 2
if tier >= 2 and t3_done >= t3_total:
    tier = 3
if tier >= 3 and t4_done >= t4_total:
    tier = 4
tier_labels = {
    0: "T0 未开始",
    1: "T1 可识别",
    2: "T2 有组织",
    3: "T3 可运转",
    4: "T4 充分运转",
}
missing = [k for k, v in all_items.items() if not v]
missing_descriptions = {
    "T1_project_name": "项目名称未填写",
    "T1_project_code": "项目编号未填写",
    "T1_project_address": "项目地址未填写",
    "T1_project_type": "项目类型无法区分",
    "T1_lifecycle_stage": "生命周期阶段未设定",
    "T1_admin_user": "无项目管理员账户",
    "T1_admin_email": "管理员无可邮箱",
    "T2_builder_company": "缺少建设单位",
    "T2_management_company": "缺少代建/管理单位",
    "T2_general_contractor": "缺少施工总承包单位",
    "T2_supervisor_company": "缺少监理单位",
    "T2_project_manager": "缺少项目经理",
    "T2_chief_supervisor": "缺少总监理工程师",
}
```



```

        "T3_node_tree_created": "节点树未创建",
        "T3_milestone_deadlines": "里程碑无截止日期",
        "T3_node_responsible_persons": "关键节点无责任人",
        "T3_sop_adapted": "无适配 SOP",
        "T3_pm_im_bound": "项目经理未绑定 IM",
        "T4_all_companies": "参建单位不全",
        "T4_all_node_responsible": "部分节点无责任人",
        "T4_dependency_coverage": "节点依赖关系不足50%",
        "T4_knowledge_filled": "知识库文件不足5个",
        "T4_morning_report_sent": "晨报从未成功发送",
    }
    missing_desc = [missing_descriptions.get(k, k) for k in missing]
    summary_by_tier = {
        "T1": {"done": t1_done, "total": t1_total, "pass": t1_done >=
t1_total},
        "T2": {"done": t2_done, "total": t2_total, "pass": t2_done >=
t2_total},
        "T3": {"done": t3_done, "total": t3_total, "pass": t3_done >=
t3_total},
        "T4": {"done": t4_done, "total": t4_total, "pass": t4_done >=
t4_total},
    }
    return {
        "project_id": project_id,
        "tier": tier,
        "tier_label": tier_labels.get(tier, "未知"),
        "is_activated": tier >= 3,
        "items": all_items,
        "missing": missing_desc,
        "summary_by_tier": summary_by_tier,
        "total_done": done_items,
        "total_items": total_items,
    }

```

文件编号：100 文件路径：emily-core\emily_core\services\meeting_service.py

```

import json
import logging
from typing import Optional
from ..repositories.meeting_repo import MeetingRepository
from ..adapters.standard.command import MeetingCommand
from ..infrastructure.database.models import Meeting
logger = logging.getLogger("emily.service.meeting")
class MeetingService:
    def __init__(self):
        self.repo = MeetingRepository()
    def create_meeting(self, cmd: MeetingCommand) -> Meeting:
        meeting_no = self.repo.generate_meeting_no()
        meeting = self.repo.create(
            meeting_no=meeting_no,
            title=cmd.title,
            project_id=cmd.project_id or None,

```

```

        summary=cmd.summary or None,
        attendees=cmd.attendees or [],
        source_message_id=cmd.source_message_id or None,
        created_by=cmd.creator_id or None,
    )
    logger.info("Meeting %s archived: %s", meeting_no, cmd.title)
    return meeting
@staticmethod
def format_reply(meeting: Meeting) -> str:
    try:
        att = json.loads(meeting.attendees) if meeting.attendees else []
    except (json.JSONDecodeError, TypeError):
        att = []
    attendees_str = ", ".join(att) if att else "未指定"
    return (
        f"📄 会议纪要已归档\n"
        f"—————\n"
        f"编号: {meeting.meeting_no}\n"
        f"标题: {meeting.title}\n"
        f"参会人: {attendees_str}\n"
        f"—————"
    )

```

文件编号：101 文件路径：emily-core\emily_core\services\message_service.py

```

import logging
from typing import Optional
from ..repositories.message_repo import MessageRepository
from ..adapters.standard.message import StandardMessage
from ..adapters.standard.route_decision import RouteDecision
from ..infrastructure.database.models import Message
logger = logging.getLogger("emily.service.message")
class MessageService:
    def __init__(self):
        self.repo = MessageRepository()
    def record_message(
        self,
        event_id: str,
        msg: StandardMessage,
        decision: RouteDecision,
    ) -> Message:
        existing = self.repo.get_by_event_id(event_id)
        if existing:
            logger.debug("Message already exists: %s", event_id)
            return existing
        return self.repo.create_from_standard(event_id, msg, decision)
    def mark_processed(self, message_id: str) -> None:
        self.repo.mark_processed(message_id)
    def get_by_event_id(self, event_id: str) -> Optional[Message]:
        return self.repo.get_by_event_id(event_id)
    def bind_sender(self, message_id: str, user_id: str) -> None:
        self.repo.update_sender_user_id(message_id, user_id)

```

```
def update_route_result(self, message_id: str, intent: str, project_id:
Optional[str] = None) -> None:
    self.repo.update_intent(message_id, intent)
    if project_id:
        self.repo.update_project_id(message_id, project_id)
    logger.info("Route result saved: msg=%s, intent=%s, project=%s",
message_id, intent, project_id)
```

文件编号: 102 文件路径: emily-core\emily_core\services\node_batch.py

```
from __future__ import annotations
import asyncio
import hashlib
import logging
import re
from typing import Any
logger = logging.getLogger("emily.node_batch")
def flatten_nodes(
    nodes: list[dict],
    *,
    project_id: str = "",
    parent_node_id: str = "",
) -> list[dict]:
    flat: list[dict] = []
    for node_def in nodes:
        node_id = node_def.get("node_id", "")
        if not node_id:
            node_name = node_def.get("node_name", "未命名")
            node_id = _generate_node_id(node_name, project_id)
            logger.debug("自动生成 node_id: %s → %s", node_name, node_id)
        record = {
            "node_id": node_id,
            "node_name": node_def.get("node_name", ""),
            "deadline": node_def.get("deadline", ""),
            "owner_dept_id": node_def.get("owner_dept_id", "项目总"),
            "related_company_id": node_def.get("related_company_id", "建设单位"),
            "stage_id": node_def.get("stage_id", 0),
            "remark": node_def.get("remark", ""),
            "sort_order": node_def.get("sort_order", 0),
            "child_weight": node_def.get("child_weight", 1.0),
            "deliverables": node_def.get("deliverables", []),
            "dependencies": node_def.get("dependencies", []),
            "parent_node_id": parent_node_id,
        }
        flat.append(record)
        children = node_def.get("children", [])
        if children:
            child_records = flatten_nodes(
                children,
                project_id=project_id,
                parent_node_id=node_id,
            )
```

```

        flat.extend(child_records)
    return flat

def _generate_node_id(node_name: str, project_id: str) -> str:
    clean = re.sub(r'^[\u00a0-\u009f]', '', node_name)
    hash_part = hashlib.md5(f"{clean}:{project_id}".encode()).hexdigest()
    [:4].upper()
    return f"NODE-{hash_part}"

async def _resolve_deliverable_id(
    node_id: str,
    deliverable_name: str,
) -> str | None:
    from ..repositories.node_repo import NodeDeliverableRepo
    delivs = await asyncio.to_thread(NodeDeliverableRepo.find_by_node, node_id)
    for d in delivs:
        if d.deliverable_name == deliverable_name:
            return d.deliverable_id
    for d in delivs:
        if deliverable_name in d.deliverable_name or d.deliverable_name in
deliverable_name:
            return d.deliverable_id
    return None

async def _resolve_node_id_by_name(
    project_id: str,
    node_name: str,
) -> str | None:
    from ..repositories.node_repo import ProjectNodeRepo
    nodes = await asyncio.to_thread(ProjectNodeRepo.find_by_project, project_id)
    for n in nodes:
        if n.node_name == node_name:
            return n.node_id
    for n in nodes:
        if node_name in n.node_name or n.node_name in node_name:
            return n.node_id
    return None

async def create_node_tree(
    project_id: str,
    creator_id: str,
    nodes: list[dict],
    *,
    auto_activate: bool = True,
    dry_run: bool = False,
) -> list[dict]:
    from .node_service import NodeService
    from .node_commands import (
        CreateNodeCommand,
        CreateDeliverableCommand,
        AddDependencyCommand,
        MountChildCommand,
    )
    from ..repositories.permission_repo import PermissionRepository
    svc = NodeService(user_repo=PermissionRepository())
    results: list[dict] = []
    flat_nodes = flatten_nodes(nodes, project_id=project_id)
    logger.info("Phase 1: 创建 %d 个节点", len(flat_nodes))

```

```
for fn in flat_nodes:
    node_id = fn["node_id"]
    existing = await asyncio.to_thread(
        svc._node_repo.get_by_node_id, node_id,
    )
    if existing and not existing.is_discarded:
        logger.info(" 节点 %s 已存在, 跳过创建", node_id)
        results.append({
            "node_id": node_id,
            "success": True,
            "phase": "create_node",
            "message": f"节点「{fn['node_name']}」已存在, 跳过",
            "skipped": True,
        })
        continue
    if dry_run:
        results.append({
            "node_id": node_id,
            "success": True,
            "phase": "create_node",
            "message": f"[DRY-RUN] 将创建节点「{fn['node_name']}」",
            "dry_run": True,
        })
        continue
    cmd = CreateNodeCommand(
        project_id=project_id,
        node_id=node_id,
        node_name=fn["node_name"],
        deadline=fn.get("deadline", ""),
        owner_dept_id=fn.get("owner_dept_id", "项目总"),
        related_company_id=fn.get("related_company_id", "建设单位"),
        stage_id=fn.get("stage_id", 0),
        parent_node_id="",
        creator_id=creator_id,
        remark=fn.get("remark", ""),
        sort_order=fn.get("sort_order", 0),
    )
    try:
        r = await svc.create_node(cmd)
        results.append({
            "node_id": node_id,
            "success": r.success,
            "phase": "create_node",
            "status": r.status,
            "message": r.message,
        })
        if r.success:
            logger.info("  ✓ 节点 %s 创建成功: %s", node_id, r.message)
        else:
            logger.error("  ✕ 节点 %s 创建失败: %s", node_id, r.message)
    except Exception as e:
        results.append({
            "node_id": node_id,
            "success": False,
```

```

        "phase": "create_node",
        "message": str(e),
    })
    logger.error(" X 节点 %s 创建异常: %s", node_id, e)
    deliverable_specs: list[dict] = []
    for fn in flat_nodes:
        for d in fn.get("deliverables", []):
            deliverable_specs.append({"node_id": fn["node_id"], **d})
    if deliverable_specs:
        logger.info("Phase 2: 添加 %d 个成果", len(deliverable_specs))
    for ds in deliverable_specs:
        node_id = ds["node_id"]
        deliv_name = ds.get("name", ds.get("deliverable_name", "未命名成果"))
        if dry_run:
            results.append({
                "node_id": node_id,
                "success": True,
                "phase": "create_deliverable",
                "message": f"[DRY-RUN] 将为节点 {node_id} 添加成果「{deliv_name}」",
                "dry_run": True,
            })
            continue
        cmd = CreateDeliverableCommand(
            node_id=node_id,
            deliverable_name=deliv_name,
            target_amount=float(ds.get("target", ds.get("target_amount", 1))),
            unit=ds.get("unit", "份"),
            is_required=ds.get("is_required", True),
            operator_id=creator_id,
        )
        try:
            r = await svc.create_deliverable(cmd)
            results.append({
                "node_id": node_id,
                "success": r.success,
                "phase": "create_deliverable",
                "message": r.message,
            })
            if r.success:
                logger.info(" ✓ 成果「%s」→ %s", deliv_name, node_id)
            else:
                logger.error(" X 成果「%s」→ %s: %s", deliv_name, node_id,
r.message)
        except Exception as e:
            results.append({
                "node_id": node_id,
                "success": False,
                "phase": "create_deliverable",
                "message": str(e),
            })
            logger.error(" X 成果「%s」异常: %s", deliv_name, e)
    mount_specs: list[dict] = []
    for fn in flat_nodes:
        parent_node_id = fn.get("parent_node_id")

```

```

        if parent_node_id:
            mount_specs.append({
                "parent_node_id": parent_node_id,
                "child_node_id": fn["node_id"],
                "child_weight": fn.get("child_weight", 1.0),
            })
    if mount_specs:
        logger.info("Phase 3: 挂载 %d 个父子关系", len(mount_specs))
    for ms in mount_specs:
        if dry_run:
            results.append({
                "node_id": ms["child_node_id"],
                "success": True,
                "phase": "mount_child",
                "message": f"[DRY-RUN] 将 {ms['child_node_id']} 挂载到 {ms['parent_node_id']}",
                "dry_run": True,
            })
            continue
        cmd = MountChildCommand(
            parent_node_id=ms["parent_node_id"],
            child_node_id=ms["child_node_id"],
            child_weight=ms["child_weight"],
            operator_id=creator_id,
        )
        try:
            r = await svc.mount_child(cmd)
            results.append({
                "node_id": ms["child_node_id"],
                "success": r.success,
                "phase": "mount_child",
                "message": r.message,
            })
            if r.success:
                logger.info("  ✓ 挂载 %s → %s", ms["child_node_id"],
ms["parent_node_id"])
            else:
                logger.error("  ✗ 挂载 %s → %s: %s", ms["child_node_id"],
ms["parent_node_id"], r.message)
        except Exception as e:
            results.append({
                "node_id": ms["child_node_id"],
                "success": False,
                "phase": "mount_child",
                "message": str(e),
            })
            logger.error("  ✗ 挂载异常: %s", e)
    dep_specs: list[dict] = []
    for fn in flat_nodes:
        for dep in fn.get("dependencies", []):
            dep_specs.append({"node_id": fn["node_id"], **dep})
    if dep_specs:
        logger.info("Phase 4: 添加 %d 个依赖关系", len(dep_specs))
    for ds in dep_specs:

```

```

        downstream_node_id = ds["node_id"]
        upstream_node_id = ds.get("upstream_node_id", "")
        upstream_node_name = ds.get("upstream_node_name", ds.get("node_name", ""))
        if not upstream_node_id and upstream_node_name:
            upstream_node_id = await _resolve_node_id_by_name(project_id,
upstream_node_name)
        deliverable_id = ds.get("depends_on_deliverable_id", "")
        deliverable_name = ds.get("deliverable_name", "")
        if not deliverable_id and deliverable_name and upstream_node_id:
            deliverable_id = await _resolve_deliverable_id(upstream_node_id,
deliverable_name)
        if not deliverable_id:
            msg = (
                f"无法解析依赖: 节点 {downstream_node_id} 的依赖"
                f"(upstream={upstream_node_id or upstream_node_name}, "
                f"deliverable={deliverable_id or deliverable_name}) 未找到"
            )
            logger.error(" X %s", msg)
            results.append({
                "node_id": downstream_node_id,
                "success": False,
                "phase": "add_dependency",
                "message": msg,
            })
            continue
        weight = float(ds.get("weight", 1.0))
        if dry_run:
            results.append({
                "node_id": downstream_node_id,
                "success": True,
                "phase": "add_dependency",
                "message": f"[DRY-RUN] 将为 {downstream_node_id} 添加依赖
{deliverable_id} (weight={weight})",
                "dry_run": True,
            })
            continue
        cmd = AddDependencyCommand(
            node_id=downstream_node_id,
            depends_on_deliverable_id=deliverable_id,
            weight=weight,
            operator_id=creator_id,
        )
        try:
            r = await svc.add_dependency(cmd)
            results.append({
                "node_id": downstream_node_id,
                "success": r.success,
                "phase": "add_dependency",
                "message": r.message,
            })
            if r.success:
                logger.info(" ✓ 依赖 %s → %s", downstream_node_id,
deliverable_id)
            else:

```



```

        logger.error(" X 依赖 %s → %s: %s", downstream_node_id,
deliverable_id, r.message)
    except Exception as e:
        results.append({
            "node_id": downstream_node_id,
            "success": False,
            "phase": "add_dependency",
            "message": str(e),
        })
        logger.error(" X 依赖异常: %s", e)
    return results

```

文件编号: 103 文件路径: emily-core\emily_core\services\node_batch_update.py

```

from __future__ import annotations
import asyncio
import logging
from typing import Any
logger = logging.getLogger("emily.node_batch_update")
async def _resolve_deliverable_id(node_id: str, deliverable_name: str) -> str |
None:
    from ..repositories.node_repo import NodeDeliverableRepo
    delivs = await asyncio.to_thread(NodeDeliverableRepo.find_by_node, node_id)
    for d in delivs:
        if d.deliverable_name == deliverable_name:
            return d.deliverable_id
    for d in delivs:
        if deliverable_name in d.deliverable_name or d.deliverable_name in
deliverable_name:
            return d.deliverable_id
    return None
async def _resolve_node_id_by_name(project_id: str, node_name: str) -> str | None:
    from ..repositories.node_repo import ProjectNodeRepo
    nodes = await asyncio.to_thread(ProjectNodeRepo.find_by_project, project_id)
    for n in nodes:
        if n.node_name == node_name:
            return n.node_id
    for n in nodes:
        if node_name in n.node_name or n.node_name in node_name:
            return n.node_id
    return None
async def batch_update_nodes(
    updates: list[dict],
    *,
    operator_id: str = "",
    dry_run: bool = False,
) -> list[dict]:
    from .node_service import NodeService
    from .node_commands import UpdateNodeCommand
    from ..repositories.permission_repo import PermissionRepository
    svc = NodeService(user_repo=PermissionRepository())
    results: list[dict] = []

```

```
logger.info("批量更新 %d 个节点", len(updates))
for u in updates:
    node_id = u.get("node_id", "")
    if not node_id:
        results.append({
            "node_id": "",
            "success": False,
            "phase": "update_node",
            "message": "缺少 node_id",
        })
        continue
    if dry_run:
        results.append({
            "node_id": node_id,
            "success": True,
            "phase": "update_node",
            "message": f"[DRY-RUN] 将更新节点 {node_id}: {list(k for k in u if k != 'node_id'))}",
            "dry_run": True,
        })
        continue
    cmd = UpdateNodeCommand(
        node_id=node_id,
        operator_id=operator_id,
        node_name=u.get("node_name"),
        deadline=u.get("deadline"),
        owner_dept_id=u.get("owner_dept_id"),
        related_company_id=u.get("related_company_id"),
        remark=u.get("remark"),
        stage_id=u.get("stage_id"),
        sort_order=u.get("sort_order"),
        land_parcel_id=u.get("land_parcel_id"),
        startup_doc_id=u.get("startup_doc_id"),
    )
    try:
        r = await svc.update_node(cmd)
        results.append({
            "node_id": node_id,
            "success": r.success,
            "phase": "update_node",
            "status": r.status,
            "progress": r.progress,
            "message": r.message,
        })
    except Exception as e:
        results.append({
            "node_id": node_id,
            "success": False,
            "phase": "update_node",
            "message": str(e),
        })
    return results

async def batch_activate_nodes(
    node_ids: list[str],
```

```

*,
approver_id: str = "",
approved: bool = True,
remark: str = "",
dry_run: bool = False,
) -> list[dict]:
    from .node_service import NodeService
    from .node_commands import ActivateNodeCommand
    from ..repositories.permission_repo import PermissionRepository
    svc = NodeService(user_repo=PermissionRepository())
    results: list[dict] = []
    action = "激活" if approved else "拒绝"
    logger.info("批量%s %d 个节点", action, len(node_ids))
    for node_id in node_ids:
        if dry_run:
            results.append({
                "node_id": node_id,
                "success": True,
                "phase": "activate_node",
                "message": f"[DRY-RUN] 将{action}节点 {node_id}",
                "dry_run": True,
            })
            continue
        cmd = ActivateNodeCommand(
            node_id=node_id,
            approver_id=approver_id,
            approved=approved,
            remark=remark,
        )
        try:
            r = await svc.activate_node(cmd)
            results.append({
                "node_id": node_id,
                "success": r.success,
                "phase": "activate_node",
                "status": r.status,
                "message": r.message,
            })
        except Exception as e:
            results.append({
                "node_id": node_id,
                "success": False,
                "phase": "activate_node",
                "message": str(e),
            })
    return results

async def batch_discard_nodes(
    node_ids: list[str],
    *,
    operator_id: str = "",
    dry_run: bool = False,
) -> list[dict]:
    from .node_service import NodeService
    from .node_commands import DiscardNodeCommand

```

```

from ..repositories.permission_repo import PermissionRepository
svc = NodeService(user_repo=PermissionRepository())
results: list[dict] = []
logger.info("批量废弃 %d 个节点", len(node_ids))
for node_id in node_ids:
    if dry_run:
        results.append({
            "node_id": node_id,
            "success": True,
            "phase": "discard_node",
            "message": f"[DRY-RUN] 将废弃节点 {node_id}",
            "dry_run": True,
        })
        continue
    cmd = DiscardNodeCommand(
        node_id=node_id,
        operator_id=operator_id,
    )
    try:
        r = await svc.discard_node(cmd)
        results.append({
            "node_id": node_id,
            "success": r.success,
            "phase": "discard_node",
            "message": r.message,
        })
    except Exception as e:
        results.append({
            "node_id": node_id,
            "success": False,
            "phase": "discard_node",
            "message": str(e),
        })
return results

async def batch_update_progress(
    progress_updates: list[dict],
    *,
    operator_id: str = "",
    dry_run: bool = False,
) -> list[dict]:
    from .node_service import NodeService
    from .node_commands import UpdateDeliverableProgressCommand
    from ..repositories.permission_repo import PermissionRepository
    svc = NodeService(user_repo=PermissionRepository())
    results: list[dict] = []
    logger.info("批量更新 %d 个成果进度", len(progress_updates))
    for p in progress_updates:
        deliverable_id = p.get("deliverable_id", "")
        if not deliverable_id:
            deliverable_name = p.get("deliverable_name", "")
            node_id = p.get("node_id", "")
            if not deliverable_name or not node_id:
                results.append({
                    "node_id": node_id,

```

```

        "success": False,
        "phase": "update_progress",
        "message": "缺少 deliverable_id 或 (deliverable_name +
node_id)",
    })
    continue
    deliverable_id = await _resolve_deliverable_id(node_id,
deliverable_name)
    if not deliverable_id:
        results.append({
            "node_id": node_id,
            "success": False,
            "phase": "update_progress",
            "message": f"成果「{deliverable_name}」在节点 {node_id} 中未找
到",
        })
        continue
    current_amount = float(p.get("current_amount", 0))
    if dry_run:
        results.append({
            "node_id": p.get("node_id", ""),
            "success": True,
            "phase": "update_progress",
            "message": f"[DRY-RUN] 将更新成果 {deliverable_id} 进度为
{current_amount}",
            "dry_run": True,
        })
        continue
    cmd = UpdateDeliverableProgressCommand(
        deliverable_id=deliverable_id,
        current_amount=current_amount,
        file_id=p.get("file_id", ""),
        operator_id=operator_id,
    )
    try:
        r = await svc.update_deliverable_progress(cmd)
        results.append({
            "node_id": r.node_id,
            "success": r.success,
            "phase": "update_progress",
            "status": r.status,
            "progress": r.progress,
            "message": r.message,
            "affected_ancestors": r.affected_downstream,
        })
    except Exception as e:
        results.append({
            "node_id": p.get("node_id", ""),
            "success": False,
            "phase": "update_progress",
            "message": str(e),
        })
    return results
_VALID_ACTIONS = frozenset({

```

```

        "add_shared_file",
        "remove_shared_file",
        "set_startup_doc",
        "clear_startup_doc",
        "link_deliverable_file",
        "unlink_deliverable_file",
    })
    _FILE_REQUIRED_ACTIONS = frozenset({
        "add_shared_file",
        "remove_shared_file",
        "set_startup_doc",
        "link_deliverable_file",
    })
    _DELIVERABLE_ACTIONS = frozenset({
        "link_deliverable_file",
        "unlink_deliverable_file",
    })
    async def batch_link_files(
        links: list[dict],
        *,
        operator_id: str = "",
        dry_run: bool = False,
    ) -> list[dict]:
        from ..repositories.file_repo import FileRepository
        from ..repositories.node_repo import (
            NodeAccessibleFileRepo,
            NodeDeliverableRepo,
            ProjectNodeRepo,
        )
        results: list[dict] = []
        logger.info("批量文件关联 %d 条", len(links))
        for lk in links:
            node_id = lk.get("node_id", "")
            action = lk.get("action", "")
            if action not in _VALID_ACTIONS:
                results.append({
                    "node_id": node_id,
                    "success": False,
                    "phase": action or "unknown",
                    "message": f"未知 action 「{action}」，合法值: {'',
''.join(sorted(_VALID_ACTIONS))}",
                })
                continue
            if not node_id:
                results.append({
                    "node_id": "",
                    "success": False,
                    "phase": action,
                    "message": "缺少 node_id",
                })
                continue
            file_id = ""
            if action in _FILE_REQUIRED_ACTIONS:
                file_id = lk.get("file_id", "")

```

```

        file_no = lk.get("file_no", "")
        if not file_id and not file_no:
            results.append({
                "node_id": node_id,
                "success": False,
                "phase": action,
                "message": f"action [{action}] 需要 file_no 或 file_id",
            })
            continue
        if file_no and not file_id:
            file_record = await
asyncio.to_thread(FileRepository.get_by_file_no, file_no)
            if file_record is None:
                results.append({
                    "node_id": node_id,
                    "success": False,
                    "phase": action,
                    "message": f"文件编号 [{file_no}] 不存在",
                })
                continue
            file_id = file_record.id
        deliverable_id = ""
        if action in _DELIVERABLE_ACTIONS:
            deliverable_id = lk.get("deliverable_id", "")
            if not deliverable_id:
                deliverable_name = lk.get("deliverable_name", "")
                if deliverable_name:
                    deliverable_id = await _resolve_deliverable_id(node_id,
deliverable_name)
                if not deliverable_id:
                    results.append({
                        "node_id": node_id,
                        "success": False,
                        "phase": action,
                        "message": "缺少 deliverable_id 或 deliverable_name（在节点
中未找到对应成果）",
                    })
                    continue
            if dry_run:
                results.append({
                    "node_id": node_id,
                    "success": True,
                    "phase": action,
                    "message": f"[DRY-RUN] 将执行 {action} (node={node_id}, file=
{file_id or '-'}, deliverable={deliverable_id or '-'})",
                    "dry_run": True,
                })
                continue
            try:
                if action == "add_shared_file":
                    exists = await asyncio.to_thread(NodeAccessibleFileRepo.exists,
node_id, file_id)
                    if exists:
                        results.append({

```

```
        "node_id": node_id,
        "success": True,
        "phase": action,
        "message": f"共享文件已存在, 跳过 (file_id={file_id})",
        "skipped": True,
    })
else:
    await asyncio.to_thread(
        NodeAccessibleFileRepo.create,
        node_id=node_id,
        file_id=file_id,
        added_by=operator_id,
    )
    results.append({
        "node_id": node_id,
        "success": True,
        "phase": action,
        "message": f"已增加共享文件 (file_id={file_id})",
    })
elif action == "remove_shared_file":
    exists = await asyncio.to_thread(NodeAccessibleFileRepo.exists,
node_id, file_id)
    if not exists:
        results.append({
            "node_id": node_id,
            "success": True,
            "phase": action,
            "message": f"共享文件不存在, 跳过 (file_id={file_id})",
            "skipped": True,
        })
    else:
        await asyncio.to_thread(NodeAccessibleFileRepo.remove,
node_id, file_id)
        results.append({
            "node_id": node_id,
            "success": True,
            "phase": action,
            "message": f"已移除共享文件 (file_id={file_id})",
        })
elif action == "set_startup_doc":
    node = await asyncio.to_thread(ProjectNodeRepo.get_by_node_id,
node_id)
    if node is None:
        results.append({
            "node_id": node_id,
            "success": False,
            "phase": action,
            "message": f"节点「{node_id}」不存在",
        })
    else:
        await asyncio.to_thread(
            ProjectNodeRepo.update_fields, node_id,
startup_doc_id=file_id,
        )
```



```
        results.append({
            "node_id": node_id,
            "success": True,
            "phase": action,
            "message": f"已设置条件文件 (startup_doc_id={file_id})",
        })
    elif action == "clear_startup_doc":
node_id)
        node = await asyncio.to_thread(ProjectNodeRepo.get_by_node_id,
node_id)
        if node is None:
            results.append({
                "node_id": node_id,
                "success": False,
                "phase": action,
                "message": f"节点「{node_id}」不存在",
            })
        else:
            await asyncio.to_thread(
                ProjectNodeRepo.update_fields, node_id, startup_doc_id="",
            )
            results.append({
                "node_id": node_id,
                "success": True,
                "phase": action,
                "message": "已清除条件文件",
            })
    elif action == "link_deliverable_file":
        deliv = await asyncio.to_thread(
            NodeDeliverableRepo.get_by_deliverable_id, deliverable_id,
        )
        if deliv is None:
            results.append({
                "node_id": node_id,
                "success": False,
                "phase": action,
                "message": f"成果「{deliverable_id}」不存在",
            })
        else:
            await asyncio.to_thread(
                NodeDeliverableRepo.update_file_id, deliverable_id,
file_id,
            )
            results.append({
                "node_id": node_id,
                "success": True,
                "phase": action,
                "message": f"已关联成果文件 (deliverable={deliverable_id},
file_id={file_id})",
            })
    elif action == "unlink_deliverable_file":
        deliv = await asyncio.to_thread(
            NodeDeliverableRepo.get_by_deliverable_id, deliverable_id,
        )
        if deliv is None:
```

```

        results.append({
            "node_id": node_id,
            "success": False,
            "phase": action,
            "message": f"成果「{deliverable_id}」不存在",
        })
    else:
        await asyncio.to_thread(
            NodeDeliverableRepo.update_file_id, deliverable_id, "",
        )
        results.append({
            "node_id": node_id,
            "success": True,
            "phase": action,
            "message": f"已取消成果文件关联 (deliverable={
{deliverable_id})",
        })
    except Exception as e:
        logger.exception("文件关联操作失败: %s node=%s", action, node_id)
        results.append({
            "node_id": node_id,
            "success": False,
            "phase": action,
            "message": str(e),
        })
    return results

```

文件编号: 104 文件路径: emily-core\emily_core\services\node_commands.py

```

from dataclasses import dataclass, field
@dataclass
class CreateNodeCommand:
    project_id: str
    node_id: str
    node_name: str
    owner_dept_id: str = "项目总"
    related_company_id: str = "建设单位"
    deadline: str = ""
    creator_id: str = ""
    parent_node_id: str = ""
    stage_id: int = 0
    child_weight: float = 1.0
    remark: str = ""
    land_parcel_id: str = ""
    startup_doc_id: str = ""
    sort_order: int = 0
    responsible_user_id: str = ""
    node_type: str = "WORK_PACKAGE"
@dataclass
class UpdateNodeCommand:
    node_id: str
    operator_id: str = ""

```

```
node_name: str | None = None
deadline: str | None = None
owner_dept_id: str | None = None
related_company_id: str | None = None
remark: str | None = None
stage_id: int | None = None
sort_order: int | None = None
land_parcel_id: str | None = None
startup_doc_id: str | None = None

@dataclass
class ActivateNodeCommand:
    node_id: str
    approver_id: str = ""
    approved: bool = True
    remark: str = ""

@dataclass
class DiscardNodeCommand:
    node_id: str
    operator_id: str = ""

@dataclass
class CreateDeliverableCommand:
    node_id: str
    deliverable_name: str
    target_amount: float
    unit: str
    is_required: bool = True
    operator_id: str = ""

@dataclass
class UpdateDeliverableProgressCommand:
    deliverable_id: str
    current_amount: float
    file_id: str = ""
    operator_id: str = ""

@dataclass
class AddDependencyCommand:
    node_id: str
    depends_on_deliverable_id: str
    weight: float = 1.0
    dependency_type: str = "DELIVERABLE"
    operator_id: str = ""

@dataclass
class RemoveDependencyCommand:
    dependency_id: str
    operator_id: str = ""

@dataclass
class MountChildCommand:
    parent_node_id: str
    child_node_id: str
    child_weight: float = 1.0
    operator_id: str = ""

@dataclass
class UnmountChildCommand:
    parent_node_id: str
    child_node_id: str
```

```

    operator_id: str = ""
@dataclass
class NodeOperationResult:
    success: bool = True
    node_id: str = ""
    status: str = ""
    progress: float = 0.0
    message: str = ""
    error_code: str = ""
    affected_downstream: list[str] = field(default_factory=list)
@dataclass
class CycleCheckResult:
    has_cycle: bool = False
    cycle_path: list[str] = field(default_factory=list)
    message: str = ""
@dataclass
class StateTransitionResult:
    node_id: str = ""
    old_status: str = ""
    new_status: str = ""
    old_progress: float = 0.0
    new_progress: float = 0.0
    should_transition: bool = False
    reason: str = ""
    affected_ancestors: list[str] = field(default_factory=list)
@dataclass
class AssignNodeCommand:
    node_id: str
    responsible_user_id: str
    operator_id: str = ""
@dataclass
class SubmitNodeDeliverableCommand:
    deliverable_id: str
    content: str = ""
    file_url: str = ""
    file_name: str = ""
    attachment_file_id: str = ""
    submitted_by: str = ""
    is_acceptance_check: bool = False
@dataclass
class ConfirmNodeDeliverableCommand:
    deliverable_id: str
    confirmed_by: str = ""
@dataclass
class ReturnNodeDeliverableCommand:
    deliverable_id: str
    returned_by: str = ""
    reason: str = ""
@dataclass
class ResubmitNodeDeliverableCommand:
    deliverable_id: str
    content: str = ""
    file_url: str = ""
    file_name: str = ""

```

```

        attachment_file_id: str = ""
        submitted_by: str = ""
    @dataclass
    class CreateTaskNodeCommand:
        project_id: str
        node_name: str
        responsible_user_id: str = ""
        deadline: str = ""
        parent_node_id: str = ""
        owner_dept_id: str = "项目总"
        description: str = ""
        creator_id: str = ""
    @dataclass
    class MyTasksQuery:
        user_id: str
        project_id: str = ""
        submission_status: str = ""
        page: int = 1
        page_size: int = 20

```

文件编号：105 文件路径：emily-core\emily_core\services\node_service.py

```

from __future__ import annotations
import asyncio
import json
import logging
from datetime import datetime, timezone, timedelta
from typing import TYPE_CHECKING
from .node_commands import (
    CreateNodeCommand,
    UpdateNodeCommand,
    DiscardNodeCommand,
    ActivateNodeCommand,
    CreateDeliverableCommand,
    UpdateDeliverableProgressCommand,
    AddDependencyCommand,
    RemoveDependencyCommand,
    MountChildCommand,
    UnmountChildCommand,
    AssignNodeCommand,
    SubmitNodeDeliverableCommand,
    ConfirmNodeDeliverableCommand,
    ReturnNodeDeliverableCommand,
    ResubmitNodeDeliverableCommand,
    NodeOperationResult,
    CycleCheckResult,
    StateTransitionResult,
)
from .node_state_machine import (
    NOT_ACTIVATED,
    CONDITIONS_NOT_MET,
    IN_PROGRESS,

```

```

    COMPLETED,
    NodeSnapshot,
    DependencySnapshot,
    DeliverableSnapshot,
    ChildSnapshot,
    calc_dependency_satisfaction,
    calc_deliverable_completion,
    determine_node_status,
    calc_parent_progress,
    detect_cycle,
    check_parent_child_cycle,
)
from ..repositories.node_repo import (
    ProjectNodeRepo,
    NodeDependencyRepo,
    NodeDeliverableRepo,
    NodeEventRepo,
    _parse_decimal,
    _to_decimal_str,
)
from ..infrastructure.database.models import _new_id
if TYPE_CHECKING:
    from ..infrastructure.database.models import (
        ProjectNode,
        NodeDependency,
        NodeDeliverable,
    )
logger = logging.getLogger("emily.node_service")
BEIJING_TZ = timezone(timedelta(hours=8))
MAX_CHILDREN_PER_PARENT = 100
MAX_ANCESTOR_RECALC_DEPTH = 3
class NodeService:
    def __init__(
        self,
        node_repo: ProjectNodeRepo | None = None,
        dependency_repo: NodeDependencyRepo | None = None,
        deliverable_repo: NodeDeliverableRepo | None = None,
        event_repo: NodeEventRepo | None = None,
        user_repo=None,
        outbound_bus=None,
    ):
        self._node_repo = node_repo or ProjectNodeRepo()
        self._dep_repo = dependency_repo or NodeDependencyRepo()
        self._deliv_repo = deliverable_repo or NodeDeliverableRepo()
        self._event_repo = event_repo or NodeEventRepo()
        self._user_repo = user_repo
        self._outbound_bus = outbound_bus
    @staticmethod
    def _now_iso() -> str:
        return datetime.now(timezone.utc).isoformat()
    def _record_event(self, node_id: str, event_type: str,
        old_value: str = "", new_value: str = "",
        operator_id: str = "", remark: str = "") -> None:
        try:

```

```

        self._event_repo.create(
            event_id=_new_id("EVT"),
            node_id=node_id,
            event_type=event_type,
            old_value=old_value,
            new_value=new_value,
            operator_id=operator_id,
            remark=remark,
        )
    except Exception:
        logger.exception("Failed to record event for node %s", node_id)
    async def create_node(self, cmd: CreateNodeCommand) -> NodeOperationResult:
        responsible_user_id = getattr(cmd, 'responsible_user_id', '') or
cmd.creator_id
        if self._user_repo:
            user = await asyncio.to_thread(self._user_repo.get_user,
responsible_user_id)
            if user is None:
                return NodeOperationResult(
                    success=False, node_id=cmd.node_id,
                    message=f"责任人 {responsible_user_id} 不存在于用户表中",
                    error_code="40002",
                )
            if cmd.parent_node_id:
                parent = await asyncio.to_thread(self._node_repo.get_by_node_id,
cmd.parent_node_id)
                if parent and getattr(parent, 'node_type', '') == "TASK":
                    return NodeOperationResult(
                        success=False, node_id=cmd.node_id,
                        message="TASK 类型节点不允许挂载子节点",
                        error_code="40003",
                    )
            is_admin = False
            if cmd.creator_id and self._user_repo:
                user = await asyncio.to_thread(self._user_repo.get_user,
cmd.creator_id)
                if user:
                    is_admin = getattr(user, "level", 0) >= 5
                    if not is_admin:
                        company_type = await asyncio.to_thread(
                            self._get_creator_company_type, cmd.creator_id,
                        )
                        if company_type and company_type != "建设单位":
                            return NodeOperationResult(
                                success=False, node_id=cmd.node_id,
                                message="仅建设单位人员可创建全景节点",
                                error_code="40301",
                            )
            child_weight_str = _to_decimal_str(cmd.child_weight, precision=4)
            initial_status = CONDITIONS_NOT_MET if is_admin else NOT_ACTIVATED
            node = await asyncio.to_thread(
                self._node_repo.create,
                project_id=cmd.project_id,
                node_id=cmd.node_id,

```

```

        node_name=cmd.node_name,
        owner_dept_id=cmd.owner_dept_id,
        related_company_id=cmd.related_company_id,
        deadline=cmd.deadline,
        creator_id=cmd.creator_id,
        parent_node_id=cmd.parent_node_id,
        stage_id=cmd.stage_id,
        child_weight=child_weight_str,
        remark=cmd.remark,
        land_parcel_id=cmd.land_parcel_id,
        startup_doc_id=cmd.startup_doc_id,
        sort_order=cmd.sort_order,
        status=initial_status,
        responsible_user_id=responsible_user_id,
        node_type=getattr(cmd, 'node_type', 'WORK_PACKAGE'),
    )
    now_iso = self._now_iso()
    if is_admin:
        await asyncio.to_thread(
            self._node_repo.update_fields,
            cmd.node_id,
            approver_id=cmd.creator_id,
            approved_at=now_iso,
        )
        self._record_event(
            node_id=cmd.node_id,
            event_type="node_created_auto_activated",
            new_value=json.dumps({
                "node_name": cmd.node_name,
                "project_id": cmd.project_id,
                "owner_dept_id": cmd.owner_dept_id,
                "creator_id": cmd.creator_id,
                "auto_activated": True,
            }),
            operator_id=cmd.creator_id,
            remark=f"管理员创建节点，自动激活（跳过审批）",
        )
    if self._outbound_bus:
        try:
            self._outbound_bus.publish("node_auto_activated", {
                "node_id": cmd.node_id,
                "node_name": cmd.node_name,
                "project_id": cmd.project_id,
                "owner_dept_id": cmd.owner_dept_id,
                "creator_id": cmd.creator_id,
            })
        except Exception:
            logger.exception("Failed to publish node_auto_activated
event")
    logger.info("Node created (auto-activated by admin): %s (project=%s,
dept=%s)",
                cmd.node_id, cmd.project_id, cmd.owner_dept_id)
    return NodeOperationResult(
        success=True,

```



```

        node_id=cmd.node_id,
        status=CONDITIONS_NOT_MET,
        progress=_parse_decimal(node.progress),
        message=f"节点「{cmd.node_name}」已创建并激活",
    )
else:
    self._record_event(
        node_id=cmd.node_id,
        event_type="node_created_pending_approval",
        new_value=json.dumps({
            "node_name": cmd.node_name,
            "project_id": cmd.project_id,
            "owner_dept_id": cmd.owner_dept_id,
            "creator_id": cmd.creator_id,
        }),
        operator_id=cmd.creator_id,
        remark=f"节点创建, 待「{cmd.owner_dept_id}」部门负责人审批",
    )
    if self._outbound_bus:
        try:
            self._outbound_bus.publish("node_pending_approval", {
                "node_id": cmd.node_id,
                "node_name": cmd.node_name,
                "project_id": cmd.project_id,
                "owner_dept_id": cmd.owner_dept_id,
                "creator_id": cmd.creator_id,
            })
        except Exception:
            logger.exception("Failed to publish node_pending_approval
event")
    logger.info("Node created (pending approval): %s (project=%s,
dept=%s)",
                cmd.node_id, cmd.project_id, cmd.owner_dept_id)
    return NodeOperationResult(
        success=True,
        node_id=cmd.node_id,
        status=node.status,
        progress=_parse_decimal(node.progress),
        message=f"节点「{cmd.node_name}」已创建, 待「{cmd.owner_dept_id}」部
门负责人审批后启用",
    )

async def update_node(self, cmd: UpdateNodeCommand) -> NodeOperationResult:
    updates = {}
    if cmd.node_name is not None:
        updates["node_name"] = cmd.node_name
    if cmd.deadline is not None:
        updates["deadline"] = cmd.deadline
    if cmd.owner_dept_id is not None:
        updates["owner_dept_id"] = cmd.owner_dept_id
    if cmd.related_company_id is not None:
        updates["related_company_id"] = cmd.related_company_id
    if cmd.remark is not None:
        updates["remark"] = cmd.remark
    if cmd.stage_id is not None:

```

```

        updates["stage_id"] = cmd.stage_id
    if cmd.sort_order is not None:
        updates["sort_order"] = cmd.sort_order
    if cmd.land_parcel_id is not None:
        updates["land_parcel_id"] = cmd.land_parcel_id
    if cmd.startup_doc_id is not None:
        updates["startup_doc_id"] = cmd.startup_doc_id
    if not updates:
        return NodeOperationResult(success=False, node_id=cmd.node_id,
message="无更新字段")
    old_node = await asyncio.to_thread(self._node_repo.get_by_node_id,
cmd.node_id)
    if old_node is None:
        return NodeOperationResult(success=False, node_id=cmd.node_id,
message="节点不存在")
    node = await asyncio.to_thread(self._node_repo.update_fields, cmd.node_id,
**updates)
    if node is None:
        return NodeOperationResult(success=False, node_id=cmd.node_id,
message="更新失败")
    self._record_event(
        node_id=cmd.node_id,
        event_type="node_updated",
        old_value=json.dumps({"node_name": old_node.node_name, "deadline":
old_node.deadline}),
        new_value=json.dumps(updates),
        operator_id=cmd.operator_id,
        remark="节点字段更新",
    )
    return NodeOperationResult(
        success=True,
        node_id=cmd.node_id,
        status=node.status,
        progress=_parse_decimal(node.progress),
        message="节点更新成功",
    )

    async def discard_node(self, cmd: DiscardNodeCommand) -> NodeOperationResult:
        node = await asyncio.to_thread(self._node_repo.get_by_node_id,
cmd.node_id)
        if node is None:
            return NodeOperationResult(success=False, node_id=cmd.node_id,
message="节点不存在")
        children = await asyncio.to_thread(self._node_repo.find_by_parent,
cmd.node_id)
        for child in children:
            if child.status == COMPLETED:
                return NodeOperationResult(
                    success=False, node_id=cmd.node_id,
                    message=f"子节点「{child.node_id}」已完成, 不可废弃父节点",
                )
        await asyncio.to_thread(self._node_repo.discard, cmd.node_id)
        self._record_event(
            node_id=cmd.node_id,
            event_type="node_discarded",

```

```

        old_value=json.dumps({"status": node.status}),
        operator_id=cmd.operator_id,
        remark="节点废弃",
    )
    return NodeOperationResult(success=True, node_id=cmd.node_id, message="节点已废弃")
    async def activate_node(self, cmd: ActivateNodeCommand) ->
NodeOperationResult:
    node = await asyncio.to_thread(self._node_repo.get_by_node_id,
cmd.node_id)
    if node is None:
        return NodeOperationResult(success=False, node_id=cmd.node_id,
message="节点不存在")
    if node.status != NOT_ACTIVATED:
        return NodeOperationResult(
            success=False, node_id=cmd.node_id,
            message=f"节点状态为「{node.status}」，非待审批状态，无法审批",
        )
    if cmd.approver_id and self._user_repo:
        is_authorized = await asyncio.to_thread(
            self._check_approver_permission,
            cmd.approver_id, node.owner_dept_id,
        )
        if not is_authorized:
            return NodeOperationResult(
                success=False, node_id=cmd.node_id,
                message=f"仅「{node.owner_dept_id}」部门负责人或管理员（L5+）可审
批此节点",
                error_code="40302",
            )
    now_iso = self._now_iso()
    if cmd.approved:
        await asyncio.to_thread(self._node_repo.update_status, cmd.node_id,
CONDITIONS_NOT_MET)
        await asyncio.to_thread(
            self._node_repo.update_fields,
            cmd.node_id,
            approver_id=cmd.approver_id,
            approved_at=now_iso,
        )
        self._record_event(
            node_id=cmd.node_id,
            event_type="node_activated",
            old_value=json.dumps({"status": NOT_ACTIVATED}),
            new_value=json.dumps({"status": CONDITIONS_NOT_MET, "approver_id":
cmd.approver_id}),
            operator_id=cmd.approver_id,
            remark=f"审批通过: {cmd.remark}" if cmd.remark else "审批通过",
        )
        logger.info("Node activated: %s by %s", cmd.node_id, cmd.approver_id)
    return NodeOperationResult(
        success=True,
        node_id=cmd.node_id,
        status=CONDITIONS_NOT_MET,

```

```

        progress=_parse_decimal(node.progress),
        message=f"节点「{node.node_name}」审批通过，已启用",
    )
else:
    await asyncio.to_thread(self._node_repo.discard, cmd.node_id)
    self._record_event(
        node_id=cmd.node_id,
        event_type="node_activation_rejected",
        old_value=json.dumps({"status": NOT_ACTIVATED}),
        new_value=json.dumps({"is_discarded": True, "approver_id":
cmd.approver_id}),
        operator_id=cmd.approver_id,
        remark=f"审批拒绝: {cmd.remark}" if cmd.remark else "审批拒绝",
    )
    logger.info("Node activation rejected: %s by %s", cmd.node_id,
cmd.approver_id)
    return NodeOperationResult(
        success=True,
        node_id=cmd.node_id,
        status="DISCARDED",
        progress=0.0,
        message=f"节点「{node.node_name}」审批拒绝，已废弃",
    )

def _get_creator_company_type(self, creator_id: str) -> str:
    if self._user_repo is None:
        return ""
    user = self._user_repo.get_user(creator_id)
    if user is None:
        return ""
    company = self._user_repo.get_company(user.company) if user.company else
None
    return company.type if company else ""

def _check_approver_permission(self, approver_id: str, owner_dept_id: str) ->
bool:
    if self._user_repo is None:
        return True
    user = self._user_repo.get_user(approver_id)
    if user is None:
        return False
    if user.level >= 5:
        return True
    if user.department and owner_dept_id:
        import json as _json
        try:
            depts = _json.loads(user.department) if
isinstance(user.department, str) else user.department
            if isinstance(depts, list) and owner_dept_id in depts:
                return True
        except (_json.JSONDecodeError, TypeError):
            pass
        if user.department == owner_dept_id:
            return True
    return False

def _check_submission_permission(self, submitter_id: str, node) -> bool:

```

```

        if not submitter_id:
            return True
        if self._user_repo is None:
            return True
        resp_id = getattr(node, 'responsible_user_id', '')
        if resp_id and submitter_id == resp_id:
            return True
        user = self._user_repo.get_user(submitter_id)
        if user is None:
            return False
        owner_dept = getattr(node, 'owner_dept_id', '')
        if user.department and owner_dept:
            if user.department == owner_dept:
                return True
            import json as _json
            try:
                depts = _json.loads(user.department) if
isinstance(user.department, str) else user.department
                if isinstance(depts, list) and owner_dept in depts:
                    return True
            except (_json.JSONDecodeError, TypeError):
                pass
            return False
        async def create_deliverable(self, cmd: CreateDeliverableCommand) ->
NodeOperationResult:
            node = await asyncio.to_thread(self._node_repo.get_by_node_id,
cmd.node_id)
            if node is None:
                return NodeOperationResult(success=False, node_id=cmd.node_id,
message="节点不存在")
            seq = await asyncio.to_thread(self._deliv_repo.get_next_seq, cmd.node_id)
            deliverable_id = self._deliv_repo.generate_deliverable_id(cmd.node_id,
seq)
            await asyncio.to_thread(
                self._deliv_repo.create,
                deliverable_id=deliverable_id,
                node_id=cmd.node_id,
                deliverable_name=cmd.deliverable_name,
                target_amount=_to_decimal_str(cmd.target_amount, precision=2),
                unit=cmd.unit,
                is_required=cmd.is_required,
            )
            self._record_event(
                node_id=cmd.node_id,
                event_type="deliverable_updated",
                new_value=json.dumps({"deliverable_id": deliverable_id, "name":
cmd.deliverable_name}),
                operator_id=cmd.operator_id,
                remark=f"新增成果: {cmd.deliverable_name}",
            )
            await self._recalc_node_status(cmd.node_id)
            return NodeOperationResult(
                success=True,
                node_id=cmd.node_id,

```

```

        message=f"成果「{cmd.deliverable_name}」创建成功",
    )
    async def update_deliverable_progress(self, cmd:
UpdateDeliverableProgressCommand) -> NodeOperationResult:
        deliv = await asyncio.to_thread(self._deliv_repo.get_by_deliverable_id,
cmd.deliverable_id)
        if deliv is None:
            return NodeOperationResult(
                success=False, node_id="",
                message=f"成果 {cmd.deliverable_id} 不存在",
            )
        old_amount = deliv.current_amount
        amount_str = _to_decimal_str(cmd.current_amount, precision=2)
        await asyncio.to_thread(
            self._deliv_repo.update_progress,
            cmd.deliverable_id,
            amount_str,
            cmd.file_id,
        )
        self._record_event(
            node_id=deliv.node_id,
            event_type="deliverable_updated",
            old_value=json.dumps({"current_amount": old_amount}),
            new_value=json.dumps({"current_amount": amount_str, "file_id":
cmd.file_id}),
            operator_id=cmd.operator_id,
            remark=f"成果进度更新: {old_amount} → {amount_str}",
        )
        result = await self._recalc_node_status(deliv.node_id)
        return result
    async def add_dependency(self, cmd: AddDependencyCommand) ->
NodeOperationResult:
        dep_deliv = await asyncio.to_thread(
            self._deliv_repo.get_by_deliverable_id, cmd.depends_on_deliverable_id,
        )
        if dep_deliv is None:
            return NodeOperationResult(
                success=False, node_id=cmd.node_id,
                message=f"成果 {cmd.depends_on_deliverable_id} 不存在",
            )
        upstream_node_id = dep_deliv.node_id
        if upstream_node_id == cmd.node_id:
            return NodeOperationResult(
                success=False, node_id=cmd.node_id,
                message="节点不能依赖自己的成果",
                error_code="40001",
            )
        if await self._is_ancestor_dependency(cmd.node_id, upstream_node_id):
            return NodeOperationResult(
                success=False, node_id=cmd.node_id,
                message="子节点不能依赖父节点（任何层级）",
                error_code="40001",
            )
        cycle_result = await self._check_cycle(cmd.node_id,

```

```

cmd.depends_on_deliverable_id)
    if cycle_result.has_cycle:
        return NodeOperationResult(
            success=False, node_id=cmd.node_id,
            message=f"循环依赖: {' → '.join(cycle_result.cycle_path)}",
            error_code="40001",
        )
    if await asyncio.to_thread(
        self._dep_repo.exists, cmd.node_id, cmd.depends_on_deliverable_id,
    ):
        return NodeOperationResult(
            success=False, node_id=cmd.node_id,
            message="该依赖关系已存在",
        )
    weight_str = _to_decimal_str(cmd.weight, precision=4)
    await asyncio.to_thread(
        self._dep_repo.create,
        node_id=cmd.node_id,
        depends_on_deliverable_id=cmd.depends_on_deliverable_id,
        depends_on_node_id=upstream_node_id,
        weight=weight_str,
        dependency_type=cmd.dependency_type,
    )
    self._record_event(
        node_id=cmd.node_id,
        event_type="dependency_added",
        new_value=json.dumps({
            "depends_on_deliverable_id": cmd.depends_on_deliverable_id,
            "depends_on_node_id": upstream_node_id,
            "weight": weight_str,
        })),
        operator_id=cmd.operator_id,
        remark=f"新增依赖: {cmd.depends_on_deliverable_id} (权重{weight_str})",
    )
    if cmd.weight >= 999.0:
        self._record_event(
            node_id=cmd.node_id,
            event_type="BLOCKING_CONDITION_ADDED",
            new_value=json.dumps({"deliverable_id":
cmd.depends_on_deliverable_id}),
            operator_id=cmd.operator_id,
            remark="人工阻塞条件",
        )
    await self._recalc_node_status(cmd.node_id)
    return NodeOperationResult(
        success=True,
        node_id=cmd.node_id,
        message="依赖添加成功",
    )

    async def remove_dependency(self, cmd: RemoveDependencyCommand) ->
NodeOperationResult:
    dep = await asyncio.to_thread(self._dep_repo.get_by_id, cmd.dependency_id)
    if dep is None:
        return NodeOperationResult(success=False, message="依赖不存在")

```

```

node_id = dep.node_id
is_blocking = _parse_decimal(dep.weight) >= 999.0
await asyncio.to_thread(self._dep_repo.delete, cmd.dependency_id)
self._record_event(
    node_id=node_id,
    event_type="dependency_removed",
    old_value=json.dumps({"depends_on_deliverable_id":
dep.depends_on_deliverable_id}),
    operator_id=cmd.operator_id,
    remark="移除依赖",
)
if is_blocking:
    self._record_event(
        node_id=node_id,
        event_type="BLOCKING_CONDITION_REMOVED",
        operator_id=cmd.operator_id,
        remark="解除阻塞条件",
    )
await self._recalc_node_status(node_id)
return NodeOperationResult(success=True, node_id=node_id, message="依赖已
移除")

async def mount_child(self, cmd: MountChildCommand) -> NodeOperationResult:
    count = await asyncio.to_thread(self._node_repo.count_children,
cmd.parent_node_id)
    if count >= MAX_CHILDREN_PER_PARENT:
        return NodeOperationResult(
            success=False, node_id=cmd.parent_node_id,
            message=f"子节点数量已达上限（{MAX_CHILDREN_PER_PARENT}）",
            error_code="40002",
        )
    ancestors = await asyncio.to_thread(
        self._node_repo.get_ancestor_chain, cmd.parent_node_id, max_depth=2,
    )
    if len(ancestors) >= 2:
        return NodeOperationResult(
            success=False, node_id=cmd.parent_node_id,
            message="嵌套深度已达上限（3层），无法继续挂载子节点",
        )
    all_parents = {cmd.parent_node_id}
    for a in ancestors:
        all_parents.add(a.node_id)
    child_ancestors = await asyncio.to_thread(
        self._node_repo.get_ancestor_chain, cmd.child_node_id, max_depth=3,
    )
    for ca in child_ancestors:
        if ca.node_id in all_parents:
            return NodeOperationResult(
                success=False, node_id=cmd.parent_node_id,
                message="不能将祖先节点挂载为子节点",
                error_code="40001",
            )
    weight_str = _to_decimal_str(cmd.child_weight, precision=4)
    await asyncio.to_thread(
        self._node_repo.update_fields,

```



```

        cmd.child_node_id,
        parent_node_id=cmd.parent_node_id,
        child_weight=weight_str,
    )
    self._record_event(
        node_id=cmd.child_node_id,
        event_type="child_node_mounted",
        new_value=json.dumps({"parent_node_id": cmd.parent_node_id}),
        operator_id=cmd.operator_id,
        remark=f"挂载到父节点 {cmd.parent_node_id}",
    )
    await self._recalc_node_status(cmd.parent_node_id)
    return NodeOperationResult(
        success=True,
        node_id=cmd.child_node_id,
        message=f"子节点已挂载到 {cmd.parent_node_id}",
    )

    async def unmount_child(self, cmd: UnmountChildCommand) ->
NodeOperationResult:
    child = await asyncio.to_thread(self._node_repo.get_by_node_id,
cmd.child_node_id)
    if child is None or child.parent_node_id != cmd.parent_node_id:
        return NodeOperationResult(
            success=False, node_id=cmd.child_node_id,
            message="父子关系不匹配",
        )
    await asyncio.to_thread(
        self._node_repo.update_fields,
        cmd.child_node_id,
        parent_node_id="",
        child_weight="1.0000",
    )
    self._record_event(
        node_id=cmd.child_node_id,
        event_type="child_node_unmounted",
        old_value=json.dumps({"parent_node_id": cmd.parent_node_id}),
        operator_id=cmd.operator_id,
        remark=f"从父节点 {cmd.parent_node_id} 移除",
    )
    await self._recalc_node_status(cmd.parent_node_id)
    return NodeOperationResult(
        success=True, node_id=cmd.child_node_id, message="子节点已移除",
    )

    async def assign_node(self, cmd: AssignNodeCommand) -> NodeOperationResult:
        node = await asyncio.to_thread(self._node_repo.get_by_node_id,
cmd.node_id)
        if node is None:
            return NodeOperationResult(success=False, node_id=cmd.node_id,
message="节点不存在")
        if self._user_repo:
            user = await asyncio.to_thread(self._user_repo.get_user,
cmd.responsible_user_id)
            if user is None:
                return NodeOperationResult(

```

```

        success=False, node_id=cmd.node_id,
        message=f"目标责任人 {cmd.responsible_user_id} 不存在于用户表
中",
        error_code="40002",
    )
    if cmd.operator_id and self._user_repo:
        is_authorized = await asyncio.to_thread(
            self._check_approver_permission, cmd.operator_id,
node.owner_dept_id,
        )
        if not is_authorized:
            return NodeOperationResult(
                success=False, node_id=cmd.node_id,
                message=f"仅「{node.owner_dept_id}」部门负责人或管理员（L5+）可变
更责任人",
                error_code="40302",
            )
        await asyncio.to_thread(
            self._node_repo.update_fields, cmd.node_id,
            responsible_user_id=cmd.responsible_user_id,
        )
        self._record_event(
            node_id=cmd.node_id,
            event_type="responsible_user_changed",
            new_value=json.dumps({"responsible_user_id":
cmd.responsible_user_id}),
            operator_id=cmd.operator_id,
            remark=f"责任人变更",
        )
        return NodeOperationResult(success=True, node_id=cmd.node_id, message="责
任人变更成功")

    async def submit_deliverable(self, cmd: SubmitNodeDeliverableCommand) ->
NodeOperationResult:
        deliv = await asyncio.to_thread(self._deliv_repo.get_by_deliverable_id,
cmd.deliverable_id)
        if deliv is None:
            return NodeOperationResult(success=False, node_id="", message=f"成果
{cmd.deliverable_id} 不存在")
        if deliv.submission_status not in ("PENDING", "RETURNED"):
            return NodeOperationResult(
                success=False, node_id=deliv.node_id,
                message=f"成果当前状态为「{deliv.submission_status}」，无法提交",
            )
        node = await asyncio.to_thread(self._node_repo.get_by_node_id,
deliv.node_id)
        if node is None or node.status != "IN_PROGRESS":
            return NodeOperationResult(
                success=False, node_id=deliv.node_id,
                message=f"节点状态为「{getattr(node, 'status', '未知')}」，非
「IN_PROGRESS」",
            )
        if not self._check_submission_permission(cmd.submitted_by, node):
            return NodeOperationResult(
                success=False, node_id=deliv.node_id,

```

```
        message="仅节点责任人或同部门人员可提交成果",
        error_code="40303",
    )
    await asyncio.to_thread(
        self._deliv_repo.update_submission_status,
        cmd.deliverable_id,
        "SUBMITTED",
        submitted_by=cmd.submitted_by,
        attachment_file_id=cmd.attachment_file_id,
    )
    self._record_event(
        node_id=deliv.node_id,
        event_type="deliverable_submitted",
        new_value=json.dumps({"deliverable_id": cmd.deliverable_id}),
        operator_id=cmd.submitted_by,
        remark="成果提交",
    )
    return NodeOperationResult(success=True, node_id=deliv.node_id,
message="成果提交成功")
    async def confirm_deliverable(self, cmd: ConfirmNodeDeliverableCommand) ->
NodeOperationResult:
        deliv = await asyncio.to_thread(self._deliv_repo.get_by_deliverable_id,
cmd.deliverable_id)
        if deliv is None:
            return NodeOperationResult(success=False, node_id="", message=f"成果
{cmd.deliverable_id} 不存在")
        if deliv.submission_status != "SUBMITTED":
            return NodeOperationResult(
                success=False, node_id=deliv.node_id,
                message=f"成果当前状态为「{deliv.submission_status}」, 非
「SUBMITTED」",
            )
        await asyncio.to_thread(
            self._deliv_repo.update_submission_status,
            cmd.deliverable_id,
            "CONFIRMED",
            confirmed_by=cmd.confirmed_by,
        )
        await asyncio.to_thread(
            self._deliv_repo.update_progress,
            cmd.deliverable_id,
            deliv.target_amount,
            file_id=getattr(deliv, 'attachment_file_id', ''),
        )
        self._record_event(
            node_id=deliv.node_id,
            event_type="deliverable_confirmed",
            new_value=json.dumps({"deliverable_id": cmd.deliverable_id}),
            operator_id=cmd.confirmed_by,
            remark="成果确认",
        )
        await self._recalc_node_status(deliv.node_id)
        return NodeOperationResult(success=True, node_id=deliv.node_id,
message="成果确认成功")
```

```

    async def return_deliverable(self, cmd: ReturnNodeDeliverableCommand) ->
NodeOperationResult:
    deliv = await asyncio.to_thread(self._deliv_repo.get_by_deliverable_id,
cmd.deliverable_id)
    if deliv is None:
        return NodeOperationResult(success=False, node_id="", message=f"成果
{cmd.deliverable_id} 不存在")
    if deliv.submission_status != "SUBMITTED":
        return NodeOperationResult(
            success=False, node_id=deliv.node_id,
            message=f"成果当前状态为「{deliv.submission_status}」, 非
「SUBMITTED」",
        )
    if not cmd.reason:
        return NodeOperationResult(
            success=False, node_id=deliv.node_id,
            message="退回必须填写原因",
        )
    await asyncio.to_thread(
        self._deliv_repo.update_submission_status,
        cmd.deliverable_id,
        "RETURNED",
        return_reason=cmd.reason,
    )
    self._record_event(
        node_id=deliv.node_id,
        event_type="deliverable_returned",
        new_value=json.dumps({"deliverable_id": cmd.deliverable_id, "reason":
cmd.reason}),
        operator_id=cmd.returned_by,
        remark=f"成果退回: {cmd.reason}",
    )
    return NodeOperationResult(success=True, node_id=deliv.node_id,
message="成果已退回")
    async def resubmit_deliverable(self, cmd: ResubmitNodeDeliverableCommand) ->
NodeOperationResult:
    deliv = await asyncio.to_thread(self._deliv_repo.get_by_deliverable_id,
cmd.deliverable_id)
    if deliv is None:
        return NodeOperationResult(success=False, node_id="", message=f"成果
{cmd.deliverable_id} 不存在")
    if deliv.submission_status != "RETURNED":
        return NodeOperationResult(
            success=False, node_id=deliv.node_id,
            message=f"成果当前状态为「{deliv.submission_status}」, 非
「RETURNED」",
        )
    await asyncio.to_thread(
        self._deliv_repo.update_submission_status,
        cmd.deliverable_id,
        "SUBMITTED",
        submitted_by=cmd.submitted_by,
        attachment_file_id=cmd.attachment_file_id,
    )

```

```

        self._record_event(
            node_id=deliv.node_id,
            event_type="deliverable_resubmitted",
            new_value=json.dumps({"deliverable_id": cmd.deliverable_id}),
            operator_id=cmd.submitted_by,
            remark="成果重新提交",
        )
        return NodeOperationResult(success=True, node_id=deliv.node_id,
message="成果重新提交成功")
    async def find_near_deadline(self, before_minutes: int = 60, limit: int = 100)
-> list:
        return await asyncio.to_thread(
            self._node_repo.find_near_deadline, before_minutes, limit,
        )
    async def find_overdue(self, limit: int = 100) -> list:
        return await asyncio.to_thread(self._node_repo.find_overdue, limit)
    async def get_node_detail(self, node_id: str) -> dict | None:
        node = await asyncio.to_thread(self._node_repo.get_by_node_id, node_id)
        if node is None:
            return None
        children = await asyncio.to_thread(self._node_repo.find_by_parent,
node_id)
        delivs = await asyncio.to_thread(self._deliv_repo.find_by_node, node_id)
        deps = await asyncio.to_thread(self._dep_repo.find_by_node, node_id)
        return {
            "node_id": node.node_id,
            "node_name": node.node_name,
            "project_id": node.project_id,
            "status": node.status,
            "progress": _parse_decimal(node.progress),
            "deadline": node.deadline,
            "owner_dept_id": node.owner_dept_id,
            "related_company_id": node.related_company_id,
            "parent_node_id": node.parent_node_id,
            "stage_id": node.stage_id,
            "remark": node.remark,
            "is_discarded": node.is_discarded,
            "created_at": node.created_at,
            "children": [
                {
                    "node_id": c.node_id,
                    "node_name": c.node_name,
                    "status": c.status,
                    "progress": _parse_decimal(c.progress),
                }
                for c in children
            ],
            "deliverables": [
                {
                    "deliverable_id": d.deliverable_id,
                    "deliverable_name": d.deliverable_name,
                    "target_amount": _parse_decimal(d.target_amount),
                    "current_amount": _parse_decimal(d.current_amount),
                    "unit": d.unit,
                }
            ]
        }

```

```

        "is_required": d.is_required,
        "file_id": d.file_id,
        "completed_at": d.completed_at,
    }
    for d in delivs
],
"dependencies": [
    {
        "id": d.id,
        "depends_on_deliverable_id": d.depends_on_deliverable_id,
        "depends_on_node_id": d.depends_on_node_id,
        "weight": _parse_decimal(d.weight),
        "dependency_type": d.dependency_type,
    }
    for d in deps
],
}
async def _recalc_node_status(self, node_id: str) -> NodeOperationResult:
    snap = await self._build_snapshot(node_id)
    if snap is None:
        return NodeOperationResult(success=False, node_id=node_id, message="节点不存在")
    if snap.status == NOT_ACTIVATED:
        return NodeOperationResult(
            success=True, node_id=node_id, status=NOT_ACTIVATED, progress=0.0,
            message="节点尚未启用，不进行状态计算",
        )
    file_status = {}
    for dep in snap.dependencies:
        dep_deliv = await asyncio.to_thread(
            self._deliv_repo.get_by_deliverable_id,
            dep.depends_on_deliverable_id,
        )
        if dep_deliv:
            current = _parse_decimal(dep_deliv.current_amount)
            target = max(_parse_decimal(dep_deliv.target_amount), 0.001)
            file_status[dep.depends_on_deliverable_id] = (current >= target)
    old_status = snap.status
    old_progress = snap.progress
    new_status = determine_node_status(
        snap.dependencies, snap.deliverables, file_status, snap.children,
    )
    if snap.children:
        new_progress = calc_parent_progress(snap.children)
    else:
        new_progress = calc_deliverable_completion(snap.deliverables) * 100.0
    status_changed = (new_status != old_status)
    progress_changed = abs(new_progress - old_progress) > 0.001
    if not status_changed and not progress_changed:
        return NodeOperationResult(
            success=True, node_id=node_id, status=old_status,
            progress=old_progress,
            message="状态无变化",
        )

```

```

        await asyncio.to_thread(self._node_repo.update_progress, node_id,
new_progress)
        if status_changed:
            await asyncio.to_thread(self._node_repo.update_status, node_id,
new_status)
            self._record_event(
                node_id=node_id,
                event_type="status_changed",
                old_value=json.dumps({"status": old_status}),
                new_value=json.dumps({"status": new_status, "progress":
new_progress}),
                remark="状态自动流转",
            )
            if new_status == COMPLETED:
                self._record_event(
                    node_id=node_id,
                    event_type="auto_triggered",
                    remark="节点已完成",
                )
            affected = []
            current_id = node_id
            for depth in range(MAX_ANCESTOR_RECALC_DEPTH):
                current = await asyncio.to_thread(self._node_repo.get_by_node_id,
current_id)
                if current is None or not current.parent_node_id:
                    break
                parent_id = current.parent_node_id
                await self._recalc_parent_progress(parent_id)
                affected.append(parent_id)
                current_id = parent_id
            logger.info(
                "Node %s recalc: status %s->%s, progress %.2f->%.2f, ancestors=%s",
                node_id, old_status, new_status, old_progress, new_progress, affected,
            )
            return NodeOperationResult(
                success=True,
                node_id=node_id,
                status=new_status,
                progress=new_progress,
                message=f"状态重算完成: {old_status} → {new_status}",
                affected_downstream=affected,
            )
        async def _recalc_parent_progress(self, parent_node_id: str) -> None:
            children = await asyncio.to_thread(self._node_repo.find_by_parent,
parent_node_id)
            if not children:
                return
            child_snapshots = [
                ChildSnapshot(
                    node_id=c.node_id,
                    status=c.status,
                    progress=_parse_decimal(c.progress),
                    child_weight=_parse_decimal(c.child_weight),
                )

```

```

        for c in children
    ]
    new_progress = calc_parent_progress(child_snapshots)
    new_status = determine_node_status([], [], {}, child_snapshots)
    node = await asyncio.to_thread(self._node_repo.get_by_node_id,
parent_node_id)
    if node:
        old_status = node.status
        await asyncio.to_thread(self._node_repo.update_progress,
parent_node_id, new_progress)
        if new_status != old_status:
            await asyncio.to_thread(self._node_repo.update_status,
parent_node_id, new_status)
    async def _build_snapshot(self, node_id: str) -> NodeSnapshot | None:
        node = await asyncio.to_thread(self._node_repo.get_by_node_id, node_id)
        if node is None:
            return None
        snap = NodeSnapshot(
            node_id=node.node_id,
            status=node.status,
            progress=_parse_decimal(node.progress),
            parent_node_id=node.parent_node_id,
        )
        deps = await asyncio.to_thread(self._dep_repo.find_by_node, node_id)
        snap.dependencies = [
            DependencySnapshot(
                depends_on_deliverable_id=d.depends_on_deliverable_id,
                depends_on_node_id=d.depends_on_node_id,
                weight=_parse_decimal(d.weight),
                dependency_type=d.dependency_type,
            )
            for d in deps
        ]
        delivs = await asyncio.to_thread(self._deliv_repo.find_by_node, node_id)
        snap.deliverables = [
            DeliverableSnapshot(
                deliverable_id=d.deliverable_id,
                target_amount=_parse_decimal(d.target_amount),
                current_amount=_parse_decimal(d.current_amount),
                is_required=d.is_required,
                file_id=d.file_id,
            )
            for d in delivs
        ]
        children = await asyncio.to_thread(self._node_repo.find_by_parent,
node_id)
        snap.children = [
            ChildSnapshot(
                node_id=c.node_id,
                status=c.status,
                progress=_parse_decimal(c.progress),
                child_weight=_parse_decimal(c.child_weight),
            )
            for c in children

```



```

    ]
    return snap
    async def _check_cycle(self, node_id: str, depends_on_deliverable_id: str) ->
CycleCheckResult:
    dep_deliv = await asyncio.to_thread(
        self._deliv_repo.get_by_deliverable_id, depends_on_deliverable_id,
    )
    if dep_deliv is None:
        return CycleCheckResult(has_cycle=False)
    upstream_node = dep_deliv.node_id
    if upstream_node == node_id:
        return CycleCheckResult(has_cycle=True, cycle_path=[node_id, node_id],
                                message="节点不能依赖自己的成果")
    node_obj = await asyncio.to_thread(self._node_repo.get_by_node_id,
node_id)
    if node_obj is None:
        return CycleCheckResult(has_cycle=False)
    all_nodes = await asyncio.to_thread(self._node_repo.find_by_project,
node_obj.project_id)
    all_node_ids = [n.node_id for n in all_nodes]
    deliverable_to_node: dict[str, str] = {}
    for nid in all_node_ids:
        delivs = await asyncio.to_thread(self._deliv_repo.find_by_node, nid)
        for d in delivs:
            deliverable_to_node[d.deliverable_id] = d.node_id
    node_deps: dict[str, list[str]] = {}
    for nid in all_node_ids:
        deps = await asyncio.to_thread(self._dep_repo.find_by_node, nid)
        upstream_ids = list(set(d.depends_on_node_id for d in deps))
        node_deps[nid] = upstream_ids
    has_cycle, path = detect_cycle(
        node_id, depends_on_deliverable_id, deliverable_to_node, node_deps,
    )
    return CycleCheckResult(
        has_cycle=has_cycle,
        cycle_path=path,
        message=" → ".join(path) if has_cycle else "",
    )
    async def _is_ancestor_dependency(self, node_id: str, upstream_node_id: str) -
> bool:
    ancestors = await asyncio.to_thread(self._node_repo.get_ancestor_chain,
node_id, max_depth=3)
    for a in ancestors:
        if a.node_id == upstream_node_id:
            return True
    return False

```

文件编号: 106 文件路径: emily-core\emily_core\services\node_state_machine.py

```

from __future__ import annotations
import logging
from collections import deque

```

```

logger = logging.getLogger("emily.node_state_machine")
NOT_ACTIVATED = "NOT_ACTIVATED"
CONDITIONS_NOT_MET = "CONDITIONS_NOT_MET"
IN_PROGRESS = "IN_PROGRESS"
COMPLETED = "COMPLETED"
VALID_STATUSES = frozenset({NOT_ACTIVATED, CONDITIONS_NOT_MET, IN_PROGRESS,
COMPLETED})
VALID_TRANSITIONS: dict[str, frozenset[str]] = {
    NOT_ACTIVATED: frozenset({CONDITIONS_NOT_MET}),
    CONDITIONS_NOT_MET: frozenset({IN_PROGRESS}),
    IN_PROGRESS: frozenset({COMPLETED, CONDITIONS_NOT_MET}),
    COMPLETED: frozenset(),
}
}
class NodeSnapshot:
    __slots__ = ("node_id", "status", "progress", "parent_node_id",
        "dependencies", "deliverables", "children")
    def __init__(self, node_id: str, status: str = NOT_ACTIVATED,
        progress: float = 0.0, parent_node_id: str = ""):
        self.node_id = node_id
        self.status = status
        self.progress = progress
        self.parent_node_id = parent_node_id
        self.dependencies: list[DependencySnapshot] = []
        self.deliverables: list[DeliverableSnapshot] = []
        self.children: list[ChildSnapshot] = []
class DependencySnapshot:
    __slots__ = ("depends_on_deliverable_id", "depends_on_node_id", "weight",
"dependency_type")
    def __init__(self, depends_on_deliverable_id: str, depends_on_node_id: str,
        weight: float = 1.0, dependency_type: str = "DELIVERABLE"):
        self.depends_on_deliverable_id = depends_on_deliverable_id
        self.depends_on_node_id = depends_on_node_id
        self.weight = weight
        self.dependency_type = dependency_type
class DeliverableSnapshot:
    __slots__ = ("deliverable_id", "target_amount", "current_amount",
"is_required", "file_id")
    def __init__(self, deliverable_id: str, target_amount: float, current_amount:
float,
        is_required: bool = True, file_id: str = ""):
        self.deliverable_id = deliverable_id
        self.target_amount = target_amount
        self.current_amount = current_amount
        self.is_required = is_required
        self.file_id = file_id
class ChildSnapshot:
    __slots__ = ("node_id", "status", "progress", "child_weight")
    def __init__(self, node_id: str, status: str, progress: float, child_weight:
float = 1.0):
        self.node_id = node_id
        self.status = status
        self.progress = progress
        self.child_weight = child_weight
def calc_dependency_satisfaction(dependencies: list[DependencySnapshot],

```

```

                                deliverable_file_status: dict[str, bool]) ->
float:
    if not dependencies:
        return 1.0
    total = 0.0
    for dep in dependencies:
        file_ready = deliverable_file_status.get(dep.depends_on_deliverable_id,
False)
        if file_ready:
            total += dep.weight
    return total
def calc_deliverable_completion(deliverables: list[DeliverableSnapshot]) -> float:
    required = [d for d in deliverables if d.is_required]
    if not required:
        return 1.0
    total_ratio = 0.0
    for d in required:
        target = max(d.target_amount, 0.001)
        current = min(d.current_amount, target)
        total_ratio += current / target
    return total_ratio / len(required)
def determine_node_status(dependencies: list[DependencySnapshot],
                            deliverables: list[DeliverableSnapshot],
                            deliverable_file_status: dict[str, bool],
                            children: list[ChildSnapshot]) -> str:

    if children:
        return _determine_parent_status(children)
    dep_satisfaction = calc_dependency_satisfaction(dependencies,
deliverable_file_status)
    if dep_satisfaction < 1.0:
        return CONDITIONS_NOT_MET
    if not deliverables:
        return CONDITIONS_NOT_MET
    completion = calc_deliverable_completion(deliverables)
    if completion >= 1.0:
        return COMPLETED
    return IN_PROGRESS
def _determine_parent_status(children: list[ChildSnapshot]) -> str:
    if not children:
        return CONDITIONS_NOT_MET
    statuses = [c.status for c in children]
    if all(s == CONDITIONS_NOT_MET for s in statuses):
        return CONDITIONS_NOT_MET
    if all(s == COMPLETED for s in statuses):
        return COMPLETED
    return IN_PROGRESS
def calc_parent_progress(children: list[ChildSnapshot]) -> float:
    if not children:
        return 0.0
    weighted_sum = sum(c.progress * c.child_weight for c in children)
    total_weight = max(sum(c.child_weight for c in children), 0.0001)
    return min(weighted_sum / total_weight, 100.0)
def detect_cycle(node_id: str, depends_on_deliverable_id: str,
                    deliverable_to_node: dict[str, str],

```

```

        node_dependencies: dict[str, list[str]],
        max_depth: int = 100) -> tuple[bool, list[str]]:
    upstream_node = deliverable_to_node.get(depends_on_deliverable_id)
    if upstream_node is None:
        return False, []
    if upstream_node == node_id:
        return True, [node_id, node_id]
    visited = {upstream_node}
    queue = deque([upstream_node])
    parent_map: dict[str, str | None] = {upstream_node: None}
    for _ in range(max_depth):
        if not queue:
            break
        current = queue.popleft()
        current_upstreams = node_dependencies.get(current, [])
        for next_upstream in current_upstreams:
            if next_upstream == node_id:
                path = _reconstruct_path(parent_map, current, upstream_node,
node_id)
                return True, path
            if next_upstream not in visited:
                visited.add(next_upstream)
                parent_map[next_upstream] = current
                queue.append(next_upstream)
    return False, []
def _reconstruct_path(parent_map: dict[str, str | None], end_node: str,
                      start_node: str, upstream_node: str) -> list[str]:
    path = []
    current = end_node
    while current is not None and current != start_node:
        path.append(current)
        current = parent_map.get(current)
    path.append(start_node)
    path.reverse()
    path.append(upstream_node)
    return path
def check_parent_child_cycle(parent_node_id: str, child_node_id: str,
                             parent_child_map: dict[str, str]) -> bool:
    current = parent_node_id
    visited = set()
    while current:
        if current == child_node_id:
            return True
        if current in visited:
            return True
        visited.add(current)
        current = parent_child_map.get(current, "")
    return False

```

文件编号: 107 文件路径: emily-core\emily_core\services\pending_issues.py

```

import logging
import os
from datetime import datetime, timezone
from pathlib import Path
logger = logging.getLogger("emily.service.pending_issues")
DEFAULT_ISSUES_PATH = "emily-data/notebooks/待解决问题.md"
class PendingIssuesService:
    def __init__(self, issues_path: str = ""):
        self._path = Path(issues_path) if issues_path else None
        self._resolved_path = False
    @property
    def path(self) -> Path:
        if self._resolved_path:
            return self._path
        self._resolved_path = True
        if self._path is not None:
            return self._path
        project_root = Path(__file__).parent.parent.parent.parent.parent
        self._path = project_root / DEFAULT_ISSUES_PATH
        return self._path
    def _ensure_file(self) -> None:
        p = self.path
        if not p.exists():
            p.parent.mkdir(parents=True, exist_ok=True)
            p.write_text(
                """
                "> 此文件由 Emy 守护Agent 自动维护，项目总经理处理并给出决策。\\n"
                "> 处理后移出本文，决策以事件形式入库并关联原事件。\\n\\n"
                """
                encoding="utf-8",
            )
            logger.info("Created pending issues file: %s", p)
    def _read(self) -> str:
        self._ensure_file()
        return self.path.read_text(encoding="utf-8")
    def _write(self, content: str) -> None:
        self._ensure_file()
        self.path.write_text(content, encoding="utf-8")
        logger.debug("Pending issues file updated: %s", self.path)
    def _generate_id(self, content: str) -> str:
        today = datetime.now(timezone.utc).strftime("%Y%m%d")
        import re
        pattern = rf"PND-{today}-(\\d+)"
        matches = re.findall(pattern, content)
        if matches:
            max_seq = max(int(m) for m in matches)
            seq = max_seq + 1
        else:
            seq = 1
        return f"PND-{today}-{seq:04d}"
    def add(

```

```

        self,
        raised_by: str,
        source: str,
        description: str,
        suggestion: str = "",
        related_events: list[str] | None = None,
    ) -> str:
        content = self._read()
        issue_id = self._generate_id(content)
        now = datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M")
        related = ""
        if related_events:
            related = "\n".join(related_events)
        entry = (
            f"
            f"- **提出时间**: {now}\n"
            f"- **提出人**: {raised_by}\n"
            f"- **来源**: {source}\n"
        )
        if related:
            entry += f"- **关联事件**: {related}\n"
        entry += (
            f"- **问题描述**: {description}\n"
            f"- **建议**: {suggestion}\n\n"
        )
        pending_marker = "
        if pending_marker in content:
            pos = content.index(pending_marker) + len(pending_marker)
            newline_pos = content.index("\n", pos)
            content = content[:newline_pos + 1] + "\n" + entry +
content[newline_pos + 1:]
        else:
            content += "\n#
        self._write(content)
        logger.info("Pending issue added: %s - %s", issue_id, description[:50])
        return issue_id

    def resolve(
        self,
        issue_id: str,
        handler: str,
        decision: str,
        decision_event_id: str = "",
    ) -> bool:
        content = self._read()
        marker = f"
        if marker not in content:
            logger.warning("Pending issue not found: %s", issue_id)
            return False
        start = content.index(marker)
        rest = content[start + len(marker):]
        import re
        next_section = re.search(r"\n(
        if next_section:
            entry_end = start + len(marker) + next_section.start()

```

```

else:
    entry_end = len(content)
    entry_content = content[start:entry_end]
    now = datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M")
    resolved_entry = marker + "    ✓ 已处理\n"
    lines = entry_content[len(marker):].strip().split("\n")
    resolved_entry += "\n".join(lines) + "\n"
    resolved_entry += f"- **处理时间**: {now}\n"
    resolved_entry += f"- **处理人**: {handler}\n"
    if decision_event_id:
        resolved_entry += f"- **决策事件**: {decision_event_id}\n"
    resolved_entry += f"- **决策**: {decision}\n\n"
    content = content[:start] + content[entry_end:]
    resolved_marker = "
    if resolved_marker in content:
        pos = content.index(resolved_marker) + len(resolved_marker)
        newline_pos = content.index("\n", pos)
        content = content[:newline_pos + 1] + "\n" + resolved_entry +
content[newline_pos + 1:]
    else:
        content += "\n#
    self._write(content)
    logger.info("Pending issue resolved: %s by %s", issue_id, handler)
    return True
def list_pending(self) -> list[dict]:
    content = self._read()
    pending_marker = "
    if pending_marker not in content:
        return []
    start = content.index(pending_marker) + len(pending_marker)
    rest = content[start:]
    import re
    next_section = re.search(r"\n#
    if next_section:
        section = rest[:next_section.start()]
    else:
        section = rest
    return self._parse_entries(section)
def list_resolved(self) -> list[dict]:
    content = self._read()
    resolved_marker = "
    if resolved_marker not in content:
        return []
    start = content.index(resolved_marker) + len(resolved_marker)
    section = content[start:]
    return self._parse_entries(section)
def _parse_entries(self, section: str) -> list[dict]:
    import re
    entries = []
    parts = re.split(r"\n#
    for part in parts:
        if not part.strip():
            continue
        entry = {"id": "", "is_resolved": False}

```

```

lines = part.strip().split("\n")
title = lines[0].strip()
if "✅ 已处理" in title:
    entry["is_resolved"] = True
    title = title.replace("✅ 已处理", "").strip()
entry["id"] = title
for line in lines[1:]:
    match = re.match(r"- \*\*(.+?)\*\*: (.*)", line)
    if match:
        key = match.group(1).strip()
        value = match.group(2).strip()
        entry[key] = value
if entry.get("id"):
    entries.append(entry)
return entries

```

文件编号: 108 文件路径: emily-core\emily_core\services\permission_service.py

```

from __future__ import annotations
import json
import logging
from typing import Optional
from emily_core.infrastructure.database.models import (
    CompanyInfo,
    PermissionGroup,
    User,
    _utc_now,
)
from emily_core.permission.level import can_access, LEVEL_NAME, level_label
from emily_core.repositories.permission_grant_repo import
PermissionGrantRepository
from emily_core.repositories.permission_repo import PermissionRepository
logger = logging.getLogger("emily.permission")
def _extract_node_ids(fs) -> list[str]:
    nodes: list[str] = []
    if isinstance(fs, list):
        for item in fs:
            if isinstance(item, dict):
                if "nodeIds" in item:
                    nodes.extend(item["nodeIds"] or [])
            else:
                for v in item.values():
                    if isinstance(v, list):
                        nodes.extend(v)
                    elif isinstance(v, str):
                        nodes.append(v)
    elif isinstance(fs, dict):
        for v in fs.values():
            if isinstance(v, list):
                nodes.extend(v)
            elif isinstance(v, str):
                nodes.append(v)

```



```

    return nodes
_SCOPE_NODE_MAP: dict[str, str] = {
    "室内精装": "design",
    "软装深化": "design",
    "BIM建模": "design",
    "幕墙精装": "design",
    "精装": "design",
    "设计": "design",
    "施工": "construction",
    "工程": "construction",
    "监理": "supervision",
    "景观": "landscape",
    "总包": "construction",
    "分包": "construction",
    "采购": "procurement",
    "供货": "procurement",
}

def _scope_to_node_ids(scopes: list[str]) -> list[str]:
    node_set: set[str] = set()
    for scope_text in scopes:
        for cn_keyword, node_id in _SCOPE_NODE_MAP.items():
            if cn_keyword in scope_text:
                node_set.add(node_id)
    return sorted(node_set) if node_set else []

_CONSTRUCTION_TYPES: frozenset[str] = frozenset({
    "施工单位", "总包", "总承包", "施工总包",
})

_PARTICIPANT_DB_PERMS: dict[str, dict[str, str]] = {
    "construction": {
        "project": "read",
        "event": "read_write",
        "task": "read_write",
        "meeting": "read_write",
    },
    "non_construction": {
        "project": "read",
        "event": "read_write",
        "task": "read",
        "meeting": "read_write",
    },
}

class PermissionService:
    def __init__(self,
                 repo: Optional[PermissionRepository] = None,
                 grant_repo: Optional[PermissionGrantRepository] = None,
                 fail_open: bool = True,
                 cache=None,
                 auth_engine=None,
                 audit_repo=None,
                 skill_registry=None):
        self._repo = repo or PermissionRepository()
        self._grant_repo = grant_repo or PermissionGrantRepository()
        self._fail_open = fail_open
        self._cache = cache

```

```

self._auth_engine = auth_engine
self._audit_repo = audit_repo
self._skill_registry = skill_registry
def build_permission_dict(self, user_id: str) -> dict:
    try:
        return self._do_build_snapshot(user_id)
    except Exception as e:
        logger.warning("build_permission_dict failed user=%s: %s", user_id, e)
        if self._fail_open:
            return {"level": 1}
        raise
build_permission_snapshot = build_permission_dict
def _do_build_snapshot(self, user_id: str) -> dict:
    user = self._repo.get_user(user_id)
    if user is None:
        logger.warning("user not found, fallback to L1: %s", user_id)
        return {"level": 1}
    company = self._repo.get_company(user.company) if user.company else None
    grants = self._grant_repo.get_active_grants(user_id)
    if self._cache is not None:
        sop_allow, denied_sop_ids = self._cache.get_user_whitelist(
            user_id, user.level,
            company.type if company else "",
            self._all_departments(company),
        )
    else:
        sop_allow, denied_sop_ids = self._compute_sop_allow(user, company)
    granted_codes = [g.perm_code for g in grants if g.grant_type != "AUTO"]
    denied_codes = [f"SOP-INTERNAL-*--{sid}" for sid in denied_sop_ids]
    perm_version = self._cache.get_version() if self._cache else 0
    return {
        "level": user.level,
        "user_id": user_id,
        "company_id": user.company or "",
        "company_type": company.type if company else "",
        "company_name": company.company_name if company else "",
        "department": self._all_departments(company),
        "project_ids": self._derive_project_ids(user, company),
        "partner_ids": self._load_json_list(company.partners) if company else
[],
        "scopes": self._load_json_list(company.scope) if company else [],
        "sop_allow": sop_allow,
        "db_perms": self._derive_db_perms(user.level, company.type if company
else ""),
        "is_management_unit": bool(company and getattr(company, 'is_admin',
False)),
        "info_level": self._derive_info_level(
            user.level,
            is_management_unit=bool(company and getattr(company, 'is_admin',
False)),
        ),
        "supervisor_id": user.supervisor_id or "",
        "authorized_node_ids": self._derive_authorized_nodes(company),
        "granted_codes": granted_codes,

```

```

        "denied_codes": denied_codes,
        "permissions_loaded_at": _utc_now(),
        "permission_version": perm_version,
    }
def _compute_sop_allow(self, user: User, company: Optional[CompanyInfo]):
    sop_flows = self._repo.list_active_sop_flows()
    if sop_flows:
        bindings = self._repo.list_sop_bindings()
        groups = self._repo.list_permission_groups()
        user_company_type = company.type if company else ""
        user_departments = self._all_departments(company)
        matched_group_ids = {
            g.id for g in groups
            if self._group_matches_user(g, user_company_type,
user_departments)
        }
        sop_allow: list[str] = []
        denied_sop_ids: list[str] = []
        for flow in sop_flows:
            flow_bindings = [b for b in bindings if b.sop_business_flow_id ==
flow.id]
            if any(b.binding_type == "deny" and b.permission_group_id in
matched_group_ids
                    for b in flow_bindings):
                denied_sop_ids.append(flow.sop_id)
                continue
            if flow.is_public:
                sop_allow.append(flow.sop_id)
                continue
            if not can_access(user.level, flow.min_level):
                continue
            sop_allow.append(flow.sop_id)
        return sop_allow, denied_sop_ids
    if self._skill_registry is not None:
        try:
            skill_ids = self._skill_registry.list_sop_ids()
            if skill_ids:
                logger.info(
                    "sop_business_flows empty, using SkillRegistry fallback:
%d skills for user=%s",
                    len(skill_ids), user.id,
                )
                return skill_ids, []
        except Exception as e:
            logger.warning("SkillRegistry fallback failed: %s", e)
    return [], []
@staticmethod
def _group_matches_user(group: PermissionGroup, user_company_type: str,
                        user_departments: list[str]) -> bool:
    if group.company_type and group.company_type != user_company_type:
        return False
    if group.department and group.department not in user_departments:
        return False
    return True

```

```

@staticmethod
def _all_departments(company: Optional[CompanyInfo]) -> list[str]:
    if not company or not company.department:
        return []
    return PermissionService._load_json_list(company.department)

@staticmethod
def _primary_department(company: Optional[CompanyInfo]) -> str:
    depts = PermissionService._all_departments(company)
    return depts[0] if depts else ""

@staticmethod
def _derive_info_level(level: int, is_management_unit: bool = False) -> str:
    if level >= 5:
        return "confidential"
    if is_management_unit:
        return "confidential"
    if level >= 2:
        return "internal"
    return "public"

@staticmethod
def _derive_db_perms(level: int, company_type: str = "") -> dict[str, str]:
    if level < 2:
        return {"project": "read"} if level >= 1 else {}
    if level >= 5:
        perms = {
            "project": "read_write",
            "event": "read_write",
            "task": "read_write",
            "meeting": "read_write",
        }
        if level >= 5:
            perms["financial"] = "read"
        return perms
    if level == 4:
        return {
            "project": "read_write",
            "event": "read_write",
            "task": "read_write",
            "meeting": "read_write",
        }
    key = "construction" if company_type in _CONSTRUCTION_TYPES else
"non_construction"
    return dict(_PARTICIPANT_DB_PERMS[key])

@staticmethod
def _derive_authorized_nodes(company: Optional[CompanyInfo]) -> list[str]:
    if not company:
        return []
    if company.function_scope and company.function_scope not in ("{}", "[]",
    ""):
        try:
            fs = json.loads(company.function_scope)
            nodes = _extract_node_ids(fs)
            if nodes:
                return nodes
        except (json.JSONDecodeError, TypeError):

```

```

        pass
        scopes = PermissionService._load_json_list(company.scope) if company.scope
    else []
    if scopes:
        return _scope_to_node_ids(scopes)
    return []
    @staticmethod
    def _derive_project_ids(user: User, company: Optional[CompanyInfo]) ->
list[str]:
        pid = getattr(user, "project_id", None)
        if pid:
            return [pid]
        return []
    @staticmethod
    def _load_json_list(raw: str) -> list:
        if not raw:
            return []
        try:
            data = json.loads(raw)
            return data if isinstance(data, list) else []
        except (json.JSONDecodeError, TypeError):
            return []
    async def check(self, user_id: str, sop_id: str) -> dict:
        import asyncio
        perm_dict = await asyncio.to_thread(self.build_permission_dict, user_id)
        if self._auth_engine is not None:
            result = await self._auth_engine.check_sop_access(perm_dict, sop_id)
            return {
                "allowed": result.allowed,
                "reason": result.reason,
                "suggested_approver": result.suggested_approver,
                "details": result.matched_details,
            }
        allowed = sop_id in perm_dict.get("sop_allow", [])
        return {
            "allowed": allowed,
            "reason": "" if allowed else f"SOP {sop_id} 不在用户白名单中",
            "suggested_approver": perm_dict.get("supervisor_id", "") if not
allowed else "",
        }
    async def grant(self, *, grantee_id: str, grantor_id: str, perm_code: str,
                    grant_type: str = "TEMP", operations: str = '["read"]',
                    expire_time: Optional[str] = None, remark: str = "",
                    client_ip: str = "") -> dict:
        import asyncio
        if grant_type == "PERMANENT" and not remark:
            return {"success": False, "reply": "永久授权必须填写授权原因 (remark)"}
        if grant_type == "TEMP" and not expire_time:
            return {"success": False, "reply": "临时授权必须设置过期时间
(expire_time)"}
        grantor = await asyncio.to_thread(self._repo.get_user, grantor_id)
        if grantor is None:
            return {"success": False, "reply": f"授权人 {grantor_id} 不存在"}
        from emily_core.permission.level import is_admin as _is_admin

```

```

        if not _is_admin(grantor.level):
            grantor_snapshot = await
asyncio.to_thread(self.build_permission_snapshot, grantor_id)
        if not self._grantor_has_permission(grantor_snapshot, perm_code):
            return {"success": False, "reply": "授权人无此资源权限, 无法授权"}
    try:
        grant_no = await asyncio.to_thread(
            self._grant_repo.generate_grant_no,
        )
        grant_record = await asyncio.to_thread(
            self._grant_repo.create,
            grant_no=grant_no,
            grantee_id=grantee_id,
            perm_code=perm_code,
            grant_type=grant_type,
            grantor_id=grantor_id,
            operations=operations,
            expire_time=expire_time,
            remark=remark,
            client_ip=client_ip,
        )
        if self._audit_repo is not None:
            await asyncio.to_thread(
                self._audit_repo.log_grant,
                grantor_id=grantor_id,
                grantee_id=grantee_id,
                perm_code=perm_code,
                grant_type=grant_type,
                remark=remark,
            )
        if self._cache is not None:
            self._cache.invalidate_user(grantee_id)
        return {
            "success": True,
            "grant_no": grant_no,
            "reply": f"授权成功 ({grant_type}, 编号 {grant_no})",
        }
    except Exception as e:
        logger.error("grant failed: %s", e)
        return {"success": False, "reply": f"授权失败: {e}"}
    async def revoke(self, *, grant_no: str, revoke_reason: str = "",
                    operator_id: str = "") -> dict:
        import asyncio
        try:
            grant = await asyncio.to_thread(
                self._grant_repo.get_by_grant_no, grant_no,
            )
            if grant is None:
                return {"success": False, "reply": f"授权记录 {grant_no} 不存在"}
            if grant.status != "ACTIVE":
                return {"success": False, "reply": f"授权记录 {grant_no} 状态为 {grant.status}, 无法撤销"}
            operator = await asyncio.to_thread(self._repo.get_user, operator_id)
            if operator is not None:

```

```

        from emily_core.permission.level import is_admin as _is_admin
        is_force = _is_admin(operator.level) and operator_id !=
grant.grantor_id
        if operator_id != grant.grantor_id and not is_force:
            return {"success": False, "reply": "仅授权人或管理员可撤销授权"}
        else:
            is_force = False
        await asyncio.to_thread(
            self._grant_repo.revoke, grant_no, revoke_reason,
        )
        if self._audit_repo is not None:
            await asyncio.to_thread(
                self._audit_repo.log_revoke,
                grantor_id=operator_id,
                grantee_id=grant.grantee_id,
                perm_code=grant.perm_code,
                remark=revoke_reason or ("强制撤销" if is_force else "主动撤
销"),
            )
        if self._cache is not None:
            self._cache.invalidate_user(grant.grantee_id)
            return {"success": True, "reply": f"授权 {grant_no} 已撤销"}
    except Exception as e:
        logger.error("revoke failed: %s", e)
        return {"success": False, "reply": f"撤销失败: {e}"}
    async def query_user_permissions(self, user_id: str) -> dict:
        import asyncio
        try:
            perm_dict = await asyncio.to_thread(self.build_permission_dict,
user_id)
            grants = await asyncio.to_thread(self._grant_repo.get_active_grants,
user_id)
            return {
                "success": True,
                "permissions": {
                    "user_id": user_id,
                    "level": perm_dict["level"],
                    "level_name": level_label(perm_dict["level"]),
                    "company_id": perm_dict.get("company_id", ""),
                    "company_type": perm_dict.get("company_type", ""),
                    "company_name": perm_dict.get("company_name", ""),
                    "department": perm_dict.get("department", []),
                    "is_management_unit": perm_dict.get("is_management_unit",
False),
                    "info_level": perm_dict.get("info_level", "public"),
                    "sop_allow": perm_dict.get("sop_allow", []),
                    "db_perms": perm_dict.get("db_perms", {}),
                    "authorized_node_ids": perm_dict.get("authorized_node_ids",
[]),
                    "granted_codes": perm_dict.get("granted_codes", []),
                    "denied_codes": perm_dict.get("denied_codes", []),
                    "active_grants": [
                        {
                            "grant_no": g.grant_no,

```

```

        "perm_code": g.perm_code,
        "grant_type": g.grant_type,
        "operations": g.operations,
        "expire_time": g.expire_time,
        "status": g.status,
    }
    for g in grants
],
    "permission_version": perm_dict.get("permission_version", 0),
    "permissions_loaded_at":
perm_dict.get("permissions_loaded_at", ""),
    },
}
except Exception as e:
    logger.error("query_user_permissions failed: %s", e)
    return {"success": False, "reply": f"查询权限失败: {e}"}
@staticmethod
def _grantor_has_permission(grantor_snapshot: dict, perm_code: str) -> bool:
    from emily_core.permission.code_compiler import code_matches_any
    granted = grantor_snapshot.get("granted_codes", [])
    if perm_code in granted:
        return True
    if code_matches_any(perm_code, granted):
        return True
    return False

```

文件编号: 109 文件路径: emily-core\emily_core\services\query_service.py

```

import logging
from datetime import datetime, timezone, timedelta
from typing import Any
from ..repositories.event_repo import EventRepository
from ..repositories.task_repo import TaskRepository
from ..repositories.meeting_repo import MeetingRepository
from ..repositories.file_repo import FileRepository
from ..repositories.message_repo import MessageRepository
from ..repositories.user_repo import UserRepository
from ..adapters.standard.command import QueryCommand
logger = logging.getLogger("emily.service.query")
def _resolve_time_range(time_range: str) -> tuple[str | None, str | None]:
    if time_range == "all":
        return None, None
    now = datetime.now(timezone.utc)
    if time_range == "today":
        start = now.replace(hour=0, minute=0, second=0, microsecond=0)
        end = now
    elif time_range == "this_week":
        start = now - timedelta(days=now.weekday())
        start = start.replace(hour=0, minute=0, second=0, microsecond=0)
        end = now
    elif time_range == "this_month":
        start = now.replace(day=1, hour=0, minute=0, second=0, microsecond=0)

```



```
        end = now
    else:
        return None, None
    return start.isoformat(), end.isoformat()
class QueryService:
    def __init__(self):
        self.event_repo = EventRepository()
        self.task_repo = TaskRepository()
        self.meeting_repo = MeetingRepository()
        self.file_repo = FileRepository()
        self.message_repo = MessageRepository()
        self.user_repo = UserRepository()
        self._journal = None
    def set_journal(self, journal) -> None:
        self._journal = journal
    def execute(self, cmd: QueryCommand) -> dict[str, Any]:
        qt = cmd.query_type
        if qt == "event":
            items = self.query_events(
                project_id=cmd.project_id,
                project_ids=cmd.project_ids,
                time_range=cmd.time_range,
                status=cmd.status_filter,
                limit=cmd.limit,
            )
            return {
                "query_type": qt,
                "items": items,
                "total": len(items),
            }
        elif qt == "task":
            items = self.query_tasks(
                project_id=cmd.project_id,
                project_ids=cmd.project_ids,
                time_range=cmd.time_range,
                status=cmd.status_filter,
                assignee=cmd.assignee,
                limit=cmd.limit,
            )
            return {
                "query_type": qt,
                "items": items,
                "total": len(items),
            }
        elif qt == "meeting":
            items = self.query_meetings(
                project_id=cmd.project_id,
                project_ids=cmd.project_ids,
                time_range=cmd.time_range,
                limit=cmd.limit,
            )
            return {
                "query_type": qt,
                "items": items,
```

```
        "total": len(items),
    }
elif qt == "file":
    items = self.query_files(
        project_id=cmd.project_id,
        project_ids=cmd.project_ids,
        file_type=cmd.file_type,
        limit=cmd.limit,
    )
    return {
        "query_type": qt,
        "items": items,
        "total": len(items),
    }
elif qt == "message":
    items = self.query_messages(
        project_id=cmd.project_id,
        time_range=cmd.time_range,
        conversation_id=cmd.conversation_id,
        sender_name=cmd.sender_name,
        keyword=cmd.keyword,
        intent=cmd.intent,
        limit=cmd.limit,
    )
    return {
        "query_type": qt,
        "items": items,
        "total": len(items),
    }
elif qt == "conversation":
    items = self.query_conversations(
        time_range=cmd.time_range,
        limit=cmd.limit,
    )
    return {
        "query_type": qt,
        "items": items,
        "total": len(items),
    }
elif qt == "user":
    items = self.query_users(
        time_range=cmd.time_range,
        limit=cmd.limit,
    )
    return {
        "query_type": qt,
        "items": items,
        "total": len(items),
    }
elif qt == "project":
    items = self.query_projects()
    return {
        "query_type": qt,
        "items": items,
```

```
        "total": len(items),
    }
    elif qt == "journal":
        items = self.query_journal(
            time_range=cmd.time_range,
            keyword=cmd.keyword,
            limit=cmd.limit,
        )
        return {
            "query_type": qt,
            "items": items,
            "total": len(items),
        }
    elif qt == "summary":
        summary = self.get_summary(project_id=cmd.project_id)
        return {
            "query_type": qt,
            "summary": summary,
            "items": None,
            "total": 0,
        }
    else:
        logger.warning("Unknown query_type: %s", qt)
        return {
            "query_type": qt,
            "items": [],
            "total": 0,
        }
}

def query_events(
    self,
    project_id: str | None = None,
    project_ids: list[str] | None = None,
    time_range: str = "all",
    status: str | None = None,
    limit: int = 50,
):
    return self.event_repo.query_events(
        project_id=project_id,
        project_ids=project_ids,
        time_range=time_range,
        status=status,
        limit=limit,
    )

def query_tasks(
    self,
    project_id: str | None = None,
    project_ids: list[str] | None = None,
    time_range: str = "all",
    status: str | None = None,
    assignee: str | None = None,
    limit: int = 50,
):
    return self.task_repo.query_tasks(
        project_id=project_id,
```

```
        project_ids=project_ids,
        time_range=time_range,
        status=status,
        assignee=assignee,
        limit=limit,
    )
def query_meetings(
    self,
    project_id: str | None = None,
    project_ids: list[str] | None = None,
    time_range: str = "all",
    limit: int = 50,
):
    return self.meeting_repo.query_meetings(
        project_id=project_id,
        project_ids=project_ids,
        time_range=time_range,
        limit=limit,
    )
def query_files(
    self,
    project_id: str | None = None,
    project_ids: list[str] | None = None,
    file_type: str | None = None,
    limit: int = 50,
):
    return self.file_repo.query_files(
        project_id=project_id,
        project_ids=project_ids,
        file_type=file_type,
        limit=limit,
    )
def query_messages(
    self,
    project_id: str | None = None,
    time_range: str = "all",
    conversation_id: str | None = None,
    sender_name: str | None = None,
    keyword: str | None = None,
    intent: str | None = None,
    limit: int = 50,
):
    return self.message_repo.query_messages(
        project_id=project_id,
        time_range=time_range,
        conversation_id=conversation_id,
        sender_name=sender_name,
        keyword=keyword,
        intent=intent,
        limit=limit,
    )
def query_conversations(
    self,
    time_range: str = "all",
```

```
        limit: int = 20,
    ) -> list[dict]:
        return self.message_repo.get_active_conversations(
            time_range=time_range,
            limit=limit,
        )
    def query_users(
        self,
        time_range: str = "all",
        limit: int = 50,
    ) -> list[dict]:
        users = self.user_repo.list_users(limit=limit)
        results = []
        for u in users:
            results.append({
                "user_id": u.id,
                "username": u.username,
                "real_name": u.username,
                "status": u.status or "active",
                "created_at": u.created_at,
            })
        return results
    def query_projects(self) -> list[dict]:
        projects = self.event_repo.list_projects(status="active")
        archived = self.event_repo.list_projects(status="archived")
        results = []
        for p in projects + archived:
            results.append({
                "project_id": p.id,
                "code": p.code,
                "name": p.name,
                "description": p.description,
                "status": p.status,
            })
        return results
    def query_journal(
        self,
        time_range: str = "all",
        keyword: str = "",
        limit: int = 50,
    ) -> list[str]:
        if self._journal is None:
            return ["项目日志服务未启用"]
        return self._journal.search(keyword=keyword, limit=limit)
    def get_summary(self, project_id: str | None = None) -> dict:
        event_counts = self.event_repo.count_by_status(project_id=project_id)
        task_counts = self.task_repo.count_by_status(project_id=project_id)
        messages = self.message_repo.query_messages(
            project_id=project_id, limit=10000,
        )
        message_count = len(messages)
        conversations = self.message_repo.get_active_conversations(limit=100)
        conversation_count = len(conversations)
        user_count = self.user_repo.count_users(status="active")
```

```
projects = self.event_repo.list_projects(status="active")
project_count_total = len(projects)
files = self.file_repo.query_files(project_id=project_id, limit=10000)
file_count = len(files)
meetings = self.meeting_repo.query_meetings(
    project_id=project_id, limit=10000,
)
meeting_count = len(meetings)
return {
    "events": event_counts,
    "events_total": sum(event_counts.values()),
    "tasks": task_counts,
    "tasks_total": sum(task_counts.values()),
    "messages_total": message_count,
    "conversations_total": conversation_count,
    "users_total": user_count,
    "projects_total": project_count_total,
    "files_total": file_count,
    "meetings_total": meeting_count,
}
}

@staticmethod
def format_reply(query_type: str, results: dict) -> str:
    total = results.get("total", 0)
    items = results.get("items")
    if query_type == "event":
        if total == 0:
            return "暂无事件记录。"
        lines = [f"共找到 {total} 条事件记录: "]
        for i, ev in enumerate(items, 1):
            if i > 10 and total > 10:
                lines.append(f"... 还有 {total - 10} 条")
                break
            title = getattr(ev, "title", "") or ""
            event_no = getattr(ev, "event_no", "") or ""
            status = getattr(ev, "status", "") or ""
            lines.append(f" {event_no} [{status}] {title}")
        return "\n".join(lines)
    elif query_type == "task":
        if total == 0:
            return "暂无任务记录。"
        lines = [f"共找到 {total} 条任务: "]
        for i, tk in enumerate(items, 1):
            if i > 10 and total > 10:
                lines.append(f"... 还有 {total - 10} 条")
                break
            title = getattr(tk, "title", "") or ""
            task_no = getattr(tk, "task_no", "") or ""
            status = getattr(tk, "status", "") or "todo"
            owner = getattr(tk, "owner_text", "") or ""
            line = f" {task_no} [{status}] {title}"
            if owner:
                line += f" - {owner}"
            lines.append(line)
        return "\n".join(lines)
```

```
elif query_type == "meeting":
    if total == 0:
        return "暂无会议记录。"
    lines = [f"共找到 {total} 场会议: "]
    for i, mt in enumerate(items, 1):
        if i > 10 and total > 10:
            lines.append(f"... 还有 {total - 10} 条")
            break
        title = getattr(mt, "title", "") or "未命名会议"
        meeting_no = getattr(mt, "meeting_no", "") or ""
        lines.append(f" {meeting_no} {title}")
    return "\n".join(lines)
elif query_type == "file":
    if total == 0:
        return "暂无文件记录。"
    lines = [f"共找到 {total} 个文件: "]
    for i, f in enumerate(items, 1):
        if i > 10 and total > 10:
            lines.append(f"... 还有 {total - 10} 个")
            break
        filename = getattr(f, "filename", "") or "未知文件"
        file_no = getattr(f, "file_no", "") or ""
        file_type = getattr(f, "file_type", "") or ""
        line = f" {file_no} {filename}"
        if file_type:
            line += f" ({file_type})"
        lines.append(line)
    return "\n".join(lines)
elif query_type == "message":
    if total == 0:
        return "暂无消息记录。"
    lines = [f"共找到 {total} 条消息: "]
    for i, msg in enumerate(items, 1):
        if i > 10 and total > 10:
            lines.append(f"... 还有 {total - 10} 条")
            break
        sender = getattr(msg, "sender_name", "") or "未知"
        content = getattr(msg, "content", "") or ""
        if len(content) > 80:
            content = content[:80] + "..."
        created = getattr(msg, "created_at", "") or ""
        time_str = ""
        if created:
            try:
                dt = datetime.fromisoformat(created)
                time_str = dt.strftime("%m-%d %H:%M")
            except (ValueError, TypeError):
                pass
        line = f" [{time_str}] {sender}: {content}"
        lines.append(line)
    return "\n".join(lines)
elif query_type == "conversation":
    if total == 0:
        return "暂无活跃会话。"
```

```

lines = [f"活跃会话 Top {min(total, 10)}: "]
for i, conv in enumerate(items[:10], 1):
    title = conv.get("title", "未知会话")
    count = conv.get("count", 0)
    last = conv.get("last_active", "")
    time_str = ""
    if last:
        try:
            dt = datetime.fromisoformat(last)
            time_str = dt.strftime("%m-%d %H:%M")
        except (ValueError, TypeError):
            pass
    lines.append(f"  {i}. {title} ({count} 条消息, 最后活跃:
{time_str})")
return "\n".join(lines)
elif query_type == "user":
    if total == 0:
        return "暂无注册用户。"
    lines = [f"共有 {total} 位用户: "]
    for i, u in enumerate(items[:10], 1):
        name = u.get("real_name") or u.get("username") or "未知"
        status = u.get("status", "active")
        line = f"  {name}"
        if status != "active":
            line += f" [{status}]"
        lines.append(line)
    if total > 10:
        lines.append(f"... 还有 {total - 10} 位")
    return "\n".join(lines)
elif query_type == "project":
    if total == 0:
        return "暂无项目。"
    lines = [f"共有 {total} 个项目: "]
    for i, p in enumerate(items[:10], 1):
        name = p.get("name", "未知")
        code = p.get("code", "")
        status = p.get("status", "active")
        line = f"  {name}"
        if code:
            line += f" ({code})"
        if status != "active":
            line += f" [{status}]"
        lines.append(line)
    return "\n".join(lines)
elif query_type == "journal":
    items = results.get("items") or []
    if not items:
        return "暂无项目事件日志。"
    lines = [f"项目事件日志 (最近 {len(items)} 条): "]
    for entry in items:
        lines.append(f"  {entry}")
    return "\n".join(lines)
elif query_type == "summary":
    s = results.get("summary") or {}

```



```

        lines = ["项目概览: "]
        lines.append(f"  事件: {s.get('events_total', 0)} 条 (pending: {s.get('events', {}).get('pending', 0)})")
        lines.append(f"  任务: {s.get('tasks_total', 0)} 条 (todo: {s.get('tasks', {}).get('todo', 0)})")
        lines.append(f"  会议: {s.get('meetings_total', 0)} 场")
        lines.append(f"  文件: {s.get('files_total', 0)} 个")
        lines.append(f"  消息: {s.get('messages_total', 0)} 条")
        lines.append(f"  活跃会话: {s.get('conversations_total', 0)} 个")
        lines.append(f"  用户: {s.get('users_total', 0)} 人")
        lines.append(f"  活跃项目: {s.get('projects_total', 0)} 个")
        return "\n".join(lines)
    logger.warning("Unknown query_type in format_reply: %s", query_type)
    return "查询结果暂不支持显示。"

```

文件编号: 110 文件路径: emily-core\emily_core\services\rule_book_loader.py

```

from __future__ import annotations
import logging
import os
from pathlib import Path
from typing import Optional
logger = logging.getLogger("emily.rule_book_loader")
class RuleBookLoader:
    def __init__(self):
        self._content: str = ""
        self._loaded: bool = False
    def load(self) -> str:
        candidates = []
        candidates.append("/app/rules/规则书.md")
        env_dir = os.environ.get("EMILY_RULE_BOOK_DIR", "")
        if env_dir:
            candidates.append(str(Path(env_dir) / "规则书.md"))
        dev_path = Path(__file__).resolve().parents[2] / "emily-data" / "rules" / "规则书.md"
        candidates.append(str(dev_path))
        for path in candidates:
            p = Path(path)
            if p.exists() and p.is_file():
                try:
                    self._content = p.read_text(encoding="utf-8")
                    self._loaded = True
                    logger.info("RuleBook loaded from %s (%d chars)", path, len(self._content))
                except Exception as e:
                    logger.warning("Failed to read rule book from %s: %s", path, e)
            self._content = ""
            self._loaded = False
        logger.warning("RuleBook file not found in any candidate path, using empty string")

```

```

        return ""
    def reload(self) -> dict:
        old_len = len(self._content)
        self.load()
        new_len = len(self._content)
        changed = old_len != new_len
        logger.info("RuleBook reload: %d->%d chars, changed=%s", old_len, new_len,
changed)
        return {
            "ok": True,
            "content_length": new_len,
            "changed": changed,
        }
    @property
    def content(self) -> str:
        if not self._loaded:
            self.load()
        return self._content
    @property
    def is_loaded(self) -> bool:
        return self._loaded

```

文件编号: 111 文件路径: emily-core\emily_core\services\schema_drift_detector.py

```

from __future__ import annotations
import logging
from typing import Optional
from ..repositories.system_description_repo import SystemDescriptionRepo
logger = logging.getLogger("emily.schema_drift_detector")
class SchemaDriftDetector:
    def detect(self) -> dict:
        stored = SystemDescriptionRepo.get_latest()
        if stored is None:
            return {
                "has_description": False,
                "needs_build": True,
                "has_drift": True,
                "stale_domains": ["database", "file", "permission"],
                "drift": {},
            }
        drift = {}
        drift["database"] = self._check_schema_drift(stored.schema_hash)
        drift["file"] = self._check_file_model_drift(stored.file_model_hash)
        drift["permission"] = self._check_permission_drift(stored.permission_hash)
        stale_domains = [k for k, v in drift.items() if v.get("stale", False)]
        return {
            "has_description": True,
            "needs_build": False,
            "has_drift": len(stale_domains) > 0,
            "stale_domains": stale_domains,
            "drift": drift,
        }

```

```

def _check_schema_drift(self, stored_hash: str) -> dict:
    signals = []
    stale = False
    try:
        from ..services.system_description_builder import
SystemDescriptionBuilder
        current_hash = SystemDescriptionBuilder._compute_schema_hash()
        if current_hash != stored_hash:
            signals.append(f"schema_hash \u53d8\u5316:
{stored_hash[:12]}...\u2192{current_hash[:12]}...")
            stale = True
        except Exception as e:
            signals.append(f"\u68c0\u6d4b\u5f02\u5e38: {e}")
        return {"stale": stale, "signals": signals}
def _check_file_model_drift(self, stored_hash: str) -> dict:
    signals = []
    stale = False
    try:
        from ..services.system_description_builder import
SystemDescriptionBuilder
        current_hash = SystemDescriptionBuilder._compute_file_model_hash()
        if current_hash != stored_hash:
            signals.append(f"file_model_hash \u53d8\u5316:
{stored_hash[:12]}...\u2192{current_hash[:12]}...")
            stale = True
        except Exception as e:
            signals.append(f"\u68c0\u6d4b\u5f02\u5e38: {e}")
        return {"stale": stale, "signals": signals}
def _check_permission_drift(self, stored_hash: str) -> dict:
    signals = []
    stale = False
    try:
        from ..services.system_description_builder import
SystemDescriptionBuilder
        current_hash = SystemDescriptionBuilder._compute_permission_hash()
        if current_hash != stored_hash:
            signals.append(f"permission_hash \u53d8\u5316:
{stored_hash[:12]}...\u2192{current_hash[:12]}...")
            stale = True
        except Exception as e:
            signals.append(f"\u68c0\u6d4b\u5f02\u5e38: {e}")
        return {"stale": stale, "signals": signals}

```

文件编号: 112 文件路径: emily-core\emily_core\services\system_description_builder.py

```

from __future__ import annotations
import json
import logging
from typing import Optional
from ..infrastructure.database.session import get_session
from ..infrastructure.database.models import (
    Base, PublicFieldRegistry, File, FileCategory,

```

```
)
logger = logging.getLogger("emily.system_description_builder")
_TABLE_DISPLAY_NAMES = {
    "project": "项目表",
    "events": "事件表",
    "tasks": "任务表",
    "meetings": "会议表",
    "files": "文件表",
    "messages": "消息表",
    "users": "用户表",
    "financial": "财务表",
    "project_nodes": "节点表",
    "node_deliverables": "节点成果表",
    "companies": "单位表",
    "conversations": "会话表",
    "permissions": "权限表",
    "permission_grants": "授权表",
    "scheduler_jobs": "调度作业表",
    "scheduler_executions": "调度执行表",
}
_TABLE_DESCRIPTIONS = {
    "project": "项目基本信息",
    "events": "记录项目现场发生的事件",
    "tasks": "跟踪工作任务的执行情况",
    "meetings": "记录会议安排和纪要",
    "files": "项目文件归档",
    "messages": "对话消息记录",
    "users": "用户账户信息",
    "financial": "财务数据",
    "project_nodes": "全景节点树（工作分解结构）",
    "node_deliverables": "节点成果交付物",
    "companies": "参建单位信息",
    "conversations": "IM 会话记录",
    "permissions": "权限规则定义",
    "permission_grants": "跨线授权记录",
    "scheduler_jobs": "系统调度作业",
    "scheduler_executions": "调度执行日志",
}
_VISIBLE_TABLES = {"project", "events", "tasks", "meetings", "files", "financial"}
_KEY_FIELDS = {
    "project": {"name", "code", "address", "status", "lifecycle_stage"},
    "events": {"title", "event_type", "event_date", "created_by", "project_id"},
    "tasks": {"title", "status", "node_id", "assignee_id", "project_id"},
    "meetings": {"title", "meeting_type", "meeting_date", "project_id",
"created_by"},
    "files": {"filename", "file_type", "file_category", "confidentiality",
"project_id", "uploaded_by"},
    "financial": {"title", "amount", "project_id", "category"},
}
_KEY_RELATIONS = [
    {"from": "events.project_id", "to": "projects.id", "description": "每个事件属于一个项目"},
    {"from": "tasks.node_id", "to": "project_nodes.node_id", "description": "每个任务关联一个全景节点"},
]
```

```

    {"from": "files.project_id", "to": "projects.id", "description": "每个文件属于一个项目"},
    {"from": "files.uploaded_by", "to": "users.id", "description": "文件上传者"},
    {"from": "users.company", "to": "companies.id", "description": "用户归属参建单位"},
    {"from": "events.created_by", "to": "users.id", "description": "事件记录人"},
    {"from": "files.file_category", "to": "FileCategory 枚举", "description": "文件按分类体系归档"},
]
class SystemDescriptionBuilder:
    def build(self, *, generated_by: str = "manual", dry_run: bool = False) -> dict:
        database = self._build_database()
        file_cognition = self._build_file()
        permission = self._build_permission()
        content_json = {
            "database": database,
            "file": file_cognition,
            "permission": permission,
        }
        schema_hash = self._compute_schema_hash()
        permission_hash = self._compute_permission_hash()
        file_model_hash = self._compute_file_model_hash()
        content_text = self._format_content_text(content_json)
        token_count = int(len(content_text) / 1.5) if content_text else 0
        domain_versions = {
            "database": 1,
            "file": 1,
            "permission": 1,
        }
        result = {
            "content_json": json.dumps(content_json, ensure_ascii=False),
            "content_text": content_text,
            "domain_versions": json.dumps(domain_versions),
            "schema_hash": schema_hash,
            "permission_hash": permission_hash,
            "file_model_hash": file_model_hash,
            "token_count": token_count,
            "generated_by": generated_by,
            "status": "preview" if dry_run else "built",
        }
        if not dry_run:
            from ..repositories.system_description_repo import SystemDescriptionRepo
            existing = SystemDescriptionRepo.get_latest()
            if existing:
                new_version = existing.version + 1
                old_dv = json.loads(existing.domain_versions or "{}")
                merged_dv = {**old_dv}
                for k in domain_versions:
                    if k in merged_dv:
                        merged_dv[k] += 1
                    else:
                        merged_dv[k] = 1

```

```

        SystemDescriptionRepo.update_content(
            content_json=result["content_json"],
            content_text=content_text,
            domain_versions=json.dumps(merged_dv),
            version=new_version,
            schema_hash=schema_hash,
            permission_hash=permission_hash,
            file_model_hash=file_model_hash,
            token_count=token_count,
            generated_by=generated_by,
        )
        result["version"] = new_version
    else:
        desc = SystemDescriptionRepo.create(
            content_json=result["content_json"],
            content_text=content_text,
            domain_versions=result["domain_versions"],
            schema_hash=schema_hash,
            permission_hash=permission_hash,
            file_model_hash=file_model_hash,
            token_count=token_count,
            generated_by=generated_by,
        )
        result["version"] = 1
    return result

def _build_database(self) -> dict:
    try:
        tables_info = self._reflect_tables()
        visible_tables = tables_info["visible_tables"]
        all_tables = tables_info["all_tables"]
        key_relations = _KEY_RELATIONS
        return {
            "total_tables": all_tables["total"],
            "visible_tables": visible_tables,
            "hidden_table_count": all_tables["total"] - len(visible_tables),
            "key_relations": key_relations,
        }
    except Exception as e:
        logger.error("_build_database failed: %s", e)
        return {"total_tables": 0, "visible_tables": [], "hidden_table_count":
0, "key_relations": [], "error": str(e)}

def _reflect_tables(self) -> dict:
    all_table_names = sorted(Base.metadata.tables.keys())
    total = len(all_table_names)
    visible_tables = []
    for tbl_name in _VISIBLE_TABLES:
        if tbl_name not in Base.metadata.tables:
            continue
        table = Base.metadata.tables[tbl_name]
        display_name = _TABLE_DISPLAY_NAMES.get(tbl_name, tbl_name)
        description = _TABLE_DESCRIPTIONS.get(tbl_name, "")
        key_fields = self._extract_key_fields(tbl_name, table)
        field_descriptions = self._load_field_descriptions(tbl_name)
        for field in key_fields:

```

```

        if not field.get("description") and field["name"] in
field_descriptions:
            field["description"] = field_descriptions[field["name"]]
        visible_tables.append({
            "table_name": tbl_name,
            "display_name": display_name,
            "description": description,
            "key_fields": key_fields,
        })
    return {
        "visible_tables": visible_tables,
        "all_tables": {"total": total, "names": all_table_names},
    }
def _extract_key_fields(self, tbl_name: str, table) -> list[dict]:
    selected = _KEY_FIELDS.get(tbl_name, set())
    fields = []
    for col in table.columns:
        if selected and col.name not in selected:
            continue
        field = {
            "name": col.name,
            "type": self._simplify_type(col.type),
            "required": not col.nullable if col.nullable is not None else
False,
            "description": col.comment or "",
        }
        for fk in col.foreign_keys:
            target = str(fk.target_fullname)
            field["type"] = f"FK\u2192{target.split('.')[0]}"
            break
        fields.append(field)
    return fields
@staticmethod
def _simplify_type(col_type) -> str:
    type_name = type(col_type).__name__
    mapping = {
        "String": "\u6587\u672c",
        "Text": "\u957f\u6587\u672c",
        "Integer": "\u6574\u6570",
        "BigInteger": "\u5927\u6574\u6570",
        "Float": "\u6d6e\u70b9\u6570",
        "Boolean": "\u5e03\u5c14",
        "DateTime": "\u65e5\u671f\u65f6\u95f4",
        "Date": "\u65e5\u671f",
        "Numeric": "\u6570\u503c",
        "JSON": "JSON",
        "Enum": "\u679a\u4e3e",
    }
    return mapping.get(type_name, type_name)
@staticmethod
def _load_field_descriptions(model_name: str) -> dict[str, str]:
    try:
        with get_session() as session:
            records = session.query(PublicFieldRegistry).filter(

```

```

        PublicFieldRegistry.model_name == model_name,
        PublicFieldRegistry.is_deleted == False,
    ).all()
    return {r.field_name: r.description for r in records if
r.description}
    except Exception as e:
        logger.debug("_load_field_descriptions failed for %s: %s", model_name,
e)

    return {}
def _build_file(self) -> dict:
    try:
        categories = []
        for cat_value in FileCategory.ALL:
            categories.append({
                "name": FileCategory.DISPLAY_NAMES.get(cat_value, cat_value),
                "value": cat_value,
            })
        if "files" in Base.metadata.tables:
            file_table = Base.metadata.tables["files"]
            file_key_fields = ["filename", "file_type", "file_category",
"confidentiality", "project_id", "uploaded_by"]
            key_fields = []
            for col in file_table.columns:
                if col.name not in file_key_fields:
                    continue
                field = {
                    "name": col.name,
                    "type": self._simplify_type(col.type),
                    "required": not col.nullable if col.nullable is not None
else False,

                    "description": col.comment or "",
                }
                for fk in col.foreign_keys:
                    target = str(fk.target_fullname)
                    field["type"] = f"FK\u2192{target.split('.')[0]}"
                    break
                key_fields.append(field)
        else:
            key_fields = []
        access_rules = [
            "confidentiality=0 公开文件所有用户可见",
            "confidentiality\u22651 内部/机密/绝密文件按权限级别控制",
            "上传人始终可查看自己上传的文件",
            "L5+ 管理员可查看所有文件",
        ]
        return {
            "categories": categories,
            "key_fields": key_fields,
            "access_rules": access_rules,
        }
    except Exception as e:
        logger.error("_build_file failed: %s", e)
        return {"categories": [], "key_fields": [], "access_rules": [],
"error": str(e)}

```



```

def _build_permission(self) -> dict:
    try:
        from ..permission.level import (
            PermissionLevel, INHERITANCE_CHAIN, LEVEL_NAME,
        )
        levels = []
        line_map = {
            1: "\u516c\u5171",
            2: "\u53c2\u5efa\u7ebf",
            3: "\u53c2\u5efa\u7ebf",
            4: "\u5efa\u8bbe\u7ebf",
            5: "\u5efa\u8bbe\u7ebf",
            6: "\u5efa\u8bbe\u7ebf",
        }
        capability_map = {
            1:
                "\u57fa\u672c\u67e5\u8be2\u3001\u516c\u5f00\u4fe1\u606f\u67e5\u770b",
            2:
                "\u5f55\u5165\u81ea\u5df1\u8d1f\u8d23\u7684\u4e8b\u4ef6/\u4efb\u52a1\u3001\u67e5\u770b\u9879\u76ee\u516c\u5f00\u4fe1\u606f",
            3:
                "\u7ba1\u7406\u53c2\u5efa\u5355\u4f4d\u5185\u4eba\u5458\u3001\u5ba1\u6838\u53c2\u5efa\u65b9\u63d0\u4ea4",
            4:
                "\u7ba1\u7406\u5168\u666f\u8282\u70b9\u3001\u5ba1\u6279\u8de8\u5355\u4f4d\u4e8b\u9879",
            5:
                "\u5168\u5c40\u7ba1\u7406\u3001\u6743\u9650\u6388\u4e88\u3001\u6570\u636e\u8131\u654f\u89c4\u5219\u914d\u7f6e",
            6:
                "\u7cfb\u7edf\u914d\u7f6e\u3001\u6743\u9650\u89c4\u5219\u5b9a\u4e49\u3001\u5f3a\u5236\u64a4\u9500",
        }
        for level_value in sorted(INHERITANCE_CHAIN.keys()):
            levels.append({
                "level": level_value,
                "name": LEVEL_NAME.get(level_value, "\u672a\u77e5"),
                "line": line_map.get(level_value, ""),
                "capabilities": capability_map.get(level_value, ""),
            })
        inheritance = {
            "\u53c2\u5efa\u7ebf": sorted(list(INHERITANCE_CHAIN.get(3,
frozenset()))),
            "\u5efa\u8bbe\u7ebf": sorted(list(INHERITANCE_CHAIN.get(6,
frozenset()))),
            "note": "L4 \u4e0d\u7ee7\u627f L2/L3\u5424\u7ebf\u5728 L1
\u540e\u5206\u53c9",
        }
        cross_line_mechanism = "\u8de8\u7ebf\u8bbf\u95ee\u901a\u8fc7
PermissionGrant\u5424\u65f6/\u6c38\u4e45\u6388\u6743\u5b9e\u73b0"
        return {
            "levels": levels,
            "inheritance": inheritance,
            "cross_line_mechanism": cross_line_mechanism,
        }

```

```

    }
    except Exception as e:
        logger.error("_build_permission failed: %s", e)
        return {"levels": [], "inheritance": {}, "cross_line_mechanism": "",
"error": str(e)}
    @staticmethod
    def _compute_schema_hash() -> str:
        import hashlib
        parts = []
        for tbl_name in sorted(Base.metadata.tables.keys()):
            table = Base.metadata.tables[tbl_name]
            col_parts = []
            for col in table.columns:
                col_parts.append(f"{col.name}:{type(col.type).__name__}:
{col.nullable}")
            parts.append(f"{tbl_name}({'.'.join(sorted(col_parts))}")
            canonical = "|".join(parts)
            return hashlib.sha256(canonical.encode("utf-8")).hexdigest()
    @staticmethod
    def _compute_permission_hash() -> str:
        import hashlib
        from ..permission.level import PermissionLevel, INHERITANCE_CHAIN,
LEVEL_NAME
        parts = []
        for level in sorted(PermissionLevel, key=lambda x: x.value):
            parts.append(f"{level.name}={level.value}")
        for k in sorted(INHERITANCE_CHAIN.keys()):
            parts.append(f"chain_{k}={sorted(INHERITANCE_CHAIN[k])}")
        for k in sorted(LEVEL_NAME.keys()):
            parts.append(f"name_{k}={LEVEL_NAME[k]}")
        canonical = "|".join(parts)
        return hashlib.sha256(canonical.encode("utf-8")).hexdigest()
    @staticmethod
    def _compute_file_model_hash() -> str:
        import hashlib
        parts = []
        for cat in FileCategory.ALL:
            parts.append(f"cat_{cat}={FileCategory.DISPLAY_NAMES.get(cat, cat)}")
        if "files" in Base.metadata.tables:
            file_table = Base.metadata.tables["files"]
            for col in file_table.columns:
                parts.append(f"file_{col.name}:{type(col.type).__name__}")
            canonical = "|".join(sorted(parts))
            return hashlib.sha256(canonical.encode("utf-8")).hexdigest()
    def _format_content_text(self, content_json: dict) -> str:
        lines = []
        db = content_json.get("database", {})
        visible = db.get("visible_tables", [])
        if visible:
            lines.append("\U0001f4ca 数据库资源（你有权访问的表）")
            for tbl in visible:
                lines.append(f"\u00b7 {tbl['display_name']}:
{tbl.get('description', '')}")
                fields = tbl.get("key_fields", [])

```

```

        if fields:
            field_strs = []
            for f in fields[:5]:
                req_mark = ",\u5fc5\u586b" if f.get("required") else ""
                type_str = f.get("type", "")
                desc = f.get("description", "")
                if desc:
                    field_strs.append(f"{f['name']}({desc}{req_mark})")
                else:
                    field_strs.append(f"{f['name']}({type_str}
{req_mark})")
            lines.append(f" \u6838\u5fc3\u5b57\u6bb5\u5411\u5217\u8868:
'.'.join(field_strs)}")
            hidden_count = db.get("hidden_table_count", 0)
            total = db.get("total_tables", 0)
            lines.append(f"\u5411\u5217\u8868\u603b\u8ba1: {hidden_count}
\u5411\u5217\u8868\u603b\u8ba1: {total} \u5411\u5217\u8868")
            relations = db.get("key_relations", [])
            if relations:
                rel_strs = [f"{r['from']} \u2192 {r['to']}" for r in
relations[:5]]
                lines.append("\u0001f517 表间关系")
                lines.append(" / ".join(rel_strs))
            fc = content_json.get("file", {})
            categories = fc.get("categories", [])
            if categories:
                lines.append("")
                lines.append("\u0001f4c1 文件分类体系")
                cat_strs = [f"{c['name']}" for c in categories]
                lines.append(" / ".join(cat_strs))
            access_rules = fc.get("access_rules", [])
            if access_rules:
                lines.append("\u6587\u4ef6\u8bbf\u95ee\u89c4\u5219\u5411\u5217\u8868:
'\u5411\u5217\u8868'.join(access_rules[:3]))
            perm = content_json.get("permission", {})
            levels = perm.get("levels", [])
            if levels:
                lines.append("")
                lines.append("\u0001f510 权限分级体系")
                participant_levels = [l for l in levels if l["line"] ==
"\u53c2\u53c2\u7b7e"]
                if participant_levels:
                    pl_str = " \u2192 ".join(f"L{1['level']} {1['name']}" for l in
participant_levels)
                    lines.append(f"\u53c2\u53c2\u7b7e: L1 \u8bbf\u5ba2 \u2192
{pl_str}")
                construction_levels = [l for l in levels if l["line"] ==
"\u5efa\u8bbe\u7b7e"]
                if construction_levels:
                    cl_str = " \u2192 ".join(f"L{1['level']} {1['name']}" for l in
construction_levels)
                    lines.append(f"\u5efa\u8bbe\u7b7e: L1 \u8bbf\u5ba2 \u2192
{cl_str}")
            inheritance = perm.get("inheritance", {})

```

```
        note = inheritance.get("note", "") if isinstance(inheritance, dict)
    else ""
        if note:
            lines.append(f"\u26a0\ufe0f
{note}\uff1b\u8de8\u7ebf\u8bbf\u95ee\u9700\u4e34\u65f6/\u6c38\u4e45\u6388\u6743")
        return "\n".join(lines)
```

文件编号：113 文件路径：emily-core\emily_core\services\system_description_service.py

```

from __future__ import annotations
import asyncio
import logging
from ..services.schema_drift_detector import SchemaDriftDetector
from ..services.system_description_builder import SystemDescriptionBuilder
logger = logging.getLogger("emily.system_description_service")
class SystemDescriptionService:
    def __init__(self, llm_client=None):
        self._builder = SystemDescriptionBuilder()
        self._detector = SchemaDriftDetector()
        self._llm = llm_client
    async def check_and_update(self, *, dry_run: bool = False) -> dict:
        drift_result = self._detector.detect()
        if not drift_result.get("has_description"):
            result = await asyncio.to_thread(
                self._builder.build, generated_by="startup", dry_run=dry_run
            )
            result["status"] = "built" if not dry_run else "preview"
            return result
        if not drift_result.get("has_drift"):
            return {
                "status": "no_drift",
                "message":
"\u7cfb\u7edf\u63cf\u8ff0\u4e0e\u5f53\u524d\u4ee3\u7801\u7ed3\u6784\u4e00\u81f4\u7f0c\u65e0\u9700\u66f4\u65b0",
            }
        stale_domains = drift_result.get("stale_domains", [])
        logger.info("SystemDescription drift detected in domains: %s",
stale_domains)
        result = await asyncio.to_thread(
            self._builder.build, generated_by="scheduler", dry_run=dry_run
        )
        result["updated_domains"] = stale_domains
        result["drift_details"] = drift_result.get("drift", {})
        result["status"] = "updated" if not dry_run else "preview"
        return result
    async def force_rebuild(self, *, dry_run: bool = False) -> dict:
        result = await asyncio.to_thread(
            self._builder.build, generated_by="manual", dry_run=dry_run
        )
        result["status"] = "rebuilt" if not dry_run else "preview"
        return result

```

文件编号: 114 文件路径: emily-core\emily_core\services\task_service.py

```

import logging
from typing import Optional
from ..repositories.task_repo import TaskRepository
from ..adapters.standard.command import TaskCommand
from ..infrastructure.database.models import Task

```

```
logger = logging.getLogger("emily.service.task")
class TaskService:
    def __init__(self):
        self.repo = TaskRepository()
    def create_task(self, cmd: TaskCommand) -> Task:
        task_no = self.repo.generate_task_no()
        task = self.repo.create(
            task_no=task_no,
            title=cmd.title,
            project_id=cmd.project_id or None,
            source_message_id=cmd.source_message_id or None,
            description=cmd.description or None,
            owner_text=cmd.assignee_text or None,
            due_date=cmd.due_date,
            due_text=cmd.due_text or None,
            created_by=cmd.creator_id or None,
            status="todo",
        )
        logger.info("Task %s created: %s", task_no, cmd.title)
        return task
```

文件编号: 115 文件路径: emily-core\emily_core\services\user_binding_service.py

```
import logging
from typing import Tuple
from ..repositories.user_repo import UserRepository
from ..infrastructure.database.models import User, UserImBinding
logger = logging.getLogger("emily.service.user_binding")
class UserNotAllowedError(Exception):
    pass
class UserBindingService:
    def __init__(self, auto_create: bool = True, whitelist: list | None = None):
        self.repo = UserRepository()
        self._auto_create = auto_create
        self._whitelist = whitelist or []
    @staticmethod
    def _looks_like_uuid(value: str) -> bool:
        if not value:
            return False
        parts = value.split("-")
        if len(parts) == 5 and all(p.isalnum() for p in parts):
            return True
        if len(value) == 32 and value.isalnum():
            return True
        return False
    def get_or_create_user(
        self,
        im_platform: str,
        im_user_id: str,
        im_display_name: str | None = None,
    ) -> Tuple[User, bool]:
        existing = self.repo.get_by_im(im_platform, im_user_id)
```

```

if existing:
    logger.debug(
        "User already bound: %s -> %s (%s)",
        im_user_id, existing.id, existing.username,
    )
    return existing, False
if self._looks_like_uuid(im_user_id):
    direct_user = self.repo.get_by_id(im_user_id)
    if direct_user:
        logger.debug(
            "User resolved by UUID: %s -> %s (%s)",
            im_user_id, direct_user.id, direct_user.username,
        )
        return direct_user, False
if not self._allow_auto_create(im_platform, im_user_id):
    logger.warning(
        "User auto-create denied: platform=%s im_user_id=%s "
        "(auto_create=%s, whitelist=%s)",
        im_platform, im_user_id,
        self._auto_create, bool(self._whitelist),
    )
    raise UserNotAllowedError(
        f"未授权的发送者 {im_user_id}, 请联系管理员注册"
    )
user, _ = self.repo.create_user_and_bind(
    im_platform=im_platform,
    im_user_id=im_user_id,
    im_display_name=im_display_name,
)
logger.info(
    "Auto-created user: %s (%s) bound to %s/%s",
    user.id, user.username, im_platform, im_user_id,
)
return user, True
def _allow_auto_create(self, im_platform: str, im_user_id: str) -> bool:
    if self._auto_create:
        return True
    return im_user_id in self._whitelist

```

文件编号: 116 文件路径: emily-core\emily_core\services\user_memory_service.py

```

import os
import re
import logging
from datetime import datetime, timezone, timedelta
from typing import Optional
logger = logging.getLogger("emily.service.memory")
class UserMemoryService:
    def __init__(
        self,
        memory_dir: str = "",
        enabled: bool = True,

```

```

        max_entries: int = 50,
    ):
        self.enabled = enabled
        self.max_entries = max_entries
        self.memory_dir = memory_dir or ""
        if not self.memory_dir:
            self.memory_dir = os.path.join(
                os.path.dirname(os.path.dirname(os.path.dirname(
                    os.path.dirname(os.path.abspath(__file__)))
                )),
                "memory",
            )
        if self.enabled:
            os.makedirs(self.memory_dir, exist_ok=True)
    def _sanitize_filename(self, user_name: str) -> str:
        safe = re.sub(r'[\\/:*?"<>|]', '_', user_name)
        safe = safe.strip().replace(' ', '_')
        if not safe:
            safe = "unknown_user"
        return f"{safe}-长期记忆.md"
    def _get_filepath(self, user_name: str) -> str:
        filename = self._sanitize_filename(user_name)
        return os.path.join(self.memory_dir, filename)
    def _ensure_file(self, filepath: str, user_name: str) -> None:
        if not os.path.exists(filepath):
            try:
                with open(filepath, "w", encoding="utf-8") as f:
                    f.write(f"> 此文件由 Emy 自动维护，记录用户的长期工作要求。\\n")
                    f.write("> 每次新对话时作为上下文加载。\\n\\n")
                    f.write("---\\n\\n")
                logger.info("Created memory file for user: %s", user_name)
            except OSError as e:
                logger.error("Failed to create memory file %s: %s", filepath, e)
    def save_memory(
        self,
        user_name: str,
        content: str,
        title: str = "",
    ) -> Optional[str]:
        if not self.enabled or not user_name or not content:
            return None
        filepath = self._get_filepath(user_name)
        self._ensure_file(filepath, user_name)
        date_str = ""
        try:
            local = datetime.now(timezone.utc) + timedelta(hours=8)
            date_str = local.strftime("%Y-%m-%d %H:%M")
        except Exception:
            date_str = datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M")
        if not title:
            title = content[:30] + ("..." if len(content) > 30 else "")
        entry = f"
try:

```



```
        with open(filepath, "a", encoding="utf-8") as f:
            f.write(entry)
        self._trim_if_needed(filepath)
        logger.info(
            "Memory saved for user %s: %s", user_name, title,
        )
        return title
    except OSError as e:
        logger.error("Failed to save memory for %s: %s", user_name, e)
        return None

def load_memory(self, user_name: str) -> str:
    if not self.enabled or not user_name:
        return ""
    filepath = self._get_filepath(user_name)
    if not os.path.exists(filepath):
        return ""
    try:
        with open(filepath, "r", encoding="utf-8") as f:
            content = f.read()
        logger.info(
            "Memory loaded for user %s: %d chars",
            user_name, len(content),
        )
        return content
    except OSError as e:
        logger.error("Failed to load memory for %s: %s", user_name, e)
        return ""

def load_memory_context(self, user_name: str) -> str:
    full = self.load_memory(user_name)
    if not full:
        return ""
    lines = full.split("\n")
    entries = []
    current_title = ""
    current_content = []
    for line in lines:
        if line.startswith("    "):
            if current_title:
                content_text = " ".join(current_content).strip()
                entries.append(f"- {current_title}: {content_text}")
                current_title = line[3:].strip()
                current_content = []
            elif current_title and line.strip():
                current_content.append(line.strip())
    if current_title:
        content_text = " ".join(current_content).strip()
        entries.append(f"- {current_title}: {content_text}")
    if not entries:
        return ""
    return "用户长期工作要求: \n" + "\n".join(entries)

def _trim_if_needed(self, filepath: str) -> None:
    try:
        with open(filepath, "r", encoding="utf-8") as f:
            content = f.read()
```

```

        entries = re.findall(r'^
        if len(entries) <= self.max_entries:
            return
        parts = re.split(r'(?=^
        header = ""
        entry_parts = []
        for part in parts:
            if part.startswith("
                entry_parts.append(part)
            else:
                header = part
        kept = entry_parts[-self.max_entries:]
        new_content = header + "".join(kept)
        with open(filepath, "w", encoding="utf-8") as f:
            f.write(new_content)
        logger.info(
            "Memory trimmed for %s: %d -> %d entries",
            filepath, len(entries), len(kept),
        )
    except OSError as e:
        logger.warning("Failed to trim memory file %s: %s", filepath, e)
def list_users(self) -> list[str]:
    if not self.enabled:
        return []
    try:
        users = []
        for filename in os.listdir(self.memory_dir):
            if filename.endswith("-长期记忆.md"):
                name = filename[:-len("-长期记忆.md")]
                name = name.replace("_", " ")
                users.append(name)
        return users
    except OSError as e:
        logger.error("Failed to list memory users: %s", e)
    return []

```

文件编号：117 文件路径：emily-core\emily_core\services\world_book_builder.py

```

from __future__ import annotations
import json
import logging
from datetime import datetime, timezone, timedelta
from typing import Optional
from ..repositories.world_book_repo import ProjectWorldBookRepo
from ..infrastructure.database.session import get_session
from ..infrastructure.database.models import (
    Project, User, CompanyInfo, ProjectNode, NodeDependency, Event,
)
logger = logging.getLogger("emily.world_book_builder")
BEIJING_TZ = timezone(timedelta(hours=8))
LIFECYCLE_LABELS = {0: "立项", 1: "规划设计", 2: "工程施工", 3: "交付结算"}
class ProjectWorldBookBuilder:

```

```

def build(self, project_id: str, *, generated_by: str = "manual", dry_run:
bool = False) -> dict:
    ontology = self._build_ontology(project_id)
    personnel = self._build_personnel(project_id)
    structure = self._build_structure(project_id)
    temporal = self._build_temporal(project_id)
    relation = self._build_relation(project_id)
    knowledge = self._build_knowledge(project_id)
    introspection = self._build_introspection(project_id)
    content_json = {
        "ontology": ontology,
        "personnel": personnel,
        "structure": structure,
        "temporal": temporal,
        "relation": relation,
        "knowledge": knowledge,
        "introspection": introspection,
    }
    content_text = self._format_content_text(content_json)
    token_count = int(len(content_text) / 1.5) if content_text else 0
    init_tier = introspection.get("initialization_tier", 0)
    init_status = introspection.get("initialization_status", {})
    is_activated = introspection.get("is_activated", False)
    layer_versions = {k: 1 for k in content_json.keys()}
    result = {
        "project_id": project_id,
        "content_json": json.dumps(content_json, ensure_ascii=False),
        "content_text": content_text,
        "layer_versions": json.dumps(layer_versions),
        "initialization_tier": init_tier,
        "initialization_status": json.dumps(init_status, ensure_ascii=False),
        "is_activated": is_activated,
        "token_count": token_count,
        "generated_by": generated_by,
        "status": "preview" if dry_run else "built",
    }
    if not dry_run:
        existing = ProjectWorldBookRepo.get_by_project(project_id)
        if existing:
            new_version = existing.version + 1
            old_lv = json.loads(existing.layer_versions or "{}")
            merged_lv = {**old_lv}
            for k in layer_versions:
                if k in merged_lv:
                    merged_lv[k] += 1
                else:
                    merged_lv[k] = 1
            ProjectWorldBookRepo.update_content(
                project_id=project_id,
                content_json=result["content_json"],
                content_text=content_text,
                layer_versions=json.dumps(merged_lv),
                version=new_version,
                initialization_tier=init_tier,

```

```

        initialization_status=result["initialization_status"],
        is_activated=is_activated,
        token_count=token_count,
        generated_by=generated_by,
    )
    result["version"] = new_version
else:
    wb = ProjectWorldBookRepo.create(
        project_id=project_id,
        content_json=result["content_json"],
        content_text=content_text,
        layer_versions=result["layer_versions"],
        initialization_tier=init_tier,
        initialization_status=result["initialization_status"],
        is_activated=is_activated,
        token_count=token_count,
        generated_by=generated_by,
    )
    result["version"] = 1
return result
def _build_ontology(self, project_id: str) -> dict:
    try:
        with get_session() as session:
            project = session.query(Project).filter(Project.id == project_id,
Project.is_deleted == False).first()
            if project is None:
                return {"name": "", "code": "", "error": "项目不存在"}
            companies = []
            users = session.query(User).filter(User.project_id == project_id,
User.is_deleted == False).all()
            company_ids = list(set(u.company for u in users if u.company))
            if company_ids:
                company_records =
session.query(CompanyInfo).filter(CompanyInfo.id.in_(company_ids)).all()
                for c in company_records:
                    companies.append({
                        "name": c.company_name,
                        "type": c.type or "",
                        "role": c.business_desc or "",
                    })
            stage = project.lifecycle_stage or 0
            return {
                "name": project.name or "",
                "code": project.code or "",
                "address": project.address or "",
                "lifecycle_stage": stage,
                "lifecycle_stage_label": LIFECYCLE_LABELS.get(stage, "未知"),
                "organizations": companies,
                "project_summary": f"{project.name or '未命名项目'}, 当前处于
{LIFECYCLE_LABELS.get(stage, '未知')}阶段",
            }
    except Exception as e:
        logger.error("_build_ontology failed: %s", e)
        return {"name": "", "code": "", "error": str(e)}

```

```

def _build_personnel(self, project_id: str) -> dict:
    try:
        with get_session() as session:
            users = session.query(User).filter(
                User.project_id == project_id,
                User.is_deleted == False,
                User.status == "active",
            ).all()
            key_personnel = []
            department_leads = []
            for u in users:
                positions = []
                try:
                    positions = json.loads(u.position or "[]")
                except (json.JSONDecodeError, TypeError):
                    positions = []
                company_name = ""
                if u.company:
                    company = session.query(CompanyInfo).filter(CompanyInfo.id
== u.company).first()
                    if company:
                        company_name = company.company_name
                person = {
                    "name": u.username or "",
                    "role": ", ".join(positions) if positions else "",
                    "company": company_name,
                    "level": u.level or 1,
                    "is_admin": u.is_admin or False,
                }
                if u.is_admin or any(p in ["项目经理", "总监理工程师", "总监理"]
for p in positions):
                    key_personnel.append(person)
                elif positions:
                    for p in positions:
                        if p not in ["项目经理", "总监理工程师", "总监理"]:
                            department_leads.append({"department": p, "name":
u.username or ""})
            return {
                "key_personnel": key_personnel,
                "department_leads": department_leads,
                "total_users": len(users),
            }
        except Exception as e:
            logger.error("_build_personnel failed: %s", e)
            return {"key_personnel": [], "department_leads": [], "error": str(e)}

def _build_structure(self, project_id: str) -> dict:
    try:
        with get_session() as session:
            nodes = session.query(ProjectNode).filter(
                ProjectNode.project_id == project_id,
                ProjectNode.is_discarded == False,
            ).all()
            total = len(nodes)
            completed = sum(1 for n in nodes if n.status == "COMPLETED")

```

```

        in_progress = sum(1 for n in nodes if n.status == "IN_PROGRESS")
        conditions_not_met = sum(1 for n in nodes if n.status ==
"CONDITIONS_NOT_MET")
        not_activated = sum(1 for n in nodes if n.status ==
"NOT_ACTIVATED")
        now_beijing = datetime.now(BEIJING_TZ)
        overdue = 0
        for n in nodes:
            if n.status != "COMPLETED" and n.deadline:
                try:
                    dl = datetime.fromisoformat(n.deadline)
                    if dl.tzinfo is None:
                        dl = dl.replace(tzinfo=BEIJING_TZ)
                    if dl < now_beijing:
                        overdue += 1
                except (ValueError, TypeError):
                    pass
        milestones = []
        for n in nodes:
            if getattr(n, 'node_type', '') == 'MILESTONE':
                milestones.append({
                    "name": n.node_name,
                    "status": n.status,
                    "progress": n.progress or "0.00",
                    "deadline": n.deadline or "",
                })
        total_progress = 0.0
        if total > 0:
            total_progress = sum(float(n.progress or "0") for n in nodes)
/ total

        return {
            "total_nodes": total,
            "completed": completed,
            "in_progress": in_progress,
            "conditions_not_met": conditions_not_met,
            "not_activated": not_activated,
            "overdue": overdue,
            "overall_progress": f"{total_progress:.1f}%",
            "milestones": milestones[:10],
        }
    except Exception as e:
        logger.error("_build_structure failed: %s", e)
        return {"total_nodes": 0, "error": str(e)}
def _build_temporal(self, project_id: str) -> dict:
    try:
        now_beijing = datetime.now(BEIJING_TZ)
        week_ago = (now_beijing - timedelta(days=7)).strftime("%Y-%m-%d")
        week_later = (now_beijing + timedelta(days=7)).strftime("%Y-%m-%d")
        with get_session() as session:
            events = session.query(Event).filter(
                Event.project_id == project_id,
            ).order_by(Event.created_at.desc()).limit(5).all()
            recent_events = []
            for e in events:

```

```

        recent_events.append({
            "date": (e.event_date or e.created_at or "")[:10],
            "summary": e.title or "",
            "type": e.event_type or "",
        })
    nodes = session.query(ProjectNode).filter(
        ProjectNode.project_id == project_id,
        ProjectNode.is_discarded == False,
        ProjectNode.status != "COMPLETED",
    ).all()
    upcoming_deadlines = []
    overdue_items = []
    for n in nodes:
        if not n.deadline:
            continue
        try:
            dl = datetime.fromisoformat(n.deadline)
            if dl.tzinfo is None:
                dl = dl.replace(tzinfo=BEIJING_TZ)
            dl_str = dl.strftime("%Y-%m-%d")
            if dl < now_beijing:
                overdue_items.append({
                    "name": n.node_name,
                    "deadline": dl_str,
                    "status": n.status,
                })
            elif dl_str <= week_later:
                upcoming_deadlines.append({
                    "name": n.node_name,
                    "deadline": dl_str,
                    "status": n.status,
                })
        except (ValueError, TypeError):
            pass
    return {
        "recent_events": recent_events,
        "upcoming_deadlines": upcoming_deadlines[:5],
        "overdue_items": overdue_items[:5],
    }
except Exception as e:
    logger.error("_build_temporal failed: %s", e)
    return {"recent_events": [], "upcoming_deadlines": [],
"overdue_items": [], "error": str(e)}
def _build_relation(self, project_id: str) -> dict:
    try:
        with get_session() as session:
            deps = session.query(NodeDependency).all()
            project_node_ids = set(
                n.node_id for n in session.query(ProjectNode).filter(
                    ProjectNode.project_id == project_id,
                    ProjectNode.is_discarded == False,
                ).all()
            )
            key_dependencies = []

```

```

        blocked_nodes = []
        for dep in deps:
            if dep.node_id not in project_node_ids:
                continue
            upstream_node = session.query(ProjectNode).filter(
                ProjectNode.node_id == dep.depends_on_node_id
            ).first()
            downstream_node = session.query(ProjectNode).filter(
                ProjectNode.node_id == dep.node_id
            ).first()
            if upstream_node and downstream_node:
                key_dependencies.append({
                    "upstream": upstream_node.node_name,
                    "downstream": downstream_node.node_name,
                    "deliverable": dep.depends_on_deliverable_id,
                })
                if upstream_node.status != "COMPLETED":
                    blocked_nodes.append({
                        "node": downstream_node.node_name,
                        "blocked_by": f"{upstream_node.node_name}未完成",
                        "impact": "",
                    })
        return {
            "key_dependencies": key_dependencies[:10],
            "blocked_nodes": blocked_nodes[:5],
        }
    except Exception as e:
        logger.error("_build_relation failed: %s", e)
        return {"key_dependencies": [], "blocked_nodes": [], "error": str(e)}

def _build_knowledge(self, project_id: str) -> dict:
    try:
        from ..skill.registry import SkillRegistry
        sop_ids = []
        try:
            skill_dir = "/app/skills"
            if not __import__('pathlib').Path(skill_dir).exists():
                skill_dir = ""
            if not skill_dir:
                from pathlib import Path as _P
                dev_dir = str(_P(__file__).resolve().parents[2] / "emily-data"
                               / "skills")

                if _P(dev_dir).exists():
                    skill_dir = dev_dir
            if skill_dir:
                reg = SkillRegistry(skill_directory=skill_dir)
                reg.load()
                sop_ids = reg.list_sop_ids()
        except Exception:
            pass
        rag_available = False
        rag_collections = []
        return {
            "sop_count": len(sop_ids),
            "sop_ids": sop_ids[:15],
        }

```



```

        "rag_available": rag_available,
        "rag_collections": rag_collections,
        "coverage_gaps": [],
    }
    except Exception as e:
        logger.error("_build_knowledge failed: %s", e)
        return {"sop_count": 0, "sop_ids": [], "rag_available": False,
"rag_collections": [], "error": str(e)}
    def _build_introspection(self, project_id: str) -> dict:
        try:
            with get_session() as session:
                project = session.query(Project).filter(Project.id == project_id,
Project.is_deleted == False).first()
                if project is None:
                    return {
                        "initialization_tier": 0,
                        "initialization_status": {},
                        "is_activated": False,
                        "missing_items": ["项目不存在"],
                    }
                init_status = {}
                init_status["T1_project_name"] = bool(project.name and
project.name != "未命名项目")
                init_status["T1_project_code"] = bool(project.code)
                init_status["T1_project_address"] = bool(project.address)
                init_status["T1_lifecycle_stage"] = (project.lifecycle_stage or 0)
                != 0

                admins = session.query(User).filter(
                    User.project_id == project_id,
                    User.is_deleted == False,
                    User.is_admin == True,
                ).all()
                init_status["T1_admin_user"] = len(admins) > 0
                init_status["T1_admin_email"] = any(u.email for u in admins if
u.email)

                t1_items = [k for k, v in init_status.items() if
k.startswith("T1_") and v]
                t1_total = sum(1 for k in init_status if k.startswith("T1_"))
                t1_done = len(t1_items)
                tier = 0
                if t1_done >= t1_total:
                    tier = 1
                missing = [k for k, v in init_status.items() if not v]
                return {
                    "initialization_tier": tier,
                    "initialization_status": init_status,
                    "is_activated": tier >= 3,
                    "missing_items": missing,
                }
        except Exception as e:
            logger.error("_build_introspection failed: %s", e)
            return {
                "initialization_tier": 0,
                "initialization_status": {},

```

```

        "is_activated": False,
        "missing_items": [str(e)],
    }
def _format_content_text(self, content_json: dict) -> str:
    lines = []
    o = content_json.get("ontology", {})
    if o.get("name"):
        code = f"({o['code']})" if o.get("code") else ""
        lines.append(f"📄 项目: {o['name']}{code}")
        addr = o.get("address", "")
        stage = o.get("lifecycle_stage_label", "")
        if addr or stage:
            lines.append(f"📍 {addr} | 阶段: {stage}")
        orgs = o.get("organizations", [])
        if orgs:
            org_str = " / ".join(f"{c['name']}{(c['type'])}" for c in
orgs[:4])
            lines.append(f"🏢 {org_str}")
        p = content_json.get("personnel", {})
        kp = p.get("key_personnel", [])
        if kp:
            ppl_str = " / ".join(f"{u['name']}{(u['role'])}" for u in kp[:4])
            lines.append(f"👤 {ppl_str}")
        s = content_json.get("structure", {})
        if s.get("total_nodes", 0) > 0:
            lines.append(
                f"📊 {s['total_nodes']}节点: {s.get('completed', 0)}完成 / "
                f"{s.get('in_progress', 0)}进行中 / {s.get('overdue', 0)}逾期 | "
                f"整体{s.get('overall_progress', '0%')}"
            )
        ms = s.get("milestones", [])
        if ms:
            ms_str = " | ".join(
                f"{m['name']}{('✓' if m['status']=='COMPLETED' else
m.get('progress', ''))}"
                for m in ms[:3]
            )
            lines.append(f"📅 {ms_str}")
        t = content_json.get("temporal", {})
        re = t.get("recent_events", [])
        if re:
            ev_str = " / ".join(f"{e['date'][:5]} if len(e.get('date', ''))>=5 else
e.get('date', '')}{e['summary']}" for e in re[:3])
            lines.append(f"📅 近期: {ev_str}")
        ud = t.get("upcoming_deadlines", [])
        if ud:
            ud_str = " / ".join(f"{d['name']}{(d['deadline'][:5] if
len(d.get('deadline', ''))>=5 else d.get('deadline', ''))}" for d in ud[:3])
            lines.append(f"🕒 7天内: {ud_str}")
        oi = t.get("overdue_items", [])
        if oi:
            oi_str = " / ".join(f"{i['name']}{(i['deadline'][:5] if
len(i.get('deadline', ''))>=5 else i.get('deadline', ''))}" for i in oi[:3])
            lines.append(f"🕒 逾期: {oi_str}")

```

```

        r = content_json.get("relation", {})
        bn = r.get("blocked_nodes", [])
        if bn:
            bn_str = " / ".join(f"{b['blocked_by']} → {b['node']}" for b in
bn[:2])
            lines.append(f"🔒 阻塞: {bn_str}")
        intro = content_json.get("introspection", {})
        tier = intro.get("initialization_tier", 0)
        tier_labels = {0: "未开始", 1: "T1 可识别", 2: "T2 有组织", 3: "T3 可运转",
4: "T4 充分运转"}
        if tier < 3:
            missing = intro.get("missing_items", [])
            missing_str = " / ".join(missing[:3])
            lines.append(f"🟡 {tier_labels.get(tier, '未知')}级 - 缺失:
{missing_str}")
        else:
            lines.append(f"🟢 {tier_labels.get(tier, '未知')}级")
        return "\n".join(lines)

```

文件编号: 118 文件路径: emily-core\emily_core\services\world_book_service.py

```

from __future__ import annotations
import json
import logging
from typing import Optional
from ..repositories.world_book_repo import ProjectWorldBookRepo
from ..services.world_book_builder import ProjectWorldBookBuilder
from ..services.cognition_drift_detector import CognitionDriftDetector
logger = logging.getLogger("emily.world_book_service")
DATA_DRIVEN_LAYERS = {"personnel", "structure", "temporal", "relation",
"introspection"}
LLM_DRIVEN_LAYERS = {"ontology"}
class ProjectWorldBookService:
    def __init__(self, llm_client=None):
        self._llm = llm_client
        self._builder = ProjectWorldBookBuilder()
        self._detector = CognitionDriftDetector()
    async def update_stale(self, project_id: str, *, dry_run: bool = False) ->
dict:
        drift_result = self._detector.detect(project_id)
        if not drift_result.get("has_world_book"):
            import asyncio
            return await asyncio.to_thread(
                self._builder.build, project_id, generated_by="startup",
dry_run=dry_run
            )
        if not drift_result.get("has_drift"):
            return {
                "project_id": project_id,
                "status": "no_drift",
                "message": "世界书与实际数据一致, 无需更新",
            }

```

```

        stale_layers = drift_result.get("stale_layers", [])
        if not stale_layers:
            return {"project_id": project_id, "status": "no_drift"}
        import asyncio
        result = await asyncio.to_thread(
            self._builder.build, project_id, generated_by="scheduler_data",
dry_run=dry_run
        )
        result["updated_layers"] = stale_layers
        result["drift_details"] = drift_result.get("drift", {})
        result["status"] = "updated" if not dry_run else "preview"
        return result
    async def update_all(self, *, dry_run: bool = False) -> list[dict]:
        import asyncio
        wbs = await asyncio.to_thread(ProjectWorldBookRepo.list_all)
        results = []
        for wb in wbs:
            try:
                r = await self.update_stale(wb.project_id, dry_run=dry_run)
                results.append(r)
            except Exception as e:
                logger.error("update_stale failed for project %s: %s",
wb.project_id, e)
                results.append({"project_id": wb.project_id, "status": "error",
"error": str(e)})
        return results

```

文件编号: 119 文件路径: emily-core\emily_core\session\confirm_queue.py

```

from __future__ import annotations
import heapq
import itertools
import logging
from dataclasses import dataclass, field
from typing import Any
logger = logging.getLogger("emily.confirm_queue")
@dataclass(order=True)
class _Entry:
    priority: int
    seq: int
    workitem_id: str = field(compare=False)
    prompt: str = field(compare=False)
    payload: Any = field(default=None, compare=False)
class ConfirmQueue:
    def __init__(self):
        self._heap: list[_Entry] = []
        self._counter = itertools.count()
    def add(self, workitem_id: str, prompt: str, priority: int = 1, payload: Any =
None) -> None:
        entry = _Entry(
            priority=priority,
            seq=next(self._counter),

```

```

        workitem_id=workitem_id,
        prompt=prompt,
        payload=payload,
    )
    heapq.heappush(self._heap, entry)
    logger.debug("ConfirmQueue add: WI=%s priority=%d (size=%d)",
                 workitem_id, priority, len(self._heap))
def pop(self) -> _Entry | None:
    if not self._heap:
        return None
    return heapq.heappop(self._heap)
def peek(self) -> _Entry | None:
    return self._heap[0] if self._heap else None
def remove(self, workitem_id: str) -> bool:
    if workitem_id == "__all__":
        self._heap.clear()
        return True
    before = len(self._heap)
    self._heap = [e for e in self._heap if e.workitem_id != workitem_id]
    heapq.heapify(self._heap)
    return len(self._heap) < before
def clear(self) -> None:
    self._heap.clear()
@property
def is_empty(self) -> bool:
    return not self._heap
def __len__(self) -> int:
    return len(self._heap)

```

文件编号: 120 文件路径: emily-core\emily_core\session\fetchers\fetch_available_tools.py

```

from __future__ import annotations
import json
import logging
import argparse
logger = logging.getLogger("emily.session.fetchers.fetch_available_tools")
import os
DB_URL_DEFAULT = os.getenv(
    "EMILY_DATABASE_URL",
    "postgresql://emily:emily_secret_2026@localhost:25432/emily"
)
def fetch(perms: dict) -> list[dict]:
    try:
        from ...repositories.tool_registry_repo import ToolRegistryRepo
        level = perms.get("level", 1)
        sop_allow = perms.get("sop_allow", [])
        return ToolRegistryRepo.get_available(
            level=level,
            sop_allow=sop_allow,
        )
    except Exception as e:
        logger.error("fetch_available_tools failed: %s", e)

```

```

        return []
def main():
    import sys
    sys.stdout.reconfigure(encoding="utf-8")
    parser = argparse.ArgumentParser(description="获取用户可用的 API 工具列表")
    parser.add_argument("--user-id", required=True, help="用户 UUID")
    parser.add_argument("--db-url", default=DB_URL_DEFAULT, help="PostgreSQL 连接 URL")
    args = parser.parse_args()
    from ...infrastructure.database import init_db
    init_db(db_url=args.db_url)
    from ...services.permission_service import PermissionService
    perms = PermissionService().build_permission_dict(args.user_id)
    result = fetch(perms)
    print(json.dumps(result, ensure_ascii=False, indent=2))
if __name__ == "__main__":
    main()

```

文件编号: 121 文件路径: emily-core\emily_core\session\fetchers\fetch_rag_info.py

```

from __future__ import annotations
import json
import logging
import argparse
from typing import Any
logger = logging.getLogger("emily.session.fetchers.fetch_rag_info")
def fetch(core: Any = None) -> dict:
    try:
        if core is None:
            return {"available": False, "collections": []}
        rag_provider = getattr(core, "_rag_provider", None)
        if rag_provider is None:
            return {"available": False, "collections": []}
        is_available_fn = getattr(rag_provider, "is_available", None)
        if is_available_fn is not None:
            import asyncio
            try:
                loop = asyncio.get_running_loop()
                available = True
            except RuntimeError:
                available = asyncio.run(is_available_fn())
        else:
            available = True
        collections = getattr(rag_provider, "collections", [])
        return {
            "available": available,
            "collections": list(collections) if collections else [],
        }
    except Exception as e:
        logger.error("fetch_rag_info failed: %s", e)
        return {"available": False, "collections": []}
def main():

```

```

import sys
sys.stdout.reconfigure(encoding="utf-8")
parser = argparse.ArgumentParser(description="获取 RAG 知识库可用性信息")
parser.add_argument("--check", action="store_true", help="实际调用 RAG 检查可用性")
args = parser.parse_args()
if args.check:
    print("需要 EmilyCore 实例, 请在 emily-core 容器内运行或使用 emy-test 工具")
    return
result = fetch(core=None)
result["note"] = "无 EmilyCore 实例, available 为默认值 False"
print(json.dumps(result, ensure_ascii=False, indent=2))
if __name__ == "__main__":
    main()

```

文件编号: 122 文件路径: emily-core\emily_core\session\fetchers\fetch_system_description.py

```

from __future__ import annotations
import json
import logging
import argparse
logger = logging.getLogger("emily.session.fetchers.fetch_system_description")
import os
DB_URL_DEFAULT = os.getenv(
    "EMILY_DATABASE_URL",
    "postgresql://emily:emily_secret_2026@localhost:25432/emily"
)
_TABLE_DISPLAY_NAMES = {
    "project": "项目表",
    "events": "事件表",
    "tasks": "任务表",
    "meetings": "会议表",
    "files": "文件表",
    "financial": "财务表",
}
def fetch(perms: dict) -> str:
    try:
        from ...repositories.system_description_repo import SystemDescriptionRepo
        desc = SystemDescriptionRepo.get_latest()
        if desc is None:
            return ""
        if not desc.content_json:
            return desc.content_text or ""
        content = json.loads(desc.content_json)
        return _format_filtered_text(content, perms)
    except Exception as e:
        logger.error("fetch_system_description failed: %s", e)
        return ""
def _format_filtered_text(content: dict, perms: dict) -> str:
    db_perms = perms.get("db_perms", {})
    lines = []
    db = content.get("database", {})

```

```

visible_tables = db.get("visible_tables", [])
if visible_tables and db_perms:
    lines.append("🗄️ 数据库资源（你有权访问的表）")
    for tbl in visible_tables:
        tbl_name = tbl.get("table_name", "")
        perm = db_perms.get(tbl_name, "")
        if not perm:
            continue
        perm_cn = "读写" if perm == "read_write" else "只读"
        display_name = tbl.get("display_name",
            _TABLE_DISPLAY_NAMES.get(tbl_name, tbl_name))
        desc = tbl.get("description", "")
        lines.append(f"· {display_name}: {perm_cn} – {desc}")
        fields = tbl.get("key_fields", [])
        if fields:
            field_strs = []
            for f in fields[:5]:
                req_mark = ",必填" if f.get("required") else ""
                ftype = f.get("type", "")
                fdesc = f.get("description", "")
                if fdesc:
                    field_strs.append(f"{f['name']}({fdesc}{req_mark})")
                else:
                    field_strs.append(f"{f['name']}({ftype}{req_mark})")
            lines.append(f"  核心字段: {' / '.join(field_strs)}")
        hidden_count = db.get("hidden_table_count", 0)
        no_perm_count = sum(1 for tbl in visible_tables if not
            db_perms.get(tbl.get("table_name", ""), ""))
        total_hidden = hidden_count + no_perm_count
        total = db.get("total_tables", 0)
        lines.append(f"（另有 {total_hidden} 张表按权限不可直接查询，共 {total}
            张）")
        relations = db.get("key_relations", [])
        if relations:
            user_tables = set(db_perms.keys())
            rel_strs = []
            for r in relations:
                from_tbl = r.get("from", "").split(".")[0]
                if from_tbl in user_tables:
                    rel_strs.append(f"{r['from']} → {r['to']}")
            if rel_strs:
                lines.append("🔗 表间关系")
                lines.append(" / ".join(rel_strs[:5]))
        elif not db_perms:
            lines.append("🗄️ 数据库资源（无数据库访问权限）")
    fc = content.get("file", {})
    categories = fc.get("categories", [])
    if categories:
        lines.append("")
        lines.append("📁 文件分类体系")
        cat_strs = [c["name"] for c in categories]
        lines.append(" / ".join(cat_strs))
    access_rules = fc.get("access_rules", [])
    if access_rules:

```



```

        lines.append("文件访问规则: " + "; ".join(access_rules[:3]))
    perm = content.get("permission", {})
    levels = perm.get("levels", [])
    if levels:
        lines.append("")
        lines.append("🔒 权限分级体系")
        participant_levels = [l for l in levels if l.get("line") == "参建线"]
        if participant_levels:
            pl_str = " → ".join(f"L{1['level']} {1['name']}" for l in
participant_levels)
            lines.append(f"参建线: L1 访客 → {pl_str}")
        construction_levels = [l for l in levels if l.get("line") == "建设线"]
        if construction_levels:
            cl_str = " → ".join(f"L{1['level']} {1['name']}" for l in
construction_levels)
            lines.append(f"建设线: L1 访客 → {cl_str}")
        inheritance = perm.get("inheritance", {})
        note = inheritance.get("note", "") if isinstance(inheritance, dict) else
""
        if note:
            lines.append(f"⚠️ {note}; 跨线访问需临时/永久授权")
            user_level = perms.get("level", 1)
            from ...permission.level import LEVEL_NAME, effective_levels
            level_name = LEVEL_NAME.get(user_level, "未知")
            eff = effective_levels(user_level)
            chain_str = "→".join(f"L{1}" for l in sorted(eff, reverse=True))
            line_map = {1: "公共", 2: "参建线", 3: "参建线", 4: "建设线", 5: "建设线",
6: "建设线"}
            user_line = line_map.get(user_level, "")
            lines.append(f"你的位置: {level_name}(L{user_level}) {user_line}, 继承链
{chain_str}")
        return "\n".join(lines)
def main():
    import sys
    sys.stdout.reconfigure(encoding="utf-8")
    parser = argparse.ArgumentParser(description="获取系统自我描述文本")
    parser.add_argument("--user-id", required=True, help="用户 UUID")
    parser.add_argument("--db-url", default=DB_URL_DEFAULT, help="PostgreSQL 连接
URL")
    args = parser.parse_args()
    from ...infrastructure.database import init_db
    init_db(db_url=args.db_url)
    from ...services.permission_service import PermissionService
    perms = PermissionService().build_permission_dict(args.user_id)
    result = fetch(perms)
    print(result)
if __name__ == "__main__":
    main()

```

文件编号: 123 文件路径: emily-core\emily_core\session\fetchers\fetch_visible_files.py

```

from __future__ import annotations
import json
import logging
import argparse
logger = logging.getLogger("emily.session.fetchers.fetch_visible_files")
import os
DB_URL_DEFAULT = os.getenv(
    "EMILY_DATABASE_URL",
    "postgresql://emily:emily_secret_2026@localhost:25432/emily"
)
def fetch(user_id: str) -> dict:
    try:
        from ...repositories.session_accessible_file_repo import
        SessionAccessibleFileRepo
        return SessionAccessibleFileRepo.get_file_summary(user_id)
    except Exception as e:
        logger.error("fetch_visible_files failed user=%s: %s", user_id, e)
        return {"count": 0, "by_type": {}}
def format_summary(visible_files: dict) -> str:
    count = visible_files.get("count", 0)
    if count == 0:
        return "无可见文件"
    by_type = visible_files.get("by_type", {})
    type_parts = []
    for ft, cnt in sorted(by_type.items(), key=lambda x: x[1], reverse=True):
        type_parts.append(f"{ft}({cnt}个)")
    by_category = visible_files.get("by_category", {})
    cat_parts = []
    from ...infrastructure.database.models import FileCategory
    for cat, cnt in sorted(by_category.items(), key=lambda x: x[1], reverse=True):
        cat_parts.append(f"{FileCategory.display(cat)}({cnt}个)")
    cat_str = f", 分类: {' '.join(cat_parts)}" if cat_parts else ""
    return f"共 {count} 个文件 ({' '.join(type_parts)}) {cat_str}" if type_parts
else f"共 {count} 个文件{cat_str}"
def main():
    import sys
    sys.stdout.reconfigure(encoding="utf-8")
    parser = argparse.ArgumentParser(description="获取用户可见文件摘要")
    parser.add_argument("--user-id", required=True, help="用户 UUID")
    parser.add_argument("--db-url", default=DB_URL_DEFAULT, help="PostgreSQL 连接 URL")
    parser.add_argument("--text", action="store_true", help="输出文本摘要而非 JSON")
    args = parser.parse_args()
    from ...infrastructure.database import init_db
    init_db(db_url=args.db_url)
    result = fetch(args.user_id)
    if args.text:
        print(format_summary(result))
    else:
        print(json.dumps(result, ensure_ascii=False, indent=2))

```

```
if __name__ == "__main__":  
    main()
```

文件编号: 124 文件路径: emily-core\emily_core\session\fetchers\fetch_visible_schema.py

```
from __future__ import annotations  
import json  
import logging  
import argparse  
logger = logging.getLogger("emily.session.fetchers.fetch_visible_schema")  
import os  
DB_URL_DEFAULT = os.getenv(  
    "EMILY_DATABASE_URL",  
    "postgresql://emily:emily_secret_2026@localhost:25432/emily"  
)  
def fetch(perms: dict) -> str:  
    try:  
        db_perms = perms.get("db_perms", {})  
        if not db_perms:  
            return "（无数据库访问权限）"  
        table_names = {  
            "project": "项目表",  
            "events": "事件表",  
            "tasks": "任务表",  
            "meetings": "会议表",  
            "files": "文件表",  
            "messages": "消息表",  
            "users": "用户表",  
            "financial": "财务表",  
            "project_nodes": "节点表",  
            "node_deliverables": "节点成果表",  
        }  
        lines = []  
        for tbl, perm in sorted(db_perms.items()):  
            label = table_names.get(tbl, tbl)  
            perm_cn = "读写" if perm == "read_write" else "只读"  
            lines.append(f"    {label}: {perm_cn}")  
        return "\n".join(lines) if lines else "（无数据库访问权限）"  
    except Exception as e:  
        logger.error("fetch_visible_schema failed: %s", e)  
        return ""  
def main():  
    import sys  
    sys.stdout.reconfigure(encoding="utf-8")  
    parser = argparse.ArgumentParser(description="获取用户可见的数据库 Schema 摘要")  
    parser.add_argument("--user-id", required=True, help="用户 UUID")  
    parser.add_argument("--db-url", default=DB_URL_DEFAULT, help="PostgreSQL 连接 URL")  
    args = parser.parse_args()  
    from ...infrastructure.database import init_db  
    init_db(db_url=args.db_url)  
    from ...services.permission_service import PermissionService
```

```

perms = PermissionService().build_permission_dict(args.user_id)
result = fetch(perms)
print(result)
if __name__ == "__main__":
    main()

```

文件编号: 125 文件路径: emily-core\emily_core\session\focus_lock.py

```

from __future__ import annotations
import logging
logger = logging.getLogger("emily.focus_lock")
_SWITCH_HINTS = ("先不管", "先处理", "等一下", "先说", "改成先")
class FocusLock:
    def __init__(self):
        self._current_focus: str | None = None
    @property
    def current_focus(self) -> str | None:
        return self._current_focus
    def set_focus(self, workitem_id: str) -> None:
        if self._current_focus != workitem_id:
            logger.debug("FocusLock: focus %s -> %s", self._current_focus,
workitem_id)
            self._current_focus = workitem_id
    def clear_focus(self) -> None:
        self._current_focus = None
    @staticmethod
    def wants_switch(content: str) -> bool:
        text = (content or "").strip()
        return any(hint in text for hint in _SWITCH_HINTS)

```

文件编号: 126 文件路径: emily-core\emily_core\session\session_agent.py

```

from __future__ import annotations
import asyncio
import logging
from datetime import datetime, timezone, timedelta
from typing import TYPE_CHECKING
from .session_state import SessionState
from .session_context import SessionContext
from .focus_lock import FocusLock
from .confirm_queue import ConfirmQueue
from ..adapters.standard.reply import ReplyMessage
from ..services.event_journal import EventJournal
from ..workitem import WorkItem
if TYPE_CHECKING:
    from ..adapters.standard.message import StandardMessage
    from ..workitem.pipeline.bus import PipelineBUS
logger = logging.getLogger("emily.session_agent")
_SIMPLE_GREETINGS = {

```

```

    "你好", "早", "早上好", "下午好", "晚上好", "早安", "午安",
    "hi", "hello", "hey", "nihao", "在吗", "在不", "在不在",
}
_SIMPLE_THANKS = {"谢谢", "感谢", "多谢", "thanks", "thank you", "thx", "3q"}
_SIMPLE_FAREWELLS = {"再见", "拜拜", "bye", "goodbye", "晚安", "回头见", "下次见"}
_SIMPLE_SELF_INTRO = {"你是谁", "你叫什么", "你是什么", "who are you", "介绍自己",
"介绍一下自己"}
def _load_session_prompt() -> str:
    from ..infrastructure.llm.prompt_loader import load_prompt
    return load_prompt("session")
_SESSION_SYSTEM_PROMPT = _load_session_prompt()
def _beijing_now_str() -> str:
    return (datetime.now(timezone.utc) + timedelta(hours=8)).strftime("%Y-%m-%d
%H:%M:%S")
class SessionAgent:
    def __init__(
        self,
        conversation_id: str,
        context: SessionContext,
        bus: "PipelineBUS",
        llm_client=None,
        skill_registry=None,
    ):
        self.conversation_id = conversation_id
        self.context = context
        self.state = SessionState.CREATED
        self.focus = FocusLock()
        self.confirm_queue = ConfirmQueue()
        from ..workitem import SessionScheduler
        self.scheduler = SessionScheduler(conversation_id, bus,
session_context=context)
        self._llm = llm_client
        self._skill_registry = skill_registry
        from datetime import datetime, timezone
        self._created_at = datetime.now(timezone.utc).isoformat()
        self.state = SessionState.ACTIVE
        _log_session_lifecycle(self.conversation_id, self.context.user_id,
"created")
        async def handle(self, message: "StandardMessage") -> ReplyMessage | None:
            reply = await self._handle_impl(message)
            if reply is not None:
                self._record_turn(message, reply.content)
                if len(self.context.message_history) > 40 and self._llm:
                    asyncio.ensure_future(self.context.compress_overflow(self._llm))
                _detect_feedback(message.content or "", reply.content or "",
                                self.conversation_id, self.context.user_id)
            return reply
        async def _handle_impl(self, message: "StandardMessage") -> ReplyMessage |
None:
            content = (message.content or "").strip()
            logger.debug("Session[%s] handle: %s", self.conversation_id, content[:60])
            fast = self._try_fast_reply(content, message.sender_name)
            if fast is not None:
                return self._reply(message, fast)

```

```

        if FocusLock.wants_switch(content):
            self.focus.clear_focus()
            logger.debug("Session[%s] focus cleared by user switch",
self.conversation_id)
        work_items = await self._split_into_workitems(message)
        if not work_items:
            return self._reply(
                message,
                "我暂时不太确定你的意思，可以换个说法吗？比如"
                "「帮我创建事件：样板段放线完成」。",
            )
        self.focus.set_focus(work_items[0].id)
        for wi in work_items:
            if wi.sop_id == "SYS-confirm":
                confirm_reply = await self._handle_confirm(wi)
                if confirm_reply:
                    return self._reply(message, confirm_reply)
                return self._reply(message, "确认处理完成。")
        for wi in work_items:
            self.scheduler.enqueue(wi)
        done = await self.scheduler.run_all_with_message(message)
        pending = self._collect_pending_confirms(done)
        if pending:
            return self._reply(message, pending)
        replies = [wi.result_text for wi in done if wi.result_text]
        if not replies:
            return self._reply(message, "Emily 已处理完毕。")
        return self._reply(message, "\n\n".join(replies))
    async def _recognize_intent(self, message: "StandardMessage") -> dict:
        content = message.content or ""
        catalog_source = self._skill_registry
        if not self._llm or not catalog_source:
            return {"sop_id": None, "confidence": "none", "reasoning": "",
                    "is_compound": False, "sub_tasks": [], "fallback": True}
        if not content.strip():
            return {"sop_id": None, "confidence": "none", "reasoning": "空消息",
                    "is_compound": False, "sub_tasks": [], "fallback": True}
        try:
            sop_catalog = catalog_source.dump_as_text()
        except Exception as e:
            logger.warning("Failed to dump SOP catalog: %s", e)
            return {"sop_id": None, "confidence": "none", "reasoning": f"SOP目录加
载失败: {e}",
                    "is_compound": False, "sub_tasks": [], "fallback": True}
        system_prompt = (_SESSION_SYSTEM_PROMPT
            .replace("{sop_catalog}", sop_catalog)
            .replace("{current_datetime}", _beijing_now_str()))
        prompt_vars = self.context.get_prompt_variables()
        for key, value in prompt_vars.items():
            if value:
                system_prompt = system_prompt.replace(key, str(value))
        full_messages = [{"role": "system", "content": system_prompt}]
        full_messages.extend(self.context.message_history)
        pending_event = self._get_pending_event()

```

```

if pending_event:
    full_messages.append({
        "role": "system",
        "content": (
            f"⚠️ 当前存在待确认的录入项: \n"
            f"  编号: {pending_event.event_no}\n"
            f"  内容: {pending_event.title}\n"
            f"  状态: 等待用户确认\n"
            f"  如果用户表达了确认/取消/修改意图, 请路由到 SYS-confirm, "
            f"不要走其他 SOP 路由。"
        ),
    })
sender = getattr(message, "sender_name", "") or ""
full_messages.append({
    "role": "user",
    "content": content,
    "name": sender if sender else None,
})
try:
    result = await self._llm.chat_messages(full_messages, json_mode=True)
    data = result.get("data", {})
    logger.debug("SessionAgent intent for '%s': sop=%s conf=%s
compound=%s",
                content[:40], data.get("sop_id"), data.get("confidence"),
                data.get("is_compound"))
    return data
except Exception as e:
    logger.warning("SessionAgent intent recognition failed: %s", e)
    return {"sop_id": None, "confidence": "none", "reasoning": f"LLM调用失
败: {e}",
            "is_compound": False, "sub_tasks": [], "fallback": True}
async def _split_into_workitems(self, message: "StandardMessage") ->
list[WorkItem]:
    content = message.content or ""
    intent = await self._recognize_intent(message)
    sop_id = intent.get("sop_id")
    is_compound = intent.get("is_compound", False)
    sub_tasks = intent.get("sub_tasks", [])
    fallback = intent.get("fallback", False)
    confidence = intent.get("confidence", "none")
    logger.info(
        "Session[%s] intent: sop=%s conf=%s compound=%s fallback=%s
sub_tasks=%d",
        self.conversation_id, sop_id, confidence, is_compound, fallback,
        len(sub_tasks),
    )
    if sop_id == "SYS-confirm":
        action = (intent.get("data") or {}).get("action", "confirm")
        pending_event = self._get_pending_event()
        if pending_event:
            wi = WorkItem(
                session_id=self.conversation_id,
                user_input=content,
                user_id=self.context.user_id,

```

```

        sop_id="SYS-confirm",
        intent_type="sop",
        priority=0,
    )
    setattr(wi, "_confirm_action", action)
    setattr(wi, "_confirm_event_id", pending_event.id)
    return [wi]
else:
    return [WorkItem(
        session_id=self.conversation_id,
        user_input=content,
        user_id=self.context.user_id,
        sop_id=None,
        intent_type="fallback",
        priority=1,
    )]
if fallback or not sop_id:
    return [WorkItem(
        session_id=self.conversation_id,
        user_input=content,
        user_id=self.context.user_id,
        sop_id=None,
        intent_type="fallback",
        priority=1,
    )]
if is_compound and sub_tasks:
    items = []
    for i, st in enumerate(sub_tasks[:5]):
        wi = WorkItem(
            session_id=self.conversation_id,
            user_input=st.get("user_input", content) if isinstance(st,
dict) else content,
            user_id=self.context.user_id,
            sop_id=st.get("sop_id", sop_id) if isinstance(st, dict) else
sop_id,
            intent_type="sop",
            priority=st.get("priority", 1) if isinstance(st, dict) else 1,
        )
        items.append(wi)
    return items
return [WorkItem(
    session_id=self.conversation_id,
    user_input=content,
    user_id=self.context.user_id,
    sop_id=sop_id,
    intent_type="sop",
    priority=1,
)]
def _get_pending_event(self):
    try:
        from ..repositories.event_repo import EventRepository
        repo = EventRepository()
        return repo.find_pending_by_conversation_id(self.conversation_id)
    except Exception as e:

```



```

        logger.debug("_get_pending_event failed: %s", e)
        return None
    async def _handle_confirm(self, wi) -> str | None:
        action = getattr(wi, "_confirm_action", "confirm")
        event_id = getattr(wi, "_confirm_event_id", "")
        if not event_id:
            logger.warning("SYS-confirm: no _confirm_event_id on WorkItem %s",
wi.id)

            return "没有待确认的事件。请重新录入。"
        try:
            from ..repositories.event_repo import EventRepository
            from ..services.event_service import EventService
            from ..application.event_app import EventApplication
            event_repo = EventRepository()
            event = event_repo.get_by_id(event_id)
            if event is None:
                return "找不到该事件记录，可能已被处理。"
            if event.status != "pending":
                return f"该事件（{event.event_no}）已经处理过了，当前状态为
「{event.status}」。"
            event_service = EventService()
            event_app = EventApplication(event_service)
            journal = EventJournal(path="", enabled=True)
            event_app.set_journal(journal)
            result = event_app.handle_confirmation(event_id=event_id,
action=action)
            logger.info(
                "SYS-confirm: event=%s action=%s success=%s",
                event.event_no, action, result.success,
            )
            return result.reply or "确认操作完成。"
        except Exception as e:
            logger.error("SYS-confirm handling failed: %s", e)
            return f"确认处理失败: {e}"
    def _collect_pending_confirms(self, done_workitems: list) -> str | None:
        from ..workitem.workitem_state import WorkItemState
        needs_confirm = [
            wi for wi in done_workitems
            if wi.state == WorkItemState.WAITING_CONFIRM
        ]
        for wi in needs_confirm:
            self.confirm_queue.add(
                workitem_id=wi.id,
                prompt=f"关于「{wi.user_input[:50]}」...」需要你的确认",
                priority=wi.priority,
            )
        if not self.confirm_queue.is_empty:
            entry = self.confirm_queue.pop()
            if entry:
                return entry.prompt
        return None
    async def archive(self) -> None:
        if self.state in (SessionState.CLOSED, SessionState.ARCHIVING):
            return

```

```

self.state = SessionState.ARCHIVING
logger.info("Session[%s] archiving (history_msgs=%d)...",
            self.conversation_id, len(self.context.message_history))
try:
    self.confirm_queue.clear()
    from ..workitem.workitem_state import WorkItemState
    for wi in list(self.scheduler._active.values()):
        if not wi.is_terminal:
            try:
                wi.transition_to(WorkItemState.FAILED)
            except ValueError:
                pass
    await self.context.persist_and_consolidate(llm_client=self._llm)
    logger.info("Session[%s] archived successfully", self.conversation_id)
    _log_session_lifecycle(self.conversation_id, self.context.user_id,
"archived")
except Exception as e:
    logger.warning("Session[%s] archive warning: %s",
self.conversation_id, e)
finally:
    self.state = SessionState.CLOSED
async def _maybe_refresh_context(self) -> None:
    from .session_data_fetcher import SessionDataFetcher
    try:
        data = SessionDataFetcher.fetch(
            user_id=self.context.user_id,
            conversation_id=self.conversation_id,
        )
        updated = self.context.refresh(data)
        if updated:
            logger.info("Session[%s] auto-refreshed: %s",
self.conversation_id, updated)
    except Exception as e:
        logger.debug("Session[%s] refresh skipped: %s", self.conversation_id,
e)
def _record_turn(self, message: "StandardMessage", reply_content: str) ->
None:
    sender = getattr(message, "sender_name", "") or ""
    self.context.record_turn(
        user_content=(message.content or "")[:2000],
        assistant_content=(reply_content or "")[:2000],
        sender_name=sender,
    )
    if len(self.context.message_history) % 20 == 0:
        asyncio.ensure_future(self._maybe_refresh_context())
def _reply(self, message: "StandardMessage", content: str) -> ReplyMessage:
    msg_id = None
    if getattr(message, "conversation_type", "") == "group":
        msg_id = message.message_id
    return ReplyMessage(
        conversation_id=message.conversation_id,
        content=content,
        reply_to_message_id=msg_id,
    )

```

```

@staticmethod
def _try_fast_reply(content: str, sender_name: str = "") -> str | None:
    text = content.strip().lower().replace(" ", "")
    if not text:
        return None
    if text in _SIMPLE_GREETINGS:
        greeting = f"你好呀, {sender_name}! " if sender_name else "你好呀! "
        return f"{greeting} 有什么需要帮忙的吗? "
    if text in _SIMPLE_THANKS or any(k in text for k in ["谢谢", "感谢",
"thank"]):
        return "不客气! 随时为你效劳。"
    if text in _SIMPLE_FAREWELLS:
        return "再见, 有事随时找我! "
    if text in _SIMPLE_SELF_INTRO or any(k in text for k in ["你是谁", "你叫什
么"]):
        return ("我是 Emily, 你的工程项目管理助手。可以帮你记录事件、管理任务、"
"归档会议、查询项目数据, 随时吩咐! ")
    return None
def _log_session_lifecycle(conversation_id: str, user_id: str, event_type: str) ->
None:
    try:
        from ..infrastructure.logging.session_lifecycle_logger import
SessionLifecycleLogger
        asyncio.ensure_future(SessionLifecycleLogger.log(
            conversation_id=conversation_id,
            user_id=user_id or "",
            event_type=event_type,
            message_count=0,
            duration_ms=0,
        ))
    except Exception:
        pass
def _detect_feedback(user_message: str, assistant_reply: str,
                    conversation_id: str, user_id: str) -> None:
    try:
        from ..infrastructure.logging.feedback_detector import
FeedbackSignalDetector
        signals = FeedbackSignalDetector.detect(
            user_message,
            conversation_id=conversation_id,
            user_id=user_id or "",
            assistant_reply=assistant_reply,
        )
        if signals:
            asyncio.ensure_future(FeedbackSignalDetector.log_signals(
                signals,
                pipeline_run_id="",
                conversation_id=conversation_id,
                user_id=user_id or "",
            ))
    except Exception:
        pass

```

文件编号: 127 文件路径: emily-core\emily_core\session\session_context.py

```
from __future__ import annotations
import asyncio
import json
import logging
from dataclasses import dataclass, field
from datetime import datetime, timezone
from typing import Any
logger = logging.getLogger("emily.session_context")
@dataclass
class SessionContext:
    conversation_id: str = ""
    user_id: str = ""
    user_name: str = ""
    user_position: str = ""
    created_at: str = ""
    project_name: str = ""
    project_type: str = ""
    project_status: str = ""
    long_term_memory: str = ""
    conversation_summary: str = ""
    level: int = 1
    is_management_unit: bool = False
    company_id: str = ""
    company_type: str = ""
    company_name: str = ""
    department: list[str] = field(default_factory=list)
    project_ids: list[str] = field(default_factory=list)
    partner_ids: list[str] = field(default_factory=list)
    scopes: list[str] = field(default_factory=list)
    sop_allow: list[str] = field(default_factory=list)
    db_perms: dict[str, str] = field(default_factory=dict)
    info_level: str = "public"
    supervisor_id: str = ""
    granted_codes: list[str] = field(default_factory=list)
    denied_codes: list[str] = field(default_factory=list)
    authorized_node_ids: list[str] = field(default_factory=list)
    permission_version: int = 0
    permissions_loaded_at: str = ""
    message_history: list[dict] = field(default_factory=list)
    sop_catalog_summary: str = ""
    current_datetime: str = ""
    available_skills: list[str] = field(default_factory=list)
    available_tools: list[dict] = field(default_factory=list)
    visible_schema_summary: str = ""
    visible_files_count: int = 0
    visible_files_summary: str = ""
    rag_available: bool = False
    rag_collections: list[str] = field(default_factory=list)
    project_world_book: str = ""
    rule_book: str = ""
    system_description: str = ""
```

```

@property
def history_summary(self) -> str:
    parts = [p for p in (self.long_term_memory, self.conversation_summary) if
p]
    return "\n".join(parts)
@classmethod
def create(cls, user_id: str, conversation_id: str,
            sender_name: str, core) -> "SessionContext":
    from .session_data_fetcher import SessionDataFetcher
    now = datetime.now(timezone.utc).isoformat()
    ctx = cls(
        conversation_id=conversation_id,
        user_id=user_id,
        user_name=sender_name,
        current_datetime=now,
        created_at=now,
    )
    data = SessionDataFetcher.fetch(user_id, conversation_id, core=core)
    snapshot = data.get("session_snapshot", {})
    runtime = data.get("session_runtime", {})
    errors = data.get("errors", [])
    ctx.user_position = snapshot.get("user_position", "")
    ctx.project_name = snapshot.get("project_name", "")
    ctx.project_type = snapshot.get("project_type", "")
    ctx.project_status = snapshot.get("project_status", "")
    ctx.long_term_memory = snapshot.get("long_term_memory", "")
    ctx.conversation_summary = snapshot.get("conversation_summary", "")
    ctx.level = snapshot.get("level", 1)
    ctx.is_management_unit = snapshot.get("is_management_unit", False)
    ctx.company_id = snapshot.get("company_id", "")
    ctx.company_type = snapshot.get("company_type", "")
    ctx.company_name = snapshot.get("company_name", "")
    ctx.department = list(snapshot.get("department", []))
    ctx.project_ids = list(snapshot.get("project_ids", []))
    ctx.partner_ids = list(snapshot.get("partner_ids", []))
    ctx.scopes = list(snapshot.get("scopes", []))
    ctx.sop_allow = list(snapshot.get("sop_allow", []))
    ctx.db_perms = dict(snapshot.get("db_perms", {}))
    ctx.info_level = snapshot.get("info_level", "public")
    ctx.supervisor_id = snapshot.get("supervisor_id", "")
    ctx.granted_codes = list(snapshot.get("granted_codes", []))
    ctx.denied_codes = list(snapshot.get("denied_codes", []))
    ctx.authorized_node_ids = list(snapshot.get("authorized_node_ids", []))
    ctx.permission_version = snapshot.get("permission_version", 0)
    ctx.permissions_loaded_at = snapshot.get("permissions_loaded_at", "")
    if core is not None:
        skill_registry = getattr(core, "_skill_registry", None)
        if skill_registry is not None:
            try:
                skill_ids = skill_registry.list_sop_ids()
                if skill_ids:
                    ctx.available_skills = list(skill_ids)
            except Exception:
                ctx.available_skills = list(ctx.sop_allow)

```

```

        else:
            ctx.available_skills = list(ctx.sop_allow)
    else:
        ctx.available_skills = list(ctx.sop_allow)
    ctx.available_tools = list(snapshot.get("available_tools", []))
    ctx.visible_schema_summary = snapshot.get("visible_schema_summary", "")
    ctx.visible_files_count = snapshot.get("visible_files_count", 0)
    ctx.visible_files_summary = snapshot.get("visible_files_summary", "")
    ctx.rag_available = snapshot.get("rag_available", False)
    ctx.rag_collections = list(snapshot.get("rag_collections", []))
    ctx.project_world_book = snapshot.get("project_world_book", "")
    ctx.rule_book = snapshot.get("rule_book", "")
    ctx.system_description = snapshot.get("system_description", "")
    recent_turns = runtime.get("recent_turns", [])
    for turn in recent_turns:
        role = turn.get("role", "user")
        name = turn.get("sender_name", "") if role == "user" else None
        ctx.message_history.append({
            "role": role if role in ("user", "assistant") else "user",
            "content": turn.get("content", "") or "",
            "name": name if name else None,
        })
    if core is not None:
        skill_registry = getattr(core, "_skill_registry", None)
        if skill_registry is not None:
            try:
                skill_ids = skill_registry.list_sop_ids()
                if skill_ids:
                    ctx.sop_catalog_summary = (
                        f"可用业务流程 ({len(skill_ids)}): {'',
'.join(skill_ids[:15])}'
                    )
            except Exception:
                pass
    if errors:
        logger.warning("SessionContext.create: %d data fetch errors for
user=%s",
                        len(errors), user_id)

    return ctx

def _format_tools_summary(self) -> str:
    if not self.available_tools:
        return "（无可用工具）"
    lines = []
    for t in self.available_tools:
        lines.append(f"  · {t['api_id']}: {t['display_name']}")
    return "\n".join(lines)

def _format_rag_summary(self) -> str:
    if not self.rag_available:
        return "知识库不可用"
    collections = "、".join(self.rag_collections) if self.rag_collections else
"默认知识库"
    return f"知识库可用 ({collections}) "

def has_sop_permission(self, sop_id: str) -> bool:
    return sop_id in self.sop_allow or "all" in self.sop_allow

```

```

def has_db_permission(self, table: str, operation: str = "read") -> bool:
    perm = self.db_perms.get(table)
    if perm is None:
        return False
    if operation == "read":
        return perm in ["read", "read_write"]
    if operation == "write":
        return perm == "read_write"
    return False
def meets_level_requirement(self, required_level: int) -> bool:
    from ..permission.level import can_access
    return can_access(self.level, required_level)
def meets_grouping_requirement(self, required_grouping: int) -> bool:
    return self.meets_level_requirement(required_grouping)
def record_turn(self, user_content: str, assistant_content: str,
                sender_name: str = "") -> None:
    self.message_history.append({
        "role": "user",
        "content": (user_content or "")[:2000],
        "name": sender_name if sender_name else None,
    })
    self.message_history.append({
        "role": "assistant",
        "content": (assistant_content or "")[:2000],
    })
    if len(self.message_history) > 40:
        logger.debug("SessionContext message_history overflow (%d), triggering
compress",
                    len(self.message_history))
def build_llm_messages(self, system_prompt_template: str,
                       current_user_msg: str = "",
                       sender_name: str = "",
                       pending_context: str = "") -> list[dict]:
    prompt_vars = self.get_prompt_variables()
    system_prompt = system_prompt_template
    for key, value in prompt_vars.items():
        if value:
            system_prompt = system_prompt.replace(key, str(value))
    full_messages: list[dict] = [{"role": "system", "content": system_prompt}]
    full_messages.extend(self.message_history)
    if pending_context:
        full_messages.append({
            "role": "system",
            "content": pending_context,
        })
    if current_user_msg:
        full_messages.append({
            "role": "user",
            "content": current_user_msg,
            "name": sender_name if sender_name else None,
        })
    return full_messages
def get_prompt_variables(self) -> dict[str, str]:
    from ..permission.level import level_label as _level_label

```

```

return {
    "{project_name}": self.project_name,
    "{project_type}": self.project_type,
    "{project_status}": self.project_status,
    "{user_name}": self.user_name,
    "{user_position}": self.user_position,
    "{user_company}": self.company_name,
    "{user_company_type}": self.company_type,
    "{user_department}": "\ ".join(self.department) if self.department
else "",
    "{user_level}": _level_label(self.level),
    "{user_permission_level}": _level_label(self.level),
    "{current_node_ids}": "\ ".join(self.authorized_node_ids),
    "{conversation_summary}": self.conversation_summary,
    "{user_memory}": self.long_term_memory,
    "{sop_catalog}": self.sop_catalog_summary,
    "{current_datetime}": self.current_datetime,
    "{available_skills}": "\ ".join(self.available_skills) or "（无）",
    "{recent_turns}": "",
    "{available_tools}": self._format_tools_summary(),
    "{visible_schema}": self.visible_schema_summary,
    "{visible_files}": self.visible_files_summary,
    "{rag_info}": self._format_rag_summary(),
    "{project_world_book}": self.project_world_book,
    "{rule_book}": self.rule_book,
    "{system_description}": self.system_description,
}

async def persist_and_consolidate(self, llm_client=None) -> None:
    await self._persist_archive()
    if self.user_id and llm_client:
        await self._consolidate_conversation_summary(llm_client)
async def _persist_archive(self) -> None:
    try:
        from ..repositories.session_archive_repo import SessionArchiveRepo
        turn_count = len(self.message_history) // 2
        history_snapshot = json.dumps(
            self.message_history[-40:], ensure_ascii=False
        )
        context_snapshot = json.dumps({
            "user_name": self.user_name,
            "sop_catalog_summary": self.sop_catalog_summary,
            "level": self.level,
            "company_name": self.company_name,
        }, ensure_ascii=False)
        SessionArchiveRepo.create(
            conversation_id=self.conversation_id,
            user_id=self.user_id or None,
            user_name=self.user_name,
            turn_count=turn_count,
            message_history_snapshot=history_snapshot,
            context_snapshot=context_snapshot,
            started_at=self.created_at or None,
            archive_reason="expired",
        )

```



```

        logger.info(
            "SessionContext archive persisted: conv=%s turns=%d",
            self.conversation_id, turn_count,
        )
    except Exception as e:
        logger.warning("SessionContext archive persist failed: %s", e)
    async def _consolidate_conversation_summary(self, llm_client) -> None:
        from ..repositories.user_repo import UserRepository
        user = UserRepository.get_by_id(self.user_id)
        if not user:
            return
        existing_summary = user.conversation_summary or ""
        current_conversation = _format_message_history(self.message_history)
        if not current_conversation or current_conversation == "(无历史消息)":
            return
        compress_messages = [
            {"role": "system", "content": (
                "你是一个对话摘要助手。将用户的「已有历史摘要」和「本次对话」合并为一份
新的摘要。"
                "只保留关键事实：人物、事件、决策、任务、时间。不超过 500 字。"
            )},
            {"role": "user", "content": (
                f"
                f"
                f"请输出合并后的完整摘要："
            )},
        ]
    try:
        result = await llm_client.chat_messages(compress_messages)
        new_summary = result.get("content", "") or ""
        if new_summary and len(new_summary) > 20:
            UserRepository.update_user(self.user_id,
conversation_summary=new_summary)
            logger.info(
                "SessionContext summary consolidated for user %s (%d→%d
chars)",
                self.user_id, len(existing_summary), len(new_summary),
            )
    except Exception as e:
        logger.warning("SessionContext summary consolidation failed: %s", e)
    async def compress_overflow(self, llm_client) -> None:
        if not llm_client:
            batch = self.message_history[:20]
            self.message_history = self.message_history[20:]
            logger.debug("SessionContext compression skipped (no LLM): %d msgs
dropped", len(batch))
            return
        batch = self.message_history[:20]
        self.message_history = self.message_history[20:]
        existing_summary = ""
        if (self.message_history
            and self.message_history[0].get("name") == "system"
            and "[对话历史摘要]" in self.message_history[0].get("content",
"")):

```

```

        existing_summary = self.message_history[0]["content"]
        self.message_history = self.message_history[1:]
        compress_msgs = _build_compress_messages(batch, existing_summary)
    try:
        result = await llm_client.chat_messages(compress_msgs)
        summary_content = result.get("content", "") or ""
        if summary_content and len(summary_content) > 20:
            self.message_history.insert(0, {
                "role": "user",
                "content": f"[对话历史摘要] {summary_content.strip()}",
                "name": "system",
            })
            logger.info("SessionContext compressed %d msgs → summary (%d
chars), history now %d",
                        len(batch), len(summary_content),
                        len(self.message_history))
        except Exception as e:
            logger.warning("SessionContext compression failed (msgs dropped): %s",
e)

    def refresh(self, data: dict) -> list[str]:
        updated: list[str] = []
        snapshot = data.get("session_snapshot", {})
        _hot_fields = {
            "level": snapshot.get("level"),
            "is_management_unit": snapshot.get("is_management_unit"),
            "company_id": snapshot.get("company_id"),
            "company_type": snapshot.get("company_type"),
            "company_name": snapshot.get("company_name"),
            "department": snapshot.get("department"),
            "project_ids": snapshot.get("project_ids"),
            "partner_ids": snapshot.get("partner_ids"),
            "scopes": snapshot.get("scopes"),
            "sop_allow": snapshot.get("sop_allow"),
            "db_perms": snapshot.get("db_perms"),
            "info_level": snapshot.get("info_level"),
            "supervisor_id": snapshot.get("supervisor_id"),
            "granted_codes": snapshot.get("granted_codes"),
            "denied_codes": snapshot.get("denied_codes"),
            "authorized_node_ids": snapshot.get("authorized_node_ids"),
            "permission_version": snapshot.get("permission_version"),
            "permissions_loaded_at": snapshot.get("permissions_loaded_at"),
            "available_tools": snapshot.get("available_tools"),
            "visible_schema_summary": snapshot.get("visible_schema_summary"),
            "visible_files_count": snapshot.get("visible_files_count"),
            "visible_files_summary": snapshot.get("visible_files_summary"),
            "rag_available": snapshot.get("rag_available"),
            "rag_collections": snapshot.get("rag_collections"),
            "project_world_book": snapshot.get("project_world_book"),
            "rule_book": snapshot.get("rule_book"),
            "system_description": snapshot.get("system_description"),
        }
        for field, new_val in _hot_fields.items():
            if new_val is not None:
                old_val = getattr(self, field, None)

```

```

        if new_val != old_val:
            setattr(self, field, new_val)
            updated.append(field)
    _cautious_fields = {
        "project_name": snapshot.get("project_name"),
        "project_type": snapshot.get("project_type"),
        "project_status": snapshot.get("project_status"),
    }
    for field, new_val in _cautious_fields.items():
        if new_val is not None:
            old_val = getattr(self, field, None)
            if new_val != old_val:
                setattr(self, field, new_val)
                updated.append(field)
    sop_allow = snapshot.get("sop_allow", [])
    if sop_allow:
        old_skills = set(self.available_skills)
        new_skills = set(sop_allow)
        if old_skills != new_skills:
            self.available_skills = list(sop_allow)
            updated.append("available_skills")
    if updated:
        logger.info("SessionContext refreshed: %s", updated)
    return updated

def register_skill(self, skill_id: str) -> None:
    if skill_id not in self.available_skills:
        self.available_skills.append(skill_id)

def unregister_skill(self, skill_id: str) -> None:
    if skill_id in self.available_skills:
        self.available_skills.remove(skill_id)

def has_skill(self, skill_id: str) -> bool:
    return skill_id in self.available_skills

def _format_message_history(message_history: list[dict]) -> str:
    if not message_history:
        return "（无历史消息）"
    lines = []
    for msg in message_history:
        role = msg.get("role", "?")
        role_label = "用户" if role == "user" else ("Emy" if role == "assistant"
else "系统")
        content = (msg.get("content", "") or "")[:100]
        name = msg.get("name", "")
        name_part = f"({name})" if name and name != "system" else ""
        lines.append(f"[{role_label}]{name_part} {content}")
    return "\n".join(lines)

def _build_compress_messages(history: list[dict], existing_summary: str) ->
list[dict]:
    if not history:
        return []
    history_text = _format_message_history(history)
    return [
        {"role": "system", "content": (
            "你是一个对话摘要助手。请将以下对话压缩为简短的要点摘要（中文，不超过 300
字），"

```

```

        "只保留关键事实：人物、事件、决策、任务、时间。不要包含套话。"
    )},
    {"role": "user", "content": (
        f"
        f"
        f"请输出合并后的完整摘要（不超过 300 字）： "
    )},
]
format_message_history = _format_message_history
build_compress_messages = _build_compress_messages

```

文件编号：128 文件路径：emily-core\emily_core\session\session_data_fetcher.py

```

from __future__ import annotations
import json
import logging
from datetime import datetime, timezone, timedelta
from typing import Any, Optional
logger = logging.getLogger("emily.session_data_fetcher")
_SENTINEL = "XXXXXXXXXX"
def _beijing_now_str() -> str:
    return (datetime.now(timezone.utc) + timedelta(hours=8)).strftime("%Y-%m-%d
%H:%M:%S")
def _parse_position_json(position_raw: str) -> str:
    if not position_raw or position_raw == "[]":
        return ""
    try:
        positions = json.loads(position_raw)
        if isinstance(positions, list) and positions:
            return ", ".join(str(p) for p in positions if p)
    except (json.JSONDecodeError, IndexError, TypeError):
        return ""
    return ""
def _resolve_display_name(user) -> str:
    try:
        bindings = getattr(user, "im_bindings", None)
        if bindings:
            for binding in bindings:
                display = getattr(binding, "im_display_name", None)
                if display:
                    return display
    except Exception:
        pass
    return getattr(user, "username", "") or ""
def _format_recent_turns(recent_turns: list[dict]) -> str:
    if not recent_turns:
        return ""
    lines = []
    for turn in recent_turns:
        role_label = "用户" if turn.get("role") == "user" else "Emy"
        time_str = turn.get("time", "")[:16]
        content = turn.get("content", "")

```

```

        lines.append(f"[{time_str}] {role_label}: {content}")
    return "\n".join(lines)
def _translate_project_status(status: str) -> str:
    status_map = {
        "active": "进行中",
        "planning": "规划中",
        "paused": "已暂停",
        "completed": "已竣工",
        "cancelled": "已取消",
        "draft": "草稿",
    }
    return status_map.get(status, status)
class SessionDataFetcher:
    @staticmethod
    def fetch(user_id: str, conversation_id: str = "", core=None) -> dict:
        if not user_id:
            raise ValueError("user_id 不能为空")
        errors: list[str] = []
        def _record(value, label: str):
            if isinstance(value, str) and value == _SENTINEL:
                errors.append(label)
            elif isinstance(value, dict):
                for k, v in value.items():
                    if v == _SENTINEL:
                        errors.append(f"{label}.{k}")
        from ..repositories.user_repo import UserRepository
        user = UserRepository.get_by_id(user_id)
        if user is None:
            return _empty_result(conversation_id, user_id, ["用户不存在"])
        user_name = _resolve_display_name(user)
        user_position = _parse_position_json(user.position or "")
        long_term_memory = user.long_term_memory or ""
        conversation_summary = user.conversation_summary or ""
        project_id = getattr(user, "project_id", None)
        perms = _sub_fetch_permissions(user_id, core)
        project = _sub_fetch_project(project_id)
        recent_turns = _sub_fetch_recent_turns(user_id)
        available_tools = _sub_fetch_available_tools(perms)
        visible_schema = _sub_fetch_visible_schema(perms)
        visible_files = _sub_fetch_visible_files(user_id)
        rag_info = _sub_fetch_rag_info(core)
        system_description = _sub_fetch_system_description(perms)
        created_at = datetime.now(timezone.utc).isoformat()
        session_snapshot = {
            "conversation_id": conversation_id,
            "user_id": user_id,
            "user_name": user_name,
            "user_position": user_position,
            "created_at": created_at,
            "project_name": project["project_name"],
            "project_type": project["project_type"],
            "project_status": project["project_status"],
            "level": perms.get("level", 1),
            "company_id": perms.get("company_id", ""),

```

```

        "company_type": perms.get("company_type", ""),
        "company_name": perms.get("company_name", ""),
        "department": perms.get("department", []),
        "project_ids": perms.get("project_ids", []),
        "partner_ids": perms.get("partner_ids", []),
        "scopes": perms.get("scopes", []),
        "sop_allow": perms.get("sop_allow", []),
        "db_perms": perms.get("db_perms", {}),
        "info_level": perms.get("info_level", "public"),
        "supervisor_id": perms.get("supervisor_id", ""),
        "granted_codes": perms.get("granted_codes", []),
        "denied_codes": perms.get("denied_codes", []),
        "authorized_node_ids": perms.get("authorized_node_ids", []),
        "permission_version": perms.get("permission_version", 0),
        "permissions_loaded_at": perms.get("permissions_loaded_at", ""),
        "long_term_memory": long_term_memory,
        "conversation_summary": conversation_summary,
        "available_tools": available_tools,
        "visible_schema_summary": visible_schema,
        "visible_files_count": visible_files.get("count", 0),
        "visible_files_summary": _format_visible_files_summary(visible_files),
        "rag_available": rag_info.get("available", False),
        "rag_collections": rag_info.get("collections", []),
        "project_world_book": _sub_fetch_world_book(project_id),
        "rule_book": _sub_fetch_rule_book(),
        "system_description": system_description,
    }
    session_runtime = {
        "recent_turns": recent_turns,
    }
    for key in ["user_name", "user_position"]:
        _record(session_snapshot[key], key)
    for key in ["level", "company_id", "company_type", "company_name",
               "supervisor_id", "info_level"]:
        _record(session_snapshot[key], key)
    for key in ["project_name", "project_type", "project_status"]:
        _record(project[key], f"project.{key}")
    _record(long_term_memory, "long_term_memory")
    _record(conversation_summary, "conversation_summary")
    logger.info("SessionDataFetcher.fetch done: user=%s errors=%d", user_id,
len(errors))
    if errors:
        logger.warning("SessionDataFetcher.fetch errors: %s", " |
".join(errors))
    return {
        "session_snapshot": session_snapshot,
        "session_runtime": session_runtime,
        "errors": errors,
    }
def _sub_fetch_permissions(user_id: str, core=None) -> dict:
    try:
        perm_service = getattr(core, "_permission_service", None) if core else
None
        if perm_service is None:

```

```

        from ..services.permission_service import PermissionService
        skill_registry = getattr(core, "_skill_registry", None) if core else
None
        perm_service = PermissionService(skill_registry=skill_registry)
        perm_dict = perm_service.build_permission_dict(user_id)
        return perm_dict
    except Exception as e:
        logger.error("_sub_fetch_permissions failed user=%s: %s", user_id, e)
        return {"level": 1}
def _sub_fetch_project(project_id: Optional[str]) -> dict:
    if not project_id:
        return {"project_name": "", "project_type": "", "project_status": ""}
    try:
        from ..infrastructure.database import get_session
        from ..infrastructure.database.models import Project
        with get_session() as session:
            project = session.query(Project).filter(
                Project.id == project_id,
                Project.is_deleted == False,
            ).first()
            if project is None:
                return {"project_name": "", "project_type": "", "project_status":
""}
            stage = getattr(project, "lifecycle_stage", 0) or 0
            type_map = {0: "工程项目", 1: "工程项目", 2: "房屋建筑", 3: "工程项目"}
            return {
                "project_name": project.name or "",
                "project_type": type_map.get(stage, ""),
                "project_status": _translate_project_status(project.status or ""),
            }
    except Exception as e:
        logger.error("_sub_fetch_project failed project_id=%s: %s", project_id, e)
        return {"project_name": _SENTINEL, "project_type": _SENTINEL,
"project_status": _SENTINEL}
def _sub_fetch_recent_turns(user_id: str) -> list[dict]:
    try:
        from ..repositories.message_repo import MessageRepository
        return MessageRepository.get_recent_by_user_id(user_id, limit=20)
    except Exception as e:
        logger.error("_sub_fetch_recent_turns failed user=%s: %s", user_id, e)
        return []
def _sub_fetch_available_tools(perms: dict) -> list[dict]:
    from .fetchers.fetch_available_tools import fetch
    return fetch(perms=perms)
def _sub_fetch_visible_schema(perms: dict) -> str:
    from .fetchers.fetch_visible_schema import fetch
    return fetch(perms=perms)
def _sub_fetch_visible_files(user_id: str) -> dict:
    from .fetchers.fetch_visible_files import fetch
    return fetch(user_id=user_id)
def _sub_fetch_rag_info(core=None) -> dict:
    from .fetchers.fetch_rag_info import fetch
    return fetch(core=core)
def _format_visible_files_summary(visible_files: dict) -> str:

```

```

from .fetchers.fetch_visible_files import format_summary
return format_summary(visible_files)
def _empty_result(conversation_id: str, user_id: str, errors: list[str]) -> dict:
    return {
        "session_snapshot": {
            "conversation_id": conversation_id,
            "user_id": user_id,
            "user_name": _SENTINEL,
            "user_position": _SENTINEL,
            "created_at": _beijing_now_str(),
            "project_name": _SENTINEL,
            "project_type": _SENTINEL,
            "project_status": _SENTINEL,
            "level": 1,
            "company_id": "",
            "company_type": "",
            "company_name": "",
            "department": [],
            "project_ids": [],
            "partner_ids": [],
            "scopes": [],
            "sop_allow": [],
            "db_perms": {},
            "info_level": "public",
            "supervisor_id": "",
            "granted_codes": [],
            "denied_codes": [],
            "authorized_node_ids": [],
            "permission_version": 0,
            "permissions_loaded_at": "",
            "long_term_memory": _SENTINEL,
            "conversation_summary": _SENTINEL,
            "available_tools": [],
            "visible_schema_summary": "",
            "visible_files_count": 0,
            "visible_files_summary": "",
            "rag_available": False,
            "rag_collections": [],
            "project_world_book": "",
            "rule_book": "",
            "system_description": "",
        },
        "session_runtime": {
            "recent_turns": [],
        },
        "errors": errors,
    }
def _sub_fetch_system_description(perms: dict) -> str:
    from .fetchers.fetch_system_description import fetch
    return fetch(perms=perms)
def _sub_fetch_world_book(project_id: Optional[str]) -> str:
    if not project_id:
        return ""
    try:

```



```

from ..repositories.world_book_repo import ProjectWorldBookRepo
wb = ProjectWorldBookRepo.get_by_project(project_id)
if wb is None:
    return ""
return wb.content_text or ""
except Exception as e:
    logger.error("_sub_fetch_world_book failed project=%s: %s", project_id, e)
    return ""
def _sub_fetch_rule_book() -> str:
    try:
        from ..services.rule_book_loader import RuleBookLoader
        loader = RuleBookLoader()
        return loader.content
    except Exception as e:
        logger.error("_sub_fetch_rule_book failed: %s", e)
        return ""

```

文件编号: 129 文件路径: emily-core\emily_core\session\session_state.py

```

from __future__ import annotations
from enum import Enum
class SessionState(Enum):
    CREATED = "CREATED"
    ACTIVE = "ACTIVE"
    WAITING_CONFIRM = "WAITING_CONFIRM"
    ARCHIVING = "ARCHIVING"
    CLOSED = "CLOSED"
TRANSITIONS: dict[SessionState, list[SessionState]] = {
    SessionState.CREATED: [SessionState.ACTIVE],
    SessionState.ACTIVE: [
        SessionState.ACTIVE,
        SessionState.WAITING_CONFIRM,
        SessionState.ARCHIVING,
    ],
    SessionState.WAITING_CONFIRM: [SessionState.ACTIVE, SessionState.ARCHIVING],
    SessionState.ARCHIVING: [SessionState.CLOSED],
    SessionState.CLOSED: [],
}
TERMINAL_STATES = frozenset({SessionState.CLOSED})

```

文件编号: 130 文件路径: emily-core\emily_core\skill\definition.py

```

from __future__ import annotations
from dataclasses import dataclass, field
from typing import Any
@dataclass(frozen=True)
class ParamMapping:
    source: str
    path: str = ""

```

```

    value: Any = None
    extraction: str = ""
    required: bool = False
    default: Any = None
    enum: list[str] = field(default_factory=list)
    max_length: int = 0
@dataclass(frozen=True)
class SkillStep:
    id: str
    description: str
    tool_name: str
    tool_params: dict[str, ParamMapping] = field(default_factory=dict)
    output_key: str = ""
@dataclass(frozen=True)
class SkillTool:
    name: str
    description: str = ""
@dataclass(frozen=True)
class SkillDefinition:
    skill_id: str
    sop_id: str
    version: str
    display_name: str
    instructions: str
    tools: list[SkillTool]
    steps: list[SkillStep]

```

文件编号: 131 文件路径: emily-core\emily_core\skill\executor.py

```

from __future__ import annotations
import logging
import time as _time
from dataclasses import dataclass, field
from typing import Any
from .definition import SkillDefinition
from .param_extractor import ParamExtractor
from ..workitem.pipeline.interfaces.execution import StepResult, ToolCallRecord, DbResult
from ..tools.business_flow_tools import BusinessFlowToolRegistry
logger = logging.getLogger("emily.skill.executor")
@dataclass
class SkillExecutionContext:
    skill: SkillDefinition
    user_input: str
    user_id: str
    message_id: str
    conversation_id: str
    session_context: dict
    step_results: dict[str, dict]
    business_flow_tools: BusinessFlowToolRegistry
    llm_client: Any = None
    session_available_tools: list[dict] = field(default_factory=list)

```

```

class SkillExecutor:
    def __init__(self):
        self._param_extractor: ParamExtractor | None = None
    async def execute(self, ctx: SkillExecutionContext) -> list[StepResult]:
        results: list[StepResult] = []
        self._param_extractor = ParamExtractor(llm_client=ctx.llm_client)
        allowed_tools = {t.name for t in ctx.skill.tools}
        session_api_ids = {t["api_id"] for t in ctx.session_available_tools}
        for step in ctx.skill.steps:
            t_start = _time.monotonic()
            if not step.tool_name:
                if step.tool_params:
                    extracted = await self._extract_null_step_data(step, ctx)
                    if step.output_key and extracted:
                        ctx.step_results[step.output_key] = extracted
                    results.append(StepResult(
                        step_id=step.id,
                        success=True,
                        output=str(extracted) if extracted else step.description,
                        business_data=extracted,
                    ))
                else:
                    logger.debug("Step %s: pure logic step, no extraction needed",
step.id)

                    results.append(StepResult(
                        step_id=step.id,
                        success=True,
                        output=step.description,
                    ))
                    continue
            if step.tool_name not in session_api_ids:
                results.append(StepResult(
                    step_id=step.id,
                    success=False,
                    output=f"工具 '{step.tool_name}' 不在 Session 可见 API 中",
                ))
                break
            tool = ctx.business_flow_tools.get(step.tool_name)
            if tool is None:
                results.append(StepResult(
                    step_id=step.id,
                    success=False,
                    output=f"工具 '{step.tool_name}' 未在 BusinessFlowToolRegistry
中注册",
                ))
                break
            try:
                if step.tool_name in allowed_tools:
                    tool_params = await self._param_extractor.resolve_params(
                        step.tool_params, ctx.user_input, ctx.session_context,
ctx.step_results,
                    )
                else:
                    tool_params = await self._llm_resolve_params(step.tool_name,

```

```

tool, ctx)

    tool_params["_user_id"] = ctx.user_id
    tool_params["_message_id"] = ctx.message_id
    tool_params["_conversation_id"] = ctx.conversation_id
    tool_params["_session_scope"] =
self._build_session_scope(ctx.session_context)
    import inspect
    sig = inspect.signature(tool.handler)
    handler_kwargs = {"params": tool_params}
    if "user_id" in sig.parameters:
        handler_kwargs["user_id"] = ctx.user_id
    if "message_id" in sig.parameters:
        handler_kwargs["message_id"] = ctx.message_id
    handler_result = await tool.handler(**handler_kwargs)
    handler_dict = handler_result if isinstance(handler_result, dict)

else {}

    elapsed_ms = int((_time.monotonic() - t_start) * 1000)
    tool_call = ToolCallRecord(
        tool_name=step.tool_name,
        tool_input=tool_params,
        tool_output=handler_dict,
        success=handler_dict.get("success", True),
        elapsed_ms=elapsed_ms,
    )
    db_results = []
    object_id = handler_dict.get("object_id", "")
    if object_id:
        db_results.append(DbResult(
            operation="insert",
            table=step.tool_name.replace("record_", "") + "s",
            affected_rows=1,
            result_data=handler_dict,
        ))
    output = handler_dict.get("reply", step.description)
    success = handler_dict.get("success", True)
    sr = StepResult(
        step_id=step.id,
        success=success,
        output=str(output),
        tool_calls=[tool_call],
        db_results=db_results,
        business_data=handler_dict,
    )
    if step.output_key and handler_dict:
        ctx.step_results[step.output_key] = handler_dict
except Exception as e:
    logger.error("Step %s failed: %s", step.id, e, exc_info=True)
    sr = StepResult(
        step_id=step.id,
        success=False,
        output=f"步骤执行异常: {e}",
    )
results.append(sr)
if not sr.success:

```

```

        break
    return results
    async def _extract_null_step_data(
        self, step, ctx: SkillExecutionContext,
    ) -> dict:
        if self._param_extractor is None:
            self._param_extractor = ParamExtractor(llm_client=ctx.llm_client)
        try:
            extracted = await self._param_extractor.resolve_params(
                step.tool_params, ctx.user_input, ctx.session_context,
                ctx.step_results,
            )
            if extracted:
                logger.info(
                    "Step %s: null-tool extraction produced %d fields: %s",
                    step.id, len(extracted), list(extracted.keys()),
                )
            return extracted
        except ValueError as e:
            logger.warning("Step %s: null-tool extraction failed: %s", step.id, e)
            return {}
        except Exception as e:
            logger.warning("Step %s: null-tool extraction error: %s", step.id, e)
            return {}
    async def _llm_resolve_params(self, tool_name: str, tool, ctx:
SkillExecutionContext) -> dict:
        if ctx.llm_client is None:
            return {"query": ctx.user_input}
        try:
            import json
            prompt = (
                f"用户请求: [{ctx.user_input}] \n"
                f"需要调用工具: {tool_name}\n"
                f"工具描述: {tool.description}\n"
                f"工具参数定义: {json.dumps(tool.parameters,
ensure_ascii=False)}\n"
                f"Session 上下文: {json.dumps(ctx.session_context,
ensure_ascii=False, default=str)}\n"
                f"\n请从用户消息和 Session 上下文中提取工具参数, 只返回 JSON 对象。"
            )
            result = await ctx.llm_client.chat_messages([
                {"role": "user", "content": prompt},
            ])
            content = result.get("content", "{}") if isinstance(result, dict) else
"{}"
            if content.startswith("`"):
                content = content.strip("`").strip()
                if content.startswith("json"):
                    content = content[4:].strip()
                return json.loads(content) if content else {}
        except Exception as e:
            logger.warning("_llm_resolve_params failed for %s: %s", tool_name, e)
            return {"query": ctx.user_input}
    @staticmethod

```

```
def _build_session_scope(session_context: dict) -> dict:
    return {
        "project_ids": session_context.get("project_ids", []),
        "db_perms": session_context.get("db_perms", {}),
        "info_level": session_context.get("info_level", "public"),
        "company_type": session_context.get("company_type", ""),
        "department": session_context.get("department", []),
    }
```

文件编号: 132 文件路径: emily-core\emily_core\skill\param_extractor.py

```
from __future__ import annotations
import logging
from datetime import datetime, timezone
from typing import Any
from .definition import ParamMapping
logger = logging.getLogger("emily.skill.param_extractor")
def _resolve_dot_path(data: dict, path: str) -> Any:
    keys = path.split(".")
    current = data
    for key in keys:
        if isinstance(current, dict):
            current = current.get(key)
        elif isinstance(current, list):
            try:
                current = current[int(key)]
            except (ValueError, IndexError):
                return None
        else:
            return None
        if current is None:
            return None
    return current
class ParamExtractor:
    def __init__(self, llm_client=None):
        self._llm = llm_client
    async def resolve_params(
        self,
        tool_params_mapping: dict[str, ParamMapping],
        user_input: str,
        session_context: dict,
        step_results: dict[str, dict],
    ) -> dict[str, Any]:
        resolved: dict[str, Any] = {}
        for pname, mapping in tool_params_mapping.items():
            value = await self._resolve_one(mapping, user_input, session_context,
            step_results)
            if value is None:
                if mapping.required:
                    raise ValueError(
                        f"必填参数 '{pname}' 提取失败 (source={mapping.source})"
                    )
```

```

        value = mapping.default
        if mapping.enum and value is not None and value not in mapping.enum:
            logger.warning("参数 '%s' 值 '%s' 不在枚举 %s 中, 使用 default=%s",
                           pname, value, mapping.enum, mapping.default)
            value = mapping.default
        if mapping.max_length and isinstance(value, str) and len(value) >
mapping.max_length:
            value = value[:mapping.max_length]
        if value is not None:
            resolved[pname] = value
    return resolved
async def _resolve_one(
    self,
    mapping: ParamMapping,
    user_input: str,
    session_context: dict,
    step_results: dict[str, dict],
) -> Any:
    source = mapping.source
    if source == "fixed":
        return self._extract_from_fixed(mapping)
    elif source == "context":
        return self._extract_from_context(mapping, session_context)
    elif source == "prev_step":
        return self._extract_from_prev_step(mapping, step_results)
    elif source == "user_input":
        return await self._extract_from_user_input(mapping, user_input)
    else:
        logger.warning("未知 source: %s", source)
        return None
def _extract_from_fixed(self, mapping: ParamMapping) -> Any:
    value = mapping.value
    if isinstance(value, str):
        if value == "today":
            return datetime.now(timezone.utc).strftime("%Y-%m-%d")
        if value == "now":
            return datetime.now(timezone.utc).isoformat()
    return value
def _extract_from_context(self, mapping: ParamMapping, session_context: dict)
-> Any:
    return _resolve_dot_path(session_context, mapping.path)
def _extract_from_prev_step(self, mapping: ParamMapping, step_results:
dict[str, dict]) -> Any:
    return _resolve_dot_path(step_results, mapping.path)
async def _extract_from_user_input(self, mapping: ParamMapping, user_input:
str) -> Any:
    if self._llm is None:
        logger.warning("LLM 不可用, 无法提取 user_input 参数: %s",
mapping.extraction)
        return mapping.default
    try:
        from ..infrastructure.llm.prompt_loader import load_prompt
        prompt_template = load_prompt("param_extraction")
    except Exception:

```

```

        prompt_template = (
            "请从用户消息中提取指定字段的值。\\n\\n"
            "字段: {extraction}\\n"
            "约束: {constraints}\\n\\n"
            "用户消息: \\n{user_input}\\n\\n"
            '仅输出 JSON: {{ "value": "提取的值" }}\\n'
            '若无法提取且非必填, 输出: {{ "value": null }}'
        )
        constraints_parts = []
        if mapping.required:
            constraints_parts.append("required=true")
        if mapping.max_length:
            constraints_parts.append(f"max_length={mapping.max_length}")
        if mapping.enum:
            constraints_parts.append(f"enum={mapping.enum}")
        prompt = prompt_template.format(
            extraction=mapping.extraction,
            constraints=", ".join(constraints_parts) if constraints_parts else "无特殊约束",
            user_input=user_input[:1000],
        )
        try:
            result = await self._llm.chat_json(prompt)
            data = result.get("data", {})
            value = data.get("value")
            return value
        except Exception as e:
            logger.warning("LLM 参数提取失败: %s - %s", mapping.extraction, e)
            return mapping.default

```

文件编号: 133 文件路径: emily-core\emily_core\skill\parser.py

```

from __future__ import annotations
import logging
from pathlib import Path
import yaml
from .definition import ParamMapping, SkillStep, SkillTool, SkillDefinition
from .validator import validate_skill
logger = logging.getLogger("emily.skill.parser")
def _extract_sop_id_from_stem(stem: str) -> str:
    if not stem:
        return ""
    parts = stem.split("-")
    if len(parts) >= 3 and parts[0] == "SOP":
        return f"{parts[0]}-{parts[1]}-{parts[2]}"
    return stem
class SkillParseError(Exception):
def _parse_param_mapping(name: str, data: dict) -> ParamMapping:
    if not isinstance(data, dict):
        return ParamMapping(source="fixed", value=data)
    source = data.get("source", "fixed")
    return ParamMapping(

```



```

        source=source,
        path=data.get("path", ""),
        value=data.get("value"),
        extraction=data.get("extraction", ""),
        required=data.get("required", False),
        default=data.get("default"),
        enum=data.get("enum", []),
        max_length=data.get("max_length", 0),
    )
def _parse_step(data: dict) -> SkillStep:
    tool_params: dict[str, ParamMapping] = {}
    raw_params = data.get("tool_params", {})
    if isinstance(raw_params, dict):
        for pname, pdata in raw_params.items():
            tool_params[pname] = _parse_param_mapping(pname, pdata)
    elif isinstance(raw_params, list):
        for item in raw_params:
            if not isinstance(item, dict):
                continue
            pname = item.get("name", "")
            if not pname:
                continue
            pdata = {k: v for k, v in item.items() if k != "name"}
            tool_params[pname] = _parse_param_mapping(pname, pdata)
    return SkillStep(
        id=data.get("id", ""),
        description=data.get("description", ""),
        tool_name=data.get("tool_name", "") or None,
        tool_params=tool_params,
        output_key=data.get("output_key", ""),
    )
def _parse_tool(data: dict) -> SkillTool:
    return SkillTool(
        name=data.get("name", ""),
        description=data.get("description", ""),
    )
def parse_skill_text(text: str, source_name: str = "<text>") -> SkillDefinition:
    try:
        data = yaml.safe_load(text)
    except yaml.YAMLError as e:
        raise SkillParseError(f"YAML 解析失败 ({source_name}): {e}") from e
    if not isinstance(data, dict):
        raise SkillParseError(f"Skill 文件根节点必须是 dict ({source_name})")
    filename_stem = Path(source_name).stem.replace(".skill", "") if source_name != "<text>" else ""
    _inferred_sop_id = _extract_sop_id_from_stem(filename_stem)
    if not data.get("skill_id") and filename_stem:
        data["skill_id"] = filename_stem
    if not data.get("sop_id") and _inferred_sop_id:
        data["sop_id"] = _inferred_sop_id
    if not data.get("version"):
        data["version"] = "1.0"
    if not data.get("display_name") and _inferred_sop_id:
        data["display_name"] = _inferred_sop_id

```

```

tools = [_parse_tool(t) for t in data.get("tools", [])]
steps = [_parse_step(s) for s in data.get("steps", [])]
if not steps and tools:
    logger.info("Skill %s: steps 为空, 从 tools 自动生成兜底步骤", source_name)
    for i, tool in enumerate(tools, 1):
        steps.append(SkillStep(
            id=f"step-{i:02d}",
            description=f"调用 {tool.name}",
            tool_name=tool.name,
            tool_params={},
            output_key=f"{tool.name}_result",
        ))
skill = SkillDefinition(
    skill_id=data.get("skill_id", ""),
    sop_id=data.get("sop_id", ""),
    version=str(data.get("version", "")),
    display_name=data.get("display_name", ""),
    instructions=data.get("instructions", ""),
    tools=tools,
    steps=steps,
)
result = validate_skill(skill)
if not result.is_valid:
    raise SkillParseError(
        f"Skill 校验失败 ({source_name}): {'; '.join(result.errors)}"
    )
if result.warnings:
    for w in result.warnings:
        logger.warning("Skill %s 校验警告: %s", source_name, w)
return skill

def parse_skill_file(path: Path) -> SkillDefinition:
    path = Path(path)
    if not path.exists():
        raise SkillParseError(f"文件不存在: {path}")
    text = path.read_text(encoding="utf-8")
    return parse_skill_text(text, source_name=path.name)

```

文件编号: 134 文件路径: emily-core\emily_core\skill\registry.py

```

from __future__ import annotations
import logging
import threading
from dataclasses import dataclass, field
from datetime import datetime, timezone
from pathlib import Path
from .definition import SkillDefinition
from .parser import parse_skill_file, SkillParseError
logger = logging.getLogger("emily.skill.registry")
def _extract_sop_type(sop_id: str) -> str:
    parts = sop_id.split("-")
    for p in parts[1:]:
        if p in ("REC", "FILE", "QRY", "FLOW", "SYS"):

```

```

        return p
    return "UNKNOWN"
@dataclass
class SkillRegistryStatus:
    total_files: int = 0
    successfully_parsed: int = 0
    failed_parsed: int = 0
    failed_files: list[str] = field(default_factory=list)
    is_ready: bool = False
    last_reload_at: str = ""
class SkillRegistry:
    _SKILL_FILE_PATTERN = "*.skill.yaml"
    def __init__(self, skill_directory: str):
        self.skill_directory = Path(skill_directory)
        self._lock = threading.RLock()
        self._registry: dict[str, SkillDefinition] = {}
        self._by_skill_id: dict[str, SkillDefinition] = {}
        self._is_ready: bool = False
    def load(self) -> SkillRegistryStatus:
        with self._lock:
            new_registry, new_by_skill_id, status = self._scan_and_parse()
            self._registry = new_registry
            self._by_skill_id = new_by_skill_id
            self._is_ready = status.successfully_parsed > 0
            logger.info(
                "SkillRegistry loaded: %d skills, %d ok, %d failed",
                status.total_files, status.successfully_parsed,
                status.failed_parsed,
            )
            return status
    def reload(self) -> SkillRegistryStatus:
        with self._lock:
            new_registry, new_by_skill_id, status = self._scan_and_parse()
            if status.successfully_parsed == 0 and len(self._registry) > 0:
                logger.warning("SkillRegistry reload: all failed, keeping old
registry")
            return self._get_status()
            self._registry = new_registry
            self._by_skill_id = new_by_skill_id
            self._is_ready = status.successfully_parsed > 0
            logger.info(
                "SkillRegistry reloaded: %d ok, %d failed",
                status.successfully_parsed, status.failed_parsed,
            )
            return status
    def get_by_sop_id(self, sop_id: str) -> SkillDefinition | None:
        with self._lock:
            return self._registry.get(sop_id)
    def get_by_skill_id(self, skill_id: str) -> SkillDefinition | None:
        with self._lock:
            return self._by_skill_id.get(skill_id)
    def has_skill(self, sop_id: str) -> bool:
        with self._lock:
            return sop_id in self._registry

```

```

def list_sop_ids(self) -> list[str]:
    with self._lock:
        return sorted(self._registry.keys())
def list_skills(self) -> list[SkillDefinition]:
    with self._lock:
        return list(self._registry.values())
def dump_as_text(self) -> str:
    with self._lock:
        skills = list(self._registry.values())
    if not skills:
        return "（暂无已加载的业务流程/Skill）"
    grouped: dict[str, list[SkillDefinition]] = {}
    for skill in skills:
        sop_type = _extract_sop_type(skill.sop_id)
        grouped.setdefault(sop_type, []).append(skill)
    TYPE_DESC = {
        "REC": "记录与录入（事件/任务/会议等）",
        "FILE": "文件管理（归档/查询/分享）",
        "QRY": "数据查询（项目/进度/人员等）",
        "FLOW": "深度调查（跨维度分析/审计）",
        "SYS": "系统管理（确认/取消/设置等）",
    }
    FALLBACK_BY_TYPE = {
        "REC": "这是记录/录入类请求。若以上 REC 流程均不匹配，使用 record_event /
record_task 原子工具自由推理录入。",
        "FILE": "这是文件管理类请求。若以上 FILE 流程均不匹配，使用 file_storage
原子工具处理。",
        "QRY": "这是数据查询类请求。若以上 QRY 流程均不匹配，使用 query_data 工具
查询，query_type 根据用户意图选择 event/task/meeting/file/message/summary。",
        "FLOW": "这是深度调查类请求。若以上 FLOW 流程均不匹配，使用系统内置的守护调
查Agent执行跨维度分析。",
        "SYS": "这是系统管理类请求。若以上 SYS 流程均不匹配，但仍需系统功能，使用对
应原子工具自由推理。",
    }
    lines: list[str] = []
    lines.append("")
    lines.append("")
    lines.append("请先将用户消息归类到以下类型之一，再在该类型下精匹配具体流程：")
    lines.append("")
    for sop_type, type_skills in grouped.items():
        names = ",".join(s.display_name for s in type_skills)
        sop_ids = ",".join(s.sop_id for s in type_skills)
        desc = TYPE_DESC.get(sop_type, sop_type)
        lines.append(f"**{sop_type}** - {desc}")
        lines.append(f"  包含流程: {names}")
        lines.append(f"  编号: {sop_ids}")
        lines.append("")
    lines.append("---")
    lines.append("")
    lines.append("")
    lines.append("")
    for sop_type, type_skills in grouped.items():
        desc = TYPE_DESC.get(sop_type, sop_type)
        lines.append(f"

```

```

        lines.append("")
        for skill in type_skills:
            tool_names = ", ".join(t.name for t in skill.tools) if skill.tools
else "（无工具声明）"
        lines.append(f"***[{skill.sop_id}] {skill.display_name}** | 工具:
{tool_names}")
        if skill.steps:
            step_descs = " → ".join(
                f"{s.id}({s.tool_name})" for s in skill.steps[:5]
            )
            lines.append(f"  步骤: {step_descs}")
        else:
            lines.append("  步骤: （无预定义步骤）")
        if skill.instructions:
            first_line = skill.instructions.strip().split("\n")[0][:80]
            lines.append(f"  说明: {first_line}")
        lines.append("")
        fallback = FALLBACK_BY_TYPE.get(sop_type, "")
        if fallback:
            lines.append(f"> **{sop_type} 类型兜底**: {fallback}")
            lines.append("")
        lines.append("---")
    return "\n".join(lines)

def _scan_and_parse(self) -> tuple[dict[str, SkillDefinition], dict[str,
SkillDefinition], SkillRegistryStatus]:
    new_registry: dict[str, SkillDefinition] = {}
    new_by_skill_id: dict[str, SkillDefinition] = {}
    now = datetime.now(timezone.utc).isoformat()
    if not self.skill_directory.exists():
        logger.warning("Skill directory not found: %s", self.skill_directory)
        status = SkillRegistryStatus(last_reload_at=now)
        return new_registry, new_by_skill_id, status
    skill_files = sorted(self.skill_directory.glob(self._SKILL_FILE_PATTERN))
    ok_count = 0
    failed_count = 0
    failed_files: list[str] = []
    for file_path in skill_files:
        try:
            skill = parse_skill_file(file_path)
            new_registry[skill.sop_id] = skill
            new_by_skill_id[skill.skill_id] = skill
            ok_count += 1
        except (SkillParseError, Exception) as e:
            failed_count += 1
            failed_files.append(file_path.name)
            logger.error("Skill parse failed: %s - %s", file_path.name, e)
    status = SkillRegistryStatus(
        total_files=ok_count + failed_count,
        successfully_parsed=ok_count,
        failed_parsed=failed_count,
        failed_files=failed_files,
        is_ready=ok_count > 0,
        last_reload_at=now,
    )

```

```

        return new_registry, new_by_skill_id, status
    def _get_status(self) -> SkillRegistryStatus:
        with self._lock:
            ok = sum(1 for _ in self._registry.values())
            return SkillRegistryStatus(
                total_files=ok,
                successfully_parsed=ok,
                is_ready=self._is_ready,
            )

```

文件编号：135 文件路径：emily-core\emily_core\skill\validator.py

```

from __future__ import annotations
from dataclasses import dataclass, field
from .definition import SkillDefinition, SkillStep, ParamMapping
_VALID_SOURCES = {"user_input", "prev_step", "fixed", "context"}
@dataclass
class SkillValidationResult:
    is_valid: bool = True
    errors: list[str] = field(default_factory=list)
    warnings: list[str] = field(default_factory=list)
def validate_skill(skill: SkillDefinition) -> SkillValidationResult:
    result = SkillValidationResult()
    if not skill.skill_id:
        result.errors.append("skill_id 不能为空")
    if not skill.sop_id:
        result.errors.append("sop_id 不能为空")
    if not skill.version:
        result.errors.append("version 不能为空")
    if not skill.display_name:
        result.errors.append("display_name 不能为空")
    if not skill.steps:
        result.errors.append("steps 不能为空—Skill 必须至少有一个步骤")
    else:
        step_ids: set[str] = set()
        for i, step in enumerate(skill.steps):
            if not step.id:
                result.errors.append(f"steps[{i}].id 不能为空")
            if step.id in step_ids:
                result.errors.append(f"steps[{i}].id 重复: {step.id}")
            step_ids.add(step.id)
            if not step.tool_name:
                result.warnings.append(f"steps[{i}].tool_name 为空（纯逻辑步骤，不调用工具）")
        for pname, mapping in step.tool_params.items():
            if mapping.source not in _VALID_SOURCES:
                result.errors.append(
                    f"steps[{i}].tool_params.{pname}.source 非法: {mapping.source}"
                )
            if mapping.source == "prev_step" and not mapping.path:
                result.errors.append(

```

```

        f"steps[{i}].tool_params.{pname}: source=prev_step 时 path
不能为空"
    )
    if mapping.source == "context" and not mapping.path:
        result.errors.append(
            f"steps[{i}].tool_params.{pname}: source=context 时 path
不能为空"
        )
    if mapping.source == "fixed" and mapping.value is None:
        result.errors.append(
            f"steps[{i}].tool_params.{pname}: source=fixed 时 value 不
能为 None"
        )
    if mapping.source == "user_input" and not mapping.extraction:
        result.errors.append(
            f"steps[{i}].tool_params.{pname}: source=user_input 时
extraction 不能为空"
        )
    if not skill.tools:
        result.warnings.append("tools 为空—Skill 未声明任何可用工具")
    else:
        tool_names = {t.name for t in skill.tools}
        for i, step in enumerate(skill.steps):
            if step.tool_name and step.tool_name not in tool_names:
                result.warnings.append(
                    f"steps[{i}].tool_name='{step.tool_name}' 不在 tools 白名单中"
                )
        result.is_valid = len(result.errors) == 0
    return result

```

文件编号: 136 文件路径: emily-core\emily_core\tools\business_flow_tools.py

```

import logging
from dataclasses import dataclass, field
from typing import Callable, Any
logger = logging.getLogger("emily.tools.business_flow")
@dataclass
class BusinessFlowTool:
    name: str
    description: str
    parameters: dict
    handler: Callable
    category: str = "base"
    permission_flag: str = "all"
class BusinessFlowToolRegistry:
    def __init__(self):
        self._tools: dict[str, BusinessFlowTool] = {}
    def register(self, tool: BusinessFlowTool) -> None:
        if tool.name in self._tools:
            raise ValueError(f"BusinessFlowTool '{tool.name}' 已注册")
        self._tools[tool.name] = tool
        logger.debug("BusinessFlowTool registered: %s", tool.name)

```

```
def get(self, name: str) -> BusinessFlowTool | None:
    return self._tools.get(name)
def has(self, name: str) -> bool:
    return name in self._tools
def list_names(self) -> list[str]:
    return list(self._tools.keys())
def get_tools_schema(self, names: list[str]) -> list[dict]:
    schemas = []
    for name in names:
        tool = self._tools.get(name)
        if tool is not None:
            schemas.append({
                "name": tool.name,
                "description": tool.description,
                "parameters": tool.parameters,
            })
    return schemas
def __len__(self) -> int:
    return len(self._tools)
def __contains__(self, name: str) -> bool:
    return name in self._tools
```

文件编号: 137 文件路径: emily-core\emily_core\tools\chat_archive_tool.py

```
import logging
from .definitions import ToolDefinition
from ..services.chat_archive_service import ChatArchiveService
logger = logging.getLogger(__name__)
def create_chat_archive_tool(chat_archive_service) -> ToolDefinition:
    async def execute(args: dict) -> dict:
        action = args.get("action", "search").strip()
        if action == "search":
            keyword = args.get("keyword", "").strip()
            if not keyword:
                return {"success": False, "error": "请提供搜索关键词"}
            time_range = args.get("time_range", "all")
            limit = min(int(args.get("limit", 20)), 50)
            conversation_id = args.get("conversation_id")
            messages = chat_archive_service.search_messages(
                keyword=keyword,
                conversation_id=conversation_id or None,
                time_range=time_range,
                limit=limit,
            )
            return {
                "success": True,
                "total": len(messages),
                "messages": [
                    {
                        "id": m.id,
                        "sender_name": m.sender_name,
                        "direction": getattr(m, "direction", "user_to_agent"),
                    }
                ]
            }
    return ToolDefinition(
        name="chat_archive_tool",
        description="chat archive tool",
        parameters={
            "action": {"type": "string", "required": True, "enum": ["search"]},
            "keyword": {"type": "string", "required": False},
            "time_range": {"type": "string", "required": False, "enum": ["all", "1h", "24h", "7d", "30d"]},
            "limit": {"type": "integer", "required": False, "minimum": 1, "maximum": 50},
            "conversation_id": {"type": "string", "required": False},
        },
        execute=execute,
    )
```



```
        "content": (m.content or "")[:300],
        "created_at": str(m.created_at) if m.created_at else "",
    }
    for m in messages
],
}
elif action == "history":
    conversation_id = args.get("conversation_id", "").strip()
    if not conversation_id:
        return {"success": False, "error": "请提供会话ID"}
    (conversation_id) "}"
    limit = min(int(args.get("limit", 50)), 100)
    include_progress = bool(args.get("include_progress", False))
    messages = chat_archive_service.get_conversation_history(
        conversation_id=conversation_id,
        limit=limit,
        include_progress=include_progress,
    )
    return {
        "success": True,
        "total": len(messages),
        "messages": [
            {
                "id": m.id,
                "sender_name": m.sender_name,
                "direction": getattr(m, "direction", "user_to_agent"),
                "content": (m.content or "")[:300],
                "created_at": str(m.created_at) if m.created_at else "",
            }
            for m in messages
        ],
    }
elif action == "user":
    user_name = args.get("user_name", "").strip()
    user_id = args.get("user_id", "").strip()
    if not user_name and not user_id:
        return {"success": False, "error": "请提供用户名 (user_name) 或用户ID (user_id) "}"
    limit = min(int(args.get("limit", 20)), 50)
    messages = chat_archive_service.search_messages(
        keyword=user_name if user_name else "",
        user_id=user_id if user_id else None,
        limit=limit,
    )
    return {
        "success": True,
        "total": len(messages),
        "messages": [
            {
                "id": m.id,
                "sender_name": m.sender_name,
                "direction": getattr(m, "direction", "user_to_agent"),
                "content": (m.content or "")[:300],
                "created_at": str(m.created_at) if m.created_at else "",
            }
            for m in messages
        ],
    }
```

```

        }
        for m in messages
    ],
    }
else:
    return {
        "success": False,
        "error": f"不支持的操作类型: {action}。支持: search / history /
user",
    }
return ToolDefinition(
    name="chat_archive",
    description=(
        "检索聊天记录档案。支持三种操作: \n"
        "- action='search': 全文搜索消息 (参数: keyword, time_range, limit,
conversation_id) \n"
        "- action='history': 查看会话完整对话历史 (参数: conversation_id, limit,
include_progress) \n"
        "- action='user': 查看用户历史消息 (参数: user_name 或 user_id, limit)"
    ),
    parameters={
        "type": "object",
        "properties": {
            "action": {
                "type": "string",
                "description": "操作类型: search (全文搜索) / history (会话历
史) / user (用户历史)",
                "enum": ["search", "history", "user"],
            },
            "keyword": {
                "type": "string",
                "description": "搜索关键词 (action=search时使用)",
            },
            "conversation_id": {
                "type": "string",
                "description": "会话ID (action=history时使用)",
            },
            "user_name": {
                "type": "string",
                "description": "用户名 (action=user时使用, 与user_id二选一)",
            },
            "user_id": {
                "type": "string",
                "description": "用户ID (action=user时使用, 与user_name二选一)",
            },
            "time_range": {
                "type": "string",
                "description": "时间范围: today / week / month / all (默认
all)",
            },
            "limit": {
                "type": "integer",
                "description": "返回条数上限 (默认20, 最大50)",
            },
        },
    },

```

```

        "include_progress": {
            "type": "boolean",
            "description": "是否包含前导消息（action=history时，默认
false）",
        },
    },
    "required": ["action"],
},
execute=execute,
)

```

文件编号：138 文件路径：emily-core\emily_core\tools\definitions.py

```

from dataclasses import dataclass
from typing import Any, Awaitable, Callable
@dataclass
class ToolDefinition:
    name: str
    description: str
    parameters: dict
    execute: Callable[[dict[str, Any]], Awaitable[dict[str, Any]]]
    require_admin: bool = False

```

文件编号：139 文件路径：emily-core\emily_core\tools\email_tool.py

```

from __future__ import annotations
import logging
import os
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from ..config import Config
    from ..services.email_service import EmailService
logger = logging.getLogger(__name__)
_SEND_EMAIL_SCHEMA = {
    "type": "object",
    "properties": {
        "to": {
            "type": "string",
            "description": "收件人邮箱地址",
        },
        "subject": {
            "type": "string",
            "description": "邮件主题",
        },
        "body": {
            "type": "string",
            "description": "邮件正文（支持 Markdown）",
        },
    },
    "credential_source": {

```

```

        "type": "string",
        "enum": ["user_memory", "env"],
        "description": "user_memory=从用户长期记忆中获取凭证；env=使用环境变量中的
默认邮箱凭证。默认 env。",
    },
},
"required": ["to", "subject", "body"],
}
_FETCH_INBOX_SCHEMA = {
    "type": "object",
    "properties": {
        "credential_source": {
            "type": "string",
            "enum": ["user_memory", "env"],
            "description": "凭证来源。默认 env。",
        },
        "limit": {
            "type": "integer",
            "description": "最多返回邮件数，默认 10。",
        },
    },
    "required": [],
}
def create_send_email_tool(email_service: "EmailService", config: "Config" =
None):
    from emily_core.tools.definitions import ToolDefinition
    async def execute(args: dict) -> dict:
        to = (args.get("to", "") or "").strip()
        subject = (args.get("subject", "") or "").strip()
        body = (args.get("body", "") or "").strip()
        credential_source = args.get("credential_source", "env") or "env"
        if not to or "@" not in to:
            return {"success": False, "error": "收件人邮箱地址无效"}
        if not subject:
            return {"success": False, "error": "邮件主题不能为空"}
        if not body:
            return {"success": False, "error": "邮件正文不能为空"}
        creds = _get_credentials(credential_source, config)
        if creds is None:
            return {"success": False, "error": "无法获取邮箱凭证。请设置
EMILY_EMAIL_IDKEY 和 EMILY_EMAIL_PASSWORD 环境变量。"}
        try:
            result = await email_service.send(
                creds=creds,
                to=to,
                subject=subject,
                body=body,
                html=False,
            )
            if result.success:
                return {
                    "success": True,
                    "message": f"邮件已发送至 {to}",
                    "message_id": result.message_id or "",

```

```

    }
    else:
        return {"success": False, "error": result.error or "发送失败"}
except Exception as e:
    logger.warning("send_email tool failed: %s", e)
    return {"success": False, "error": f"邮件发送异常: {e}"}
return ToolDefinition(
    name="send_email",
    description=(
        "发送邮件。需要调用者提供邮箱凭证（默认从环境变量获取）。"
        "当用户要求发送邮件、发通知、发报告时使用此工具。"
    ),
    parameters=_SEND_EMAIL_SCHEMA,
    execute=execute,
    require_admin=False,
)
def create_fetch_inbox_tool(email_service: "EmailService", config: "Config" =
None):
    from emily_core.tools.definitions import ToolDefinition
    async def execute(args: dict) -> dict:
        credential_source = args.get("credential_source", "env") or "env"
        limit = args.get("limit", 10) or 10
        creds = _get_credentials(credential_source, config)
        if creds is None:
            return {"success": False, "error": "无法获取邮箱凭证。请设置
EMILY_EMAIL_IDKEY 和 EMILY_EMAIL_PASSWORD 环境变量。"}
        try:
            envelopes = await email_service.fetch_inbox(
                creds=creds,
                unread_only=True,
                limit=min(limit, 50),
            )
            if not envelopes:
                return {"success": True, "message": "收件箱中没有未读邮件。",
"count": 0}
            lines = [f"收件箱中共 {len(envelopes)} 封未读邮件: "]
            for i, env in enumerate(envelopes, 1):
                sender = env.sender or "(未知)"
                date_str = env.date.strftime("%m-%d %H:%M") if env.date else "(无
日期)"
                subject = env.subject or "(无主题)"
                preview = env.body_plain[:100] + "..." if len(env.body_plain) >
100 else env.body_plain
                lines.append(f"\n{i}. **{subject}**")
                lines.append(f"    发件人: {sender} | {date_str}")
                lines.append(f"    预览: {preview}")
            return {"success": True, "message": "\n".join(lines), "count":
len(envelopes)}
        except Exception as e:
            logger.warning("fetch_inbox tool failed: %s", e)
            return {"success": False, "error": f"邮件查询异常: {e}"}
    return ToolDefinition(
        name="fetch_inbox",
        description=(

```

```

        "检查邮箱收件箱，获取未读邮件列表。"
        "当用户要求查邮件、查收件箱、看看有没有新邮件时使用此工具。"
    ),
    parameters=_FETCH_INBOX_SCHEMA,
    execute=execute,
    require_admin=False,
)
def _get_credentials(source: str, config: "Config" = None):
    from ..providers.email.base import EmailCredentials
    if source == "user_memory":
        logger.debug("credential_source=user_memory 暂不支持, fallback 到 env")
        username = os.getenv("EMILY_EMAIL_IDKEY", "")
        password = os.getenv("EMILY_EMAIL_PASSWORD", "")
        if not username or not password:
            return None
        smtp_host = getattr(config, "email_smtp_host", "smtp.qq.com") if config else "smtp.qq.com"
        smtp_port = getattr(config, "email_smtp_port", 465) if config else 465
        imap_host = getattr(config, "email_imap_host", "imap.qq.com") if config else "imap.qq.com"
        imap_port = getattr(config, "email_imap_port", 993) if config else 993
        return EmailCredentials(
            smtp_host=smtp_host,
            smtp_port=smtp_port,
            imap_host=imap_host,
            imap_port=imap_port,
            username=username,
            password=password,
            use_ssl=True,
        )

```

文件编号：140 文件路径：emily-core\emily_core\tools\event_tool.py

```

import json
import logging
from ..adapters.standard.result import RouteResult
from ..application.event_app import EventApplication
logger = logging.getLogger("emily.tool.event")
_EVENT_TOOL_SCHEMA = {
    "type": "object",
    "properties": {
        "project_name": {
            "type": "string",
            "description": "项目名称（如 '未来城'），可选",
        },
        "project_id": {
            "type": "string",
            "description": "项目 UUID，可选",
        },
        "data": {
            "type": "object",
            "description": "事件详细参数",
        },
    },
}

```

```

        "properties": {
            "title": {
                "type": "string",
                "description": "事件简述（10字以内）",
            },
            "event_type": {
                "type": "string",
                "enum": [
                    "construction_progress",
                    "inspection",
                    "material_arrival",
                    "quality_issue",
                    "safety_issue",
                    "weather",
                    "design_change",
                    "decision",
                    "general",
                ],
                "description": "事件类型（decision=决策事件，用于处理待解决问
题）",
            },
            "event_date": {
                "type": "string",
                "description": "事件日期（YYYY-MM-DD 格式）",
            },
            "description": {
                "type": "string",
                "description": "事件完整描述",
            },
        },
        "required": ["title", "event_type"],
    },
    "force": {
        "type": "boolean",
        "description": "是否强制录入（跳过核验，默认 false）。核验不通过且用户坚持
录入时设为 true。",
    },
    "guardian_notes": {
        "type": "string",
        "description": "守护核验发现的问题描述（force=true 时填写，将标记在事件备
注中）",
    },
    "related_event_ids": {
        "type": "array",
        "items": {"type": "string"},
        "description": "关联事件编号列表（如 ['EVT-20260612-0001']）",
    },
},
"required": ["data"],
}
_EVENT_TOOL_DESCRIPTION = (
    "记录一个工程项目现场事件。\\n"
    "⚠ 调用前必须完成拟录入单流程（见系统指令第10条）。\\n"
    "\\n"

```

```

"字段分级: \n"
" [必有] title - 事件简述 (10字以内) \n"
" [应有] event_type - construction_progress/inspection/material_arrival/"
"quality_issue/safety_issue/weather/design_change/general\n"
" [应有] event_date - 发生日期 YYYY-MM-DD (默认今天) \n"
" [应有] project_name - 关联项目名称\n"
" [可有] description - 事件完整描述\n"
" [可有] related_event_ids - 关联事件编号列表\n"
"\n"
"守护核验三选一 (仅在核验不通过时): \n"
" force=false - 正常录入 (核验不通过时会返回 needs_review 信号) \n"
" force=true + guardian_notes - 坚持录入 (自动标记异常备注 + 写入待解决清单) "
)
async def handle_record_event(
    params: dict,
    event_app: EventApplication,
    user_id: str = "",
    message_id: str = "",
    pending_issues=None,
    config=None,
) -> dict:
    data = params.get("data", {})
    force = params.get("force", False)
    guardian_notes = params.get("guardian_notes", "")
    related_event_ids = params.get("related_event_ids", [])
    if not data and ("title" in params or "event_type" in params):
        data = {
            "title": params.get("title", "未命名事件"),
            "event_type": params.get("event_type", "general"),
            "event_date": params.get("event_date"),
            "description": params.get("description", ""),
        }
        logger.debug("event_tool: flat params detected, auto-wrapped into data dict")
    route_result = RouteResult(
        intent="event_record",
        project_name=params.get("project_name"),
        project_id=params.get("project_id"),
        data={
            "title": data.get("title", "未命名事件"),
            "event_type": data.get("event_type", "general"),
            "event_date": data.get("event_date"),
            "description": data.get("description", ""),
            "related_event_ids": related_event_ids,
            "_conversation_id": params.get("_conversation_id", ""),
        },
    )
    result = await event_app.handle_event(route_result, user_id, message_id)
    if force and guardian_notes and result.success:
        try:
            from ..infrastructure.database.session import get_session
            from ..infrastructure.database.models import Event
            with get_session() as session:
                event = session.query(Event).filter(Event.id ==

```



```

result.object_id).first()
    if event:
        original_remarks = event.remarks or ""
        event.remarks = (
            original_remarks
            + ("\n" if original_remarks else "")
            + f"[守护标记] 核验发现: {guardian_notes}"
        )
        session.commit()
        logger.info("M8a guardian mark added to event %s",
result.object_id)
    except Exception as e:
        logger.warning("M8a failed to add guardian mark to event: %s", e)
    reply_text = result.reply or ""
    if force and guardian_notes and result.success and pending_issues is not None:
        try:
            issue_id = pending_issues.add(
                raised_by=user_id or "用户",
                source=f"录入事件「{data.get('title', '未命名')}」时守护核验发现异
常",
                description=f"核验发现: {guardian_notes}。用户坚持录入。",
                suggestion="请核实并给出处理意见。",
                related_events=[result.object_id] if result.object_id else [],
            )
            reply_text = (reply_text or "") + f"\n📄 已记录待处理: {issue_id}"
            logger.info("M8a pending issue created: %s", issue_id)
        except Exception as e:
            logger.warning("M8a failed to create pending issue: %s", e)
    return {
        "success": result.success,
        "object_type": result.object_type,
        "object_id": result.object_id,
        "reply": reply_text,
        "pending_confirmation": result.pending_confirmation,
        "error_code": result.error_code,
        "needs_review": False,
    }

```

文件编号: 141 文件路径: emily-core\emily_core\tools\file_tool.py

```

import logging
import os
from pathlib import Path
from ..adapters.standard.result import RouteResult
from ..application.file_app import FileApplication
logger = logging.getLogger("emily.tool.file")
_FILE_TOOL_SCHEMA = {
    "type": "object",
    "properties": {
        "project_name": {
            "type": "string",
            "description": "项目名称, 可选",

```

```

    },
    "project_id": {
        "type": "string",
        "description": "项目 UUID, 可选",
    },
    "data": {
        "type": "object",
        "description": "文件详细参数",
        "properties": {
            "filename": {
                "type": "string",
                "description": "文件名（如 '施工图纸v3.pdf'）",
            },
            "file_type": {
                "type": "string",
                "description": "文件类型（如 pdf、docx、图纸）",
            },
            "file_category": {
                "type": "string",
                "description": "文件业务分类: PROJECT_LICENSE(项目证  
照)/CONTRACT(承包合同)/WORK_RECORD(工作记录)/PHASE_DELIVERABLE(阶段成  
果)/PROCESS_DOC(过程文件)/MANAGEMENT_SPEC(管理规程)/OTHER(其他文件)",
                "default": "OTHER",
            },
        },
    },
    "required": ["filename"],
},
"force": {
    "type": "boolean",
    "description": "是否强制录入（跳过核验, 默认 false）",
},
"guardian_notes": {
    "type": "string",
    "description": "守护核验发现的问题描述（force=true 时填写）",
},
},
"required": ["data"],
}
_FILE_TOOL_DESCRIPTION = (
    "记录一个文件信息。\\n"
    "⚠️ 调用前必须完成拟录入单流程（见系统指令第10条）。\\n"
    "\\n"
    "字段分级: \\n"
    " [必有] filename - 文件名\\n"
    " [应有] project_name - 关联项目名称\\n"
    " [应有] file_type - 文件类型（从后缀推断, 如 pdf/docx/图纸）\\n"
    "\\n"
    "守护核验三选一（仅在核验不通过时）: \\n"
    " force=false - 正常录入（核验不通过时会返回 needs_review 信号）\\n"
    " force=true + guardian_notes - 坚持录入（自动写入待解决清单）"
)
async def handle_record_file(
    params: dict,
    file_app: FileApplication,

```

```

    user_id: str = "",
    message_id: str = "",
    pending_issues=None,
    config=None,
) -> dict:
    data = params.get("data") or {}
    if not data:
        data = {k: v for k, v in params.items()
                if not k.startswith("_") and k not in ("data", "force",
"guardian_notes",
"project_name",
"project_id")}}
    force = params.get("force", False)
    guardian_notes = params.get("guardian_notes", "")
    attachment_url = params.get("_attachment_url", "")
    attachment_type = params.get("_attachment_type", 0)
    source_filename = data.get("filename", "")
    route_result = RouteResult(
        intent="file_record",
        project_name=params.get("project_name"),
        project_id=params.get("project_id"),
        data={
            "filename": source_filename or "未命名文件",
            "file_type": data.get("file_type", ""),
            "file_category": data.get("file_category", "OTHER"),
        },
    )
    result = await file_app.handle_file(
        route_result, user_id, message_id,
        attachment_url=attachment_url,
        attachment_type=attachment_type,
        source_filename=source_filename,
    )
    reply_text = result.reply or ""
    if force and guardian_notes and result.success and pending_issues is not None:
        try:
            issue_id = pending_issues.add(
                raised_by=user_id or "用户",
                source=f"录入文件「{data.get('filename', '未命名')}」时守护核验发现异常",
                description=f"核验发现: {guardian_notes}。用户坚持录入。",
                suggestion="请核实并给出处理意见。",
                related_events=[],
            )
            reply_text = (reply_text or "") + f"\n📄 已记录待处理: {issue_id}"
            logger.info("M8a pending issue created: %s", issue_id)
        except Exception as e:
            logger.warning("M8a failed to create pending issue: %s", e)
    return {
        "success": result.success,
        "object_type": result.object_type,
        "object_id": result.object_id,
        "reply": reply_text,
        "error_code": result.error_code,
    }

```

```

        "needs_review": False,
    }
_QUERY_FILES_SCHEMA = {
    "type": "object",
    "properties": {
        "file_category": {
            "type": "string",
            "description": "按分类过滤：
PROJECT_LICENSE/CONTRACT/WORK_RECORD/PHASE_DELIVERABLE/PROCESS_DOC/MANAGEMENT_SPEC
/OTHER",
        },
        "keyword": {
            "type": "string",
            "description": "按文件名关键词搜索（模糊匹配）",
        },
        "project_id": {
            "type": "string",
            "description": "项目 UUID（可选，默认取当前用户项目）",
        },
        "limit": {
            "type": "integer",
            "description": "返回数量上限，默认 10",
            "default": 10,
        },
    },
}

_QUERY_FILES_DESCRIPTION = (
    "按分类或关键词查询项目文件列表。\\n"
    "\\n"
    "字段分级：\\n"
    " [应有] file_category - 按业务分类过滤（如 CONTRACT 查承包合同类）\\n"
    " [应有] keyword - 按文件名关键词模糊搜索\\n"
    " [可选] project_id - 指定项目范围\\n"
    " [可选] limit - 返回数量上限"
)

async def handle_query_files(
    params: dict,
    file_app: FileApplication,
    user_id: str = "",
    message_id: str = "",
    project_ids: list[str] | None = None,
    **kwargs,
) -> dict:
    file_category = params.get("file_category")
    keyword = params.get("keyword", "")
    project_id = params.get("project_id")
    limit = params.get("limit", 10)
    result = await file_app.handle_list_by_category(
        file_category=file_category,
        project_id=project_id,
        project_ids=project_ids,
        keyword=keyword,
        limit=limit,
    )

```

```

    return {
        "success": result.success,
        "reply": result.reply,
        "data": getattr(result, 'data', {}),
        "error_code": result.error_code,
    }
_UPDATE_CATEGORY_SCHEMA = {
    "type": "object",
    "properties": {
        "file_no": {
            "type": "string",
            "description": "文件编号（如 FIL-20260709-0001）",
        },
        "file_category": {
            "type": "string",
            "description": "目标分类：
PROJECT_LICENSE/CONTRACT/WORK_RECORD/PHASE_DELIVERABLE/PROCESS_DOC/MANAGEMENT_SPEC
/OTHER",
        },
    },
    "required": ["file_no", "file_category"],
}
_UPDATE_CATEGORY_DESCRIPTION = (
    "修改文件的分类归属。\\n"
    "\\n"
    "必填字段：\\n"
    "  file_no - 文件编号\\n"
    "  file_category - 目标分类枚举值"
)
async def handle_update_file_category(
    params: dict,
    file_app: FileApplication,
    user_id: str = "",
    message_id: str = "",
    **kwargs,
) -> dict:
    file_no = params.get("file_no", "")
    file_category = params.get("file_category", "OTHER")
    if not file_no:
        return {"success": False, "reply": "请提供文件编号", "error_code":
"missing_file_no"}
    result = await file_app.handle_update_category(
        file_no=file_no,
        file_category=file_category,
        user_id=user_id,
    )
    return {
        "success": result.success,
        "reply": result.reply,
        "data": getattr(result, 'data', {}),
        "error_code": result.error_code,
    }

```

文件编号：142 文件路径：emily-core\emily_core\tools\knowledge_search_tool.py

```
import logging
import time as _time
from ..providers.rag.base import RagProvider
logger = logging.getLogger("emily.tool.knowledge_search")
_KNOWLEDGE_SEARCH_SCHEMA = {
    "type": "object",
    "properties": {
        "query": {
            "type": "string",
            "description": "自然语言检索关键词或问题",
        },
        "top_k": {
            "type": "integer",
            "description": "返回结果条数，默认 5，最大 10",
        },
        "stage": {
            "type": "string",
            "description": (
                "按项目阶段过滤，可选：投资决策、专项审查、规划报建、"
                "招标采购、施工建设、竣工验收、交付售后"
            ),
        },
        "role": {
            "type": "string",
            "description": (
                "按岗位过滤，如'工程部经理'、'设计部经理'、"
                "'规划报建专员'、'成本部经理'等"
            ),
        },
    },
    "required": ["query"],
}

_KNOWLEDGE_SEARCH_DESCRIPTION = (
    "搜索知识库获取项目相关的领域知识。"
    "适用场景：查询规范标准、施工工艺、政策法规、项目文档等。"
    "参数 query 为自然语言查询，top_k 控制返回条数（默认 5，最大 10）。"
    "可选参数 stage 按项目阶段过滤（如'投资决策'、'规划报建'、'施工建设'、'竣工验收'等），"
    "可选参数 role 按岗位过滤（如'工程部经理'、'设计部经理'、'规划报建专员'等）。"
)

async def handle_knowledge_search(
    params: dict,
    rag_provider: RagProvider,
) -> dict:
    query = params.get("query", "").strip()
    if not query:
        return {"success": False, "reply": "请提供查询关键词 query"}
    top_k = min(int(params.get("top_k", 5)), 10)
    stage = params.get("stage", None)
    role = params.get("role", None)
    if not await rag_provider.is_available():
```

```

        return {
            "success": False,
            "reply": "知识库暂不可用，请稍后重试",
        }
    t_start = _time.monotonic()
    try:
        response = await rag_provider.search(
            query, top_k=top_k,
            stage=stage, role=role,
        )
        elapsed_ms = int((_time.monotonic() - t_start) * 1000)
        chunks = [
            {
                "content": r.content,
                "score": r.score,
                "doc_name": r.source_document,
            }
            for r in response.results
        ]
        reply = response.context_text or "未找到相关知识"
        return {
            "success": True,
            "query": response.query,
            "total": response.total,
            "reply": reply,
            "rag_results_data": {
                "query": response.query,
                "provider": response.provider_name,
                "chunks": chunks,
                "hit_count": response.total,
                "elapsed_ms": elapsed_ms,
            },
        }
    except Exception as e:
        logger.error("knowledge_search handler failed: %s", e, exc_info=True)
        return {"success": False, "reply": f"知识库检索失败: {e}"}

```

文件编号：143 文件路径：emily-core\emily_core\tools\meeting_tool.py

```

import logging
from ..adapters.standard.result import RouteResult
from ..application.meeting_app import MeetingApplication
logger = logging.getLogger("emily.tool.meeting")
_MEETING_TOOL_SCHEMA = {
    "type": "object",
    "properties": {
        "project_name": {
            "type": "string",
            "description": "项目名称，可选",
        },
        "project_id": {
            "type": "string",

```

```

        "description": "项目 UUID, 可选",
    },
    "data": {
        "type": "object",
        "description": "会议详细参数",
        "properties": {
            "title": {
                "type": "string",
                "description": "会议名称或主题",
            },
            "summary": {
                "type": "string",
                "description": "会议摘要/纪要内容",
            },
            "attendees": {
                "type": "array",
                "items": {"type": "string"},
                "description": "参会人员名单（字符串数组）",
            },
        },
        "required": ["title"],
    },
    "force": {
        "type": "boolean",
        "description": "是否强制录入（跳过核验，默认 false）",
    },
    "guardian_notes": {
        "type": "string",
        "description": "守护核验发现的问题描述（force=true 时填写）",
    },
},
"required": ["data"],
}

_MEETING_TOOL_DESCRIPTION = (
    "记录一场会议。\\n"
    "⚠ 调用前必须完成拟录入单流程（见系统指令第10条）。\\n"
    "\\n"
    "字段分级：\\n"
    " [必有] title - 会议名称或主题\\n"
    " [应有] summary - 会议摘要/纪要内容\\n"
    " [应有] attendees - 参会人员（字符串数组）\\n"
    " [应有] project_name - 关联项目名称\\n"
    "\\n"
    "守护核验三选一（仅在核验不通过时）：\\n"
    " force=false - 正常录入（核验不通过时会返回 needs_review 信号）\\n"
    " force=true + guardian_notes - 坚持录入（自动写入待解决清单）"
)

async def handle_record_meeting(
    params: dict,
    meeting_app: MeetingApplication,
    user_id: str = "",
    message_id: str = "",
    pending_issues=None,
    config=None,

```



```

) -> dict:
    data = params.get("data") or {}
    if not data:
        data = {k: v for k, v in params.items()
                 if not k.startswith("_") and k not in ("data", "force",
"guardian_notes",
                                                                    "project_name",
"project_id")}}
    force = params.get("force", False)
    guardian_notes = params.get("guardian_notes", "")
    route_result = RouteResult(
        intent="meeting_record",
        project_name=params.get("project_name"),
        project_id=params.get("project_id"),
        data={
            "title": data.get("title", "未命名会议"),
            "summary": data.get("summary", ""),
            "attendees": data.get("attendees") or [],
        },
    )
    result = await meeting_app.handle_meeting(route_result, user_id, message_id)
    reply_text = result.reply or ""
    if force and guardian_notes and result.success and pending_issues is not None:
        try:
            issue_id = pending_issues.add(
                raised_by=user_id or "用户",
                source=f"录入会议「{data.get('title', '未命名')}」时守护核验发现异
常",
                description=f"核验发现: {guardian_notes}。用户坚持录入。",
                suggestion="请核实并给出处理意见。",
                related_events=[],
            )
            reply_text = (reply_text or "") + f"\n📝 已记录待处理: {issue_id}"
            logger.info("M8a pending issue created: %s", issue_id)
        except Exception as e:
            logger.warning("M8a failed to create pending issue: %s", e)
    return {
        "success": result.success,
        "object_type": result.object_type,
        "object_id": result.object_id,
        "reply": reply_text,
        "error_code": result.error_code,
        "needs_review": False,
    }

```

文件编号: 144 文件路径: emily-core\emily_core\tools\memory_tool.py

```

import logging
from typing import Optional
from .definitions import ToolDefinition
logger = logging.getLogger("emily.tool.memory")
def create_memory_tool(user_memory_service) -> ToolDefinition:

```

```
service = user_memory_service
async def execute(args: dict) -> dict:
    content = args.get("content", "")
    title = args.get("title", "")
    if not service or not service.enabled:
        return {
            "success": False,
            "message": "长期记忆服务未启用",
        }
    user_name = ""
    user_name = args.get("user_name", "")
    if not user_name:
        user_id = args.get("_user_id", "")
        if user_id:
            try:
                from ..repositories.user_repo import UserRepository
                u = UserRepository.get(user_id)
                if u:
                    user_name = u.username or ""
            except Exception:
                pass
        if not user_name:
            logger.warning("write_user_memory: cannot resolve user_name,
            _user_id=%s", args.get("_user_id", "?"))
            return {
                "success": False,
                "message": "无法确定用户名，记忆未写入",
            }
        try:
            result_title = service.save_memory(
                user_name=user_name,
                content=content,
                title=title,
            )
            if result_title:
                logger.info(
                    "write_user_memory: user=%s, title=%s", user_name,
                    result_title,
                )
                return {
                    "success": True,
                    "message": f"已记录长期工作要求: {result_title}",
                    "title": result_title,
                }
            else:
                return {
                    "success": False,
                    "message": "记忆写入失败",
                }
        except Exception as e:
            logger.error("write_user_memory failed: %s", e, exc_info=True)
            return {
                "success": False,
                "message": f"记忆写入异常: {e}",
```

```

    }
    return ToolDefinition(
        name="write_user_memory",
        description=(
            "将用户的长期工作要求写入记忆文件。"
            "当用户表达明显的长期或持续性需求时使用，例如："
            "'随时跟踪...','每天检查...','以后...','今后...','"
            "'每周...','定期...'等。"
            "不用于记录一次性对话或临时查询。"
        ),
        parameters={
            "type": "object",
            "properties": {
                "content": {
                    "type": "string",
                    "description": "用户的长期工作要求描述，清晰完整地记录",
                },
                "title": {
                    "type": "string",
                    "description": "记忆标题，可选，为空时从内容截取前30字",
                },
            },
            "required": ["content"],
        },
        execute=execute,
    )

```

文件编号：145 文件路径：emily-core\emily_core\tools\node_task_tool.py

```

from __future__ import annotations
import asyncio
import logging
from typing import Any
logger = logging.getLogger("emily.tools.node_task")
async def handle_create_task_node(
    params: dict[str, Any],
    node_service=None,
    user_id: str = "",
    **kwargs,
) -> dict:
    if node_service is None:
        return {"success": False, "reply": "NodeService 未初始化"}
    from ..services.node_commands import CreateTaskNodeCommand
    cmd = CreateTaskNodeCommand(
        project_id=params.get("project_id", ""),
        node_name=params.get("title", params.get("node_name", "")),
        responsible_user_id=params.get("executor_id",
params.get("responsible_user_id", "")),
        deadline=params.get("deadline_at", ""),
        parent_node_id=params.get("parent_node_id", params.get("node_id", "")),
        owner_dept_id=params.get("owner_dept_id", "项目总"),
        description=params.get("description", ""),

```

```

        creator_id=user_id,
    )
    result = await node_service.create_node(cmd)
    return {
        "success": result.success,
        "object_type": "node",
        "object_id": result.node_id,
        "reply": result.message,
    }
async def handle_submit_node_deliverable(
    params: dict[str, Any],
    node_service=None,
    user_id: str = "",
    **kwargs,
) -> dict:
    if node_service is None:
        return {"success": False, "reply": "NodeService 未初始化"}
    from ..services.node_commands import SubmitNodeDeliverableCommand
    cmd = SubmitNodeDeliverableCommand(
        deliverable_id=params.get("deliverable_id", ""),
        content=params.get("content", ""),
        file_url=params.get("file_url", ""),
        file_name=params.get("file_name", ""),
        attachment_file_id=params.get("attachment_file_id", ""),
        submitted_by=user_id,
        is_acceptance_check=params.get("is_acceptance_check", False),
    )
    result = await node_service.submit_deliverable(cmd)
    return {"success": result.success, "reply": result.message}
async def handle_confirm_node_deliverable(
    params: dict[str, Any],
    node_service=None,
    user_id: str = "",
    **kwargs,
) -> dict:
    if node_service is None:
        return {"success": False, "reply": "NodeService 未初始化"}
    from ..services.node_commands import ConfirmNodeDeliverableCommand
    cmd = ConfirmNodeDeliverableCommand(
        deliverable_id=params.get("deliverable_id", ""),
        confirmed_by=user_id,
        reason=params.get("reason", ""),
    )
    result = await node_service.confirm_deliverable(cmd)
    return {"success": result.success, "reply": result.message}
async def handle_return_node_deliverable(
    params: dict[str, Any],
    node_service=None,
    user_id: str = "",
    **kwargs,
) -> dict:
    if node_service is None:
        return {"success": False, "reply": "NodeService 未初始化"}
    from ..services.node_commands import ReturnNodeDeliverableCommand

```

```

cmd = ReturnNodeDeliverableCommand(
    deliverable_id=params.get("deliverable_id", ""),
    returned_by=user_id,
    reason=params.get("reason", ""),
)
result = await node_service.return_deliverable(cmd)
return {"success": result.success, "reply": result.message}
async def handle_query_my_nodes(
    params: dict[str, Any],
    node_service=None,
    user_id: str = "",
    **kwargs,
) -> dict:
    if node_service is None:
        return {"success": False, "reply": "NodeService 未初始化"}
    from ..repositories.node_repo import ProjectNodeRepo
    nodes = await asyncio.to_thread(
        ProjectNodeRepo.find_by_responsible_user,
        user_id,
        project_id=params.get("project_id") or None,
        node_type=params.get("node_type") or None,
        status="IN_PROGRESS",
        limit=int(params.get("limit", 20)),
    )
    items = [
        {
            "node_id": n.node_id,
            "node_name": n.node_name,
            "node_type": getattr(n, "node_type", ""),
            "status": n.status,
            "deadline": n.deadline,
            "project_id": n.project_id,
        }
        for n in nodes
    ]
    return {"success": True, "reply": f"找到 {len(items)} 个节点", "data": items}

```

文件编号：146 文件路径：emily-core\emily_core\tools\node_tool.py

```

from __future__ import annotations
import logging
from typing import Any
logger = logging.getLogger("emily.tools.node_tool")
_CREATE_NODE_SCHEMA = {
    "type": "object",
    "properties": {
        "project_id": {"type": "string", "description": "项目归属ID"},
        "node_id": {"type": "string", "description": "节点编号（业务主键），如 SG-  
JG-01-2026。单节点模式必填"},
        "node_name": {"type": "string", "description": "节点名称/工作项描述。单节点  
模式必填"},
        "deadline": {"type": "string", "description": "截止时间（ISO8601格式）。单节点"}
    }
}

```

```

点模式必填"},
    "owner_dept_id": {"type": "string", "description": "主责条线/部门，默认'项目总'"},
    "stage_id": {"type": "integer", "description": "阶段ID: 0=立项 1=规划 2=施工 3=交付，默认0"},
    "parent_node_id": {"type": "string", "description": "父节点编号（如果是子节点则填写）"},
    "remark": {"type": "string", "description": "备注/说明"},
    "nodes": {
        "type": "array",
        "description": "批量创建模式：节点树列表。每项含node_id/node_name/deadline/deliverables/dependencies/children",
        "items": {"type": "object"},
    },
},
"required": ["project_id"],
}

_CREATE_NODE_DESCRIPTION = (
    "创建项目全景节点。支持两种模式："
    "1) 单节点：提供 node_id+node_name+deadline 创建单个节点；"
    "2) 批量创建：提供 nodes 列表，一次性创建节点树（含成果、依赖、子节点）。"
    "批量模式下 nodes 中每个节点可含 deliverables/dependencies/children 字段。"
    "创建后节点初始状态为 CONDITIONS_NOT_MET（条件不足），需进一步添加成果和依赖。"
)

_QUERY_NODE_SCHEMA = {
    "type": "object",
    "properties": {
        "node_id": {"type": "string", "description": "节点编号（业务主键）"},
    },
    "required": ["node_id"],
}

_QUERY_NODE_DESCRIPTION = (
    "查询节点详情。返回节点的状态、进度、成果列表、依赖关系、子节点等信息。"
)

_UPDATE_PROGRESS_SCHEMA = {
    "type": "object",
    "properties": {
        "deliverable_id": {"type": "string", "description": "成果编号"},
        "current_amount": {"type": "number", "description": "当前完成量"},
        "file_id": {"type": "string", "description": "关联文件ID（上传文件后获得）"},
    },
    "required": ["deliverable_id", "current_amount"],
}

_UPDATE_PROGRESS_DESCRIPTION = (
    "更新节点成果进度。更新后自动触发状态机重算："
    "当所有必需成果 100% 完成且前置依赖满足时，节点自动流转至 COMPLETED（已完成）。"
    "当有阻塞条件（权重999）未满足时，节点回退至 CONDITIONS_NOT_MET。"
)

_ADD_DEPENDENCY_SCHEMA = {
    "type": "object",
    "properties": {
        "node_id": {"type": "string", "description": "下游节点编号（需要等待的节点）"},
    },
}

```

```

        "depends_on_deliverable_id": {"type": "string", "description": "依赖的上游成果编号"},
        "weight": {"type": "number", "description": "权重（0.0000-1.0000，阻塞场景用≥999）"},
    },
    "required": ["node_id", "depends_on_deliverable_id"],
}

_ADD_DEPENDENCY_DESCRIPTION = (
    "为节点添加前置依赖。节点需等待上游成果完成才能启动。"
    "系统自动进行循环依赖检测（BFS），非法依赖会被拒绝。"
    "设置 weight≥999 可创建人工阻塞条件。"
)

_MOUNT_CHILD_SCHEMA = {
    "type": "object",
    "properties": {
        "parent_node_id": {"type": "string", "description": "父节点编号"},
        "child_node_id": {"type": "string", "description": "子节点编号"},
        "child_weight": {"type": "number", "description": "子节点权重（0.0-1.0），默认为1.0"},
    },
    "required": ["parent_node_id", "child_node_id"],
}

_MOUNT_CHILD_DESCRIPTION = (
    "将子节点挂载到父节点。父节点进度由子节点进度加权汇总。"
    "嵌套深度上限3层，子节点上限100个。"
    "系统自动检测父子循环。"
)

async def handle_create_node(
    params: dict[str, Any],
    user_id: str = "",
    message_id: str = "",
    **kw,
) -> dict[str, Any]:
    batch_nodes = params.get("nodes")
    if batch_nodes and isinstance(batch_nodes, list):
        from emily_core.services.node_batch import create_node_tree
        results = await create_node_tree(
            project_id=params.get("project_id", ""),
            creator_id=user_id,
            nodes=batch_nodes,
            auto_activate=True,
            dry_run=False,
        )
        success_count = sum(1 for r in results if r.get("success"))
        fail_count = sum(1 for r in results if not r.get("success"))
        return {
            "success": fail_count == 0,
            "batch": True,
            "total": len(results),
            "success_count": success_count,
            "fail_count": fail_count,
            "results": results,
            "message": f"批量创建完成: {success_count} 成功, {fail_count} 失败",
        }
    }

```

```

from emily_core.services.node_commands import CreateNodeCommand
from emily_core.services.node_service import NodeService
from emily_core.repositories.permission_repo import PermissionRepository
svc = NodeService(user_repo=PermissionRepository())
cmd = CreateNodeCommand(
    project_id=params.get("project_id", ""),
    node_id=params.get("node_id", ""),
    node_name=params.get("node_name", ""),
    deadline=params.get("deadline", ""),
    owner_dept_id=params.get("owner_dept_id", "项目总"),
    stage_id=params.get("stage_id", 0),
    parent_node_id=params.get("parent_node_id", ""),
    remark=params.get("remark", ""),
    creator_id=user_id,
)
result = await svc.create_node(cmd)
return {
    "success": result.success,
    "node_id": result.node_id,
    "status": result.status,
    "message": result.message,
}
}

async def handle_query_node(
    params: dict[str, Any],
    user_id: str = "",
    message_id: str = "",
    **kw,
) -> dict[str, Any]:
    from emily_core.services.node_service import NodeService
    from emily_core.repositories.permission_repo import PermissionRepository
    svc = NodeService(user_repo=PermissionRepository())
    node_id = params.get("node_id", "")
    detail = await svc.get_node_detail(node_id)
    if detail is None:
        return {"success": False, "message": f"节点 {node_id} 不存在"}
    return {
        "success": True,
        "data": detail,
        "message": f"节点「{detail['node_name']}」当前状态: {detail['status']}, 进
度: {detail['progress']}%",
    }
}

async def handle_update_node_progress(
    params: dict[str, Any],
    user_id: str = "",
    message_id: str = "",
    **kw,
) -> dict[str, Any]:
    from emily_core.services.node_commands import UpdateDeliverableProgressCommand
    from emily_core.services.node_service import NodeService
    from emily_core.repositories.permission_repo import PermissionRepository
    svc = NodeService(user_repo=PermissionRepository())
    cmd = UpdateDeliverableProgressCommand(
        deliverable_id=params.get("deliverable_id", ""),
        current_amount=float(params.get("current_amount", 0)),
    )

```



```

        file_id=params.get("file_id", ""),
        operator_id=user_id,
    )
    result = await svc.update_deliverable_progress(cmd)
    return {
        "success": result.success,
        "node_id": result.node_id,
        "status": result.status,
        "progress": result.progress,
        "message": result.message,
        "affected_ancestors": result.affected_downstream,
    }
async def handle_add_node_dependency(
    params: dict[str, Any],
    user_id: str = "",
    message_id: str = "",
    **kw,
) -> dict[str, Any]:
    from emily_core.services.node_commands import AddDependencyCommand
    from emily_core.services.node_service import NodeService
    from emily_core.repositories.permission_repo import PermissionRepository
    svc = NodeService(user_repo=PermissionRepository())
    cmd = AddDependencyCommand(
        node_id=params.get("node_id", ""),
        depends_on_deliverable_id=params.get("depends_on_deliverable_id", ""),
        weight=float(params.get("weight", 1.0)),
        operator_id=user_id,
    )
    result = await svc.add_dependency(cmd)
    return {
        "success": result.success,
        "node_id": result.node_id,
        "message": result.message,
        "error_code": result.error_code,
    }
async def handle_mount_child_node(
    params: dict[str, Any],
    user_id: str = "",
    message_id: str = "",
    **kw,
) -> dict[str, Any]:
    from emily_core.services.node_commands import MountChildCommand
    from emily_core.services.node_service import NodeService
    from emily_core.repositories.permission_repo import PermissionRepository
    svc = NodeService(user_repo=PermissionRepository())
    cmd = MountChildCommand(
        parent_node_id=params.get("parent_node_id", ""),
        child_node_id=params.get("child_node_id", ""),
        child_weight=float(params.get("child_weight", 1.0)),
        operator_id=user_id,
    )
    result = await svc.mount_child(cmd)
    return {
        "success": result.success,

```

```

        "node_id": result.node_id,
        "message": result.message,
        "error_code": result.error_code,
    }
    _UPDATE_NODES_SCHEMA = {
        "type": "object",
        "properties": {
            "updates": {
                "type": "array",
                "description": "节点更新列表。每项含 node_id（必填）+ 要更新的字段（node_name/deadline/owner_dept_id/remark/stage_id 等）",
                "items": {"type": "object"},
            },
        },
        "required": ["updates"],
    }
    _UPDATE_NODES_DESCRIPTION = (
        "批量更新节点字段。每项指定 node_id + 要修改的字段（只填要改的），"
        "支持："
        "node_name/deadline/owner_dept_id/related_company_id/remark/stage_id/sort_order。"
    )
    _ACTIVATE_NODES_SCHEMA = {
        "type": "object",
        "properties": {
            "node_ids": {
                "type": "array",
                "description": "要激活（审批通过）的节点编号列表",
                "items": {"type": "string"},
            },
            "approved": {
                "type": "boolean",
                "description": "True=审批通过，False=审批拒绝（默认 True）",
            },
            "remark": {"type": "string", "description": "审批备注"},
        },
        "required": ["node_ids"],
    }
    _ACTIVATE_NODES_DESCRIPTION = (
        "批量激活（审批通过/拒绝）节点。审批通过：NOT_ACTIVATED → CONDITIONS_NOT_MET。"
        "审批拒绝：节点废弃。需部门负责人或 L5+ 管理员权限。"
    )
    _DISCARD_NODES_SCHEMA = {
        "type": "object",
        "properties": {
            "node_ids": {
                "type": "array",
                "description": "要废弃的节点编号列表",
                "items": {"type": "string"},
            },
        },
        "required": ["node_ids"],
    }
    _DISCARD_NODES_DESCRIPTION = (
        "批量废弃节点。已完成子节点的父节点不可废弃。废弃为软删除。"
    )

```

```
)
async def handle_update_nodes(
    params: dict[str, Any],
    user_id: str = "",
    message_id: str = "",
    **kw,
) -> dict[str, Any]:
    from emily_core.services.node_batch_update import batch_update_nodes
    updates = params.get("updates", [])
    if not updates:
        return {"success": False, "message": "updates 列表为空"}
    results = await batch_update_nodes(
        updates=updates,
        operator_id=user_id,
    )
    success_count = sum(1 for r in results if r.get("success"))
    fail_count = sum(1 for r in results if not r.get("success"))
    return {
        "success": fail_count == 0,
        "total": len(results),
        "success_count": success_count,
        "fail_count": fail_count,
        "results": results,
        "message": f"批量更新完成: {success_count} 成功, {fail_count} 失败",
    }
async def handle_activate_nodes(
    params: dict[str, Any],
    user_id: str = "",
    message_id: str = "",
    **kw,
) -> dict[str, Any]:
    from emily_core.services.node_batch_update import batch_activate_nodes
    node_ids = params.get("node_ids", [])
    if not node_ids:
        return {"success": False, "message": "node_ids 列表为空"}
    results = await batch_activate_nodes(
        node_ids=node_ids,
        approver_id=user_id,
        approved=params.get("approved", True),
        remark=params.get("remark", ""),
    )
    success_count = sum(1 for r in results if r.get("success"))
    fail_count = sum(1 for r in results if not r.get("success"))
    return {
        "success": fail_count == 0,
        "total": len(results),
        "success_count": success_count,
        "fail_count": fail_count,
        "results": results,
        "message": f"批量审批完成: {success_count} 成功, {fail_count} 失败",
    }
async def handle_discard_nodes(
    params: dict[str, Any],
    user_id: str = "",
```

```

        message_id: str = "",
        **kw,
    ) -> dict[str, Any]:
        from emily_core.services.node_batch_update import batch_discard_nodes
        node_ids = params.get("node_ids", [])
        if not node_ids:
            return {"success": False, "message": "node_ids 列表为空"}
        results = await batch_discard_nodes(
            node_ids=node_ids,
            operator_id=user_id,
        )
        success_count = sum(1 for r in results if r.get("success"))
        fail_count = sum(1 for r in results if not r.get("success"))
        return {
            "success": fail_count == 0,
            "total": len(results),
            "success_count": success_count,
            "fail_count": fail_count,
            "results": results,
            "message": f"批量废弃完成: {success_count} 成功, {fail_count} 失败",
        }

```

文件编号: 147 文件路径: emily-core\emily_core\tools\node_voice_entry_tool.py

```

from __future__ import annotations
import json
import logging
import hashlib
import re
from dataclasses import dataclass, field
logger = logging.getLogger("emily.tool.node_voice_entry")
VOICE_NODE_SCHEMA = {
    "type": "object",
    "properties": {
        "action": {
            "type": "string",
            "enum": ["create_node", "add_child", "add_deliverable",
"add_dependency", "query", "unknown"],
            "description": "用户意图"
        },
        "extracted_data": {
            "type": "object",
            "properties": {
                "node_name": {"type": "string", "description": "节点名称/工作项描述"},
                "deadline": {"type": "string", "description": "截止时间 (ISO8601)"},
                "owner_dept_id": {"type": "string", "description": "主责部门/线条"},
                "parent_node_id": {"type": "string", "description": "父节点ID (如果是子节点)"},
                "stage_id": {"type": "integer", "description": "阶段ID"},

```

```

        "deliverable_name": {"type": "string", "description": "成果/产出物
名称"},
        "deliverable_amount": {"type": "number", "description": "成果数
量"},
        "deliverable_unit": {"type": "string", "description": "成果单位"},
        "depends_on_node": {"type": "string", "description": "依赖于哪个节
点的成果"},
    },
    },
    "missing_fields": {
        "type": "array",
        "items": {"type": "string"},
        "description": "缺失的必要字段（如 ['deadline', 'owner_dept_id']）"
    },
    "follow_up_question": {
        "type": "string",
        "description": "用自然语言询问用户缺失的信息"
    },
    "confidence": {
        "type": "number",
        "description": "提取置信度 0.0-1.0"
    },
    },
    "required": ["action", "extracted_data", "confidence"],
}

@dataclass
class VoiceEntryState:
    user_id: str = ""
    project_id: str = ""
    draft: dict = field(default_factory=dict)
    step: str = "idle"
    created_node_ids: list[str] = field(default_factory=list)
    _voice_states: dict[str, VoiceEntryState] = {}
    def get_voice_state(user_id: str) -> VoiceEntryState:
        if user_id not in _voice_states:
            _voice_states[user_id] = VoiceEntryState(user_id=user_id)
        return _voice_states[user_id]
    async def voice_parse_input(user_text: str, user_id: str = "", project_id: str =
    "") -> dict:
        try:
            from ..providers.llm_client import chat_json
            messages = [
                {"role": "system", "content": VOICE_ENTRY_SYSTEM_PROMPT},
                {"role": "user", "content": f"当前项目: {project_id}\n用户输入:
{user_text}"},
            ]
            result = await chat_json(
                messages=messages,
                schema=VOICE_NODE_SCHEMA,
                temperature=0.1,
            )
            if result is None:
                return {
                    "action": "unknown",

```

```

        "extracted": {},
        "missing": [],
        "follow_up": "抱歉，我没能理解您的意思。请再说一遍您要录入的节点信息？",
        "confidence": 0.0,
    }
    return {
        "action": result.get("action", "unknown"),
        "extracted": result.get("extracted_data", {}),
        "missing": result.get("missing_fields", []),
        "follow_up": result.get("follow_up_question", ""),
        "confidence": result.get("confidence", 0.0),
    }
except Exception as e:
    logger.error("voice_parse_input error: %s", e)
    return {
        "action": "unknown",
        "extracted": {},
        "missing": [],
        "follow_up": "解析出错，请稍后重试。",
        "confidence": 0.0,
    }
}

async def voice_execute_create(user_id: str, project_id: str, extracted: dict) -> dict:
    try:
        from ..services.node_commands import CreateNodeCommand
        from ..services.node_service import NodeService
        from ..repositories.permission_repo import PermissionRepository
        node_id = _generate_node_id(extracted.get("node_name", ""), project_id)
        svc = NodeService(user_repo=PermissionRepository())
        cmd = CreateNodeCommand(
            project_id=project_id,
            node_id=node_id,
            node_name=extracted.get("node_name", "未命名节点"),
            deadline=extracted.get("deadline", ""),
            owner_dept_id=extracted.get("owner_dept_id", "项目总"),
            stage_id=extracted.get("stage_id", 0),
            creator_id=user_id,
        )
        result = await svc.create_node(cmd)
        if result.success:
            state = get_voice_state(user_id)
            state.created_node_ids.append(node_id)
            return {
                "success": True,
                "node_id": node_id,
                "message": f"节点「{cmd.node_name}」创建成功（编号：{node_id}）。"
                           f"您可以继续添加子节点、成果或依赖。",
            }
        else:
            return {"success": False, "node_id": "", "message": result.message}
    except Exception as e:
        return {"success": False, "node_id": "", "message": str(e)}

def _generate_node_id(node_name: str, project_id: str) -> str:

```

```

clean = re.sub(r'^[-\u00a0a-zA-Z0-9]', '', node_name)
hash_part = hashlib.md5(clean.encode()).hexdigest()[4].upper()
return f"NODE-{hash_part}"

```

文件编号: 148 文件路径: emily-core\emily_core\tools\pending_issue_tool.py

```

from .definitions import ToolDefinition
from ..services.pending_issues import PendingIssuesService
def create_pending_issue_tool(
    pending_issues: PendingIssuesService,
    is_admin: bool = False,
) -> ToolDefinition:
    async def execute(args: dict) -> dict:
        action = args.get("action", "list_pending")
        if action == "list_pending":
            items = pending_issues.list_pending()
            if not items:
                return {"success": True, "reply": "当前没有待解决的问题。", "items":
[]
            lines = [f"共 {len(items)} 条待处理: "]
            for item in items:
                lines.append(
                    f"  {item.get('id', '?')} · {item.get('提出人', '?')} · "
                    f"{item.get('问题描述', '?')[:50]}"
                )
            return {"success": True, "reply": "\n".join(lines), "items": items}
        elif action == "list_resolved":
            items = pending_issues.list_resolved()
            if not items:
                return {"success": True, "reply": "没有已处理的记录。", "items":
[]
            lines = [f"最近已处理 {len(items)} 条: "]
            for item in items[:10]:
                handler = item.get("处理人", "?")
                decision = item.get("决策", "?")[:50]
                lines.append(f"  {item.get('id', '?')} · 处理人: {handler} ·
{decision}")
            return {"success": True, "reply": "\n".join(lines), "items": items}
        elif action == "add":
            raised_by = args.get("raised_by", "用户")
            source = args.get("source", "")
            description = args.get("description", "")
            suggestion = args.get("suggestion", "")
            related_events = args.get("related_events", [])
            if not description:
                return {
                    "success": False,
                    "error_code": "missing_description",
                    "reply": "请描述需要记录的问题。",
                }
            if raised_by == "用户" or not raised_by:
                user_id = args.get("_user_id", "")

```

```
        if user_id:
            try:
                from ..repositories.user_repo import UserRepository
                u = UserRepository.get(user_id)
                if u:
                    raised_by = (
                        u.username or
                        "用户"
                    )
                except Exception:
                    pass
            issue_id = pending_issues.add(
                raised_by=raised_by,
                source=source or "用户通过对话添加",
                description=description,
                suggestion=suggestion,
                related_events=related_events if isinstance(related_events, list)
            else [],
        )
        return {
            "success": True,
            "reply": f"已记录待解决问题 {issue_id}",
            "issue_id": issue_id,
        }
    elif action == "resolve":
        if not is_admin:
            return {
                "success": False,
                "error_code": "permission_denied",
                "reply": "只有项目经理（管理员）可以处理待解决问题。",
            }
        issue_id = args.get("issue_id", "")
        if not issue_id:
            return {"success": False, "error_code": "missing_id", "reply": "请指定要处理的问题编号（如 PND-20260612-0001）。"}
        handler = args.get("handler", "管理员")
        decision = args.get("decision", "")
        decision_event_id = args.get("decision_event_id", "")
        if not decision:
            return {"success": False, "error_code": "missing_decision",
                    "reply": "请说明处理决策。"}
        ok = pending_issues.resolve(
            issue_id=issue_id,
            handler=handler,
            decision=decision,
            decision_event_id=decision_event_id,
        )
        if ok:
            return {
                "success": True,
                "reply": f"已处理 {issue_id}: {decision}",
            }
        else:
            return {
```



```

        "success": False,
        "error_code": "not_found",
        "reply": f"未找到待解决问题 {issue_id}。",
    }
    else:
        return {"success": False, "error_code": "unknown_action", "reply":
f"不支持的操作: {action}"}
    return ToolDefinition(
        name="manage_pending_issues",
        description=(
            "管理待解决问题清单。\\n"
            "操作类型: \\n"
            "  list_pending - 列出所有待处理问题\\n"
            "  list_resolved - 列出最近已处理的问题\\n"
            "  add - 添加新的待解决问题, 需 description; 可选
raised_by/source/suggestion/related_events\\n"
            "  resolve - 标记问题为已处理 (需管理员权限), 需 issue_id、handler、
decision"
        ),
        parameters={
            "type": "object",
            "properties": {
                "action": {
                    "type": "string",
                    "enum": ["list_pending", "list_resolved", "add", "resolve"],
                    "description": "操作类型",
                },
                "issue_id": {
                    "type": "string",
                    "description": "问题编号 (resolve 时必填, 如 PND-20260612-
0001)",
                },
                "raised_by": {
                    "type": "string",
                    "description": "提出人姓名 (add 时填写, 如'彭工'、'守护Agent')",
                },
                "source": {
                    "type": "string",
                    "description": "问题来源描述 (add 时填写)",
                },
                "description": {
                    "type": "string",
                    "description": "问题详细描述 (add 时必填)",
                },
                "suggestion": {
                    "type": "string",
                    "description": "建议处理方式 (add 时可选)",
                },
                "related_events": {
                    "type": "array",
                    "items": {"type": "string"},
                    "description": "关联事件编号列表 (add 时可选)",
                },
                "handler": {

```

```

        "type": "string",
        "description": "处理人姓名 (resolve 时填写)",
    },
    "decision": {
        "type": "string",
        "description": "处理决策描述 (resolve 时必填)",
    },
    "decision_event_id": {
        "type": "string",
        "description": "决策事件编号 (可选, 如 EVT-20260612-0015)",
    },
    },
    "required": ["action"],
},
execute=execute,
require_admin=(not is_admin),
)

```

文件编号: 149 文件路径: emily-core\emily_core\tools\query_tool.py

```

import logging
from ..adapters.standard.command import QueryCommand
from ..services.query_service import QueryService
logger = logging.getLogger("emily.tool.query")
_QUERY_TOOL_SCHEMA = {
    "type": "object",
    "properties": {
        "query_type": {
            "type": "string",
            "enum": [
                "event", "task", "meeting", "file",
                "message", "conversation", "user",
                "project", "summary",
            ],
            "description": "查询类型",
        },
        "project_id": {
            "type": "string",
            "description": "项目 UUID, 可选",
        },
        "project_name": {
            "type": "string",
            "description": "项目名称, 可选",
        },
        "time_range": {
            "type": "string",
            "enum": ["today", "this_week", "this_month", "all"],
            "description": "时间范围, 默认 all",
        },
        "status_filter": {
            "type": "string",
            "description": "按状态筛选 (event: pending/confirmed/cancelled, task:

```

```

todo/doing/done) ",
    },
    "assignee": {
        "type": "string",
        "description": "按负责人筛选（仅 task 类型）",
    },
    "sender_name": {
        "type": "string",
        "description": "按发送者筛选（仅 message 类型）",
    },
    "keyword": {
        "type": "string",
        "description": "关键词搜索（仅 message 类型）",
    },
    "intent": {
        "type": "string",
        "description": "按意图筛选（仅 message 类型）",
    },
    "file_type": {
        "type": "string",
        "description": "按文件类型筛选（仅 file 类型）",
    },
    "conversation_id": {
        "type": "string",
        "description": "按会话 ID 筛选（仅 message 类型）",
    },
    "limit": {
        "type": "integer",
        "description": "返回结果上限，默认 50",
    },
},
"required": ["query_type"],
}

_QUERY_TOOL_DESCRIPTION = (
    "查询项目管理系统中的数据。支持 9 种查询类型：\n"
    "- event: 事件查询（支持按项目、时间范围、状态筛选）\n"
    "- task: 任务查询（支持按项目、时间、状态、负责人筛选）\n"
    "- meeting: 会议查询\n"
    "- file: 文件查询（支持按文件类型筛选）\n"
    "- message: 通讯记录查询（支持按发送者、关键词、意图筛选）\n"
    "- conversation: 活跃会话排行\n"
    "- user: 用户列表\n"
    "- project: 项目列表\n"
    "- summary: 全局统计概览（事件数/任务数/消息数等）\n"
    "时间范围可选 today / this_week / this_month / all（默认 all）。"
)

_QUERY_TYPE_TO_TABLE = {
    "event": "events", "task": "tasks", "meeting": "meetings",
    "file": "files", "message": "messages", "conversation": "conversations",
    "user": "users", "project": "projects", "journal": "events",
    "summary": "summary",
}

async def handle_query_data(
    params: dict,

```

```

        query_service: QueryService,
    ) -> dict:
        try:
            session_scope = params.pop("_session_scope", None) or {}
            query_type = params.get("query_type", "event")
            target_table = _QUERY_TYPE_TO_TABLE.get(query_type, "")
            db_perms = session_scope.get("db_perms", {})
            if db_perms and target_table and target_table not in db_perms:
                logger.info("query_data: db_perms denied table=%s for query_type=%s",
                    target_table, query_type)
                return {"success": False, "reply": f"无权限查询{target_table}",
                    "total": 0}
            project_ids = session_scope.get("project_ids", [])
            if project_ids and not params.get("project_id"):
                params["project_ids"] = project_ids
            cmd = QueryCommand(
                query_type=params.get("query_type", "event"),
                project_id=params.get("project_id"),
                project_ids=params.get("project_ids"),
                project_name=params.get("project_name"),
                time_range=params.get("time_range", "all"),
                status_filter=params.get("status_filter"),
                assignee=params.get("assignee"),
                sender_name=params.get("sender_name"),
                keyword=params.get("keyword"),
                intent=params.get("intent"),
                file_type=params.get("file_type"),
                conversation_id=params.get("conversation_id"),
                limit=params.get("limit", 50),
            )
            logger.info(
                "query_data tool: type=%s, project=%s, time=%s",
                cmd.query_type, cmd.project_id, cmd.time_range,
            )
            results = query_service.execute(cmd)
            reply = query_service.format_reply(cmd.query_type, results)
            return {
                "success": True,
                "query_type": cmd.query_type,
                "total": results.get("total", 0),
                "reply": reply,
            }
        except Exception as e:
            logger.error("query_data tool failed: %s", e, exc_info=True)
            return {"success": False, "error": str(e)}

```

文件编号: 150 文件路径: emily-core\emily_core\tools\registry.py

```

from __future__ import annotations
import logging
from functools import partial
from typing import TYPE_CHECKING

```

```

if TYPE_CHECKING:
    from ... import EmilyCore
logger = logging.getLogger("emily.tool.registry")
def register_all(core: "EmilyCore") -> None:
    reg = getattr(core, "_business_flow_tools", None)
    if reg is None:
        logger.warning("register_all: _business_flow_tools is None - skip")
        return
    _register_base(core, reg)
    _register_business(core, reg)
    _register_project(core, reg)
    logger.info("registry: %d tools (base=%d, business=%d, project=%d)",
                len(reg), _bc, _buc, _pjc)
_bc, _buc, _pjc = 0, 0, 0
def _tool(name: str, desc: str, params: dict, handler, category: str = "base",
permission_flag: str = "all"):
    from .business_flow_tools import BusinessFlowTool
    return BusinessFlowTool(name=name, description=desc, parameters=params,
handler=handler,
                            category=category, permission_flag=permission_flag)
def _register_base(core, reg):
    global _bc
    _bc = 0
    from .query_tool import handle_query_data
    qs = getattr(core, "_query_service", None)
    if qs is None:
        from ..services.query_service import QueryService
        qs = QueryService()
    reg.register(_tool("query_data", "查询项目数据（事件/任务/会议/文件/消息/用户/项目/日志）",
                        {"type": "object", "properties": {}},
                        partial(handle_query_data, query_service=qs)))
    _bc += 1
    rp = getattr(core, "_rag_provider", None)
    from .knowledge_search_tool import (
        _KNOWLEDGE_SEARCH_SCHEMA, _KNOWLEDGE_SEARCH_DESCRIPTION,
    )
    if rp is not None:
        from .knowledge_search_tool import handle_knowledge_search
        async def _rag(params, **kw):
            return await handle_knowledge_search(params, rag_provider=rp)
        reg.register(_tool("knowledge_search", _KNOWLEDGE_SEARCH_DESCRIPTION,
                            _KNOWLEDGE_SEARCH_SCHEMA, _rag))
    else:
        async def _rag_stub(params, **kw):
            query = (params.get("query", "") or "").strip()
            return {
                "success": False,
                "reply": f"知识库服务暂未就绪（查询: {query}），请稍后重试或联系管理员
检查 RAG 配置。",
            }
        reg.register(_tool("knowledge_search", _KNOWLEDGE_SEARCH_DESCRIPTION,
                            _KNOWLEDGE_SEARCH_SCHEMA, _rag_stub))
        logger.warning("knowledge_search registered with stub handler

```

```

(rag_provider is None)")
    _bc += 1
def _register_business(core, reg):
    global _buc
    _buc = 0
    cfg = core.config
    _buc += _reg_biz(reg, "record_event", "记录项目事件",
                    partial(_h("event_tool", "handle_record_event"),
                            event_app=core._event_app), "business", "write")
    _buc += _reg_biz(reg, "record_task", "创建任务",
                    partial(_h("task_tool", "handle_record_task"),
                            task_app=core._task_app), "business", "write")
    _buc += _reg_biz(reg, "record_meeting", "归档会议纪要",
                    partial(_h("meeting_tool", "handle_record_meeting"),
                            meeting_app=core._meeting_app), "business", "write")
    _buc += _reg_biz(reg, "record_file", "记录文件元数据",
                    partial(_h("file_tool", "handle_record_file"),
                            file_app=core._file_app), "business", "write")
    _buc += _reg_biz(reg, "query_files", "按分类或关键词查询项目文件",
                    partial(_h("file_tool", "handle_query_files"),
                            file_app=core._file_app), "business", "all")
    _buc += _reg_biz(reg, "update_file_category", "修改文件分类归属",
                    partial(_h("file_tool", "handle_update_file_category"),
                            file_app=core._file_app), "business", "write")
    mem = getattr(core, "_user_memory_service", None)
    if mem is not None and not reg.has("write_user_memory"):
        from .memory_tool import create_memory_tool
        bt = create_memory_tool(mem)
        reg.register(_tool(bt.name, bt.description, bt.parameters, bt.execute))
        _buc += 1
def _reg_biz(reg, name, desc, handler, category="business",
permission_flag="write"):
    try:
        reg.register(_tool(name, desc, {"type": "object", "properties": {}},
handler,
                                category=category, permission_flag=permission_flag))
        return 1
    except Exception as e:
        logger.warning("tool '%s' registration failed: %s", name, e)
        return 0
def _h(mod, fn):
    import importlib
    m = importlib.import_module(f".{mod}", package="emily_core.tools")
    return getattr(m, fn)
def _register_project(core, reg):
    global _pjc
    _pjc = 0
    na = getattr(core, "_node_app", None)
    if na is not None:
        try:
            from .node_tool import (
                handle_create_node, handle_query_node,
handle_update_node_progress,
                handle_add_node_dependency, handle_mount_child_node,

```

```

        handle_update_nodes, handle_activate_nodes, handle_discard_nodes,
        _CREATE_NODE_SCHEMA, _CREATE_NODE_DESCRIPTION,
        _QUERY_NODE_SCHEMA, _QUERY_NODE_DESCRIPTION,
        _UPDATE_PROGRESS_SCHEMA, _UPDATE_PROGRESS_DESCRIPTION,
        _ADD_DEPENDENCY_SCHEMA, _ADD_DEPENDENCY_DESCRIPTION,
        _MOUNT_CHILD_SCHEMA, _MOUNT_CHILD_DESCRIPTION,
        _UPDATE_NODES_SCHEMA, _UPDATE_NODES_DESCRIPTION,
        _ACTIVATE_NODES_SCHEMA, _ACTIVATE_NODES_DESCRIPTION,
        _DISCARD_NODES_SCHEMA, _DISCARD_NODES_DESCRIPTION,
    )
    for name, desc, schema, handler in [
        ("create_node", _CREATE_NODE_DESCRIPTION, _CREATE_NODE_SCHEMA,
handle_create_node),
        ("query_node", _QUERY_NODE_DESCRIPTION, _QUERY_NODE_SCHEMA,
handle_query_node),
        ("update_node_progress", _UPDATE_PROGRESS_DESCRIPTION,
_UPDATE_PROGRESS_SCHEMA, handle_update_node_progress),
        ("add_node_dependency", _ADD_DEPENDENCY_DESCRIPTION,
_ADD_DEPENDENCY_SCHEMA, handle_add_node_dependency),
        ("mount_child_node", _MOUNT_CHILD_DESCRIPTION,
_MOUNT_CHILD_SCHEMA, handle_mount_child_node),
        ("update_nodes", _UPDATE_NODES_DESCRIPTION, _UPDATE_NODES_SCHEMA,
handle_update_nodes),
        ("activate_nodes", _ACTIVATE_NODES_DESCRIPTION,
_ACTIVATE_NODES_SCHEMA, handle_activate_nodes),
        ("discard_nodes", _DISCARD_NODES_DESCRIPTION,
_DISCARD_NODES_SCHEMA, handle_discard_nodes),
    ]:
        if not reg.has(name):
            reg.register(_tool(name, desc, schema, handler,
category="project",
permission_flag="admin"))
            _pjc += 1
    except Exception as e:
        logger.warning("node tools registration failed: %s", e)
    ns = getattr(core, "_node_service", None)
    if ns is not None:
        try:
            from .node_task_tool import (
                handle_create_task_node, handle_submit_node_deliverable,
                handle_confirm_node_deliverable,
handle_return_node_deliverable,
                handle_query_my_nodes,
            )
            for name, desc, handler in [
                ("create_task_node", "创建TASK类型叶子节点（替代
record_plan_task）",
                partial(handle_create_task_node, node_service=ns)),
                ("submit_node_deliverable", "提交节点成果（替代
submit_plan_task）",
                partial(handle_submit_node_deliverable, node_service=ns)),
                ("confirm_node_deliverable", "确认节点成果",
                partial(handle_confirm_node_deliverable, node_service=ns)),
                ("return_node_deliverable", "退回节点成果",

```

```

        partial(handle_return_node_deliverable, node_service=ns)),
        ("query_my_nodes", "查询我负责的节点（替代query_plan_tasks）",
        partial(handle_query_my_nodes, node_service=ns)),
    ]:
        if not reg.has(name):
            reg.register(_tool(name, desc, {"type": "object",
"properties": {}}, handler,
                                category="business",
permission_flag="write"))
            _pjc += 1
        except Exception as e:
            logger.warning("node task tools registration failed: %s", e)
    es = getattr(core, "_email_service", None)
    if es is not None:
        _pjc += _reg_email(reg, es, core.config)
    ca = getattr(core, "_chat_archive_service", None)
    if ca is not None:
        _pjc += _reg_chat_archive(reg, ca)
    pi = getattr(core, "_pending_issues_service", None)
    if pi is not None:
        _pjc += _reg_pending(reg, pi)
    from .project import _VOICE_ENTRY_SCHEMA, _VOICE_ENTRY_DESCRIPTION,
handle_voice_entry
    reg.register(_tool("voice_entry", _VOICE_ENTRY_DESCRIPTION,
_VOICE_ENTRY_SCHEMA, handle_voice_entry))
    _pjc += 1
def _reg_email(reg, es, config):
    try:
        from .email_tool import create_send_email_tool, create_fetch_inbox_tool
        from .project import (_SEND_EMAIL_SCHEMA, _SEND_EMAIL_DESCRIPTION,
                               _FETCH_INBOX_SCHEMA, _FETCH_INBOX_DESCRIPTION,
                               handle_send_email, handle_fetch_inbox)
        reg.register(_tool("send_email", _SEND_EMAIL_DESCRIPTION,
_SEND_EMAIL_SCHEMA,
                                partial(handle_send_email, email_service=es,
config=config)))
        reg.register(_tool("fetch_inbox", _FETCH_INBOX_DESCRIPTION,
_FETCH_INBOX_SCHEMA,
                                partial(handle_fetch_inbox, email_service=es,
config=config)))
        return 2
    except Exception as e:
        logger.warning("email tools registration failed: %s", e)
        return 0
def _reg_chat_archive(reg, ca):
    try:
        from .chat_archive_tool import create_chat_archive_tool
        from .project import _CHAT_ARCHIVE_SCHEMA, _CHAT_ARCHIVE_DESCRIPTION,
handle_chat_archive
        reg.register(_tool("chat_archive", _CHAT_ARCHIVE_DESCRIPTION,
_CHAT_ARCHIVE_SCHEMA,
                                partial(handle_chat_archive, chat_archive_service=ca)))
        return 1
    except Exception as e:

```



```

        logger.warning("chat_archive tool registration failed: %s", e)
        return 0
def _reg_pending(reg, pi):
    try:
        from .pending_issue_tool import create_pending_issue_tool
        from .project import _PENDING_ISSUE_SCHEMA, _PENDING_ISSUE_DESCRIPTION,
        handle_manage_pending_issues
        reg.register(_tool("manage_pending_issues", _PENDING_ISSUE_DESCRIPTION,
        _PENDING_ISSUE_SCHEMA,
                        partial(handle_manage_pending_issues,
        pending_issues_service=pi, is_admin=False)))
        return 1
    except Exception as e:
        logger.warning("pending_issue tool registration failed: %s", e)
        return 0

```

文件编号: 151 文件路径: emily-core\emily_core\tools\scripts\search_files.py

```

from __future__ import annotations
import argparse
import json
import os
import subprocess
import sys
from pathlib import Path
SEARCH_FILES_SCHEMA = {
    "type": "object",
    "properties": {
        "query": {
            "type": "string",
            "description": "自然语言描述或关键词，用于匹配文件名或文件一句话描述",
        },
        "top_k": {
            "type": "integer",
            "description": "返回结果数上限，默认 5",
        },
    },
    "required": ["query"],
}
SEARCH_FILES_DISPLAY_NAME = "根据自然语言描述搜索可见文件"
async def handle_search_files(params: dict, **kwargs) -> dict:
    query = params.get("query", "").strip()
    top_k = min(int(params.get("top_k", 5)), 20)
    user_id = params.get("_user_id", "") or kwargs.get("user_id", "")
    if not query:
        return {"success": False, "reply": "请提供搜索关键词"}
    if not user_id:
        return {"success": False, "reply": "用户身份未识别"}
    from ...repositories.session_accessible_file_repo import
    SessionAccessibleFileRepo
    results = SessionAccessibleFileRepo.search(user_id, query, top_k=top_k)
    if not results:

```

```

        return {
            "success": True,
            "total": 0,
            "files": [],
            "reply": f"未找到与「{query}」相关的文件",
        }
    reply_parts = [f"找到 {len(results)} 个相关文件: "]
    for f in results:
        desc = f.get("description", f.get("file_type", ""))
        reply_parts.append(f"  · {f['filename']} ({desc}) ")
    return {
        "success": True,
        "total": len(results),
        "files": results,
        "reply": "\n".join(reply_parts),
    }
def register(core=None):
    from ..business_flow_tools import BusinessFlowTool
    return BusinessFlowTool(
        name="search_files",
        description=SEARCH_FILES_DISPLAY_NAME,
        parameters=SEARCH_FILES_SCHEMA,
        handler=handle_search_files,
    )
def _detect_docker_pg_port() -> int | None:
    try:
        result = subprocess.run(
            ["docker", "port", "emily-postgres", "5432/tcp"],
            capture_output=True, text=True, timeout=5,
        )
        if result.returncode == 0 and result.stdout.strip():
            port_str = result.stdout.strip().rsplit(":", 1)[-1]
            return int(port_str)
    except (FileNotFoundError, subprocess.TimeoutExpired, ValueError, OSError):
        pass
    return None
def _init_db():
    from emily_core.infrastructure.database import init_db
    _HERE = Path(__file__).resolve().parent.parent.parent.parent
    _CORE_DIR = _HERE / "emily-core"
    if str(_CORE_DIR) not in sys.path:
        sys.path.insert(0, str(_CORE_DIR))
    db_url = os.environ.get("EMILY_DATABASE_URL", "")
    if db_url:
        init_db(db_url=db_url)
    else:
        pg_host = os.environ.get("EMILY_PG_HOST", os.environ.get("PG_HOST",
"127.0.0.1"))
        pg_port_env = os.environ.get("EMILY_PG_PORT", os.environ.get("PG_PORT"))
        if pg_port_env:
            pg_port = int(pg_port_env)
        else:
            pg_port = _detect_docker_pg_port() or 5432
        pg_db = os.environ.get("EMILY_PG_DB", "emily")

```

```

pg_user = os.environ.get("EMILY_PG_USER", "emily")
pg_password = os.environ.get("EMILY_PG_PASSWORD", "emily_secret_2026")
init_db(pg_host=pg_host, pg_port=pg_port, pg_db=pg_db, pg_user=pg_user,
pg_password=pg_password)
def main():
    parser = argparse.ArgumentParser(description="搜索可见文件（独立脚本模式）")
    parser.add_argument("query", help="搜索关键词")
    parser.add_argument("--user-id", required=True, help="用户 UUID")
    parser.add_argument("--top-k", type=int, default=5, help="返回结果数上限")
    parser.add_argument("--json", action="store_true", help="JSON 格式输出")
    parser.add_argument("--sync", action="store_true", help="先同步可见文件再搜索")
    args = parser.parse_args()
    _init_db()
    from emily_core.repositories.session_accessible_file_repo import
SessionAccessibleFileRepo
    if args.sync:
        n = SessionAccessibleFileRepo.sync_for_user(
            user_id=args.user_id,
            project_ids=[],
            info_level="internal",
        )
        print(f"[sync] {n} files synced for user {args.user_id}")
        results = SessionAccessibleFileRepo.search(args.user_id, args.query,
top_k=args.top_k)
        if args.json:
            print(json.dumps({"total": len(results), "files": results},
ensure_ascii=False, indent=2))
        else:
            if not results:
                print(f"未找到与「{args.query}」相关的文件")
            else:
                print(f"找到 {len(results)} 个相关文件: ")
                for f in results:
                    desc = f.get("description", f.get("file_type", ""))
                    print(f"    {f['filename']} ({desc})")
if __name__ == "__main__":
    main()

```

文件编号：152 文件路径：emily-core\emily_core\tools\task_tool.py

```

import logging
from ..adapters.standard.result import RouteResult
from ..application.task_app import TaskApplication
logger = logging.getLogger("emily.tool.task")
_TASK_TOOL_SCHEMA = {
    "type": "object",
    "properties": {
        "project_name": {
            "type": "string",
            "description": "项目名称（如 '未来城'），可选",
        },
        "project_id": {

```

```

        "type": "string",
        "description": "项目 UUID, 可选",
    },
    "data": {
        "type": "object",
        "description": "任务详细参数",
        "properties": {
            "title": {
                "type": "string",
                "description": "任务简述（10字以内）",
            },
            "description": {
                "type": "string",
                "description": "任务详细描述",
            },
            "assignee": {
                "type": "string",
                "description": "负责人姓名（如 '张三'），可选",
            },
            "due_date": {
                "type": "string",
                "description": "截止日期（YYYY-MM-DD），可选",
            },
            "due_text": {
                "type": "string",
                "description": "截止日期自然语言描述（如 '下周五前'），可选",
            },
        },
        "required": ["title"],
    },
    "force": {
        "type": "boolean",
        "description": "是否强制录入（跳过核验，默认 false）",
    },
    "guardian_notes": {
        "type": "string",
        "description": "守护核验发现的问题描述（force=true 时填写）",
    },
},
"required": ["data"],
}

_TASK_TOOL_DESCRIPTION = (
    "创建一个工作任务。\\n"
    "⚠️ 调用前必须完成拟录入单流程（见系统指令第10条）。\\n"
    "\\n"
    "字段分级：\\n"
    " [必有] title - 任务简述（10字以内）\\n"
    " [应有] assignee - 负责人姓名\\n"
    " [应有] due_date - 截止日期 YYYY-MM-DD（从'明天/下周五'等短语换算）\\n"
    " [应有] project_name - 关联项目名称\\n"
    " [可有] description - 任务详细描述\\n"
    "\\n"
    "守护核验三选一（仅在核验不通过时）：\\n"
    " force=false - 正常录入（核验不通过时会返回 needs_review 信号）\\n"

```

```

" force=true + guardian_notes - 坚持录入（自动写入待解决清单）"
)
async def handle_record_task(
    params: dict,
    task_app: TaskApplication,
    user_id: str = "",
    message_id: str = "",
    pending_issues=None,
    config=None,
) -> dict:
    data = params.get("data") or {}
    if not data:
        data = {k: v for k, v in params.items()
                if not k.startswith("_") and k not in ("data", "force",
"guardian_notes",
"project_name",
"project_id")}}
    force = params.get("force", False)
    guardian_notes = params.get("guardian_notes", "")
    route_result = RouteResult(
        intent="task_record",
        project_name=params.get("project_name"),
        project_id=params.get("project_id"),
        data={
            "title": data.get("title", "未命名任务"),
            "description": data.get("description", ""),
            "assignee": data.get("assignee") or data.get("owner", ""),
            "due_date": data.get("due_date"),
            "due_text": data.get("due_text", ""),
        },
    )
    result = await task_app.handle_task(route_result, user_id, message_id)
    reply_text = result.reply or ""
    if force and guardian_notes and result.success and pending_issues is not None:
        try:
            issue_id = pending_issues.add(
                raised_by=user_id or "用户",
                source=f"录入任务「{data.get('title', '未命名')}」时守护核验发现异常",
                description=f"核验发现: {guardian_notes}。用户坚持录入。",
                suggestion="请核实并给出处理意见。",
                related_events=[],
            )
            reply_text = (reply_text or "") + f"\n📄 已记录待处理: {issue_id}"
            logger.info("M8a pending issue created: %s", issue_id)
        except Exception as e:
            logger.warning("M8a failed to create pending issue: %s", e)
    return {
        "success": result.success,
        "object_type": result.object_type,
        "object_id": result.object_id,
        "reply": reply_text,
        "error_code": result.error_code,
    }

```

```

        "needs_review": False,
    }

```

文件编号: 153 文件路径: emily-core\emily_core\workitem\injector.py

```

from __future__ import annotations
import logging
from dataclasses import dataclass, field
logger = logging.getLogger("emily.injector")
@dataclass
class InjectionResult:
    new_sops: set[str] = field(default_factory=set)
    new_tools: set[str] = field(default_factory=set)
    new_tables: set[str] = field(default_factory=set)
    @property
    def is_empty(self) -> bool:
        return not (self.new_sops or self.new_tools or self.new_tables)
_KNOWN_TABLES: dict[str, str] = {
    "events": "events(id, event_no, title, event_type, category, event_date,
location, description, payload, project_id, status, created_at)",
    "tasks": "tasks(id, task_no, title, description, status, priority, assignee,
due_date, project_id, created_at)",
    "meetings": "meetings(id, meeting_no, title, meeting_type, meeting_date,
location, attendees, conclusion, action_items, project_id)",
    "files": "files(id, file_no, file_name, file_type, file_path, file_size,
version, confidentiality, project_id)",
    "projects": "projects(id, name, code, lifecycle_stage, city, address,
is_deleted)",
    "users": "users(id, name, role, grouping, position, company)",
    "messages": "messages(id, conversation_id, sender_id, direction, content,
msg_type, created_at)",
}
class KnowledgeInjector:
    def __init__(
        self,
        sop_loader=None,
        max_sop_text_chars: int = 8000,
        max_context_tokens_est: int = 32000,
    ):
        self._loaded_sops: set[str] = set()
        self._loaded_tools: set[str] = set()
        self._loaded_tables: set[str] = set()
        self._sop_texts: dict[str, str] = {}
        self._table_schemas: dict[str, str] = {}
        self._sop_loader = sop_loader
        self._max_sop_text_chars = max_sop_text_chars
        self._max_context_tokens_est = max_context_tokens_est
    def get_skill_instructions(self, sop_id: str) -> str | None:
        return None
    def analyze(self, work_item) -> InjectionResult:
        required_sops = {work_item.sop_id} if work_item.sop_id else set()
        required_tools = set(getattr(work_item, "required_tools", set()) or set())

```

```

        required_tables = set(getattr(work_item, "required_tables", set()) or
set())
        new_sops = required_sops - self._loaded_sops
        new_tools = required_tools - self._loaded_tools
        new_tables = required_tables - self._loaded_tables
        for sop_id in new_sops:
            text = self._load_sop_text(sop_id)
            if text:
                self._sop_texts[sop_id] = text
                self._loaded_sops.add(sop_id)
            else:
                self._loaded_sops.add(sop_id)
        for table_name in new_tables:
            schema = self._load_table_schema(table_name)
            if schema:
                self._table_schemas[table_name] = schema
                self._loaded_tables.add(table_name)
        work_item.injected_sops |= new_sops
        work_item.injected_tools |= new_tools
        work_item.injected_tables |= new_tables
        self._enforce_token_budget()
        if not new_sops and not new_tools and not new_tables:
            logger.debug("Injector: WI %s - all knowledge already loaded",
work_item.id)
        else:
            logger.info(
                "Injector: WI %s - +%d sop, +%d tool, +%d table (loaded:
%d/%d/%d)",
                work_item.id,
                len(new_sops), len(new_tools), len(new_tables),
                len(self._loaded_sops), len(self._loaded_tools),
len(self._loaded_tables),
            )
            return InjectionResult(
                new_sops=new_sops,
                new_tools=new_tools,
                new_tables=new_tables,
            )
    def release(self, work_item) -> None:
        removed = set()
        for sop_id in work_item.injected_sops:
            if sop_id in self._loaded_sops:
                self._loaded_sops.discard(sop_id)
                self._sop_texts.pop(sop_id, None)
                removed.add(sop_id)
        for table_name in work_item.injected_tables:
            if table_name in self._loaded_tables:
                self._loaded_tables.discard(table_name)
                self._table_schemas.pop(table_name, None)
                removed.add(table_name)
        if removed:
            logger.debug("Injector: WI %s released %d items", work_item.id,
len(removed))
    def get_context_text(self) -> str:

```

```

parts = []
if self._sop_texts:
    for sop_id, text in self._sop_texts.items():
        parts.append(f"--- SOP: {sop_id} ---\n{text}")
if self._table_schemas:
    parts.append("--- 可用数据表 ---")
    for name, schema in self._table_schemas.items():
        parts.append(f"- {name}: {schema}")
return "\n\n".join(parts)
def loaded_summary(self) -> dict:
    return {
        "sops": sorted(self._loaded_sops),
        "tables": sorted(self._loaded_tables),
        "sop_texts_count": len(self._sop_texts),
        "table_schemas_count": len(self._table_schemas),
    }
def _load_sop_text(self, sop_id: str) -> str | None:
    if self._sop_loader is None:
        return None
    try:
        text = self._sop_loader.load_full_text(sop_id)
        if text and len(text) > self._max_sop_text_chars:
            text = text[:self._max_sop_text_chars] + "\n\n... (内容已截断)"
        return text
    except Exception as e:
        logger.warning("Failed to load SOP text for %s: %s", sop_id, e)
        return None
@staticmethod
def _load_table_schema(table_name: str) -> str | None:
    return _KNOWN_TABLES.get(table_name)
def _enforce_token_budget(self) -> None:
    total_chars = (
        sum(len(v) for v in self._sop_texts.values()) +
        sum(len(v) for v in self._table_schemas.values())
    )
    est_tokens = total_chars // 3
    if est_tokens > self._max_context_tokens_est:
        logger.warning(
            "KnowledgeInjector: estimated %d tokens exceeds budget %d,
trimming...",
            est_tokens, self._max_context_tokens_est,
        )
        while est_tokens > self._max_context_tokens_est and self._sop_texts:
            oldest = next(iter(self._sop_texts))
            del self._sop_texts[oldest]
            self._loaded_sops.discard(oldest)
            total_chars = (
                sum(len(v) for v in self._sop_texts.values()) +
                sum(len(v) for v in self._table_schemas.values())
            )
            est_tokens = total_chars // 3

```



```

from __future__ import annotations
import json
import logging
import time
from datetime import datetime, timezone
from pathlib import Path
from typing import TYPE_CHECKING
from .hook_registry import HookRegistry
from .hook import HookDecision, HOOK_TYPE_MAP
from ...infrastructure.logging.llm_logger import LLMInteractionLogger
from ...infrastructure.logging.business_event_logger import BusinessEventLogger
if TYPE_CHECKING:
    from .node import PipelineNode
    from .context import BusContext
logger = logging.getLogger("emily.bus")
class PipelineBUS:
    def __init__(self, name: str = "emily_bus"):
        self.name = name
        self._nodes: list["PipelineNode"] = []
        self._hooks = HookRegistry()
    def add_node(self, node: "PipelineNode") -> "PipelineBUS":
        self._nodes.append(node)
        return self
    def add_nodes(self, nodes: list["PipelineNode"]) -> "PipelineBUS":
        self._nodes.extend(nodes)
        return self
    def register_hook(self, mount_point: str, hook) -> "PipelineBUS":
        self._hooks.register(mount_point, hook)
        return self
    def register_hooks_from_config(self, config: dict, **injected_services) ->
None:
        hooks_section = config.get("hooks", {})
        if not hooks_section:
            logger.info("No hooks found in BUS config")
            return
        count = 0
        for mount_point, hook_specs in hooks_section.items():
            if not isinstance(hook_specs, list):
                logger.warning("Invalid hook config at %s: expected list",
mount_point)
                continue
            for spec in hook_specs:
                if not isinstance(spec, dict):
                    continue
                hook = self._build_hook_from_spec(spec, **injected_services)
                if hook:
                    self.register_hook(mount_point, hook)
                    count += 1
        logger.info(
            "BUS registered %d hook(s) at %d mount point(s)",
            count, len(hooks_section),
        )

```

```

def _build_hook_from_spec(self, spec: dict, **injected_services):
    hook_type = spec.get("type", "")
    hook_name = spec.get("name", "unnamed_hook")
    if hook_type not in HOOK_TYPE_MAP:
        logger.warning("Unknown hook type '%s' for '%s'", hook_type,
hook_name)
        return None
    cls = HOOK_TYPE_MAP[hook_type]
    try:
        kwargs: dict = {}
        if hook_type == "auth":
            kwargs["resource_type"] = spec.get("resource_type", "")
            kwargs["action"] = spec.get("action", "")
        elif hook_type == "audit":
            kwargs["event_type"] = spec.get("event_type", "")
        elif hook_type == "trace":
            kwargs["trace_level"] = spec.get("trace_level", "summary")
            if "agent_trace_service" in injected_services:
                kwargs["agent_trace_service"] =
injected_services["agent_trace_service"]
        elif hook_type == "progress":
            if "progress_sender" in injected_services:
                kwargs["progress_sender"] =
injected_services["progress_sender"]
            if "progress_template" in injected_services:
                kwargs["progress_template"] =
injected_services["progress_template"]
            kwargs["enable_progress"] = spec.get("enabled", True)
        elif hook_type == "plan_task_match":
            pass
        kwargs["name"] = hook_name
        kwargs["priority"] = spec.get("priority", 10)
        kwargs["enabled"] = spec.get("enabled", True)
        return cls(**kwargs)
    except Exception as e:
        logger.error("Failed to build hook '%s' (type=%s): %s", hook_name,
hook_type, e)
        return None
def hook_count(self) -> int:
    return self._hooks.hook_count()
async def run(self, context: "BusContext") -> "BusContext":
    wi = context.work_item
    started_at = datetime.now(timezone.utc).isoformat()
    node_timings: dict[str, int] = {}
    LLMInteractionLogger.set_context(
        pipeline_run_id=context.pipeline_run_id,
        conversation_id=context.message.conversation_id if context.message
else "",
        user_id=context.user_id,
    )
    BusinessEventLogger.set_context(
        pipeline_run_id=context.pipeline_run_id,
        conversation_id=context.message.conversation_id if context.message
else "",

```

```

    )
    logger.info(
        "BUS[%s] running WorkItem %s: %d nodes",
        self.name, wi.id if wi else "?", len(self._nodes),
    )
    try:
        for node in self._nodes:
            context.current_stage = node.name
            t_node = time.monotonic()
            if not await self._fire_before_hooks(node.name, context):
                logger.warning("BUS blocked at node %s: %s", node.name,
context.abort_reason)
                context.should_abort = True
                node_timings[node.name] = int((time.monotonic() - t_node) *
1000)

                await self._log_execution(context, started_at, node_timings)
                return context
            try:
                await node.handler(context)
            except Exception as e:
                logger.error("BUS node [%s] failed: %s", node.name, e,
exc_info=True)

                await self._fire_error_hooks(node.name, context, e)
                node_timings[node.name] = int((time.monotonic() - t_node) *
1000)

                if node.required:
                    context.should_abort = True
                    context.abort_reason = str(e)
                    if wi is not None:
                        wi.error_message = str(e)
                    await self._log_execution(context, started_at,
node_timings)

                    return context
                continue
            await self._fire_after_hooks(node.name, context)
            node_timings[node.name] = int((time.monotonic() - t_node) * 1000)
        logger.info("BUS[%s] completed WorkItem %s", self.name, wi.id if wi
else "?")

        await self._log_execution(context, started_at, node_timings)
        return context
    finally:
        LLMInteractionLogger.clear_context()
        BusinessEventLogger.clear_context()
    async def _log_execution(self, context: "BusContext",
                           started_at: str, node_timings: dict) -> None:
        try:
            from ...infrastructure.logging.pipeline_logger import
PipelineExecutionLogger
            await PipelineExecutionLogger.log(context, started_at, node_timings)
        except Exception as e:
            logger.warning("Pipeline execution log write failed: %s", e)
        try:
            from ...infrastructure.logging.legacy_log_bridge import
write_legacy_logs

```

```

        await write_legacy_logs(context, started_at)
    except Exception as e:
        logger.warning("Legacy log bridge write failed: %s", e)
    async def _fire_before_hooks(self, node_name: str, context: "BusContext") ->
bool:
        mount = f"before:{node_name}"
        for hook in self._hooks.get_enabled(mount):
            try:
                result = await hook.execute(context)
                if result.is_blocked:
                    logger.info(
                        "BUS blocked by hook '%s' at %s: %s",
                        hook.name, mount, result.message,
                    )
                    context.abort_reason = result.message
                    return False
                if result.decision == HookDecision.WARN:
                    context.add_warning(result.message)
            except Exception as e:
                logger.error("Before hook '%s' at %s failed: %s", hook.name,
mount, e)
                context.abort_reason = f"鉴权/核验服务异常: {e}"
                return False
        return True
    async def _fire_after_hooks(self, node_name: str, context: "BusContext") ->
None:
        mount = f"after:{node_name}"
        for hook in self._hooks.get_enabled(mount):
            try:
                await hook.execute(context)
            except Exception as e:
                logger.warning(
                    "After hook '%s' at %s failed (non-blocking): %s",
                    hook.name, mount, e,
                )
    async def _fire_error_hooks(self, node_name: str, context: "BusContext",
error: Exception) -> None:
        mount = f"on_error:{node_name}"
        for hook in self._hooks.get_enabled(mount):
            try:
                await hook.execute(context)
            except Exception as e:
                logger.error("Error hook '%s' at %s also failed: %s", hook.name,
mount, e)
        @classmethod
        def build_default(cls, node_handlers: dict, name: str = "emily_bus") ->
"PipelineBUS":
            from .node import PipelineNode
            bus = cls(name=name)
            bus.add_nodes([
                PipelineNode("wi_node1", "意图+拆分", node_handlers["wi_node1"],
required=True),
                PipelineNode("wi_node2", "计划+标准", node_handlers["wi_node2"],
required=True),

```

```

        PipelineNode("wi_node3", "执行+验收", node_handlers["wi_node3"],
required=True),
        PipelineNode("wi_node4", "成果总结", node_handlers["wi_node4"],
required=False),
    ])
    return bus

```

文件编号: 155 文件路径: emily-core\emily_core\workitem\pipeline\context.py

```

from __future__ import annotations
import uuid
from dataclasses import dataclass, field
from typing import Any, TYPE_CHECKING, Optional
if TYPE_CHECKING:
    from ..workitem import WorkItem
    from ...adapters.standard.message import StandardMessage
    from ...session.session_context import SessionContext
def _new_run_id() -> str:
    return str(uuid.uuid4())[:8]
@dataclass
class BusContext:
    work_item: "WorkItem" = None # type: ignore[assignment]
    _session_context: Optional["SessionContext"] = None
    pipeline_run_id: str = field(default_factory=_new_run_id)
    current_stage: str = ""
    message: "StandardMessage" = None # type: ignore[assignment]
    user_id: str = ""
    is_admin: bool = False
    intent: Any = None
    sub_tasks: list[Any] = field(default_factory=list)
    db_message_id: str = ""
    agent_result: Any = None
    agent_reply: str = ""
    verified_reply: str = ""
    verify_warnings: list[str] = field(default_factory=list)
    should_abort: bool = False
    abort_reason: str = ""
    warnings: list[str] = field(default_factory=list)
    baggage: dict[str, Any] = field(default_factory=dict)
    def get_session_context(self) -> Optional["SessionContext"]:
        return self._session_context
    def has_sop_permission(self, sop_id: str) -> bool:
        if self._session_context is None:
            return False
        return self._session_context.has_sop_permission(sop_id)
    def has_db_permission(self, table: str, operation: str = "read") -> bool:
        if self._session_context is None:
            return False
        return self._session_context.has_db_permission(table, operation)
    def meets_grouping_requirement(self, required_grouping: int) -> bool:
        if self._session_context is None:
            return False

```

```

        return self._session_context.meets_grouping_requirement(required_grouping)
    def add_warning(self, msg: str) -> None:
        self.warnings.append(msg)
    def get(self, key: str, default: Any = None) -> Any:
        return self.baggage.get(key, default)
    def set(self, key: str, value: Any) -> None:
        self.baggage[key] = value

```

文件编号: 156 文件路径: emily-core\emily_core\workitem\pipeline\hook.py

```

from __future__ import annotations
import logging
from dataclasses import dataclass, field
from enum import Enum
from typing import Any, TYPE_CHECKING
if TYPE_CHECKING:
    from .context import BusContext as PipelineContext
logger = logging.getLogger("emily.pipeline.hook")
class HookDecision(Enum):
    ALLOW = "allow"
    WARN = "warn"
    BLOCK = "block"
@dataclass
class HookResult:
    decision: HookDecision = HookDecision.ALLOW
    message: str = ""
    metadata: dict = field(default_factory=dict)
    @property
    def is_blocked(self) -> bool:
        return self.decision == HookDecision.BLOCK
    @staticmethod
    def allow() -> "HookResult":
        return HookResult(decision=HookDecision.ALLOW)
    @staticmethod
    def warn(msg: str) -> "HookResult":
        return HookResult(decision=HookDecision.WARN, message=msg)
    @staticmethod
    def block(msg: str) -> "HookResult":
        return HookResult(decision=HookDecision.BLOCK, message=msg)
@dataclass
class Hook:
    name: str
    priority: int = 10
    enabled: bool = True
    async def execute(self, context: "PipelineContext") -> HookResult:
        raise NotImplementedError(f"Hook [{self.name}] must implement execute()")
@dataclass
class AuthHook(Hook):
    resource_type: str = ""
    action: str = ""
    async def execute(self, context: "PipelineContext") -> HookResult:
        user_id = context.user_id

```

```

if not user_id:
    if self.action and self.action not in ("read",):
        logger.info(
            "AuthHook[%s] blocking: no user_id for action=%s res=%s",
            self.name, self.action, self.resource_type,
        )
        return HookResult.block(f"需要登录才能执行 {self.action} 操作")
    return HookResult.allow()
session_ctx = context.get_session_context()
if self.resource_type == "system" and self.action == "execute":
    if session_ctx is None:
        is_admin_flag = getattr(context, "is_admin", False) or
context.get("is_admin", False)
        if not is_admin_flag:
            logger.info("AuthHook[%s] blocking: user %s is not admin",
self.name, user_id)
            result = HookResult.block("仅管理员可执行系统级操作")
            await _log_auth_block(user_id, self.resource_type,
result.message)
            return result
        else:
            from ...permission.level import is_admin as _is_admin
            if not _is_admin(session_ctx.level):
                logger.info(
                    "AuthHook[%s] blocking: user %s level=%d < L5",
                    self.name, user_id, session_ctx.level,
                )
                result = HookResult.block("仅管理员（L5+）可执行系统级操作")
                await _log_auth_block(user_id, self.resource_type,
result.message)
                return result
            intent = context.intent
            sop_id = getattr(intent, "sop_id", None) if intent else None
            if sop_id and session_ctx is not None:
                if sop_id not in session_ctx.sop_allow and "all" not in
session_ctx.sop_allow:
                    logger.info(
                        "AuthHook[%s] blocking: user %s no access to SOP %s",
                        self.name, user_id, sop_id,
                    )
                    reason = f"无权访问 {sop_id}"
                    if session_ctx.supervisor_id:
                        reason += f", 可联系主管 {session_ctx.supervisor_id} 申请权限"
                    result = HookResult.block(reason)
                    await _log_auth_block(user_id, sop_id, reason)
                    return result
                logger.debug("AuthHook[%s] allow: user=%s res=%s action=%s",
                    self.name, user_id, self.resource_type, self.action)
                return HookResult.allow()
async def _log_auth_block(user_id: str, resource: str, reason: str) -> None:
    try:
        from ...permission.row_security import PermissionAuditLogRepository
        repo = PermissionAuditLogRepository()
        repo.log_access_denied(

```

```

        grantee_id=user_id,
        perm_code=f"AUTH-{{resource}}",
        reason=reason,
    )
except Exception as e:
    logger.warning("Auth audit log write failed (non-blocking): %s", e)

@dataclass
class AuditHook(Hook):
    event_type: str = ""
    async def execute(self, context: "PipelineContext") -> HookResult:
        try:
            from ...infrastructure.database.session import get_session
            from ...infrastructure.database.models import HookExecutionLog
            from datetime import datetime, timezone
            import time as _time
            t0 = _time.time()
            run_id = context.pipeline_run_id
            user_id = context.user_id or ""
            sop_id = ""
            session_level = None
            session_ctx = context.get_session_context()
            if session_ctx is not None:
                session_level = session_ctx.level
            intent = context.intent
            if intent:
                sop_id = getattr(intent, "sop_id", "") or ""
            block_reason = ""
            if context.should_abort and context.abort_reason:
                block_reason = context.abort_reason[:500]
            log_entry = HookExecutionLog(
                hook_name=self.name,
                mount_point=f"after:{{context.current_stage}}",
                pipeline_run_id=run_id,
                phase="after",
                decision="block" if block_reason else "allow",
                message=f"audit: {{self.event_type}}",
                duration_ms=int((_time.time() - t0) * 1000),
                metadata_json={},
                user_id=user_id,
                sop_id=sop_id,
                block_reason=block_reason,
                session_level=session_level,
                created_at=datetime.now(timezone.utc).isoformat(),
            )
            with get_session() as session:
                session.add(log_entry)
                session.commit()
        except Exception as e:
            logger.warning("AuditHook[%s] failed (non-blocking): %s", self.name,
e)

        return HookResult.allow()

@dataclass
class TraceHook(Hook):
    trace_level: str = "summary"

```



```

agent_trace_service: Any = None
async def execute(self, context: "PipelineContext") -> HookResult:
    if self.agent_trace_service is None:
        return HookResult.allow()
    try:
        if self.name == "trace.reasoning_start":
            reasoning_id = self.agent_trace_service.create_reasoning_log(
                message_id=context.db_message_id,
                user_id=context.user_id,
                conversation_id=context.message.conversation_id if
context.message else "",
                matched_sop_id=getattr(context.intent, "sop_id", None),
                match_confidence=getattr(context.intent, "confidence",
"none"),
                is_compound=bool(context.sub_tasks),
                fallback=getattr(context.intent, "fallback", False),
            )
            context.set("_trace_reasoning_id", reasoning_id)
        elif self.name == "trace.reasoning_end":
            reasoning_id = context.get("_trace_reasoning_id", "")
            if reasoning_id:
                agent_result = context.agent_result
                self.agent_trace_service.update_reasoning_log(
                    reasoning_log_id=reasoning_id,
                    iteration_count=context.get("trace_iteration_count", 0),
                    elapsed_ms=context.get("trace_elapsed_ms", 0),
                    execution_result="success" if agent_result and
agent_result.success else "failed",
                    reply_preview=(context.agent_reply or "")[:500],
                    error_message=getattr(agent_result, "error_code", "") if
agent_result else "",
                )
            return HookResult.allow()
        except Exception as e:
            logger.warning("TraceHook[%s] failed (non-blocking): %s", self.name,
e)
            return HookResult.allow()

@dataclass
class ProgressHook(Hook):
    progress_sender: Any = None
    progress_template: str = "收到, 正在为你{action}, 请稍候..."
    enable_progress: bool = True
    async def execute(self, context: "PipelineContext") -> HookResult:
        sender = context.baggage.get("progress_sender", self.progress_sender)
        if not self.enable_progress or sender is None:
            return HookResult.allow()
        if not callable(sender):
            logger.debug("ProgressHook[%s] sender is not callable, skip",
self.name)
            return HookResult.allow()
        template = context.baggage.get("progress_template",
self.progress_template) or self.progress_template
        try:
            intent = context.intent

```

```

        action = "处理"
        if intent and getattr(intent, "sop_id", None):
            sop_id = intent.sop_id
            action_map = {
                "SOP-001": "整理会议纪要",
                "SOP-002": "录入事件",
                "SOP-003": "管理任务",
                "SOP-004": "归档文件",
                "SOP-005": "查询数据",
                "SOP-006": "执行守护审计",
                "SOP-007": "管理记忆",
                "SOP-008": "处理待办问题",
            }
            action = action_map.get(sop_id, "处理")
            progress_text = str(template).format(action=action)
            result = sender(progress_text)
            if result is not None and hasattr(result, "__await__"):
                await result
            logger.info("ProgressHook[%s] sent: %s", self.name, progress_text)
        except (TypeError, AttributeError) as e:
            logger.warning(
                "ProgressHook[%s] sender call failed (non-blocking): %s",
                self.name, e)
        except Exception as e:
            logger.warning("ProgressHook[%s] failed (non-blocking): %s",
                self.name, e)
        return HookResult.allow()
HOOK_TYPE_MAP: dict[str, type[Hook]] = {
    "auth": AuthHook,
    "audit": AuditHook,
    "trace": TraceHook,
    "progress": ProgressHook,
}

```

文件编号: 157 文件路径: emily-core\emily_core\workitem\pipeline\hook_registry.py

```

from __future__ import annotations
import logging
from typing import TYPE_CHECKING
if TYPE_CHECKING:
    from .hook import Hook
logger = logging.getLogger("emily.pipeline.registry")
class HookRegistry:
    def __init__(self):
        self._hooks: dict[str, list["Hook"]] = {}
    def register(self, mount_point: str, hook: "Hook") -> None:
        hook_name = getattr(hook, "name", "?")
        if mount_point not in self._hooks:
            self._hooks[mount_point] = []
        self._hooks[mount_point].append(hook)
        self._hooks[mount_point].sort(key=lambda h: getattr(h, "priority", 10))
        logger.info(

```

```

        "Hook registered: %s -> %s (priority=%d)",
        mount_point, hook_name, getattr(hook, "priority", 10),
    )
def unregister(self, mount_point: str, hook_name: str) -> bool:
    if mount_point not in self._hooks:
        return False
    before = len(self._hooks[mount_point])
    self._hooks[mount_point] = [
        h for h in self._hooks[mount_point]
        if getattr(h, "name", "") != hook_name
    ]
    removed = len(self._hooks[mount_point]) < before
    if removed:
        logger.info("Hook unregistered: %s from %s", hook_name, mount_point)
    return removed
def get(self, mount_point: str) -> list["Hook"]:
    return self._hooks.get(mount_point, [])
def get_enabled(self, mount_point: str) -> list["Hook"]:
    return [
        h for h in self.get(mount_point)
        if getattr(h, "enabled", True)
    ]
def list_all(self) -> dict[str, list[str]]:
    return {
        mp: [getattr(h, "name", "?") for h in hooks]
        for mp, hooks in self._hooks.items()
    }
def clear(self) -> None:
    self._hooks.clear()
    logger.info("HookRegistry cleared")
def mount_point_count(self) -> int:
    return len(self._hooks)
def hook_count(self) -> int:
    return sum(len(hooks) for hooks in self._hooks.values())

```

文件编号: 158 文件路径: emily-core\emily_core\workitem\pipeline\interfaces\auth.py

```

from __future__ import annotations
from dataclasses import dataclass, field
from enum import Enum
class AuthDecision(Enum):
    ALLOW = "allow"
    DENY = "deny"
@dataclass
class AuthResult:
    decision: AuthDecision
    reason: str = ""
    matched_roles: list[str] = field(default_factory=list)
    _source: str = ""
    @property
    def is_allowed(self) -> bool:
        return self.decision == AuthDecision.ALLOW

```

```
@property
def is_denied(self) -> bool:
    return self.decision == AuthDecision.DENY
```

文件编号: 159 文件路径: emily-core\emily_core\workitem\pipeline\interfaces\execution.py

```
from __future__ import annotations
from abc import ABC, abstractmethod
from dataclasses import dataclass, field
from typing import Any, TYPE_CHECKING
if TYPE_CHECKING:
    from .routing import RouteDecision
    from .planning import ExecutionPlan

@dataclass
class ToolCallRecord:
    tool_name: str
    tool_input: dict = field(default_factory=dict)
    tool_output: dict = field(default_factory=dict)
    success: bool = True
    elapsed_ms: int = 0

@dataclass
class RagChunk:
    content: str
    score: float = 0.0
    doc_name: str = ""

@dataclass
class RagResult:
    query: str
    provider: str = ""
    chunks: list[RagChunk] = field(default_factory=list)
    hit_count: int = 0
    elapsed_ms: int = 0

@dataclass
class DbResult:
    operation: str = ""
    table: str = ""
    affected_rows: int = 0
    result_data: dict = field(default_factory=dict)
    elapsed_ms: int = 0

@dataclass
class GuardianStepVerdict:
    verdict: str = "PASS"
    reason: str = ""
    suggestions: list[str] = field(default_factory=list)

@dataclass
class StepResult:
    step_id: str
    success: bool = True
    output: str = ""
    tool_calls: list[ToolCallRecord] = field(default_factory=list)
    rag_results: list[RagResult] = field(default_factory=list)
    db_results: list[DbResult] = field(default_factory=list)
```

```

    business_data: dict = field(default_factory=dict)
    guardian: GuardianStepVerdict | None = None
class WorkAgent(ABC):
    @abstractmethod
    async def plan(
        self, route_decision: "RouteDecision", context: Any
    ) -> "ExecutionPlan":
        ...
    @abstractmethod
    async def execute(
        self, plan: "ExecutionPlan", context: Any
    ) -> list[StepResult]:
        ...

```

文件编号: 160 文件路径: emily-core\emily_core\workitem\pipeline\interfaces\guardian.py

```

from __future__ import annotations
from enum import Enum
class GuardianVerdict(Enum):
    PASS = "pass"
    FLAG = "flag"
    REJECT = "reject"

```

文件编号: 161 文件路径: emily-core\emily_core\workitem\pipeline\interfaces\planning.py

```

from __future__ import annotations
from dataclasses import dataclass, field
@dataclass
class PlanStep:
    step_id: str
    description: str
    tool_name: str | None = None
    tool_params: dict = field(default_factory=dict)
    expected_output: str = ""
    depends_on: list[str] = field(default_factory=list)
@dataclass
class ExecutionPlan:
    risk_level: str = "L2"
    steps: list[PlanStep] = field(default_factory=list)
    acceptance_criteria: list[str] = field(default_factory=list)
    estimated_steps: int = 0
    _source: str = ""

```

文件编号: 162 文件路径: emily-core\emily_core\workitem\pipeline\interfaces\risk.py

```

```python

```

文件编号: 163

文件路径: emily-core\emily\_core\workitem\pipeline\interfaces\routing.py

```
```python
from __future__ import annotations
from dataclasses import dataclass, field
from enum import Enum

class IntentType(Enum):
    SOP = "sop"
    COMPOUND = "compound"
    FALLBACK = "fallback"
    FAST_REPLY = "fast_reply"

@dataclass
class SubTask:
    id: str
    sop_id: str
    user_input: str = ""
    depends_on: list[str] = field(default_factory=list)
    priority: int = 1

@dataclass
class RouteDecision:
    intent_type: str = "fallback"
    sop_id: str | None = None
    confidence: str = "none"
    is_compound: bool = False
    sub_tasks: list[SubTask] = field(default_factory=list)
    fallback_reason: str = ""
    _source: str = ""
```

文件编号: 164 文件路径: emily-core\emily_core\workitem\pipeline\mocks\mock_execution.py

```
from __future__ import annotations
from typing import Any, TYPE_CHECKING
from ..interfaces.execution import (
    WorkAgent,
    StepResult,
    ToolCallRecord,
    RagResult,
    RagChunk,
    DbResult,
    GuardianStepVerdict,
)
from ..interfaces.planning import ExecutionPlan, PlanStep
if TYPE_CHECKING:
    from ..interfaces.routing import RouteDecision
class MockWorkAgent(WorkAgent):
    async def plan(
        self, route_decision: "RouteDecision", context: Any
    ) -> ExecutionPlan:
        return ExecutionPlan(
            risk_level="L2",
            steps=[
```

```

PlanStep(
    step_id="step-01",
    description="解析用户输入",
    tool_name=None,
    expected_output="提取事件要素",
),
PlanStep(
    step_id="step-02",
    description="执行业务操作",
    tool_name="record_event",
    expected_output="事件创建成功",
),
PlanStep(
    step_id="step-03",
    description="确认结果",
    tool_name=None,
    expected_output="返回确认信息",
),
],
acceptance_criteria=["事件要素完整", "数据写入成功"],
estimated_steps=3,
_source="mock",
)
)
async def execute(
    self, plan: ExecutionPlan, context: Any
) -> list[StepResult]:
    return [
        StepResult(
            step_id="step-01",
            success=True,
            output="已解析: 事件=样板段放线完成, 日期=2026-06-22, 位置=样板段",
            tool_calls=[],
            rag_results=[],
            db_results=[],
            business_data={
                "event_title": "样板段放线完成",
                "event_date": "2026-06-22",
            },
            guardian=None,
        ),
        StepResult(
            step_id="step-02",
            success=True,
            output="事件
            tool_calls=[
                ToolCallRecord(
                    tool_name="record_event",
                    tool_input={
                        "title": "样板段放线完成",
                        "date": "2026-06-22",
                    },
                ),
            ],
            tool_output={
                "event_id": "mock-42",
                "status": "created",
            },
        ),
    ]

```

```

        },
        success=True,
    )
],
rag_results=[],
db_results=[
    DbResult(
        operation="insert",
        table="events",
        affected_rows=1,
        result_data={
            "event_id": "mock-42",
            "title": "样板段放线完成",
        },
    ),
],
business_data={"event_id": "mock-42"},
guardian=GuardianStepVerdict(
    verdict="PASS", reason="要素完整，写入成功"
),
),
StepResult(
    step_id="step-03",
    success=True,
    output="操作完成，事件已录入",
    tool_calls=[],
    rag_results=[],
    db_results=[],
    business_data={},
    guardian=None,
),
]
class MockWorkAgentQuery(WorkAgent):
    async def plan(
        self, route_decision: "RouteDecision", context: Any
    ) -> ExecutionPlan:
        return ExecutionPlan(
            risk_level="L1",
            steps=[
                PlanStep(
                    step_id="step-01",
                    description="知识库检索",
                    tool_name="knowledge_search",
                    expected_output="返回相关文档片段",
                ),
            ],
            acceptance_criteria=["检索结果相关"],
            estimated_steps=1,
            _source="mock",
        )
    async def execute(
        self, plan: ExecutionPlan, context: Any
    ) -> list[StepResult]:
        return [

```



```

StepResult(
    step_id="step-01",
    success=True,
    output="已检索到 3 条相关知识",
    tool_calls=[
        ToolCallRecord(
            tool_name="knowledge_search",
            tool_input={"query": "消防验收材料", "top_k": 3},
            tool_output={"hit_count": 3},
            success=True,
        )
    ],
    rag_results=[
        RagResult(
            query="消防验收材料",
            provider="maxkb",
            chunks=[
                RagChunk(
                    content="消防验收需提交: 1.竣工验收报告 2.消防设施检
测报告 3.消防产品质量证明文件",
                    score=0.92,
                    doc_name="消防验收指南_v2.pdf",
                ),
                RagChunk(
                    content="根据《消防法》第十三条, 建设工程竣工后建设单
位应当向公安机关消防机构申请消防验收",
                    score=0.85,
                    doc_name="消防法规汇编.pdf",
                ),
                RagChunk(
                    content="消防验收申请表需加盖建设单位公章, 并附施工单
位自检报告",
                    score=0.71,
                    doc_name="验收流程手册.pdf",
                ),
            ],
            hit_count=3,
            elapsed_ms=120,
        )
    ],
    db_results=[],
    business_data={"knowledge_hits": 3},
    guardian=None,
),
]

```

文件编号: 165 文件路径: emily-core\emily_core\workitem\pipeline\mocks\mock_planning.py

```

from __future__ import annotations
from typing import Any, TYPE_CHECKING
from ..interfaces.planning import ExecutionPlan, PlanStep
if TYPE_CHECKING:

```

```

from ..interfaces.routing import RouteDecision
class MockPlanner:
    async def plan(
        self, route_decision: "RouteDecision", context: Any
    ) -> ExecutionPlan:
        return ExecutionPlan(
            risk_level="L2",
            steps=[
                PlanStep(
                    step_id="step-01",
                    description="解析用户输入",
                    tool_name=None,
                    expected_output="提取事件要素",
                ),
                PlanStep(
                    step_id="step-02",
                    description="执行业务操作",
                    tool_name="record_event",
                    expected_output="事件创建成功",
                ),
                PlanStep(
                    step_id="step-03",
                    description="确认结果",
                    tool_name=None,
                    expected_output="返回确认信息",
                ),
            ],
            acceptance_criteria=["事件要素完整", "数据写入成功"],
            estimated_steps=3,
            _source="mock",
        )

```

文件编号：166 文件路径：emily-core\emily_core\workitem\pipeline\node.py

```

from __future__ import annotations
from dataclasses import dataclass
from typing import Awaitable, Callable, TYPE_CHECKING
if TYPE_CHECKING:
    from .context import BusContext
NodeHandler = Callable[["BusContext"], Awaitable[None]]
@dataclass
class PipelineNode:
    name: str
    title: str
    handler: "NodeHandler"
    required: bool = True

```

文件编号：167 文件路径：emily-core\emily_core\workitem\pipeline\real_guardian.py

```

from __future__ import annotations
import json
import logging
from dataclasses import dataclass, field
from typing import Any
logger = logging.getLogger("emily.guardian")
def _load_guardian_prompt(name: str) -> str:
    from emily_core.infrastructure.llm.prompt_loader import load_prompt
    return load_prompt(name)
_STEP_REVIEW_PROMPT = None
_REPLY_REVIEW_PROMPT = None
def _get_step_review_prompt() -> str:
    global _STEP_REVIEW_PROMPT
    if _STEP_REVIEW_PROMPT is None:
        _STEP_REVIEW_PROMPT = _load_guardian_prompt("guardian_step")
    return _STEP_REVIEW_PROMPT
def _get_reply_review_prompt() -> str:
    global _REPLY_REVIEW_PROMPT
    if _REPLY_REVIEW_PROMPT is None:
        _REPLY_REVIEW_PROMPT = _load_guardian_prompt("guardian_reply")
    return _REPLY_REVIEW_PROMPT
@dataclass
class GuardianNote:
    source: str = ""
    issues: list[str] = field(default_factory=list)
class RealGuardian:
    def __init__(self, llm_client: Any, config: Any = None) -> None:
        self._llm = llm_client
        self._config = config
    async def review_step(self, step_result: Any) -> GuardianNote | None:
        if not self._llm:
            return None
        prompt = self._build_step_prompt(step_result)
        try:
            data = await self._llm.chat_json(
                prompt,
                f"审核步骤: {getattr(step_result, 'step_id', '?')}",
            )
            issues: list[str] = data.get("issues", []) if isinstance(data, dict)
        except Exception as e:
            logger.debug("Guardian review_step failed (silent skip): %s", e)
            return None
        else:
            if issues:
                return GuardianNote(
                    source=f"step:{getattr(step_result, 'step_id', '?')}",
                    issues=issues,
                )
            return None
        except Exception as e:
            logger.debug("Guardian review_step failed (silent skip): %s", e)
            return None
    async def review_reply(self, draft_reply: str, work_item: Any) -> GuardianNote | None:
        if not self._llm or not draft_reply:

```

```

        return None
    prompt = self._build_reply_prompt(draft_reply, work_item)
    try:
        data = await self._llm.chat_json(
            prompt,
            f"审核回复: {draft_reply[:100]}",
        )
        issues: list[str] = data.get("issues", []) if isinstance(data, dict)
    except Exception as e:
        logger.debug("Guardian review_reply failed (silent skip): %s", e)
        return None
    else:
        if issues:
            return GuardianNote(source="reply", issues=issues)
        return None
    def _build_step_prompt(self, sr: Any) -> str:
        output = (getattr(sr, "output", "") or "")[:1500]
        tool_info = ""
        for tc in getattr(sr, "tool_calls", []) or []:
            tool_info += (
                f"工具: {getattr(tc, 'tool_name', '?')}\n"
                f"输入: {json.dumps(getattr(tc, 'tool_input', {}),
                ensure_ascii=False)}\n"
                f"输出: {json.dumps(getattr(tc, 'tool_output', {}),
                ensure_ascii=False)}\n"
            )
        rag_info = ""
        for rr in getattr(sr, "rag_results", []) or []:
            for chunk in (getattr(rr, "chunks", []) or [])[:3]:
                rag_info += (
                    f"引用: 《{getattr(chunk, 'doc_name', '?')}》 "
                    f"{getattr(chunk, 'content', '')[:200]}\n"
                )
        return _get_step_review_prompt().format(
            step_id=getattr(sr, "step_id", "?"),
            output=output,
            tool_info=tool_info[:2000],
            rag_info=rag_info[:1000],
        )
    def _build_reply_prompt(self, draft: str, wi: Any) -> str:
        sop_id = getattr(wi, "sop_id", "") or "无"
        user_input = (getattr(wi, "user_input", "") or "")[:500]
        steps_summary = ""
        for sr in getattr(wi, "step_results", []) or []:
            steps_summary += (
                f"[{getattr(sr, 'step_id', '?')}] "
                f"{'OK' if getattr(sr, 'success', True) else 'FAIL'} "
                f"{(getattr(sr, 'output', '') or '')[:200]}\n"
            )
        return _get_reply_review_prompt().format(
            draft_reply=draft[:2000],
            sop_id=sop_id,
            user_input=user_input,

```

```

        steps_summary=steps_summary[:1500],
    )

```

文件编号: 168 文件路径: emily-core\emily_core\workitem\scheduler.py

```

from __future__ import annotations
import logging
from .workitem import WorkItem
from .workitem_state import WorkItemState
from .pipeline.bus import PipelineBUS
from .pipeline.context import BusContext
logger = logging.getLogger("emily.scheduler")
class SessionScheduler:
    def __init__(self, session_id: str, bus: PipelineBUS, session_context=None):
        self.session_id = session_id
        self._bus = bus
        self._session_context = session_context
        self._queue: list[WorkItem] = []
        self._active: dict[str, WorkItem] = {}
        self._done: list[WorkItem] = []
    def enqueue(self, work_item: WorkItem) -> None:
        work_item.session_id = self.session_id
        self._queue.append(work_item)
        self._queue.sort(key=lambda wi: wi.priority)
        logger.debug(
            "Scheduler[%s] enqueued %s (priority=%d, queue=%d)",
            self.session_id, work_item.id, work_item.priority, len(self._queue),
        )
    async def run_next(self) -> WorkItem | None:
        if not self._queue:
            return None
        wi = self._queue.pop(0)
        return await self._run_one(wi)
    async def run_all(self) -> list[WorkItem]:
        results: list[WorkItem] = []
        while self._queue:
            wi = self._queue.pop(0)
            results.append(await self._run_one(wi))
        return results
    async def run_all_with_message(self, message) -> list[WorkItem]:
        results: list[WorkItem] = []
        while self._queue:
            wi = self._queue.pop(0)
            results.append(await self._run_one(wi, message=message))
        return results
    async def _run_one(self, wi: WorkItem, message=None) -> WorkItem:
        self._active[wi.id] = wi
        try:
            wi.transition_to(WorkItemState.PLANNING)
            context = BusContext(
                work_item=wi,
                message=message,

```

```

        user_id=wi.user_id,
        is_admin=wi.is_admin,
        _session_context=self._session_context,
    )
    if context.message is None:
        stored = getattr(wi, '_source_message', None)
        if stored is not None:
            context.message = stored
    wi.transition_to(WorkItemState.EXECUTING)
    await self._bus.run(context)
    if context.should_abort:
        wi.transition_to(WorkItemState.FAILED)
        wi.error_message = wi.error_message or context.abort_reason
        logger.warning("Scheduler[%s] WI %s FAILED: %s",
                       self.session_id, wi.id, wi.error_message)
    else:
        wi.transition_to(WorkItemState.DONE)
        logger.info("Scheduler[%s] WI %s DONE", self.session_id, wi.id)
except Exception as e:
    logger.error("Scheduler[%s] WI %s crashed: %s",
                self.session_id, wi.id, e, exc_info=True)
if not wi.is_terminal:
    try:
        wi.transition_to(WorkItemState.FAILED)
    except ValueError:
        wi.state = WorkItemState.FAILED
    wi.error_message = str(e)
finally:
    self._active.pop(wi.id, None)
    self._done.append(wi)
return wi

@property
def has_pending(self) -> bool:
    return bool(self._queue)

@property
def active_count(self) -> int:
    return len(self._active)

```

文件编号: 169 文件路径: emily-core\emily_core\workitem\workitem.py

```

from __future__ import annotations
import uuid
from dataclasses import dataclass, field
from datetime import datetime, timezone
from typing import Any
from .workitem_state import WorkItemState, TRANSITIONS, TERMINAL_STATES

@dataclass
class WorkItem:
    id: str = field(default_factory=lambda: f"WI-{uuid.uuid4().hex[:8]}")
    session_id: str = ""
    state: WorkItemState = WorkItemState.CREATED
    user_input: str = ""

```

```

sop_id: str = ""
intent_type: str = ""
user_id: str = ""
is_admin: bool = False
priority: int = 1
required_permissions: list[str] = field(default_factory=list)
required_tools: set[str] = field(default_factory=set)
required_tables: set[str] = field(default_factory=set)
route_decision: Any = None
execution_plan: Any = None
risk_level: str = ""
acceptance_criteria: list[str] = field(default_factory=list)
step_results: list[Any] = field(default_factory=list)
result_text: str = ""
injected_sops: set[str] = field(default_factory=set)
injected_tools: set[str] = field(default_factory=set)
injected_tables: set[str] = field(default_factory=set)
llm_call_count: int = 0
warnings: list[str] = field(default_factory=list)
error_message: str = ""
created_at: str = field(
    default_factory=lambda: datetime.now(timezone.utc).isoformat()
)
updated_at: str = ""
completed_at: str = ""
def transition_to(self, new_state: WorkItemState) -> None:
    allowed = TRANSITIONS.get(self.state, [])
    if new_state not in allowed:
        allowed_names = [s.value for s in allowed]
        raise ValueError(
            f"非法 WorkItem 状态转换: {self.state.value} -> {new_state.value} "
            f"(允许: {allowed_names})"
        )
    self.state = new_state
    self.updated_at = datetime.now(timezone.utc).isoformat()
@property
def is_terminal(self) -> bool:
    return self.state in TERMINAL_STATES
def add_step_result(self, step_result: Any) -> None:
    self.step_results.append(step_result)
def add_warning(self, msg: str) -> None:
    self.warnings.append(msg)
def to_summary(self) -> dict:
    return {
        "id": self.id,
        "state": self.state.value,
        "sop_id": self.sop_id,
        "user_input": self.user_input[:200],
        "steps_executed": len(self.step_results),
        "rag_hits": sum(
            len(getattr(sr, "rag_results", [])) for sr in self.step_results
        ),
        "tool_calls": sum(
            len(getattr(sr, "tool_calls", [])) for sr in self.step_results

```

```

    ),
    "db_operations": sum(
        len(getattr(sr, "db_results", [])) for sr in self.step_results
    ),
    "risk_level": self.risk_level,
    "llm_call_count": self.llm_call_count,
}

```

文件编号: 170 文件路径: emily-core\emily_core\workitem\workitem_agent.py

```

from __future__ import annotations
import asyncio
import logging
import re
import time as _time
from .injector import KnowledgeInjector
from .pipeline.context import BusContext
from .pipeline.interfaces.routing import RouteDecision, SubTask
from .pipeline.interfaces.planning import ExecutionPlan, PlanStep
from .pipeline.interfaces.execution import StepResult, ToolCallRecord, DbResult,
RagResult, RagChunk, GuardianStepVerdict
from .pipeline.interfaces.auth import AuthResult, AuthDecision
from .pipeline.mocks import MockPlanner, MockWorkAgent
from .pipeline.real_guardian import RealGuardian, GuardianNote
from ..session.session_context import format_message_history
logger = logging.getLogger("emily.workitem_agent")
def _load_planner_prompt() -> str:
    from ..infrastructure.llm.prompt_loader import load_prompt
    return load_prompt("planner")
def _load_workitem_prompt() -> str:
    from ..infrastructure.llm.prompt_loader import load_prompt
    return load_prompt("workitem")
def _fallback_steps() -> list[PlanStep]:
    return [
        PlanStep("step-01", "解析用户输入并确认意图"),
        PlanStep("step-02", "执行核心业务操作"),
        PlanStep("step-03", "确认结果并生成回复"),
    ]
class WorkItemAgent:
    def __init__(
        self,
        injector: KnowledgeInjector | None = None,
        llm_client=None,
        config=None,
        business_flow_tools=None,
        rag_provider=None,
        permission_engine=None,
        skill_registry=None,
        skill_executor=None,
    ):
        self.injector = injector or KnowledgeInjector()
        self.config = config

```



```

self._llm = llm_client
self._business_flow_tools = business_flow_tools
self._rag_provider = rag_provider
self._permission_engine = permission_engine
self._skill_registry = skill_registry
self._skill_executor = skill_executor
self._planner = MockPlanner()
self._work_agent = MockWorkAgent()
if self._llm:
    self._guardian = RealGuardian(llm_client=self._llm, config=config)
    logger.info("Guardian enabled: RealGuardian (lightweight LLM review)")
else:
    self._guardian = None
    logger.info("LLM not available – Guardian disabled (silent skip)")
def _resolve_mode(self, component: str) -> str:
    if self.config is None:
        return "mock"
    mode = getattr(self.config, f"{component}_mode", "mock") or "mock"
    if mode in ("real", "review", "agent") and self._llm is None:
        logger.warning(
            "Component '%s' mode=%s but no LLM client – falling back to mock",
            component, mode,
        )
        return "mock"
    return mode
@staticmethod
def _skill_to_execution_plan(skill) -> ExecutionPlan:
    from ..workitem.pipeline.interfaces.planning import PlanStep,
    ExecutionPlan
    steps = [
        PlanStep(
            step_id=s.id,
            description=s.description,
            tool_name=s.tool_name,
            tool_params={},
            expected_output="",
            depends_on=[],
        )
        for s in skill.steps
    ]
    return ExecutionPlan(
        risk_level="L2",
        steps=steps,
        acceptance_criteria=[],
        estimated_steps=len(steps),
        _source="skill_definition",
    )
async def _execute_skill(self, wi, context: BusContext) -> list[StepResult]:
    from ..skill.executor import SkillExecutionContext
    skill = self._skill_registry.get_by_sop_id(wi.sop_id)
    if skill is None:
        logger.error("Skill not found for sop_id=%s, falling back to
_real_execute", wi.sop_id)
        return await self._real_execute(wi.execution_plan, context)

```

```

session_ctx = context.get_session_context() if context else None
session_context_dict = {}
session_available_tools: list[dict] = []
if session_ctx:
    session_context_dict = {
        "user_id": getattr(session_ctx, "user_id", ""),
        "user_name": getattr(session_ctx, "user_name", ""),
        "project_ids": list(getattr(session_ctx, "project_ids", [])),
        "project_name": getattr(session_ctx, "project_name", ""),
        "db_perms": dict(getattr(session_ctx, "db_perms", {})),
        "info_level": getattr(session_ctx, "info_level", "public"),
        "company_type": getattr(session_ctx, "company_type", ""),
        "department": list(getattr(session_ctx, "department", [])),
    }
    session_available_tools = list(getattr(session_ctx, "available_tools",
[[]))

skill_ctx = SkillExecutionContext(
    skill=skill,
    user_input=wi.user_input,
    user_id=context.user_id if hasattr(context, "user_id") else "",
    message_id=context.db_message_id if hasattr(context, "db_message_id")
else "",
    conversation_id=context.message.conversation_id if context.message
else "",
    session_context=session_context_dict,
    step_results={},
    business_flow_tools=self._business_flow_tools,
    llm_client=self._llm,
    session_available_tools=session_available_tools,
)
return await self._skill_executor.execute(skill_ctx)
async def node1_intent(self, context: BusContext) -> None:
    wi = context.work_item
    self.injector.analyze(wi)
    if wi.route_decision is None:
        wi.route_decision = RouteDecision(
            intent_type=getattr(wi, "intent_type", "fallback") or "fallback",
            sop_id=wi.sop_id or None,
            confidence="medium" if wi.sop_id else "none",
            is_compound=False,
            sub_tasks=[
                SubTask(
                    id="subtask-001",
                    sop_id=wi.sop_id or "",
                    user_input=wi.user_input,
                )
            ] if wi.sop_id else [],
            _source="session_agent",
        )
    wi.llm_call_count += 1
    context.intent = wi.route_decision
    logger.debug("WI %s node1: sop=%s intent=%s _source=%s",
                wi.id, wi.sop_id, wi.route_decision.intent_type,
                getattr(wi.route_decision, "_source", ""))

```

```

    async def node2_plan(self, context: BusContext) -> None:
        wi = context.work_item
        if self._skill_registry and self._skill_registry.has_skill(wi.sop_id or
        "")):
            skill = self._skill_registry.get_by_sop_id(wi.sop_id)
            plan = self._skill_to_execution_plan(skill)
            plan._source = "skill_definition"
            wi.execution_plan = plan
            wi.risk_level = plan.risk_level
            wi.acceptance_criteria = list(getattr(plan, "acceptance_criteria",
            []))
            wi.llm_call_count += 1
            logger.info("WI %s node2: using Skill definition (sop=%s)", wi.id,
            wi.sop_id)
            return
        planner_mode = self._resolve_mode("planner")
        if planner_mode == "real" and self._llm:
            plan = await self._llm_plan(wi, context)
        else:
            plan = await self._planner.plan(wi.route_decision, context)
        wi.execution_plan = plan
        wi.risk_level = plan.risk_level
        wi.acceptance_criteria = list(getattr(plan, "acceptance_criteria", []))
        wi.llm_call_count += 1
        logger.debug("WI %s node2: risk=%s steps=%d _source=%s",
            wi.id, plan.risk_level, getattr(plan, "estimated_steps", 0),
            getattr(plan, "_source", "mock"))
    async def _llm_plan(self, wi, context) -> ExecutionPlan:
        sop_text = ""
        if hasattr(self.injector, 'get_context_text'):
            sop_text = self.injector.get_context_text()
        tools_text = ""
        if self._business_flow_tools:
            tool_entries = []
            for name in sorted(self._business_flow_tools.list_names()):
                tool = self._business_flow_tools.get(name)
                if tool:
                    tool_entries.append(f"- {name}: {tool.description}")
            tools_text = "\n".join(tool_entries) if tool_entries else "（无可用工
具）"
        planner_prompt = _load_planner_prompt()
        system_prompt = planner_prompt.format(
            sop_text=sop_text[:4000] if sop_text else f"SOP: {wi.sop_id or '未知'}
（全文未加载）",
            user_input=wi.user_input,
            available_tools=tools_text,
        )
        session_ctx = context.get_session_context() if context else None
        message_history = getattr(session_ctx, 'message_history', []) if
        session_ctx else []
        full_messages = [{"role": "system", "content": system_prompt}]
        full_messages.extend(message_history)
        full_messages.append({
            "role": "user",

```

```

        "content": f"Plan for: {wi.user_input[:200]}",
    })
    try:
        result = await self._llm.chat_messages(full_messages, json_mode=True)
        data = result.get("data", {})
        logger.debug("LLM planner response: %s", data)
    except Exception as e:
        logger.error("LLM planner failed: %s, falling back to MockPlanner", e)
        return await self._planner.plan(wi.route_decision, context)
    return self._map_to_execution_plan(data)

    @staticmethod
    def _map_to_execution_plan(data: dict) -> ExecutionPlan:
        steps = []
        for i, s in enumerate(data.get("steps", [])):
            steps.append(PlanStep(
                step_id=s.get("step_id", f"step-{i+1:02d}"),
                description=s.get("description", ""),
                tool_name=s.get("tool_name"),
                tool_params=s.get("tool_params", {}),
                expected_output=s.get("expected_output", ""),
                depends_on=s.get("depends_on", []),
            ))
        if len(steps) > 8:
            logger.warning("Plan has %d steps, truncating to 8", len(steps))
            steps = steps[:8]
        return ExecutionPlan(
            risk_level=data.get("risk_level", "L2"),
            steps=steps if steps else _fallback_steps(),
            acceptance_criteria=data.get("acceptance_criteria", []),
            estimated_steps=data.get("estimated_steps", len(steps)),
            _source="llm_planner",
        )

    async def node3_execute(self, context: BusContext) -> None:
        wi = context.work_item
        if wi.execution_plan is None:
            return
        if getattr(wi.execution_plan, "_source", "") == "skill_definition" and self._skill_executor:
            step_results = await self._execute_skill(wi, context)
        else:
            executor_mode = self._resolve_mode("executor")
            if executor_mode == "real":
                step_results = await self._real_execute(wi.execution_plan, context)
            else:
                step_results = await self._work_agent.execute(wi.execution_plan, context)
        guardian_tasks: list[asyncio.Task] = []
        if self._guardian:
            for sr in step_results:
                task = asyncio.create_task(
                    self._guardian.review_step(sr),
                    name=f"guardian_step_{sr.step_id}",
                )

```

```

        guardian_tasks.append(task)
    if guardian_tasks:
        try:
            notes = await asyncio.gather(*guardian_tasks,
return_exceptions=True)
            for sr, note in zip(step_results, notes):
                if isinstance(note, GuardianNote) and note.issues:
                    for issue in note.issues:
                        wi.add_warning(f"[{note.source}] {issue}")
                    sr.guardian = GuardianStepVerdict(
                        verdict="FLAG",
                        reason="; ".join(note.issues),
                    )
        except Exception as e:
            logger.warning("Guardian gather failed: %s", e)
    for sr in step_results:
        wi.add_step_result(sr)
    wi.llm_call_count += len(step_results)
    if step_results:
        context.agent_result = step_results[-1]
        context.agent_reply = step_results[-1].output
    logger.debug(
        "WI %s node3: %d steps, executor=%s guardian=%s",
        wi.id, len(step_results), executor_mode,
        "enabled" if self._guardian else "disabled",
    )
    async def _real_execute(self, plan: ExecutionPlan, context: BusContext) ->
list[StepResult]:
        if self._business_flow_tools is None:
            logger.warning("RealExecutor: no BusinessFlowToolRegistry, falling
back to MockWorkAgent")
            return await self._work_agent.execute(plan, context)
        results: list[StepResult] = []
        for step in plan.steps:
            t_start = _time.monotonic()
            tool_name = step.tool_name
            tool_params = dict(getattr(step, 'tool_params', {}) or {})
            tool_params["_user_id"] = context.user_id or ""
            tool_params["_message_id"] = context.db_message_id or ""
            tool_params["_conversation_id"] = (
                context.message.conversation_id if context.message else ""
            )
            if context.message is not None:
                raw_attachments = getattr(context.message, "attachments", None) or
[]

                if raw_attachments:
                    tool_params["_attachments"] = raw_attachments
                    first = raw_attachments[0] if isinstance(raw_attachments[0],
dict) else {}

                    tool_params["_attachment_url"] = first.get("url", "")
                    tool_params["_attachment_type"] = first.get("type", 0)
                    logger.debug(
                        "Injected attachments: %d item(s), primary_url=%s",
                        len(raw_attachments), first.get("url", "")[:80],

```

```

    )
    try:
        if tool_name and tool_name in self._business_flow_tools:
            tool = self._business_flow_tools.get(tool_name)
            import inspect
            sig = inspect.signature(tool.handler)
            handler_kwargs = {"params": tool_params}
            if "user_id" in sig.parameters:
                handler_kwargs["user_id"] = context.user_id if
hasattr(context, 'user_id') else ""
            if "message_id" in sig.parameters:
                handler_kwargs["message_id"] = context.db_message_id if
hasattr(context, 'db_message_id') else ""
            handler_result = await tool.handler(**handler_kwargs)
            handler_dict = handler_result if isinstance(handler_result,
dict) else {}

            elapsed_ms = int((_time.monotonic() - t_start) * 1000)
            tool_call = ToolCallRecord(
                tool_name=tool_name,
                tool_input=tool_params,
                tool_output=handler_dict,
                success=handler_dict.get("success", True),
                elapsed_ms=elapsed_ms,
            )
            db_results = []
            object_id = handler_dict.get("object_id", "") or ""
            if object_id:
                db_results.append(DbResult(
                    operation="insert",
                    table=tool_name.replace("record_", "") + "s",
                    affected_rows=1,
                    result_data=handler_dict,
                ))
            rag_results = []
            if handler_dict.get("rag_results_data"):
                rrd = handler_dict["rag_results_data"]
                rag_chunks = [
                    RagChunk(content=c["content"], score=c.get("score",
0.0), doc_name=c.get("doc_name", ""))
                    for c in rrd.get("chunks", [])
                ]
                rag_results.append(RagResult(
                    query=rrd.get("query", ""),
                    provider=rrd.get("provider", ""),
                    chunks=rag_chunks,
                    hit_count=rrd.get("hit_count", len(rag_chunks)),
                    elapsed_ms=rrd.get("elapsed_ms", 0),
                ))
            output = handler_dict.get("reply", step.description)
            success = handler_dict.get("success", True)
            sr = StepResult(
                step_id=step.step_id,
                success=success,
                output=str(output),

```

```

        tool_calls=[tool_call],
        db_results=db_results,
        rag_results=rag_results,
        business_data=handler_dict,
    )
elif tool_name:
    sr = StepResult(
        step_id=step.step_id,
        success=False,
        output=f"工具 '{tool_name}' 未在 BusinessFlowToolRegistry
中注册",
    )
else:
    sr = StepResult(
        step_id=step.step_id,
        success=True,
        output=step.description,
    )
except Exception as e:
    logger.error(
        "Step %s failed: %s (tool=%s params_keys=%s)",
        step.step_id, e, tool_name or "(none)",
        list(tool_params.keys()) if tool_params else [],
        exc_info=True,
    )
    sr = StepResult(
        step_id=step.step_id,
        success=False,
        output=f"步骤执行异常: {e}",
    )
results.append(sr)
if not sr.success:
    break
return results

async def node4_summary(self, context: BusContext) -> None:
    wi = context.work_item
    summary = wi.to_summary()
    steps = summary.get("steps_executed", 0)
    tool_calls = summary.get("tool_calls", 0)
    executor_mode = self._resolve_mode("executor")
    mock_prefix = "" if executor_mode == "real" else "[Mock 模式] "
    session_ctx = context.get_session_context() if context else None
    message_history = getattr(session_ctx, 'message_history', []) if
session_ctx else []
    draft = await self._llm_synthesize_reply(wi, mock_prefix,
                                                message_history=message_history
if message_history else None,
                                                session_ctx=session_ctx)

    if self._guardian:
        try:
            note = await self._guardian.review_reply(draft, wi)
            if note and note.issues:
                for issue in note.issues:
                    wi.add_warning(f"[reply] {issue}")

```

```

        except Exception as e:
            logger.debug("Guardian review_reply failed (silent skip): %s", e)
    if wi.warnings:
        warning_text = (
            "\n\n⚠️ Emily 提醒（系统自动审核标记，供参考）：\n"
            + "\n".join(f"    • {w}" for w in wi.warnings[-5:])
        )
        wi.result_text = draft + warning_text
    else:
        wi.result_text = draft
    wi.llm_call_count += 1
    context.verified_reply = wi.result_text
    logger.debug(
        "WI %s node4: reply_len=%d guardian=%s warnings=%d",
        wi.id, len(wi.result_text),
        "enabled" if self._guardian else "disabled",
        len(wi.warnings),
    )
    async def _llm_synthesize_reply(self, wi, mock_prefix: str = "",
                                    message_history: list[dict] | None = None,
                                    session_ctx=None) -> str:
        steps_text = ""
        for sr in getattr(wi, "step_results", []) or []:
            status = "OK" if getattr(sr, "success", True) else "FAIL"
            output = (getattr(sr, "output", "") or "")[:300]
            steps_text += f"[{getattr(sr, 'step_id', '?')}] {status}: {output}\n"
        warnings_text = "\n".join(f"    • {w}" for w in (getattr(wi, "warnings", [])
or []))
        if not warnings_text:
            warnings_text = "（无）"
        if self._llm:
            try:
                prompt_template = _load_workitem_prompt()
                wi_vars = {
                    "{available_tools}": self._build_tools_text(),
                    "{sop_text}": self.injector.get_context_text()[:3000] if
self.injector else f"SOP: {wi.sop_id or '未知'}",
                    "{user_input}": (getattr(wi, "user_input", "") or "")[:1000],
                    "{step_results}": steps_text[:2000],
                    "{warnings}": warnings_text,
                }
                system_prompt = prompt_template
                for key, value in wi_vars.items():
                    system_prompt = system_prompt.replace(key, str(value))
                if session_ctx is not None:
                    session_vars = session_ctx.get_prompt_variables()
                    for key, value in session_vars.items():
                        if value:
                            system_prompt = system_prompt.replace(key, str(value))
                system_prompt = re.sub(r'\{[a-z_]+\}', '', system_prompt)
                full_messages = [{"role": "system", "content": system_prompt}]
                if message_history:
                    full_messages.extend(message_history)
                full_messages.append({

```



```

        "role": "user",
        "content": f"合成回复: {getattr(wi, 'user_input', '?')}
[:100]}",
    })
    result = await self._llm.chat_messages(full_messages,
json_mode=True)
    data = result.get("data", {})
    reply = data.get("reply", "") if isinstance(data, dict) else ""
    if reply and len(reply) > 20:
        logger.debug("node4: LLM synthesized reply (%d chars)",
len(reply))
        return mock_prefix + reply
    except Exception as e:
        logger.warning("node4: LLM reply synthesis failed, falling back:
%s", e)
    summary = wi.to_summary()
    rag_hits = summary.get("rag_hits", 0)
    tool_calls = summary.get("tool_calls", 0)
    steps = summary.get("steps_executed", 0)
    if rag_hits > 0:
        rag_texts = []
        for sr in wi.step_results:
            for rag_result in getattr(sr, "rag_results", []):
                for chunk in getattr(rag_result, "chunks", []):
                    doc_name = getattr(chunk, "doc_name", "") or "未知来源"
                    rag_texts.append(f"根据《{doc_name}》: {chunk.content}")
        if rag_texts:
            return mock_prefix + "根据知识库检索, 找到以下相关信息: \n\n" +
"\n".join(rag_texts[:5])
            return mock_prefix + f"已完成知识库查询, 共找到 {rag_hits} 条相关信息。"
        elif tool_calls > 0:
            return (
                f"{mock_prefix}操作已完成! 共执行 {steps} 个步骤, "
                f"调用 {tool_calls} 个工具, 数据库操作 {summary.get('db_operations',
0)} 次。"
            )
        else:
            return mock_prefix + "Emily 已处理完毕。"
    @staticmethod
    def _build_tools_text() -> str:
        try:
            from ..tools.business_flow_tools import BusinessFlowToolRegistry
            registry = BusinessFlowToolRegistry.get_instance()
            if registry:
                entries = []
                for name in sorted(registry.list_names()):
                    tool = registry.get(name)
                    if tool:
                        entries.append(f"- {name}: {tool.description}")
                return "\n".join(entries) if entries else "（无可用工具）"
        except Exception:
            pass
        return "（无可用工具）"
    async def authorize(self, context: BusContext, route_decision) -> AuthResult:

```

```

auth_mode = self._resolve_mode("auth")
if auth_mode != "real":
    return AuthResult(decision=AuthDecision.ALLOW, matched_roles=["all"],
                      _source="mock_auth")
sop_id = getattr(route_decision, "sop_id", None)
if not sop_id:
    return AuthResult(decision=AuthDecision.ALLOW, _source="real_auth")
session_ctx = context.get_session_context()
if session_ctx is None:
    return AuthResult(decision=AuthDecision.DENY, reason="无会话上下文",
                      _source="real_auth")
engine = getattr(self, "_permission_engine", None)
if engine is None:
    if sop_id in session_ctx.sop_allow:
        return AuthResult(decision=AuthDecision.ALLOW,
                          matched_roles=[f"L{session_ctx.level}"],
                          _source="real_auth_sop_allow")
    return AuthResult(
        decision=AuthDecision.DENY,
        reason=f"SOP {sop_id} 不在用户白名单中",
        _source="real_auth_sop_allow",
    )
perms_dict = {
    "level": session_ctx.level,
    "user_id": session_ctx.user_id,
    "sop_allow": list(session_ctx.sop_allow),
    "granted_codes": list(session_ctx.granted_codes),
    "denied_codes": list(session_ctx.denied_codes),
    "info_level": session_ctx.info_level,
    "company_type": session_ctx.company_type,
    "department": list(session_ctx.department),
    "authorized_node_ids": list(session_ctx.authorized_node_ids),
    "supervisor_id": session_ctx.supervisor_id,
}
result = await engine.check_sop_access(perms_dict, sop_id, context)
if result.allowed:
    return AuthResult(decision=AuthDecision.ALLOW,
                      matched_roles=[f"L{session_ctx.level}"],
                      _source="real_auth_engine")
return AuthResult(
    decision=AuthDecision.DENY,
    reason=result.reason,
    _source="real_auth_engine",
)

def grade_risk(self, route_decision, operation_type: str = "") -> str:
    risk_mode = self._resolve_mode("risk")
    if risk_mode != "real":
        return "L2"
    intent_type = getattr(route_decision, "intent_type", "fallback")
    confidence = getattr(route_decision, "confidence", "none")
    is_compound = getattr(route_decision, "is_compound", False)
    if intent_type == "fast_reply":
        return "L1"
    if intent_type == "fallback" or confidence == "none":

```

```
        return "L3"
    if is_compound:
        return "L3"
    if operation_type == "delete":
        return "L3"
    if operation_type == "write":
        return "L2"
    if confidence == "low":
        return "L2"
    return "L1"
def node_handlers(self) -> dict:
    return {
        "wi_node1": self.node1_intent,
        "wi_node2": self.node2_plan,
        "wi_node3": self.node3_execute,
        "wi_node4": self.node4_summary,
    }
```

文件编号: 171 文件路径: emily-core\emily_core\workitem\workitem_state.py

```
from __future__ import annotations
from enum import Enum
class WorkItemState(Enum):
    CREATED = "CREATED"
    PLANNING = "PLANNING"
    EXECUTING = "EXECUTING"
    WAITING_CONFIRM = "WAITING_CONFIRM"
    DONE = "DONE"
    FAILED = "FAILED"
TRANSITIONS: dict[WorkItemState, list[WorkItemState]] = {
    WorkItemState.CREATED: [WorkItemState.PLANNING, WorkItemState.FAILED],
    WorkItemState.PLANNING: [WorkItemState.EXECUTING,
    WorkItemState.FAILED],
    WorkItemState.EXECUTING: [
        WorkItemState.DONE,
        WorkItemState.WAITING_CONFIRM,
        WorkItemState.FAILED,
    ],
    WorkItemState.WAITING_CONFIRM: [WorkItemState.EXECUTING,
    WorkItemState.FAILED],
    WorkItemState.DONE: [],
    WorkItemState.FAILED: [],
}
TERMINAL_STATES = frozenset({WorkItemState.DONE, WorkItemState.FAILED})
```

文件编号: 172 文件路径: emily-core\api\middleware\auth.py

```
from __future__ import annotations
import logging
```

```
import os
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.requests import Request
logger = logging.getLogger("emily.api.auth")
class AuthMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        expected = os.environ.get("EMILY_API_TOKEN", "")
        if expected and not request.url.path.endswith("/health"):
            token = request.headers.get("X-Emily-Token", "")
            if token != expected:
                from starlette.responses import JSONResponse
                return JSONResponse({"detail": "unauthorized"}, status_code=401)
        return await call_next(request)
```

文件编号: 173 文件路径: emily-core\api\routes\evolution.py

```
from __future__ import annotations
import logging
from datetime import datetime
from fastapi import APIRouter, HTTPException, Query
logger = logging.getLogger("emily.api.evolution")
router = APIRouter(prefix="/api/v1/evolution", tags=["evolution"])
@router.post("/insights/generate")
async def generate_insight(
    date: str = Query(default="", description="复盘结束日期 YYYY-MM-DD"),
    days: int = Query(default=1, description="复盘天数（默认1，最小1）"),
):
    if not date:
        date = datetime.now().strftime("%Y-%m-%d")
    try:
        from emily_core.services.evolution.insight_generator import
        InsightGenerator
        generator = InsightGenerator()
        result = await generator.generate(date, days=days, dry_run=False)
        return {"status": "ok", "data": result}
    except Exception as e:
        logger.error("generate_insight failed: %s", e, exc_info=True)
        raise HTTPException(status_code=500, detail=str(e))
@router.get("/insights/{date}")
async def get_insight(date: str):
    from emily_core.repositories.evolution_repo import EvolutionRepo
    import asyncio
    insight = await asyncio.to_thread(EvolutionRepo.get_insight_by_date, date)
    if insight is None:
        raise HTTPException(status_code=404, detail=f"洞察 {date} 不存在")
    return {
        "insight_date": insight.insight_date,
        "analysis_days": insight.analysis_days,
        "total_messages": insight.total_messages,
        "sop_hit_rate": insight.sop_hit_rate,
        "fallback_rate": insight.fallback_rate,
        "health_score": insight.health_score,
```

```
        "anomaly_flags": insight.anomaly_flags,
        "insight_text": insight.insight_text,
        "created_at": insight.created_at,
    }
}
@router.post("/rules/induct")
async def induct_rules(
    end_date: str = Query(default="", description="分析截止日期"),
    days: int = Query(default=7, description="回顾天数"),
):
    if not end_date:
        end_date = datetime.now().strftime("%Y-%m-%d")
    try:
        from emily_core.services.evolution.rule_inductor import RuleInductor
        inductor = RuleInductor()
        rules = await inductor.induct(end_date, days=days)
        return {"status": "ok", "rules": rules}
    except Exception as e:
        logger.error("induct_rules failed: %s", e, exc_info=True)
        raise HTTPException(status_code=500, detail=str(e))
@router.get("/rules")
async def list_rules(
    status: str = Query(default="CONFIRMED", description="状态过滤"),
):
    from emily_core.repositories.evolution_repo import EvolutionRepo
    import asyncio
    rules = await asyncio.to_thread(EvolutionRepo.get_rules_by_status, status)
    return {
        "rules": [
            {
                "rule_no": r.rule_no,
                "title": r.title,
                "category": r.category,
                "confidence": r.confidence,
                "status": r.status,
                "suggested_action": r.suggested_action,
                "created_at": r.created_at,
            }
            for r in rules
        ]
    }
}
@router.post("/rules/{rule_no}/confirm")
async def confirm_rule(rule_no: str):
    from emily_core.repositories.evolution_repo import EvolutionRepo
    import asyncio
    success = await asyncio.to_thread(
        EvolutionRepo.update_rule_status,
        rule_no,
        "CONFIRMED",
        confirmed_at=datetime.now().isoformat(),
    )
    if not success:
        raise HTTPException(status_code=404, detail=f"规则 {rule_no} 不存在")
    return {"status": "ok", "rule_no": rule_no, "new_status": "CONFIRMED"}
@router.post("/rules/{rule_no}/discard")
```

```

async def discard_rule(rule_no: str):
    from emily_core.repositories.evolution_repo import EvolutionRepo
    import asyncio
    success = await asyncio.to_thread(
        EvolutionRepo.update_rule_status,
        rule_no,
        "DISCARDED",
    )
    if not success:
        raise HTTPException(status_code=404, detail=f"规则 {rule_no} 不存在")
    return {"status": "ok", "rule_no": rule_no, "new_status": "DISCARDED"}

@router.post("/patches/generate")
async def generate_patches(
    rule_no: str = Query(default="", description="指定规则编号"),
):
    try:
        from emily_core.services.evolution.patch_generator import PatchGenerator
        generator = PatchGenerator()
        rule_nos = [rule_no] if rule_no else None
        patches = await generator.generate(rule_nos)
        return {"status": "ok", "patches": patches}
    except Exception as e:
        logger.error("generate_patches failed: %s", e, exc_info=True)
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/patches")
async def list_patches(
    status: str = Query(default="DRAFT", description="状态过滤"),
):
    from emily_core.repositories.evolution_repo import EvolutionRepo
    import asyncio
    patches = await asyncio.to_thread(EvolutionRepo.get_patches_by_status, status)
    return {
        "patches": [
            {
                "patch_no": p.patch_no,
                "rule_no": p.rule_no,
                "target_path": p.target_path,
                "patch_type": p.patch_type,
                "risk_level": p.risk_level,
                "status": p.status,
                "risk_reasoning": p.risk_reasoning,
                "created_at": p.created_at,
            }
            for p in patches
        ]
    }

@router.post("/patches/{patch_no}/approve")
async def approve_patch(patch_no: str):
    from emily_core.services.evolution.patch_applier import PatchApplier
    applier = PatchApplier()
    result = await applier.approve(patch_no)
    if result.get("status") == "error":
        raise HTTPException(status_code=400, detail=result.get("message", "Unknown error"))

```

```

        return result
    @router.post("/patches/{patch_no}/reject")
    async def reject_patch(patch_no: str, reason: str = ""):
        from emily_core.services.evolution.patch_applier import PatchApplier
        applier = PatchApplier()
        result = await applier.reject(patch_no, reason)
        if result.get("status") == "error":
            raise HTTPException(status_code=400, detail=result.get("message", "Unknown
error"))
        return result
    @router.post("/patches/{patch_no}/rollback")
    async def rollback_patch(patch_no: str):
        from emily_core.services.evolution.patch_applier import PatchApplier
        applier = PatchApplier()
        result = await applier.rollback(patch_no)
        if result.get("status") == "error":
            raise HTTPException(status_code=400, detail=result.get("message", "Unknown
error"))
        return result

```

文件编号：174 文件路径：emily-core\api\routes\health.py

```

from __future__ import annotations
from fastapi import APIRouter
from ..server import get_core
router = APIRouter()
@router.get("/health")
async def health():
    core = get_core()
    return core.health()

```

文件编号：175 文件路径：emily-core\api\routes\message.py

```

from __future__ import annotations
import logging
from fastapi import APIRouter, Response
from pydantic import BaseModel, Field
from emily_core.adapters.standard.message import StandardMessage
from ..server import get_core
logger = logging.getLogger("emily.api.message")
router = APIRouter()
class MessageIn(BaseModel):
    message_id: str = ""
    platform: str = ""
    conversation_type: str = ""
    conversation_id: str = ""
    sender_id: str = ""
    sender_name: str = ""
    group_id: str | None = None

```

```

group_name: str | None = None
content: str = ""
is_at_bot: bool = False
mentioned_user_ids: list[str] = Field(default_factory=list)
reply_to_message_id: str | None = None
msg_type: int = 1
attachments: list[dict] = Field(default_factory=list)
event_id: str = ""
def to_standard(self) -> StandardMessage:
    return StandardMessage(
        message_id=self.message_id,
        platform=self.platform,
        conversation_type=self.conversation_type,
        conversation_id=self.conversation_id,
        sender_id=self.sender_id,
        sender_name=self.sender_name,
        group_id=self.group_id,
        group_name=self.group_name,
        content=self.content,
        is_at_bot=self.is_at_bot,
        mentioned_user_ids=self.mentioned_user_ids,
        reply_to_message_id=self.reply_to_message_id,
        msg_type=self.msg_type,
        attachments=self.attachments,
    )
class ReplyOut(BaseModel):
    conversation_id: str
    content: str
    reply_to_message_id: str | None = None
@router.post("/message/send")
async def handle_message(msg_in: MessageIn):
    core = get_core()
    reply = await core.handle_message(msg_in.to_standard(),
event_id=msg_in.event_id)
    if reply is None:
        return Response(status_code=204)
    return ReplyOut(
        conversation_id=reply.conversation_id,
        content=reply.content,
        reply_to_message_id=reply.reply_to_message_id,
    )

```

文件编号：176 文件路径：emily-core\api\routes\meta_cognition.py

```

from __future__ import annotations
import asyncio
import logging
from fastapi import APIRouter, HTTPException, Query
logger = logging.getLogger("emily.api.meta_cognition")
router = APIRouter(prefix="/api/v1/meta-cognition", tags=["meta-cognition"])
@router.post("/system-description/build")
async def build_system_description(

```



```

    force: bool = Query(default=False, description="是否强制重建（无视偏差检测）"),
    dry_run: bool = Query(default=False, description="预览模式（不写 DB）"),
):
    try:
        from emily_core.services.system_description_service import
SystemDescriptionService
        svc = SystemDescriptionService()
        if force:
            result = await svc.force_rebuild(dry_run=dry_run)
        else:
            result = await svc.check_and_update(dry_run=dry_run)
        return {"status": "ok", "data": result}
    except Exception as e:
        logger.error("build_system_description failed: %s", e, exc_info=True)
        raise HTTPException(status_code=500, detail=str(e))
@router.get("/system-description")
async def get_system_description():
    try:
        from emily_core.repositories.system_description_repo import
SystemDescriptionRepo
        desc = await asyncio.to_thread(SystemDescriptionRepo.get_latest)
        if desc is None:
            return {"status": "not_found", "message": "系统描述尚未构建"}
        return {
            "status": "ok",
            "version": desc.version,
            "token_count": desc.token_count,
            "generated_at": desc.generated_at,
            "generated_by": desc.generated_by,
            "schema_hash": desc.schema_hash[:12] + "...",
            "permission_hash": desc.permission_hash[:12] + "...",
            "file_model_hash": desc.file_model_hash[:12] + "...",
            "content_text": desc.content_text,
        }
    except Exception as e:
        logger.error("get_system_description failed: %s", e, exc_info=True)
        raise HTTPException(status_code=500, detail=str(e))
@router.post("/system-description/check")
async def check_system_description_drift():
    try:
        from emily_core.services.schema_drift_detector import SchemaDriftDetector
        detector = SchemaDriftDetector()
        result = detector.detect()
        return {"status": "ok", "data": result}
    except Exception as e:
        logger.error("check_system_description_drift failed: %s", e,
exc_info=True)
        raise HTTPException(status_code=500, detail=str(e))

```

文件编号: 177 文件路径: emily-core\api\routes\node.py

```
from __future__ import annotations
import logging
from fastapi import APIRouter, HTTPException, Query
from .node_schemas import (
    CreateNodeRequest,
    UpdateNodeRequest,
    ActivateNodeRequest,
    CreateDeliverableRequest,
    UpdateDeliverableProgressRequest,
    AddDependencyRequest,
    MountChildRequest,
    AssignNodeRequest,
    SubmitDeliverableRequest,
    ConfirmDeliverableRequest,
    ReturnDeliverableRequest,
    ResubmitDeliverableRequest,
    ApiResponse,
)
logger = logging.getLogger("emily.api.node")
router = APIRouter(prefix="/project-nodes", tags=["project-nodes"])
cross_router = APIRouter(tags=["node-cross"])
_service = None
def set_node_service(service) -> None:
    global _service
    _service = service
def _get_service():
    global _service
    if _service is not None:
        return _service
    try:
        from api.server import get_core
        core = get_core()
        core._ensure_initialized()
        _service = core._node_service
    except Exception:
        pass
    if _service is None:
        raise HTTPException(status_code=503, detail="Node graph module not
initialized")
    return _service
@router.post("", status_code=201)
async def create_node(body: CreateNodeRequest):
    from emily_core.services.node_commands import CreateNodeCommand
    svc = _get_service()
    cmd = CreateNodeCommand(
        project_id=body.project_id,
        node_id=body.node_id,
        node_name=body.node_name,
        owner_dept_id=body.owner_dept_id,
        related_company_id=body.related_company_id,
        deadline=body.deadline,
        creator_id=body.creator_id,
```

```

        parent_node_id=body.parent_node_id,
        stage_id=body.stage_id,
        child_weight=body.child_weight,
        remark=body.remark,
        land_parcel_id=body.land_parcel_id,
        startup_doc_id=body.startup_doc_id,
        sort_order=body.sort_order,
        responsible_user_id=getattr(body, 'responsible_user_id', ''),
        node_type=getattr(body, 'node_type', 'WORK_PACKAGE'),
    )
    result = await svc.create_node(cmd)
    if not result.success:
        raise HTTPException(status_code=400, detail=result.message)
    return ApiResponse(message=result.message, data={"node_id": result.node_id,
"status": result.status})
@router.get("/my-tasks")
async def my_tasks(user_id: str = Query(..., description="当前用户ID"),
                    project_id: str = Query(default=""),
                    submission_status: str = Query(default=""),
                    page: int = Query(default=1),
                    page_size: int = Query(default=20)):
    import asyncio
    from emily_core.repositories.node_repo import NodeDeliverableRepo
    rows = await asyncio.to_thread(
        NodeDeliverableRepo.find_pending_by_responsible_user,
        user_id, project_id or None, submission_status or "", "IN_PROGRESS",
        page_size, (page - 1) * page_size,
    )
    total = await asyncio.to_thread(
        NodeDeliverableRepo.count_pending_by_responsible_user,
        user_id, project_id or None, submission_status or "", "IN_PROGRESS",
    )
    items = []
    for deliv, node in rows:
        items.append({
            "node_id": node.node_id,
            "node_name": node.node_name,
            "project_id": node.project_id,
            "deadline": node.deadline,
            "node_type": getattr(node, "node_type", ""),
            "submission_status": deliv.submission_status,
            "parent_node_name": "",
            "deliverables": [{
                "deliverable_name": deliv.deliverable_name,
                "target_amount": deliv.target_amount,
                "current_amount": deliv.current_amount,
            }],
        })
    return ApiResponse(data={"items": items, "total": total, "page": page,
"page_size": page_size})
@router.get("/{node_id}")
async def get_node_detail(
    node_id: str,
    include: str = Query(default=""),

```

```
description="children,deliverables,dependencies（逗号分隔，预留）"),
):
    svc = _get_service()
    detail = await svc.get_node_detail(node_id)
    if detail is None:
        raise HTTPException(status_code=404, detail=f"节点 {node_id} 不存在")
    return ApiResponse(data=detail)
@router.patch("/{node_id}")
async def update_node(node_id: str, body: UpdateNodeRequest):
    from emily_core.services.node_commands import UpdateNodeCommand
    svc = _get_service()
    cmd = UpdateNodeCommand(
        node_id=node_id,
        operator_id=body.operator_id,
        node_name=body.node_name,
        deadline=body.deadline,
        owner_dept_id=body.owner_dept_id,
        related_company_id=body.related_company_id,
        remark=body.remark,
        stage_id=body.stage_id,
        sort_order=body.sort_order,
        land_parcel_id=body.land_parcel_id,
        startup_doc_id=body.startup_doc_id,
    )
    result = await svc.update_node(cmd)
    if not result.success:
        raise HTTPException(status_code=400, detail=result.message)
    return ApiResponse(message=result.message)
@router.patch("/{node_id}/assign")
async def assign_node(node_id: str, body: AssignNodeRequest):
    from emily_core.services.node_commands import AssignNodeCommand
    svc = _get_service()
    cmd = AssignNodeCommand(
        node_id=node_id,
        responsible_user_id=body.responsible_user_id,
        operator_id=body.operator_id,
    )
    result = await svc.assign_node(cmd)
    if not result.success:
        status_code = 403 if result.error_code == "40302" else 400
        raise HTTPException(status_code=status_code, detail=result.message)
    return ApiResponse(message=result.message)
@cross_router.post("/node-deliverables/{deliverable_id}/submit")
async def submit_deliverable(deliverable_id: str, body: SubmitDeliverableRequest,
                             user_id: str = Query(default="")):
    from emily_core.services.node_commands import SubmitNodeDeliverableCommand
    svc = _get_service()
    cmd = SubmitNodeDeliverableCommand(
        deliverable_id=deliverable_id,
        content=body.content,
        file_url=body.file_url,
        file_name=body.file_name,
        attachment_file_id=body.attachment_file_id,
        submitted_by=user_id,
```

```

        is_acceptance_check=body.is_acceptance_check,
    )
    result = await svc.submit_deliverable(cmd)
    if not result.success:
        raise HTTPException(status_code=400, detail=result.message)
    return ApiResponse(message=result.message)
@cross_router.post("/node-deliverables/{deliverable_id}/confirm")
async def confirm_deliverable(d deliverable_id: str, body:
ConfirmDeliverableRequest,
                                user_id: str = Query(default="")):
    from emily_core.services.node_commands import ConfirmNodeDeliverableCommand
    svc = _get_service()
    cmd = ConfirmNodeDeliverableCommand(
        deliverable_id=deliverable_id,
        confirmed_by=user_id,
    )
    result = await svc.confirm_deliverable(cmd)
    if not result.success:
        raise HTTPException(status_code=400, detail=result.message)
    return ApiResponse(message=result.message)
@cross_router.post("/node-deliverables/{deliverable_id}/return")
async def return_deliverable(d deliverable_id: str, body: ReturnDeliverableRequest,
                                user_id: str = Query(default="")):
    from emily_core.services.node_commands import ReturnNodeDeliverableCommand
    svc = _get_service()
    cmd = ReturnNodeDeliverableCommand(
        deliverable_id=deliverable_id,
        returned_by=user_id,
        reason=body.reason,
    )
    result = await svc.return_deliverable(cmd)
    if not result.success:
        raise HTTPException(status_code=400, detail=result.message)
    return ApiResponse(message=result.message)
@cross_router.post("/node-deliverables/{deliverable_id}/resubmit")
async def resubmit_deliverable(d deliverable_id: str, body:
ResubmitDeliverableRequest,
                                user_id: str = Query(default="")):
    from emily_core.services.node_commands import ResubmitNodeDeliverableCommand
    svc = _get_service()
    cmd = ResubmitNodeDeliverableCommand(
        deliverable_id=deliverable_id,
        content=body.content,
        file_url=body.file_url,
        file_name=body.file_name,
        attachment_file_id=body.attachment_file_id,
        submitted_by=user_id,
    )
    result = await svc.resubmit_deliverable(cmd)
    if not result.success:
        raise HTTPException(status_code=400, detail=result.message)
    return ApiResponse(message=result.message)
@router.delete("/{node_id}")
async def discard_node(node_id: str, operator_id: str = Query(default="")):

```

```
from emily_core.services.node_commands import DiscardNodeCommand
svc = _get_service()
cmd = DiscardNodeCommand(node_id=node_id, operator_id=operator_id)
result = await svc.discard_node(cmd)
if not result.success:
    raise HTTPException(status_code=400, detail=result.message)
return ApiResponse(message=result.message)

@router.post("/{node_id}/activate")
async def activate_node(node_id: str, body: ActivateNodeRequest):
    from emily_core.services.node_commands import ActivateNodeCommand
    svc = _get_service()
    cmd = ActivateNodeCommand(
        node_id=node_id,
        approver_id=body.approver_id,
        approved=body.approved,
        remark=body.remark,
    )
    result = await svc.activate_node(cmd)
    if not result.success:
        status_code = 403 if result.error_code == "40302" else 400
        raise HTTPException(status_code=status_code, detail=result.message)
    return ApiResponse(
        message=result.message,
        data={"node_id": result.node_id, "status": result.status},
    )

@router.post("/{node_id}/deliverables", status_code=201)
async def create_deliverable(node_id: str, body: CreateDeliverableRequest):
    from emily_core.services.node_commands import CreateDeliverableCommand
    svc = _get_service()
    cmd = CreateDeliverableCommand(
        node_id=node_id,
        deliverable_name=body.deliverable_name,
        target_amount=body.target_amount,
        unit=body.unit,
        is_required=body.is_required,
        operator_id=body.operator_id,
    )
    result = await svc.create_deliverable(cmd)
    if not result.success:
        raise HTTPException(status_code=400, detail=result.message)
    return ApiResponse(message=result.message)

@cross_router.patch("/node-deliverables/{deliverable_id}")
async def update_deliverable_progress(deliverable_id: str, body:
UpdateDeliverableProgressRequest):
    from emily_core.services.node_commands import UpdateDeliverableProgressCommand
    svc = _get_service()
    cmd = UpdateDeliverableProgressCommand(
        deliverable_id=deliverable_id,
        current_amount=body.current_amount,
        file_id=body.file_id,
        operator_id=body.operator_id,
    )
    result = await svc.update_deliverable_progress(cmd)
    if not result.success:
```

```

        raise HTTPException(status_code=400, detail=result.message)
    return ApiResponse(
        message=result.message,
        data={
            "node_id": result.node_id,
            "status": result.status,
            "progress": result.progress,
            "affected_ancestors": result.affected_downstream,
        },
    )
)

@router.post("/{node_id}/dependencies", status_code=201)
async def add_dependency(node_id: str, body: AddDependencyRequest):
    from emily_core.services.node_commands import AddDependencyCommand
    svc = _get_service()
    cmd = AddDependencyCommand(
        node_id=node_id,
        depends_on_deliverable_id=body.depends_on_deliverable_id,
        weight=body.weight,
        dependency_type=body.dependency_type,
        operator_id=body.operator_id,
    )
    result = await svc.add_dependency(cmd)
    if not result.success:
        status_code = 400
        if result.error_code == "40001":
            status_code = 400
        raise HTTPException(status_code=status_code, detail=result.message)
    return ApiResponse(message=result.message)

@cross_router.delete("/node-dependencies/{dependency_id}")
async def remove_dependency(dependency_id: str, operator_id: str =
Query(default="")):
    from emily_core.services.node_commands import RemoveDependencyCommand
    svc = _get_service()
    cmd = RemoveDependencyCommand(dependency_id=dependency_id,
operator_id=operator_id)
    result = await svc.remove_dependency(cmd)
    if not result.success:
        raise HTTPException(status_code=400, detail=result.message)
    return ApiResponse(message=result.message)

@router.post("/{parent_node_id}/children", status_code=201)
async def mount_child(parent_node_id: str, body: MountChildRequest):
    from emily_core.services.node_commands import MountChildCommand
    svc = _get_service()
    cmd = MountChildCommand(
        parent_node_id=parent_node_id,
        child_node_id=body.child_node_id,
        child_weight=body.child_weight,
        operator_id=body.operator_id,
    )
    result = await svc.mount_child(cmd)
    if not result.success:
        status_code = 400
        if result.error_code == "40001":
            status_code = 400

```

```

        elif result.error_code == "40002":
            status_code = 400
            raise HTTPException(status_code=status_code, detail=result.message)
        return ApiResponse(message=result.message)
@router.delete("/{parent_node_id}/children/{child_node_id}")
async def unmount_child(parent_node_id: str, child_node_id: str, operator_id: str
= Query(default="")):
    from emily_core.services.node_commands import UnmountChildCommand
    svc = _get_service()
    cmd = UnmountChildCommand(
        parent_node_id=parent_node_id,
        child_node_id=child_node_id,
        operator_id=operator_id,
    )
    result = await svc.unmount_child(cmd)
    if not result.success:
        raise HTTPException(status_code=400, detail=result.message)
    return ApiResponse(message=result.message)

```

文件编号：178 文件路径：emily-core\api\routes\node_schemas.py

```

from __future__ import annotations
from pydantic import BaseModel, Field
class ApiResponse(BaseModel):
    code: int = 0
    message: str = "success"
    data: dict | None = None
class ErrorResponse(BaseModel):
    code: int = 40001
    message: str = ""
    detail: str = ""
class CreateNodeRequest(BaseModel):
    project_id: str = Field(..., description="项目归属ID")
    node_id: str = Field(..., description="节点编号（业务主键），例：SG-JG-01-2026")
    node_name: str = Field(..., description="节点名称")
    owner_dept_id: str = Field(default="项目总", description="主责条线")
    related_company_id: str = Field(default="建设单位", description="关联单位")
    deadline: str = Field(..., description="截止时间（ISO8601）")
    parent_node_id: str = Field(default="", description="父节点ID")
    stage_id: int = Field(default=0, description="所属阶段ID")
    child_weight: float = Field(default=1.0, description="子节点权重")
    remark: str = Field(default="", description="备注")
    land_parcel_id: str = Field(default="", description="关联地块ID")
    sort_order: int = Field(default=0, description="排序序号")
    creator_id: str = Field(default="", description="创建人ID")
    startup_doc_id: str = Field(default="", description="启动文档ID")
    responsible_user_id: str = Field(default="", description="责任人ID（为空时取creator_id）")
    node_type: str = Field(default="WORK_PACKAGE", description="节点类型：MILESTONE / WORK_PACKAGE / TASK")
class UpdateNodeRequest(BaseModel):
    node_name: str | None = Field(default=None, description="节点名称")

```



```

deadline: str | None = Field(default=None, description="截止时间")
owner_dept_id: str | None = Field(default=None, description="主责条线")
related_company_id: str | None = Field(default=None, description="关联单位")
remark: str | None = Field(default=None, description="备注")
stage_id: int | None = Field(default=None, description="阶段ID")
sort_order: int | None = Field(default=None, description="排序序号")
land_parcel_id: str | None = Field(default=None, description="地块ID")
startup_doc_id: str | None = Field(default=None, description="启动文档ID")
operator_id: str = Field(default="", description="操作人ID")
class ActivateNodeRequest(BaseModel):
    approved: bool = Field(..., description="是否通过审批")
    approver_id: str = Field(..., description="审批人ID")
    remark: str = Field(default="", description="审批意见")
class CreateDeliverableRequest(BaseModel):
    deliverable_name: str = Field(..., description="成果名称")
    target_amount: float = Field(..., description="目标量")
    unit: str = Field(..., description="量纲（份/吨/平方米...）")
    is_required: bool = Field(default=True, description="是否必需成果")
    operator_id: str = Field(default="", description="操作人ID")
class UpdateDeliverableProgressRequest(BaseModel):
    current_amount: float = Field(..., description="当前量")
    file_id: str = Field(default="", description="关联文件ID")
    operator_id: str = Field(default="", description="操作人ID")
class AddDependencyRequest(BaseModel):
    depends_on_deliverable_id: str = Field(..., description="依赖的成果ID")
    weight: float = Field(default=1.0, description="权重（0.0000-1.0000，阻塞场景用
≥999）")
    dependency_type: str = Field(default="DELIVERABLE", description="DELIVERABLE /
TIME")
    operator_id: str = Field(default="", description="操作人ID")
class MountChildRequest(BaseModel):
    child_node_id: str = Field(..., description="子节点编号")
    child_weight: float = Field(default=1.0, description="子节点权重")
    operator_id: str = Field(default="", description="操作人ID")
class AssignNodeRequest(BaseModel):
    responsible_user_id: str = Field(..., description="新责任人ID（users.id）")
    operator_id: str = Field(default="", description="操作人ID")
class SubmitDeliverableRequest(BaseModel):
    content: str = Field(default="", description="提交内容")
    file_url: str = Field(default="", description="文件URL")
    file_name: str = Field(default="", description="文件名")
    attachment_file_id: str = Field(default="", description="附件文件ID")
    is_acceptance_check: bool = Field(default=False, description="是否为完工确认报
告")
class ConfirmDeliverableRequest(BaseModel):
    reason: str = Field(default="", description="确认说明")
    operator_id: str = Field(default="", description="确认人ID")
class ReturnDeliverableRequest(BaseModel):
    reason: str = Field(..., description="退回原因（必填）")
    operator_id: str = Field(default="", description="退回人ID")
class ResubmitDeliverableRequest(BaseModel):
    content: str = Field(default="", description="重新提交内容")
    file_url: str = Field(default="", description="文件URL")
    file_name: str = Field(default="", description="文件名")

```

```

attachment_file_id: str = Field(default="", description="附件文件ID")
class MyTasksRequest(BaseModel):
    project_id: str = Field(default="", description="项目ID（可选）")
    submission_status: str = Field(default="", description="提交状态过滤")
    page: int = Field(default=1, description="页码")
    page_size: int = Field(default=20, description="每页数量")

```

文件编号：179 文件路径：emily-core\api\routes\permission.py

```

from __future__ import annotations
import logging
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel, Field
from emily_core.application.permission_app import PermissionApplication
logger = logging.getLogger("emily.api.permission")
router = APIRouter(prefix="/permission", tags=["permission"])
_app: PermissionApplication | None = None
def set_permission_app(app: PermissionApplication) -> None:
    global _app
    _app = app
def _get_app() -> PermissionApplication:
    global _app
    if _app is not None:
        return _app
    try:
        from api.server import get_core
        core = get_core()
        core._ensure_initialized()
        _app = core._permission_app
    except Exception:
        pass
    if _app is None:
        raise HTTPException(status_code=503, detail="Permission module not
initialized")
    return _app
class CheckRequest(BaseModel):
    user_id: str = Field(..., description="用户 ID")
    sop_id: str = Field(..., description="SOP 编号")
class GrantRequest(BaseModel):
    grantee_id: str = Field(..., description="被授权人 ID")
    grantor_id: str = Field(..., description="授权人 ID")
    perm_code: str = Field(..., description="权限编码")
    grant_type: str = Field(default="TEMP", description="授权类型:
AUTO/TEMP/PERMANENT")
    operations: str = Field(default='["read"]', description="操作 JSON")
    expire_time: str | None = Field(default=None, description="过期时间（TEMP 必
填）")
    remark: str = Field(default="", description="授权原因（PERMANENT 必填）")
    client_ip: str = Field(default="", description="授权人 IP")
class RevokeRequest(BaseModel):
    grant_no: str = Field(..., description="授权编号")
    revoke_reason: str = Field(default="", description="撤销原因")

```

```

    operator_id: str = Field(default="", description="操作人 ID")
class PermissionRequest(BaseModel):
    requester_id: str = Field(..., description="申请人 ID")
    perm_code: str = Field(..., description="申请的权限编码")
    request_type: str = Field(default="TEMP_GRANT", description="申请类型:
TEMP_GRANT/UNIT_BIND/LEVEL_UP/ANOMALY_DATA")
    reason: str = Field(default="", description="申请理由")
    priority: str = Field(default="NORMAL", description="优先级:
NORMAL/HIGH/URGENT")
class ApproveRequest(BaseModel):
    request_no: str = Field(..., description="申请编号")
    approver_id: str = Field(..., description="审批人 ID")
    approved: bool = Field(..., description="是否通过")
    remark: str = Field(default="", description="审批意见")
@router.post("/check")
async def check_permission(body: CheckRequest):
    app = _get_app()
    result = await app.check_permission(body.user_id, body.sop_id)
    if not result.get("success"):
        raise HTTPException(status_code=400, detail=result.get("reply", ""))
    return result
@router.post("/grant")
async def grant_permission(body: GrantRequest):
    app = _get_app()
    result = await app.grant_permission(
        grantee_id=body.grantee_id,
        grantor_id=body.grantor_id,
        perm_code=body.perm_code,
        grant_type=body.grant_type,
        operations=body.operations,
        expire_time=body.expire_time,
        remark=body.remark,
        client_ip=body.client_ip,
    )
    if not result.get("success"):
        raise HTTPException(status_code=400, detail=result.get("reply", ""))
    return result
@router.post("/revoke")
async def revoke_permission(body: RevokeRequest):
    app = _get_app()
    result = await app.revoke_permission(
        grant_no=body.grant_no,
        revoke_reason=body.revoke_reason,
        operator_id=body.operator_id,
    )
    if not result.get("success"):
        raise HTTPException(status_code=400, detail=result.get("reply", ""))
    return result
@router.get("/user/{user_id}")
async def query_user_permissions(user_id: str):
    app = _get_app()
    result = await app.query_user_permissions(user_id)
    if not result.get("success"):
        raise HTTPException(status_code=404, detail=result.get("reply", ""))

```

```

        return result
    @router.post("/request")
    async def request_permission(body: PermissionRequest):
        app = _get_app()
        result = await app.request_permission(
            requester_id=body.requester_id,
            perm_code=body.perm_code,
            request_type=body.request_type,
            reason=body.reason,
            priority=body.priority,
        )
        if not result.get("success"):
            raise HTTPException(status_code=400, detail=result.get("reply", ""))
        return result
    @router.post("/approve")
    async def approve_request(body: ApproveRequest):
        app = _get_app()
        result = await app.approve_request(
            request_no=body.request_no,
            approver_id=body.approver_id,
            approved=body.approved,
            remark=body.remark,
        )
        if not result.get("success"):
            raise HTTPException(status_code=400, detail=result.get("reply", ""))
        return result

```

文件编号: 180 文件路径: emily-core\api\routes\session.py

```

from __future__ import annotations
import logging
from fastapi import APIRouter
from pydantic import BaseModel
from ..server import get_core
logger = logging.getLogger("emily.api.session")
router = APIRouter()
class TerminateIn(BaseModel):
    conversation_id: str
    @router.post("/session/terminate")
    async def terminate_session(body: TerminateIn):
        core = get_core()
        ok = await core.terminate_session(body.conversation_id)
        return {"terminated": ok, "conversation_id": body.conversation_id}

```

文件编号: 181 文件路径: emily-core\api\routes\skills.py

```

from __future__ import annotations
from fastapi import APIRouter
from ..server import get_core

```

```
router = APIRouter(tags=["skills"])
@router.post("/skills/reload")
async def reload_skills():
    core = get_core()
    return core.reload_skills()
```

文件编号：182 文件路径：emily-core\api\server.py

```
from __future__ import annotations
import logging
from contextlib import asynccontextmanager
from fastapi import FastAPI
logger = logging.getLogger("emily.api")
_core = None
def get_core():
    if _core is None:
        raise RuntimeError("EmilyCore not initialized")
    return _core
@asynccontextmanager
async def lifespan(app: FastAPI):
    global _core
    from emily_core import bootstrap
    logger.info("Emily Core API starting - bootstrapping EmilyCore...")
    _core = bootstrap.init()
    logger.info("Emily Core API ready")
    yield
    logger.info("Emily Core API shutting down")
    _core = None
app = FastAPI(title="Emily Core API", version="1.0", lifespan=lifespan)
from .middleware.auth import AuthMiddleware # noqa: E402
app.add_middleware(AuthMiddleware)
from .routes import health, message, session, permission, skills # noqa: E402
from .sse import outbound # noqa: E402
app.include_router(health.router, prefix="/api/v1")
app.include_router(message.router, prefix="/api/v1")
app.include_router(session.router, prefix="/api/v1")
app.include_router(permission.router, prefix="/api/v1")
app.include_router(skills.router, prefix="/api/v1")
app.include_router(outbound.router, prefix="/api/v1")
from .routes import node as node_routes # noqa: E402
from .sse import node_events # noqa: E402
app.include_router(node_routes.router, prefix="/api/v1")
app.include_router(node_routes.cross_router, prefix="/api/v1")
app.include_router(node_events.router, prefix="/api/v1")
from .routes import evolution # noqa: E402
app.include_router(evolution.router)
from .routes import meta_cognition # noqa: E402
app.include_router(meta_cognition.router)
```

文件编号：183 文件路径：emily-core\api\sse\node_events.py

```

from __future__ import annotations
import asyncio
import json
import logging
import uuid
from fastapi import APIRouter, Query, Request
from fastapi.responses import StreamingResponse
logger = logging.getLogger("emily.sse.node_events")
router = APIRouter(prefix="/events", tags=["sse"])
_bus = None
def set_node_event_bus(bus) -> None:
    global _bus
    _bus = bus
async def _event_generator(request: Request, project_id: str = ""):
    sub_id = str(uuid.uuid4())
    if _bus is None:
        yield f"event: error\ndata: {json.dumps({'message': 'NodeEventBus not initialized'})}\n\n"
        return
    from emily_core.node_event_bus import SubscriptionFilter
    filter_ = SubscriptionFilter(project_id=project_id) if project_id else None
    queue: asyncio.Queue = _bus.subscribe(sub_id, filter_)
    try:
        yield f"event: connected\ndata: {json.dumps({'sub_id': sub_id, 'project_id': project_id})}\n\n"
        while True:
            if await request.is_disconnected():
                break
            try:
                event = await asyncio.wait_for(queue.get(), timeout=15.0)
            except asyncio.TimeoutError:
                yield f": heartbeat\n\n"
                continue
            yield (
                f"event: node_event\n"
                f"data: {json.dumps(event.to_outbound(), ensure_ascii=False)}\n\n"
            )
    finally:
        _bus.unsubscribe(sub_id)
        logger.debug("SSE node_events connection closed: %s", sub_id)
@router.get("/nodes")
async def node_events_stream(
    request: Request,
    project_id: str = Query(default="", description="按项目过滤（为空则接收所有项目事件）"),
):
    return StreamingResponse(
        _event_generator(request, project_id),
        media_type="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "Connection": "keep-alive",
        }
    )

```

```
        "X-Accel-Buffering": "no",
    },
)
```

文件编号: 184 文件路径: emily-core\api\sse\outbound.py

```
from __future__ import annotations
import asyncio
import json
import logging
from fastapi import APIRouter, Request
from fastapi.responses import StreamingResponse
from ..server import get_core
logger = logging.getLogger("emily.api.sse")
router = APIRouter()
@router.get("/events/outbound")
async def outbound_events(request: Request):
    core = get_core()
    bus = core.outbound_bus
    queue = bus.subscribe()
    async def event_generator():
        try:
            while True:
                if await request.is_disconnected():
                    break
                try:
                    event = await asyncio.wait_for(queue.get(), timeout=15.0)
                except asyncio.TimeoutError:
                    yield ": keep-alive\n\n"
                    continue
                etype = event.get("type", "message")
                data = json.dumps(event.get("data", {}), ensure_ascii=False)
                yield f"event: {etype}\ndata: {data}\n\n"
        finally:
            bus.unsubscribe(queue)
    return StreamingResponse(event_generator(), media_type="text/event-stream")
```

文件编号: 185 文件路径: scripts\build_system_description.py

```
from __future__ import annotations
import argparse
import io
import json
import logging
import os
import subprocess
import sys
from pathlib import Path
_HERE = Path(__file__).resolve().parent
```

```

_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger("build_system_description")
def _detect_docker_pg_port() -> int | None:
    try:
        result = subprocess.run(
            ["docker", "port", "emily-postgres", "5432/tcp"],
            capture_output=True, text=True, timeout=5,
        )
        if result.returncode == 0 and result.stdout.strip():
            port_str = result.stdout.strip().rsplit(":", 1)[-1]
            return int(port_str)
    except (FileNotFoundError, subprocess.TimeoutExpired, ValueError, OSError):
        pass
    return None
def _init_db(db_url: str = "") -> None:
    from emily_core.infrastructure.database.session import init_db
    if db_url:
        init_db(db_url=db_url)
    else:
        db_url_env = os.environ.get("EMILY_DATABASE_URL", "")
        if db_url_env:
            init_db(db_url=db_url_env)
        else:
            pg_host = os.environ.get("EMILY_PG_HOST", "127.0.0.1")
            pg_port_env = os.environ.get("EMILY_PG_PORT")
            if pg_port_env:
                pg_port = int(pg_port_env)
            else:
                pg_port = _detect_docker_pg_port() or 5432
            pg_db = os.environ.get("EMILY_PG_DB", "emily")
            pg_user = os.environ.get("EMILY_PG_USER", "emily")
            pg_password = os.environ.get("EMILY_PG_PASSWORD", "emily_secret_2026")
            init_db(pg_host=pg_host, pg_port=pg_port, pg_db=pg_db,
pg_user=pg_user, pg_password=pg_password)
def build_system_description(*, generated_by: str = "manual", db_url: str = "",
dry_run: bool = False) -> dict:
    _init_db(db_url)
    from emily_core.services.system_description_builder import
SystemDescriptionBuilder
    builder = SystemDescriptionBuilder()
    return builder.build(generated_by=generated_by, dry_run=dry_run)
def check_drift(*, db_url: str = "") -> dict:
    _init_db(db_url)
    from emily_core.services.schema_drift_detector import SchemaDriftDetector
    detector = SchemaDriftDetector()
    return detector.detect()
def main():

```



```

sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',
errors='replace')
parser = argparse.ArgumentParser(description="构建/重建/检测系统自我描述")
parser.add_argument("--dry-run", action="store_true", help="预览模式（不写
DB）")
parser.add_argument("--check-only", action="store_true", help="仅检测偏差（不构
建）")
parser.add_argument("--force", action="store_true", help="强制重建（无视偏差检
测）")
parser.add_argument("--db-url", default="", help="PostgreSQL 连接 URL")
args = parser.parse_args()
if args.check_only:
    result = check_drift(db_url=args.db_url)
    print(f"has_description: {result.get('has_description')}")
    print(f"has_drift: {result.get('has_drift')}")
    stale = result.get("stale_domains", [])
    if stale:
        print(f"stale_domains: {stale}")
        drift = result.get("drift", {})
        for domain, info in drift.items():
            if info.get("stale"):
                signals = info.get("signals", [])
                print(f"  {domain}: {signals}")
    else:
        print("系统描述与当前代码结构一致，无需更新")
    return
if args.force:
    result = build_system_description(generated_by="manual_force",
db_url=args.db_url, dry_run=args.dry_run)
else:
    result = build_system_description(generated_by="manual",
db_url=args.db_url, dry_run=args.dry_run)
    summary = {k: v for k, v in result.items() if k not in ("content_json",
"content_text")}
    print(json.dumps(summary, ensure_ascii=False, indent=2, default=str))
    if args.dry_run:
        print(f"\n--- content_text 预览 ---")
        print(result.get("content_text", ""))
if __name__ == "__main__":
    main()

```

文件编号：186 文件路径：scripts\build_world_book.py

```

from __future__ import annotations
import argparse
import io
import json
import logging
import os
import subprocess

```

```

import sys
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger("build_world_book")
def _detect_docker_pg_port() -> int | None:
    try:
        result = subprocess.run(
            ["docker", "port", "emily-postgres", "5432/tcp"],
            capture_output=True, text=True, timeout=5,
        )
        if result.returncode == 0 and result.stdout.strip():
            port_str = result.stdout.strip().rsplit(":", 1)[-1]
            return int(port_str)
    except (FileNotFoundError, subprocess.TimeoutExpired, ValueError, OSError):
        pass
    return None
def _init_db(db_url: str = "") -> None:
    from emily_core.infrastructure.database.session import init_db
    if db_url:
        init_db(db_url=db_url)
    else:
        db_url_env = os.environ.get("EMILY_DATABASE_URL", "")
        if db_url_env:
            init_db(db_url=db_url_env)
        else:
            pg_host = os.environ.get("EMILY_PG_HOST", "127.0.0.1")
            pg_port_env = os.environ.get("EMILY_PG_PORT")
            if pg_port_env:
                pg_port = int(pg_port_env)
            else:
                pg_port = _detect_docker_pg_port() or 5432
            pg_db = os.environ.get("EMILY_PG_DB", "emily")
            pg_user = os.environ.get("EMILY_PG_USER", "emily")
            pg_password = os.environ.get("EMILY_PG_PASSWORD", "emily_secret_2026")
            init_db(pg_host=pg_host, pg_port=pg_port, pg_db=pg_db,
pg_user=pg_user, pg_password=pg_password)
def build_world_book(project_id: str, *, generated_by: str = "manual", db_url: str
= "", dry_run: bool = False) -> dict:
    _init_db(db_url)
    from emily_core.services.world_book_builder import ProjectWorldBookBuilder
    builder = ProjectWorldBookBuilder()
    return builder.build(project_id, generated_by=generated_by, dry_run=dry_run)
def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',

```

```

errors='replace')
parser = argparse.ArgumentParser(description="构建/重建项目世界书")
parser.add_argument("--project-id", help="项目 ID (UUID)")
parser.add_argument("--all", action="store_true", help="构建所有 active 项目")
parser.add_argument("--db-url", default="", help="PostgreSQL 连接 URL")
parser.add_argument("--dry-run", action="store_true", help="预览模式 (不写
DB)")
args = parser.parse_args()
if not args.project_id and not args.all:
    parser.error("请指定 --project-id 或 --all")
if args.all:
    _init_db(args.db_url)
    from emily_core.infrastructure.database.models import Project
    from emily_core.infrastructure.database.session import get_session
    with get_session() as session:
        projects = session.query(Project).filter(Project.is_deleted == False,
Project.status == "active").all()
        print(f"找到 {len(projects)} 个 active 项目")
        for p in projects:
            print(f"\n=== 项目: {p.name} ({p.id}) ===")
            result = build_world_book(p.id, generated_by="manual",
db_url=args.db_url, dry_run=args.dry_run)
            print(f"状态: {result.get('status')}")
            print(f"初始化层级: T{result.get('initialization_tier', 0)}")
            print(f"Token 数: {result.get('token_count', 0)}")
            if args.dry_run:
                print(f"\n--- content_text 预览 ---")
                print(result.get("content_text", ""))
        else:
            result = build_world_book(args.project_id, generated_by="manual",
db_url=args.db_url, dry_run=args.dry_run)
            print(json.dumps(
                {k: v for k, v in result.items() if k not in ("content_json",)},
                ensure_ascii=False, indent=2, default=str,
            ))
            if args.dry_run:
                print(f"\n--- content_text 预览 ---")
                print(result.get("content_text", ""))
if __name__ == "__main__":
    main()

```

文件编号: 187 文件路径: scripts/check_initialization.py

```

from __future__ import annotations
import argparse
import io
import json
import logging
import os
import subprocess
import sys
from pathlib import Path

```

```

_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger("check_initialization")
def _detect_docker_pg_port() -> int | None:
    try:
        result = subprocess.run(
            ["docker", "port", "emily-postgres", "5432/tcp"],
            capture_output=True, text=True, timeout=5,
        )
        if result.returncode == 0 and result.stdout.strip():
            return int(result.stdout.strip().rsplit(":", 1)[-1])
    except Exception:
        pass
    return None
def _init_db(db_url: str = "") -> None:
    from emily_core.infrastructure.database.session import init_db
    if db_url:
        init_db(db_url=db_url)
    else:
        db_url_env = os.environ.get("EMILY_DATABASE_URL", "")
        if db_url_env:
            init_db(db_url=db_url_env)
        else:
            pg_host = os.environ.get("EMILY_PG_HOST", "127.0.0.1")
            pg_port = int(os.environ.get("EMILY_PG_PORT", "")) if
os.environ.get("EMILY_PG_PORT") else _detect_docker_pg_port() or 5432
            init_db(pg_host=pg_host, pg_port=pg_port,
                    pg_db=os.environ.get("EMILY_PG_DB", "emily"),
                    pg_user=os.environ.get("EMILY_PG_USER", "emily"),
                    pg_password=os.environ.get("EMILY_PG_PASSWORD",
"emily_secret_2026"))
def check_initialization(project_id: str, *, db_url: str = "") -> dict:
    _init_db(db_url)
    from emily_core.services.initialization_checker import InitializationChecker
    checker = InitializationChecker()
    return checker.check(project_id)
def _format_report(result: dict) -> str:
    lines = []
    lines.append("Emily 项目初始化检查报告")
    lines.append("=" * 40)
    lines.append(f"初始化层级: {result['tier_label']}")
    lines.append(f"({result['total_done']}/{result['total_items']} 必备项)")
    lines.append("=" * 40)
    for tier_key in ["T1", "T2", "T3", "T4"]:
        summary = result["summary_by_tier"].get(tier_key, {})
        done = summary.get("done", 0)
        total = summary.get("total", 0)

```

```

        icon = "PASS" if done >= total else "FAIL" if done == 0 else "WARN"
        lines.append(f"\n[{icon}] {tier_key} ({done}/{total}) ")
        tier_items = {k: v for k, v in result["items"].items() if
k.startswith(tier_key + "_")}
        for k, v in tier_items.items():
            lines.append(f"  {'[x]'} if v else '[' ]' {k}")
        if result["missing"]:
            lines.append(f"\n下一步: 补充缺失项以提升初始化层级")
        return "\n".join(lines)
def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',
errors='replace')
    parser = argparse.ArgumentParser(description="检查项目初始化层级")
    parser.add_argument("--project-id", help="项目 ID (UUID)")
    parser.add_argument("--all", action="store_true", help="检查所有 active 项目")
    parser.add_argument("--db-url", default="", help="PostgreSQL 连接 URL")
    parser.add_argument("--dry-run", action="store_true", help="仅预览（本项目无副作
用，始终可安全运行）")
    parser.add_argument("--json", action="store_true", help="输出原始 JSON")
    args = parser.parse_args()
    if not args.project_id and not args.all:
        parser.error("请指定 --project-id 或 --all")
    if args.all:
        _init_db(args.db_url)
        from emily_core.infrastructure.database.models import Project
        from emily_core.infrastructure.database.session import get_session
        with get_session() as session:
            projects = session.query(Project).filter(Project.is_deleted == False,
Project.status == "active").all()
            for p in projects:
                result = check_initialization(p.id, db_url=args.db_url)
                if args.json:
                    print(json.dumps(result, ensure_ascii=False, indent=2))
                else:
                    print(_format_report(result))
                    print()
    else:
        result = check_initialization(args.project_id, db_url=args.db_url)
        if args.json:
            print(json.dumps(result, ensure_ascii=False, indent=2))
        else:
            print(_format_report(result))
if __name__ == "__main__":
    main()

```

文件编号: 188 文件路径: scripts\cognition_cycle.py

```

from __future__ import annotations
import argparse
import asyncio

```

```

import io
import json
import logging
import os
import subprocess
import sys
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(name)s: %(message)s")
logger = logging.getLogger("cognition_cycle")
def _init_db(db_url: str = "") -> None:
    from emily_core.infrastructure.database.session import init_db
    if db_url:
        init_db(db_url=db_url)
    else:
        db_url_env = os.environ.get("EMILY_DATABASE_URL", "")
        if db_url_env:
            init_db(db_url=db_url_env)
        else:
            pg_port = int(os.environ.get("EMILY_PG_PORT", "")) if
os.environ.get("EMILY_PG_PORT") else None
            if not pg_port:
                try:
                    r = subprocess.run(["docker", "port", "emily-postgres",
"5432/tcp"], capture_output=True, text=True, timeout=5)
                    pg_port = int(r.stdout.strip().rsplit(":", 1)[-1]) if
r.returncode == 0 and r.stdout.strip() else 5432
                except Exception:
                    pg_port = 5432
            init_db(pg_host=os.environ.get("EMILY_PG_HOST", "127.0.0.1"),
pg_port=pg_port,
                    pg_db=os.environ.get("EMILY_PG_DB", "emily"),
                    pg_user=os.environ.get("EMILY_PG_USER", "emily"),
                    pg_password=os.environ.get("EMILY_PG_PASSWORD",
"emily_secret_2026"))
async def run_cognition_cycle(project_id: str = "", *, db_url: str = "", dry_run:
bool = False) -> dict:
    _init_db(db_url)
    from emily_core.services.world_book_service import ProjectWorldBookService
    service = ProjectWorldBookService()
    if project_id:
        return await service.update_stale(project_id, dry_run=dry_run)
    else:
        results = await service.update_all(dry_run=dry_run)
        return {"total": len(results), "results": results}
def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',
errors='replace')

```

```

parser = argparse.ArgumentParser(description="认知进化周期执行")
parser.add_argument("--project-id", help="项目 ID")
parser.add_argument("--all", action="store_true", help="所有项目")
parser.add_argument("--db-url", default="")
parser.add_argument("--dry-run", action="store_true")
args = parser.parse_args()
result = asyncio.run(run_cognition_cycle(
    args.project_id or "", db_url=args.db_url, dry_run=args.dry_run,
))
output = {k: v for k, v in result.items() if k not in ("content_json",
"drift_details")}
if "results" in output:
    for r in output["results"]:
        r.pop("content_json", None)
        r.pop("drift_details", None)
    print(json.dumps(output, ensure_ascii=False, indent=2, default=str))
if __name__ == "__main__":
    main()

```

文件编号: 189 文件路径: scripts\cold_start.py

```

from __future__ import annotations
import argparse
import asyncio
import io
import json
import logging
import os
import subprocess
import sys
from datetime import datetime, timezone, timedelta
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(
name)s: %(message)s")
logger = logging.getLogger("cold_start")
BEIJING_TZ = timezone(timedelta(hours=8))
def _init_db(db_url: str = "") -> None:
    from emily_core.infrastructure.database.session import init_db
    if db_url:
        init_db(db_url=db_url)
    else:
        db_url_env = os.environ.get("EMILY_DATABASE_URL", "")
        if db_url_env:
            init_db(db_url=db_url_env)
        else:
            pg_port = int(os.environ.get("EMILY_PG_PORT", "")) if
os.environ.get("EMILY_PG_PORT") else None
            if not pg_port:

```

```

        try:
            r = subprocess.run(["docker", "port", "emily-postgres",
                                "5432/tcp"], capture_output=True, text=True, timeout=5)
            pg_port = int(r.stdout.strip().rsplit(":", 1)[-1]) if
r.returncode == 0 and r.stdout.strip() else 5432
        except Exception:
            pg_port = 5432
        init_db(pg_host=os.environ.get("EMILY_PG_HOST", "127.0.0.1"),
pg_port=pg_port,
                pg_db=os.environ.get("EMILY_PG_DB", "emily"),
                pg_user=os.environ.get("EMILY_PG_USER", "emily"),
                pg_password=os.environ.get("EMILY_PG_PASSWORD",
"emily_secret_2026"))
    async def run_cold_start(*, db_url: str = "", dry_run: bool = False) -> dict:
        _init_db(db_url)
        from self_check import self_check
        check_result = self_check(db_url=db_url, dry_run=dry_run)
        print(f"[1/4] 系统自检完成: {check_result.get('projects', {}).get('active', 0)}
个活跃项目")
        from emily_core.infrastructure.database.models import Project
        from emily_core.infrastructure.database.session import get_session
        from emily_core.services.initialization_checker import InitializationChecker
        checker = InitializationChecker()
        init_results = []
        with get_session() as session:
            projects = session.query(Project).filter(Project.is_deleted == False,
Project.status == "active").all()
            for p in projects:
                init_result = checker.check(p.id)
                init_results.append({
                    "project_id": p.id,
                    "project_name": p.name,
                    **init_result,
                })
            print(f"[2/4] 初始化检查完成: {len(init_results)} 个项目")
            from emily_core.services.world_book_builder import ProjectWorldBookBuilder
            from emily_core.repositories.world_book_repo import ProjectWorldBookRepo
            builder = ProjectWorldBookBuilder()
            build_results = []
            for p in projects:
                existing = ProjectWorldBookRepo.get_by_project(p.id)
                if existing is None:
                    build_result = builder.build(p.id, generated_by="startup",
dry_run=dry_run)
                    build_results.append({
                        "project_id": p.id,
                        "project_name": p.name,
                        "action": "created",
                        **build_result,
                    })
                print(f" 世界书构建: {p.name} ->
tier=T{build_result.get('initialization_tier', 0)}")
            print(f"[3/4] 世界书构建完成: {len(build_results)} 个新建")
            email_sent = 0

```



```

email_failed = 0
if not dry_run:
    from emily_core.infrastructure.database.models import User
    for init_r in init_results:
        try:
            with get_session() as session:
                admins = session.query(User).filter(
                    User.project_id == init_r["project_id"],
                    User.is_deleted == False,
                    User.is_admin == True,
                ).all()
            for admin in admins:
                if admin.email:
                    logger.info("Cold start notification: project=%s admin=%s
email=%s tier=T%d",
                                init_r["project_name"], admin.username,
                                admin.email, init_r["tier"])
                    email_sent += 1
            except Exception as e:
                logger.warning("Email notification failed for project %s: %s",
init_r["project_id"], e)
                email_failed += 1
    print(f"[4/4] 邮件通知: {email_sent} 已发送, {email_failed} 失败")
    return {
        "self_check": check_result,
        "initialization": init_results,
        "world_books_built": len(build_results),
        "email_sent": email_sent,
        "email_failed": email_failed,
    }
def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',
errors='replace')
    parser = argparse.ArgumentParser(description="Emily 冷启动流程")
    parser.add_argument("--db-url", default="")
    parser.add_argument("--dry-run", action="store_true")
    args = parser.parse_args()
    result = asyncio.run(run_cold_start(db_url=args.db_url, dry_run=args.dry_run))
    print(json.dumps(
        {k: v for k, v in result.items() if k != "self_check"},
        ensure_ascii=False, indent=2, default=str,
    ))
if __name__ == "__main__":
    main()

```

文件编号: 190 文件路径: scripts\collect_session_data.py

```

from __future__ import annotations
import logging
import os

```

```

import subprocess
import sys
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
logger = logging.getLogger("emily.collect_session_data")
_SENTINEL = "XXXXXXXXXX"
def _detect_docker_pg_port() -> int | None:
    try:
        result = subprocess.run(
            ["docker", "port", "emily-postgres", "5432/tcp"],
            capture_output=True, text=True, timeout=5,
        )
        if result.returncode == 0 and result.stdout.strip():
            port_str = result.stdout.strip().rsplit(":", 1)[-1]
            return int(port_str)
    except (FileNotFoundError, subprocess.TimeoutExpired, ValueError, OSError):
        pass
    return None
def _init_db_if_needed() -> None:
    from emily_core.infrastructure.database import init_db, get_db_path
    db_url = os.environ.get("EMILY_DATABASE_URL", "")
    if db_url:
        init_db(db_url=db_url)
    else:
        pg_host = os.environ.get("EMILY_PG_HOST", os.environ.get("PG_HOST",
"127.0.0.1"))
        pg_port_env = os.environ.get("EMILY_PG_PORT", os.environ.get("PG_PORT"))
        if pg_port_env:
            pg_port = int(pg_port_env)
        else:
            pg_port = _detect_docker_pg_port() or 5432
        pg_db = os.environ.get("EMILY_PG_DB", "emily")
        pg_user = os.environ.get("EMILY_PG_USER", "emily")
        pg_password = os.environ.get("EMILY_PG_PASSWORD", "emily_secret_2026")
        init_db(pg_host=pg_host, pg_port=pg_port, pg_db=pg_db, pg_user=pg_user,
pg_password=pg_password)
        logger.debug("DB connected: %s", get_db_path())
def collect_session_data(
    user_id: str,
    conversation_id: str = "",
) -> dict:
    if not user_id:
        raise ValueError("user_id 不能为空")
    _init_db_if_needed()
    from emily_core.session.session_data_fetcher import SessionDataFetcher
    return SessionDataFetcher.fetch(user_id, conversation_id, core=None)
if __name__ == "__main__":
    import io
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',

```

```

errors='replace')
hot_update = "--hot-update" in sys.argv
args = [a for a in sys.argv[1:] if a != "--hot-update"]
test_user = args[0] if args else "80137af6-78e0-41fd-9795-435e0b9eaeab"
test_conv = args[1] if len(args) > 1 else ""
print(f"=== collect_session_data({test_user!r}) ===")
data = collect_session_data(user_id=test_user, conversation_id=test_conv)
print("\n--- ERRORS (获取过程中异常) ---")
if data["errors"]:
    for err in data["errors"]:
        print(f"  X {err}")
else:
    print("  (无异常)")
_HOT_CLASS = {
    "conversation_id": "🔒", "user_id": "🔒", "user_name": "🔒",
    "user_position": "🔒", "created_at": "🔒",
    "permission_level": "🔥", "company_id": "🔥", "company_type": "🔥",
    "company_name": "🔥", "department": "🔥", "project_ids": "🔥",
    "partner_ids": "🔥", "scopes": "🔥", "sop_allow": "🔥",
    "db_perms": "🔥", "info_level": "🔥", "supervisor_id": "🔥",
    "granted_codes": "🔥", "denied_codes": "🔥",
    "authorized_node_ids": "🔥", "permission_version": "🔥",
    "permissions_loaded_at": "🔥",
    "project_name": "📁", "project_type": "📁", "project_status": "📁",
    "long_term_memory": "📄", "conversation_summary": "📄",
}
_SNAPSHOT_GROUPS = [
    ("🔒 标识字段", ["conversation_id", "user_id", "user_name",
"user_position", "created_at"]),
    ("🔥 权限字段", [
        "permission_level", "company_id", "company_type", "company_name",
        "department", "project_ids", "partner_ids", "scopes",
        "sop_allow", "db_perms", "info_level", "supervisor_id",
        "granted_codes", "denied_codes", "authorized_node_ids",
        "permission_version", "permissions_loaded_at",
    ]),
    ("📁 项目字段", ["project_name", "project_type", "project_status"]),
    ("📄 记忆字段", ["long_term_memory", "conversation_summary"]),
]
snapshot = data["session_snapshot"]
print("\n--- SessionSnapshot (扁平化) ---")
for group_label, group_keys in _SNAPSHOT_GROUPS:
    tag = group_label[:1]
    if hot_update:
        print(f"\n {group_label}")
    for k in group_keys:
        v = snapshot.get(k, "(缺失)")
        flag = " <= ERROR" if (isinstance(v, str) and v == _SENTINEL) else ""
        hot_tag = f" {tag}" if hot_update else ""
        val_str = str(v)[:150] + "..." if len(str(v)) > 150 else str(v)
        if k == "conversation_id" and v == "":
            val_str = "(未指定, recent_turns 为跨会话全局查询)"
        print(f"  {k:<25s} = {val_str}{flag}{hot_tag}")
print("\n--- SessionRuntime (DB采集) ---")

```

```

for k, v in data["session_runtime"].items():
    val_str = str(v)[:120] + "..." if len(str(v)) > 120 else str(v)
    print(f" {k:<25s} = {val_str}")
sentinel_count = sum(
    1 for v in snapshot.values()
    if (isinstance(v, str) and v == _SENTINEL)
)
if data["errors"]:
    print(f"\n共 {len(data['errors'])} 个异常（含 {sentinel_count} 个哨兵值），需排查。")
else:
    print(f"\n全部获取成功（哨兵值：{sentinel_count}）。")
print("Done.")

```

文件编号：191 文件路径：scripts\detect_cognition_drift.py

```

from __future__ import annotations
import argparse
import io
import json
import logging
import os
import subprocess
import sys
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(name)s: %(message)s")
logger = logging.getLogger("detect_cognition_drift")
def _init_db(db_url: str = "") -> None:
    from emily_core.infrastructure.database.session import init_db
    if db_url:
        init_db(db_url=db_url)
    else:
        db_url_env = os.environ.get("EMILY_DATABASE_URL", "")
        if db_url_env:
            init_db(db_url=db_url_env)
        else:
            pg_port = int(os.environ.get("EMILY_PG_PORT", "")) if
os.environ.get("EMILY_PG_PORT") else None
            if not pg_port:
                try:
                    r = subprocess.run(["docker", "port", "emily-postgres",
"5432/tcp"], capture_output=True, text=True, timeout=5)
                    pg_port = int(r.stdout.strip().rsplit(":", 1)[-1]) if
r.returncode == 0 and r.stdout.strip() else 5432
                except Exception:
                    pg_port = 5432
            init_db(pg_host=os.environ.get("EMILY_PG_HOST", "127.0.0.1"),

```

```

pg_port=pg_port,
        pg_db=os.environ.get("EMILY_PG_DB", "emily"),
        pg_user=os.environ.get("EMILY_PG_USER", "emily"),
        pg_password=os.environ.get("EMILY_PG_PASSWORD",
"emily_secret_2026"))
def detect_cognition_drift(project_id: str, *, db_url: str = "") -> dict:
    _init_db(db_url)
    from emily_core.services.cognition_drift_detector import
CognitionDriftDetector
    detector = CognitionDriftDetector()
    return detector.detect(project_id)
def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',
errors='replace')
    parser = argparse.ArgumentParser(description="检测认知偏差")
    parser.add_argument("--project-id", help="项目 ID (UUID)")
    parser.add_argument("--all", action="store_true", help="检测所有项目")
    parser.add_argument("--db-url", default="", help="PostgreSQL 连接 URL")
    parser.add_argument("--dry-run", action="store_true", help="预览模式")
    args = parser.parse_args()
    if not args.project_id and not args.all:
        parser.error("请指定 --project-id 或 --all")
    if args.all:
        _init_db(args.db_url)
        from emily_core.infrastructure.database.models import Project
        from emily_core.infrastructure.database.session import get_session
        with get_session() as session:
            projects = session.query(Project).filter(Project.is_deleted == False,
Project.status == "active").all()
            print(f"检测 {len(projects)} 个 active 项目")
            for p in projects:
                result = detect_cognition_drift(p.id, db_url=args.db_url)
                print(f"\n--- {p.name} ({p.id}) ---")
                if not result.get("has_world_book"):
                    print(" [INFO] 项目无世界书, 需首次生成")
                elif result.get("has_drift"):
                    print(f" [DRIFT] 过时层: {result.get('stale_layers', [])}")
                    for layer_name, layer_data in result.get("drift", {}).items():
                        if layer_data.get("stale"):
                            print(f"    {layer_name}: {layer_data.get('signals',
[])}")
                else:
                    print(" [OK] 无偏差")
            else:
                result = detect_cognition_drift(args.project_id, db_url=args.db_url)
                print(json.dumps(result, ensure_ascii=False, indent=2, default=str))
if __name__ == "__main__":
    main()

```

```

from __future__ import annotations
import argparse
import asyncio
import io
import sys
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
if str(_HERE) not in sys.path:
    sys.path.insert(0, str(_HERE))
from evolution_metrics import collect_metrics
from evolution_anomaly import detect_anomalies
async def run_daily_pipeline(date: str, *, days: int = 1, dry_run: bool = False) -
> dict:
    print(f"=== 洞察流水线 - {date} ({days}天) ===")
    metrics = await collect_metrics(date, days=days)
    anomalies = detect_anomalies(metrics, days=days)
    print(f"  指标聚合完成: {len(metrics)} 个数据源, {len(anomalies)} 条异常")
    try:
        from evolution_insight import generate_insight
        insight = await generate_insight(date, days=days, dry_run=dry_run)
        print(f"  日洞察生成: status={insight.get('status')}")
    except Exception as e:
        insight = {"status": "error", "error": str(e)}
        print(f"  日洞察生成失败: {e}")
    return {"metrics_summary": f"{len(metrics)} sources", "anomalies": anomalies,
"insight": insight}
async def run_weekly_pipeline(end_date: str, *, days: int = 7, dry_run: bool =
False) -> dict:
    print(f"=== 每周流水线 - {end_date} (近 {days} 天) ===")
    try:
        from evolution_rules import induct_rules
        rules = await induct_rules(end_date, days=days, dry_run=dry_run)
        print(f"  规则归纳: {len(rules)} 条规则")
    except Exception as e:
        rules = [{"status": "error", "error": str(e)}]
        print(f"  规则归纳失败: {e}")
    try:
        from evolution_patch import generate_patches
        patches = await generate_patches(dry_run=dry_run)
        print(f"  补丁生成: {len(patches)} 个补丁")
    except Exception as e:
        patches = [{"status": "error", "error": str(e)}]
        print(f"  补丁生成失败: {e}")
    return {"rules": rules, "patches": patches}
async def run_morning(date: str, *, dry_run: bool = False) -> dict:
    print(f"=== 晨报流水线 - {date} ===")
    try:
        from evolution_morning import generate_morning_report
        reports = await generate_morning_report(date, dry_run=dry_run)

```

```

        return {"reports": reports}
    except Exception as e:
        print(f" 晨报生成失败: {e}")
        return {"reports": [], "error": str(e)}
async def run_validation(*, dry_run: bool = False) -> dict:
    print(f"=== 补丁验证流水线 ===")
    try:
        from evolution_validate import validate_patches
        results = await validate_patches(dry_run=dry_run)
        return {"validation_results": results}
    except Exception as e:
        print(f" 验证失败: {e}")
        return {"validation_results": [], "error": str(e)}
async def run_full_pipeline(date: str, *, dry_run: bool = False) -> dict:
    daily = await run_daily_pipeline(date, dry_run=dry_run)
    morning = await run_morning(date, dry_run=dry_run)
    return {"daily": daily, "morning": morning}
def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
    errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',
    errors='replace')
    parser = argparse.ArgumentParser(description="进化闭环薄聚合脚本")
    sub = parser.add_subparsers(dest="command", required=True)
    p = sub.add_parser("daily", help="洞察流水线: metrics -> anomaly -> insight")
    p.add_argument("--date", "-d", required=True)
    p.add_argument("--days", type=int, default=1, help="复盘天数（默认1）")
    p.add_argument("--dry-run", action="store_true")
    p = sub.add_parser("weekly", help="每周流水线: rules -> patches")
    p.add_argument("--end-date", "-d", required=True)
    p.add_argument("--days", type=int, default=7)
    p.add_argument("--dry-run", action="store_true")
    p = sub.add_parser("morning", help="晨报")
    p.add_argument("--date", "-d", required=True)
    p.add_argument("--dry-run", action="store_true")
    p = sub.add_parser("validate", help="补丁验证")
    p.add_argument("--dry-run", action="store_true")
    p = sub.add_parser("full", help="全流程: daily + morning")
    p.add_argument("--date", "-d", required=True)
    p.add_argument("--dry-run", action="store_true")
    args = parser.parse_args()
    dry_run = getattr(args, 'dry_run', False)
    if args.command == "daily":
        asyncio.run(run_daily_pipeline(args.date, days=args.days,
    dry_run=dry_run))
    elif args.command == "weekly":
        asyncio.run(run_weekly_pipeline(args.end_date, days=args.days,
    dry_run=dry_run))
    elif args.command == "morning":
        asyncio.run(run_morning(args.date, dry_run=dry_run))
    elif args.command == "validate":
        asyncio.run(run_validation(dry_run=dry_run))
    elif args.command == "full":
        asyncio.run(run_full_pipeline(args.date, dry_run=dry_run))

```

```
if __name__ == "__main__":  
    main()
```

文件编号: 193 文件路径: scripts\evolution_anomaly.py

```
from __future__ import annotations  
import argparse  
import asyncio  
import io  
import json  
import logging  
import sys  
from pathlib import Path  
_HERE = Path(__file__).resolve().parent  
_CORE_DIR = _HERE.parent / "emily-core"  
if str(_CORE_DIR) not in sys.path:  
    sys.path.insert(0, str(_CORE_DIR))  
logging.basicConfig(  
    level=logging.INFO,  
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",  
    datefmt="%Y-%m-%d %H:%M:%S",  
)  
logger = logging.getLogger("evolution_anomaly")  
THRESHOLDS = {  
    "high_fallback": 0.30,  
    "sop_correction_spike": 3,  
    "low_rag_hit": 0.50,  
    "slow_pipeline": 10000,  
    "volume_anomaly": 0.50,  
    "tool_failure": 0.20,  
    "iteration_overflow": 3,  
    "node_overdue": 1,  
}  
  
def detect_anomalies(metrics: dict, *, days: int = 1) -> list[dict]:  
    anomalies = []  
    p = metrics.get("pipeline", {})  
    fb = metrics.get("feedback", {})  
    rag = metrics.get("rag", {})  
    tc = metrics.get("tool_calls", {})  
    ar = metrics.get("agent_reasoning", {})  
    pn = metrics.get("project_nodes", {})  
    fallback_rate = p.get("fallback_rate", 0.0)  
    if fallback_rate > THRESHOLDS["high_fallback"]:  
        anomalies.append({  
            "flag": "high_fallback",  
            "severity": "high",  
            "message": f"Fallback 率 {fallback_rate:.1%} 超过阈值  
{THRESHOLDS['high_fallback']:.0%}",  
            "detail": {"fallback_rate": fallback_rate, "threshold":  
THRESHOLDS["high_fallback"]},  
        })  
    correction_count = 0
```



```

for t in fb.get("type_distribution", []):
    if t["type"] == "explicit_correction":
        correction_count = t["count"]
        break
correction_threshold = THRESHOLDS["sop_correction_spike"] * days
if correction_count > correction_threshold:
    anomalies.append({
        "flag": "sop_correction_spike",
        "severity": "high",
        "message": f"纠正信号 {correction_count} 次超过阈值
{correction_threshold}",
        "detail": {"correction_count": correction_count},
    })
zero_hit_rate = rag.get("zero_hit_rate", 0.0)
if zero_hit_rate > THRESHOLDS["low_rag_hit"]:
    anomalies.append({
        "flag": "low_rag_hit",
        "severity": "medium",
        "message": f"RAG 零命中率 {zero_hit_rate:.1%} 超过阈值
{THRESHOLDS['low_rag_hit']:.0%}",
        "detail": {"zero_hit_rate": zero_hit_rate},
    })
avg_elapsed = p.get("avg_elapsed_ms", 0)
if avg_elapsed > THRESHOLDS["slow_pipeline"]:
    anomalies.append({
        "flag": "slow_pipeline",
        "severity": "medium",
        "message": f"Pipeline 平均耗时 {avg_elapsed}ms 超过阈值
{THRESHOLDS['slow_pipeline']}ms",
        "detail": {"avg_elapsed_ms": avg_elapsed},
    })
tool_dist = tc.get("tool_distribution", {})
failure_details = tc.get("failure_details", [])
failure_by_tool = {}
for f in failure_details:
    failure_by_tool[f["tool"]] = failure_by_tool.get(f["tool"], 0) +
f["count"]
for tool_name, fail_count in failure_by_tool.items():
    total_calls = tool_dist.get(tool_name, 0)
    if total_calls > 0:
        fail_rate = fail_count / total_calls
        if fail_rate > THRESHOLDS["tool_failure"]:
            anomalies.append({
                "flag": "tool_failure",
                "severity": "high",
                "message": f"工具 {tool_name} 失败率 {fail_rate:.1%} 超过阈值
{THRESHOLDS['tool_failure']:.0%}",
                "detail": {"tool": tool_name, "fail_rate": fail_rate,
"fail_count": fail_count, "total": total_calls},
            })
max_iter = ar.get("max_iterations_reached", 0)
iter_threshold = THRESHOLDS["iteration_overflow"] * days
if max_iter > iter_threshold:
    anomalies.append({

```

```

        "flag": "iteration_overflow",
        "severity": "medium",
        "message": f"最大迭代触达 {max_iter} 次超过阈值 {iter_threshold}",
        "detail": {"max_iterations_reached": max_iter},
    })
    overdue = pn.get("overdue_nodes", [])
    if len(overdue) >= THRESHOLDS["node_overdue"]:
        anomalies.append({
            "flag": "node_overdue",
            "severity": "high",
            "message": f"存在 {len(overdue)} 个逾期节点",
            "detail": {"overdue_count": len(overdue), "nodes": [{"node_id":
n["node_id"], "name": n["name"]} for n in overdue[:5]]},
        })
    return anomalies

def _print_anomalies(anomalies: list[dict], date_label: str) -> None:
    severity_icon = {"high": "HIGH", "medium": "MED"}
    print(f"\n{'=' * 60}")
    print(f"  异常检测报告 - {date_label}")
    print(f"{'=' * 60}")
    if anomalies:
        for a in anomalies:
            icon = severity_icon.get(a["severity"], "WARN")
            print(f"  [{icon}] {a['flag']}: {a['message']}")
    else:
        print("  [OK] 全部 8 条规则未触发异常")
    high_count = sum(1 for a in anomalies if a["severity"] == "high")
    medium_count = sum(1 for a in anomalies if a["severity"] == "medium")
    print(f"\n异常数: {len(anomalies)} (HIGH {high_count} / MED {medium_count})")
    print(f"{'=' * 60}")

def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',
errors='replace')
    parser = argparse.ArgumentParser(description="硬规则异常检测脚本")
    parser.add_argument("--date", "-d", default="", help="复盘结束日期 YYYY-MM-DD")
    parser.add_argument("--days", type=int, default=1, help="复盘天数（默认1，最小
1）")
    parser.add_argument("--metrics-file", "-f", default="", help="从 JSON 文件加载
metrics")
    parser.add_argument("--json", action="store_true", help="输出原始 JSON")
    args = parser.parse_args()
    if args.metrics_file:
        with open(args.metrics_file, "r", encoding="utf-8") as f:
            metrics = json.load(f)
            date_label = metrics.get("end_date", args.metrics_file)
    elif args.date:
        from evolution_metrics import collect_metrics
        metrics = asyncio.run(collect_metrics(args.date, days=args.days))
        date_label = args.date
    else:
        print("错误: 必须指定 --date 或 --metrics-file")
        sys.exit(1)

```

```

    anomalies = detect_anomalies(metrics, days=args.days)
    if args.json:
        print(json.dumps(anomalies, ensure_ascii=False, indent=2, default=str))
    else:
        _print_anomalies(anomalies, date_label)
if __name__ == "__main__":
    main()

```

文件编号: 194 文件路径: scripts\evolution_apply.py

```

from __future__ import annotations
import argparse
import asyncio
import io
import json
import sys
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
if str(_HERE) not in sys.path:
    sys.path.insert(0, str(_HERE))
from evolution_metrics import _init_db
async def approve_patch(patch_no: str) -> dict:
    from emily_core.services.evolution.patch_applier import PatchApplier
    applier = PatchApplier()
    return await applier.approve(patch_no)
async def apply_patch(patch_no: str, *, dry_run: bool = False) -> dict:
    from emily_core.services.evolution.patch_applier import PatchApplier
    applier = PatchApplier()
    return await applier.apply(patch_no, dry_run=dry_run)
async def rollback_patch(patch_no: str, *, dry_run: bool = False) -> dict:
    from emily_core.services.evolution.patch_applier import PatchApplier
    applier = PatchApplier()
    return await applier.rollback(patch_no, dry_run=dry_run)
async def reject_patch(patch_no: str, reason: str = "") -> dict:
    from emily_core.services.evolution.patch_applier import PatchApplier
    applier = PatchApplier()
    return await applier.reject(patch_no, reason)
async def list_patches(status: str = "DRAFT") -> list[dict]:
    from emily_core.repositories.evolution_repo import EvolutionRepo
    patches = EvolutionRepo.get_patches_by_status(status)
    return [
        {
            "patch_no": p.patch_no,
            "rule_no": p.rule_no,
            "target_path": p.target_path,
            "patch_type": p.patch_type,
            "risk_level": p.risk_level,
            "status": p.status,
            "created_at": p.created_at,

```

```

    }
    for p in patches
]
def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',
errors='replace')
    parser = argparse.ArgumentParser(description="补丁审批/应用/回滚脚本")
    sub = parser.add_subparsers(dest="command", required=True)
    p = sub.add_parser("list", help="列出补丁")
    p.add_argument("--status", default="DRAFT", help="状态过滤")
    p = sub.add_parser("approve", help="审批补丁")
    p.add_argument("--patch-no", required=True)
    p = sub.add_parser("reject", help="拒绝补丁")
    p.add_argument("--patch-no", required=True)
    p.add_argument("--reason", default="")
    p = sub.add_parser("apply", help="应用补丁")
    p.add_argument("--patch-no", required=True)
    p.add_argument("--dry-run", action="store_true")
    p = sub.add_parser("rollback", help="回滚补丁")
    p.add_argument("--patch-no", required=True)
    p.add_argument("--dry-run", action="store_true")
    p = sub.add_parser("status", help="查看补丁详情")
    p.add_argument("--patch-no", required=True)
    args = parser.parse_args()
    _init_db()
    if args.command == "list":
        patches = asyncio.run(list_patches(args.status))
        print(f"\n补丁列表 (status={args.status}): {len(patches)} 条")
        for p in patches:
            print(f"    {p['patch_no']}: {p['rule_no']} -> {p['target_path']}
[{p.get('risk_level', '?')}]")
    elif args.command == "approve":
        result = asyncio.run(approve_patch(args.patch_no))
        print(f"\n审批结果: {result}")
    elif args.command == "reject":
        result = asyncio.run(reject_patch(args.patch_no, args.reason))
        print(f"\n拒绝结果: {result}")
    elif args.command == "apply":
        result = asyncio.run(apply_patch(args.patch_no, dry_run=args.dry_run))
        if args.dry_run:
            print(f"\n预览应用:\n{json.dumps(result, ensure_ascii=False, indent=2,
default=str)}")
        else:
            print(f"\n应用结果: {result.get('status')}")
    elif args.command == "rollback":
        result = asyncio.run(rollback_patch(args.patch_no, dry_run=args.dry_run))
        print(f"\n回滚结果: {result.get('status')}")
    elif args.command == "status":
        from emily_core.repositories.evolution_repo import EvolutionRepo
        p = EvolutionRepo.get_patch_by_no(args.patch_no)
        if p:
            print(f"\n补丁 {p.patch_no}:")

```

```

        print(f" 规则: {p.rule_no}")
        print(f" 目标: {p.target_path}")
        print(f" 类型: {p.patch_type}")
        print(f" 风险: {p.risk_level}")
        print(f" 状态: {p.status}")
        print(f" 理由: {p.risk_reasoning}")
        print(f" 锚点: {p.search_anchor}")
        print(f" 内容: {p.patch_content[:200]}")
    else:
        print(f"补丁 {args.patch_no} 不存在")
if __name__ == "__main__":
    main()

```

文件编号: 195 文件路径: scripts\evolution_insight.py

```

from __future__ import annotations
import argparse
import asyncio
import io
import json
import logging
import os
import sys
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
if str(_HERE) not in sys.path:
    sys.path.insert(0, str(_HERE))
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger("evolution_insight")
async def generate_insight(end_date: str, *, days: int = 1, db_url: str = "",
dry_run: bool = False) -> dict:
    from emily_core.infrastructure.database.session import get_session
    from emily_core.services.evolution.insight_generator import InsightGenerator
    from evolution_metrics import _init_db
    _init_db(db_url)
    llm_client = None
    if not dry_run:
        try:
            from emily_core.infrastructure.llm.client import LLMClient
            api_key = os.environ.get("EMILY_LLM_API_KEY",
os.environ.get("OPENAI_API_KEY", ""))
            base_url = os.environ.get("EMILY_LLM_BASE_URL",
os.environ.get("OPENAI_BASE_URL", ""))
            model = os.environ.get("EMILY_LLM_MODEL",
os.environ.get("OPENAI_MODEL", "gpt-4o"))

```

```

        if api_key and base_url:
            llm_client = LLMClient(
                api_key=api_key,
                base_url=base_url,
                model=model,
            )
            logger.info("LLM client initialized: model=%s", model)
        else:
            logger.warning("LLM credentials not configured, running in preview
mode")

            dry_run = True
    except Exception as e:
        logger.warning("Failed to init LLM client: %s, running in preview
mode", e)

        dry_run = True
    generator = InsightGenerator(llm_client=llm_client)
    result = await generator.generate(end_date, days=days, dry_run=dry_run)
    return result

async def show_insight(date: str) -> dict | None:
    from emily_core.repositories.evolution_repo import EvolutionRepo
    from evolution_metrics import _init_db
    _init_db()
    insight = EvolutionRepo.get_insight_by_date(date)
    if insight is None:
        print(f"未找到日期 {date} 的洞察记录")
        return None
    return {
        "insight_date": insight.insight_date,
        "analysis_days": insight.analysis_days,
        "sop_hit_rate": insight.sop_hit_rate,
        "fallback_rate": insight.fallback_rate,
        "health_score": insight.health_score,
        "anomaly_flags": json.loads(insight.anomaly_flags) if
insight.anomaly_flags else [],
        "insight": json.loads(insight.insight_text) if insight.insight_text else
{},
    }

def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',
errors='replace')
    parser = argparse.ArgumentParser(description="日洞察生成脚本（支持可变周期）")
    parser.add_argument("--date", "-d", required=True, help="复盘结束日期 YYYY-MM-
DD")
    parser.add_argument("--days", type=int, default=1, help="复盘天数（默认1，最小
1）")
    parser.add_argument("--db-url", default="", help="PostgreSQL 连接 URL")
    parser.add_argument("--preview", action="store_true", help="预览模式（不调
LLM、不写 DB）")
    parser.add_argument("--show", action="store_true", help="仅显示已有洞察（不生
成）")
    parser.add_argument("--json", action="store_true", help="输出原始 JSON")
    args = parser.parse_args()

```

```

if args.show:
    result = asyncio.run(show_insight(args.date))
    if result:
        print(json.dumps(result, ensure_ascii=False, indent=2, default=str))
    return
result = asyncio.run(generate_insight(
    args.date,
    days=args.days,
    db_url=args.db_url,
    dry_run=args.preview,
))
if args.json:
    output = {k: v for k, v in result.items() if k != "metrics"}
    print(json.dumps(output, ensure_ascii=False, indent=2, default=str))
else:
    status = result.get("status", "unknown")
    print(f"\n洞察生成完成: status={status}")
    if status == "preview":
        print("预览模式: metrics 聚合已完成, LLM 未调用")
    elif status == "generated":
        insight = result.get("insight", {})
        if insight:
            print(f"健康评分: {insight.get('health_score', 'N/A')}/100")
            print(f"摘要: {insight.get('summary', 'N/A')}")
            findings = insight.get("key_findings", [])
            if findings:
                print(f"关键发现: {len(findings)} 条")
                for f in findings[:5]:
                    print(f" - [{f.get('category', '')}] {f.get('finding',
''}})")
if __name__ == "__main__":
    main()

```

文件编号: 196 文件路径: scripts\evolution_metrics.py

```

from __future__ import annotations
import argparse
import asyncio
import io
import json
import logging
import os
import subprocess
import sys
from datetime import datetime, timedelta
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
logging.basicConfig(
    level=logging.INFO,

```

```

        format="%(%asctime)s [%(levelname)s] %(name)s: %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S",
    )
    logger = logging.getLogger("evolution_metrics")
def _detect_docker_pg_port() -> int | None:
    try:
        result = subprocess.run(
            ["docker", "port", "emily-postgres", "5432/tcp"],
            capture_output=True, text=True, timeout=5,
        )
        if result.returncode == 0 and result.stdout.strip():
            port_str = result.stdout.strip().rsplit(":", 1)[-1]
            return int(port_str)
    except (FileNotFoundError, subprocess.TimeoutExpired, ValueError, OSError):
        pass
    return None
def _init_db(db_url: str = "") -> None:
    from emily_core.infrastructure.database.session import init_db
    if db_url:
        init_db(db_url=db_url)
    else:
        db_url_env = os.environ.get("EMILY_DATABASE_URL", "")
        if db_url_env:
            init_db(db_url=db_url_env)
        else:
            pg_host = os.environ.get("EMILY_PG_HOST", "127.0.0.1")
            pg_port_env = os.environ.get("EMILY_PG_PORT")
            if pg_port_env:
                pg_port = int(pg_port_env)
            else:
                pg_port = _detect_docker_pg_port() or 5432
            pg_db = os.environ.get("EMILY_PG_DB", "emily")
            pg_user = os.environ.get("EMILY_PG_USER", "emily")
            pg_password = os.environ.get("EMILY_PG_PASSWORD", "emily_secret_2026")
            init_db(pg_host=pg_host, pg_port=pg_port, pg_db=pg_db,
pg_user=pg_user, pg_password=pg_password)
async def collect_metrics(end_date: str, *, days: int = 1, db_url: str = "") -> dict:
    _init_db(db_url)
    from emily_core.repositories.evolution_repo import EvolutionRepo
    from emily_core.infrastructure.database.session import get_session
    start_dt = datetime.fromisoformat(end_date) - timedelta(days=days - 1)
    start_date = start_dt.strftime("%Y-%m-%d")
    with get_session() as sess:
        metrics = {
            "start_date": start_date,
            "end_date": end_date,
            "analysis_days": days,
            "pipeline": EvolutionRepo.aggregate_pipeline_logs(start_date,
end_date, session=sess),
            "sop_routing": EvolutionRepo.aggregate_sop_routing_logs(start_date,
end_date, session=sess),
            "feedback": EvolutionRepo.aggregate_feedback_signals(start_date,
end_date, session=sess),

```



```

        "rag": EvolutionRepo.aggregate_rag_logs(start_date, end_date,
        session=sess),
        "business_events": EvolutionRepo.aggregate_business_events(start_date,
        end_date, session=sess),
        "session_lifecycle":
        EvolutionRepo.aggregate_session_lifecycle(start_date, end_date, session=sess),
        "agent_reasoning": EvolutionRepo.aggregate_agent_reasoning(start_date,
        end_date, session=sess),
        "tool_calls": EvolutionRepo.aggregate_tool_calls(start_date, end_date,
        session=sess),
        "project_nodes": EvolutionRepo.aggregate_project_nodes(end_date,
        session=sess),
        "cognition_drift": EvolutionRepo.aggregate_cognition_drift(end_date,
        session=sess),
    }
    return metrics

async def collect_single_source(source: str, end_date: str, *, days: int = 1,
db_url: str = "") -> dict:
    _init_db(db_url)
    from emily_core.repositories.evolution_repo import EvolutionRepo
    from emily_core.infrastructure.database.session import get_session
    start_dt = datetime.fromisoformat(end_date) - timedelta(days=days - 1)
    start_date = start_dt.strftime("%Y-%m-%d")
    source_map = {
        "pipeline": EvolutionRepo.aggregate_pipeline_logs,
        "sop_routing": EvolutionRepo.aggregate_sop_routing_logs,
        "feedback": EvolutionRepo.aggregate_feedback_signals,
        "rag": EvolutionRepo.aggregate_rag_logs,
        "business_events": EvolutionRepo.aggregate_business_events,
        "session_lifecycle": EvolutionRepo.aggregate_session_lifecycle,
        "agent_reasoning": EvolutionRepo.aggregate_agent_reasoning,
        "tool_calls": EvolutionRepo.aggregate_tool_calls,
        "project_nodes": EvolutionRepo.aggregate_project_nodes,
        "cognition_drift": EvolutionRepo.aggregate_cognition_drift,
    }
    if source not in source_map:
        raise ValueError(f"未知数据源: {source}, 可选: {list(source_map.keys())}")
    with get_session() as sess:
        if source in ("project_nodes", "cognition_drift"):
            return source_map[source](end_date, session=sess)
        return source_map[source](start_date, end_date, session=sess)

def _print_metrics(metrics: dict) -> None:
    end_date = metrics["end_date"]
    days = metrics.get("analysis_days", 1)
    period_label = f"{end_date} ({days}天)" if days > 1 else end_date
    print(f"\n{'=' * 60}")
    print(f"  指标聚合报告 - {period_label}")
    print(f"{'=' * 60}")
    p = metrics["pipeline"]
    print(f"\n[A] Pipeline 执行日志")
    print(f"  总执行: {p['total']} | SOP 命中: {p['hit']} ({p['sop_hit_rate']:.1%})
| Fallback: {p['fallback']} ({p['fallback_rate']:.1%})")
    if p["status_distribution"]:
        print(f"  状态: {' ', ' '.join(f'{k}={v}' for k, v in

```

```

p['status_distribution'].items()))")
    print(f" 平均耗时: {p['avg_elapsed_ms']}ms | 最大: {p['max_elapsed_ms']}ms |
被阻断: {p['blocked']}")
    sr = metrics["sop_routing"]
    print(f"\n[B] SOP 路由日志")
    print(f" 未命中: {sr['not_hit_count']} 次")
    if sr["miss_samples"]:
        print(f" 未命中样本:")
        for s in sr["miss_samples"][:5]:
            print(f"    - \"{s['message'][:50]}\" (confidence={s['confidence']})")
    fb = metrics["feedback"]
    print(f"\n[C] 用户反馈信号")
    if fb["type_distribution"]:
        for t in fb["type_distribution"]:
            print(f"  {t['type']}: {t['count']} 次 (avg_strength=
{t['avg_strength']:.2f})")
    rag = metrics["rag"]
    print(f"\n[D] RAG 检索日志")
    print(f" 总查询: {rag['total']} | 命中: {rag['hit']} | 零命中:
{rag['zero_hit_count']} ({rag['zero_hit_rate']:.1%})")
    print(f" 平均最高分: {rag['avg_top_score']:.4f} | 平均延迟:
{rag['avg_latency_ms']}ms")
    be = metrics["business_events"]
    print(f"\n[E] 业务事件日志")
    if be["category_distribution"]:
        print(f" 类别: {'', '.join(f'{k}={v}' for k, v in
be['category_distribution'].items())}")
    sl = metrics["session_lifecycle"]
    print(f"\n[F] Session 生命周期")
    print(f" 新建: {sl['sessions_created']} | 归档: {sl['sessions_archived']} | 平
均时长: {sl['avg_duration_ms']}ms")
    ar = metrics["agent_reasoning"]
    print(f"\n[G] Agent 推理日志")
    print(f"  Fallback: {ar['fallback_count']} | 最大迭代触达:
{ar['max_iterations_reached']} | 平均迭代: {ar['avg_iterations']}")
    tc = metrics["tool_calls"]
    print(f"\n[H] 工具调用日志")
    if tc["tool_distribution"]:
        top3 = list(tc["tool_distribution"].items())[:3]
        print(f"  Top 工具: {'', '.join(f'{k}={v}' for k, v in top3)}")
    pn = metrics["project_nodes"]
    print(f"\n[I] 项目节点")
    if pn["status_distribution"]:
        print(f" 状态: {'', '.join(f'{k}={v}' for k, v in
pn['status_distribution'].items())}")
    if pn["overdue_nodes"]:
        print(f" 逾期节点: {len(pn['overdue_nodes'])} 个")
        for n in pn["overdue_nodes"][:3]:
            print(f"    - {n['node_id']} {n['name']} (截止 {n['deadline']})")
    cd = metrics["cognition_drift"]
    print(f"\n[J] 认知偏差")
    print(f" 总项目: {cd['total_projects']} | 有世界书:
{cd['projects_with_world_book']} | 无世界书: {cd['projects_without_world_book']}")
    print(f" 世界书覆盖率: {cd['world_book_coverage']:.1%}")

```

```

    print(f" 有偏差项目: {cd['projects_with_drift']} ({cd['drift_rate']:.1%}) | 总
过时层数: {cd['total_stale_layers']}")
    if cd["stale_layer_distribution"]:
        layer_summary = ", ".join(f"{k}={v}" for k, v in
cd["stale_layer_distribution"].items() if v > 0)
        if layer_summary:
            print(f" 过时层分布: {layer_summary}")
    if cd.get("project_drifts"):
        drifted = [p for p in cd["project_drifts"] if p["has_drift"]]
        for p in drifted[:3]:
            print(f" - {p['project_name']} ({p['project_id'][:8]}...): {'',
'.join(p['stale_layers'])}")
        print(f"\n{'=' * 60}")
def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',
errors='replace')
    parser = argparse.ArgumentParser(description="日指标聚合脚本")
    parser.add_argument("--date", "-d", required=True, help="复盘结束日期 YYYY-MM-
DD")
    parser.add_argument("--days", type=int, default=1, help="复盘天数（默认1, 最小
1, 支持7/30等）")
    parser.add_argument("--db-url", default="", help="PostgreSQL 连接 URL")
    parser.add_argument("--source", "-s", default="", help="只查单个数据源
(pipeline/sop_routing/feedback/rag/business_events/session_lifecycle/agent_reasoni
ng/tool_calls/project_nodes/cognition_drift)")
    parser.add_argument("--preview", action="store_true", help="预览模式（只读不
写）")
    parser.add_argument("--json", action="store_true", help="输出原始 JSON")
    args = parser.parse_args()
    if args.source:
        result = asyncio.run(collect_single_source(args.source, args.date,
days=args.days, db_url=args.db_url))
        if args.json:
            print(json.dumps(result, ensure_ascii=False, indent=2, default=str))
        else:
            print(json.dumps(result, ensure_ascii=False, indent=2, default=str))
    else:
        metrics = asyncio.run(collect_metrics(args.date, days=args.days,
db_url=args.db_url))
        if args.json:
            print(json.dumps(metrics, ensure_ascii=False, indent=2, default=str))
        else:
            _print_metrics(metrics)
if __name__ == "__main__":
    main()

```

文件编号: 197 文件路径: scripts\evolution_morning.py

```

from __future__ import annotations
import argparse

```

```

import asyncio
import io
import json
import sys
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
if str(_HERE) not in sys.path:
    sys.path.insert(0, str(_HERE))
from evolution_metrics import _init_db
async def generate_morning_report(date: str, *, user_id: str = "", push: bool =
False, db_url: str = "", dry_run: bool = False) -> dict | list[dict]:
    from emily_core.services.evolution.morning_report_builder import
MorningReportBuilder
    from emily_core.repositories.evolution_repo import EvolutionRepo
    _init_db(db_url)
    builder = MorningReportBuilder()
    if user_id:
        user = EvolutionRepo.get_active_users()
        target_user = None
        for u in user:
            if u.id == user_id:
                target_user = u
                break
        if not target_user:
            return {"error": f"User {user_id} not found or not active"}
        report = await builder.build_for_user(target_user, date)
        result = [report]
    else:
        result = await builder.build_for_date(date)
    return result
def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',
errors='replace')
    parser = argparse.ArgumentParser(description="晨报生成脚本")
    parser.add_argument("--date", "-d", required=True, help="晨报日期 YYYY-MM-DD")
    parser.add_argument("--user-id", default="", help="指定用户 ID（为空则全量）")
    parser.add_argument("--db-url", default="", help="PostgreSQL 连接 URL")
    parser.add_argument("--push", action="store_true", help="生成并推送")
    parser.add_argument("--preview", action="store_true", help="预览模式")
    args = parser.parse_args()
    reports = asyncio.run(generate_morning_report(
        args.date,
        user_id=args.user_id,
        push=args.push,
        db_url=args.db_url,
        dry_run=args.preview,
    ))
    result_list = reports if isinstance(reports, list) else [reports]
    for r in result_list:

```

```

    if "error" in r:
        print(f"错误: {r['error']}")
    else:
        print(f"\n{'=' * 50}")
        print(f"晨报 - {r.get('user_name', '?')}")
        print(f"{'=' * 50}")
        print(r.get("report_text", ""))
        print()
if __name__ == "__main__":
    main()

```

文件编号: 198 文件路径: scripts\evolution_node_close.py

```

from __future__ import annotations
import argparse
import asyncio
import io
import json
import sys
from datetime import datetime
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
if str(_HERE) not in sys.path:
    sys.path.insert(0, str(_HERE))
from evolution_metrics import _init_db
async def generate_node_closure(node_id: str, *, db_url: str = "", dry_run: bool =
False) -> dict:
    _init_db(db_url)
    from emily_core.infrastructure.database.session import get_session
    from emily_core.infrastructure.database.models import ProjectNode,
NodeDeliverable, BusinessEventLog
    with get_session() as sess:
        node = sess.query(ProjectNode).filter(
            ProjectNode.node_id == node_id,
            ProjectNode.is_discarded == False,
        ).first()
        if node is None:
            return {"error": f"Node {node_id} not found"}
        deliverables = sess.query(NodeDeliverable).filter(
            NodeDeliverable.node_id == node_id,
        ).all()
        events = sess.query(BusinessEventLog).filter(
            BusinessEventLog.target_id == node_id,
        ).order_by(BusinessEventLog.created_at).all()
        created_at = node.created_at
        completed_at = node.completed_at or datetime.now().isoformat()
        duration_str = ""
        try:
            start_dt = datetime.fromisoformat(created_at.replace("Z", "+00:00"))

```

```

        end_dt = datetime.fromisoformat(completed_at.replace("Z", "+00:00"))
        duration = end_dt - start_dt
        days = duration.days
        hours = duration.seconds // 3600
        duration_str = f"{days} 天 {hours} 小时"
    except Exception:
        duration_str = "无法计算"
    report_lines = []
    report_lines.append(f"
    report_lines.append("")
    report_lines.append(f"- 节点编号: {node.node_id}")
    report_lines.append(f"- 开启时间: {created_at}")
    report_lines.append(f"- 关闭时间: {completed_at}")
    report_lines.append(f"- 总耗时: {duration_str}")
    report_lines.append(f"- 最终进度: {node.progress}%")
    report_lines.append(f"- 关联事件数: {len(events)}")
    report_lines.append(f"- 成果交付数: {len(deliverables)}")
    report_lines.append("")
    if deliverables:
        report_lines.append("
        report_lines.append("")
        for d in deliverables:
            report_lines.append(f"- {d.deliverable_id}: {d.name if hasattr(d,
'name') else d.deliverable_id}")
            report_lines.append("")
    if events:
        report_lines.append("
        report_lines.append("")
        for e in events:
            report_lines.append(f"- [{e.event_category}/{e.event_action}]
{e.summary}")
            report_lines.append("")
    report_text = "\n".join(report_lines)
    if dry_run:
        return {
            "node_id": node.node_id,
            "node_name": node.node_name,
            "created_at": created_at,
            "completed_at": completed_at,
            "duration": duration_str,
            "events_count": len(events),
            "deliverables_count": len(deliverables),
            "report": report_text,
            "status": "preview",
        }
    return {
        "node_id": node.node_id,
        "node_name": node.node_name,
        "created_at": created_at,
        "completed_at": completed_at,
        "duration": duration_str,
        "events_count": len(events),
        "deliverables_count": len(deliverables),
        "report": report_text,

```

```

        "status": "generated",
    }
def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',
errors='replace')
    parser = argparse.ArgumentParser(description="节点闭合总结脚本")
    parser.add_argument("--node-id", "-n", required=True, help="节点编号")
    parser.add_argument("--db-url", default="", help="PostgreSQL 连接 URL")
    parser.add_argument("--preview", action="store_true", help="预览模式")
    args = parser.parse_args()
    result = asyncio.run(generate_node_closure(
        args.node_id,
        db_url=args.db_url,
        dry_run=args.preview,
    ))
    if "error" in result:
        print(f"错误: {result['error']}")
    else:
        print(result["report"])
if __name__ == "__main__":
    main()

```

文件编号: 199 文件路径: scripts\evolution_patch.py

```

from __future__ import annotations
import argparse
import asyncio
import io
import json
import logging
import os
import sys
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
if str(_HERE) not in sys.path:
    sys.path.insert(0, str(_HERE))
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger("evolution_patch")
async def generate_patches(rule_nos: list[str] | None = None, *, db_url: str = "",
dry_run: bool = False) -> list[dict]:
    from emily_core.services.evolution.patch_generator import PatchGenerator
    from evolution_metrics import _init_db
    _init_db(db_url)

```

```

llm_client = None
if not dry_run:
    try:
        from emily_core.infrastructure.llm.client import LLMClient
        api_key = os.environ.get("EMILY_LLM_API_KEY",
os.environ.get("OPENAI_API_KEY", ""))
        base_url = os.environ.get("EMILY_LLM_BASE_URL",
os.environ.get("OPENAI_BASE_URL", ""))
        model = os.environ.get("EMILY_LLM_MODEL",
os.environ.get("OPENAI_MODEL", "gpt-4o"))
        if api_key and base_url:
            llm_client = LLMClient(api_key=api_key, base_url=base_url,
model=model)
        else:
            logger.warning("LLM not configured, running in preview mode")
            dry_run = True
    except Exception as e:
        logger.warning("Failed to init LLM: %s", e)
        dry_run = True
generator = PatchGenerator(llm_client=llm_client)
patches = await generator.generate(rule_nos, dry_run=dry_run)
return patches
def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',
errors='replace')
    parser = argparse.ArgumentParser(description="进化补丁生成脚本")
    parser.add_argument("--all", action="store_true", help="为所有 CONFIRMED 规则生
成补丁")
    parser.add_argument("--rule-no", default="", help="为指定规则生成补丁")
    parser.add_argument("--db-url", default="", help="PostgreSQL 连接 URL")
    parser.add_argument("--preview", action="store_true", help="预览模式（不调
LLM）")
    parser.add_argument("--json", action="store_true", help="输出原始 JSON")
    args = parser.parse_args()
    rule_nos = None
    if args.rule_no:
        rule_nos = [args.rule_no]
    elif not args.all:
        print("请指定 --all 或 --rule-no R-XXX")
        sys.exit(1)
    patches = asyncio.run(generate_patches(
        rule_nos,
        db_url=args.db_url,
        dry_run=args.preview,
    ))
    if args.json:
        print(json.dumps(patches, ensure_ascii=False, indent=2, default=str))
    else:
        print(f"\n补丁生成完成: {len(patches)} 个补丁")
        for p in patches:
            status = p.get("status", "")
            patch_no = p.get("patch_no", "?")

```



```

        rule_no = p.get("rule_no", "?")
        risk = p.get("risk_level", "?")
        if status == "preview":
            print(f" [PREVIEW] {rule_no} -> {p.get('target_path', '?')}")
        elif status == "generated":
            print(f" {patch_no}: {rule_no} -> {p.get('target_path', '?')}")
    [{risk}])
        print(f"    锚点: {p.get('search_anchor', 'N/A')[:50]}")
    else:
        print(f" [ERROR] {rule_no}: {p.get('error', '?')}")
if __name__ == "__main__":
    main()

```

文件编号: 200 文件路径: scripts\evolution_rules.py

```

from __future__ import annotations
import argparse
import asyncio
import io
import json
import logging
import os
import sys
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
if str(_HERE) not in sys.path:
    sys.path.insert(0, str(_HERE))
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger("evolution_rules")
async def induct_rules(end_date: str, *, days: int = 7, db_url: str = "", dry_run:
bool = False) -> list[dict]:
    from emily_core.services.evolution.rule_inductor import RuleInductor
    from evolution_metrics import _init_db
    _init_db(db_url)
    llm_client = None
    if not dry_run:
        try:
            from emily_core.infrastructure.llm.client import LLMClient
            api_key = os.environ.get("EMILY_LLM_API_KEY",
os.environ.get("OPENAI_API_KEY", ""))
            base_url = os.environ.get("EMILY_LLM_BASE_URL",
os.environ.get("OPENAI_BASE_URL", ""))
            model = os.environ.get("EMILY_LLM_MODEL",
os.environ.get("OPENAI_MODEL", "gpt-4o"))
            if api_key and base_url:

```

```

        llm_client = LLMClient(api_key=api_key, base_url=base_url,
model=model)
    else:
        logger.warning("LLM not configured, running in preview mode")
        dry_run = True
    except Exception as e:
        logger.warning("Failed to init LLM: %s", e)
        dry_run = True
    inductor = RuleInductor(llm_client=llm_client)
    rules = await inductor.induct(end_date, days=days, dry_run=dry_run)
    return rules
def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',
errors='replace')
    parser = argparse.ArgumentParser(description="进化规则归纳脚本")
    parser.add_argument("--end-date", "-d", required=True, help="分析截止日期 YYYY-MM-DD")
    parser.add_argument("--days", type=int, default=7, help="回顾天数（默认 7）")
    parser.add_argument("--db-url", default="", help="PostgreSQL 连接 URL")
    parser.add_argument("--preview", action="store_true", help="预览模式（不调 LLM）")
    parser.add_argument("--json", action="store_true", help="输出原始 JSON")
    args = parser.parse_args()
    rules = asyncio.run(induct_rules(
        args.end_date,
        days=args.days,
        db_url=args.db_url,
        dry_run=args.preview,
    ))
    if args.json:
        print(json.dumps(rules, ensure_ascii=False, indent=2, default=str))
    else:
        print(f"\n规则归纳完成: {len(rules)} 条规则")
        for r in rules:
            status = r.get("status", "")
            if status == "preview":
                print(f"  预览模式: 趋势={r.get('trend_data', {})}, 重复异常={r.get('recurring_anomalies', [])}")
            elif status == "generated" or "rule_no" in r:
                print(f"  {r.get('rule_no', '?')}: {r.get('title', '?')} (confidence={r.get('confidence', 0)})")
            else:
                print(f"  {r}")
if __name__ == "__main__":
    main()

```

文件编号: 201 文件路径: scripts\evolution_validate.py

```

from __future__ import annotations
import argparse

```

```

import asyncio
import io
import json
import sys
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
if str(_HERE) not in sys.path:
    sys.path.insert(0, str(_HERE))
from evolution_metrics import _init_db
async def validate_patches(patch_nos: list[str] | None = None, *, db_url: str =
"", dry_run: bool = False) -> list[dict]:
    _init_db(db_url)
    from emily_core.services.evolution.patch_validator import PatchValidator
    validator = PatchValidator()
    return await validator.validate(patch_nos, dry_run=dry_run)
def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',
errors='replace')
    parser = argparse.ArgumentParser(description="补丁效果验证脚本")
    parser.add_argument("--all", action="store_true", help="验证所有已应用 >= 7 天
的补丁")
    parser.add_argument("--patch-no", default="", help="验证指定补丁")
    parser.add_argument("--db-url", default="", help="PostgreSQL 连接 URL")
    parser.add_argument("--preview", action="store_true", help="预览模式（只看指标对
比）")
    args = parser.parse_args()
    patch_nos = None
    if args.patch_no:
        patch_nos = [args.patch_no]
    elif not args.all:
        print("请指定 --all 或 --patch-no EP-XXX")
        sys.exit(1)
    results = asyncio.run(validate_patches(
        patch_nos,
        db_url=args.db_url,
        dry_run=args.preview,
    ))
    print(f"\n补丁验证结果: {len(results)} 条")
    for r in results:
        print(f"  {r.get('patch_no', '?')}: {r.get('decision', r.get('status',
'?''))} "
            f"(before={r.get('avg_health_before', '?')}, after=
{r.get('avg_health_after', '?')})")
    if __name__ == "__main__":
        main()

```

```

from __future__ import annotations
import argparse
import io
import json
import logging
import os
import subprocess
import sys
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(name)s: %(message)s")
logger = logging.getLogger("generate_session_prompt")
def _init_db(db_url: str = "") -> None:
    from emily_core.infrastructure.database.session import init_db
    if db_url:
        init_db(db_url=db_url)
    else:
        db_url_env = os.environ.get("EMILY_DATABASE_URL", "")
        if db_url_env:
            init_db(db_url=db_url_env)
        else:
            pg_port = int(os.environ.get("EMILY_PG_PORT", "")) if
os.environ.get("EMILY_PG_PORT") else None
            if not pg_port:
                try:
                    r = subprocess.run(["docker", "port", "emily-postgres",
"5432/tcp"], capture_output=True, text=True, timeout=5)
                    pg_port = int(r.stdout.strip().rsplit(":", 1)[-1]) if
r.returncode == 0 and r.stdout.strip() else 5432
                except Exception:
                    pg_port = 5432
            init_db(pg_host=os.environ.get("EMILY_PG_HOST", "127.0.0.1"),
pg_port=pg_port,
                    pg_db=os.environ.get("EMILY_PG_DB", "emily"),
                    pg_user=os.environ.get("EMILY_PG_USER", "emily"),
                    pg_password=os.environ.get("EMILY_PG_PASSWORD",
"emily_secret_2026"))
def generate_session_prompt(user_id: str, *, db_url: str = "", dry_run: bool =
False) -> dict:
    _init_db(db_url)
    from emily_core.repositories.user_repo import UserRepository
    from emily_core.repositories.world_book_repo import ProjectWorldBookRepo
    user = UserRepository.get_by_id(user_id)
    if user is None:
        return {"error": "用户不存在", "user_id": user_id}
    project_id = getattr(user, "project_id", None)
    world_book_text = ""
    world_book_tokens = 0

```

```
if project_id:
    wb = ProjectWorldBookRepo.get_by_project(project_id)
    if wb:
        world_book_text = wb.content_text or ""
        world_book_tokens = wb.token_count or 0
rule_book_text = ""
rule_book_path = Path(_CORE_DIR) / ".." / "emily-data" / "rules" / "规则书.md"
if not rule_book_path.exists():
    rule_book_path = Path("/app/rules/规则书.md")
if rule_book_path.exists():
    try:
        rule_book_text = rule_book_path.read_text(encoding="utf-8")
    except Exception as e:
        logger.warning("Failed to read rule book: %s", e)
        rule_book_text = ""
rule_book_tokens = int(len(rule_book_text) / 1.5) if rule_book_text else 0
return {
    "user_id": user_id,
    "user_name": user.username,
    "project_id": project_id or "",
    "world_book_text": world_book_text,
    "world_book_tokens": world_book_tokens,
    "rule_book_text_length": len(rule_book_text),
    "rule_book_tokens": rule_book_tokens,
    "total_tokens": world_book_tokens + rule_book_tokens,
    "dry_run": dry_run,
}
def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',
errors='replace')
    parser = argparse.ArgumentParser(description="生成 Session prompt 段")
    parser.add_argument("--user-id", required=True, help="用户 UUID")
    parser.add_argument("--db-url", default="")
    parser.add_argument("--dry-run", action="store_true")
    parser.add_argument("--show-text", action="store_true", help="显示完整文本（默
认只显示统计）")
    args = parser.parse_args()
    result = generate_session_prompt(args.user_id, db_url=args.db_url,
dry_run=args.dry_run)
    if args.show_text:
        print("=== 世界书 ===")
        print(result.get("world_book_text", "（无）"))
        print(f"\n=== 规则书（{result['rule_book_tokens']} tokens）===")
        print(result.get("rule_book_text_length", 0) > 0 and "（已加载）" or "（未
找到）")
    else:
        output = {k: v for k, v in result.items() if k not in
("world_book_text",)}
        print(json.dumps(output, ensure_ascii=False, indent=2))
if __name__ == "__main__":
    main()
```

文件编号: 203 文件路径: scripts\manage_nodes.py

```

from __future__ import annotations
import argparse
import asyncio
import logging
import sys
from pathlib import Path
import yaml
PROJECT_ROOT = Path(__file__).resolve().parent.parent
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger("manage_nodes")
def _init_db(db_url: str) -> None:
    from emily_core.infrastructure.database.session import init_db
    init_db(db_url)
def load_yaml(filepath: str | Path) -> dict:
    filepath = Path(filepath)
    if not filepath.exists():
        raise FileNotFoundError(f"YAML 文件不存在: {filepath}")
    with open(filepath, "r", encoding="utf-8") as f:
        data = yaml.safe_load(f)
    if not isinstance(data, dict):
        raise ValueError("YAML 文件顶层必须是字典")
    return data
def _assign_auto_node_ids(
    nodes: list[dict],
    project_prefix: str,
) -> list[dict]:
    used_nums = set()
    _collect_used_nums(nodes, project_prefix, used_nums)
    counter = [0]
    _assign_recursive(nodes, project_prefix, used_nums, counter)
    return nodes
def _collect_used_nums(nodes: list[dict], prefix: str, used_nums: set[int]) -> None:
    for n in nodes:
        nid = n.get("node_id", "")
        if nid and nid.startswith(f"{prefix}-"):
            suffix = nid[len(prefix) + 1:]
            try:
                used_nums.add(int(suffix))
            except ValueError:
                pass
        children = n.get("children", [])
        if children:
            _collect_used_nums(children, prefix, used_nums)
def _assign_recursive(

```

```

        nodes: list[dict], prefix: str, used_nums: set[int], counter: list[int],
    ) -> None:
        for n in nodes:
            if not n.get("node_id"):
                while True:
                    counter[0] += 1
                    if counter[0] not in used_nums:
                        break
                n["node_id"] = f"{prefix}-{counter[0]:03d}"
                logger.debug("自动编号: %s → %s", n.get("node_name", "?"),
n["node_id"])
                children = n.get("children", [])
                if children:
                    _assign_recursive(children, prefix, used_nums, counter)
def _print_report(results: list[dict], dry_run: bool = False) -> None:
    phase_labels = {
        "create_node": "创建节点",
        "create_deliverable": "添加成果",
        "mount_child": "挂载子节点",
        "add_dependency": "添加依赖",
        "update_node": "更新节点",
        "activate_node": "激活节点",
        "discard_node": "废弃节点",
        "update_progress": "更新进度",
        "add_shared_file": "增加共享文件",
        "remove_shared_file": "移除共享文件",
        "set_startup_doc": "设置条件文件",
        "clear_startup_doc": "清除条件文件",
        "link_deliverable_file": "关联成果文件",
        "unlink_deliverable_file": "取消成果文件关联",
    }
    print("\n" + "=" * 60)
    print(f"全景节点操作报告{' (DRY-RUN)' if dry_run else ''}")
    print("=" * 60)
    for phase, label in phase_labels.items():
        phase_results = [r for r in results if r.get("phase") == phase]
        if not phase_results:
            continue
        success_count = sum(1 for r in phase_results if r.get("success"))
        skip_count = sum(1 for r in phase_results if r.get("skipped"))
        fail_count = sum(1 for r in phase_results if not r.get("success"))
        print(f"\n— {label} —")
        print(f"  成功: {success_count}  跳过: {skip_count}  失败: {fail_count}")
        for r in phase_results:
            icon = "✓" if r.get("success") else "X"
            if r.get("skipped"):
                icon = "o"
            if r.get("dry_run"):
                icon = "▷"
            node_id = r.get("node_id", "?")
            msg = r.get("message", "")
            print(f"  {icon} [{node_id}] {msg}")
    total_success = sum(1 for r in results if r.get("success"))
    total_fail = sum(1 for r in results if not r.get("success"))

```

```

print("\n" + "-" * 60)
print(f"总计: 成功 {total_success}, 失败 {total_fail}")
print("=" * 60)
def main():
    parser = argparse.ArgumentParser(
        description="全景节点图管理脚本",
    )
    sub = parser.add_subparsers(dest="command", required=True)
    db_url_default = "postgres://emily:emily_secret_2026@localhost:25432/emily"
    p = sub.add_parser("create", help="从 YAML 文件批量创建节点")
    p.add_argument("--file", "-f", required=True, help="YAML 定义文件路径")
    p.add_argument("--db-url", default=db_url_default, help="PostgreSQL 连接 URL")
    p.add_argument("--dry-run", action="store_true", help="预览模式")
    p = sub.add_parser("update", help="从 YAML 文件批量更新节点字段")
    p.add_argument("--file", "-f", required=True, help="YAML 更新文件路径")
    p.add_argument("--db-url", default=db_url_default, help="PostgreSQL 连接 URL")
    p.add_argument("--dry-run", action="store_true", help="预览模式")
    p = sub.add_parser("activate", help="批量激活（审批通过）节点")
    p.add_argument("--node-ids", required=True, help="节点编号，逗号分隔")
    p.add_argument("--operator-id", required=True, help="审批人 UUID")
    p.add_argument("--reject", action="store_true", help="拒绝而非通过")
    p.add_argument("--remark", default="", help="审批备注")
    p.add_argument("--db-url", default=db_url_default, help="PostgreSQL 连接 URL")
    p.add_argument("--dry-run", action="store_true", help="预览模式")
    p = sub.add_parser("discard", help="批量废弃节点")
    p.add_argument("--node-ids", required=True, help="节点编号，逗号分隔")
    p.add_argument("--operator-id", required=True, help="操作人 UUID")
    p.add_argument("--db-url", default=db_url_default, help="PostgreSQL 连接 URL")
    p.add_argument("--dry-run", action="store_true", help="预览模式")
    p = sub.add_parser("progress", help="批量更新成果进度")
    p.add_argument("--file", "-f", required=True, help="YAML 进度文件路径")
    p.add_argument("--db-url", default=db_url_default, help="PostgreSQL 连接 URL")
    p.add_argument("--dry-run", action="store_true", help="预览模式")
    p = sub.add_parser("link-files", help="批量管理节点文件关联（共享文件/条件文件/成果文件）")
    p.add_argument("--file", "-f", required=True, help="YAML 文件关联定义路径")
    p.add_argument("--db-url", default=db_url_default, help="PostgreSQL 连接 URL")
    p.add_argument("--dry-run", action="store_true", help="预览模式")
    p = sub.add_parser("query", help="查询项目节点概览")
    p.add_argument("--project-id", required=True, help="项目 ID")
    p.add_argument("--db-url", default=db_url_default, help="PostgreSQL 连接 URL")
    args = parser.parse_args()
    handlers = {
        "create": _run_create,
        "update": _run_update,
        "activate": _run_activate,
        "discard": _run_discard,
        "progress": _run_progress,
        "link-files": _run_link_files,
        "query": _run_query,
    }
    handlers[args.command](args)
def _run_create(args) -> None:
    yaml_data = load_yaml(args.file)

```



```

project_id = yaml_data.get("project_id", "")
creator_id = yaml_data.get("creator_id", "")
nodes = yaml_data.get("nodes", [])
if not project_id:
    print("错误: YAML 文件缺少 project_id")
    sys.exit(1)
if not creator_id:
    print(
        "错误: YAML 文件缺少 creator_id。"
        "请查 users 表填入管理员 UUID: \n"
        " docker exec emily-postgres psql -U emily -d emily "
        "-c \"SELECT id, username, permission_level FROM users WHERE "
        "status='active' ORDER BY permission_level DESC LIMIT 5;\""
    )
    sys.exit(1)
if not nodes:
    print("错误: YAML 文件 nodes 列表为空")
    sys.exit(1)
_init_db(args.db_url)
from emily_core.repositories.user_repo import UserRepository
creator_user = UserRepository.get_by_id(creator_id)
if creator_user is None:
    print(f"错误: creator_id [{creator_id}] 在 users 表中不存在")
    sys.exit(1)
logger.info("项目: %s | 创建人: %s (%s) | 节点数: %d",
            project_id, creator_user.username, creator_id, len(nodes))
project_prefix = yaml_data.get("project_prefix", project_id[:4].upper())
_assign_auto_node_ids(nodes, project_prefix)
from emily_core.services.node_batch import create_node_tree
results = asyncio.run(create_node_tree(
    project_id=project_id,
    creator_id=creator_id,
    nodes=nodes,
    dry_run=args.dry_run,
))
_print_report(results, dry_run=args.dry_run)
if any(not r.get("success") for r in results):
    sys.exit(1)
def _run_update(args) -> None:
    yaml_data = load_yaml(args.file)
    operator_id = yaml_data.get("operator_id", "")
    updates = yaml_data.get("updates", [])
    if not operator_id:
        print("错误: YAML 文件缺少 operator_id")
        sys.exit(1)
    if not updates:
        print("错误: YAML 文件 updates 列表为空")
        sys.exit(1)
    _init_db(args.db_url)
    logger.info("更新节点: %d 条 | 操作人: %s", len(updates), operator_id)
    from emily_core.services.node_batch_update import batch_update_nodes
    results = asyncio.run(batch_update_nodes(
        updates=updates,
        operator_id=operator_id,
    ))

```

```
        dry_run=args.dry_run,
    ))
    _print_report(results, dry_run=args.dry_run)
    if any(not r.get("success") for r in results):
        sys.exit(1)
def _run_activate(args) -> None:
    node_ids = [n.strip() for n in args.node_ids.split(",") if n.strip()]
    if not node_ids:
        print("错误: --node-ids 不能为空")
        sys.exit(1)
    _init_db(args.db_url)
    approved = not args.reject
    action = "激活" if approved else "拒绝"
    logger.info("批量%s: %s | 审批人: %s", action, node_ids, args.operator_id)
    from emily_core.services.node_batch_update import batch_activate_nodes
    results = asyncio.run(batch_activate_nodes(
        node_ids=node_ids,
        approver_id=args.operator_id,
        approved=approved,
        remark=args.remark,
        dry_run=args.dry_run,
    ))
    _print_report(results, dry_run=args.dry_run)
    if any(not r.get("success") for r in results):
        sys.exit(1)
def _run_discard(args) -> None:
    node_ids = [n.strip() for n in args.node_ids.split(",") if n.strip()]
    if not node_ids:
        print("错误: --node-ids 不能为空")
        sys.exit(1)
    _init_db(args.db_url)
    logger.info("批量废弃: %s | 操作人: %s", node_ids, args.operator_id)
    from emily_core.services.node_batch_update import batch_discard_nodes
    results = asyncio.run(batch_discard_nodes(
        node_ids=node_ids,
        operator_id=args.operator_id,
        dry_run=args.dry_run,
    ))
    _print_report(results, dry_run=args.dry_run)
    if any(not r.get("success") for r in results):
        sys.exit(1)
def _run_progress(args) -> None:
    yaml_data = load_yaml(args.file)
    operator_id = yaml_data.get("operator_id", "")
    progress = yaml_data.get("progress", [])
    if not operator_id:
        print("错误: YAML 文件缺少 operator_id")
        sys.exit(1)
    if not progress:
        print("错误: YAML 文件 progress 列表为空")
        sys.exit(1)
    _init_db(args.db_url)
    logger.info("更新进度: %d 条 | 操作人: %s", len(progress), operator_id)
    from emily_core.services.node_batch_update import batch_update_progress
```

```

    results = asyncio.run(batch_update_progress(
        progress_updates=progress,
        operator_id=operator_id,
        dry_run=args.dry_run,
    ))
    _print_report(results, dry_run=args.dry_run)
    if any(not r.get("success") for r in results):
        sys.exit(1)
def _run_link_files(args) -> None:
    yaml_data = load_yaml(args.file)
    operator_id = yaml_data.get("operator_id", "")
    links = yaml_data.get("links", [])
    if not operator_id:
        print("错误: YAML 文件缺少 operator_id")
        sys.exit(1)
    if not links:
        print("错误: YAML 文件 links 列表为空")
        sys.exit(1)
    _init_db(args.db_url)
    logger.info("文件关联: %d 条 | 操作人: %s", len(links), operator_id)
    from emily_core.services.node_batch_update import batch_link_files
    results = asyncio.run(batch_link_files(
        links=links,
        operator_id=operator_id,
        dry_run=args.dry_run,
    ))
    _print_report(results, dry_run=args.dry_run)
    if any(not r.get("success") for r in results):
        sys.exit(1)
def _run_query(args) -> None:
    from emily_core.repositories.node_repo import ProjectNodeRepo
    _init_db(args.db_url)
    node_list = asyncio.run(
        asyncio.to_thread(ProjectNodeRepo.find_by_project, args.project_id),
    )
    if not node_list:
        print(f"项目 {args.project_id} 下没有节点")
        return
    print(f"\n项目 {args.project_id} 的全景节点 ({len(node_list)} 个)")
    print("=" * 80)
    print(f"{'节点编号':<20} {'节点名称':<20} {'状态':<20} {'进度':>6} {'主责':<10}")
    print("-" * 80)
    for n in node_list:
        progress = float(n.progress) if n.progress else 0.0
        print(f"{n.node_id:<20} {n.node_name:<20} {n.status:<20} {progress:>5.1f}% {n.owner_dept_id or '':<10}")
    if __name__ == "__main__":
        main()

```

文件编号: 204 文件路径: scripts\register_api.py

```

from __future__ import annotations
import argparse
import importlib
import json
import logging
import os
import subprocess
import sys
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
logger = logging.getLogger("emily.register_api")
_SENTINEL = "XXXXXXXXXX"
def _detect_docker_pg_port() -> int | None:
    try:
        result = subprocess.run(
            ["docker", "port", "emily-postgres", "5432/tcp"],
            capture_output=True, text=True, timeout=5,
        )
        if result.returncode == 0 and result.stdout.strip():
            port_str = result.stdout.strip().rsplit(":", 1)[-1]
            return int(port_str)
    except (FileNotFoundError, subprocess.TimeoutExpired, ValueError, OSError):
        pass
    return None
def _init_db():
    from emily_core.infrastructure.database import init_db
    db_url = os.environ.get("EMILY_DATABASE_URL", "")
    if db_url:
        init_db(db_url=db_url)
    else:
        pg_host = os.environ.get("EMILY_PG_HOST", os.environ.get("PG_HOST",
"127.0.0.1"))
        pg_port_env = os.environ.get("EMILY_PG_PORT", os.environ.get("PG_PORT"))
        if pg_port_env:
            pg_port = int(pg_port_env)
        else:
            pg_port = _detect_docker_pg_port() or 5432
        pg_db = os.environ.get("EMILY_PG_DB", "emily")
        pg_user = os.environ.get("EMILY_PG_USER", "emily")
        pg_password = os.environ.get("EMILY_PG_PASSWORD", "emily_secret_2026")
        init_db(pg_host=pg_host, pg_port=pg_port, pg_db=pg_db, pg_user=pg_user,
pg_password=pg_password)
_TOOL_SCRIPTS = [
    ("search_files", "emily_core.tools.scripts.search_files", "base", "all"),
]
def do_register(api_id: str) -> bool:
    _init_db()
    entry = None
    for e in _TOOL_SCRIPTS:

```

```

        if e[0] == api_id:
            entry = e
            break
    if entry is None:
        print(f"[ERROR] Unknown API: {api_id}")
        return False
    api_id, module_path, category, perm_flag = entry
    try:
        module = importlib.import_module(module_path)
        register_fn = getattr(module, "register", None)
        if register_fn is None:
            print(f"[ERROR] {module_path} has no register() function")
            return False
    except Exception as e:
        print(f"[ERROR] Import {module_path} failed: {e}")
        return False
    try:
        bft = register_fn(core=None)
    except Exception as e:
        print(f"[ERROR] register() failed: {e}")
        return False
    print(f" [code] Tool '{bft.name}' loaded from {module_path}")
    from emily_core.repositories.tool_registry_repo import ToolRegistryRepo
    signature = json.dumps(
        {"params": bft.parameters, "returns": "dict"},
        ensure_ascii=False,
    )
    ok = ToolRegistryRepo.upsert(
        api_id=bft.name,
        signature=signature,
        display_name=(
            bft.description.split("。")[0][:80]
            if bft.description
            else f"Tool: {bft.name}"
        ),
        category=category,
        permission_flag=perm_flag,
        handler_module=getattr(bft.handler, "__module__", module_path),
    )
    if ok:
        print(f" [DB] Tool '{bft.name}' registered in tool_registry table")
    else:
        print(f" [DB] Tool '{bft.name}' DB upsert FAILED")
        return False
    return True
def cmd_list():
    _init_db()
    from emily_core.repositories.tool_registry_repo import ToolRegistryRepo
    rows = ToolRegistryRepo.get_all()
    if not rows:
        print("(no registered APIs)")
        return
    print(f"\n{'API ID':<24s} {'Category':<10s} {'Active':<6s} Description")
    print("-" * 80)

```

```

for r in rows:
    active = "YES" if r.get("is_active") else "NO"
    print(
        f"  {r['api_id']:<24s} {r.get('category', ''):<10s} "
        f"{active:<6s} {r.get('display_name', '')}"
    )
def cmd_help_api(api_id: str):
    _init_db()
    from emily_core.repositories.tool_registry_repo import ToolRegistryRepo
    rows = ToolRegistryRepo.get_all()
    found = None
    for r in rows:
        if r["api_id"] == api_id:
            found = r
            break
    if found:
        print(f"\nAPI: {found['api_id']}")
        print(f"  描述: {found.get('display_name', '')}")
        print(f"  类别: {found.get('category', '')}")
        print(f"  权限: {found.get('permission_flag', '')}")
        print(f"  模块: {found.get('handler_module', '')}")
        try:
            sig = json.loads(found.get("signature", "{}"))
            print(f"  签名: {json.dumps(sig, ensure_ascii=False, indent=2)}")
        except json.JSONDecodeError:
            print(f"  签名: {found.get('signature', '')}")
    else:
        print(f"[ERROR] API '{api_id}' not found in tool_registry")
def main():
    parser = argparse.ArgumentParser(description="API 注册器")
    parser.add_argument("--api", help="注册单个 API")
    parser.add_argument("--all", action="store_true", help="注册全部 API")
    parser.add_argument("--list", action="store_true", help="查看已注册 API")
    parser.add_argument("--help-api", help="查看某个 API 帮助")
    args = parser.parse_args()
    if args.list:
        cmd_list()
    elif args.help_api:
        cmd_help_api(args.help_api)
    elif args.all:
        success = 0
        for e in _TOOL_SCRIPTS:
            api_id = e[0]
            print(f"\n=== Registering {api_id} ===")
            if do_register(api_id):
                success += 1
        print(f"\nDone: {success}/{len(_TOOL_SCRIPTS)} APIs registered")
    elif args.api:
        do_register(args.api)
    else:
        parser.print_help()
if __name__ == "__main__":
    main()

```

文件编号：205 文件路径：scripts\rescan_files.py

```
from __future__ import annotations
import argparse
import hashlib
import logging
import os
import uuid
from datetime import datetime, timezone
from pathlib import Path
from sqlalchemy import Boolean, Column, create_engine, Integer, String, Text
from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker
PROJECT_ROOT = Path(__file__).resolve().parent.parent
DEFAULT_SCAN_DIR = PROJECT_ROOT / "emily-data" / "files"
DEFAULT_ATTACHMENTS_DIR = PROJECT_ROOT / "emily-data" / "attachments"
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
)
logger = logging.getLogger("rescan_files")
class Base(DeclarativeBase):
    pass
class File(Base):
    __tablename__ = "files"
    id = Column(String, primary_key=True)
    file_no = Column(String(50), unique=True, nullable=False)
    project_id = Column(String, nullable=True)
    message_id = Column(String, nullable=True)
    filename = Column(String(500), nullable=False)
    file_type = Column(String(100))
    bucket = Column(String(100))
    object_key = Column(String)
    storage_path = Column(String)
    file_size = Column(Integer)
    uploaded_by = Column(String, nullable=True)
    parse_status = Column(String(50), default="pending")
    created_at = Column(String)
    file_ext = Column(String(50), default="")
    file_url = Column(String(1000), default="")
    file_hash = Column(String(256), default="")
    version = Column(String(50), default="V1.0")
    is_latest = Column(Boolean, default=True)
    parent_file_id = Column(String, nullable=True)
    confidentiality = Column(Integer, default=0)
    creator_id = Column(String, nullable=True)
    updated_at = Column(String)
    is_deleted = Column(Boolean, default=False)
    source_module_id = Column(String(100), default="")
    source_module_type = Column(String(50), default="")
    _EXT_TYPE_MAP: dict[str, str] = {
```

```
    ".pdf": "pdf",
    ".doc": "doc", ".docx": "doc",
    ".xls": "xls", ".xlsx": "xls",
    ".ppt": "ppt", ".pptx": "ppt",
    ".jpg": "image", ".jpeg": "image", ".png": "image",
    ".gif": "image", ".bmp": "image", ".webp": "image",
    ".mp3": "audio", ".wav": "audio", ".amr": "audio",
    ".mp4": "video", ".avi": "video", ".mkv": "video",
    ".zip": "archive", ".rar": "archive", ".7z": "archive",
    ".txt": "text", ".md": "text", ".csv": "csv",
    ".json": "data", ".xml": "data",
}

def _utc_now() -> str:
    return datetime.now(timezone.utc).isoformat()

def compute_sha256(file_path: Path) -> str:
    h = hashlib.sha256()
    with open(file_path, "rb") as f:
        while True:
            chunk = f.read(8192)
            if not chunk:
                break
            h.update(chunk)
    return h.hexdigest()

def infer_file_type(ext: str) -> str:
    return _EXT_TYPE_MAP.get(ext.lower(), "file")

def generate_file_no(session: Session) -> str:
    today_str = datetime.now(timezone.utc).strftime("%Y%m%d")
    prefix = f"FIL-{today_str}-"
    last = (
        session.query(File)
        .filter(File.file_no.like(f"{prefix}%"))
        .order_by(File.file_no.desc())
        .first()
    )
    if last is None:
        return f"{prefix}0001"
    seq_str = last.file_no[len(prefix):]
    try:
        seq = int(seq_str) + 1
    except ValueError:
        seq = 1
    return f"{prefix}{seq:04d}"

def scan_directory(scan_dir: Path) -> list[Path]:
    if not scan_dir.exists():
        logger.warning("扫描目录不存在: %s", scan_dir)
        return []
    files = []
    for root, _dirs, filenames in os.walk(scan_dir):
        for fn in filenames:
            fp = Path(root) / fn
            if fn.startswith(".") or fn == ".gitkeep":
                continue
            files.append(fp)
    return files
```



```
def find_existing_record(session: Session, storage_path: str, filename: str,
file_size: int):
    if storage_path:
        record = session.query(File).filter(
            File.storage_path == storage_path,
            File.is_deleted == False, # noqa: E712
        ).first()
        if record:
            return record
    record = session.query(File).filter(
        File.filename == filename,
        File.file_size == file_size,
        File.is_deleted == False, # noqa: E712
    ).first()
    if record:
        return record
    return None

def insert_file_record(session: Session, *, file_path: Path, scan_root: Path) ->
str:
    file_no = generate_file_no(session)
    ext = file_path.suffix.lower()
    file_size = file_path.stat().st_size
    file_hash = compute_sha256(file_path)
    rel_path = str(file_path.relative_to(scan_root)).replace("\\", "/")
    file_type = infer_file_type(ext)
    now_iso = _utc_now()
    new_id = str(uuid.uuid4())
    record = File(
        id=new_id,
        file_no=file_no,
        filename=file_path.name,
        project_id=None,
        message_id=None,
        file_type=file_type,
        storage_path=rel_path,
        file_size=file_size,
        uploaded_by=None,
        parse_status="pending",
        created_at=now_iso,
        file_ext=ext.lstrip("."),
        file_url=rel_path,
        file_hash=file_hash,
        version="V1.0",
        is_latest=True,
        parent_file_id=None,
        confidentiality=0,
        creator_id=None,
        updated_at=now_iso,
        is_deleted=False,
        source_module_id="",
        source_module_type="",
    )
    session.add(record)
    session.flush()
```

```

    return file_no
def run_rescan(
    scan_dirs: list[Path],
    db_url: str,
    dry_run: bool = False,
) -> None:
    engine = create_engine(db_url, echo=False, pool_pre_ping=True)
    SessionLocal = sessionmaker(
        autocommit=False, autoflush=False, bind=engine, expire_on_commit=False,
    )
    total_scanned = 0
    total_existing = 0
    total_inserted = 0
    total_error = 0
    inserted_details: list[dict] = []
    existing_details: list[dict] = []
    error_details: list[str] = []
    for scan_dir in scan_dirs:
        logger.info("扫描目录: %s", scan_dir)
        file_paths = scan_directory(scan_dir)
        logger.info("发现 %d 个文件", len(file_paths))
        for fp in file_paths:
            total_scanned += 1
            rel_path = str(fp.relative_to(scan_dir)).replace("\\", "/")
            file_size = fp.stat().st_size
            try:
                session = SessionLocal()
                try:
                    existing = find_existing_record(session, rel_path, fp.name,
file_size)

                    if existing:
                        total_existing += 1
                        existing_details.append({
                            "file": str(fp),
                            "file_no": existing.file_no,
                            "matched_by": "storage_path" if existing.storage_path
== rel_path else "filename+size",
                        })
                        logger.debug("已存在: %s (file_no=%s)", fp.name,
existing.file_no)
                    else:
                        if dry_run:
                            total_inserted += 1
                            inserted_details.append({
                                "file": str(fp),
                                "file_no": "[DRY-RUN]",
                                "rel_path": rel_path,
                                "size": file_size,
                            })
                            logger.info("[DRY-RUN] 将补录: %s (%d bytes)",
fp.name, file_size)
                        else:
                            file_no = insert_file_record(
                                session,

```

```

        file_path=fp,
        scan_root=scan_dir,
    )
    session.commit()
    total_inserted += 1
    inserted_details.append({
        "file": str(fp),
        "file_no": file_no,
        "rel_path": rel_path,
        "size": file_size,
    })
    logger.info("已补录: %s → %s", fp.name, file_no)

finally:
    session.close()
except Exception as e:
    total_error += 1
    error_details.append(f"{fp}: {e}")
    logger.error("处理失败: %s - %s", fp, e)

print("\n" + "=" * 60)
print("扫描补录报告")
print("=" * 60)
print(f"扫描目录 : {' '.join(str(d) for d in scan_dirs)}")
print(f"扫描文件 : {total_scanned}")
print(f"已存在 : {total_existing}")
print(f"新补录 : {total_inserted}" + (" (DRY-RUN)" if dry_run else ""))
print(f"失败 : {total_error}")
print("-" * 60)
if existing_details:
    print("\n已存在文件（跳过）: ")
    for item in existing_details:
        print(f" {item['file']} -> {item['file_no']} (匹配: {item['matched_by']})")
if inserted_details:
    print("\n新补录文件: ")
    for item in inserted_details:
        size_kb = item['size'] / 1024
        print(f" {item['file']} -> {item['file_no']} ({size_kb:.1f} KB)
path={item['rel_path']}")
if error_details:
    print("\n失败文件: ")
    for msg in error_details:
        print(f" {msg}")
print("=" * 60)
engine.dispose()

def main():
    parser = argparse.ArgumentParser(
        description="扫描本地文件目录，对照 files 表补录缺失记录",
    )
    parser.add_argument(
        "--dir",
        type=str,
        default=str(DEFAULT_SCAN_DIR),
        help=f"扫描目录（默认: {DEFAULT_SCAN_DIR}"),
    )

```

```

    parser.add_argument(
        "--include-attachments",
        action="store_true",
        help="同时扫描 emily-data/attachments 目录",
    )
    parser.add_argument(
        "--db-url",
        type=str,
        default="postgresql://emily:emily_secret_2026@localhost:25432/emily",
        help="PostgreSQL 连接 URL (默认: postgresql://emily:emily_secret_2026@localhost:25432/emily)",
    )
    parser.add_argument(
        "--dry-run",
        action="store_true",
        help="预览模式: 只输出报告, 不写入数据库",
    )
    args = parser.parse_args()
    scan_dirs = [Path(args.dir)]
    if args.include_attachments:
        scan_dirs.append(DEFAULT_ATTACHMENTS_DIR)
    logger.info("数据库: %s", args.db_url.replace("emily_secret_2026", "****"))
    run_rescan(scan_dirs, db_url=args.db_url, dry_run=args.dry_run)
if __name__ == "__main__":
    main()

```

文件编号: 206 文件路径: scripts\self_check.py

```

from __future__ import annotations
import argparse
import io
import json
import logging
import os
import subprocess
import sys
from datetime import datetime, timezone, timedelta
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(name)s: %(message)s")
logger = logging.getLogger("self_check")
BEIJING_TZ = timezone(timedelta(hours=8))
def _init_db(db_url: str = "") -> None:
    from emily_core.infrastructure.database.session import init_db
    if db_url:
        init_db(db_url=db_url)
    else:
        db_url_env = os.environ.get("EMILY_DATABASE_URL", "")

```

```

        if db_url_env:
            init_db(db_url=db_url_env)
        else:
            pg_port = int(os.environ.get("EMILY_PG_PORT", "")) if
os.environ.get("EMILY_PG_PORT") else None
            if not pg_port:
                try:
                    r = subprocess.run(["docker", "port", "emily-postgres",
"5432/tcp"], capture_output=True, text=True, timeout=5)
                    pg_port = int(r.stdout.strip().rsplit(":", 1)[-1]) if
r.returncode == 0 and r.stdout.strip() else 5432
                except Exception:
                    pg_port = 5432
            init_db(pg_host=os.environ.get("EMILY_PG_HOST", "127.0.0.1"),
pg_port=pg_port,
                    pg_db=os.environ.get("EMILY_PG_DB", "emily"),
                    pg_user=os.environ.get("EMILY_PG_USER", "emily"),
                    pg_password=os.environ.get("EMILY_PG_PASSWORD",
"emily_secret_2026"))
def self_check(*, db_url: str = "", dry_run: bool = False) -> dict:
    _init_db(db_url)
    from emily_core.infrastructure.database.session import get_session
    from emily_core.infrastructure.database.models import User, Project, Event,
Task, ProjectNode, ProjectWorldBook
    result = {
        "checked_at": datetime.now(BEIJING_TZ).isoformat(),
        "dry_run": dry_run,
    }
    with get_session() as session:
        total_users = session.query(User).filter(User.is_deleted == False).count()
        active_users = session.query(User).filter(User.is_deleted == False,
User.status == "active").count()
        admin_users = session.query(User).filter(User.is_deleted == False,
User.is_admin == True).count()
        result["users"] = {"total": total_users, "active": active_users, "admins":
admin_users}
        total_projects = session.query(Project).filter(Project.is_deleted ==
False).count()
        active_projects = session.query(Project).filter(Project.is_deleted ==
False, Project.status == "active").count()
        result["projects"] = {"total": total_projects, "active": active_projects}
        event_count = session.query(Event).count()
        task_count = session.query(Task).count()
        node_count = session.query(ProjectNode).filter(ProjectNode.is_discarded ==
False).count()
        result["business"] = {"events": event_count, "tasks": task_count, "nodes":
node_count}
        wb_count = session.query(ProjectWorldBook).count()
        wb_activated =
session.query(ProjectWorldBook).filter(ProjectWorldBook.is_activated ==
True).count()
        result["world_books"] = {"total": wb_count, "activated": wb_activated}
        sop_count = 0
    try:

```

```

        from emily_core.skill.registry import SkillRegistry
        skill_dir = "/app/skills"
        if not Path(skill_dir).exists():
            dev_dir = str(Path(__file__).resolve().parent.parent / "emily-
data" / "skills")
            if Path(dev_dir).exists():
                skill_dir = dev_dir
        if skill_dir and Path(skill_dir).exists():
            reg = SkillRegistry(skill_directory=skill_dir)
            reg.load()
            sop_count = len(reg.list_sop_ids())
        except Exception:
            pass
        result["knowledge"] = {"sop_count": sop_count}
    return result

def _format_self_check(result: dict) -> str:
    lines = []
    lines.append("Emily 系统自检报告")
    lines.append("=" * 40)
    lines.append(f"检查时间: {result['checked_at']}")
    lines.append("=" * 40)
    u = result.get("users", {})
    lines.append(f"\n用户: {u.get('active', 0)} 活跃 / {u.get('total', 0)} 总计 /
{u.get('admins', 0)} 管理员")
    p = result.get("projects", {})
    lines.append(f"项目: {p.get('active', 0)} 活跃 / {p.get('total', 0)} 总计")
    b = result.get("business", {})
    lines.append(f"业务: {b.get('events', 0)} 事件 / {b.get('tasks', 0)} 任务 /
{b.get('nodes', 0)} 节点")
    wb = result.get("world_books", {})
    lines.append(f"世界书: {wb.get('total', 0)} 份 / {wb.get('activated', 0)} 已激
活")
    k = result.get("knowledge", {})
    lines.append(f"知识库: {k.get('sop_count', 0)} 个 SOP")
    return "\n".join(lines)

def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',
errors='replace')
    parser = argparse.ArgumentParser(description="Emily 系统自检")
    parser.add_argument("--db-url", default="")
    parser.add_argument("--dry-run", action="store_true")
    parser.add_argument("--json", action="store_true")
    args = parser.parse_args()
    result = self_check(db_url=args.db_url, dry_run=args.dry_run)
    if args.json:
        print(json.dumps(result, ensure_ascii=False, indent=2))
    else:
        print(_format_self_check(result))

if __name__ == "__main__":
    main()

```

文件编号: 207 文件路径: scripts\sop_to_skill.py

```
import argparse
import os
import sys
from pathlib import Path
PROJECT_ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(PROJECT_ROOT / "emily-core"))
def _find_sop_dir() -> Path:
    candidates = [
        PROJECT_ROOT / "emily-data" / "sops",
        Path("/app/sops"),
    ]
    for d in candidates:
        if d.exists():
            return d
    raise FileNotFoundError("SOP 目录未找到")
def _find_skill_dir() -> Path:
    candidates = [
        PROJECT_ROOT / "emily-data" / "skills",
        Path("/app/skills"),
    ]
    for d in candidates:
        if d.exists():
            return d
    default = candidates[0]
    default.mkdir(parents=True, exist_ok=True)
    return default
def _load_sop(sop_id: str, sop_dir: Path) -> str:
    for f in sop_dir.glob(f"*{sop_id}*.md"):
        return f.read_text(encoding="utf-8")
    raise FileNotFoundError(f"SOP 文件未找到: {sop_id} (在 {sop_dir})")
def _call_llm(sop_text: str, api_key: str, base_url: str, model: str) -> str:
    from openai import OpenAI
    client = OpenAI(api_key=api_key, base_url=base_url)
    prompt_path = PROJECT_ROOT / "emily-data" / "prompts" / "sop_to_skill.md"
    if prompt_path.exists():
        system_prompt = prompt_path.read_text(encoding="utf-8")
    else:
        system_prompt = "将以下 SOP 文档转换为 Skill YAML（三段结构: instructions / tools / steps），不要输出 datasets 段。"
    system_prompt = system_prompt.replace("{sop_text}", sop_text)
    response = client.chat.completions.create(
        model=model,
        messages=[
            {"role": "system", "content": system_prompt},
            {"role": "user", "content": "请转换此 SOP 文档为 Skill YAML。"},
        ],
        temperature=0,
    )
    content = response.choices[0].message.content or ""
    if "`yaml" in content:
        content = content.split("`yaml")[1].split("`")[0]
```

```

elif ```` in content:
    content = content.split("````")[1].split("````")[0]
return content.strip()
def _validate_yaml(text: str) -> bool:
    try:
        import yaml
        data = yaml.safe_load(text)
        return isinstance(data, dict) and "skill_id" in data and "steps" in data
    except Exception:
        return False
def convert_sop(sop_id: str, dry_run: bool = False) -> str | None:
    sop_dir = _find_sop_dir()
    skill_dir = _find_skill_dir()
    sop_text = _load_sop(sop_id, sop_dir)
    print(f"加载 SOP: {sop_id} ({len(sop_text)} chars)")
    api_key = os.environ.get("DEEPSEEK_API_KEY", "")
    base_url = os.environ.get("DEEPSEEK_BASE_URL", "https://api.deepseek.com")
    model = os.environ.get("DEEPSEEK_MODEL", "deepseek-chat")
    if not api_key:
        print("错误: 未设置 DEEPSEEK_API_KEY 环境变量")
        return None
    yaml_text = _call_llm(sop_text, api_key, base_url, model)
    if not _validate_yaml(yaml_text):
        print("警告: LLM 输出不是合法 Skill YAML, 请人工审校")
    if dry_run:
        print("\n" + "=" * 60)
        print(yaml_text)
        print("=" * 60)
        return yaml_text
    import yaml
    data = yaml.safe_load(yaml_text)
    skill_id = data.get("skill_id", sop_id)
    output_path = skill_dir / f"{skill_id}.skill.yaml"
    output_path.write_text(yaml_text, encoding="utf-8")
    print(f"写入: {output_path}")
    return yaml_text
def _notify_reload() -> bool:
    import json
    try:
        import urllib.request
        core_host = os.environ.get("EMILY_CORE_HOST", "localhost")
        core_port = os.environ.get("EMILY_CORE_PORT", "18080")
        url = f"http://{core_host}:{core_port}/api/v1/skills/reload"
        req = urllib.request.Request(url, method="POST", data=b"")
        with urllib.request.urlopen(req, timeout=5) as resp:
            result = json.loads(resp.read().decode("utf-8"))
            if result.get("ok"):
                print(f"✓ EmilyCore 热重载成功: {result.get('total', 0)} skill(s)")
                return True
            else:
                print(f"X EmilyCore 热重载失败: {result.get('error', '未知错误')}")
                return False
    except Exception as e:

```



```

        print(f"⚠ 无法通知 EmilyCore 热重载（容器未运行？）：{e}")
        return False
def main():
    parser = argparse.ArgumentParser(description="SOP-to-Skill 转换器")
    parser.add_argument("--sop", help="SOP 编号（如 SOP-002-REC）")
    parser.add_argument("--all", action="store_true", help="批量转换全部 SOP")
    parser.add_argument("--dry-run", action="store_true", help="输出到 stdout 不写文件")
    parser.add_argument("--notify", action="store_true", help="转换完成后通知 EmilyCore 热重载")
    args = parser.parse_args()
    if not args.sop and not args.all:
        parser.print_help()
        return
    converted = 0
    if args.all:
        sop_dir = _find_sop_dir()
        for f in sorted(sop_dir.glob("SOP-*.md")):
            sop_id = f.stem.split("-")[0] + "-" + f.stem.split("-")[1] + "-" + f.stem.split("-")[2]
            print(f"\n转换: {sop_id}")
            result = convert_sop(sop_id, dry_run=args.dry_run)
            if result:
                converted += 1
    else:
        result = convert_sop(args.sop, dry_run=args.dry_run)
        if result:
            converted += 1
    if args.notify and converted > 0 and not args.dry_run:
        print(f"\n已转换 {converted} 个 Skill, 通知 EmilyCore 热重载...")
        _notify_reload()
if __name__ == "__main__":
    main()

```

文件编号：208 文件路径：scripts\update_world_book.py

```

from __future__ import annotations
import argparse
import asyncio
import io
import json
import logging
import os
import subprocess
import sys
from pathlib import Path
_HERE = Path(__file__).resolve().parent
_CORE_DIR = _HERE.parent / "emily-core"
if str(_CORE_DIR) not in sys.path:
    sys.path.insert(0, str(_CORE_DIR))
logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s] %(name)s: %(message)s")

```

```

logger = logging.getLogger("update_world_book")
def _init_db(db_url: str = "") -> None:
    from emily_core.infrastructure.database.session import init_db
    if db_url:
        init_db(db_url=db_url)
    else:
        db_url_env = os.environ.get("EMILY_DATABASE_URL", "")
        if db_url_env:
            init_db(db_url=db_url_env)
        else:
            pg_port = int(os.environ.get("EMILY_PG_PORT", "")) if
os.environ.get("EMILY_PG_PORT") else None
            if not pg_port:
                try:
                    r = subprocess.run(["docker", "port", "emily-postgres",
"5432/tcp"], capture_output=True, text=True, timeout=5)
                    pg_port = int(r.stdout.strip().rsplit(":", 1)[-1]) if
r.returncode == 0 and r.stdout.strip() else 5432
                except Exception:
                    pg_port = 5432
            init_db(pg_host=os.environ.get("EMILY_PG_HOST", "127.0.0.1"),
pg_port=pg_port,
                    pg_db=os.environ.get("EMILY_PG_DB", "emily"),
                    pg_user=os.environ.get("EMILY_PG_USER", "emily"),
                    pg_password=os.environ.get("EMILY_PG_PASSWORD",
"emily_secret_2026"))
async def update_world_book(project_id: str, *, db_url: str = "", dry_run: bool =
False) -> dict:
    _init_db(db_url)
    from emily_core.services.world_book_service import ProjectWorldBookService
    service = ProjectWorldBookService()
    return await service.update_stale(project_id, dry_run=dry_run)
def main():
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8',
errors='replace')
    sys.stderr = io.TextIOWrapper(sys.stderr.buffer, encoding='utf-8',
errors='replace')
    parser = argparse.ArgumentParser(description="增量更新世界书")
    parser.add_argument("--project-id", help="项目 ID")
    parser.add_argument("--all", action="store_true")
    parser.add_argument("--db-url", default="")
    parser.add_argument("--dry-run", action="store_true")
    args = parser.parse_args()
    if not args.project_id and not args.all:
        parser.error("请指定 --project-id 或 --all")
    if args.all:
        _init_db(args.db_url)
        from emily_core.services.world_book_service import ProjectWorldBookService
        service = ProjectWorldBookService()
        results = asyncio.run(service.update_all(dry_run=args.dry_run))
        for r in results:
            print(json.dumps({k: v for k, v in r.items() if k not in
("content_json", "drift_details")}, ensure_ascii=False, indent=2, default=str))
        else:

```

```

        result = asyncio.run(update_world_book(args.project_id,
        db_url=args.db_url, dry_run=args.dry_run))
        print(json.dumps({k: v for k, v in result.items() if k not in
        ("content_json",)}, ensure_ascii=False, indent=2, default=str))
if __name__ == "__main__":
    main()

```

文件编号：209 文件路径：data\plugins\emily_agent_conf_schema.json

```

{
  "emycore_url": {
    "description": "Emily Core 容器 API 地址",
    "type": "string",
    "hint": "独立业务核心容器的 HTTP 地址（Docker 内网用服务名）",
    "default": "http://emily-core:18080"
  },
  "emycore_sse_url": {
    "description": "Emily Core SSE 出站推送地址",
    "type": "string",
    "hint": "为空时自动取 {emycore_url}/api/v1/events/outbound",
    "default": ""
  },
  "emycore_api_token": {
    "description": "Emily Core API 鉴权令牌",
    "type": "string",
    "hint": "与 Core 容器 EMILY_API_TOKEN 一致；为空时不做鉴权（仅内网部署）",
    "default": ""
  }
}

```

文件编号：210 文件路径：data\plugins\emily_agent\adapters\api_client.py

```

from __future__ import annotations
import logging
from typing import TYPE_CHECKING
import aiohttp
if TYPE_CHECKING:
    from .standard.message import StandardMessage
    from .standard.reply import ReplyMessage
logger = logging.getLogger("emily.plugin.api_client")
class EmilyApiClient:
    def __init__(
        self,
        base_url: str = "http://emily-core:18080",
        api_token: str = "",
        timeout: float = 30.0,
    ):
        self.base_url = base_url.rstrip("/")
        self.api_token = api_token

```

```

        self.timeout = aiohttp.ClientTimeout(total=timeout)
        self._session: aiohttp.ClientSession | None = None
    async def _ensure_session(self) -> aiohttp.ClientSession:
        if self._session is None or self._session.closed:
            headers = {}
            if self.api_token:
                headers["X-Emily-Token"] = self.api_token
            self._session = aiohttp.ClientSession(timeout=self.timeout,
headers=headers)
        return self._session
    async def close(self) -> None:
        if self._session and not self._session.closed:
            await self._session.close()
    def get_sse_url(self) -> str:
        return f"{self.base_url}/api/v1/events/outbound"
    async def send_message(self, msg: "StandardMessage") -> "ReplyMessage | None":
        from .standard.reply import ReplyMessage
        session = await self._ensure_session()
        url = f"{self.base_url}/api/v1/message/send"
        payload = self._message_to_dict(msg)
        try:
            async with session.post(url, json=payload) as resp:
                if resp.status == 204:
                    return None
                if resp.status != 200:
                    text = await resp.text()
                    logger.warning("send_message %d: %s", resp.status, text[:200])
                    return None
                data = await resp.json()
                return ReplyMessage(
                    conversation_id=data.get("conversation_id", ""),
                    content=data.get("content", ""),
                    reply_to_message_id=data.get("reply_to_message_id"),
                )
        except Exception as e:
            logger.error("send_message failed: %s", e)
            return None
    async def terminate_session(self, conversation_id: str) -> bool:
        session = await self._ensure_session()
        url = f"{self.base_url}/api/v1/session/terminate"
        try:
            async with session.post(url, json={"conversation_id":
conversation_id}) as resp:
                if resp.status != 200:
                    return False
                data = await resp.json()
                return bool(data.get("terminated"))
        except Exception as e:
            logger.error("terminate_session failed: %s", e)
            return False
    async def health_check(self) -> dict:
        session = await self._ensure_session()
        url = f"{self.base_url}/api/v1/health"
        try:

```

```

        async with session.get(url) as resp:
            if resp.status == 200:
                return await resp.json()
            return {"status": "error", "code": resp.status}
    except Exception as e:
        logger.error("health_check failed: %s", e)
        return {"status": "unreachable", "error": str(e)}

    @staticmethod
    def _message_to_dict(msg: "StandardMessage") -> dict:
        return {
            "message_id": msg.message_id,
            "platform": msg.platform,
            "conversation_type": msg.conversation_type,
            "conversation_id": msg.conversation_id,
            "sender_id": msg.sender_id,
            "sender_name": msg.sender_name,
            "group_id": msg.group_id,
            "group_name": msg.group_name,
            "content": msg.content,
            "is_at_bot": msg.is_at_bot,
            "mentioned_user_ids": list(msg.mentioned_user_ids or []),
            "reply_to_message_id": msg.reply_to_message_id,
            "msg_type": getattr(msg, "msg_type", 1),
            "attachments": list(getattr(msg, "attachments", []) or []),
        }

```

文件编号: 211 文件路径: data\plugins\emily_agent\adapters\astrbot\inbound_adapter.py

```

import logging
from typing import TYPE_CHECKING
from ..standard.message import StandardMessage
if TYPE_CHECKING:
    from astrbot.core.platform.astr_message_event import AstrMessageEvent
logger = logging.getLogger("emily.adapter.inbound")
class AstrBotInboundAdapter:
    @staticmethod
    def to_standard_message(event: "AstrMessageEvent") -> StandardMessage:
        from astrbot.core.platform.message_type import MessageType
        msg_type = event.get_message_type()
        if msg_type == MessageType.GROUP_MESSAGE:
            conversation_type = "group"
        elif msg_type == MessageType.PRIVATE_MESSAGE:
            conversation_type = "private"
        else:
            conversation_type = "unknown"
            logger.warning("Unknown message type: %s, treating as unknown",
msg_type)
        sender_id = event.get_sender_id()
        sender_name = event.get_sender_name()
        group_id = None
        if conversation_type == "group":
            gid = event.get_group_id()

```

```
        if gid:
            group_id = gid
        content = event.message_str or ""
        msg_type = 1
        attachments: list[dict] = []
        receiver_id = ""
        self_id = event.get_self_id()
        if conversation_type == "group":
            receiver_id = self_id or ""
        messages = event.get_messages()
        for comp in messages:
            if hasattr(comp, "qq"):
                uid = str(comp.qq)
                mentioned_ids.append(uid)
                if uid == self_id:
                    is_at_bot = True
            if hasattr(comp, "type") and getattr(comp, "type", None) == "at_all":
                mentioned_ids.append("all")
            comp_type = getattr(comp, "type", None)
            comp_data = getattr(comp, "data", None) or {}
            if comp_type == "image" or (comp_data and comp_data.get("url")):
                if msg_type == 1:
                    msg_type = 2
                url = comp_data.get("url") or getattr(comp, "url", "")
                attachments.append({
                    "type": 2,
                    "url": url,
                    "file_name": comp_data.get("file", "") or
comp_data.get("summary", ""),
                    "file_size": comp_data.get("file_size", 0),
                    "summary": comp_data.get("summary", ""),
                })
            elif comp_type == "record" or (comp_data and "time" in comp_data):
                if msg_type == 1:
                    msg_type = 4
                url = comp_data.get("url") or getattr(comp, "url", "")
                attachments.append({
                    "type": 4,
                    "url": url,
                    "file_name": comp_data.get("file", ""),
                    "file_size": comp_data.get("file_size", 0),
                })
            elif comp_type == "video" or (comp_data and comp_data.get("thumb")):
                msg_type = 5
                url = comp_data.get("url") or getattr(comp, "url", "")
                attachments.append({
                    "type": 5,
                    "url": url,
                    "file_name": comp_data.get("file", ""),
                    "file_size": comp_data.get("file_size", 0),
                    "thumb": comp_data.get("thumb", ""),
                })
            elif comp_type == "file" or (comp_data and comp_data.get("name")):
                if msg_type == 1:
```

```

        msg_type = 3
        url = comp_data.get("url") or getattr(comp, "url", "")
        attachments.append({
            "type": 3,
            "url": url,
            "file_name": comp_data.get("name", ""),
            "file_size": comp_data.get("size", 0),
        })
    message_id = ""
    msg_obj = event.message_obj
    if msg_obj:
        msg_id_attr = getattr(msg_obj, "message_id", None)
        if msg_id_attr:
            message_id = str(msg_id_attr)
    reply_to = None
    if msg_obj:
        raw = getattr(msg_obj, "raw_message", None)
        if isinstance(raw, dict):
            reply_to = str(raw.get("reply_message_id", "")) or None
    standard = StandardMessage(
        message_id=message_id,
        platform=event.get_platform_name() or "napcat",
        conversation_type=conversation_type,
        conversation_id=group_id or sender_id,
        sender_id=sender_id,
        sender_name=sender_name,
        group_id=group_id,
        content=content,
        is_at_bot=is_at_bot,
        mentioned_user_ids=mentioned_ids,
        reply_to_message_id=reply_to,
        msg_type=msg_type,
        attachments=attachments,
    )
    logger.debug(
        "inbound: type=%s, sender=%s, at_bot=%s, content_preview=%.50s",
        conversation_type, sender_name, is_at_bot, content,
    )
    return standard

```

文件编号: 212 文件路径: data\plugins\emily_agent\adapters\astrbot\outbound_sender.py

```

import asyncio
import logging
from typing import TYPE_CHECKING
from ..standard.reply import ReplyMessage
if TYPE_CHECKING:
    from astrbot.core.platform.astr_message_event import AstrMessageEvent
logger = logging.getLogger("emily.adapter.outbound")
class AstrBotOutboundSender:
    @staticmethod
    async def send(reply: ReplyMessage, event: "AstrMessageEvent") -> None:

```

```

file_paths = getattr(reply, "file_paths", None) or []
if file_paths:
    result = AstrBotOutboundSender._build_file_chain_result(
        text=reply.content, file_paths=file_paths, event=event,
    )
else:
    result = event.plain_result(reply.content)
event.set_result(result)
event.stop_event()
logger.info(
    "outbound: content=%.50s, files=%d, conversation=%s",
    reply.content, len(file_paths), reply.conversation_id,
)
@staticmethod
async def send_files(
    file_paths: list[dict],
    event: "AstrMessageEvent",
    caption: str = "",
) -> None:
    try:
        result = AstrBotOutboundSender._build_file_chain_result(
            text=caption, file_paths=file_paths, event=event,
        )
        await event.send(result)
        logger.info(
            "outbound send_files: %d file(s), caption=%.50s",
            len(file_paths), caption,
        )
    except Exception as e:
        logger.warning("M13 send_files failed: %s", e)
@staticmethod
def _build_file_chain_result(
    text: str,
    file_paths: list[dict],
    event: "AstrMessageEvent",
):
    from astrbot.core.message.components import Plain, File, Image
    chain = []
    if text:
        chain.append(Plain(text))
    for fp in file_paths:
        path = fp.get("path", "")
        name = fp.get("name", "")
        file_type = fp.get("type", "file")
        if not path:
            continue
        if file_type == "image":
            chain.append(Image(file=path))
        else:
            chain.append(File(name=name or path.rsplit("/", 1)[-1],
file=path))
    return event.chain_result(chain)
@staticmethod
async def send_progress(text: str, event: "AstrMessageEvent") -> None:

```



```
try:
    msg = event.plain_result(text)
    await event.send(msg)
    logger.info(
        "outbound progress: text=%.50s",
        text,
    )
except Exception as e:
    logger.warning("M8b progress direct send failed, falling back: %s", e)
    result = event.plain_result(text)
    event.set_result(result)
```

文件编号: 213 文件路径: data\plugins\emily_agent\adapters\sse_listener.py

```
from __future__ import annotations
import asyncio
import json
import logging
from typing import TYPE_CHECKING
import aiohttp
if TYPE_CHECKING:
    from .astrbot.outbound_sender import AstrBotOutboundSender
    from .standard.reply import ReplyMessage
logger = logging.getLogger("emily.plugin.sse")
class SSEListener:
    def __init__(self, outbound: "AstrBotOutboundSender", event_registry: dict | None = None):
        self.outbound = outbound
        self._event_registry = event_registry if event_registry is not None else {}
        self._running = False
        self._handlers = {
            "reply": self._handle_reply,
            "progress": self._handle_progress,
            "file_send": self._handle_file_send,
            "session_closed": self._handle_session_closed,
        }
    async def listen(self, sse_url: str, reconnect_delay: float = 3.0) -> None:
        self._running = True
        while self._running:
            try:
                await self._listen_once(sse_url)
            except Exception as e:
                logger.warning("SSE connection lost: %s - reconnecting in %.0fs",
                               e, reconnect_delay)
                await asyncio.sleep(reconnect_delay)
    def stop(self) -> None:
        self._running = False
    async def _listen_once(self, sse_url: str) -> None:
        async with aiohttp.ClientSession() as session:
            async with session.get(sse_url) as resp:
                logger.info("SSE connected: %s (status=%d)", sse_url, resp.status)
```

```

        event_type = "message"
        data_buf: list[str] = []
        async for raw in resp.content:
            line = raw.decode("utf-8", errors="ignore").rstrip("\r\n")
            if line == "":
                if data_buf:
                    await self._dispatch(event_type, "\n".join(data_buf))
                    event_type = "message"
                    data_buf = []
                    continue
            if line.startswith(":"):
                continue
            if line.startswith("event:"):
                event_type = line[len("event:"):].strip()
            elif line.startswith("data:"):
                data_buf.append(line[len("data:"):].strip())
    async def _dispatch(self, event_type: str, data_str: str) -> None:
        try:
            data = json.loads(data_str) if data_str else {}
        except json.JSONDecodeError:
            logger.warning("SSE bad data for %s: %s", event_type, data_str[:120])
            return
        handler = self._handlers.get(event_type)
        if handler:
            try:
                await handler(data)
            except Exception as e:
                logger.warning("SSE handler '%s' failed: %s", event_type, e)
        else:
            logger.debug("SSE unhandled event type: %s", event_type)
    async def _handle_reply(self, data: dict) -> None:
        from .standard.reply import ReplyMessage
        conv_id = data.get("conversation_id", "")
        event = self._event_registry.get(conv_id)
        if event is None:
            logger.debug("SSE reply: no event for conv=%s (already replied sync?)", conv_id)
            return
        reply = ReplyMessage(
            conversation_id=conv_id,
            content=data.get("content", ""),
            reply_to_message_id=data.get("reply_to_message_id"),
        )
        await self.outbound.send(reply, event)
    async def _handle_progress(self, data: dict) -> None:
        conv_id = data.get("conversation_id", "")
        event = self._event_registry.get(conv_id)
        text = data.get("content", "")
        if event is not None and text:
            await self.outbound.send_progress(text, event)
    async def _handle_file_send(self, data: dict) -> None:
        conv_id = data.get("conversation_id", "")
        event = self._event_registry.get(conv_id)
        if event is None:

```

```

        return
    file_paths = data.get("file_paths", [])
    caption = data.get("caption", "")
    if file_paths:
        await self.outbound.send_files(file_paths=file_paths, event=event,
caption=caption)
    async def _handle_session_closed(self, data: dict) -> None:
        conv_id = data.get("conversation_id", "")
        self._event_registry.pop(conv_id, None)
        logger.debug("SSE session_closed: conv=%s", conv_id)

```

文件编号: 214 文件路径: data\plugins\emily_agent\adapters\standard\command.py

```

from dataclasses import dataclass
@dataclass
class EventCommand:
    project_id: str | None = None
    project_name: str | None = None
    title: str = ""
    event_type: str = "general"
    category: str = "待分类"
    description: str = ""
    event_date: str | None = None
    creator_id: str = ""
    source_message_id: str = ""
    related_event_ids: list[str] | None = None
    conversation_id: str = ""
@dataclass
class TaskCommand:
    project_id: str | None = None
    project_name: str | None = None
    title: str = ""
    description: str = ""
    assignee_text: str = ""
    due_date: str | None = None
    due_text: str = ""
    creator_id: str = ""
    source_message_id: str = ""
@dataclass
class MeetingCommand:
    project_id: str | None = None
    project_name: str | None = None
    title: str = ""
    summary: str = ""
    attendees: list[str] | None = None
    creator_id: str = ""
    source_message_id: str = ""
@dataclass
class FileCommand:
    project_id: str | None = None
    project_name: str | None = None
    filename: str = ""

```

```
file_type: str = ""
file_size: int = 0
storage_path: str = ""
uploaded_by: str = ""
source_message_id: str = ""
@dataclass
class QueryCommand:
    query_type: str = "event"
    project_id: str | None = None
    project_name: str | None = None
    time_range: str = "all"
    status_filter: str | None = None
    assignee: str | None = None
    sender_name: str | None = None
    keyword: str | None = None
    intent: str | None = None
    file_type: str | None = None
    conversation_id: str | None = None
    limit: int = 50
```

文件编号: 215 文件路径: data\plugins\emily_agent\adapters\standard\message.py

```
from dataclasses import dataclass, field
from datetime import datetime
@dataclass
class StandardMessage:
    message_id: str = ""
    platform: str = ""
    conversation_type: str = ""
    conversation_id: str = ""
    sender_id: str = ""
    sender_name: str = ""
    group_id: str | None = None
    group_name: str | None = None
    content: str = ""
    is_at_bot: bool = False
    mentioned_user_ids: list[str] = field(default_factory=list)
    reply_to_message_id: str | None = None
    timestamp: datetime | None = None
    raw_event: dict | None = None
    msg_type: int = 1
    attachments: list[dict] = field(default_factory=list)
```

文件编号: 216 文件路径: data\plugins\emily_agent\adapters\standard\reply.py

```
from dataclasses import dataclass, field
@dataclass
class ReplyMessage:
    conversation_id: str
```

```
content: str
reply_to_message_id: str | None = None
references: list[dict] = field(default_factory=list)
metadata: dict = field(default_factory=dict)
file_paths: list[dict] = field(default_factory=list)
```

文件编号: 217 文件路径: data\plugins\emily_agent\adapters\standard\result.py

```
from dataclasses import dataclass, field
@dataclass
class RouteResult:
    intent: str
    project_id: str | None = None
    project_name: str | None = None
    confidence: float = 0.0
    data: dict = field(default_factory=dict)
@dataclass
class HandlerResult:
    success: bool
    object_type: str | None = None
    object_id: str | None = None
    reply: str | None = None
    error_code: str | None = None
    pending_confirmation: bool = False
@dataclass
class AgentStep:
    step_index: int
    type: str
    detail: str
    timestamp: str = ""
@dataclass
class AgentResult:
    success: bool
    reply: str
    steps: list[AgentStep] = field(default_factory=list)
    error_code: str | None = None
```

文件编号: 218 文件路径: data\plugins\emily_agent\adapters\standard\route_decision.py

```

from dataclasses import dataclass
@dataclass
class RouteDecision:
    takeover: bool
    mode: str = "collaborate"
    intent: str | None = None
    confidence: float = 0.0
    handler: str | None = None
    should_reply: bool = True
    reason: str = ""

```

文件编号: 219 文件路径: data\plugins\emily_agent\main.py

```

import asyncio
import hashlib
import logging
from collections import deque
from sys import maxsize
from astrbot.api import star
from astrbot.api.event import AstrMessageEvent, filter
from .adapters.astrbot.inbound_adapter import AstrBotInboundAdapter
from .adapters.astrbot.outbound_sender import AstrBotOutboundSender
from .adapters.api_client import EmilyApiClient
from .adapters.sse_listener import SSEListener
logger = logging.getLogger("emily.plugin")
DEDUP_MAX = 200
def _event_fingerprint(event: AstrMessageEvent) -> str:
    raw = f"{event.session_id}|{event.message_str}"
    return hashlib.sha256(raw.encode()).hexdigest()[:16]
class Main(star.Star):
    def __init__(self, context: star.Context, config: dict | None = None) -> None:
        super().__init__(context, config=config)
        cfg = dict(config) if config else {}
        base_url = cfg.get("emycore_url", "http://emily-core:18080")
        api_token = cfg.get("emycore_api_token", "")
        self.inbound = AstrBotInboundAdapter()
        self.outbound = AstrBotOutboundSender()
        self.api = EmilyApiClient(base_url=base_url, api_token=api_token)
        self._event_registry: dict = {}
        self.sse = SSEListener(self.outbound, event_registry=self._event_registry)
        self._sse_url = cfg.get("emycore_sse_url", "") or self.api.get_sse_url()
        self._seen: deque[str] = deque(maxlen=DEDUP_MAX)
        self._sse_task = None
    async def initialize(self) -> None:
        self._sse_task = asyncio.create_task(self.sse.listen(self._sse_url))
        health = await self.api.health_check()
        logger.info("Emily Core health: %s", health.get("status", "?"))
    async def terminate(self) -> None:
        self.sse.stop()
        if self._sse_task is not None:

```

```
        self._sse_task.cancel()
    await self.api.close()
@filter.event_message_type(filter.EventMessageType.ALL, priority=maxsize)
async def on_message(self, event: AstrMessageEvent) -> None:
    event_id = _event_fingerprint(event)
    if event_id in self._seen:
        return
    self._seen.append(event_id)
    msg = self.inbound.to_standard_message(event)
    if msg.conversation_id:
        self._event_registry[msg.conversation_id] = event
    reply = await self.api.send_message(msg)
    if reply is not None:
        await self.outbound.send(reply, event)
```

文件编号：220 文件路径：data\plugins\emily_agent\metadata.yaml

```
name: emily_agent
desc: "Emily 薄通信层插件 — 消息去重 + 格式转换 + HTTP 转发到 emily-core 容器 + SSE
出站推送（业务逻辑全部在独立 Core 容器）"
version: 1.0.0
author: Emily
```